BUKU TUTORIAL

Edisi Odin Turbo Pascal v2.3.0

Belajar Pemrograman Bahasa Pascal

dengan **ODIN** Turbo Pascal

```
[■]                    HALO.PAS                    [↕]
1  program HaloDunia;
2
3  begin
4    WriteLn('Halo, Dunia!');
5    WriteLn('Selamat datang di Odin Turbo Pascal.');
6  end.
```




```
Halo, Dunia!
Selamat datang di Odin Turbo Pascal.
_
```

20 Bab | 89 Program Contoh | Infografik & Latihan | Proyek Akhir

Syams Ideris

syamsuddin.ideris@gmail.com



Belajar Pemrograman Bahasa Pascal

dengan ODIN Turbo Pascal

Syams Ideris

syamsuddin.ideris@gmail.com

EDISI ODIN TURBO PASCAL V2.3.0 · 2026

Belajar Pemrograman Bahasa Pascal dengan ODIN Turbo Pascal

Penulis: Syams Ideris (syamsuddin.ideris@gmail.com)
Edisi pertama, Oktober 2026, untuk Odin Turbo Pascal v2.3.0.

Lisensi GNU GPL v3 atau yang lebih baru, hak cipta (c) 2026 Syams Ideris.

Buku ini adalah bagian dari Odin Turbo Pascal dan merupakan karya bebas: Anda boleh menyebarluaskan dan/atau mengubahnya menurut ketentuan GNU General Public License yang diterbitkan Free Software Foundation, versi 3 atau (sesuai pilihan Anda) versi yang lebih baru. Buku dan program contohnya disebarkan dengan harapan bermanfaat, tetapi **tanpa jaminan apa pun**, bahkan tanpa jaminan tersirat bahwa layak diperdagangkan atau cocok untuk tujuan tertentu. Naskah lengkap lisensi ada di berkas `LICENSE` yang menyertai Odin Turbo Pascal.

Odin Turbo Pascal, termasuk kode sumber buku ini, tersedia di `github.com/Syamsuddin/ODIN-TURBO-PASCAL`.

Semua program contoh di buku ini tersedia di Odin Turbo Pascal melalui menu **File ▸ Examples**. Setiap listing dicetak langsung dari berkas contoh tersebut, dan setiap keluaran program adalah hasil kompilasi dengan kompiler Odin lalu menjalankan program itu. Keluaran di komputer Anda bisa sedikit berbeda bila bergantung pada sistem operasi, prosesor, atau bilangan acak.

Odin Turbo Pascal adalah perangkat lunak yang ditulis secara independen. Tampilannya terinspirasi IDE Turbo Pascal era DOS, tetapi Odin Turbo Pascal bukan produk Borland, tidak berafiliasi dengan Borland, dan tidak memuat berkas, logo, maupun huruf milik Borland. Turbo Pascal, macOS, Windows, dan nama produk lain yang disebut di buku ini adalah merek dagang pemiliknya masing-masing dan disebut hanya untuk keperluan identifikasi.

Gambar layar Odin Turbo Pascal di buku ini digambar ulang dari data aplikasi yang sebenarnya: menu, warna editor, kode, dan pesan kompiler diambil dari Odin Turbo Pascal v2.3.0.

Kata Pengantar

Puji syukur ke hadirat Tuhan Yang Maha Esa, buku *Belajar Pemrograman Bahasa Pascal dengan ODIN Turbo Pascal* akhirnya dapat diselesaikan. Buku ini lahir dari keinginan sederhana: membuat belajar pemrograman terasa dekat, jelas, dan menyenangkan, terutama bagi Anda yang baru pertama kali menulis program.

Banyak dari kita mengenal pemrograman lewat layar biru Turbo Pascal di laboratorium komputer sekolah atau kampus. Layar itu sederhana, tetapi justru karena kesederhanaannya kita bisa memusatkan perhatian pada hal yang paling penting: cara berpikir. Menulis perintah dengan urutan yang benar, memilih tipe data yang tepat, memecah masalah besar menjadi bagian kecil, lalu menguji hasilnya. Kebiasaan berpikir itulah yang tetap berguna di bahasa pemrograman apa pun yang nanti Anda pelajari.

Masalahnya, Turbo Pascal asli adalah program DOS yang tidak lagi berjalan dengan nyaman di komputer masa kini. Karena itu saya membangun **Odin Turbo Pascal**: IDE bergaya klasik yang berjalan di macOS, Windows, dan Linux, lengkap dengan kompiler Odin yang menghasilkan program native, layar program 80×25, debugger, dan kumpulan program contoh. Buku ini adalah pasangan Odin Turbo Pascal. Ke-89 program contoh di dalamnya tersedia di menu **File ▸ Examples**, sehingga Anda bisa membaca penjelasan di buku sambil membuka, menjalankan, dan mengubah programnya sendiri.

Buku ini disusun sebagai tutorial bertahap dalam 20 bab yang dikelompokkan menjadi enam bagian, dari mengenal IDE dan menulis program pertama sampai menyelesaikan tiga proyek akhir yang utuh. Setiap konsep dijelaskan dengan bahasa sehari-hari, digambarkan dengan infografik berwarna, lalu dipraktikkan dengan program lengkap. Setiap program disertai keluarannya dan penjelasan baris demi baris. Di akhir bab tersedia rangkuman dan latihan bertingkat untuk menguji pemahaman.

Saran saya: jangan hanya membaca. Ketik ulang sebagian program, ubah angkanya, sengaja buat kesalahan, lalu baca pesan kompilernya. Programmer belajar paling cepat dari program yang gagal dan dari usaha memperbaikinya.

Terima kasih kepada keluarga, rekan, dan semua pengguna Odin Turbo Pascal yang telah mencoba, memberi masukan, dan melaporkan kesalahan. Buku ini tentu belum sempurna. Kritik, saran, dan laporan kesalahan sangat saya harapkan melalui surel **syamsuddin.ideris@gmail.com**, agar edisi berikutnya menjadi lebih baik.

Selamat belajar, dan selamat menikmati serunya membuat program pertama Anda.

Oktober 2026
Syams Ideris

Daftar Isi

Kata Pengantar iii

Daftar Isi iv

Cara Menggunakan Buku Ini x

Bagian I Fondasi 1

Bab 1 Mengenal Pascal dan Odin Turbo Pascal 2

1.1 Apa Itu Pemrograman? 3

1.2 Sekilas Bahasa Pascal 4

1.3 Mengenal Odin Turbo Pascal 4

1.4 Memasang Odin Turbo Pascal 6

1.5 Program Pertama: Halo, Dunia! 6

1.6 Memeriksa Kompiler 8

1.7 Menyimpan Program Sendiri 10

1.8 Ketika Program Salah 10

1.9 Rangkuman 11

1.10 Latihan 12

Bab 2 Struktur Program Pascal 13

2.1 Tujuh Bagian Program 14

2.2 Kata Tercadang dan Nama Buatan Sendiri 16

2.3 Komentar 16

2.4 Blok begin..end dan Titik Koma 18

2.5 Kesalahan yang Sering Terjadi 19

2.6 Rangkuman 20

2.7 Latihan 20

Bab 3 Tipe Data, Variabel, dan Konstanta 22

3.1 Variabel dan Tipenya 23

3.2 Bilangan: Integer dan Real 23

3.3 Teks: Char dan String 25

3.4 Benar atau Salah: Boolean 26

3.5 Konstanta 27

3.6 Ukuran dan Batas Tipe 29

3.7 Rangkuman 31

3.8 Latihan 31

Bab 4 Masukan dan Keluaran 33
4.1 Menampilkan: Write dan WriteLn 34
4.2 Membaca: Read dan ReadLn 35
4.3 Merapikan Keluaran dengan Format 37
4.4 Menggabungkan: Kartu Biodata 38
4.5 Kesalahan yang Sering Terjadi 39
4.6 Rangkuman 40
4.7 Latihan 40

Bab 5 Operator dan Ekspresi 41
5.1 Operator Aritmetika 42
5.2 Pembagian Bulat: div dan mod 43
5.3 Operator Perbandingan 44
5.4 Operator Logika 46
5.5 Prioritas Operator 48
5.6 Menggabungkan Semua: Luas dan Keliling 49
5.7 Kesalahan yang Sering Terjadi 50
5.8 Rangkuman 51
5.9 Latihan 51

Bagian II Alur Kendali 52

Bab 6 Percabangan 53
6.1 Program yang Bisa Memilih 54
6.2 Percabangan Satu Arah: if..then 54
6.3 Dua Pilihan: if..then..else 57
6.4 if Bersarang dan Rantai else if 60
6.5 Banyak Pilihan: case..of 63
6.6 Latihan Terpadu: Nilai Angka Menjadi Nilai Huruf 67
6.7 Latihan Terpadu: Tahun Kabisat 69
6.8 Kesalahan yang Sering Terjadi 71
6.9 Rangkuman 74
6.10 Latihan 74

Bab 7 Perulangan 76
7.1 Mengapa Perlu Perulangan 77
7.2 Perulangan Terhitung: for..to 78
7.3 Menghitung Mundur: for..downto 81
7.4 Perulangan Bersyarat: while..do 83
7.5 Mengulang Sampai: repeat..until 86
7.6 Perulangan Bersarang 89
7.7 Menu yang Berulang 91

7.8 Melacak Perulangan dengan Debugger 94
7.9 Kesalahan yang Sering Terjadi 96
7.10 Rangkuman 98
7.11 Latihan 98

Bab 8 Berlatih Algoritma dengan Perulangan 100
8.1 Dari Masalah ke Algoritma 101
8.2 Bilangan Prima 101
8.3 FPB dan KPK: Algoritma Euclid 105
8.4 Deret Fibonacci 108
8.5 Segitiga Bintang 111
8.6 Bilangan Sempurna 113
8.7 Permainan Tebak Angka 115
8.8 Kesalahan yang Sering Terjadi 118
8.9 Rangkuman 121
8.10 Latihan 121

Bagian III Prosedur, Fungsi, dan Rekursi 123

Bab 9 Prosedur dan Fungsi 124
9.1 Memecah Program Menjadi Bagian 125
9.2 Prosedur: Perintah yang Diberi Nama 125
9.3 Fungsi: Subprogram yang Menghasilkan Nilai 128
9.4 Parameter: Mengirim Data ke Subprogram 131
9.5 Variabel Lokal dan Global 135
9.6 Melacak Panggilan dengan Debugger 137
9.7 Latihan Terpadu: Kalkulator Modular 139
9.8 Kesalahan yang Sering Terjadi 142
9.9 Rangkuman 143
9.10 Latihan 144

Bab 10 Rekursi 145
10.1 Berpikir Secara Rekursif 146
10.2 Faktorial: Rekursi yang Paling Klasik 147
10.3 Fibonacci: Rumus Indah yang Lambat 151
10.4 Pangkat: Memecah Separuh demi Separuh 154
10.5 Menara Hanoi: Teka-teki yang Dipecahkan Rekursi 156
10.6 Rekursi atau Perulangan? 161
10.7 Kesalahan yang Sering Terjadi 161
10.8 Rangkuman 163
10.9 Latihan 164

Bagian IV Struktur Data 165

Bab 11 Array 166

11.1 Satu Nama, Banyak Kotak 167
11.2 Array Satu Dimensi 168
11.3 Indeks di Luar Batas 171
11.4 Nilai Terbesar, Terkecil, dan Rata-rata 172
11.5 Array Dua Dimensi 175
11.6 Matriks dan Penjumlahannya 178
11.7 Perkalian Matriks 180
11.8 Kesalahan yang Sering Terjadi 182
11.9 Rangkuman 183
11.10 Latihan 184

Bab 12 Mengolah String 185

12.1 String sebagai Deretan Karakter 186
12.2 Fungsi dan Prosedur String 187
12.3 Membalik Huruf dan Membalik Kata 190
12.4 Palindrom 193
12.5 Menghitung Vokal dan Jenis Karakter Lain 196
12.6 Mengubah Angka dan String 198
12.7 Kesalahan yang Sering Terjadi 201
12.8 Rangkuman 203
12.9 Latihan 203

Bab 13 Record 205

13.1 Mengapa Perlu Record? 206
13.2 Mendeklarasikan Record dan Memakai Field 206
13.3 Array of Record: Record sebagai Tabel 210
13.4 Record Bersarang 213
13.5 Menyatukan Semuanya: Data Mahasiswa 216
13.6 Kesalahan yang Sering Terjadi 221
13.7 Rangkuman 222
13.8 Latihan 223

Bab 14 Enumerasi, Subrange, dan Set 224

14.1 Tipe Ordinal: Nilai yang Berurutan 225
14.2 Tipe Enumerasi 225
14.3 Tipe Subrange 228
14.4 Tipe Set (Himpunan) 231
14.5 Kesalahan yang Sering Terjadi 238
14.6 Rangkuman 240
14.7 Latihan 240

Bagian V Pencarian dan Pengurutan 241

Bab 15 Pencarian Data 242

15.1 Apa yang Dimaksud Mencari Data 243
15.2 Pencarian Beruntun 243
15.3 Pencarian Biner 247
15.4 Membandingkan Jumlah Langkah 252
15.5 Mencari Data di Array of Record 253
15.6 Kesalahan yang Sering Terjadi 257
15.7 Rangkuman 259
15.8 Latihan 260

Bab 16 Pengurutan Data 261

16.1 Mengapa Data Perlu Diurutkan 262
16.2 Menukar Dua Elemen 262
16.3 Bubble Sort 263
16.4 Selection Sort 268
16.5 Insertion Sort 271
16.6 Mengurutkan Array of Record 275
16.7 Membandingkan Jumlah Langkah 278
16.8 Kesalahan yang Sering Terjadi 283
16.9 Rangkuman 284
16.10 Latihan 284

Bagian VI File, Pointer, Unit, dan Proyek Akhir 286

Bab 17 Bekerja dengan File 287

17.1 Mengapa Perlu File? 288
17.2 Alur Kerja File 288
17.3 Menulis File Teks 290
17.4 Membaca File Teks 292
17.5 Menambah Isi File Teks 295
17.6 File Bertipe 297
17.7 Menyimpan Record ke File 300
17.8 Kesalahan yang Sering Terjadi 303
17.9 Rangkuman 304
17.10 Latihan 305

Bab 18 Pointer dan Linked List 306

18.1 Variabel Menyimpan Data, Pointer Menyimpan Alamat 307
18.2 Pointer ke Variabel yang Ada 308
18.3 Memori Dinamis: New dan Dispose 311
18.4 Simpul dan Linked List 313

18.5 Menambah Simpul 314
18.6 Menghapus Simpul 318
18.7 Kesalahan yang Sering Terjadi 321
18.8 Rangkuman 323
18.9 Latihan 323

Bab 19 Unit dan Layar Crt 324
19.1 Mengapa Memakai Unit 325
19.2 Anatomi Sebuah Unit 325
19.3 Membuat dan Memakai Unit matematika 326
19.4 Galat Seputar Unit 331
19.5 Unit Crt: Mengatur Layar Teks 332
19.6 Proyek Kecil: Menu Berwarna 337
19.7 Kesalahan yang Sering Terjadi 344
19.8 Rangkuman 344
19.9 Latihan 345

Bab 20 Proyek Akhir 346
20.1 Dari Masalah ke Program 347
20.2 Proyek 1: Kasir Sederhana 348
20.3 Proyek 2: Pengolah Nilai Siswa 357
20.4 Proyek 3: Buku Telepon 364
20.5 Kesalahan yang Sering Terjadi di Proyek 373
20.6 Peta Konsep: Perjalanan Anda 374
20.7 Rangkuman 375
20.8 Latihan 376

Penutup 377

Lampiran A Memasang Odin Turbo Pascal 379

Lampiran B Pintasan Keyboard 381

Lampiran C Kata Tercadang dan Rutin Bawaan 382

Lampiran D Memahami Pesan Kesalahan 384

Lampiran E Daftar Program Contoh 386

Lampiran F Glosarium 389

Cara Menggunakan Buku Ini

Buku ini bisa dibaca berurutan dari awal sampai akhir, dan memang dirancang untuk itu: setiap bab memakai konsep dari bab-bab sebelumnya. Bagian ini menjelaskan susunan buku dan arti tanda-tanda yang Anda temui di setiap halaman.

Peta Buku

Kedua puluh bab dikelompokkan menjadi enam bagian. Setiap bagian punya warna sendiri, dan warna itu dipakai pada judul, tabel, infografik, serta tab penanda bab di tepi kanan halaman, sehingga Anda selalu tahu sedang berada di bagian mana.



Gambar Enam bagian buku. Ikuti panah: setiap bagian dibangun di atas bagian sebelumnya.

Susunan Setiap Bab

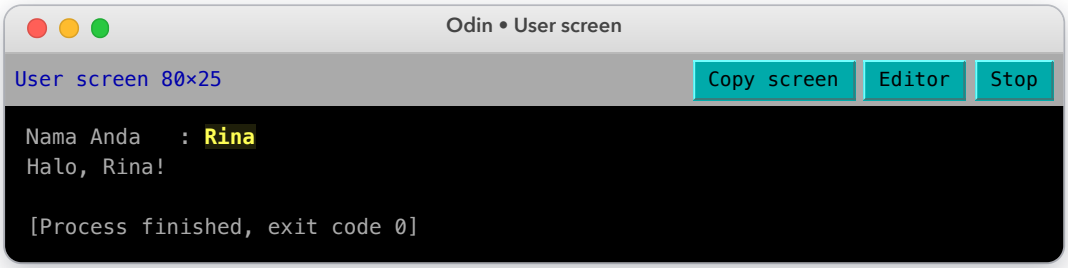
Setiap bab dibuka dengan halaman berwarna yang memuat **tujuan pembelajaran** dan **daftar program** yang akan dibahas. Isi bab mengikuti pola yang sama: konsep dijelaskan lebih dulu, lengkap dengan bentuk umum penulisannya dan infografik, lalu dipraktikkan dengan program contoh. Bab ditutup dengan **kesalahan yang sering terjadi**, **rangkuman**, dan **latihan**.



Membaca Listing dan Keluaran

Listing program dicetak dengan warna yang sama dengan editor Odin Turbo Pascal. Bilah judulnya memuat nomor program, nama berkas, dan judul singkat. Nomor program mengikuti nama berkas: **Program 6.4** adalah berkas `bab06_04_case_of.pas`, yang dibuka dari `File ▸ Examples...`. Angka di kiri adalah nomor baris, sama dengan nomor baris di baris status IDE, dan dipakai dalam tabel penjelasan.

Keluaran program ditampilkan dalam jendela **User screen**, tempat program berjalan di Odin Turbo Pascal. Teks yang diketik pengguna, misalnya nama atau angka, dicetak **kuning tebal** agar mudah dibedakan dari tulisan program. Seluruh sesi ditampilkan dari awal sampai pesan `[Process finished, exit code 0]`.



Gambar Contoh tampilan User screen: "Rina" diketik pengguna, sisanya ditulis program.

Gambar layar IDE kadang diberi ❶ lingkaran bernomor. Penjelasan setiap nomor ada di teks atau tabel di dekat gambar.

Kotak-Kotak Khusus

TIPS
Saran praktis agar pekerjaan lebih mudah atau program lebih rapi.

AWAS
Hal yang sering menimbulkan kesalahan. Baca dengan teliti.

INFO
Pengetahuan tambahan yang memperluas pemahaman.

ATURAN TURBO PASCAL 7
Aturan khas Turbo Pascal 7 yang dipakai buku ini.

COBA SENDIRI
Eksperimen kecil: ubah program, jalankan, amati.

INGAT
Inti penting yang perlu diingat.

Penulisan Tombol, Menu, dan Kode

Tabel Notasi yang dipakai di buku ini

Tampilan	Arti
Ctrl + F9	Tahan tombol Ctrl, lalu tekan F9. Di Mac, tombol fungsi mungkin perlu ditekan bersama fn.
`File ▸ Examples...`	Buka menu **File**, lalu pilih **Examples...**.
`WriteLn`	Kode, nama berkas, atau nama di dalam program.
DASAR MENENGAH TANTANGAN	Tingkat kesulitan soal latihan.

Saran Belajar

1. **Siapkan Odin Turbo Pascal** sebelum mulai membaca (lihat Bab 1 dan Lampiran A).
2. **Buka setiap program contoh** saat membahasnya, jalankan, dan bandingkan keluarannya dengan buku.
3. **Kerjakan "Coba Sendiri"**. Simpan hasil ubahan di folder Anda sendiri dengan `File ▸ Save as...`.
4. **Selesaikan latihan** minimal tingkat Dasar dan Menengah sebelum pindah bab.
5. **Pakai lampiran** sebagai rujukan cepat: tombol pintasan, kata tercadang, rutin bawaan, dan arti pesan kesalahan.

BAGIAN I

Fondasi

Mengenal Pascal dan Odin Turbo Pascal, struktur program, tipe data, masukan dan keluaran, serta operator.

01 Mengenal Pascal dan Odin Turbo Pascal

02 Struktur Program Pascal

03 Tipe Data, Variabel, dan Konstanta

04 Masukan dan Keluaran

05 Operator dan Ekspresi

BAB

01

BAGIAN I · FONDASI

Mengenal Pascal dan Odin Turbo Pascal

TUJUAN PEMBELAJARAN

- Memahami apa itu program, kode sumber, kompiler, dan program executable.
- Mengenal sejarah singkat bahasa Pascal dan alasan bahasa ini cocok untuk belajar.
- Mengenal bagian-bagian IDE Odin Turbo Pascal: menu, jendela editor, tombol fungsi, dan baris status.
- Membuka, menjalankan, dan menyimpan program contoh.
- Membaca pesan kesalahan kompiler dan memperbaikinya.

PROGRAM DALAM BAB INI

1.1 `bab01_01_halo_dunia.pas`
Halo, Dunia!

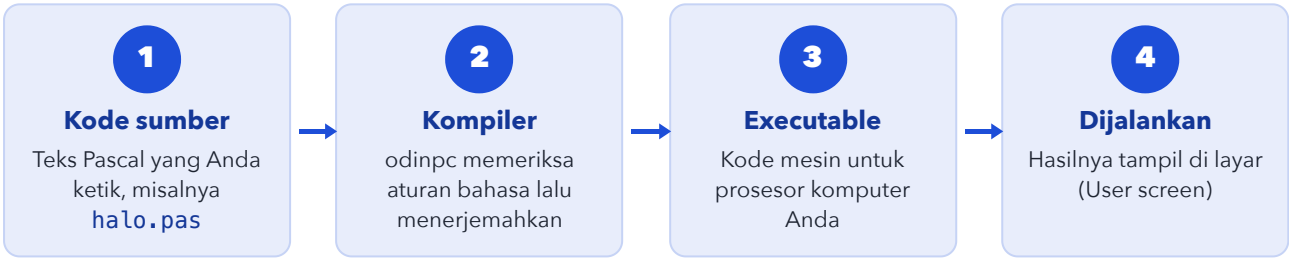
1.2 `bab01_02_cek_kompiler.pas`
Memeriksa kompiler

Selamat datang di dunia pemrograman. Di bab pertama ini Anda belum perlu menghafal apa pun. Tugas Anda hanya berkenalan: dengan cara komputer menjalankan perintah, dengan bahasa Pascal, dan dengan Odin Turbo Pascal, alat yang akan menemani Anda sepanjang buku. Di akhir bab, Anda sudah menjalankan dua program pertama dan tahu apa yang harus dilakukan ketika kompiler menolak program Anda.

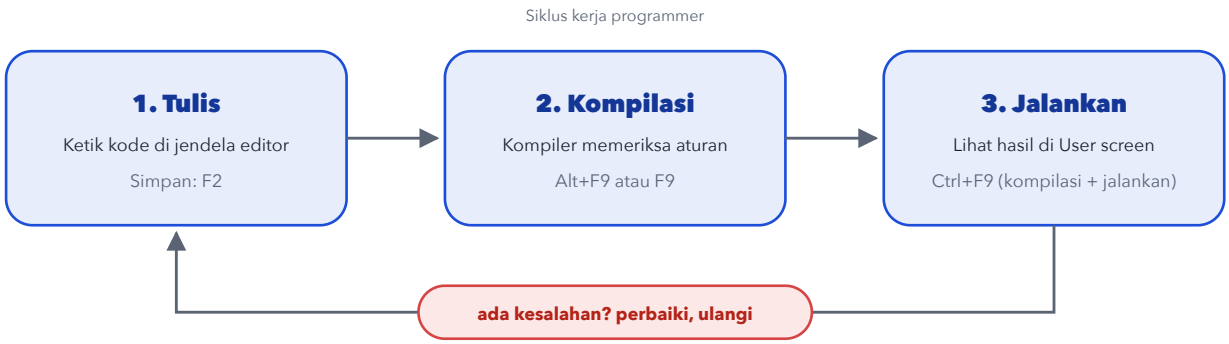
1.1 Apa Itu Pemrograman?

Komputer sangat cepat, tetapi tidak bisa menebak keinginan kita. Ia hanya mengerjakan perintah yang ditulis dengan sangat jelas, satu per satu, berurutan. Kumpulan perintah seperti itu disebut **program**, dan kegiatan menyusunnya disebut **pemrograman**. Bayangkan resep masakan: resep yang baik menyebut bahan, takaran, dan langkah dengan urutan yang tepat. Bila satu langkah tertukar, hasilnya bisa gagal. Program juga begitu.

Program ditulis dalam **bahasa pemrograman**, yaitu bahasa buatan yang aturannya ketat sehingga bisa dipahami mesin dan juga manusia. Teks program yang kita ketik disebut **kode sumber** (*source code*). Prosesor komputer tidak bisa membaca kode sumber secara langsung. Ia hanya memahami **kode mesin**, deretan angka biner. Penerjemahnya adalah **kompiler**: program yang membaca kode sumber, memeriksa kebenarannya, lalu menghasilkan **program executable** yang bisa dijalankan.



Bila kode sumber melanggar aturan bahasa, misalnya ada tanda titik koma yang lupa ditulis, kompiler berhenti dan memberi **pesan kesalahan**. Tidak ada executable yang dibuat sampai kesalahan itu diperbaiki. Karena itu pekerjaan seorang programmer selalu berupa siklus: menulis, mengompilasi, menjalankan, lalu memperbaiki dan mengulang.



Gambar 1.1 Siklus kerja: tulis, kompilasi, jalankan. Setiap kali ada kesalahan, Anda kembali ke editor.

1.2 Sekilas Bahasa Pascal

Pascal dirancang oleh **Niklaus Wirth**, profesor ilmu komputer di ETH Zürich, Swiss, dan diterbitkan pada tahun 1970. Namanya diambil dari Blaise Pascal, matematikawan Prancis abad ke-17 yang membuat salah satu mesin hitung mekanis pertama. Sejak awal Pascal dirancang untuk **mengajarkan pemrograman yang terstruktur dan rapi**: setiap variabel harus diumumkan beserta tipenya, blok program jelas awal dan akhirnya, dan kompiler menolak banyak kesalahan sebelum program sempat berjalan.

Pascal sangat populer di sekolah dan kampus sejak Borland merilis **Turbo Pascal** pada tahun 1983. Turbo Pascal memadukan editor, kompiler, dan debugger dalam satu layar biru yang cepat, sehingga siklus tulis-kompilasi-jalankan bisa dilakukan dalam hitungan detik. Versi terakhirnya untuk DOS, Turbo Pascal 7 (1992), menjadi standar de facto yang dipakai di banyak buku pelajaran hingga kini. Gaya bahasa Turbo Pascal 7 inilah yang dipakai di buku ini.

1970
Pascal diterbitkan oleh Niklaus Wirth

1983
Turbo Pascal memopulerkan IDE layar biru

1992
Turbo Pascal 7, gaya bahasa yang dipakai buku ini

Kini
Odin Turbo Pascal membawa pengalaman itu ke komputer modern

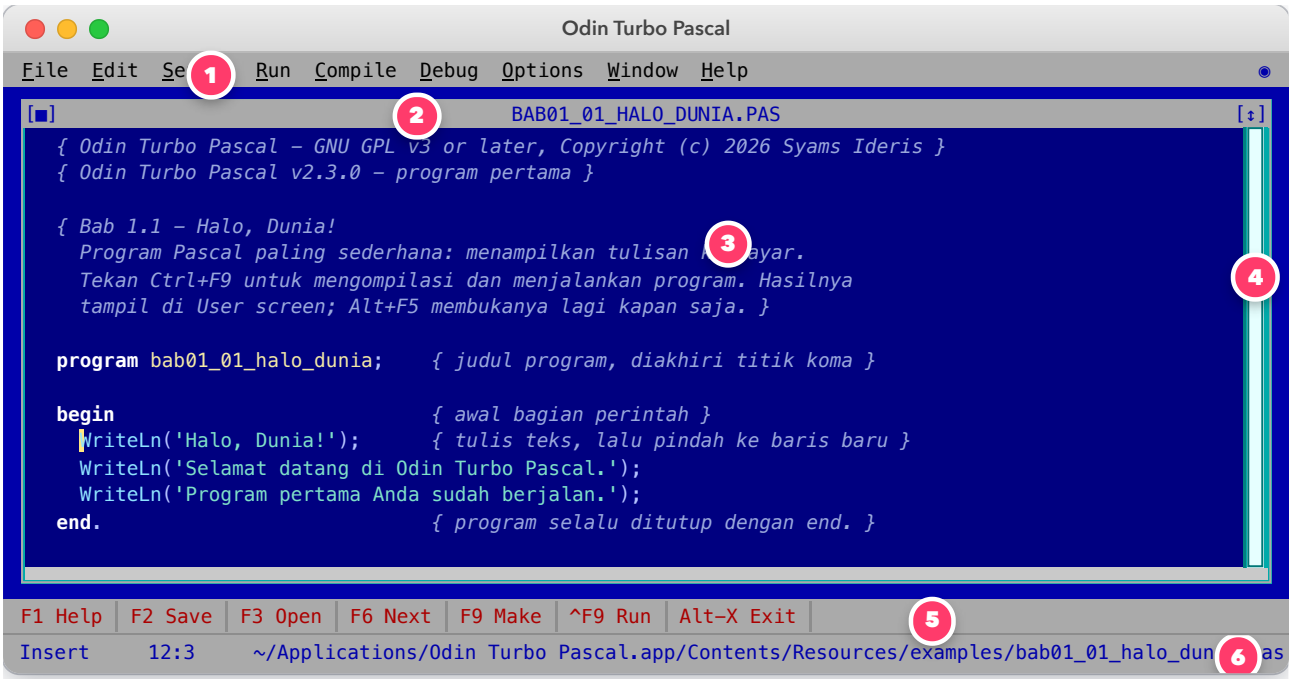
Mengapa belajar Pascal sekarang? Karena Pascal memaksa Anda membangun kebiasaan baik: memberi nama yang jelas, memikirkan tipe data, dan menyusun program dari bagian-bagian kecil. Konsep yang Anda kuasai di sini, seperti variabel, percabangan, perulangan, prosedur, array, record, file, dan pointer, ada di hampir semua bahasa pemrograman lain. Setelah fasih berpikir dengan Pascal, pindah ke bahasa lain akan terasa jauh lebih mudah.

1.3 Mengenal Odin Turbo Pascal

Odin Turbo Pascal (OTP) adalah IDE (*Integrated Development Environment*, lingkungan pengembangan terpadu) bergaya Turbo Pascal klasik yang berjalan di macOS, Windows, dan Linux. Di dalamnya ada editor kode, kompiler **Odin** (`odinpc`) yang menghasilkan program native untuk prosesor komputer Anda, terminal 80×25 tempat program berjalan (*User screen*), dan debugger. Semuanya buatan Odin sendiri, sehingga Anda tidak perlu memasang kompiler lain.

> *i* **INFO**
>
> Odin Turbo Pascal adalah perangkat lunak bebas (GNU GPL v3 atau yang lebih baru) yang ditulis secara independen. Tampilannya terinspirasi IDE Turbo Pascal era DOS, tetapi Odin Turbo Pascal **bukan** produk Borland dan tidak memakai berkas apa pun milik Borland.

Saat dibuka pertama kali, OTP langsung menampilkan program contoh pertama, `bab01_01_halo_dunia.pas`, di jendela editor. Perhatikan bagian-bagian layarnya pada gambar berikut. Angka merah pada gambar dijelaskan di tabel sesudahnya.



Gambar 1.2 Tampilan Odin Turbo Pascal dengan program contoh pertama terbuka di editor

Tabel 1.1 Bagian-bagian layar Odin Turbo Pascal (lihat **Gambar 1.2**)

No.	Bagian	Fungsi
1	Baris menu	Semua perintah: **File** (berkas), **Edit**, **Search** (cari), **Run** (jalankan), **Compile**, **Debug**, **Options** (pengaturan), **Window**, dan **Help**. Klik nama menu, atau tekan F10.
2	Judul jendela	Nama berkas yang sedang disunting. Tanda `*` muncul bila ada perubahan yang belum disimpan. `[■]` menutup jendela, `[↕]` memperbesarnya.
3	Editor	Tempat mengetik kode. Kata kunci, nama, angka, teks, dan komentar diberi warna berbeda agar mudah dibaca.
4	Penggulung	Menggeser isi editor bila program lebih panjang dari layar.
5	Tombol fungsi	Pintasan yang paling sering dipakai: F1 Help, F2 Save, F3 Open, F9 Make, Ctrl+F9 Run, Alt+X Exit. Bisa juga diklik.
6	Baris status	Mode ketik (*Insert* atau *Overwrite*), posisi kursor *baris:kolom*, lokasi berkas, dan mode bahasa `[TP]` (Turbo Pascal).

Bila semua jendela ditutup, desktop biru menampilkan logo Odin dan sebuah plakat berisi versi OTP yang terpasang. Bila suatu saat Anda ragu versi mana yang sedang dipakai, lihat plakat ini atau menu `Help ▸ About Odin`.



Gambar 1.3 Desktop Odin Turbo Pascal tanpa jendela terbuka: logo dan plakat versi

1.4 Memasang Odin Turbo Pascal

Cara memasang OTP berbeda untuk setiap sistem operasi. Ringkasannya ada di bawah ini; langkah lengkapnya ada di **Lampiran A**.

> **INFO**
> Paket OTP dan kode sumbernya tersedia gratis di repositori GitHub `github.com/Syamsuddin/ODIN-TURBO-PASCAL`. Pilih **Code ▸ Download ZIP** untuk mengunduh paketnya.

macOS
Ekstrak paket, lalu buka `Install.command`. Pemasang memeriksa Python dan Tk, lalu menyalin aplikasi ke folder *Applications* di folder rumah Anda.

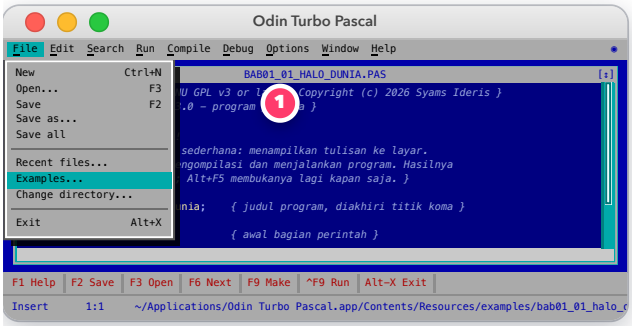
Windows
Ekstrak paket, lalu klik dua kali `Install.cmd`. Pemasang memeriksa Python dan Tk, menguji kompilasi, lalu membuat pintasan di menu Start dan Desktop.

Linux
Pasang `python3-tk`, `git`, dan Rust, unduh kode sumbernya, bangun kompiler dengan `cargo build --release`, lalu jalankan `python3 main.py`.

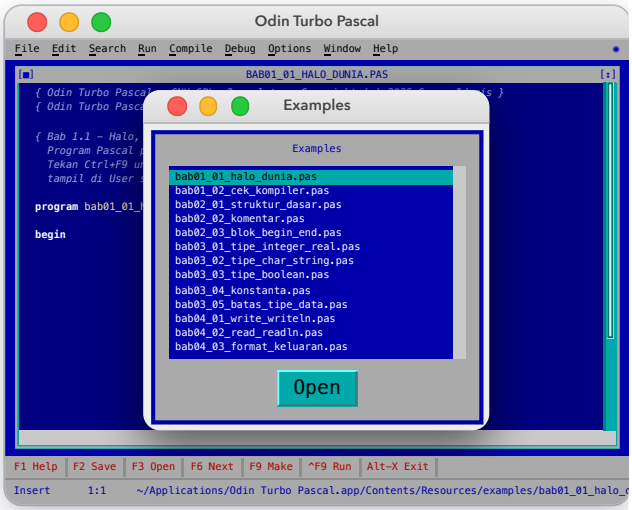
1.5 Program Pertama: Halo, Dunia!

Sudah menjadi tradisi bahwa program pertama dalam bahasa apa pun menampilkan tulisan "Halo, Dunia!". Semua program contoh buku ini sudah tersedia di dalam OTP, jadi Anda tidak perlu mengetik ulang. Ikuti langkah berikut.

1. **Buka menu File.** Klik `File` di baris menu, atau tekan **F10**. Pilih `Examples...`.
2. **Pilih program.** Daftar 90 berkas contoh muncul, urut sesuai bab. Pilih `bab01_01_halo_dunia.pas`, lalu tekan **Enter** atau klik **Open**.
3. **Jalankan.** Tekan **Ctrl**+**F9**. OTP menyimpan, mengompilasi, lalu menjalankan program.
4. **Lihat hasilnya** di jendela *User screen*. Klik tombol **Editor** untuk kembali menyunting, atau tekan **Alt**+**F5** kapan saja untuk melihat User screen lagi.



Gambar 1.4 Langkah 1: menu File ▸ Examples...



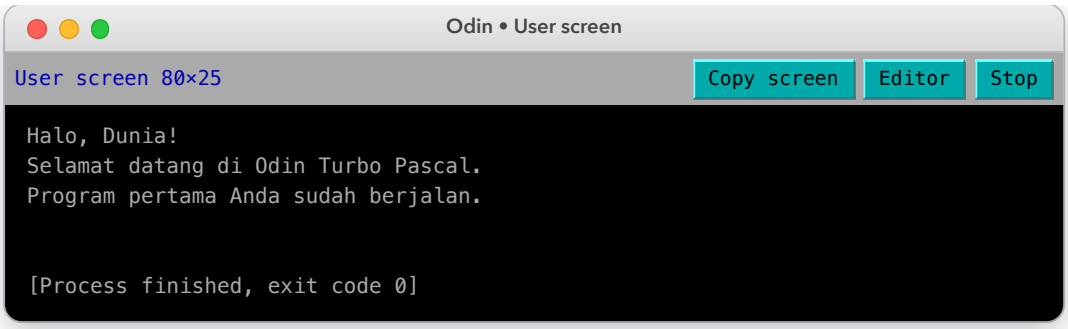
Gambar 1.5 Langkah 2: memilih berkas contoh

Berikut isi program tersebut. Setiap listing di buku ini diberi nomor sesuai nama berkasnya: **Program 1.1** adalah berkas `bab01_01_halo_dunia.pas`. Nomor di sebelah kiri adalah nomor baris; nomor ini sama persis dengan nomor baris yang tampil di baris status OTP.

Program 1.1 `bab01_01_halo_dunia.pas` *Halo, Dunia!* File ▸ Examples...

```
1  { Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Odin Turbo Pascal v2.3.0 – program pertama }
3
4  { Bab 1.1 – Halo, Dunia!
5    Program Pascal paling sederhana: menampilkan tulisan ke layar.
6    Tekan Ctrl+F9 untuk mengompilasi dan menjalankan program. Hasilnya
7    tampil di User screen; Alt+F5 membukanya lagi kapan saja. }
8
9  program bab01_01_halo_dunia;     { judul program, diakhiri titik koma }
10
11 begin                            { awal bagian perintah }
12   WriteLn('Halo, Dunia!');       { tulis teks, lalu pindah ke baris baru }
13   WriteLn('Selamat datang di Odin Turbo Pascal.');
14   WriteLn('Program pertama Anda sudah berjalan.');
15 end.                             { program selalu ditutup dengan end. }
```

Setelah **Ctrl**+**F9** ditekan, User screen menampilkan hasil berikut. Baris terakhir, `[Process finished, exit code 0]`, ditambahkan OTP untuk memberi tahu bahwa program sudah selesai dengan normal. Kode keluar `0` berarti tidak ada masalah.



Gambar 1.6 Keluaran `bab01_01_halo_dunia.pas` di User screen

1.5.1 Membaca Program Baris demi Baris

Baris	Penjelasan
1–7	Teks di antara kurung kurawal `{ ... }` adalah **komentar**. Komentar diabaikan kompiler; gunanya menjelaskan program kepada pembaca manusia. Baris 2 menyimpan versi OTP yang sedang Anda pakai.
9	`program bab01_01_halo_dunia;` memberi nama program. Kata `program` adalah **kata tercadang** (*reserved word*), kata yang artinya sudah ditetapkan bahasa Pascal. Pernyataan diakhiri titik koma `;`.
11	`begin` menandai awal bagian perintah. Semua yang dikerjakan program ditulis antara `begin` dan `end.`
12–14	`WriteLn` menulis teks ke layar lalu pindah ke baris baru. Teks ditulis di antara tanda petik tunggal `'...'`. Tiga perintah dijalankan berurutan dari atas ke bawah.
15	`end.` dengan **titik** menutup program. Apa pun yang ditulis setelahnya tidak dibaca kompiler.

COBA SENDIRI

Ubah teks pada baris 12 menjadi nama Anda, misalnya `WriteLn('Halo, Rina!');`, lalu tekan Ctrl + F9 lagi. Karena berkas contoh adalah templat bawaan, OTP akan meminta Anda menyimpannya sebagai berkas baru terlebih dahulu (lihat subbab berikutnya).

1.6 Memeriksa Kompiler

Program kedua memastikan kompiler terpasang dengan benar dan sekaligus memperkenalkan beberapa hal baru. Buka `bab01_02_cek_kompiler.pas` dari `File ▸ Examples...` lalu jalankan.

Program 1.2 `bab01_02_cek_kompiler.pas` *Memeriksa kompiler* — File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 1.2 — Memeriksa kompiler
3    Program ini memastikan kompiler terpasang dan bisa membuat program.
4    Baris berawalan $IFDEF dibaca sebelum kompilasi: hanya bagian yang
5    cocok dengan kompiler dan sistem Anda yang ikut dikompilasi. }
6
7  program bab01_02_cek_kompiler;
8
9  begin
10   WriteLn('PEMERIKSAAN KOMPILER');
11   WriteLn('====================');
```

```
12
13    { Nama kompiler: kompiler Odin mendefinisikan simbol ODIN }
14    {$IFDEF ODIN} WriteLn('Kompiler : Odin (odinpc)'); {$ENDIF}
15
16    { Sistem operasi tempat program berjalan }
17    {$IFDEF DARWIN} WriteLn('Sistem   : macOS'); {$ENDIF}
18    {$IFDEF LINUX} WriteLn('Sistem   : Linux'); {$ENDIF}
19    {$IFDEF WINDOWS} WriteLn('Sistem   : Windows'); {$ENDIF}
20
21    { Jenis prosesor }
22    {$IFDEF CPUX86_64} WriteLn('Prosesor : x86-64 (Intel/AMD)'); {$ENDIF}
23    {$IFDEF CPUAARCH64} WriteLn('Prosesor : ARM64'); {$ENDIF}
24
25    { Mode Turbo Pascal: Integer berukuran 2 byte, seperti TP7 }
26    WriteLn('Integer  : ', SizeOf(Integer), ' byte');
27    WriteLn('Uji hitung 2 + 3 = ', 2 + 3);
28    WriteLn;
29    WriteLn('Kompiler siap dipakai.');
30  end.
```



Gambar 1.7 Keluaran `bab01_02_cek_kompiler.pas` di Mac Intel. Di Mac Apple Silicon baris Prosesor berbunyi ARM64.

Baris yang diawali `{$IFDEF ...}` adalah **direktif kompiler**, perintah untuk kompiler, bukan untuk program. `{$IFDEF ODIN}` berarti "ikutkan bagian ini hanya bila dikompilasi oleh Odin". Bagian yang tidak cocok dengan kompiler, sistem operasi, atau prosesor Anda dibuang sebelum kompilasi, sehingga keluaran program ini bisa sedikit berbeda di komputer lain. Direktif berbeda dengan komentar: keduanya memakai kurung kurawal, tetapi direktif selalu diawali tanda dolar `$`.

Baris 26 memakai fungsi `SizeOf` yang menghasilkan ukuran sebuah tipe data dalam byte. Hasilnya 2 byte untuk `Integer`, sama seperti Turbo Pascal 7. Baris 28, `WriteLn;` tanpa isi, hanya mencetak baris kosong. Perhatikan juga bahwa `WriteLn` bisa menerima beberapa nilai sekaligus, dipisah koma, seperti `WriteLn('Uji hitung 2 + 3 = ', 2 + 3)`: teks ditulis apa adanya, sedangkan `2 + 3` dihitung dulu, lalu hasilnya, `5`, yang ditulis.

ATURAN TURBO PASCAL 7

Buku ini memakai aturan Turbo Pascal 7 (mode `[TP]` di baris status). Semua program contoh dikompilasi dan dijalankan dengan kompiler Odin, dan keluarannya diperiksa oleh tes otomatis.

1.7 Menyimpan Program Sendiri

Program contoh di menu Examples tersimpan di dalam aplikasi dan diperlakukan sebagai **templat**: perintah **Save** padanya selalu berubah menjadi **Save as**, sehingga contoh aslinya tidak pernah rusak. Biasakan menyimpan pekerjaan Anda di folder sendiri, misalnya folder `Pascal` di folder rumah Anda.

1. **Buat program baru** dengan `File ▸ New` (Ctrl + N). Jendela baru berjudul `NONAME` berisi kerangka program kecil.
2. **Simpan** dengan F2. Pilih folder `Pascal`, beri nama berkas, misalnya `halo.pas`. Akhiran `.pas` menandai berkas kode Pascal.
3. **Ketik dan jalankan.** Setiap kali menekan Ctrl + F9, OTP menyimpan semua perubahan lebih dulu, jadi program yang dijalankan selalu versi terbaru.

TIPS

Beri nama berkas dengan huruf kecil, tanpa spasi, dan mirip dengan nama program di dalamnya, seperti contoh-contoh di buku ini. Nama yang rapi memudahkan Anda mencari program lama.

1.8 Ketika Program Salah

Semua programmer, termasuk yang paling berpengalaman, setiap hari membuat kesalahan ketik. Yang penting adalah bisa membaca pesan kesalahan. Coba program berikut: ada satu titik koma yang terlupa di akhir baris 5.

halo.pas

```
1  program Halo;
2  begin
3    WriteLn('Halo, Dunia!');
4    WriteLn('Ini baris kedua.');
5    WriteLn('Titik koma di akhir baris ini terlupa')
6    WriteLn('Baris keempat.');
7  end.
```

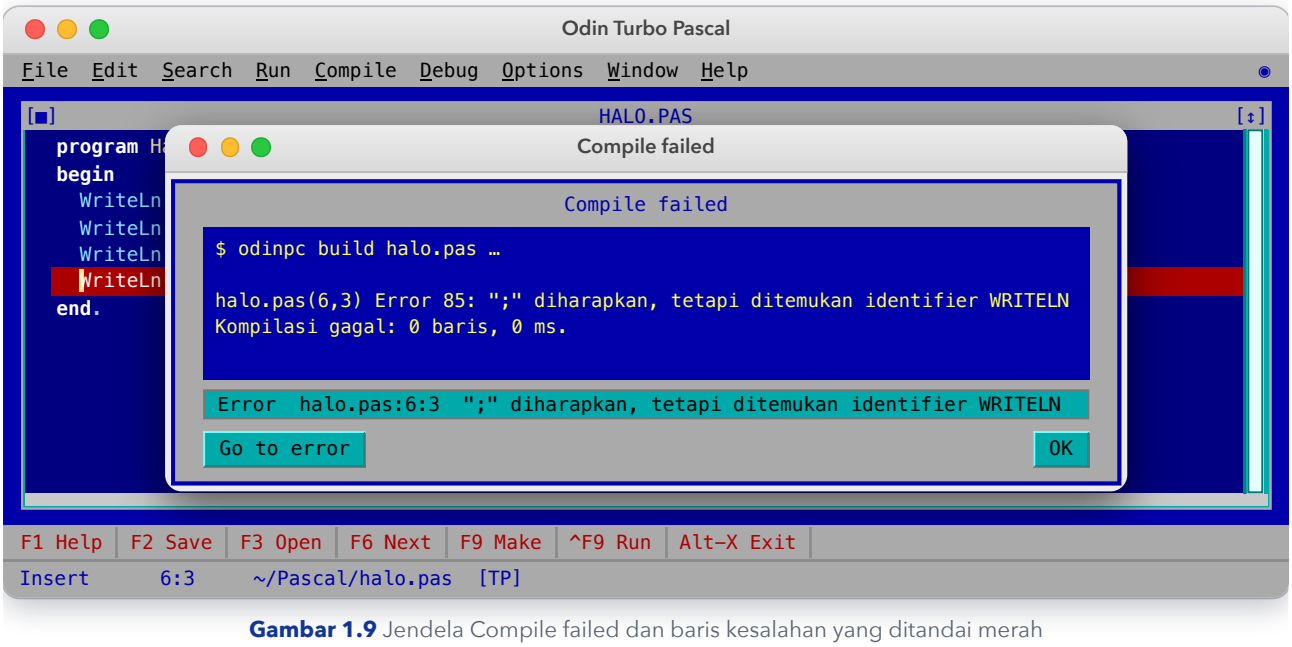
PESAN KOMPILER

```
halo.pas(6,3) Error: ";" diharapkan, tetapi ditemukan identifier WRITELN
```

Gambar 1.8 Pesan kesalahan menunjuk baris 6 kolom 3, tempat kompiler tersandung

Pesan `halo.pas(6,3) Error 85: ";" diharapkan, tetapi ditemukan identifier WRITELN` dibaca sebagai berikut: di berkas `halo.pas`, baris 6, kolom 3, kompiler mengharapkan titik koma tetapi malah menemukan kata `WriteLn`. Mengapa baris 6, padahal yang salah baris 5? Karena kompiler baru sadar ada yang kurang ketika membaca kata berikutnya. Jadi bila pesan menunjuk sebuah baris, periksa juga **akhir baris sebelumnya**.

Di dalam OTP, kompilasi yang gagal membuka jendela **Compile failed**. Pilih pesan kesalahan lalu klik **Go to error**: kursor langsung melompat ke baris dan kolom yang dimaksud, dan baris itu diberi latar merah. Perbaiki, lalu tekan Ctrl+F9 lagi. Bila ada beberapa kesalahan, Alt+F8 melompat ke kesalahan berikutnya.



Gambar 1.9 Jendela Compile failed dan baris kesalahan yang ditandai merah

> **AWAS**
>
> Kompiler berhenti pada kesalahan pertama yang ditemukannya. Setelah satu kesalahan diperbaiki, kompilasi berikutnya mungkin menemukan kesalahan lain. Itu wajar. Perbaiki satu per satu dari atas.

1.9 Rangkuman

- Program adalah kumpulan perintah berurutan. Kode sumber ditulis dalam bahasa pemrograman, lalu kompiler menerjemahkannya menjadi program executable.
- Pascal dirancang Niklaus Wirth (1970) untuk mengajarkan pemrograman terstruktur. Buku ini memakai gaya Turbo Pascal 7.
- Odin Turbo Pascal memadukan editor, kompiler Odin, User screen, dan debugger dalam satu IDE bergaya klasik.
- Ctrl+F9 menyimpan, mengompilasi, dan menjalankan program; Alt+F5 membuka User screen; F2 menyimpan.
- Program contoh dibuka dari `File ▸ Examples...` dan harus disimpan dengan nama baru sebelum diubah.

- Pesan kesalahan menyebut berkas, baris, dan kolom. Bila sebuah baris tampak benar, periksa akhir baris sebelumnya.

1.10 Latihan

1. **DASAR** Buat program baru `biodata.pas` yang menampilkan nama, kelas, dan hobi Anda, masing-masing di baris sendiri.
2. **DASAR** Jalankan `bab01_02_cek_kompiler.pas`. Catat sistem operasi dan prosesor yang dilaporkan. Cocokkah dengan komputer Anda?
3. **MENENGAH** Hapus titik pada `end.` di program Anda, lalu kompilasi. Tulis pesan kesalahan yang muncul dan jelaskan artinya dengan kata-kata sendiri.
4. **MENENGAH** Buat program yang menampilkan kotak dari tanda bintang selebar 10 karakter dan setinggi 4 baris, hanya dengan `WriteLn`.
5. **TANTANGAN** Tulis `WriteLn('10 + 20 = ', 10 + 20);` dan `WriteLn('10 + 20 = 10 + 20');`. Jalankan keduanya dan jelaskan mengapa hasilnya berbeda.

BAB 02

BAGIAN I · FONDASI

Struktur Program Pascal

TUJUAN PEMBELAJARAN

- Mengenal tujuh bagian program Pascal dan urutannya yang tetap.
- Membedakan bagian deklarasi (`uses`, `const`, `type`, `var`) dari blok perintah.
- Menulis komentar dalam tiga gaya dan memahami kapan dipakai.
- Memahami peran `begin`, `end`, dan titik koma dalam mengelompokkan pernyataan.
- Membaca pesan kesalahan akibat struktur yang keliru dan memperbaikinya.

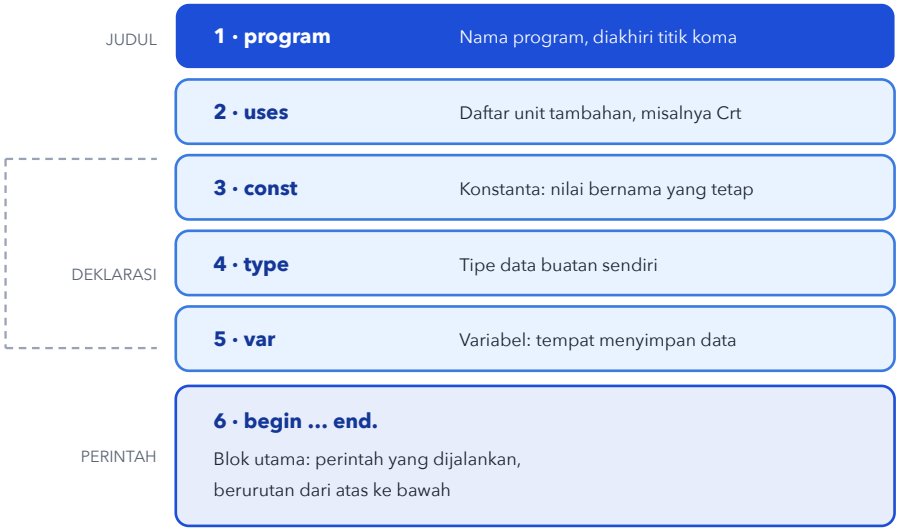
PROGRAM DALAM BAB INI

2.1	`bab02_01_struktur_dasar.pas`	Struktur dasar program Pascal
2.2	`bab02_02_komentar.pas`	Komentar
2.3	`bab02_03_blok_begin_end.pas`	Blok begin..end

Di Bab 1 Anda menjalankan program tanpa memikirkan susunannya. Sekarang kita bedah kerangkanya. Setiap program Pascal mengikuti pola yang sama: beberapa bagian pengumuman di atas, lalu satu blok perintah di bawah. Begitu Anda mengenali pola ini, program sepanjang apa pun akan terasa akrab.

2.1 Tujuh Bagian Program

Program Pascal yang lengkap tersusun dari tujuh bagian berurutan. Tidak semua bagian selalu ada: program kecil cukup punya judul dan blok perintah. Namun bila sebuah bagian dipakai, **urutannya tidak boleh ditukar**. Inilah susunannya, dari atas ke bawah.



Gambar 2.1 Kerangka program Pascal. Bagian judul dan blok perintah wajib; empat bagian deklarasi di tengah bersifat pilihan.

Bagian 2 sampai 5 disebut **bagian deklarasi** (*declaration*): di sinilah segala sesuatu "diperkenalkan" kepada kompiler sebelum dipakai, mulai dari unit, konstanta, tipe, sampai variabel. Bagian 6 adalah **blok perintah** (*statement part*): daftar perintah yang benar-benar dijalankan. Mari lihat semuanya dalam satu program.

Program 2.1 `bab02_01_struktur_dasar.pas` *Struktur dasar program Pascal* File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 2.1 — Struktur dasar program Pascal
3    Urutan bagian program: judul, uses, const, type, var, lalu blok
4    begin..end. Bagian yang tidak diperlukan boleh dihilangkan, tetapi
5    urutannya tetap seperti ini. }
6
7  program bab02_01_struktur_dasar;  { 1. judul program }
8
9  { 2. uses: daftar unit tambahan. Program ini belum memerlukannya. }
10
11 const                            { 3. konstanta: nilai yang tetap }
12   HargaTiket = 25000;
```

```
13
14  type                              { 4. tipe data buatan sendiri }
15    TJumlah = Integer;
16
17  var                               { 5. variabel: tempat menyimpan data }
18    JumlahOrang: TJumlah;
19    TotalBayar: LongInt;
20
21  begin                             { 6. blok utama, dijalankan berurutan }
22    JumlahOrang := 4;               { := artinya "isi dengan" }
23    TotalBayar := HargaTiket * JumlahOrang;
24
25    WriteLn('Harga satu tiket : ', HargaTiket);
26    WriteLn('Jumlah orang     : ', JumlahOrang);
27    WriteLn('Total bayar      : ', TotalBayar);
28  end.                              { 7. titik menandai akhir program }
```

Program ini menghitung total bayar tiket. Jalankan dengan Ctrl+F9; karena tidak meminta masukan, hasilnya langsung muncul.



Gambar 2.2 Keluaran `bab02_01_struktur_dasar.pas` di User screen

Baris	Penjelasan
7	**Judul program.** `program bab02_01_struktur_dasar;`. Nama program sebaiknya sama dengan nama berkas. Diakhiri titik koma.
9	**uses.** Di sini hanya berupa komentar karena program belum butuh unit tambahan. Bila perlu unit `Crt`, barisnya menjadi `uses Crt;`.
11–12	**const.** `HargaTiket = 25000` adalah konstanta: nilainya tetap selama program berjalan. Perhatikan tanda `=` (bukan `:=`) pada konstanta.
14–15	**type.** `TJumlah = Integer` membuat nama tipe baru yang sama dengan `Integer`. Nama tipe buatan sendiri biasanya diawali huruf `T`.
17–19	**var.** Dua variabel diumumkan beserta tipenya: `JumlahOrang` bertipe `TJumlah`, `TotalBayar` bertipe `LongInt`.
21–28	**Blok perintah.** Antara `begin` dan `end.` perintah dijalankan berurutan: mengisi variabel dengan `:=`, lalu menuliskannya.

ATURAN TURBO PASCAL 7

Tanda `:=` (titik dua lalu sama dengan) berarti "isi dengan", disebut **penugasan** (*assignment*). Tanda `=` sendirian hanya untuk konstanta dan perbandingan. Keduanya mudah tertukar; Bab 5 membahasnya lebih jauh.

2.2 Kata Tercadang dan Nama Buatan Sendiri

Perhatikan dua jenis kata di program tadi. Ada **kata tercadang** seperti `program`, `const`, `var`, `begin`, dan `end`, yang artinya sudah ditetapkan Pascal dan tidak boleh dipakai sebagai nama sendiri. Ada pula **pengenal** (*identifier*), yaitu nama yang Anda buat sendiri, seperti `HargaTiket` dan `JumlahOrang`. Di editor keduanya diberi warna berbeda.

Nama yang sah

Diawali huruf atau garis bawah, lalu boleh huruf, angka, garis bawah. Tanpa spasi.

```
JumlahOrang
Total_Bayar
Nilai2
_sementara
```

Nama yang salah

Tidak boleh diawali angka, memakai spasi, tanda hubung, atau kata tercadang.

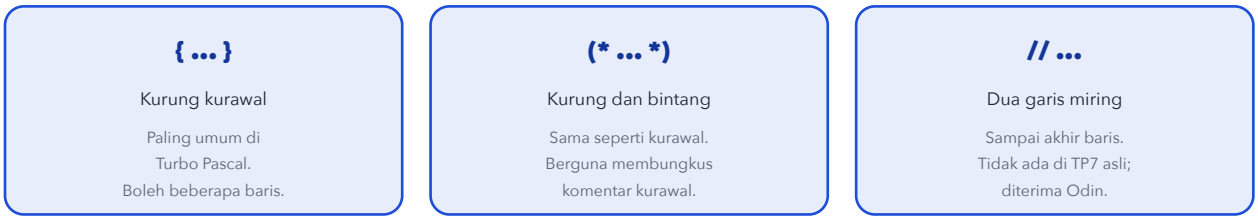
```
2Nilai      { diawali angka }
Total Bayar { ada spasi }
Harga-Tiket { ada tanda - }
begin       { kata tercadang }
```

INFO

Pascal **tidak membedakan huruf besar dan kecil** pada nama. `JumlahOrang`, `jumlahorang`, dan `JUMLAHORANG` dianggap sama. Walau begitu, biasakan menulis konsisten, misalnya gaya `HurufDepanKapital`, agar program rapi dan mudah dibaca.

2.3 Komentar

Komentar adalah catatan untuk pembaca manusia. Kompiler melewatinya sama sekali, sehingga komentar tidak pernah mengubah jalannya program. Gunanya menjelaskan maksud kode, menandai bagian penting, atau sementara "mematikan" baris tertentu. Pascal mengenal tiga gaya komentar.



Gambar 2.3 Tiga gaya komentar. Teks di dalamnya diabaikan kompiler.

Program 2.2 `bab02_02_komentar.pas` *Komentar* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 2.2 — Komentar
  Komentar adalah catatan untuk manusia. Kompiler melewatinya, jadi
  komentar tidak mengubah jalannya program. }

program bab02_02_komentar;

var
  Panjang, Lebar: Integer;   { komentar di akhir baris }

begin
  { Gaya 1: kurung kurawal. Paling umum dipakai di Turbo Pascal. }
  Panjang := 8;

  (* Gaya 2: kurung dan bintang.
     Bisa beberapa baris, sama seperti gaya kurung kurawal. *)
  Lebar := 5;

  // Gaya 3: dua garis miring, berlaku sampai akhir baris.
  // Turbo Pascal 7 asli belum mengenal gaya ini; Odin dan FPC mengenalnya.

  WriteLn('Luas = ', Panjang * Lebar);

  { Komentar juga dipakai untuk "mematikan" baris sementara.
    Baris di bawah tidak dijalankan karena berada di dalam komentar:
    WriteLn('Baris ini tidak akan tampil');
  }

  (* Gaya yang berbeda boleh disisipkan: { ini aman } di dalam *)
  WriteLn('Selesai. Hanya dua baris yang tampil di layar.');
end.
```



Gambar 2.4 Keluaran `bab02_02_komentar.pas` di User screen

Meski programnya panjang karena banyak komentar, hanya dua baris yang tampil di layar. Semua teks di dalam `{ }`, `(* *)`, dan setelah `//` dilewati. Perhatikan baris yang sengaja "dimatikan": sebuah `WriteLn` yang diletakkan di dalam komentar tidak pernah dijalankan. Teknik ini sering dipakai untuk mencoba-coba tanpa menghapus kode.

> **AWAS**
>
> Komentar kurung kurawal **tidak bisa bersarang**: `{ luar { dalam } luar }` akan membingungkan kompiler, karena `}` pertama sudah menutup komentar. Bila perlu membungkus komentar kurawal, pakai gaya berbeda di luarnya: `(* { ini aman } *)`.

2.4 Blok begin..end dan Titik Koma

Kata `begin` dan `end` mengelompokkan beberapa pernyataan menjadi **satu blok**, sehingga diperlakukan sebagai satu kesatuan. Ini penting karena banyak perintah Pascal, seperti `if` dan `for`, hanya menguasai **satu** pernyataan sesudahnya. Bila Anda ingin perintah itu menguasai beberapa pernyataan, bungkus dengan `begin`..`end`.

BENTUK UMUM

```
begin
  pernyataan1;
  pernyataan2;
  pernyataan3
end
```

Di dalam blok, **titik koma memisahkan pernyataan satu dari yang lain**. Titik koma bukan penutup pernyataan, melainkan pemisah, sehingga pernyataan terakhir sebelum `end` boleh tanpa titik koma. Banyak programmer tetap menambahkannya demi kerapian, agar mudah menyisipkan baris baru.

Program 2.3 `bab02_03_blok_begin_end.pas` *Blok begin..end* File ▸ Examples…

```
1  { Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 2.3 - Blok begin..end
3    begin..end mengelompokkan beberapa pernyataan menjadi satu blok.
4    Titik koma memisahkan pernyataan; sebelum end boleh tanpa titik koma. }
5
6  program bab02_03_blok_begin_end;
7
8  var
9    Nilai: Integer;
10
11 begin
12   Nilai := 80;
13
14   { Tanpa begin..end, if hanya menguasai SATU pernyataan sesudahnya }
15   if Nilai >= 75 then
16     WriteLn('Satu pernyataan: Lulus');
17
18   { Dengan begin..end, semua pernyataan di dalam blok ikut if }
19   if Nilai >= 75 then
20   begin
21     WriteLn('Blok dimulai');
22     WriteLn('Nilai Anda ', Nilai);
23     WriteLn('Blok selesai');
24   end;
25
26   { Blok boleh bersarang. Rapikan dengan indentasi agar mudah dibaca. }
27   begin
28     WriteLn('Blok luar');
29     begin
30       WriteLn('  Blok dalam')
31     end                          { tanpa titik koma sebelum end: tetap sah }
32   end;
33 end.
```



Gambar 2.5 Keluaran `bab02_03_blok_begin_end.pas` di User screen

Perhatikan dua pemakaian `if` di baris 14 dan 18. Yang pertama hanya menguasai satu `WriteLn`. Yang kedua, karena diikuti `begin`..`end`, menguasai tiga `WriteLn` sekaligus. Bagian akhir menunjukkan blok bersarang: sebuah `begin`..`end` di dalam `begin`..`end` lain. Indentasi (menjorokkan baris ke dalam) memperjelas blok mana milik siapa.

> ★ **INGAT**
>
> Perhatikan dua tanda akhir yang berbeda: `end;` (dengan titik koma) menutup sebuah blok di tengah program, sedangkan `end.` (dengan **titik**) menutup seluruh program dan hanya muncul sekali, di paling bawah.

2.5 Kesalahan yang Sering Terjadi

Struktur yang keliru adalah sumber kesalahan pemula yang paling umum. Berikut beberapa yang sering muncul, lengkap dengan pesan kompiler Odin yang sebenarnya.

Lupa titik di akhir program. Mengganti `end.` dengan `end;` membuat kompiler terus menunggu program yang tidak kunjung ditutup.

akhir.pas

```
1  program Akhir;
2  begin
3    WriteLn('Halo');
4  end;
```

PESAN KOMPILER

```
akhir.pas(4,4) Error: "." diharapkan, tetapi ditemukan ";"
```

Gambar 2.6 Kompiler mencapai akhir berkas tanpa menemukan titik penutup

Menukar urutan bagian. Menaruh `var` sebelum `const` masih boleh di Pascal modern, tetapi menaruh deklarasi **setelah** `begin` tidak diperbolehkan: semua pengumuman harus selesai sebelum blok perintah dimulai.

urutan.pas

```
1  program Urutan;
2  begin
3    var Jumlah: Integer;
4    Jumlah := 5;
5    WriteLn(Jumlah);
6  end.
```

PESAN KOMPILER

```
urutan.pas(3,3) Error: Kesalahan dalam statement, tetapi ditemukan VAR
```

Gambar 2.7 Variabel diumumkan setelah begin, padahal deklarasi harus mendahuluinya

Memakai variabel yang belum diumumkan. Setiap nama harus dikenalkan di bagian `var` dulu.

belum.pas

```
1  program Belum;
2  begin
3    Hasil := 10;
4    WriteLn(Hasil);
5  end.
```

PESAN KOMPILER

```
belum.pas(3,3) Error: Identifier tidak dikenal: Hasil
```

Gambar 2.8 Nama dipakai tanpa diumumkan di bagian var

2.6 Rangkuman

- Program Pascal tersusun dari tujuh bagian berurutan: `program`, `uses`, `const`, `type`, `var`, blok `begin` .. `end.` Bagian deklarasi boleh dihilangkan, tetapi urutannya tetap.
- Bagian deklarasi memperkenalkan unit, konstanta, tipe, dan variabel sebelum dipakai. Blok perintah berisi langkah yang dijalankan.
- Kata tercadang sudah ditetapkan bahasa; pengenal adalah nama buatan sendiri yang diawali huruf atau garis bawah, tanpa spasi. Pascal tidak membedakan huruf besar dan kecil.
- Komentar ditulis dengan `{ }`, `(* *)`, atau `//`. Isinya diabaikan kompiler.
- `begin` .. `end` mengelompokkan beberapa pernyataan menjadi satu blok. Titik koma memisahkan pernyataan.
- `end;` menutup blok; `end.` dengan titik menutup seluruh program.

2.7 Latihan

1. **DASAR** Tulis program yang mengumumkan konstanta `Pi = 3.14` dan variabel `Jari`, lalu menampilkan keliling lingkaran untuk `Jari := 7`.

2. **DASAR** Tambahkan komentar pada program `bab02_01_struktur_dasar.pas` yang menjelaskan maksud setiap variabel. Pastikan program tetap berjalan sama.

3. **MENENGAH** Buat program dengan dua `if`: satu menguasai satu `WriteLn`, satu lagi menguasai blok `begin`..`end` berisi tiga `WriteLn`.

4. **MENENGAH** Sengaja hapus satu titik koma di antara dua pernyataan, kompilasi, lalu catat dan jelaskan pesan kesalahannya.

5. **TANTANGAN** Pindahkan bagian `var` ke bawah `begin`. Apa pesan kompilernya? Jelaskan mengapa Pascal mengharuskan deklarasi mendahului perintah.

BAB

03

BAGIAN I · FONDASI

Tipe Data, Variabel, dan Konstanta

TUJUAN PEMBELAJARAN

- Memahami guna tipe data dan cara memilih tipe yang tepat.
- Memakai tipe bilangan (`Integer`, `LongInt`, `Real`), karakter (`Char`, `string`), dan `Boolean`.
- Mengubah antar-tipe dengan `Trunc`, `Round`, `Ord`, dan `Chr`.
- Membedakan variabel dari konstanta.
- Mengetahui ukuran dan rentang setiap tipe, serta apa yang terjadi bila nilai melampauinya.

PROGRAM DALAM BAB INI

3.1	`bab03_01_tipe_integer_real.pas`	Tipe Integer dan Real
3.2	`bab03_02_tipe_char_string.pas`	Tipe Char dan string
3.3	`bab03_03_tipe_boolean.pas`	Tipe Boolean
3.4	`bab03_04_konstanta.pas`	Konstanta
3.5	`bab03_05_batas_tipe_data.pas`	Batas tipe data

Setiap data yang disimpan program punya jenis: bilangan bulat, bilangan pecahan, satu huruf, sepotong teks, atau nilai benar/salah. Jenis itu disebut **tipe data**. Memilih tipe yang tepat membuat program benar, hemat memori, dan mudah dibaca. Bab ini mengenalkan tipe-tipe dasar yang akan Anda pakai di sepanjang buku.

3.1 Variabel dan Tipenya

Sebuah **variabel** adalah tempat bernama di memori untuk menyimpan nilai yang bisa berubah. Sebelum dipakai, variabel harus diumumkan di bagian `var` beserta tipenya. Tipe menentukan tiga hal: nilai apa yang boleh disimpan, berapa besar memori yang dipakai, dan operasi apa yang berlaku.

BENTUK UMUM

```
var
  namaVariabel: Tipe;
  namaLain, namaLagi: Tipe;   { beberapa nama bertipe sama }
```

Bayangkan variabel sebagai kotak berlabel. Labelnya adalah nama, isinya adalah nilai, dan bentuk kotaknya adalah tipe, yang menentukan isi macam apa yang muat.



3.2 Bilangan: Integer dan Real

Ada dua kelompok bilangan. **Bilangan bulat** (tanpa pecahan) disimpan dengan tipe seperti `Integer` dan `LongInt`. **Bilangan pecahan** (dengan koma) disimpan dengan `Real`. Di Pascal, pemisah desimal adalah **titik**, bukan koma: tulis `165.5`, bukan `165,5`.

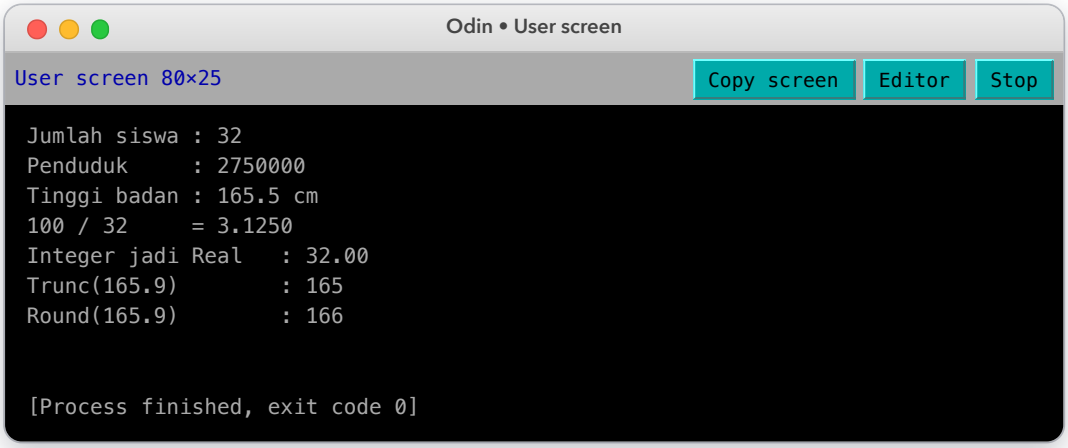
7 **ATURAN TURBO PASCAL 7**

Dalam mode Turbo Pascal, `Integer` berukuran 2 byte dengan rentang **−32768 sampai 32767**. Untuk bilangan yang lebih besar, misalnya jumlah penduduk, pakai `LongInt` (4 byte, sampai sekitar 2 miliar). Memberi nilai di luar rentang akan memicu kesalahan, bukan menghasilkan angka yang salah diam-diam.

Program 3.1 `bab03_01_tipe_integer_real.pas` *Tipe Integer dan Real* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 3.1 — Tipe Integer dan Real
3    Integer dan LongInt menyimpan bilangan bulat. Real menyimpan bilangan
4    pecahan. Bilangan Real ditampilkan dengan format :lebar:desimal. }
5
6  program bab03_01_tipe_integer_real;
7
```

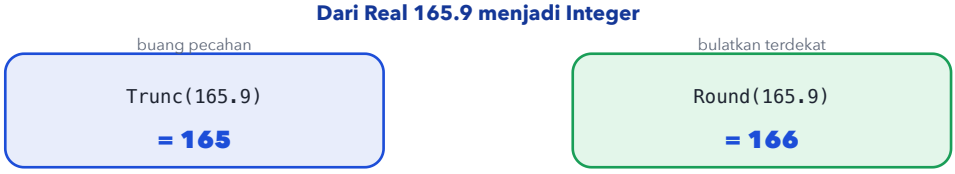
```
8  var
9    JumlahSiswa: Integer;        { bilangan bulat kecil, -32768..32767 }
10   Penduduk: LongInt;           { bilangan bulat besar, sampai 2 miliar }
11   TinggiBadan: Real;           { bilangan pecahan, memakai titik desimal }
12   RataRata: Real;
13
14 begin
15   JumlahSiswa := 32;
16   Penduduk := 2750000;
17   TinggiBadan := 165.5;        { pemisah desimal di Pascal adalah titik }
18
19   WriteLn('Jumlah siswa : ', JumlahSiswa);
20   WriteLn('Penduduk     : ', Penduduk);
21   WriteLn('Tinggi badan : ', TinggiBadan:0:1, ' cm');  { 1 angka desimal }
22
23   { Pembagian dengan / selalu menghasilkan Real, walau angkanya bulat }
24   RataRata := 100 / JumlahSiswa;
25   WriteLn('100 / 32     = ', RataRata:0:4);
26
27   { Integer boleh dimasukkan ke Real, tetapi Real tidak boleh langsung
28     dimasukkan ke Integer. Gunakan Trunc (buang pecahan) atau Round. }
29   TinggiBadan := JumlahSiswa;
30   WriteLn('Integer jadi Real   : ', TinggiBadan:0:2);
31   JumlahSiswa := Trunc(165.9);
32   WriteLn('Trunc(165.9)        : ', JumlahSiswa);
33   JumlahSiswa := Round(165.9);
34   WriteLn('Round(165.9)        : ', JumlahSiswa);
35 end.
```



Gambar 3.1 Keluaran `bab03_01_tipe_integer_real.pas` di User screen

Dua hal penting dari program ini. Pertama, **pembagian dengan `/` selalu menghasilkan `Real`**, walau kedua angkanya bulat: `100 / 32` memberi `3.1250`, bukan `3`. Kedua, bilangan `Real` ditampilkan dengan format `:lebar:desimal`. `TinggiBadan:0:1` berarti "lebar secukupnya, satu angka di belakang titik". Tanpa format, `Real` tampil dalam notasi ilmiah yang sulit dibaca, jadi **selalu format setiap `Real`** saat ditampilkan.

Perhatikan juga perubahan antar-tipe di akhir program. `Integer` boleh langsung dimasukkan ke `Real` (baris 30), tetapi sebaliknya tidak: `Real` harus diubah dulu. `Trunc` membuang bagian pecahan (`165.9` menjadi `165`), sedangkan `Round` membulatkan ke bilangan terdekat (`165.9` menjadi `166`).



Gambar 3.2 Dua cara mengubah `Real` menjadi `Integer`.

3.3 Teks: Char dan String

Tipe `Char` menyimpan **tepat satu karakter**, sedangkan `string` menyimpan **deretan karakter**. Teks ditulis di antara tanda petik tunggal: `'A'` untuk sebuah `Char`, `'Budi Santoso'` untuk sebuah `string`.

Program 3.2 `bab03_02_tipe_char_string.pas` *Tipe Char dan string* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 3.2 — Tipe Char dan string
  Char menyimpan satu karakter, string menyimpan deretan karakter
  (paling banyak 255). Teks ditulis di antara tanda petik tunggal. }

program bab03_02_tipe_char_string;

var
  Huruf: Char;                { tepat satu karakter }
  Nama: string;               { teks sampai 255 karakter }
  Kota: string[9];            { teks dibatasi 9 karakter }
  Kalimat: string;

begin
  Huruf := 'A';
  Nama := 'Budi Santoso';
  Kota := 'Kandangan Barat';    { kelebihan karakter dipotong }

  WriteLn('Huruf : ', Huruf);
  WriteLn('Nama  : ', Nama);
  WriteLn('Kota  : ', Kota, ' (dipotong menjadi 9 karakter)');

  { Setiap karakter punya nomor kode (ASCII). Ord dan Chr saling balik. }
  WriteLn('Kode huruf A   : ', Ord(Huruf));
  WriteLn('Karakter ke-66 : ', Chr(66));

  { Karakter di dalam string diambil dengan indeks, dimulai dari 1 }
  WriteLn('Huruf pertama nama: ', Nama[1]);
  WriteLn('Panjang nama      : ', Length(Nama), ' karakter');

  { Tanda + menyambung string. Petik di dalam teks ditulis dua kali. }
  Kalimat := 'Halo, ' + Nama + '! Ini Jum''at yang cerah.';
  WriteLn(Kalimat);
end.
```



Gambar 3.3 Keluaran `bab03_02_tipe_char_string.pas` di User screen

Beberapa ide baru muncul di sini:

- **Panjang string dibatasi.** `string[9]` hanya memuat 9 karakter; kelebihannya dipotong, sehingga `'Kandangan Barat'` menjadi `'Kandangan'`. `string` biasa memuat hingga 255 karakter.
- **Setiap karakter punya kode.** `Ord('A')` memberi kode ASCII-nya, `65`. Kebalikannya, `Chr(66)` memberi karakter `'B'`. `Ord` dan `Chr` saling membalik.
- **Karakter dalam string diambil lewat indeks**, mulai dari **1**: `Nama[1]` adalah huruf pertama. `Length(Nama)` memberi jumlah karakternya.
- **Tanda `+` menyambung string.** Untuk menulis tanda petik di dalam teks, ketik dua kali: `'Jum''at'` menghasilkan `Jum'at`.

> **7** **ATURAN TURBO PASCAL 7**
>
> Dalam mode Turbo Pascal, `string` adalah *short string*: panjang maksimumnya 255 karakter, dan karakter pertamanya (indeks 1) adalah isi sesungguhnya. Bab 12 membahas pengolahan string lebih dalam.

3.4 Benar atau Salah: Boolean

Tipe `Boolean` hanya punya dua nilai: `True` (benar) dan `False` (salah). Tipe ini tampak sederhana, tetapi menjadi dasar semua pengambilan keputusan di Bab 6. Hasil sebuah perbandingan, seperti `Umur >= 17`, adalah nilai `Boolean` yang bisa disimpan dan dipakai ulang.

Program 3.3 `bab03_03_tipe_boolean.pas` *Tipe Boolean* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 3.3 — Tipe Boolean
3    Boolean hanya punya dua nilai: True (benar) dan False (salah).
4    Hasil perbandingan seperti Umur >= 17 adalah nilai Boolean. }
5
6  program bab03_03_tipe_boolean;
7
8  var
9    Umur: Integer;
```

```
10    SudahDewasa: Boolean;
11    PunyaKTP: Boolean;
12    Hujan: Boolean;
13
14  begin
15    Umur := 19;
16    SudahDewasa := Umur >= 17;     { hasil perbandingan disimpan }
17    PunyaKTP := True;
18    Hujan := False;
19
20    WriteLn('Umur          : ', Umur);
21    WriteLn('Sudah dewasa? : ', SudahDewasa);    { tampil TRUE atau FALSE }
22    WriteLn('Punya KTP?    : ', PunyaKTP);
23    WriteLn('Hujan?        : ', Hujan);
24
25    { Variabel Boolean bisa langsung dipakai sebagai syarat }
26    if SudahDewasa then
27      WriteLn('Boleh membuat SIM.');
28    if not Hujan then               { not membalik True <-> False }
29      WriteLn('Tidak perlu membawa payung.');
30  end.
```



Gambar 3.4 Keluaran `bab03_03_tipe_boolean.pas` di User screen

Variabel `Boolean` ditampilkan sebagai `TRUE` atau `FALSE`. Yang menarik, variabel `Boolean` bisa langsung dipakai sebagai syarat `if` tanpa perbandingan tambahan: `if SudahDewasa then` sudah cukup. Operator `not` membalik nilainya, sehingga `not Hujan` bernilai `True` ketika `Hujan` bernilai `False`.

3.5 Konstanta

Kadang sebuah nilai memang tidak boleh berubah selama program berjalan: harga satuan, nilai pi, nama toko. Nilai seperti itu sebaiknya diberi nama lewat **konstanta** di bagian `const`. Keuntungannya dua: program lebih mudah dibaca (`Diskon` lebih jelas daripada angka `0.10` yang berserakan), dan bila nilainya perlu diganti, cukup ubah di satu tempat.

BENTUK UMUM

```
const
  NamaKonstanta = nilai;    { pakai = , bukan := }
```

Program 3.4 `bab03_04_konstanta.pas` *Konstanta* File ▸ Examples…

```
{ Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 3.4 – Konstanta
  Konstanta adalah nama untuk nilai yang tidak berubah selama program
  berjalan. Bila nilainya perlu diganti, cukup ubah di satu tempat. }

program bab03_04_konstanta;

const
  NamaToko = 'Toko Buku Pelita';   { konstanta teks }
  HargaBuku = 45000;               { konstanta bilangan bulat }
  Diskon = 0.10;                   { konstanta pecahan: 10 persen }
  JumlahBeli = 3;

var
  Total, Potongan: Real;

begin
  Total := HargaBuku * JumlahBeli;
  Potongan := Total * Diskon;

  WriteLn(NamaToko);
  WriteLn('Harga buku : Rp ', HargaBuku);
  WriteLn('Jumlah     : ', JumlahBeli, ' buku');
  WriteLn('Total      : Rp ', Total:0:0);
  WriteLn('Diskon 10% : Rp ', Potongan:0:0);
  WriteLn('Bayar      : Rp ', Total - Potongan:0:0);

  { Konstanta tidak bisa diubah. Hapus tanda komentar pada baris
    berikut, lalu kompilasi: kompiler akan menolaknya.
    HargaBuku := 50000;
  }
end.
```

Odin • User screen

User screen 80×25 | Copy screen | Editor | Stop

```
Toko Buku Pelita
Harga buku : Rp 45000
Jumlah     : 3 buku
Total      : Rp 135000
Diskon 10% : Rp 13500
Bayar      : Rp 121500


[Process finished, exit code 0]
```

Gambar 3.5 Keluaran `bab03_04_konstanta.pas` di User screen

Perbedaan penting dari variabel: konstanta memakai tanda `=` (bukan `:=`) saat diumumkan, dan nilainya **tidak bisa diubah** setelah itu. Mencoba menugaskan nilai baru ke konstanta akan ditolak kompiler:

konstanta.pas

```
program Toko;
const
```

```
  Harga = 45000;
begin
  Harga := 50000;
  WriteLn(Harga);
end.
```

PESAN KOMPILER

```
konstanta.pas(5,3) Error: Penugasan tidak sah
```

Gambar 3.6 Konstanta tidak bisa ditugasi nilai baru

3.6 Ukuran dan Batas Tipe

Setiap tipe menempati sejumlah byte di memori dan punya rentang nilai tertentu. Fungsi bawaan membantu memeriksanya: `SizeOf` memberi ukuran dalam byte, `Low` dan `High` memberi nilai terkecil dan terbesar.

Program 3.5 `bab03_05_batas_tipe_data.pas` *Batas tipe data* File ▸ Examples…

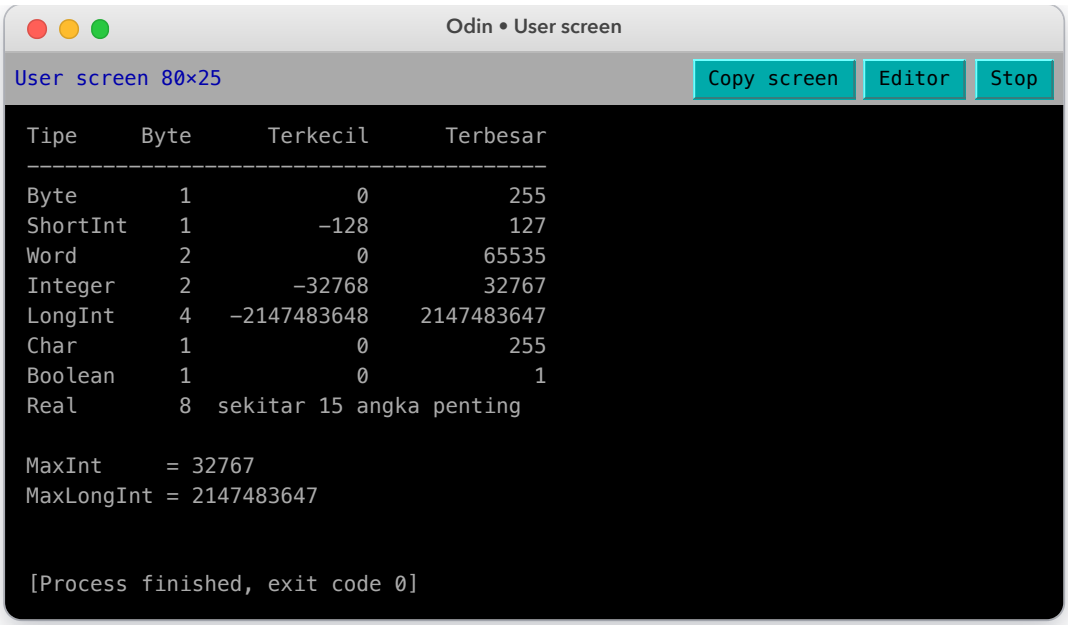
```
{ Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 3.5 - Batas tipe data
  Setiap tipe punya ukuran memori dan rentang nilai. Low dan High
  memberi nilai terkecil dan terbesar, SizeOf memberi ukurannya. }

program bab03_05_batas_tipe_data;

begin
  WriteLn('Tipe     Byte      Terkecil      Terbesar');
  WriteLn('----------------------------------------');
  WriteLn('Byte    ', SizeOf(Byte):5, Low(Byte):14, High(Byte):14);
  WriteLn('ShortInt', SizeOf(ShortInt):5, Low(ShortInt):14, High(ShortInt):14);
  WriteLn('Word    ', SizeOf(Word):5, Low(Word):14, High(Word):14);
  WriteLn('Integer ', SizeOf(Integer):5, Low(Integer):14, High(Integer):14);
  WriteLn('LongInt ', SizeOf(LongInt):5, Low(LongInt):14, High(LongInt):14);
  WriteLn('Char    ', SizeOf(Char):5, Ord(Low(Char)):14, Ord(High(Char)):14);
  WriteLn('Boolean ', SizeOf(Boolean):5, Ord(False):14, Ord(True):14);
  WriteLn('Real    ', SizeOf(Real):5, '  sekitar 15 angka penting');
  WriteLn;

  { MaxInt dan MaxLongInt adalah konstanta bawaan Pascal }
  WriteLn('MaxInt     = ', MaxInt);
  WriteLn('MaxLongInt = ', MaxLongInt);

  { Range checking aktif (Options > Compiler), jadi nilai yang keluar
    dari rentang menghentikan program dengan runtime error 201:
      var B: Byte;  ...  B := 255;  B := B + 1;
    Pilih tipe yang rentangnya cukup untuk data Anda. }
end.
```



Gambar 3.7 Keluaran `bab03_05_batas_tipe_data.pas` di User screen

Tabel 3.1 Tipe bilangan bulat dan rentangnya (mode Turbo Pascal)

Tipe	Byte	Rentang	Pakai untuk
`Byte`	1	0 … 255	Umur, nilai ujian, hitungan kecil tak-negatif
`ShortInt`	1	−128 … 127	Selisih kecil yang bisa negatif
`Word`	2	0 … 65535	Tahun, jumlah tak-negatif sedang
`Integer`	2	−32768 … 32767	Bilangan bulat umum
`LongInt`	4	≈ ±2,1 miliar	Penduduk, uang, hasil perkalian besar

Mengapa batas ini penting? Karena memberi nilai di luar rentang tipe akan **menghentikan program**. Misalnya sebuah `Byte` hanya memuat 0 sampai 255; menambah 1 pada 255 melampaui batas, dan program berhenti dengan runtime error 201.

luap.pas

```
1  program Luap;
2  var
3    B: Byte;
4  begin
5    B := 255;
6    B := B + 1;
7    WriteLn(B);
8  end.
```



Gambar 3.8 Range check menghentikan program yang melampaui batas tipe Byte (kode 201)

TIPS

Range check dan overflow check aktif secara bawaan di kompiler Odin, sehingga kesalahan seperti ini ketahuan langsung, bukan menghasilkan angka yang keliru. Biarkan tetap aktif selama belajar, dan pilih tipe yang rentangnya cukup untuk data Anda, misalnya `LongInt` untuk uang atau jumlah penduduk.

3.7 Rangkuman

- Tipe data menentukan nilai yang boleh disimpan, ukuran memori, dan operasi yang berlaku. Variabel diumumkan di `var` beserta tipenya.
- `Integer` (2 byte, ±32 ribu) dan `LongInt` (4 byte, ±2 miliar) untuk bilangan bulat; `Real` untuk pecahan dengan pemisah titik.
- Pembagian `/` selalu menghasilkan `Real`. Format setiap `Real` dengan `:lebar:desimal` saat ditampilkan.
- `Trunc` membuang pecahan, `Round` membulatkan; `Ord` dan `Chr` mengubah antara karakter dan kodenya.
- `Char` satu karakter, `string` deretan karakter (maks 255 di mode TP), diambil lewat indeks mulai 1.
- `Boolean` bernilai `True` / `False` dan bisa langsung menjadi syarat.
- Konstanta (`const`, pakai `=`) menamai nilai tetap yang tidak bisa diubah. Melampaui rentang tipe memicu runtime error 201.

3.8 Latihan

1. **DASAR** Buat program yang menyimpan nama, umur, dan tinggi badan (Real) Anda, lalu menampilkannya rapi. Format tinggi badan dengan satu angka desimal.
2. **DASAR** Tampilkan kode ASCII huruf `'a'` sampai `'e'` memakai `Ord`, dan karakter untuk kode 48 sampai 57 memakai `Chr`.
3. **MENENGAH** Hitung dan tampilkan `22 / 7` dengan 2, 4, dan 6 angka desimal memakai format `:0:n`. Amati perbedaannya.

4. **MENENGAH** Simpan hasil perbandingan `Nilai >= 75` ke variabel `Boolean` bernama `Lulus`, lalu pakai sebagai syarat `if`.

5. **TANTANGAN** Pakai tipe `Byte` untuk sebuah umur, lalu sengaja beri nilai 300. Jalankan dan jelaskan apa yang terjadi serta mengapa.

BAB 04

BAGIAN I · FONDASI

Masukan dan Keluaran

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

TUJUAN PEMBELAJARAN

- Menampilkan teks dan nilai dengan `Write` dan `WriteLn`.
- Membaca masukan dari keyboard dengan `Read` dan `ReadLn`.
- Merapikan keluaran dengan format `:lebar` dan `:lebar:desimal`.
- Menggabungkan masukan dan keluaran dalam program interaktif.
- Memahami apa yang terjadi bila masukan tidak sesuai tipe yang diminta.

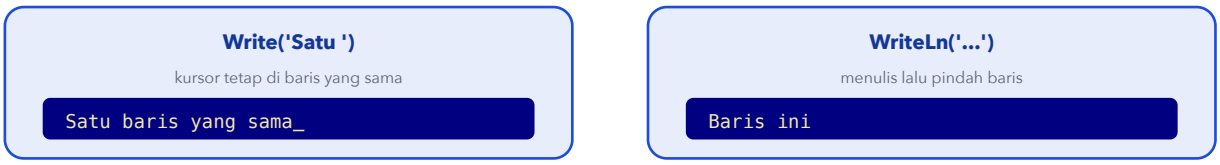
PROGRAM DALAM BAB INI

4.1	`bab04_01_write_writeln.pas`	Write dan WriteLn
4.2	`bab04_02_read_readln.pas`	Read dan ReadLn
4.3	`bab04_03_format_keluaran.pas`	Format keluaran
4.4	`bab04_04_biodata_sederhana.pas`	Latihan: biodata sederhana

Program yang berguna hampir selalu berkomunikasi: menampilkan hasil ke layar dan menerima data dari pengguna. Di Bab 1 Anda sudah memakai `WriteLn`. Bab ini melengkapinya dengan membaca masukan dan merapikan tampilan, sehingga program Anda bisa berdialog dengan penggunanya.

4.1 Menampilkan: Write dan WriteLn

Ada dua perintah untuk menulis ke layar. `WriteLn` menulis lalu **pindah ke baris baru**. `Write` menulis **tanpa pindah baris**, sehingga kursor tetap di tempat dan tulisan berikutnya menyambung. Keduanya bisa menerima beberapa nilai sekaligus, dipisah koma.



Gambar 4.1 `Write` menyambung di baris yang sama; `WriteLn` mengakhiri baris.

Program 4.1 `bab04_01_write_writeln.pas` *Write dan WriteLn* — File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 4.1 — Write dan WriteLn
  Write menulis tanpa pindah baris; WriteLn menulis lalu pindah baris.
  Keduanya bisa menerima beberapa nilai sekaligus, dipisah koma. }

program bab04_01_write_writeln;

var
  Umur: Integer;

begin
  Umur := 16;

  Write('Satu ');                { kursor tetap di baris yang sama }
  Write('baris ');
  WriteLn('yang sama.');         { pindah baris setelah menulis }

  WriteLn('Baris kedua.');
  WriteLn;                       { WriteLn kosong: baris kosong }

  { Teks, variabel, dan ekspresi boleh dicampur dalam satu WriteLn }
  WriteLn('Umur saya ', Umur, ' tahun.');
  WriteLn('Lima tahun lagi: ', Umur + 5, ' tahun.');
  WriteLn('Hasil 7 x 6 = ', 7 * 6);

  { Petik tunggal di dalam teks ditulis dua kali: '' }
  WriteLn('Hari ini hari Jum''at.');
end.
```



Gambar 4.2 Keluaran `bab04_01_write_writeln.pas` di User screen

Tiga `Write` pertama menghasilkan satu baris `Satu baris yang sama.` karena tidak ada perpindahan baris sampai `WriteLn` di akhir. `WriteLn;` tanpa isi mencetak baris kosong. Yang paling berguna: dalam satu perintah, **teks, variabel, dan ekspresi boleh dicampur**. `WriteLn('Umur saya ', Umur, ' tahun.')` menulis teks apa adanya, tetapi `Umur + 5` dan `7 * 6` dihitung dulu, lalu hasilnya yang ditulis. Ingat, teks dalam petik ditulis apa adanya; di luar petik dihitung sebagai ekspresi.

4.2 Membaca: Read dan ReadLn

Untuk menerima masukan dari keyboard, Pascal memakai `ReadLn`. Perintah ini menunggu pengguna mengetik lalu menekan **Enter**, kemudian menyimpan yang diketik ke variabel. Polanya hampir selalu sama: `Write` sebuah pertanyaan dulu (tanpa pindah baris, agar jawaban diketik di sampingnya), lalu `ReadLn` menangkap jawabannya.

BENTUK UMUM

```
Write('Pertanyaan: ');
ReadLn(variabel);
```

Program 4.2 `bab04_02_read_readln.pas` *Read dan ReadLn* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 4.2 — Read dan ReadLn
3    ReadLn membaca masukan dari keyboard sampai Enter ditekan, lalu
4    menyimpannya ke variabel. Ketik jawaban di User screen. }
5
6  program bab04_02_read_readln;
7
8  var
9    Nama: string;
10   Umur: Integer;
11   Tinggi: Real;
12   A, B: Integer;
13
14 begin
15   Write('Nama Anda   : ');      { Write: jawaban diketik di baris yang sama }
16   ReadLn(Nama);                 { membaca satu baris teks }
17   Write('Umur (tahun): ');
```

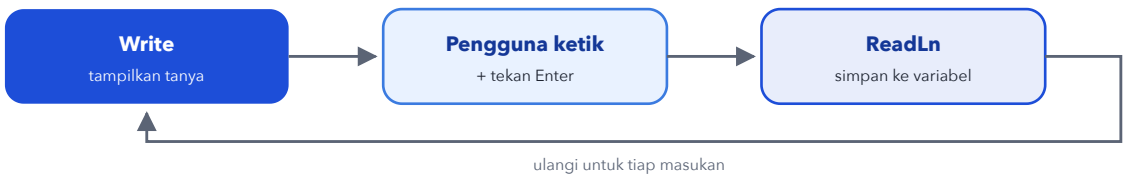
```
18    ReadLn(Umur);                { membaca bilangan bulat }
19    Write('Tinggi (cm) : ');
20    ReadLn(Tinggi);              { membaca bilangan pecahan, misal 160.5 }
21
22    { Beberapa bilangan bisa dibaca sekaligus, dipisahkan spasi }
23    Write('Ketik dua bilangan, pisahkan dengan spasi: ');
24    ReadLn(A, B);
25
26    WriteLn;
27    WriteLn('Halo, ', Nama, '!');
28    WriteLn('Tahun depan umur Anda ', Umur + 1, ' tahun.');
29    WriteLn('Tinggi Anda ', Tinggi:0:1, ' cm.');
30    WriteLn(A, ' + ', B, ' = ', A + B);
31
32    { Bila yang diketik bukan angka untuk variabel bilangan, program
33      berhenti dengan runtime error 106. Bab 12 membahas cara mencegahnya. }
34  end.
```

Jalankan dengan Ctrl + F9, lalu ketik jawaban di User screen. Pada rekaman di bawah, teks berwarna kuning adalah yang **diketik pengguna**, sisanya ditulis program.



Gambar 4.3 Keluaran `bab04_02_read_readln.pas` di User screen

`ReadLn` bisa membaca teks (`Nama`), bilangan bulat (`Umur`), maupun pecahan (`Tinggi`). Ia bahkan bisa membaca **beberapa nilai sekaligus** bila pengguna memisahkannya dengan spasi: `ReadLn(A, B)` menangkap dua bilangan dari satu baris. Alur masukan dan keluaran digambarkan berikut.



Gambar 4.4 Pola masukan: tampilkan pertanyaan, pengguna mengetik, program menyimpan ke variabel.

> **AWAS**
>
> Bila variabel bertipe bilangan tetapi pengguna mengetik huruf, program berhenti dengan runtime error 106 (masukan tidak sah). Untuk sekarang, ketiklah masukan sesuai yang diminta. Cara memeriksa masukan dibahas di Bab 12.

4.3 Merapikan Keluaran dengan Format

Keluaran yang rapi membuat program terasa profesional, terutama untuk tabel. Pascal menyediakan format lebar kolom:

Tabel 4.1 Penentu lebar dan desimal pada Write/WriteLn

Format	Arti	Contoh	Hasil
`nilai:N`	Tulis dalam N kolom, rata kanan	`42:6`	`[    42]`
`teks:N`	Teks juga rata kanan dalam N kolom	`'Ani':6`	`[   Ani]`
`real:N:D`	N kolom, D angka desimal	`3.14159:10:4`	`[    3.1416]`
`real:0:D`	Lebar secukupnya, D desimal	`3.14159:0:2`	`3.14`

Program 4.3 `bab04_03_format_keluaran.pas` *Format keluaran* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 4.3 — Format keluaran
  Nilai:N   menulis nilai rata kanan dalam N kolom.
  Nilai:N:D menulis bilangan Real dalam N kolom dengan D angka desimal.
  Dengan lebar yang sama di setiap baris, kolom tabel menjadi lurus. }

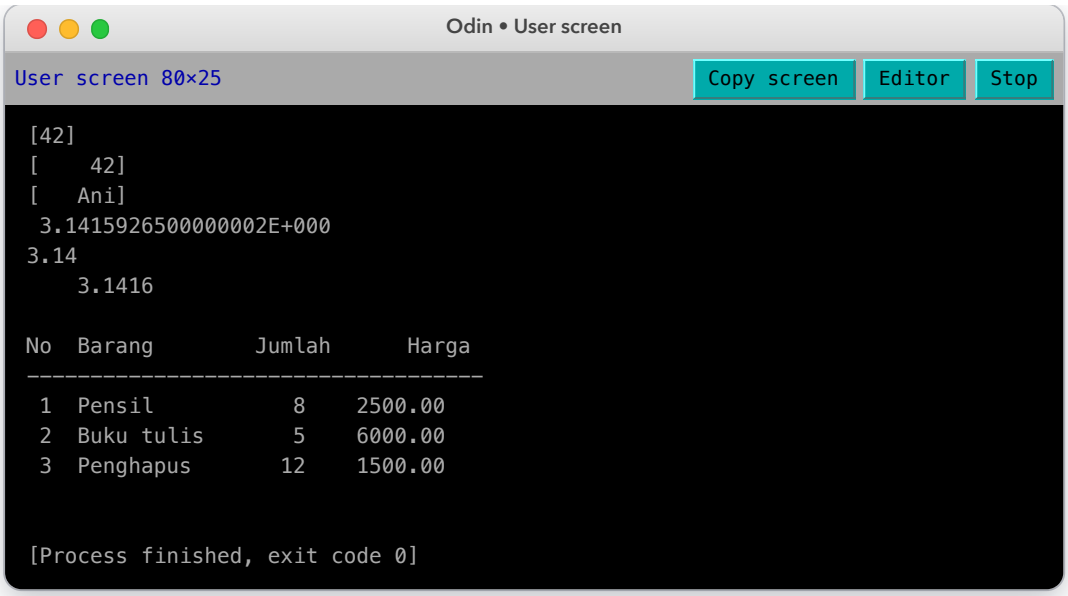
program bab04_03_format_keluaran;

var
  X: Real;

begin
  { Lebar kolom untuk bilangan bulat dan teks }
  WriteLn('[', 42, ']');          { tanpa format }
  WriteLn('[', 42:6, ']');        { 6 kolom, rata kanan }
  WriteLn('[', 'Ani':6, ']');     { teks juga rata kanan }

  { Bilangan Real tanpa format tampil dalam notasi ilmiah (E+000) }
  X := 3.14159265;
  WriteLn(X);
  WriteLn(X:0:2);                 { 2 desimal, lebar secukupnya }
  WriteLn(X:10:4);                { 10 kolom, 4 desimal }
  WriteLn;

  { Tabel: teks diberi spasi agar sama panjang, bilangan diberi lebar }
  WriteLn('No  Barang        Jumlah      Harga');
  WriteLn('-----------------------------------');
  WriteLn(1:2, '  ', 'Pensil    ', 8:8, 2500.0:11:2);
  WriteLn(2:2, '  ', 'Buku tulis', 5:8, 6000.0:11:2);
  WriteLn(3:2, '  ', 'Penghapus ', 12:8, 1500.0:11:2);
end.
```



Gambar 4.5 Keluaran `bab04_03_format_keluaran.pas` di User screen

Perhatikan baris `WriteLn(X)` tanpa format: bilangan `Real` tampil dalam notasi ilmiah yang sulit dibaca. Karena itu, **selalu beri format pada `Real`**. Dengan memberi lebar kolom yang sama di setiap baris, kolom tabel menjadi lurus rapi, seperti pada daftar barang di bagian akhir program.

> **TIPS**
>
> Bila lebar yang diminta lebih kecil dari panjang nilai, Pascal tidak memotongnya, melainkan memakai lebar secukupnya. Jadi `N` adalah lebar *minimum*, bukan maksimum.

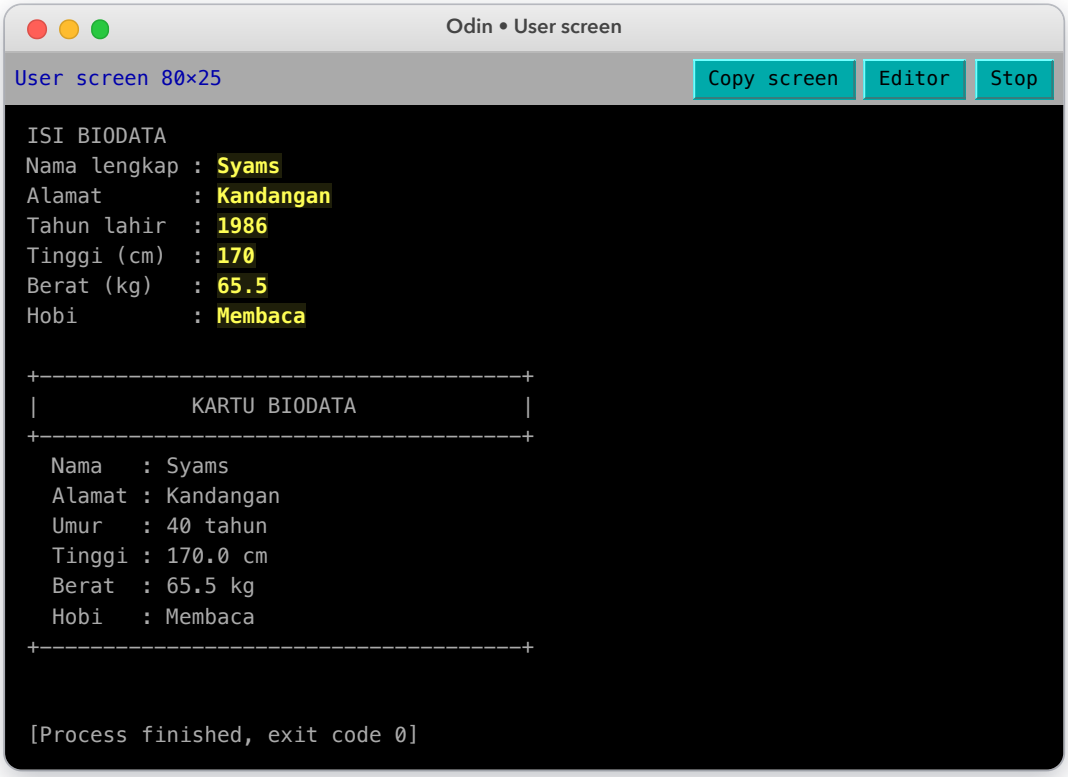
4.4 Menggabungkan: Kartu Biodata

Mari satukan semua: membaca beberapa masukan, menghitung sesuatu, lalu menampilkan hasil yang rapi. Program berikut membuat kartu biodata. Umur tidak ditanyakan, melainkan dihitung dari tahun lahir.

Program 4.4 `bab04_04_biodata_sederhana.pas` *Latihan: biodata sederhana* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 4.4 — Latihan: biodata sederhana
3    Menggabungkan masukan (ReadLn) dan keluaran berformat (WriteLn)
4    untuk mencetak kartu biodata. }
5
6  program bab04_04_biodata_sederhana;
7
8  var
9    Nama, Alamat, Hobi: string;
10   TahunLahir, Umur: Integer;
11   Tinggi, Berat: Real;
12
13 const
14   TahunIni = 2026;
15
16 begin
17   WriteLn('ISI BIODATA');
```

```
18    Write('Nama lengkap : '); ReadLn(Nama);
19    Write('Alamat       : '); ReadLn(Alamat);
20    Write('Tahun lahir  : '); ReadLn(TahunLahir);
21    Write('Tinggi (cm)  : '); ReadLn(Tinggi);
22    Write('Berat (kg)   : '); ReadLn(Berat);
23    Write('Hobi         : '); ReadLn(Hobi);
24
25    Umur := TahunIni - TahunLahir;   { umur dihitung, tidak ditanyakan }
26
27    WriteLn;
28    WriteLn('+--------------------------------------+');
29    WriteLn('|            KARTU BIODATA             |');
30    WriteLn('+--------------------------------------+');
31    WriteLn('  Nama   : ', Nama);
32    WriteLn('  Alamat : ', Alamat);
33    WriteLn('  Umur   : ', Umur, ' tahun');
34    WriteLn('  Tinggi : ', Tinggi:0:1, ' cm');
35    WriteLn('  Berat  : ', Berat:0:1, ' kg');
36    WriteLn('  Hobi   : ', Hobi);
37    WriteLn('+--------------------------------------+');
38  end.
```



Gambar 4.6 Keluaran `bab04_04_biodata_sederhana.pas` di User screen

Program ini memakai konstanta `TahunIni` agar perhitungan umur mudah diperbarui, dan menggambar bingkai kartu dengan karakter `+`, `-`, dan `|`. Perhatikan bagaimana beberapa `Write` dan `ReadLn` ditulis di satu baris dengan dipisah titik koma, agar pertanyaan dan jawaban tampak berpasangan.

4.5 Kesalahan yang Sering Terjadi

Lupa tanda petik penutup. String harus diapit dua petik tunggal.

petik.pas

```
1  program P;
2  begin
3    WriteLn('Halo, Dunia);
4  end.
```

PESAN KOMPILER

```
petik.pas(3,11) Error: Konstanta string melewati akhir baris
```

Gambar 4.7 Tanda petik penutup string terlupa

Menampilkan Real tanpa format. Ini bukan kesalahan kompilasi, tetapi hasilnya sulit dibaca. Bandingkan `WriteLn(3.14159)` yang memberi notasi ilmiah dengan `WriteLn(3.14159:0:2)` yang memberi `3.14`. Biasakan selalu memformat `Real`.

4.6 Rangkuman

- `WriteLn` menulis lalu pindah baris; `Write` tidak pindah baris. Keduanya menerima banyak nilai dipisah koma.
- Teks dalam petik ditulis apa adanya; ekspresi di luar petik dihitung dulu. `WriteLn;` mencetak baris kosong.
- `ReadLn(variabel)` membaca masukan sampai Enter. Beberapa nilai bisa dibaca sekaligus bila dipisah spasi.
- Pola interaktif: `Write` pertanyaan, lalu `ReadLn` jawaban.
- Format `:N` memberi lebar kolom, `:N:D` memberi lebar dan jumlah desimal. Selalu format setiap `Real`.
- Masukan yang tidak sesuai tipe bilangan memicu runtime error 106.

4.7 Latihan

1. **DASAR** Buat program yang menanyakan nama dan kota lahir, lalu menyapa: `Halo [nama] dari [kota]!`
2. **DASAR** Baca dua bilangan bulat dalam satu baris (dipisah spasi), lalu tampilkan jumlah dan selisihnya.
3. **MENENGAH** Buat tabel tiga barang dengan kolom Nama, Jumlah, dan Harga yang lurus rapi memakai format lebar kolom.
4. **MENENGAH** Minta jari-jari lingkaran, lalu tampilkan luasnya (`Pi * r * r`) dengan dua angka desimal.
5. **TANTANGAN** Buat kartu nilai berbingkai yang membaca tiga nilai ujian, lalu menampilkan ketiganya beserta rata-ratanya (dua desimal) dalam bingkai yang rapi.

BAB

BAGIAN I · FONDASI

05

Operator dan Ekspresi

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

TUJUAN PEMBELAJARAN

- Memakai operator aritmetika `+`, `-`, `*`, `/` dan fungsi bantu `Sqr`, `Sqrt`, `Abs`.
- Membedakan `div` (pembagian bulat) dan `mod` (sisa bagi), serta memanfaatkannya.
- Membentuk ekspresi perbandingan (`=`, `<>`, `<`, `>`, `<=`, `>=`) yang menghasilkan `Boolean`.
- Menggabungkan syarat dengan operator logika `and`, `or`, `not`, `xor`.
- Memahami prioritas operator dan memakai kurung agar ekspresi dihitung sesuai maksud.

PROGRAM DALAM BAB INI

5.1 `bab05_01_operator_aritmetika.pas`
Operator aritmetika

5.2 `bab05_02_div_mod.pas`
div dan mod

5.3 `bab05_03_operator_relasional.pas`
Operator relasional (perbandingan)

5.4 `bab05_04_operator_logika.pas`
Operator logika

5.5 `bab05_05_prioritas_operator.pas`
Prioritas operator

5.6 `bab05_06_luas_keliling.pas`
Latihan: luas dan keliling

Program mengolah data dengan **ekspresi**: gabungan nilai, variabel, dan operator yang menghasilkan sebuah nilai baru. Bab ini mengupas operator Pascal, dari hitungan biasa sampai logika benar/salah, lalu aturan prioritas yang menentukan urutan pengerjaannya. Menguasai ini adalah bekal untuk percabangan dan perulangan di bab berikutnya.

5.1 Operator Aritmetika

Empat operator dasar: `+` (tambah), `-` (kurang), `*` (kali), dan `/` (bagi). Perlu diingat dari Bab 3: `/` **selalu menghasilkan** `Real`, sedangkan `+`, `-`, `*` antara dua bilangan bulat tetap menghasilkan bilangan bulat. Pascal juga menyediakan fungsi bantu: `Sqr(x)` (kuadrat), `Sqrt(x)` (akar), dan `Abs(x)` (nilai mutlak).

Program 5.1 `bab05_01_operator_aritmetika.pas` *Operator aritmetika* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 5.1 — Operator aritmetika
  + tambah, - kurang, * kali, / bagi. Hasil / selalu Real, sedangkan
  +, -, * dari dua bilangan bulat tetap bilangan bulat. }

program bab05_01_operator_aritmetika;

var
  A, B: Integer;
  X, Y: Real;

begin
  A := 17;
  B := 5;
  WriteLn('A = ', A, ', B = ', B);
  WriteLn('A + B = ', A + B);
  WriteLn('A - B = ', A - B);
  WriteLn('A * B = ', A * B);
  WriteLn('A / B = ', A / B:0:2);      { hasil Real, perlu format }
  WriteLn('-A    = ', -A);              { minus di depan: lawan bilangan }
  WriteLn;

  X := 7.5;
  Y := 2.5;
  WriteLn('X = ', X:0:1, ', Y = ', Y:0:1);
  WriteLn('X + Y = ', X + Y:0:2);
  WriteLn('X * Y = ', X * Y:0:2);
  WriteLn('X / Y = ', X / Y:0:2);

  { Campuran Integer dan Real menghasilkan Real }
  WriteLn('A + X = ', A + X:0:2);

  { Fungsi bantu: Sqr (kuadrat), Sqrt (akar), Abs (nilai mutlak) }
  WriteLn('Sqr(B)   = ', Sqr(B));
  WriteLn('Sqrt(81) = ', Sqrt(81):0:1);
  WriteLn('Abs(-12) = ', Abs(-12));
end.
```



Gambar 5.1 Keluaran `bab05_01_operator_aritmetika.pas` di User screen

Perhatikan bahwa `A / B` perlu format `:0:2` karena hasilnya `Real`, sedangkan `A + B` tidak. Mencampur `Integer` dan `Real` dalam satu hitungan, seperti `A + X`, menghasilkan `Real`: tipe yang "lebih luas" menang.

5.2 Pembagian Bulat: div dan mod

Untuk bilangan bulat, Pascal punya dua operator khusus. `div` memberi **hasil bagi bulat** (pecahan dibuang), dan `mod` memberi **sisa pembagian**. Keduanya hanya untuk bilangan bulat.



Gambar 5.2 `17 = 3 × 5 + 2`: `div` memberi hasil bagi 3, `mod` memberi sisa 2.

Program 5.2 `bab05_02_div_mod.pas` *div dan mod* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 5.2 — div dan mod
3    div adalah pembagian bulat (pecahan dibuang), mod adalah sisa bagi.
4    Keduanya hanya untuk bilangan bulat. }
5
6  program bab05_02_div_mod;
7
8  var
9    TotalDetik: LongInt;
10   Jam, Menit, Detik: LongInt;
```

```
11    Angka: Integer;
12
13  begin
14    WriteLn('17 div 5 = ', 17 div 5);      { 17 = 3 x 5 + 2, hasil bagi 3 }
15    WriteLn('17 mod 5 = ', 17 mod 5);      { sisanya 2 }
16    WriteLn;
17
18    { Contoh: mengubah detik menjadi jam, menit, detik }
19    Write('Masukkan jumlah detik: ');
20    ReadLn(TotalDetik);
21    Jam := TotalDetik div 3600;            { 1 jam = 3600 detik }
22    Menit := (TotalDetik mod 3600) div 60;
23    Detik := TotalDetik mod 60;
24    WriteLn(TotalDetik, ' detik = ', Jam, ' jam ', Menit, ' menit ',
25      Detik, ' detik');
26    WriteLn;
27
28    { Contoh: bilangan genap habis dibagi 2, jadi sisanya 0 }
29    Write('Masukkan sebuah bilangan bulat: ');
30    ReadLn(Angka);
31    WriteLn('Sisa bagi 2 = ', Angka mod 2);
32    WriteLn('Angka satuan = ', Angka mod 10);   { digit paling kanan }
33    WriteLn('Tanpa satuan = ', Angka div 10);   { buang digit kanan }
34  end.
```



Gambar 5.3 Keluaran `bab05_02_div_mod.pas` di User screen

`div` dan `mod` sangat berguna. Contoh di program: mengubah total detik menjadi jam, menit, dan detik hanya dengan `div` dan `mod`. Trik lain yang sering dipakai: `Angka mod 10` memberi digit satuan (paling kanan), dan `Angka div 10` membuang digit itu. `Angka mod 2` bernilai 0 untuk bilangan genap, yang akan sering Anda pakai di bab-bab berikutnya.

5.3 Operator Perbandingan

Operator **relasional** membandingkan dua nilai dan menghasilkan `Boolean`. Inilah yang menjadi dasar semua keputusan.

Tabel 5.1 Enam operator perbandingan

Operator	Arti	Contoh	Hasil (A=10, B=7)
`=`	sama dengan	`A = B`	`FALSE`
`<>`	tidak sama dengan	`A <> B`	`TRUE`
`<`	kurang dari	`A < B`	`FALSE`
`>`	lebih dari	`A > B`	`TRUE`
`<=`	kurang dari atau sama	`A <= 10`	`TRUE`
`>=`	lebih dari atau sama	`A >= 10`	`TRUE`

Program 5.3 `bab05_03_operator_relasional.pas` *Operator relasional (perbandingan)* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 5.3 — Operator relasional (perbandingan)
  =  sama dengan          <>  tidak sama dengan
  <  kurang dari          <=  kurang dari atau sama dengan
  >  lebih dari           >=  lebih dari atau sama dengan
  Hasil perbandingan selalu Boolean: TRUE atau FALSE. }

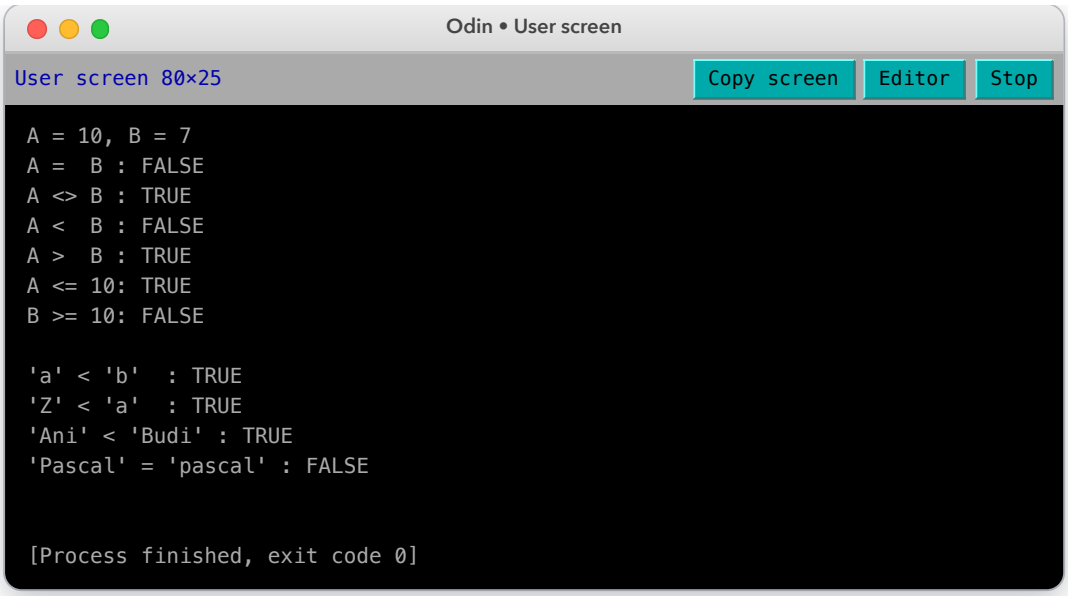
program bab05_03_operator_relasional;

var
  A, B: Integer;

begin
  A := 10;
  B := 7;
  WriteLn('A = ', A, ', B = ', B);
  WriteLn('A =  B : ', A = B);       { di Pascal, = berarti membandingkan }
  WriteLn('A <> B : ', A <> B);
  WriteLn('A <  B : ', A < B);
  WriteLn('A >  B : ', A > B);
  WriteLn('A <= 10: ', A <= 10);
  WriteLn('B >= 10: ', B >= 10);
  WriteLn;

  { Karakter dibandingkan menurut kode ASCII-nya }
  WriteLn('''a'' < ''b''  : ', 'a' < 'b');
  WriteLn('''Z'' < ''a''  : ', 'Z' < 'a');     { huruf besar lebih kecil }

  { String dibandingkan huruf demi huruf, seperti urutan kamus }
  WriteLn('''Ani'' < ''Budi'' : ', 'Ani' < 'Budi');
  WriteLn('''Pascal'' = ''pascal'' : ', 'Pascal' = 'pascal');
end.
```



Gambar 5.4 Keluaran `bab05_03_operator_relasional.pas` di User screen

> **AWAS**
>
> Di Pascal, `=` berarti **membandingkan**, bukan menugaskan. Penugasan memakai `:=`. Jangan sampai tertukar: `if A = 5` membandingkan, `A := 5` mengisi. Perbandingan juga berlaku untuk string: `'Ani' < 'Budi'` bernilai `TRUE` karena urutan abjad.

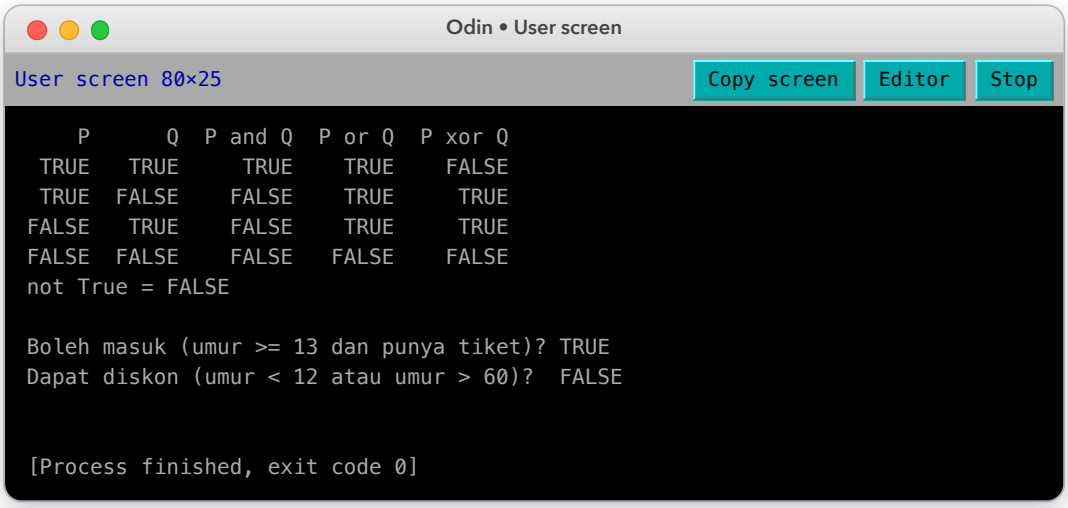
5.4 Operator Logika

Untuk menggabungkan beberapa syarat, Pascal punya operator logika. `and` benar bila **keduanya** benar; `or` benar bila **salah satu** benar; `not` membalik nilai; `xor` benar bila kedua nilai **berbeda**.

Program 5.4 `bab05_04_operator_logika.pas` *Operator logika* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 5.4 — Operator logika
3    and : benar bila keduanya benar     not : membalik nilai
4    or  : benar bila salah satu benar   xor : benar bila keduanya berbeda
5    Perbandingan yang digabung harus diberi kurung. }
6
7  program bab05_04_operator_logika;
8
9  var
10   P, Q: Boolean;
11   Umur: Integer;
12   PunyaTiket: Boolean;
13
14 begin
15   { Tabel kebenaran: ekspresi yang sama, isi P dan Q yang berganti }
16   WriteLn('    P      Q  P and Q  P or Q  P xor Q');
17   P := True;  Q := True;
18   WriteLn(P:5, Q:7, P and Q:9, P or Q:8, P xor Q:9);
19   P := True;  Q := False;
20   WriteLn(P:5, Q:7, P and Q:9, P or Q:8, P xor Q:9);
21   P := False; Q := True;
```

```
22    WriteLn(P:5, Q:7, P and Q:9, P or Q:8, P xor Q:9);
23    P := False; Q := False;
24    WriteLn(P:5, Q:7, P and Q:9, P or Q:8, P xor Q:9);
25    WriteLn('not True = ', not True);
26    WriteLn;
27
28    Umur := 15;
29    PunyaTiket := True;
30    { Kurung wajib: tanpa kurung, and dikerjakan lebih dulu dari >= }
31    WriteLn('Boleh masuk (umur >= 13 dan punya tiket)? ',
32      (Umur >= 13) and PunyaTiket);
33    WriteLn('Dapat diskon (umur < 12 atau umur > 60)?  ',
34      (Umur < 12) or (Umur > 60));
35  end.
```



Gambar 5.5 Keluaran `bab05_04_operator_logika.pas` di User screen

Keluaran baris pertama membentuk **tabel kebenaran**: ekspresi yang sama dihitung untuk setiap kemungkinan nilai `P` dan `Q`.

Tabel 5.2 Tabel kebenaran operator logika

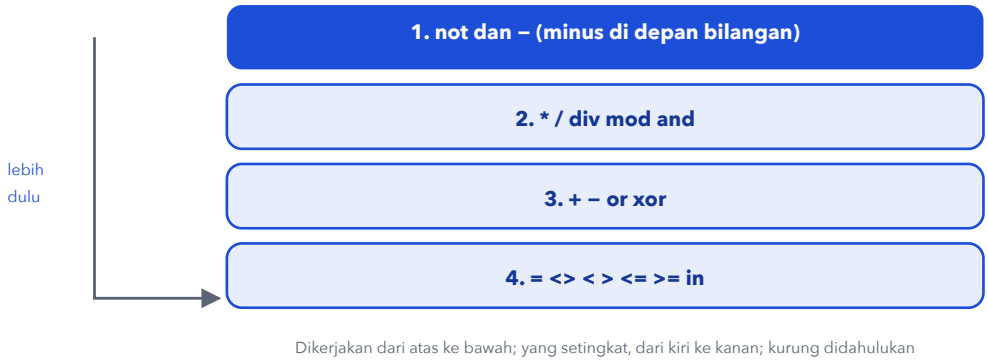
P	Q	P and Q	P or Q	P xor Q
True	True	True	True	False
True	False	False	True	True
False	True	False	True	True
False	False	False	False	False

AWAS

Saat menggabungkan perbandingan dengan `and` / `or`, **setiap perbandingan wajib diberi kurung**: tulis `(Umur >= 13) and PunyaTiket`. Tanpa kurung, `Umur >= 13 and PunyaTiket` dibaca keliru karena `and` dikerjakan lebih dulu daripada `>=`, dan kompiler menolaknya. Aturan prioritas inilah yang kita bahas berikutnya.

5.5 Prioritas Operator

Bila sebuah ekspresi memuat banyak operator, Pascal mengerjakannya dalam urutan tertentu, sama seperti "kali dan bagi sebelum tambah dan kurang" di matematika. Operator setingkat dikerjakan dari kiri ke kanan, dan **kurung selalu didahulukan**.



Gambar 5.6 Empat tingkat prioritas operator, dari yang dikerjakan paling awal.

Program 5.5 `bab05_05_prioritas_operator.pas` *Prioritas operator* File ▸ Examples…

```
{ Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 5.5 – Prioritas operator
  Urutan pengerjaan, dari yang paling dulu:
    1. not  dan  minus di depan bilangan
    2. *  /  div  mod  and
    3. +  –  or  xor
    4. =  <>  <  >  <=  >=  in
  Operator setingkat dikerjakan dari kiri ke kanan. Kurung didahulukan. }

program bab05_05_prioritas_operator;

var
  X: Integer;

begin
  WriteLn('2 + 3 * 4       = ', 2 + 3 * 4);       { kali dulu: 14 }
  WriteLn('(2 + 3) * 4     = ', (2 + 3) * 4);     { kurung dulu: 20 }
  WriteLn('20 - 6 - 4      = ', 20 - 6 - 4);      { kiri ke kanan: 10 }
  WriteLn('20 div 4 * 2    = ', 20 div 4 * 2);    { (20 div 4) * 2 = 10 }
  WriteLn('20 div (4 * 2)  = ', 20 div (4 * 2));  { 20 div 8 = 2 }
  WriteLn('-2 * 3 + 10     = ', -2 * 3 + 10);     { minus depan dulu: 4 }
  WriteLn('10 / 4 * 2      = ', 10 / 4 * 2:0:2);  { (10 / 4) * 2 = 5 }
  WriteLn;

  X := 5;
  { and lebih tinggi dari >, jadi perbandingan WAJIB diberi kurung.
    Tanpa kurung, X > 0 and X < 10 dibaca X > (0 and X) < 10 dan
    kompiler menolaknya. }
  WriteLn('(X > 0) and (X < 10)  = ', (X > 0) and (X < 10));
  WriteLn('not (X = 5)           = ', not (X = 5));
end.
```



Gambar 5.7 Keluaran `bab05_05_prioritas_operator.pas` di User screen

Pelajari tiap baris: `2 + 3 * 4` memberi 14 karena perkalian dulu, sedangkan `(2 + 3) * 4` memberi 20 karena kurung didahulukan. `20 div 4 * 2` dibaca kiri ke kanan menjadi `(20 div 4) * 2 = 10`. Inti pelajaran: **bila ragu, beri kurung**. Kurung membuat maksud Anda jelas, baik bagi kompiler maupun bagi pembaca.

5.6 Menggabungkan Semua: Luas dan Keliling

Program penutup bab ini menghitung luas dan keliling persegi panjang serta lingkaran. Ia memakai operator aritmetika, kurung untuk mengatur urutan, fungsi bawaan `Sqr` dan `Pi`, serta masukan dan keluaran berformat.

Program 5.6 `bab05_06_luas_keliling.pas` *Latihan: luas dan keliling* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 5.6 — Latihan: luas dan keliling
3    Menghitung luas dan keliling persegi panjang serta lingkaran dari
4    ukuran yang dimasukkan pengguna. Pi adalah fungsi bawaan Pascal. }
5
6  program bab05_06_luas_keliling;
7
8  var
9    Panjang, Lebar: Real;
10   JariJari: Real;
11   Luas, Keliling: Real;
12
13 begin
14   WriteLn('PERSEGI PANJANG');
15   Write('Panjang (cm): '); ReadLn(Panjang);
16   Write('Lebar   (cm): '); ReadLn(Lebar);
17   Luas := Panjang * Lebar;
18   Keliling := 2 * (Panjang + Lebar);   { kurung: jumlahkan dulu }
19   WriteLn('Luas     = ', Luas:0:2, ' cm2');
20   WriteLn('Keliling = ', Keliling:0:2, ' cm');
21   WriteLn;
22
```

```
23    WriteLn('LINGKARAN');
24    Write('Jari-jari (cm): '); ReadLn(JariJari);
25    Luas := Pi * Sqr(JariJari);          { Pi x r x r }
26    Keliling := 2 * Pi * JariJari;       { 2 x Pi x r }
27    WriteLn('Luas     = ', Luas:0:2, ' cm2');
28    WriteLn('Keliling = ', Keliling:0:2, ' cm');
29  end.
```

Odin • User screen

User screen 80×25 | Copy screen | Editor | Stop

```
PERSEGI PANJANG
Panjang (cm): 8
Lebar   (cm): 5
Luas     = 40.00 cm2
Keliling = 26.00 cm

LINGKARAN
Jari-jari (cm): 7
Luas     = 153.94 cm2
Keliling = 43.98 cm


[Process finished, exit code 0]
```

Gambar 5.8 Keluaran `bab05_06_luas_keliling.pas` di User screen

Perhatikan pemakaian kurung pada `2 * (Panjang + Lebar)`: tanpa kurung, perkalian akan dikerjakan lebih dulu dan hasilnya salah. `Pi` adalah fungsi bawaan yang memberi nilai π, dan `Sqr(JariJari)` menghitung kuadrat jari-jari.

5.7 Kesalahan yang Sering Terjadi

Lupa kurung pada perbandingan yang digabung. Karena `and` berprioritas lebih tinggi dari `>`, menggabungkan tanpa kurung membuat kompiler salah membaca ekspresi.

logika.pas

```
1  program Logika;
2  var
3    X: Integer;
4  begin
5    X := 5;
6    if X > 0 and X < 10 then
7      WriteLn('Antara 0 dan 10');
8  end.
```

PESAN KOMPILER

```
logika.pas(6,6) Error: Tipe operand tidak cocok dengan operator
```

Gambar 5.9 Perbandingan yang digabung and tanpa kurung ditolak kompiler

Perbaikannya: beri kurung pada setiap perbandingan, `if (X > 0) and (X < 10) then`.

5.8 Rangkuman

- Operator aritmetika: `+ - * /`; `/` selalu menghasilkan `Real`. Fungsi bantu: `Sqr`, `Sqrt`, `Abs`, `Pi`.
- `div` memberi hasil bagi bulat, `mod` memberi sisa. `mod 2` menguji genap/ganjil; `mod 10` dan `div 10` mengambil/membuang digit satuan.
- Operator perbandingan menghasilkan `Boolean`. `=` membandingkan, `:=` menugaskan; jangan tertukar.
- Operator logika: `and` (keduanya benar), `or` (salah satu benar), `not` (membalik), `xor` (berbeda).
- Prioritas: `not`/minus, lalu `* / div mod and`, lalu `+ - or xor`, lalu perbandingan. Kurung didahulukan.
- Perbandingan yang digabung `and` / `or` wajib diberi kurung masing-masing.

5.9 Latihan

1. **DASAR** Baca sebuah bilangan bulat, lalu tampilkan apakah genap atau ganjil memakai `mod 2` dan sebuah perbandingan (hasil `Boolean`).
2. **DASAR** Baca jumlah menit, lalu ubah menjadi jam dan menit memakai `div 60` dan `mod 60`.
3. **MENENGAH** Hitung dan tampilkan nilai dari `10 + 20 * 3`, `(10 + 20) * 3`, dan `100 div 7 mod 5`. Jelaskan urutan pengerjaan masing-masing.
4. **MENENGAH** Baca umur dan status (punya KTP, `Boolean`), lalu tampilkan apakah "boleh memilih" (umur ≥ 17 `and` punya KTP). Beri kurung dengan benar.
5. **TANTANGAN** Baca sebuah bilangan tiga digit, lalu tampilkan ketiga digitnya satu per satu memakai `div` dan `mod`.

BAGIAN II

Alur Kendali

Membuat program yang mengambil keputusan dan mengulang perintah, lalu memakainya untuk algoritma klasik.

06 Percabangan

07 Perulangan

08 Berlatih Algoritma dengan Perulangan

BAB 06

BAGIAN II · ALUR KENDALI

Percabangan

1 2 3 4 5 **6** 7 8 9 10 11 12 13 14 15 16 17 18 19 20

TUJUAN PEMBELAJARAN

- Memahami syarat (kondisi) sebagai ekspresi Boolean yang bernilai `True` atau `False`.
- Memakai `if..then` untuk menjalankan perintah hanya bila syaratnya terpenuhi.
- Memakai `if..then..else` untuk memilih satu dari dua jalan, dan memahami mengapa tidak boleh ada titik koma sebelum `else`.
- Menyusun `if` bersarang dan rantai `else if`, serta mengenali jebakan *dangling else*.
- Memakai `case..of` dengan daftar nilai, rentang, dan cabang `else`.
- Menerapkan percabangan pada masalah nyata: nilai huruf dan tahun kabisat.

PROGRAM DALAM BAB INI

6.1	`bab06_01_if_then.pas`	if..then
6.2	`bab06_02_if_else.pas`	if..then..else
6.3	`bab06_03_if_bersarang.pas`	if bersarang
6.4	`bab06_04_case_of.pas`	case..of
6.5	`bab06_05_nilai_huruf.pas`	Latihan: nilai angka menjadi nilai huruf
6.6	`bab06_06_tahun_kabisat.pas`	Latihan: tahun kabisat

Semua program di bab-bab sebelumnya berjalan lurus: perintah pertama, kedua, ketiga, sampai `end.`, tanpa pernah melompati satu baris pun. Program seperti itu tidak bisa bereaksi terhadap keadaan. Di bab ini Anda akan mengajari program mengambil keputusan: menjalankan sebagian perintah hanya bila suatu syarat terpenuhi, memilih satu dari dua jalan, atau memilih satu dari banyak kemungkinan. Kemampuan inilah yang membuat program terasa "pintar".

6.1 Program yang Bisa Memilih

Setiap hari Anda mengambil keputusan dengan pola yang sama: *kalau* hujan, *maka* bawa payung. *Kalau* nilai ujian 75 atau lebih, *maka* lulus, *kalau tidak* ikut remedial. Pola "kalau ... maka ... kalau tidak ..." ini disebut **percabangan** (*branching* atau *selection*), karena jalannya program bercabang seperti jalan di persimpangan. Program memeriksa sebuah **syarat**, lalu memilih cabang mana yang ditempuh.

Di Pascal, syarat adalah **ekspresi Boolean**: ekspresi yang hasilnya hanya salah satu dari dua nilai, `True` (benar) atau `False` (salah). Anda sudah mengenalnya di Bab 5. Syarat biasanya dibentuk dari operator pembanding (`=`, `<>`, `<`, `>`, `<=`, `>=`) dan digabung dengan operator logika `and`, `or`, serta `not`. Tabel berikut memberi beberapa contoh syarat yang akan Anda temui di bab ini.

Tabel 6.1 Contoh syarat dan nilainya untuk data tertentu

Syarat	Artinya	Data	Hasil
`Suhu >= 38`	suhu 38 atau lebih	`Suhu` = 37	`False`
`Bilangan mod 2 = 0`	sisa bagi 2 adalah nol (genap)	`Bilangan` = 14	`True`
`(Umur >= 13) and (Umur <= 22)`	umur di antara 13 dan 22	`Umur` = 25	`False`
`(Huruf = 'Y') or (Huruf = 'y')`	huruf Y besar atau kecil	`Huruf` = `'y'`	`True`
`not Kabisat`	kebalikan nilai `Kabisat`	`Kabisat` = `True`	`False`

Pascal menyediakan dua perintah percabangan. Perintah `if` memeriksa sebuah syarat Boolean dan cocok untuk keputusan "ya atau tidak". Perintah `case` memeriksa nilai sebuah variabel dan cocok untuk memilih satu dari banyak kemungkinan, misalnya nomor menu atau nama hari. Keduanya akan dibahas satu per satu.

1 if..then
Satu cabang. Perintah dijalankan bila syarat `True`, dilewati bila `False`.

2 if..then..else
Dua cabang. Tepat satu dari keduanya dijalankan, tergantung hasil syarat.

n case..of
Banyak cabang. Nilai sebuah variabel ordinal dicocokkan dengan label setiap cabang.

6.2 Percabangan Satu Arah: if..then

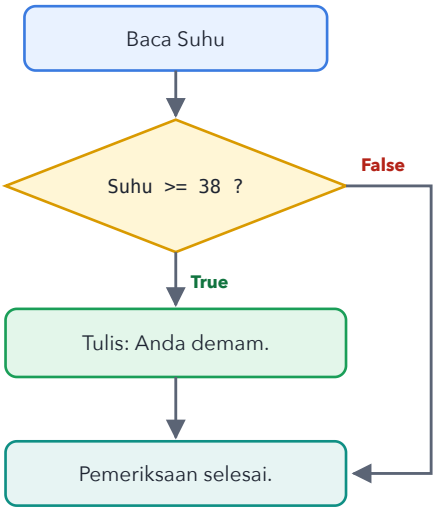
Bentuk paling sederhana adalah `if..then`. Bila syarat bernilai `True`, pernyataan sesudah `then` dijalankan. Bila `False`, pernyataan itu dilewati begitu saja, dan program langsung melanjutkan ke pernyataan berikutnya. Tidak ada yang dikerjakan sebagai gantinya.

BENTUK UMUM

```
if syarat then
  pernyataan;

if syarat then
begin
  pernyataan1;
  pernyataan2;
end;
```

Sesudah `then` hanya boleh ada **satu** pernyataan. Bila Anda ingin menjalankan beberapa perintah sekaligus, bungkus semuanya dengan `begin` dan `end`. Gabungan ini disebut **pernyataan majemuk** (*compound statement*): bagi kompiler, seluruh isi `begin..end` dihitung sebagai satu pernyataan. Perhatikan cara kerja `if..then` pada diagram alir berikut. Belah ketupat kuning adalah titik keputusan; dari sana ada dua jalan keluar, `True` dan `False`.




```
ReadLn(Suhu);
if Suhu >= 38 then
WriteLn('Anda demam. Istirahatlah.');
WriteLn('Pemeriksaan selesai.');
```

Syarat: ekspresi Boolean, hasilnya True atau False

Dijalankan hanya bila syarat bernilai True

Selalu dijalankan, apa pun hasil syaratnya

Garis warna di kiri kode menunjukkan kotak diagram yang sesuai.

Gambar 6.1 Diagram alir `if..then`: bila syarat `False`, perintah di cabang `True` dilewati.

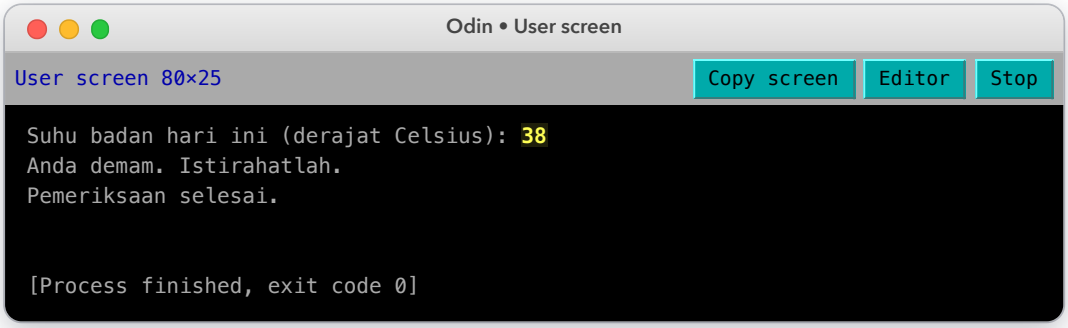
Program contoh pertama memeriksa suhu badan. Buka `bab06_01_if_then.pas` lewat `File ▸ Examples...`, lalu jalankan dengan Ctrl + F9. Program ini memiliki dua `if`: yang pertama dengan satu pernyataan, yang kedua dengan pernyataan majemuk.

Program 6.1 `bab06_01_if_then.pas` *if..then* — File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 6.1 — if..then
3    Perintah sesudah then hanya dijalankan bila syaratnya bernilai True.
4    Bila syaratnya False, perintah itu dilewati. }
5
6  program bab06_01_if_then;
7
8  var
9    Suhu: Integer;
10
11 begin
```

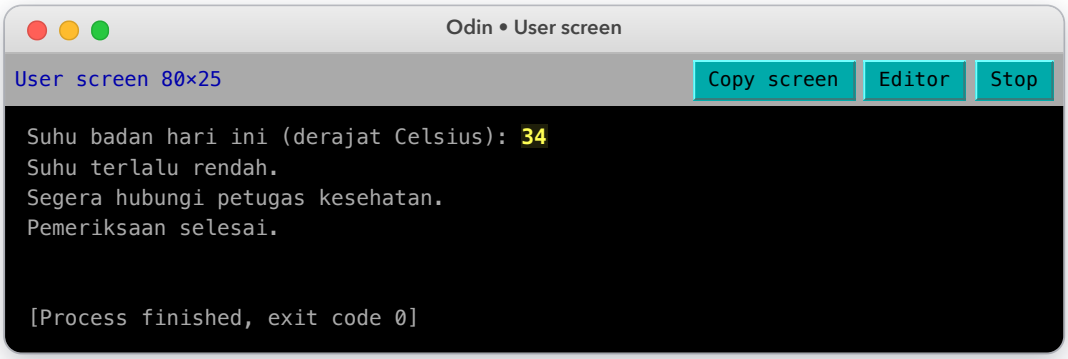
```
12    Write('Suhu badan hari ini (derajat Celsius): ');
13    ReadLn(Suhu);
14
15    if Suhu >= 38 then                    { syarat: suhu 38 atau lebih }
16      WriteLn('Anda demam. Istirahatlah.');
17
18    { Lebih dari satu perintah? Bungkus dengan begin..end }
19    if Suhu < 35 then
20    begin
21      WriteLn('Suhu terlalu rendah.');
22      WriteLn('Segera hubungi petugas kesehatan.');
23    end;
24
25    WriteLn('Pemeriksaan selesai.');       { selalu dijalankan }
26  end.
```


Bila Anda mengetik `38`, syarat pada baris 15 bernilai `True` sehingga pesan demam tampil. Syarat pada baris 19 bernilai `False`, jadi blok baris 20–23 dilewati. Baris 25 selalu dijalankan.



Gambar 6.2 Suhu 38: cabang `if` pertama dijalankan

Jalankan sekali lagi dan ketik `34`. Sekarang keadaannya terbalik: syarat pertama `False`, syarat kedua `True`, sehingga kedua `WriteLn` di dalam `begin..end` dijalankan.



Gambar 6.3 Suhu 34: blok `begin..end` pada baris 20–23 dijalankan

Baris	Penjelasan
9	Variabel `Suhu` bertipe `Integer`, jadi suhu diketik sebagai bilangan bulat.
12–13	Program meminta masukan dan menyimpannya di `Suhu`. Kursor menunggu di User screen sampai Anda menekan **Enter**.

Baris	Penjelasan
15–16	Satu pernyataan `if` utuh. Syarat `Suhu >= 38` diperiksa; bila `True`, `WriteLn` pada baris 16 dijalankan. Titik koma di akhir baris 16 mengakhiri seluruh pernyataan `if`, bukan hanya `WriteLn`.
19–23	`if` kedua dengan pernyataan majemuk. Kedua `WriteLn` di dalam `begin..end` dijalankan bersama-sama, atau dilewati bersama-sama.
25	Pernyataan di luar kedua `if`. Baris ini selalu dijalankan, berapa pun suhunya.

Bagaimana bila suhunya 36? Kedua syarat bernilai `False`, sehingga hanya baris 25 yang dijalankan. Tabel berikut merangkum baris mana yang bekerja untuk setiap kemungkinan masukan. Membuat tabel seperti ini adalah cara yang baik untuk menguji program percabangan: pilih satu masukan untuk **setiap** cabang, lalu periksa hasilnya satu per satu.

Tabel 6.2 Baris yang dijalankan Program 6.1 untuk tiga kelompok masukan

Masukan	Suhu >= 38	Suhu < 35	Baris yang dijalankan
38 atau lebih	`True`	`False`	16, 25
35 sampai 37	`False`	`False`	25 saja
kurang dari 35	`False`	`True`	21, 22, 25

TIPS

Biasakan menulis isi cabang menjorok dua spasi ke kanan, seperti di semua contoh buku ini. Kompiler tidak peduli dengan spasi, tetapi mata Anda akan langsung melihat perintah mana yang berada di dalam `if` dan mana yang tidak.

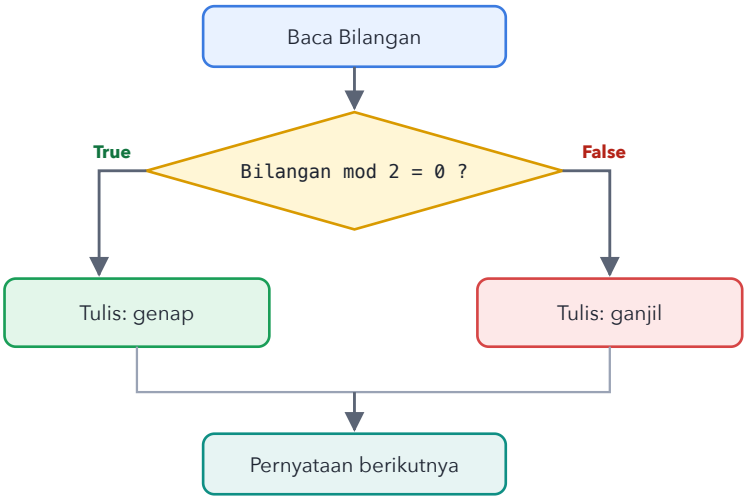
6.3 Dua Pilihan: if..then..else

Sering kali kita tidak hanya ingin melewati perintah, tetapi melakukan hal lain sebagai gantinya: bila genap tulis "genap", bila tidak tulis "ganjil". Untuk itu `if` dilengkapi bagian `else` (artinya "kalau tidak"). Dengan `if..then..else`, **tepat satu** dari dua cabang selalu dijalankan: cabang `then` bila syarat `True`, cabang `else` bila `False`. Tidak mungkin keduanya dijalankan, dan tidak mungkin keduanya dilewati.

BENTUK UMUM

```
if syarat then
  pernyataan_bila_True
else
  pernyataan_bila_False;
```

Perhatikan baik-baik: **tidak ada titik koma sebelum `else`**. Titik koma di Pascal adalah pemisah antar pernyataan. Seluruh `if..then..else` adalah **satu** pernyataan, jadi titik koma baru boleh ditulis setelah cabang `else` selesai. Bila Anda menulis titik koma sebelum `else`, kompiler mengira pernyataan `if` sudah berakhir, lalu bingung menemukan `else` yang tidak punya pasangan.




```
if Bilangan mod 2 = 0 then
WriteLn(.., ' genap')
else
WriteLn(.., ' ganjil');
...
```

Tidak ada ; sesudah baris WriteLn genap

Titik koma pertama baru muncul sesudah cabang else, karena if..then..else adalah satu pernyataan utuh.

Gambar 6.4 Diagram alir `if..then..else`: dua cabang yang bertemu kembali. Tepat satu cabang dijalankan.

Program 6.2 memakai `if..then..else` dua kali. Yang pertama menentukan genap atau ganjil dengan operator `mod` (sisa bagi). Yang kedua memutuskan kelulusan, dan setiap cabangnya berisi pernyataan majemuk.

Program 6.2 `bab06_02_if_else.pas` *if..then..else* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 6.2 — if..then..else
  else berisi perintah untuk keadaan sebaliknya. Ingat: TIDAK ADA
  titik koma tepat sebelum else. }

program bab06_02_if_else;

var
  Bilangan: Integer;
  Nilai: Integer;

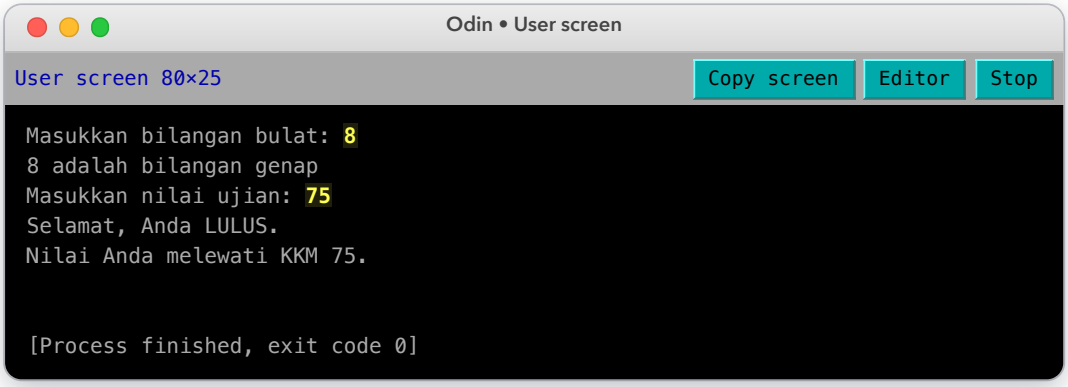
begin
  Write('Masukkan bilangan bulat: ');
  ReadLn(Bilangan);
  if Bilangan mod 2 = 0 then
    WriteLn(Bilangan, ' adalah bilangan genap')   { tanpa titik koma }
  else
    WriteLn(Bilangan, ' adalah bilangan ganjil');

  Write('Masukkan nilai ujian: ');
  ReadLn(Nilai);
  if Nilai >= 75 then
  begin
    WriteLn('Selamat, Anda LULUS.');
    WriteLn('Nilai Anda melewati KKM 75.');
  end                                  { end tanpa titik koma juga }
  else
  begin
    WriteLn('Anda belum lulus.');
    WriteLn('Kurang ', 75 - Nilai, ' poin lagi.');
  end;
end.
```



Gambar 6.5 Masukan 7 dan 60: kedua `if` menempuh cabang `else`

Bilangan 7 dibagi 2 bersisa 1, jadi syarat baris 15 `False` dan cabang `else` pada baris 18 yang dijalankan. Nilai 60 kurang dari 75, jadi blok `else` pada baris 28–31 dijalankan dan baris 30 menghitung kekurangannya, `75 - 60`. Sekarang coba masukan yang menempuh cabang lainnya: bilangan genap dan nilai tepat 75.



Gambar 6.6 Masukan 8 dan 75: kedua `if` menempuh cabang `then`

Baris	Penjelasan
15	`Bilangan mod 2` menghasilkan sisa pembagian dengan 2. Sisa 0 berarti genap.
16	Cabang `then`. Baris ini **tidak** diakhiri titik koma karena pernyataan `if` belum selesai; masih ada `else` di baris berikutnya.
17–18	Cabang `else`. Titik koma di akhir baris 18 menutup seluruh `if..then..else`.
22	Syarat `Nilai >= 75`. Nilai tepat 75 ikut lulus karena operatornya `>=`, bukan `>`.
23–26	Cabang `then` berupa pernyataan majemuk. Kata `end` pada baris 26 juga tanpa titik koma, karena masih diikuti `else`.
27–31	Cabang `else` berupa pernyataan majemuk. Baris 30 menulis teks dan hasil hitungan `75 - Nilai` dalam satu `WriteLn`.

COBA SENDIRI

Jalankan Program 6.2 dan ketik bilangan negatif, misalnya `-7` dan `-8`. Apakah genap dan ganjilnya tetap benar? Lalu ubah `>=` pada baris 22 menjadi `>` dan ketik nilai 75. Apa yang berubah, dan mengapa?

6.4 if Bersarang dan Rantai else if

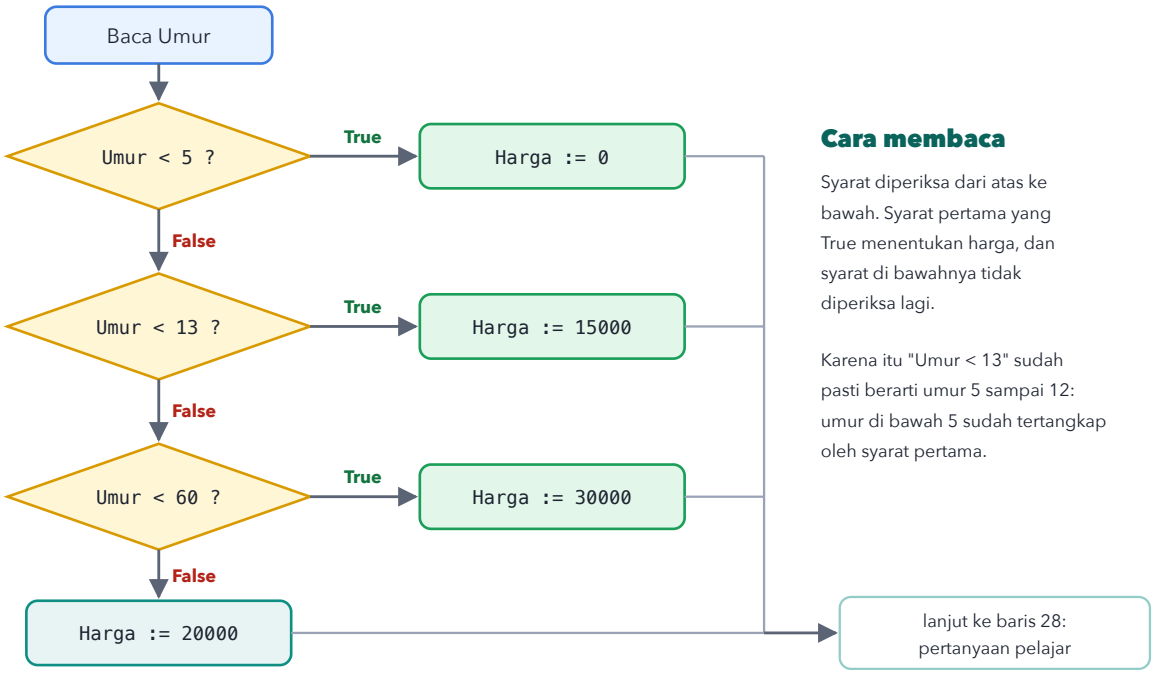
Cabang sebuah `if` boleh berisi pernyataan apa saja, termasuk `if` lain. `if` di dalam `if` ini disebut **if bersarang** (*nested if*). Ada dua pola yang sangat sering dipakai:

- **Rantai `else if`**: `if` baru ditaruh di cabang `else`. Hasilnya adalah deretan syarat yang diperiksa dari atas ke bawah, dan berhenti di syarat pertama yang `True`. Pola ini dipakai untuk memilih satu dari beberapa kelompok, misalnya kelompok umur.
- **`if` di dalam `then`**: pertanyaan kedua hanya diajukan bila jawaban pertama `True`, misalnya "apakah pelajar?" hanya ditanyakan kepada pengunjung berumur 13 sampai 22 tahun.

BENTUK UMUM

```
if syarat1 then
  pernyataan1
else if syarat2 then
  pernyataan2
else if syarat3 then
  pernyataan3
else
  pernyataan_bila_semua_False;
```

Sebenarnya Pascal tidak punya kata `else if`. Yang terjadi adalah cabang `else` berisi sebuah pernyataan `if` baru, yang cabang `else`-nya berisi `if` baru lagi, dan seterusnya. Karena penulisannya menjadi rapi seperti daftar, programmer menyebutnya rantai `else if`. Diagram berikut memperlihatkan rantai dari Program 6.3, yang menentukan harga tiket berdasarkan umur.



Gambar 6.7 Rantai `else if` pada Program 6.3, baris 19–26. Hanya satu dari empat penugasan `Harga` yang dijalankan.

Program 6.3 `bab06_03_if_bersarang.pas` *if bersarang* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 6.3 — if bersarang
3    Sebuah if boleh berada di dalam if lain. Pola else if dipakai untuk
4    memilih satu dari beberapa kemungkinan secara berurutan. }
5
6  program bab06_03_if_bersarang;
7
8  var
9    Umur: Integer;
10   Pelajar: Char;
11   Harga: LongInt;
12
13 begin
14   Write('Umur pengunjung: ');
15   ReadLn(Umur);
16
17   { Rantai else if: diperiksa dari atas, berhenti di syarat pertama
18     yang benar }
19   if Umur < 5 then
20     Harga := 0
21   else if Umur < 13 then
22     Harga := 15000
23   else if Umur < 60 then
24     Harga := 30000
25   else
26     Harga := 20000;
27
28   { if di dalam if: pertanyaan kedua hanya untuk umur 13 sampai 22 }
29   if (Umur >= 13) and (Umur <= 22) then
30   begin
31     Write('Apakah pelajar/mahasiswa (Y/T)? ');
32     ReadLn(Pelajar);
33     if (Pelajar = 'Y') or (Pelajar = 'y') then
34       Harga := Harga - 10000;          { potongan pelajar }
35   end;
36
37   if Harga = 0 then
38     WriteLn('Masuk gratis.')
39   else
40     WriteLn('Harga tiket: Rp ', Harga);
41 end.
```



Gambar 6.8 Umur 17 dan pelajar: harga dewasa dikurangi potongan pelajar

Umur 17 tidak memenuhi `Umur < 5` maupun `Umur < 13`, tetapi memenuhi `Umur < 60`, sehingga `Harga` menjadi 30000. Karena 17 berada di antara 13 dan 22, program bertanya apakah pengunjung pelajar. Jawaban `y` memenuhi syarat baris 33, sehingga harga dipotong 10000. Sekarang coba pengunjung balita:



Gambar 6.9 Umur 3: syarat pertama langsung `True`, dan pertanyaan pelajar tidak diajukan

Baris	Penjelasan
11	`Harga` bertipe `LongInt`, cukup untuk angka rupiah yang besar (batas `Integer` hanya 32767).
19–26	Rantai `else if` dengan empat kemungkinan. Hanya baris 26 yang diakhiri titik koma, karena seluruh rantai adalah satu pernyataan `if`.
29	Syarat gabungan: kedua pembanding wajib diberi tanda kurung, lalu digabung dengan `and`.
30–35	Cabang `then` berisi pernyataan majemuk, termasuk `if` lain pada baris 33. Inilah `if` di dalam `if`: pertanyaan pelajar hanya muncul untuk umur 13–22.
33	Jawaban `Y` atau `y` sama-sama diterima berkat operator `or`.
34	Penugasan `Harga := Harga - 10000` memakai nilai lama `Harga` untuk menghitung nilai barunya.
37–40	`if..else` terakhir memilih cara menampilkan hasil: kalimat khusus untuk harga nol, atau angka harganya.

AWAS

Urutan syarat dalam rantai `else if` sangat menentukan. Bila baris 19 dan 23 ditukar, sehingga `Umur < 60` diperiksa lebih dulu, balita berumur 3 tahun ikut membayar 30000, karena 3 juga kurang dari 60 dan syarat itu yang pertama bernilai `True`. Susunlah syarat dari kelompok yang paling sempit, atau dari batas terkecil ke terbesar.

6.4.1 Jebakan else yang Menggantung

Ketika `if` bersarang tanpa `begin..end`, muncul pertanyaan: sebuah `else` milik `if` yang mana? Aturan Pascal tegas: **`else` selalu menjadi pasangan `if` terdekat di atasnya yang belum punya `else`**. Indentasi tidak berpengaruh sama sekali. Masalah ini terkenal dengan nama *dangling else* (else yang menggantung). Program berikut ditulis dengan indentasi yang menipu: penulisnya ingin `else` menjadi milik `if Umur >= 13`.

gantung.pas

```
1  program Gantung;
2  var
3    Umur: Integer;
```

```
4    Pelajar: Char;
5  begin
6    Umur := 17;
7    Pelajar := 'T';
8    if Umur >= 13 then
9      if Pelajar = 'Y' then
10       WriteLn('Dapat potongan pelajar')
11   else
12     WriteLn('Anak-anak tidak perlu kartu pelajar');
13   WriteLn('Selesai.');
14 end.
```



Gambar 6.10 Umur 17, tetapi yang tampil adalah pesan untuk anak-anak: `else` menjadi milik `if` kedua

Program ini lolos kompilasi, tetapi hasilnya salah. Kompiler membaca baris 11 sebagai `else` milik `if Pelajar = 'Y'` (baris 9), karena itulah `if` terdekat yang belum punya `else`. Akibatnya, remaja yang bukan pelajar disebut anak-anak. Cara memperbaikinya adalah membungkus `if` bagian dalam dengan `begin..end`, sehingga `if` itu tertutup sebelum `else` dibaca.

✗ Indentasi menipu

```
if Umur >= 13 then
  if Pelajar = 'Y' then
    WriteLn('Potongan pelajar')
else
  WriteLn('Anak-anak');
```

`else` menempel pada `if Pelajar = 'Y'`.

✓ Batas yang jelas

```
if Umur >= 13 then
begin
  if Pelajar = 'Y' then
    WriteLn('Potongan pelajar');
end
else
  WriteLn('Anak-anak');
```

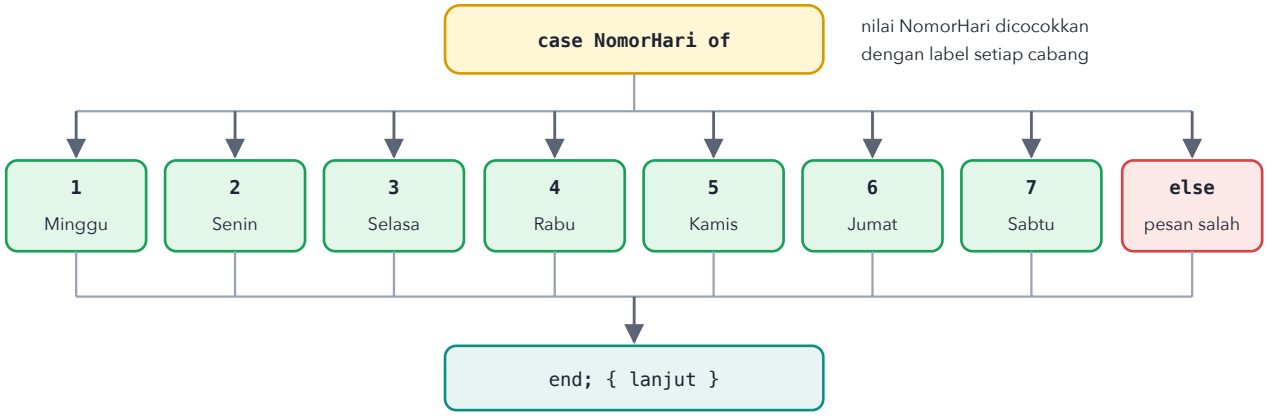
`end` menutup `if` dalam; `else` milik `if Umur >= 13`.

6.5 Banyak Pilihan: case..of

Bayangkan program yang mengubah nomor hari 1 sampai 7 menjadi nama hari. Dengan `if`, Anda butuh rantai tujuh syarat `NomorHari = 1`, `NomorHari = 2`, dan seterusnya. Pascal menyediakan cara yang lebih ringkas: `case..of`. Nilai sebuah ekspresi, yang disebut **selektor**, dicocokkan dengan **label** setiap cabang. Cabang yang labelnya cocok dijalankan, sisanya dilewati. Cara kerjanya seperti lift: Anda menekan tombol lantai, dan lift langsung berhenti di lantai itu tanpa perlu bertanya "lantai 1? lantai 2?" satu per satu.

BENTUK UMUM

```
case selektor of
  label1: pernyataan1;
  label2, label3: pernyataan2;        { daftar nilai }
  awal..akhir: pernyataan3;             { rentang nilai }
else
  pernyataan_bila_tidak_ada_yang_cocok;
end;
```



Gambar 6.11 Cara kerja `case` pertama pada Program 6.4: hanya satu cabang yang dijalankan, lalu program melompat ke sesudah `end`.

Satu cabang boleh berlaku untuk beberapa nilai. Tulis nilai-nilai itu dipisah koma (`1, 7`) atau sebagai rentang dengan dua titik (`2..6`, artinya 2 sampai 6). Contoh pengelompokan pada `case` kedua Program 6.4 tampak seperti ini: nilai 1 dan 7 masuk cabang libur, nilai 2 sampai 6 masuk cabang sekolah.

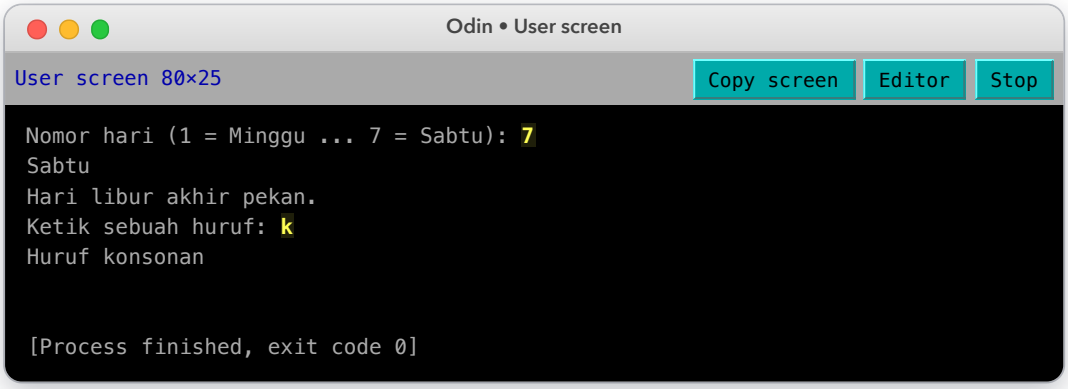
NomorHari	1	2	3	4	5	6	7
	libur	sekolah	sekolah	sekolah	sekolah	sekolah	libur

Program 6.4 memakai tiga `case`: yang pertama dengan satu label per cabang dan cabang `else`, yang kedua dengan daftar dan rentang tanpa `else`, dan yang ketiga dengan selektor bertipe `Char`.

Program 6.4 `bab06_04_case_of.pas` *case..of* File ▸ Examples...

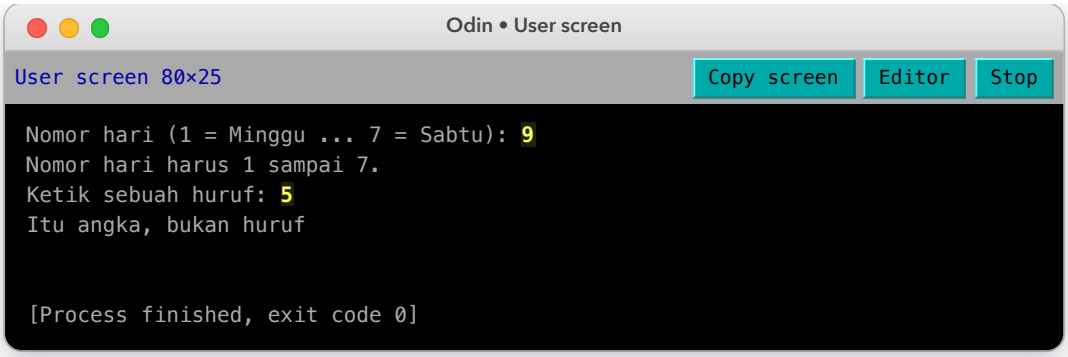
```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 6.4 — case..of
3    case memilih cabang berdasarkan nilai ordinal (Integer, Char, dst).
4    Satu cabang boleh berisi daftar nilai (1, 7) atau rentang (2..6).
5    Cabang else dipakai bila tidak ada nilai yang cocok. }
6
7  program bab06_04_case_of;
8
9  var
10   NomorHari: Integer;
11   Huruf: Char;
12
13 begin
14   Write('Nomor hari (1 = Minggu ... 7 = Sabtu): ');
```

```
15    ReadLn(NomorHari);
16    case NomorHari of
17      1: WriteLn('Minggu');
18      2: WriteLn('Senin');
19      3: WriteLn('Selasa');
20      4: WriteLn('Rabu');
21      5: WriteLn('Kamis');
22      6: WriteLn('Jumat');
23      7: WriteLn('Sabtu');
24    else
25      WriteLn('Nomor hari harus 1 sampai 7.');
26    end;                                  { case selalu ditutup end }
27
28    case NomorHari of
29      1, 7: WriteLn('Hari libur akhir pekan.');   { daftar nilai }
30      2..6: WriteLn('Hari sekolah.');             { rentang nilai }
31    end;
32
33    Write('Ketik sebuah huruf: ');
34    ReadLn(Huruf);
35    Huruf := UpCase(Huruf);               { a..z diubah menjadi A..Z }
36    { Setiap nilai hanya boleh muncul di SATU cabang. Rentang 'B'..'Z'
37      akan bentrok dengan vokal, jadi konsonan ditulis per potongan. }
38    case Huruf of
39      'A', 'I', 'U', 'E', 'O':
40        WriteLn('Huruf vokal');
41      'B'..'D', 'F'..'H', 'J'..'N', 'P'..'T', 'V'..'Z':
42        WriteLn('Huruf konsonan');
43      '0'..'9':
44        WriteLn('Itu angka, bukan huruf');
45    else
46      WriteLn('Bukan huruf');
47    end;
48  end.
```



Gambar 6.12 Hari ke-7 dan huruf `k`

Bagaimana bila nomor harinya di luar 1 sampai 7? Masukan berikut sengaja memakai nomor 9 dan karakter angka `5`. Perhatikan bahwa `case` kedua tidak menulis apa-apa: tidak ada label yang cocok dan tidak ada cabang `else`, jadi seluruh `case` dilewati.



Gambar 6.13 Nomor hari 9 dan karakter `5` : cabang `else` pertama dan cabang angka

Baris	Penjelasan
16	`case NomorHari of`: nilai `NomorHari` menjadi selektor.
17–23	Tujuh cabang, masing-masing dengan satu label konstanta, titik dua, lalu satu pernyataan.
24–25	Cabang `else` dijalankan bila tidak ada label yang cocok. Berbeda dengan `if`, sebelum `else` pada `case` **boleh** ada titik koma.
26	`end;` menutup `case`. Setiap `case` wajib ditutup `end`, walaupun tidak ada `begin` pasangannya.
29–30	Label berupa daftar (`1, 7`) dan rentang (`2..6`). `case` ini tidak punya `else`.
35	Fungsi `UpCase` mengubah huruf kecil menjadi huruf besar, sehingga `k` dan `K` diperlakukan sama. Karakter selain huruf kecil tidak diubah.
38–47	`case` dengan selektor `Char`. Label karakter ditulis di antara tanda petik. Konsonan ditulis per potongan rentang agar tidak bertumpuk dengan vokal.

Mengapa konsonan tidak cukup ditulis `'B'..'Z'`? Karena rentang itu juga memuat `'E'`, `'I'`, `'O'`, dan `'U'` yang sudah dipakai cabang vokal. Dalam satu `case`, setiap nilai hanya boleh muncul di **satu** cabang. Kompiler menolak label yang bertumpuk:

vokal.pas

```
1  program Vokal;
2  var
3    Huruf: Char;
4  begin
5    Huruf := 'K';
6    case Huruf of
7      'A', 'I', 'U', 'E', 'O': WriteLn('Huruf vokal');
8      'B'..'Z': WriteLn('Huruf konsonan');
9    end;
10 end.
```

PESAN KOMPILER

```
vokal.pas(8,5) Error: Identifier ganda: label CASE ganda
```

Gambar 6.14 Label `'B'..'Z'` bertumpuk dengan label vokal

7 **ATURAN TURBO PASCAL 7**

Selektor `case` harus bertipe **ordinal**, yaitu tipe yang nilainya bisa diurutkan dan dihitung satu per satu: `Integer`, `LongInt`, `Char`, `Boolean`, enumerasi, dan subrange. Tipe `Real` dan `String` tidak boleh menjadi selektor. Label harus berupa konstanta yang sudah diketahui saat kompilasi, bukan variabel. Untuk memilih berdasarkan teks atau bilangan pecahan, pakailah rantai `else if`.

if **Pakai if bila...**

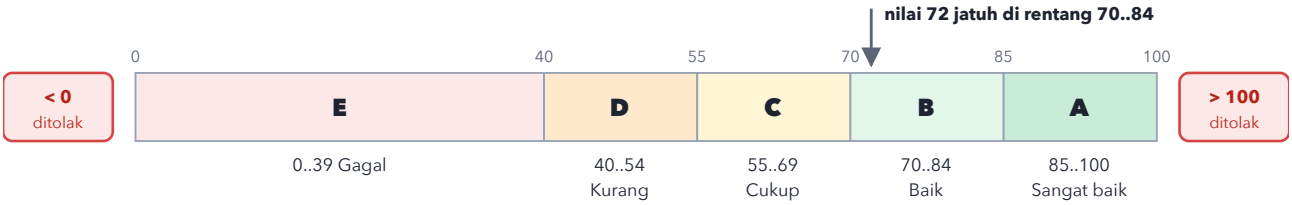
syaratnya berupa perbandingan atau gabungan logika (`Suhu >= 38`, `(A > 0) and (B > 0)`), melibatkan `Real` atau `String`, atau keputusannya hanya ya/tidak.

cs **Pakai case bila...**

pilihan bergantung pada nilai satu variabel ordinal yang bisa didaftar: nomor menu, nomor bulan, huruf jawaban, atau kelompok rentang angka bulat.

6.6 Latihan Terpadu: Nilai Angka Menjadi Nilai Huruf

Banyak sekolah mengubah nilai angka 0 sampai 100 menjadi nilai huruf. Program 6.5 memakai aturan berikut. Sebelum memilih huruf, program menolak nilai di luar 0 sampai 100 dengan `if`, karena nilai seperti 105 atau −3 tidak masuk akal. Pola "saring dulu masukan yang tidak sah, baru proses" ini akan sering Anda pakai.



Setiap rentang menjadi satu label case pada baris 21–24; nilai 0..39 ditangkap cabang else (baris 25–26).

Gambar 6.15 Skala nilai Program 6.5. Rentang tidak bertumpuk dan tidak berlubang: setiap nilai 0..100 masuk tepat satu kelompok.

Program 6.5 `bab06_05_nilai_huruf.pas` *Latihan: nilai angka menjadi nilai huruf* File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 6.5 — Latihan: nilai angka menjadi nilai huruf
3    Nilai 0..100 diubah menjadi A, B, C, D, atau E. Nilai di luar
4    rentang ditolak lebih dulu dengan if, baru kemudian dipilih case. }
5
6  program bab06_05_nilai_huruf;
7
8  var
9    Nilai: Integer;
10   Huruf: Char;
11
12 begin
13   Write('Masukkan nilai (0–100): ');
14   ReadLn(Nilai);
15
16   if (Nilai < 0) or (Nilai > 100) then
```

```
17      WriteLn('Nilai tidak sah. Harus antara 0 dan 100.')
18    else
19    begin
20      case Nilai of
21        85..100: Huruf := 'A';
22        70..84:  Huruf := 'B';
23        55..69:  Huruf := 'C';
24        40..54:  Huruf := 'D';
25      else
26        Huruf := 'E';                    { 0..39 }
27      end;
28
29      WriteLn('Nilai huruf: ', Huruf);
30      case Huruf of
31        'A': WriteLn('Keterangan: Sangat baik');
32        'B': WriteLn('Keterangan: Baik');
33        'C': WriteLn('Keterangan: Cukup');
34        'D': WriteLn('Keterangan: Kurang');
35        'E': WriteLn('Keterangan: Gagal');
36      end;
37    end;
38  end.
```



Gambar 6.16 Nilai 72 menjadi B



Gambar 6.17 Nilai 105 ditolak oleh `if` pada baris 16, sebelum `case` sempat bekerja

Baris	Penjelasan
16–17	Penyaring masukan. Syarat `(Nilai < 0) or (Nilai > 100)` bernilai `True` untuk nilai yang tidak sah.
18–37	Cabang `else` berupa pernyataan majemuk berisi dua `case` dan satu `WriteLn`. Bagian ini hanya dijalankan untuk nilai yang sah.
20–27	`case` dengan label rentang. Hasilnya disimpan di variabel `Huruf`, belum ditulis ke layar.
29	Huruf ditulis.

Baris	Penjelasan
30–36	`case` kedua memilih keterangan berdasarkan `Huruf`. Memisahkan "menghitung huruf" dari "menulis keterangan" membuat setiap bagian pendek dan mudah diperiksa.

COBA SENDIRI

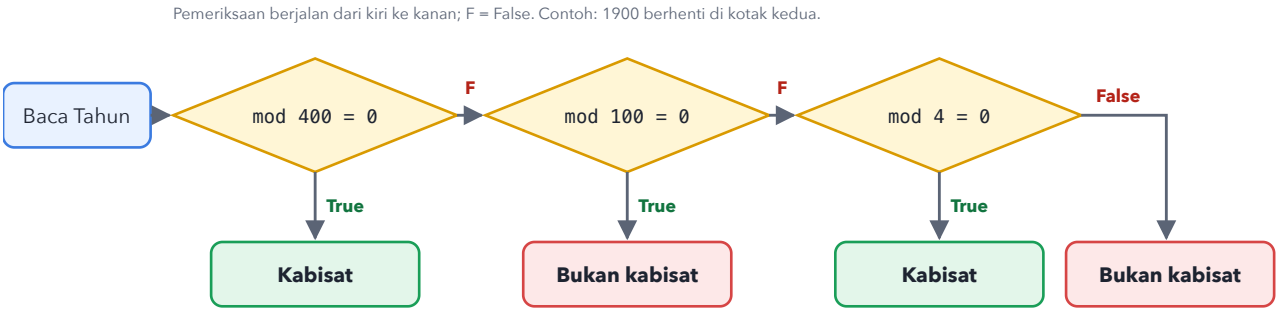
Uji Program 6.5 dengan nilai-nilai **batas**: 0, 39, 40, 54, 55, 84, 85, dan 100. Kesalahan program percabangan paling sering bersembunyi tepat di angka batas, misalnya ketika `<` tertukar dengan `<=`.

6.7 Latihan Terpadu: Tahun Kabisat

Bumi mengelilingi matahari dalam kira-kira 365,2425 hari. Agar kalender tidak bergeser, kalender Masehi menambahkan tanggal 29 Februari pada **tahun kabisat**. Aturannya ada tiga lapis:

1. Tahun yang habis dibagi 4 adalah tahun kabisat,
2. kecuali tahun yang habis dibagi 100, yang bukan tahun kabisat,
3. kecuali lagi tahun yang habis dibagi 400, yang tetap tahun kabisat.

Aturan berlapis seperti ini paling mudah diterjemahkan ke rantai `else if` bila diperiksa dari pengecualian yang paling khusus, yaitu habis dibagi 400. Diagram berikut memperlihatkan urutan pemeriksaan pada Program 6.6.



Gambar 6.18 Urutan pemeriksaan tahun kabisat pada Program 6.6, baris 18–25. Pengecualian yang paling khusus diperiksa paling dulu.

Sebelum menulis program, uji aturan itu dengan tangan menggunakan **tabel keputusan**: setiap baris adalah satu contoh tahun, setiap kolom adalah satu pertanyaan. Tabel ini sekaligus menjadi daftar uji yang lengkap, karena setiap cabang pada diagram di atas diwakili minimal satu tahun.

Tabel 6.3 Tabel keputusan tahun kabisat

Tahun	Habis dibagi 400?	Habis dibagi 100?	Habis dibagi 4?	Hasil
2023	tidak	tidak	tidak	bukan kabisat (Februari 28 hari)
2024	tidak	tidak	ya	kabisat (Februari 29 hari)
1900	tidak	ya	ya	bukan kabisat: kena pengecualian 100

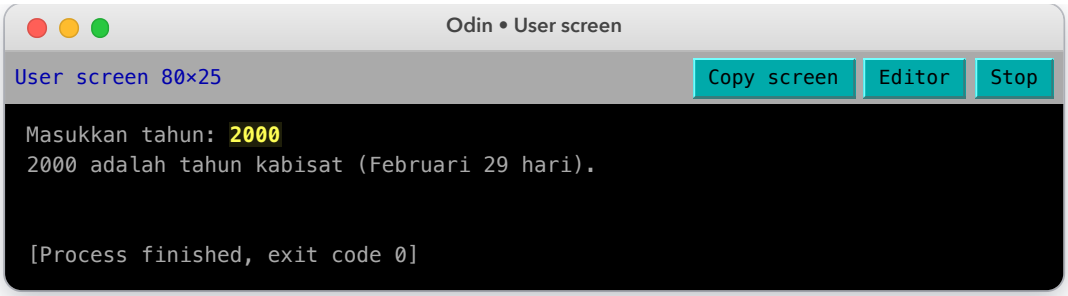
Tahun	Habis dibagi 400?	Habis dibagi 100?	Habis dibagi 4?	Hasil
2100	tidak	ya	ya	bukan kabisat: kena pengecualian 100
2000	ya	ya	ya	kabisat: pengecualian 400 mengalahkan aturan 100

Program 6.6 `bab06_06_tahun_kabisat.pas` *Latihan: tahun kabisat* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 6.6 — Latihan: tahun kabisat
3    Aturan kalender Masehi: tahun kabisat habis dibagi 4, KECUALI habis
4    dibagi 100. Tetapi yang habis dibagi 400 tetap kabisat.
5    Contoh: 2024 kabisat, 1900 bukan, 2000 kabisat. }
6
7  program bab06_06_tahun_kabisat;
8
9  var
10   Tahun: Integer;
11   Kabisat: Boolean;
12
13 begin
14   Write('Masukkan tahun: ');
15   ReadLn(Tahun);
16
17   { Cara 1: if bersarang, mengikuti aturan langkah demi langkah }
18   if Tahun mod 400 = 0 then
19     Kabisat := True
20   else if Tahun mod 100 = 0 then
21     Kabisat := False
22   else if Tahun mod 4 = 0 then
23     Kabisat := True
24   else
25     Kabisat := False;
26
27   { Cara 2: satu ekspresi logika. Hasilnya harus sama dengan cara 1. }
28   if Kabisat <> ((Tahun mod 4 = 0) and (Tahun mod 100 <> 0)
29                  or (Tahun mod 400 = 0)) then
30     WriteLn('Aneh, kedua cara memberi hasil berbeda!');
31
32   if Kabisat then
33     WriteLn(Tahun, ' adalah tahun kabisat (Februari 29 hari).')
34   else
35     WriteLn(Tahun, ' bukan tahun kabisat (Februari 28 hari).');
36 end.
```



Gambar 6.19 Tahun 1900 habis dibagi 100 tetapi tidak habis dibagi 400



Gambar 6.20 Tahun 2000 habis dibagi 400, jadi tetap kabisat

Baris	Penjelasan
11	Variabel `Kabisat` bertipe `Boolean`: tempat menyimpan hasil keputusan untuk dipakai belakangan.
18–25	Cara 1, rantai `else if` sesuai **Gambar 6.18**. Setiap cabang mengisi `Kabisat` dengan `True` atau `False`.
28–30	Cara 2, aturan yang sama dalam satu ekspresi logika. Bila hasilnya berbeda dari cara 1, program mengeluh. Dengan aturan yang benar, pesan pada baris 30 tidak pernah muncul.
32–35	`if Kabisat then` sudah cukup; tidak perlu menulis `if Kabisat = True then`, karena `Kabisat` sendiri sudah bernilai Boolean.

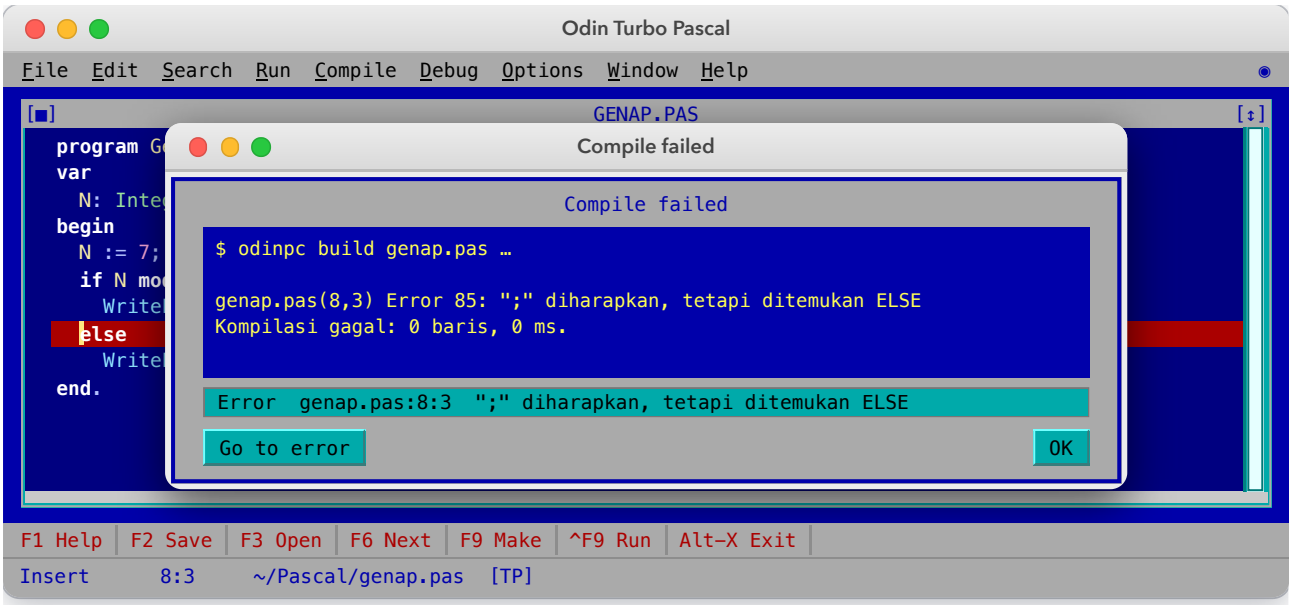
Ekspresi pada baris 28–29 layak diperhatikan. Di Pascal, `and` dikerjakan lebih dulu daripada `or`, seperti perkalian dikerjakan lebih dulu daripada penjumlahan. Jadi ekspresinya dibaca sebagai "(habis dibagi 4 **dan** tidak habis dibagi 100) **atau** habis dibagi 400". Setiap perbandingan wajib diberi tanda kurung sendiri, karena operator pembanding seperti `=` dan `<>` justru dikerjakan paling akhir. Tanpa kurung, kompiler akan mencoba menghitung `100 and Tahun` lebih dulu, dan hasilnya kacau (lihat subbab berikutnya).

> ★ **INGAT**
>
> Variabel `Boolean` bisa langsung diisi hasil perbandingan, tanpa `if`: `Kabisat := (Tahun mod 4 = 0) and (Tahun mod 100 <> 0) or (Tahun mod 400 = 0);`. Penugasan seperti ini sama artinya dengan rantai `if` pada baris 18–25.

6.8 Kesalahan yang Sering Terjadi

6.8.1 Titik koma sebelum else

Ini kesalahan percabangan nomor satu bagi pemula. Program berikut menulis titik koma di akhir baris 7, tepat sebelum `else`. Di Odin Turbo Pascal, kompilasi yang gagal membuka jendela **Compile failed**, dan baris tempat kompiler tersandung ditandai merah.



Gambar 6.21 Titik koma di akhir baris 7 membuat `else` pada baris 8 tidak punya pasangan

Kompiler menunjuk baris 8, tempat `else` berada, padahal penyebabnya ada di akhir baris 7. Titik koma itu menutup pernyataan `if`, sehingga kata `else` sesudahnya menjadi awal pernyataan baru, dan tidak ada pernyataan Pascal yang diawali `else`. Perbaikannya: hapus titik koma di akhir baris 7.

6.8.2 Titik koma sesudah then

Kebalikan dari kesalahan di atas jauh lebih berbahaya, karena kompiler tidak menolaknya. Titik koma tepat sesudah `then` membuat cabang `if` berisi **pernyataan kosong**. Perintah pada baris berikutnya tidak lagi menjadi bagian dari `if`, sehingga selalu dijalankan.

demam.pas

```
1  program Demam;
2  var
3    Suhu: Integer;
4  begin
5    Suhu := 36;
6    if Suhu >= 38 then;
7      WriteLn('Anda demam. Istirahatlah.');
8    WriteLn('Pemeriksaan selesai.');
9  end.
```



Gambar 6.22 Suhu 36, tetapi program tetap menyuruh istirahat

Indentasi baris 7 membuatnya tampak berada di dalam `if`, tetapi titik koma di akhir baris 6 sudah menutup `if` itu. Kesalahan logika seperti ini hanya bisa ditemukan dengan menguji program memakai data yang hasilnya sudah Anda ketahui. Hal yang sama berlaku untuk `else;`.

6.8.3 case tanpa end

Setiap `case` harus ditutup `end`. Karena `case` tidak diawali `begin`, `end` ini mudah terlupa, terutama bila `case` berada di akhir program.

hari.pas

```
1  program Hari;
2  var
3    N: Integer;
4  begin
5    N := 3;
6    case N of
7      1: WriteLn('Minggu');
8      2: WriteLn('Senin');
9      3: WriteLn('Selasa');
10 end.
```

PESAN KOMPILER

```
hari.pas(10,4) Error: ";" diharapkan, tetapi ditemukan "."
```

Gambar 6.23 Kompiler memakai end. terakhir untuk menutup case, sehingga program utama tidak punya penutup

Kata `end` pada baris 10 dipakai kompiler untuk menutup `case`. Akibatnya `begin` pada baris 4 tidak punya pasangan, dan titik sesudahnya muncul di tempat yang tidak terduga. Pesan kesalahan seperti ini sering membingungkan karena menunjuk baris terakhir. Bila kompiler mengeluh tentang akhir program, periksa apakah setiap `case` dan setiap `begin` sudah punya `end`.

6.8.4 Lupa tanda kurung pada syarat gabungan

rentang.pas

```
1  program Rentang;
2  var
3    Nilai: Integer;
4  begin
5    Nilai := 105;
6    if Nilai < 0 or Nilai > 100 then
7      WriteLn('Nilai tidak sah');
8  end.
```

PESAN KOMPILER

```
rentang.pas(6,6) Error: Tipe operand tidak cocok dengan operator
```

Gambar 6.24 Tanpa kurung, `0 or Nilai` dihitung lebih dulu

Karena `or` dikerjakan lebih dulu daripada `<` dan `>`, kompiler membaca baris 6 sebagai `Nilai < (0 or Nilai) > 100`, sesuatu yang tidak masuk akal. Tulislah setiap perbandingan di dalam kurung: `(Nilai < 0) or (Nilai > 100)`.

6.9 Rangkuman

- Percabangan membuat program memilih jalan berdasarkan **syarat**, yaitu ekspresi Boolean yang bernilai `True` atau `False`.
- `if..then` menjalankan satu pernyataan bila syarat `True`. Untuk beberapa perintah, bungkus dengan `begin..end` (pernyataan majemuk).
- `if..then..else` selalu menjalankan tepat satu dari dua cabang. **Tidak ada titik koma sebelum `else`**; titik koma sesudah `then` membuat cabang kosong.
- Rantai `else if` memeriksa syarat dari atas ke bawah dan berhenti di syarat pertama yang `True`, jadi urutan syarat penting.
- `else` selalu milik `if` terdekat yang belum punya `else` (*dangling else*); gunakan `begin..end` untuk memperjelas.
- `case..of` memilih cabang berdasarkan nilai selektor ordinal. Label boleh berupa daftar (`1, 7`) atau rentang (`2..6`), tidak boleh bertumpuk, dan `case` selalu ditutup `end`.
- Dalam syarat gabungan, beri kurung pada setiap perbandingan; `and` dikerjakan sebelum `or`.
- Uji program percabangan dengan satu masukan untuk setiap cabang, termasuk nilai-nilai batas.

6.10 Latihan

1. **DASAR** Buat program yang membaca sebuah bilangan bulat, lalu menulis `positif`, `negatif`, atau `nol`.
2. **DASAR** Ubah Program 6.1 agar suhu 35 sampai 37 menampilkan `Suhu normal.` Gunakan rantai `else if` sehingga hanya satu pesan suhu yang mungkin muncul.
3. **MENENGAH** Baca tiga bilangan bulat, lalu tampilkan yang terbesar. Uji dengan masukan yang terbesarnya di posisi pertama, kedua, dan ketiga, serta dengan tiga bilangan yang sama.
4. **MENENGAH** Buat program nomor bulan 1–12 yang menampilkan nama bulan dan jumlah harinya. Gunakan `case` dengan daftar nilai untuk bulan 30 dan 31 hari. Untuk Februari, tanyakan tahunnya dan pakai aturan kabisat dari Program 6.6.
5. **MENENGAH** Buat kalkulator sederhana: baca dua bilangan dan sebuah operator `Char` (`+`, `-`, `*`, `/`), lalu hitung dengan `case`. Tolak pembagian dengan nol dengan pesan yang jelas.
6. **TANTANGAN** Baca panjang tiga sisi, lalu tentukan apakah ketiganya membentuk segitiga (setiap dua sisi harus lebih panjang dari sisi ketiga). Bila ya, sebutkan jenisnya: sama sisi, sama kaki, atau sembarang.

7. **TANTANGAN** Tulis ulang `case` pertama Program 6.5 sebagai rantai `else if`. Uji kedua versi dengan nilai-nilai batas yang sama, lalu jelaskan mana yang menurut Anda lebih mudah dibaca dan mengapa.

BAB 07

BAGIAN II · ALUR KENDALI

Perulangan

1 2 3 4 5 6 **7** 8 9 10 11 12 13 14 15 16 17 18 19 20

TUJUAN PEMBELAJARAN

- Menjelaskan mengapa perulangan (*loop*) diperlukan dan membedakan perulangan terhitung dari perulangan bersyarat.
- Memakai `for..to` dan `for..downto`, termasuk pencacah bertipe `Char`, serta mengetahui aturan pencacah.
- Memakai `while..do` dan `repeat..until`, dan membedakan kapan syarat diperiksa dan kapan perulangan berhenti.
- Menyusun perulangan bersarang untuk tabel, baris, dan kolom.
- Menggabungkan `repeat..until` dengan `case` menjadi menu yang berulang.
- Melacak perulangan dengan tabel jejak nilai dan dengan debugger Odin (breakpoint, baris eksekusi, dan Watches).

PROGRAM DALAM BAB INI

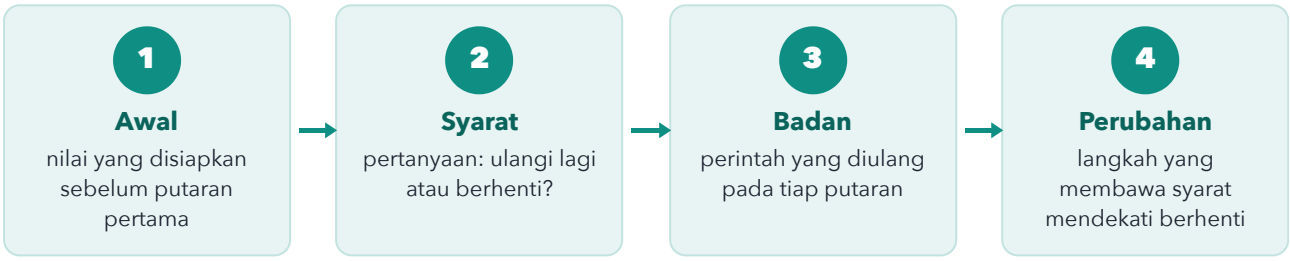
7.1	`bab07_01_for_to.pas`	Perulangan for..to
7.2	`bab07_02_for_downto.pas`	Perulangan for..downto
7.3	`bab07_03_while_do.pas`	Perulangan while..do
7.4	`bab07_04_repeat_until.pas`	Perulangan repeat..until
7.5	`bab07_05_perulangan_bersarang.pas`	Perulangan bersarang
7.6	`bab07_06_menu_berulang.pas`	Latihan: menu berulang

Komputer unggul dalam satu hal yang membosankan bagi manusia: mengerjakan hal yang sama berulang-ulang tanpa lelah dan tanpa salah. Bayangkan menulis bilangan 1 sampai 1000, atau menjumlahkan seribu nilai ujian. Menulis seribu perintah `WriteLn` tentu tidak masuk akal. Di bab ini Anda akan mempelajari **perulangan**: cara menyuruh program mengulang sekelompok perintah sebanyak yang diperlukan, dengan hanya menulisnya sekali.

7.1 Mengapa Perlu Perulangan

Misalkan Anda ingin program menulis lima baris "Halo" di layar. Dengan pengetahuan dari bab-bab sebelumnya, jalan satu-satunya adalah menulis `WriteLn('Halo')` lima kali. Untuk lima baris ini masih tertahankan, tetapi bagaimana kalau seratus baris? Atau jumlah barisnya baru diketahui saat program berjalan, misalnya diketik pengguna? Perulangan menyelesaikan kedua masalah itu: perintahnya ditulis satu kali, dan Pascal yang mengulanginya.

Setiap perulangan memiliki tiga bagian yang selalu ada, walaupun bentuk penulisannya berbeda-beda.



Satu kali menjalankan badan perulangan disebut satu **putaran** atau **iterasi** (*iteration*). Bagian "perubahan" paling sering dilupakan pemula, padahal tanpa itu perulangan tidak akan pernah berhenti. Pascal menyediakan empat bentuk perulangan, yang dibagi menjadi dua keluarga.

A Perulangan terhitung

Jumlah putaran sudah diketahui sebelum perulangan mulai. Bentuknya `for..to` dan `for..downto`. Pencacah diurus otomatis oleh Pascal.

B Perulangan bersyarat

Jumlah putaran tergantung keadaan yang berubah selama program berjalan. Bentuknya `while..do` dan `repeat..until`. Perubahan syarat Anda urus sendiri.

Tabel 7.1 Empat perulangan Pascal dan kapan memakainya

Bentuk	Kapan dipakai	Syarat diperiksa	Badan minimal
`for..to`	menghitung naik, jumlah putaran diketahui	sebelum tiap putaran	0 kali
`for..downto`	menghitung turun, jumlah putaran diketahui	sebelum tiap putaran	0 kali
`while..do`	mengulang selama syarat terpenuhi	sebelum tiap putaran	0 kali
`repeat..until`	mengulang sampai syarat terpenuhi	sesudah tiap putaran	1 kali

7.2 Perulangan Terhitung: for..to

Bentuk `for` cocok bila Anda tahu persis berapa kali sesuatu harus diulang: "tulis sepuluh baris", "jumlahkan seratus bilangan". Anda menyediakan sebuah variabel **pencacah** (*counter*), nilai awal, dan nilai akhir. Pascal mengisi pencacah dengan nilai awal, menjalankan badan, menaikkan pencacah satu angka, dan seterusnya sampai pencacah melewati nilai akhir.

BENTUK UMUM

```
for pencacah := nilai_awal to nilai_akhir do
  pernyataan;

for pencacah := nilai_awal to nilai_akhir do
begin
  pernyataan1;
  pernyataan2;
end;
```

Seperti pada `if`, badan `for` hanya satu pernyataan; gunakan `begin..end` untuk beberapa pernyataan. Diagram berikut menunjukkan perjalanan program pada perulangan yang menjumlahkan 1 sampai `N`. Perhatikan panah yang kembali ke belah ketupat: itulah "ulang" dalam perulangan.



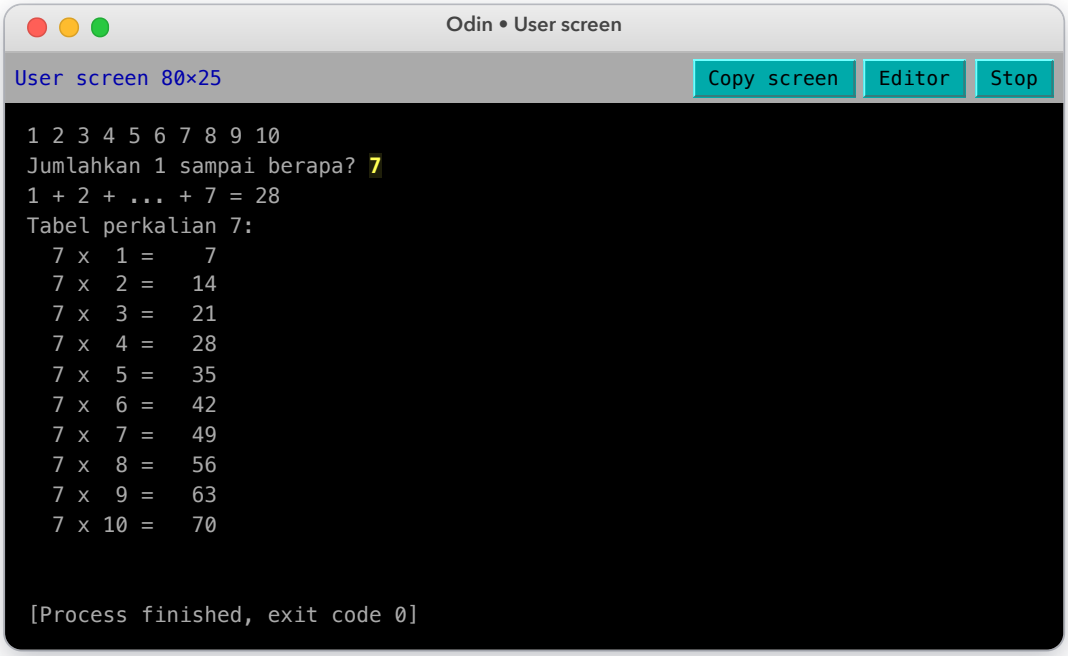
Gambar 7.1 Diagram alir `for..to`: syarat `I <= N` diperiksa sebelum setiap putaran, dan pencacah dinaikkan otomatis.

Program 7.1 memakai `for..to` tiga kali. Buka `bab07_01_for_to.pas` lewat `File ▸ Examples...`, lalu jalankan dengan Ctrl+F9. Masukan bawaan untuk run di bawah adalah 7.

Program 7.1 `bab07_01_for_to.pas` *Perulangan for..to* File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 7.1 — Perulangan for..to
3    for mengulang perintah sebanyak yang sudah diketahui. Variabel
4    pencacah naik 1 setiap putaran, dari nilai awal sampai nilai akhir. }
5
```

```
6  program bab07_01_for_to;
7
8  var
9    I, N: Integer;
10   Jumlah: LongInt;
11
12 begin
13   { Menulis 1 sampai 10 dalam satu baris }
14   for I := 1 to 10 do
15     Write(I, ' ');
16   WriteLn;
17
18   { Menjumlahkan 1 + 2 + ... + N }
19   Write('Jumlahkan 1 sampai berapa? ');
20   ReadLn(N);
21   Jumlah := 0;                        { mulai dari nol }
22   for I := 1 to N do
23     Jumlah := Jumlah + I;             { tambahkan I pada setiap putaran }
24   WriteLn('1 + 2 + ... + ', N, ' = ', Jumlah);
25
26   { Lebih dari satu perintah per putaran: pakai begin..end }
27   WriteLn('Tabel perkalian ', N, ':');
28   for I := 1 to 10 do
29   begin
30     Write(N:3, ' x ', I:2, ' = ');
31     WriteLn(N * I:4);
32   end;
33
34   { Jangan mengubah nilai I di dalam badan for: kompiler menolaknya }
35 end.
```



Gambar 7.2 Program 7.1 dengan masukan 7: jumlah 1 sampai 7 dan tabel perkalian 7

Baris	Penjelasan
9–10	`I` dan `N` bertipe `Integer`. `Jumlah` bertipe `LongInt` agar muat bila jumlahnya melewati 32767.

Baris	Penjelasan
14–15	Perulangan pertama: `I` bernilai 1, 2, ..., 10. Badannya satu pernyataan `Write(I, ' ')`, jadi tidak perlu `begin..end`. Semua angka tampil dalam satu baris karena memakai `Write`, bukan `WriteLn`.
16	`WriteLn` tanpa isi memindahkan kursor ke baris baru, setelah sepuluh angka tadi.
19–20	Menanyakan `N` kepada pengguna.
21	`Jumlah` diisi nol sebelum perulangan. Ini bagian "awal" dari penjumlahan: nilai awal harus netral terhadap penjumlahan.
22–23	Pada setiap putaran, `Jumlah` ditambah nilai `I` saat itu. Pola `Jumlah := Jumlah + I` disebut **akumulator**: nilai lama dibaca, ditambah, lalu ditulis kembali.
24	Menulis hasil penjumlahan sesudah perulangan selesai. Karena berada di luar badan `for`, baris ini hanya dijalankan sekali.
28–32	Perulangan dengan `begin..end` karena badannya dua pernyataan. `N:3`, `I:2`, dan `N * I:4` meratakan angka ke kanan pada lebar tertentu (Bab 4), sehingga kolom tabel lurus.

Untuk memahami apa yang terjadi di dalam perulangan, cara terbaik adalah membuat **tabel jejak** (*trace table*): satu baris tabel untuk setiap putaran, berisi nilai variabel pada saat itu. Tabel berikut melacak baris 22–23 untuk `N = 5`.

Tabel 7.2 Jejak penjumlahan 1 sampai 5 pada Program 7.1 (`N = 5`)

Putaran	`I`	`Jumlah` sebelum	Perhitungan	`Jumlah` sesudah
1	1	0	0 + 1	1
2	2	1	1 + 2	3
3	3	3	3 + 3	6
4	4	6	6 + 4	10
5	5	10	10 + 5	15

Sesudah putaran kelima, pencacah menjadi 6, syarat `I <= N` bernilai `False`, dan perulangan selesai dengan `Jumlah = 15`. Dua puluh kali lipat dari pekerjaan ini pun tinggal mengganti nilai `N`; programnya tetap sama.

AWAS

Jangan mengubah pencacah di dalam badan `for`. Pascal mengatur sendiri nilai `I`; bila badan ikut mengubahnya, hasil perulangan tidak lagi dapat diramalkan. Aturan Pascal melarangnya, dan sebagian kompiler menolak program semacam itu, jadi anggap ini larangan mutlak walaupun kompiler tertentu tidak memberi peringatan. Pencacah juga harus bertipe ordinal (`Integer`, `Char`, atau tipe urutan lain), tidak boleh `Real`.

COBA SENDIRI

Ubah baris 14 menjadi `for I := 1 to 10 do` yang mencetak `I * I` (kuadrat), atau ubah baris 28 agar tabel perkalian berhenti di 12. Jalankan dan cocokkan hasilnya dengan perkiraan Anda sebelum menekan **Enter**.

7.3 Menghitung Mundur: for..downto

Bila pencacah harus *turun*, ganti `to` dengan `downto`. Mekanismenya persis sama, hanya arah pencacahnya terbalik: pencacah dikurangi satu setiap putaran, dan perulangan berjalan selama pencacah masih lebih besar atau sama dengan nilai akhir.

BENTUK UMUM

```
for pencacah := nilai_awal downto nilai_akhir do
  pernyataan;
```

Bandingkan keduanya pada contoh berikut. Pada `for..to`, nilai awal harus **lebih kecil atau sama** dengan nilai akhir agar badan berjalan; pada `for..downto`, nilai awal harus **lebih besar atau sama**.

Tabel 7.3 Perbandingan `to` dan `downto`

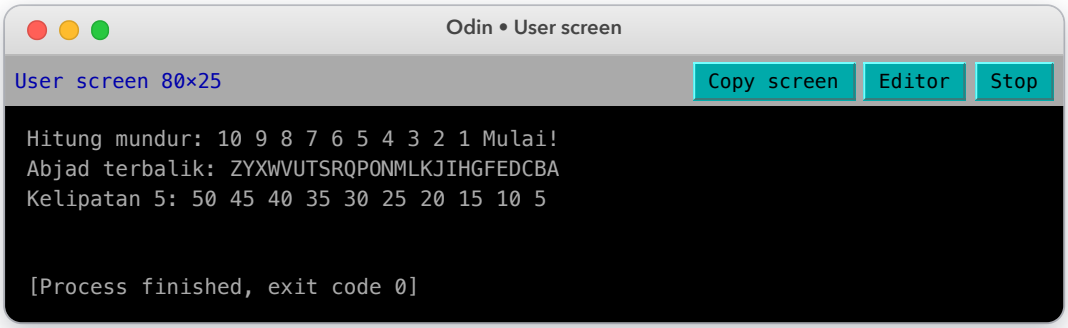
Perintah	Nilai pencacah	Jumlah putaran
`for I := 1 to 5 do`	1, 2, 3, 4, 5	5
`for I := 5 downto 1 do`	5, 4, 3, 2, 1	5
`for I := 5 to 1 do`	(tidak ada)	0
`for I := 1 downto 5 do`	(tidak ada)	0
`for I := 3 to 3 do`	3	1

Program 7.2 juga memperkenalkan pencacah bertipe `Char`. Karakter punya urutan tetap (A sebelum B, B sebelum C, dan seterusnya), sehingga bisa dihitung maju atau mundur seperti bilangan.

Program 7.2 `bab07_02_for_downto.pas` *Perulangan for..downto* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 7.2 — Perulangan for..downto
3    downto menghitung mundur: pencacah turun 1 setiap putaran.
4    Pencacah juga boleh bertipe Char. }
5
6  program bab07_02_for_downto;
7
8  var
9    I: Integer;
10   Huruf: Char;
11
12 begin
13   Write('Hitung mundur: ');
14   for I := 10 downto 1 do
15     Write(I, ' ');
16   WriteLn('Mulai!');
17
18   { Pencacah bertipe Char berjalan menurut urutan abjad }
19   Write('Abjad terbalik: ');
20   for Huruf := 'Z' downto 'A' do
21     Write(Huruf);
22   WriteLn;
23
24   { Bila nilai awal lebih kecil dari nilai akhir, downto tidak
```

```
25      berjalan sama sekali }
26    for I := 1 downto 5 do
27      WriteLn('Baris ini tidak pernah tampil');
28
29    { Kelipatan 5 dari 50 turun ke 5, memakai perkalian }
30    Write('Kelipatan 5: ');
31    for I := 10 downto 1 do
32      Write(I * 5, ' ');
33    WriteLn;
34  end.
```



Gambar 7.3 Program 7.2: hitung mundur, abjad terbalik, dan kelipatan 5

Baris	Penjelasan
13–16	Menghitung mundur dari 10 ke 1. Kata "Mulai!" ditulis dengan `WriteLn` sesudah perulangan, sehingga muncul di akhir baris yang sama.
19–22	`Huruf` bertipe `Char`. Perulangan berjalan dari `'Z'` mundur ke `'A'`, dan `Write(Huruf)` mencetak satu karakter tiap putaran: 26 huruf tanpa spasi.
26–27	Nilai awal 1 lebih kecil dari nilai akhir 5, padahal arahnya `downto`. Syarat gagal sejak awal, sehingga `WriteLn` di baris 27 tidak pernah dijalankan. Tidak tampak apa pun di layar, dan itu yang dimaksud.
30–33	Pencacah turun 10 sampai 1, tetapi yang dicetak adalah `I * 5`. Dengan begitu keluaran menjadi 50, 45, ..., 5 tanpa memerlukan "langkah" sebesar 5.

INFO

Pascal tidak mengenal `step` seperti beberapa bahasa lain: pencacah selalu berubah satu angka. Bila Anda memerlukan kelipatan atau loncatan, hitung dari pencacah, seperti `I * 5` pada baris 32.

ATURAN TURBO PASCAL 7

Pada Turbo Pascal 7, pencacah `for` boleh bertipe apa pun yang ordinal: `Integer`, `Byte`, `Char`, `Boolean`, atau tipe enumerasi dan subrentang. Tipe `Real` tidak termasuk. Pencacah juga harus variabel lokal dari blok tempat `for` itu ditulis.

7.4 Perulangan Bersyarat: while..do

Tidak semua pengulangan bisa dihitung di muka. Berapa kali sebuah bilangan harus dibagi 10 sampai habis? Berapa bilangan yang akan diketik pengguna sebelum ia mengetik 0? Untuk keadaan semacam ini, Pascal menyediakan `while..do` ("selama ... lakukan"). Perulangan ini memeriksa sebuah syarat **sebelum** setiap putaran: bila syarat `True`, badan dijalankan lalu syarat diperiksa lagi; bila `False`, perulangan selesai.

BENTUK UMUM

```
while syarat do
  pernyataan;

while syarat do
begin
  pernyataan1;
  pernyataan2;
end;
```

Ada dua konsekuensi dari sifat "periksa dulu". Pertama, bila syarat sudah `False` sejak awal, badan tidak dijalankan sama sekali. Kedua, dan ini yang berbahaya, **Anda sendiri** yang harus memastikan ada perintah di dalam badan yang pada akhirnya membuat syarat menjadi `False`. Bila tidak, perulangan berjalan selamanya (*infinite loop*). Pada `for`, Pascal mengurus perubahan itu; pada `while`, tugas itu milik programmer.

Program 7.3 berisi dua contoh klasik. Contoh pertama menjumlahkan digit suatu bilangan dengan terus mengambil digit paling kanan (`mod 10`) lalu membuangnya (`div 10`). Contoh kedua membaca bilangan sampai pengguna mengetik 0 sebagai tanda berhenti; tanda semacam ini disebut **sentinel**.

Program 7.3 `bab07_03_while_do.pas` *Perulangan while..do* File ▸ Examples...

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 7.3 — Perulangan while..do
  while memeriksa syarat SEBELUM mengulang. Selama syarat True,
  badannya diulang. Bila sejak awal False, badannya tidak dijalankan.
  Pastikan ada perintah yang akhirnya membuat syarat menjadi False. }

program bab07_03_while_do;

var
  Angka, Sisa, JumlahDigit: LongInt;
  Bilangan, Total, Banyak: Integer;

begin
  { Contoh 1: menjumlahkan digit sebuah bilangan }
  Write('Masukkan bilangan bulat positif: ');
  ReadLn(Angka);
  Sisa := Angka;
  JumlahDigit := 0;
  while Sisa > 0 do
  begin
    JumlahDigit := JumlahDigit + Sisa mod 10;   { ambil digit kanan }
    Sisa := Sisa div 10;                        { buang digit kanan }
```

```
23    end;
24    WriteLn('Jumlah digit ', Angka, ' = ', JumlahDigit);
25    WriteLn;
26
27    { Contoh 2: membaca bilangan sampai pengguna mengetik 0 }
28    WriteLn('Ketik beberapa bilangan, akhiri dengan 0.');
29    Total := 0;
30    Banyak := 0;
31    Write('Bilangan: ');
32    ReadLn(Bilangan);
33    while Bilangan <> 0 do                { 0 adalah tanda berhenti }
34    begin
35      Total := Total + Bilangan;
36      Banyak := Banyak + 1;
37      Write('Bilangan: ');
38      ReadLn(Bilangan);
39    end;
40    WriteLn('Banyak bilangan: ', Banyak, ', jumlahnya: ', Total);
41  end.
```



Gambar 7.4 Program 7.3: jumlah digit 9875, lalu tiga bilangan (4, 6, dan -2) diakhiri 0

Mari lacak contoh pertama untuk `Angka = 9875`. Gambar berikut memperlihatkan bagaimana bilangan itu "dikupas" satu digit tiap putaran, dan **Tabel 7.4** mencatat nilai variabelnya.



Gambar 7.5 Program 7.3 mengupas bilangan 9875: digit kanan diambil dengan `mod 10`, lalu dibuang dengan `div 10`.

Tabel 7.4 Jejak contoh pertama Program 7.3 untuk `Angka = 9875`

Putaran	`Sisa` awal	`Sisa > 0`	`Sisa mod 10`	`JumlahDigit`	`Sisa` akhir
1	9875	`True`	5	5	987
2	987	`True`	7	12	98
3	98	`True`	8	20	9
4	9	`True`	9	29	0
(cek)	0	`False`	perulangan berhenti		

Baris	Penjelasan
10–11	`LongInt` dipakai untuk bilangan yang dikupas, `Integer` untuk contoh kedua.
15–18	Bilangan dibaca, lalu disalin ke `Sisa` supaya `Angka` asli tetap utuh untuk ditampilkan di baris 24. `JumlahDigit` diisi nol.
19–23	`while Sisa > 0`. Baris 21 menambahkan digit paling kanan, baris 22 membuangnya. Baris 22 adalah bagian "perubahan" yang menjamin perulangan berakhir: `Sisa` menyusut sampai 0.
28–32	Persiapan contoh kedua: `Total` dan `Banyak` dinolkan, lalu bilangan **pertama** dibaca *sebelum* perulangan.
33–39	`while Bilangan <> 0`. Di dalam badan, bilangan dijumlahkan (baris 35), dihitung (baris 36), lalu bilangan berikutnya dibaca (baris 37–38). Pembacaan berikutnya ditaruh di **akhir** badan agar syarat segera menilai bilangan yang baru.
40	Nilai 0 tidak ikut dijumlahkan karena perulangan berhenti sebelum badan menyentuhnya. Bilangan negatif (-2) tetap ikut, sehingga 4 + 6 + (-2) = 8.

TIPS

Pola "baca sekali sebelum perulangan, baca lagi di akhir badan" (*priming read*) sangat umum pada `while`. Dengan pola ini nilai sentinel tidak pernah diproses sebagai data.

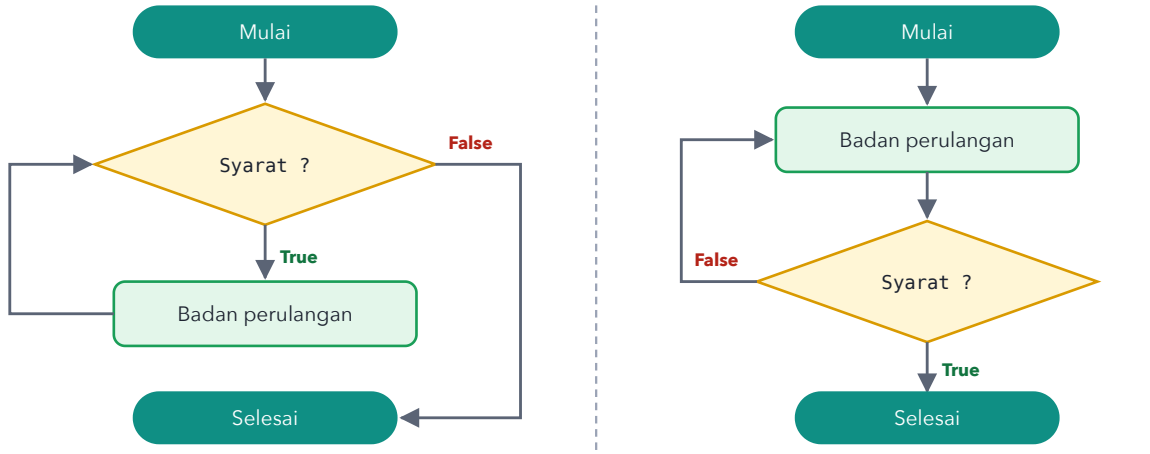
7.5 Mengulang Sampai: repeat..until

Bentuk keempat membalik urutannya: badan dijalankan **lebih dulu**, barulah syarat diperiksa. Selain itu, maknanya juga terbalik. `repeat..until` berarti "ulangi sampai ...", sehingga perulangan berhenti ketika syarat bernilai `True`. Akibatnya badan **selalu dijalankan paling sedikit satu kali**.

BENTUK UMUM

```
repeat
  pernyataan1;
  pernyataan2;
until syarat_berhenti;
```

Kata `repeat` dan `until` sudah berfungsi sebagai tanda awal dan akhir, jadi badan beberapa pernyataan **tidak** memerlukan `begin..end`. Perbedaan dengan `while..do` paling mudah dipahami dari diagram alirnya.



while..do
Periksa dulu. Ulangi selama syarat True. Badan bisa 0 kali.

repeat..until
Jalankan dulu. Ulangi sampai syarat True. Badan minimal 1 kali.

Gambar 7.6 Perbandingan diagram alir `while..do` (kiri) dan `repeat..until` (kanan). Perhatikan arti cabang `True` yang berlawanan.

Jantung perbedaannya ada pada cabang `True`. Pada `while`, `True` berarti "masuk lagi ke badan". Pada `repeat`, `True` berarti "cukup, keluar". Karena itu syarat pada keduanya biasanya merupakan kebalikan satu sama lain, seperti terlihat pada tabel berikut.

Tabel 7.5 Perbandingan `while..do` dan `repeat..until`

Aspek	`while..do`	`repeat..until`
Posisi syarat	di awal, sebelum badan	di akhir, sesudah badan
Perulangan berlanjut bila	syarat `True`	syarat `False`
Perulangan berhenti bila	syarat `False`	syarat `True`
Jumlah putaran minimal	0	1
Beberapa pernyataan	perlu `begin..end`	tidak perlu

Aspek	`while..do`	`repeat..until`
Contoh yang sama maknanya	`while Nilai <> 0 do`	`until Nilai = 0`
Cocok untuk	data yang mungkin kosong, sentinel	menu, validasi masukan, "coba lagi"

Program 7.4 memperlihatkan dua kegunaan khas `repeat..until`. Yang pertama **memvalidasi masukan**: program terus bertanya sampai nilai berada di antara 0 dan 100. Karena pertanyaan harus diajukan setidaknya sekali, `repeat` lebih tepat daripada `while`. Yang kedua menanyakan "lanjut?" setelah setiap putaran.

Program 7.4 `bab07_04_repeat_until.pas` *Perulangan repeat..until* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 7.4 — Perulangan repeat..until
  repeat menjalankan badannya DULU, baru memeriksa syarat di until.
  Perulangan berhenti saat syarat bernilai True, jadi badannya selalu
  dijalankan paling sedikit sekali. Cocok untuk memvalidasi masukan. }

program bab07_04_repeat_until;

var
  Nilai: Integer;
  Jawab: Char;
  Ulang: Integer;

begin
  { Terus bertanya sampai nilai berada di 0..100 }
  repeat
    Write('Masukkan nilai (0–100): ');
    ReadLn(Nilai);
    if (Nilai < 0) or (Nilai > 100) then
      WriteLn('  Nilai tidak sah, coba lagi.');
  until (Nilai >= 0) and (Nilai <= 100);
  WriteLn('Nilai diterima: ', Nilai);
  WriteLn;

  { repeat..until tidak memerlukan begin..end untuk banyak perintah }
  Ulang := 0;
  repeat
    Ulang := Ulang + 1;
    WriteLn('Putaran ke-', Ulang);
    Write('Lanjut (Y/T)? ');
    ReadLn(Jawab);
  until (Jawab = 'T') or (Jawab = 't');
  WriteLn('Selesai setelah ', Ulang, ' putaran.');
end.
```



Gambar 7.7 Program 7.4: 150 ditolak, 80 diterima; jawab y sekali, lalu t

Masukan pertama, 150, di luar rentang sehingga program menulis pesan galat dan bertanya lagi; 80 lolos. Pada bagian kedua, jawaban `y` membuat putaran berulang, dan jawaban `t` menghentikannya setelah putaran kedua. Bila pengguna langsung menjawab `T`, badan tetap dijalankan sekali, seperti pada run berikut.



Gambar 7.8 Nilai 100 (batas atas, sah) dan langsung menjawab T: badan hanya satu putaran

Baris	Penjelasan
16–21	Badan: tampilkan permintaan, baca `Nilai`, dan bila di luar rentang tulis peringatan. Syarat berhenti di baris 21 adalah rentang yang **sah**: `(Nilai >= 0) and (Nilai <= 100)`. Perhatikan bahwa ini kebalikan dari syarat peringatan di baris 19.
22	Dijalankan hanya setelah nilai sah diperoleh.
26	`Ulang` dinolkan sebelum perulangan, lalu dinaikkan di dalam badan sebagai penghitung putaran.
27–32	Tiga pernyataan di dalam `repeat..until` tanpa `begin..end`. Pada baris 31, `ReadLn(Jawab)` membaca satu karakter.
32	Berhenti bila jawabannya `T` atau `t`. Jawaban lain apa pun, termasuk `Y`, membuat perulangan berlanjut.

> ★ **INGAT**
>
> Menulis syarat `until` adalah menjawab pertanyaan "kapan perulangan ini boleh berhenti?", bukan "kapan ia berlanjut?". Bila Anda mengubah `while` menjadi `repeat`, syaratnya harus dibalik (`not`, atau lawan dari setiap perbandingan menurut hukum De Morgan).

7.6 Perulangan Bersarang

Badan sebuah perulangan boleh berisi perulangan lain. Susunan ini disebut **perulangan bersarang** (*nested loop*). Aturannya sederhana: untuk **setiap satu putaran luar**, perulangan dalam dijalankan **sampai habis**. Jadi bila putaran luar berulang 3 kali dan putaran dalam 4 kali, badan terdalam dijalankan 3 x 4 = 12 kali.

Bayangkan jarum jam: jarum menit berputar penuh 60 kali untuk setiap satu langkah jarum jam. Atau lembar kerja tabel: baris diisi satu per satu (luar), dan di dalam setiap baris, kolom diisi dari kiri ke kanan (dalam). Karena itu perulangan bersarang menjadi alat utama untuk tabel dan pola dua dimensi.



Gambar 7.9 Perulangan bersarang 3 baris x 4 kolom: perulangan dalam (`Kolom`) selesai penuh untuk setiap putaran `Baris`.

Program 7.5 mencetak tabel perkalian 1 sampai 9 dan menampilkan koordinat pada perulangan 3 x 4 persis seperti gambar di atas.

Program 7.5 `bab07_05_perulangan_bersarang.pas` *Perulangan bersarang* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 7.5 — Perulangan bersarang
3    Perulangan di dalam perulangan. Untuk setiap putaran luar, putaran
4    dalam berjalan sampai habis. Pola ini cocok untuk tabel dan baris–
5    kolom. }
6
7  program bab07_05_perulangan_bersarang;
8
9  var
10   Baris, Kolom: Integer;
11
12 begin
```

```
13    WriteLn('TABEL PERKALIAN 1 - 9');
14    Write('   |');
15    for Kolom := 1 to 9 do                { judul kolom }
16      Write(Kolom:4);
17    WriteLn;
18    WriteLn('---+------------------------------------');
19
20    for Baris := 1 to 9 do                { perulangan luar: baris }
21    begin
22      Write(Baris:2, ' |');
23      for Kolom := 1 to 9 do              { perulangan dalam: kolom }
24        Write(Baris * Kolom:4);
25      WriteLn;                            { pindah baris setelah 9 kolom }
26    end;
27    WriteLn;
28
29    { Koordinat (baris, kolom) untuk melihat urutan kerjanya }
30    for Baris := 1 to 3 do
31    begin
32      for Kolom := 1 to 4 do
33        Write('(', Baris, ',', Kolom, ') ');
34      WriteLn;
35    end;
36  end.
```

Odin • User screen

User screen 80×25 Copy screen Editor Stop

```
TABEL PERKALIAN 1 - 9
   |   1   2   3   4   5   6   7   8   9
---+------------------------------------
 1 |   1   2   3   4   5   6   7   8   9
 2 |   2   4   6   8  10  12  14  16  18
 3 |   3   6   9  12  15  18  21  24  27
 4 |   4   8  12  16  20  24  28  32  36
 5 |   5  10  15  20  25  30  35  40  45
 6 |   6  12  18  24  30  36  42  48  54
 7 |   7  14  21  28  35  42  49  56  63
 8 |   8  16  24  32  40  48  56  64  72
 9 |   9  18  27  36  45  54  63  72  81

(1,1) (1,2) (1,3) (1,4)
(2,1) (2,2) (2,3) (2,4)
(3,1) (3,2) (3,3) (3,4)


[Process finished, exit code 0]
```

Gambar 7.10 Program 7.5: tabel perkalian 1 sampai 9 dan koordinat 3 x 4

Baris	Penjelasan
14–17	Judul kolom. `Kolom:4` memberi lebar 4 karakter untuk setiap angka, dan `WriteLn` di baris 17 menutup baris judul.
20–26	Perulangan luar memilih `Baris` dari 1 sampai 9. Badannya memakai `begin..end` karena terdiri dari tiga bagian: judul baris, perulangan dalam, dan `WriteLn`.

Baris	Penjelasan
22	Menulis nomor baris di awal setiap baris tabel, diikuti garis `|`.
23–24	Perulangan dalam: untuk tiap `Kolom`, tulis hasil kali `Baris * Kolom` selebar 4 karakter. Pada tahap ini belum ada pindah baris.
25	`WriteLn` berada di **perulangan luar**, setelah perulangan dalam selesai: kursor pindah baris sekali per 9 kolom. Bila baris ini keliru diletakkan di dalam perulangan dalam, setiap angka akan berada di barisnya sendiri.
30–35	Pasangan koordinat (Baris, Kolom): 3 baris, masing-masing 4 koordinat. Perhatikan, `Kolom` berubah paling cepat, sedangkan `Baris` baru berubah setelah satu baris penuh.

Tabel jejak berikut menunjukkan urutan pekerjaan Program 7.5 pada perulangan koordinat (baris 30–35), yaitu urutan yang sama dengan nomor kecil di **Gambar 7.9**.

Tabel 7.6 Urutan nilai `Baris` dan `Kolom` pada perulangan bersarang 3 x 4

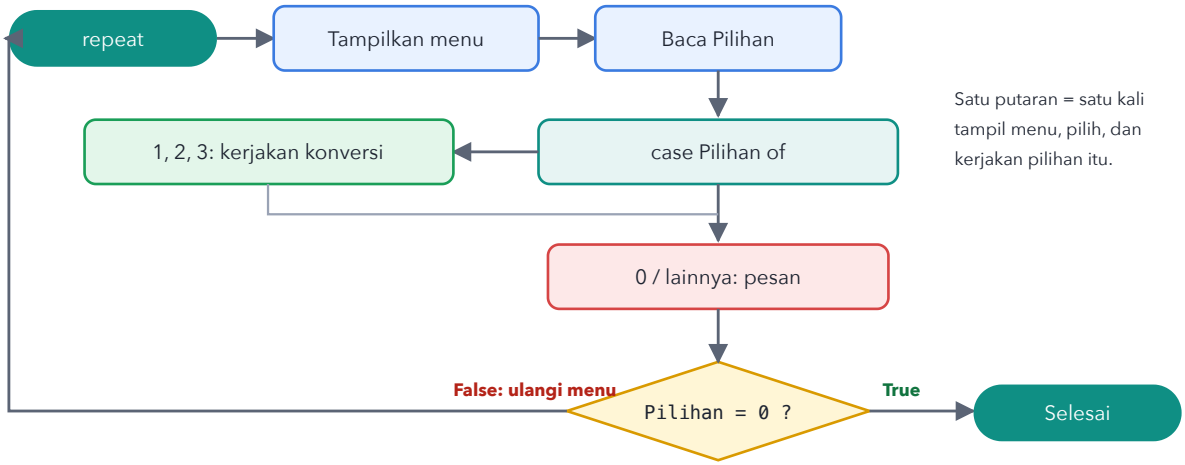
Putaran luar	`Baris`	`Kolom` yang dilalui	Keluaran baris itu
1	1	1, 2, 3, 4	`(1,1) (1,2) (1,3) (1,4)`
2	2	1, 2, 3, 4	`(2,1) (2,2) (2,3) (2,4)`
3	3	1, 2, 3, 4	`(3,1) (3,2) (3,3) (3,4)`

TIPS

Gunakan nama pencacah yang bermakna (`Baris`, `Kolom`) pada perulangan bersarang, bukan `I` dan `J` semata. Dua pencacah **harus** berbeda variabel; bila keduanya memakai variabel yang sama, perulangan dalam akan merusak hitungan perulangan luar.

7.7 Menu yang Berulang

Program interaktif yang Anda pakai sehari-hari tidak langsung selesai setelah satu pekerjaan: ia kembali menawarkan pilihan sampai Anda memilih keluar. Pola ini mudah dibuat dengan menggabungkan dua hal yang sudah Anda kuasai: `repeat..until` untuk mengulang dan `case..of` (Bab 6) untuk memilih pekerjaan. Menu harus tampil setidaknya sekali, jadi `repeat` adalah pilihan yang pas, dan syarat berhenti adalah "pengguna memilih 0".



Gambar 7.11 Struktur menu berulang: `repeat` membungkus tampilan menu dan `case`; `until Pilihan = 0` menjadi pintu keluar.

Program 7.6 membuat menu konversi satuan: Celsius ke Fahrenheit, Fahrenheit ke Celsius, dan kilometer ke mil. Dengan masukan bawaan, Anda akan melihat menu muncul berkali-kali sebelum akhirnya program pamit.

Program 7.6 `bab07_06_menu_berulang.pas` *Latihan: menu berulang* File ▸ Examples…

```
{ Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 7.6 – Latihan: menu berulang
  Gabungan repeat..until dan case: menu tampil terus sampai pengguna
  memilih 0 (keluar). Pola ini dipakai di banyak program interaktif. }

program bab07_06_menu_berulang;

var
  Pilihan: Integer;
  Celsius, Fahrenheit, Kilometer: Real;

begin
  repeat
    WriteLn;
    WriteLn('=== KONVERSI SATUAN ===');
    WriteLn('1. Celsius ke Fahrenheit');
    WriteLn('2. Fahrenheit ke Celsius');
    WriteLn('3. Kilometer ke mil');
    WriteLn('0. Keluar');
    Write('Pilihan: ');
    ReadLn(Pilihan);

    case Pilihan of
      1: begin
           Write('Suhu Celsius    : '); ReadLn(Celsius);
           WriteLn('Hasil          : ', Celsius * 9 / 5 + 32:0:1, ' F');
         end;
      2: begin
           Write('Suhu Fahrenheit: '); ReadLn(Fahrenheit);
           WriteLn('Hasil          : ', (Fahrenheit - 32) * 5 / 9:0:1, ' C');
         end;
      3: begin
```

```
33            Write('Jarak (km)     : '); ReadLn(Kilometer);
34            WriteLn('Hasil          : ', Kilometer * 0.621371:0:2, ' mil');
35          end;
36        0: WriteLn('Terima kasih!');
37      else
38        WriteLn('Pilihan tidak tersedia.');
39      end;
40    until Pilihan = 0;                    { keluar saat memilih 0 }
41  end.
```

Odin • User screen

User screen 80×25 | Copy screen | Editor | Stop

```
=== KONVERSI SATUAN ===
1. Celsius ke Fahrenheit
2. Fahrenheit ke Celsius
3. Kilometer ke mil
0. Keluar
Pilihan: 1
Suhu Celsius   : 100
Hasil          : 212.0 F

=== KONVERSI SATUAN ===
1. Celsius ke Fahrenheit
2. Fahrenheit ke Celsius
3. Kilometer ke mil
0. Keluar
Pilihan: 3
Jarak (km)     : 42.195
Hasil          : 26.22 mil

=== KONVERSI SATUAN ===
1. Celsius ke Fahrenheit
2. Fahrenheit ke Celsius
3. Kilometer ke mil
0. Keluar
Pilihan: 9
Pilihan tidak tersedia.

=== KONVERSI SATUAN ===
1. Celsius ke Fahrenheit
2. Fahrenheit ke Celsius
3. Kilometer ke mil
0. Keluar
Pilihan: 0
Terima kasih!


[Process finished, exit code 0]
```

Gambar 7.12 Program 7.6: tiga konversi, satu pilihan yang tidak ada, lalu keluar dengan 0

Baris	Penjelasan
13–21	Awal `repeat`: baris kosong, judul, daftar pilihan, dan permintaan. `ReadLn(Pilihan)` memilih jalur untuk putaran ini.
23–24	`case Pilihan of`; cabang 1 berupa `begin..end` yang membaca suhu Celsius dan menghitung `Celsius * 9 / 5 + 32`.

Baris	Penjelasan
26	`:0:1` memformat bilangan `Real` tanpa lebar tambahan dan dengan satu desimal.
28–35	Cabang 2 dan 3 memakai pola yang sama: baca nilai, hitung, tulis hasil. Cabang 3 memakai dua desimal.
36	Cabang 0 hanya menulis salam perpisahan. Perulangan tidak berhenti di sini, melainkan di baris 40.
37–38	`else` pada `case` menampung semua pilihan lain, misalnya 9, dan menulis pesan galat. Menu lalu tampil lagi.
40	`until Pilihan = 0`. Sesudah cabang 0 selesai, syarat ini `True` dan program keluar dari perulangan.

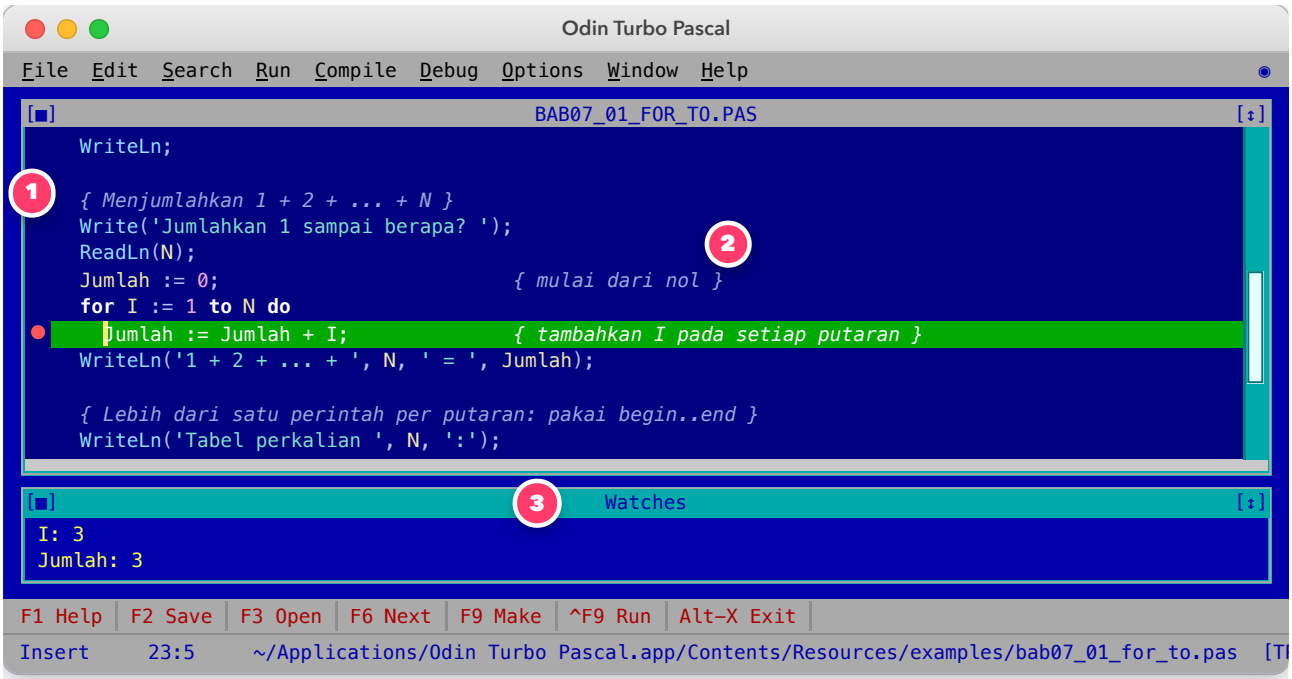
7 **ATURAN TURBO PASCAL 7**

Karena `Write` tidak bisa mencetak bilangan `Real` tanpa format yang jelas, setiap `Real` pada program ini diberi `:lebar:desimal`. Tanpa format, Turbo Pascal 7 menampilkan notasi ilmiah yang sulit dibaca (misalnya `2.1200000000E+02`).

7.8 Melacak Perulangan dengan Debugger

Tabel jejak yang Anda buat dengan tangan, bisa dikerjakan otomatis oleh debugger Odin. Dengan debugger Anda menghentikan program di tengah perulangan, melihat nilai variabel pada saat itu, dan melangkah satu baris demi satu baris. Ini cara paling andal untuk memahami perulangan yang hasilnya tidak sesuai dugaan. Debugger ada di menu `Debug`.

1. **Pasang breakpoint.** Pindahkan kursor ke baris 23 pada Program 7.1, yaitu `Jumlah := Jumlah + I`, lalu pilih `Debug ▸ Toggle breakpoint` atau tekan Ctrl+F8. Titik merah muncul di gutter (lajur di kiri editor).
2. **Tambahkan watch.** Pilih `Debug ▸ Add watch...`, lalu ketik `I`; ulangi dengan `Jumlah`. Keduanya tampil di jendela Watches.
3. **Mulai.** Pilih `Debug ▸ Start debugging`, lalu ketik masukan di User screen. Program berhenti di breakpoint: baris yang akan dijalankan ditandai hijau.
4. **Melangkah.** Tekan F8 (`Step over`) untuk satu baris, atau Ctrl+F9 (`Continue`) untuk lanjut ke breakpoint berikutnya, yaitu putaran berikutnya. Setiap kali program berhenti, nilai di Watches diperbarui.



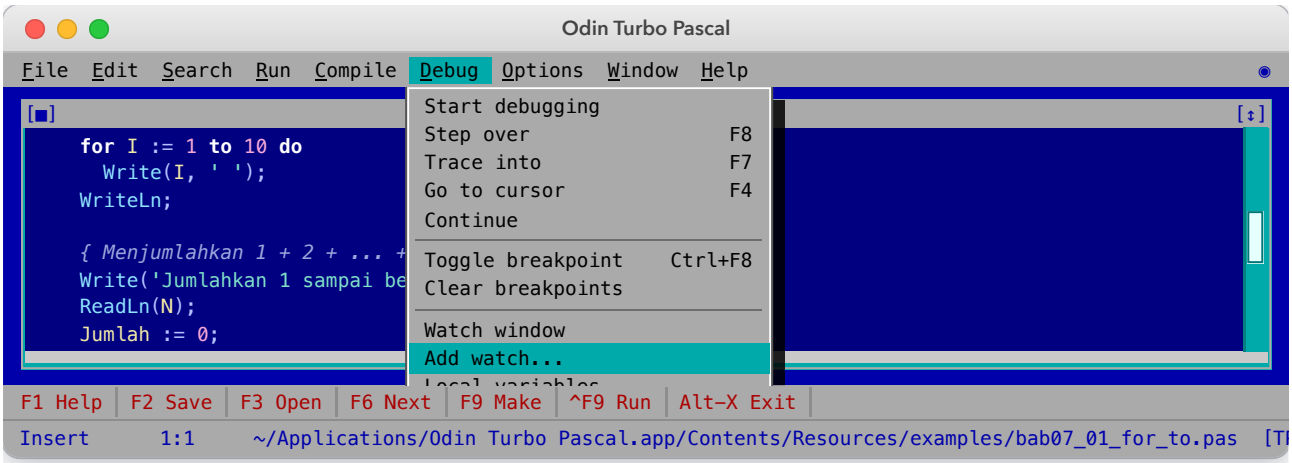
Gambar 7.13 Program 7.1 berhenti di breakpoint baris 23 pada putaran ketiga; Watches menunjukkan I = 3 dan Jumlah = 3

Nomor	Penjelasan
1	Gutter: titik merah adalah breakpoint pada baris 23.
2	Jendela editor. Baris hijau di dalamnya menandai baris berikutnya yang akan dijalankan. Karena program berhenti sebelum baris itu dikerjakan, `Jumlah` belum memuat `I` yang sekarang.
3	Jendela Watches: nilai `I` dan `Jumlah` pada saat berhenti.

Pada gambar tersebut program berhenti pada putaran ketiga: dua putaran sebelumnya sudah menambahkan 1 dan 2, jadi `Jumlah` adalah 3, sedangkan `I` bernilai 3 dan belum ditambahkan. Tekan `Continue`, dan Watches akan berubah mengikuti tabel jejak yang dibuat sebelumnya.

Tabel 7.7 Isi jendela Watches setiap kali Program 7.1 berhenti di baris 23 (`N = 5`)

Berhenti ke-	`I`	`Jumlah`	Sesudah `Continue`
1	1	0	menjadi 1
2	2	1	menjadi 3
3	3	3	menjadi 6
4	4	6	menjadi 10
5	5	10	menjadi 15, lalu perulangan selesai



Gambar 7.14 Menu Debug: Add watch... menambahkan variabel yang ingin diamati

> **INFO**
>
> Gunakan debugger saat perulangan Anda "tidak berhenti" atau berhenti di putaran yang salah. Pasang breakpoint di dalam badan, amati satu atau dua putaran, dan nilai yang tidak sesuai dugaan biasanya langsung terlihat. Untuk berhenti paksa, tekan Ctrl + F2.

7.9 Kesalahan yang Sering Terjadi

7.9.1 Lupa kata do

Setiap `for` dan `while` harus diakhiri `do` sebelum badannya. Kata ini mudah terlewat karena pada bahasa lain tidak ada padanannya.

lupa_do.pas

```
1  program LupaDo;
2  var
3    I: Integer;
4  begin
5    for I := 1 to 3
6      WriteLn(I);
7  end.
```

PESAN KOMPILER

```
lupa_do.pas(6,5) Error: DO diharapkan, tetapi ditemukan identifier WRITELN
```

Gambar 7.15 Kata do tertinggal pada perulangan for

Kompiler melaporkan pada baris badan perulangan, karena di sanalah ia menemukan kata yang tidak diharapkan. Tambahkan `do` di akhir baris `for`.

7.9.2 Pencacah bertipe Real

real.pas

```
1  program Pecahan;
```

```
var
  X: Real;
begin
  for X := 1 to 3 do
    WriteLn(X:0:1);
end.
```

PESAN KOMPILER

```
real.pas(5,7) Error: Variabel kontrol FOR tidak valid
```

Gambar 7.16 Pencacah for harus bertipe ordinal

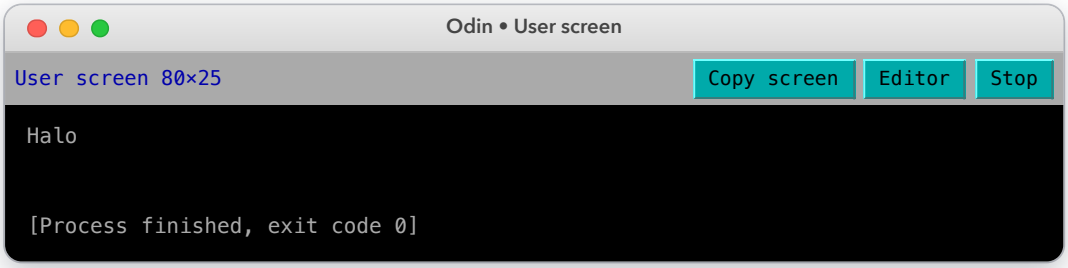
Bilangan `Real` tidak punya "bilangan berikutnya" yang pasti, jadi tidak bisa menjadi pencacah. Pakai pencacah `Integer` dan hitung nilai `Real` dari pencacah itu, misalnya `X := I / 2`.

7.9.3 Titik koma tepat sesudah do

Seperti titik koma sesudah `then` (Bab 6), titik koma sesudah `do` membuat badan perulangan menjadi **pernyataan kosong**. Kompiler tidak mengeluh, tetapi perintah di bawahnya menjadi di luar perulangan.

titik_koma.pas

```
program TitikKoma;
var
  I: Integer;
begin
  for I := 1 to 3 do;
    WriteLn('Halo');
end.
```



Gambar 7.17 Baris Halo hanya tampil sekali, bukan tiga kali, karena ada titik koma sesudah do

Perulangan tetap berjalan tiga kali, tetapi yang diulang adalah pernyataan kosong. Setelah itu, `WriteLn` dijalankan sekali sebagai pernyataan biasa. Hapus titik koma setelah `do`.

7.9.4 Perulangan yang tidak pernah berhenti

Kesalahan paling klasik pada `while` adalah lupa menulis "perubahan". Perhatikan potongan ini:

```
Sisa := Angka;
while Sisa > 0 do
  JumlahDigit := JumlahDigit + Sisa mod 10;   { Sisa tidak pernah berubah! }
  Sisa := Sisa div 10;                         { di luar while, baris ini terlambat }
```

Tanpa `begin..end`, badan `while` hanya baris pertama. `Sisa` tidak pernah berubah di dalam perulangan, sehingga syarat `Sisa > 0` selalu `True` dan program terus menjumlahkan tanpa henti. Tanda-tandanya: User screen tidak bereaksi dan kursor tidak kembali. Tekan Ctrl + F2 untuk menghentikan program, lalu periksa dua hal: apakah badannya dibungkus `begin..end`, dan apakah ada perintah yang benar-benar membawa syarat menuju `False`.

7.9.5 Satu putaran terlalu banyak atau terlalu sedikit

Kesalahan *off-by-one* terjadi ketika batas perulangan meleset satu angka, misalnya memakai `<` padahal seharusnya `<=`, atau `for I := 0 to N` padahal seharusnya `1 to N`. Cara memeriksanya: ujilah dengan nilai terkecil (misalnya `N = 1` atau `N = 0`) dan hitung dengan tangan berapa kali badan seharusnya berjalan.

7.10 Rangkuman

- Perulangan mengulang sekelompok perintah. Setiap perulangan punya nilai awal, syarat, badan, dan perubahan yang menuju berhentinya perulangan.
- `for..to` menghitung naik dan `for..downto` menghitung turun, satu angka tiap putaran. Pencacah harus bertipe ordinal (`Integer`, `Char`, dan sejenisnya), tidak boleh diubah di dalam badan, dan badan tidak berjalan bila nilai awal sudah melewati nilai akhir.
- `while..do` memeriksa syarat sebelum badan dan berlanjut selama syarat `True`; badannya bisa nol kali. Programmer wajib memastikan ada perintah yang membuat syarat akhirnya `False`.
- `repeat..until` menjalankan badan dulu, memeriksa syarat sesudahnya, dan berhenti saat syarat `True`. Badannya minimal sekali dan tidak memerlukan `begin..end`.
- Perulangan bersarang menjalankan perulangan dalam sampai habis untuk setiap putaran luar; jumlah putaran terdalam adalah hasil kali jumlah putaran masing-masing tingkat.
- Pola baku: akumulator (`Jumlah := Jumlah + I`), nilai sentinel dengan *priming read*, validasi dengan `repeat`, dan menu berulang dengan `repeat + case`.
- Tabel jejak dan debugger (breakpoint, baris eksekusi, Watches) memperlihatkan nilai variabel pada tiap putaran dan menjadi cara terbaik untuk menemukan kesalahan perulangan.

7.11 Latihan

1. **DASAR** Tulis program yang membaca `N`, lalu mencetak `N` baris berisi tulisan "Belajar Pascal itu menyenangkan". Gunakan `for..to`.
2. **DASAR** Ubah Program 7.1 agar menjumlahkan hanya bilangan genap dari 1 sampai `N`. Cobalah dengan dua cara: memakai `if I mod 2 = 0` dan memakai `I * 2` pada perulangan yang berjalan setengah `N` kali.
3. **DASAR** Buat tabel jejak, dengan tangan, untuk potongan `S := 1; for I := 1 to 4 do S := S * I;`. Berapa nilai akhir `S`? Bandingkan hasilnya dengan

jalannya program di debugger.

4. **MENENGAH** Tulis program yang membaca bilangan terus-menerus sampai 0, lalu menampilkan banyak bilangan, jumlah, rata-rata (dua desimal), serta nilai terbesar. Tangani kasus ketika pengguna langsung mengetik 0.

5. **MENENGAH** Tulis ulang bagian validasi Program 7.4 dengan `while..do`. Syarat apa yang harus Anda pakai, dan apa yang perlu Anda tambahkan sebelum perulangan?

6. **MENENGAH** Buat program yang mencetak papan "dadu" dua dadu: semua pasangan (D1, D2) untuk D1 dan D2 dari 1 sampai 6, enam pasangan per baris, beserta jumlah mata di sebelah tiap pasangan. Berapa kali badan terdalam dijalankan?

7. **TANTANGAN** Tambahkan satu pilihan ke Program 7.6: "4. Kilogram ke pon" (1 kg = 2,20462 pon). Setelah itu ubah program agar menghitung berapa kali konversi berhasil dilakukan dan menampilkannya saat pengguna keluar.

BAB 08

BAGIAN II · ALUR KENDALI

Berlatih Algoritma dengan Perulangan

1 2 3 4 5 6 7 **8** 9 10 11 12 13 14 15 16 17 18 19 20

TUJUAN PEMBELAJARAN

- Memecah masalah menjadi langkah-langkah algoritma, lalu menerjemahkannya menjadi perulangan Pascal.
- Menguji bilangan prima dengan pembagi sampai akar bilangan, dan mencetak semua prima sampai 100.
- Menghitung FPB dengan algoritma Euclid dan KPK dari FPB.
- Membangkitkan deret Fibonacci dengan dua variabel yang bergeser, serta mengenali batas tipe `LongInt`.
- Menggambar pola bintang dengan perulangan bersarang dengan mencari rumus baris dan kolom.
- Mencari bilangan sempurna dan memahami permainan tebak angka beserta strategi pencarian biner.

PROGRAM DALAM BAB INI

8.1	`bab08_01_bilangan_prima.pas`	Bilangan prima
8.2	`bab08_02_fpb_kpk.pas`	FPB dan KPK
8.3	`bab08_03_deret_fibonacci.pas`	Deret Fibonacci
8.4	`bab08_04_segitiga_bintang.pas`	Segitiga bintang
8.5	`bab08_05_bilangan_sempurna.pas`	Bilangan sempurna
8.6	`bab08_06_tebak_angka.pas`	Permainan tebak angka

Mengetahui sintaks perulangan belum sama dengan bisa memecahkan masalah. Di bab ini Anda berlatih bagian yang lebih menantang: **algoritma**, yaitu urutan langkah yang jelas untuk menyelesaikan suatu masalah. Enam program klasik akan kita bedah satu per satu: bilangan prima, FPB dan KPK, deret Fibonacci, segitiga bintang, bilangan sempurna, dan permainan tebak angka. Semuanya hanya memakai perulangan dan percabangan dari bab-bab sebelumnya, tetapi cara berpikirnya akan melatih Anda untuk menyusun solusi sendiri.

8.1 Dari Masalah ke Algoritma

Menulis program dimulai dari *berpikir*, bukan dari mengetik. Sebelum menyentuh `for` atau `while`, Anda perlu menjawab: apa masukannya, apa keluaran yang diinginkan, dan langkah apa yang membawa dari yang pertama ke yang kedua. Daftar langkah itulah algoritma. Algoritma yang baik berhingga (pasti berakhir), jelas pada setiap langkahnya, dan benar untuk semua masukan yang sah, termasuk kasus batas.



Keempat langkah ini akan terlihat berulang di setiap program bab ini. Perhatikan juga bahwa hampir setiap program menggunakan satu dari empat "pola" yang sudah Anda kenal: **akumulator** (menjumlahkan), **penghitung** (menambah satu), **penanda** (variabel Boolean seperti `Prima`), dan **pembagi bersarang** (perulangan di dalam perulangan).

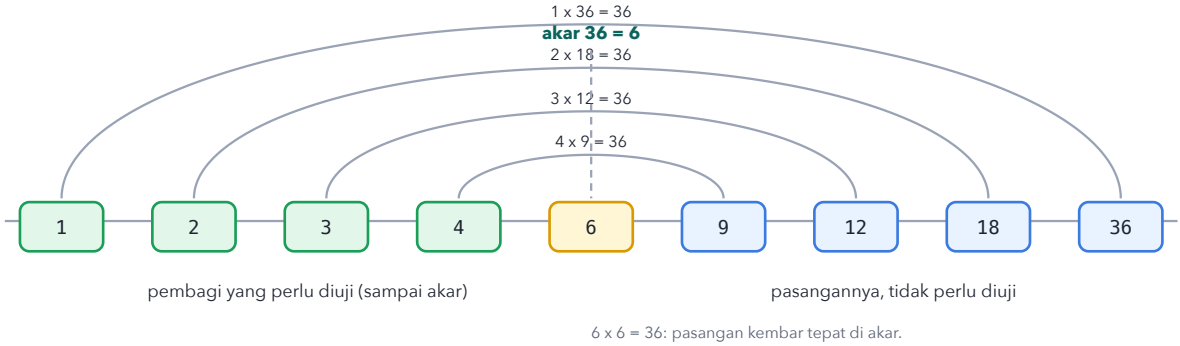
> **INFO**
>
> Seluruh program di bab ini sudah tersedia di `File ▸ Examples...` dengan nama `bab08_01` sampai `bab08_06`. Anda dianjurkan membuka satu per satu, menjalankannya, lalu mencoba kasus lain sebelum membaca penjelasannya.

8.2 Bilangan Prima

Bilangan prima adalah bilangan bulat lebih besar dari 1 yang hanya habis dibagi 1 dan dirinya sendiri: 2, 3, 5, 7, 11, 13, dan seterusnya. Angka 1 bukan prima, dan 2 adalah satu-satunya prima genap. Bagaimana memeriksa apakah `N` prima? Idenya mudah: coba bagi `N` dengan 2, 3, 4, dan seterusnya. Bila ada yang membagi habis (`N mod Pembagi = 0`), `N` bukan prima.

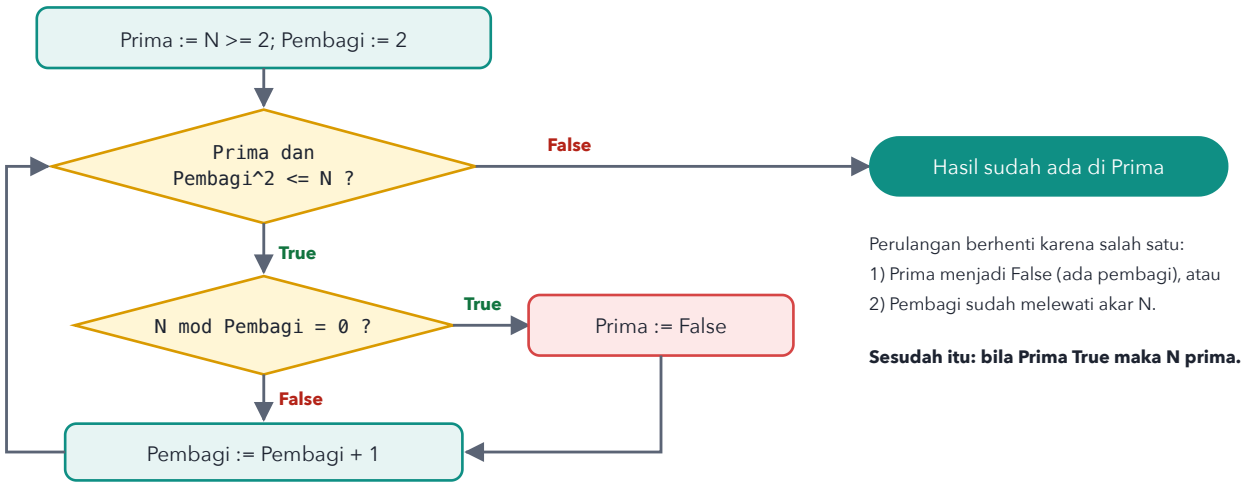
Pertanyaan berikutnya: sampai pembagi berapa kita harus mencoba? Mencoba sampai `N - 1` benar, tetapi boros. Perhatikan pasangan pembagi `N = 36` pada gambar berikut. Setiap pembagi di sisi kiri akar kuadrat (6) punya pasangan di sisi kanan. Bila `N = A x B`, tidak mungkin *keduanya* lebih besar dari akar

`N`, sebab hasil kalinya akan melebihi `N`. Jadi bila `N` punya pembagi, pasti ada satu yang tidak lebih dari akar `N`. Cukup uji pembagi sampai `Pembagi * Pembagi <= N`, dan itu menghindari perhitungan akar kuadrat.



Gambar 8.1 Pembagi 36 berpasangan di sekitar akar kuadrat 6. Cukup memeriksa pembagi sampai 6: pasangannya otomatis ikut ketemu.

Dengan penanda Boolean `Prima`, algoritmanya menjadi: anggap `N` prima (bila `N >= 2`), lalu uji pembagi 2, 3, 4, ... selama masih ada kemungkinan dan `Prima` masih `True`. Sekali ketemu pembagi, `Prima` diubah menjadi `False` dan perulangan berhenti karena syarat `while` tidak terpenuhi lagi.



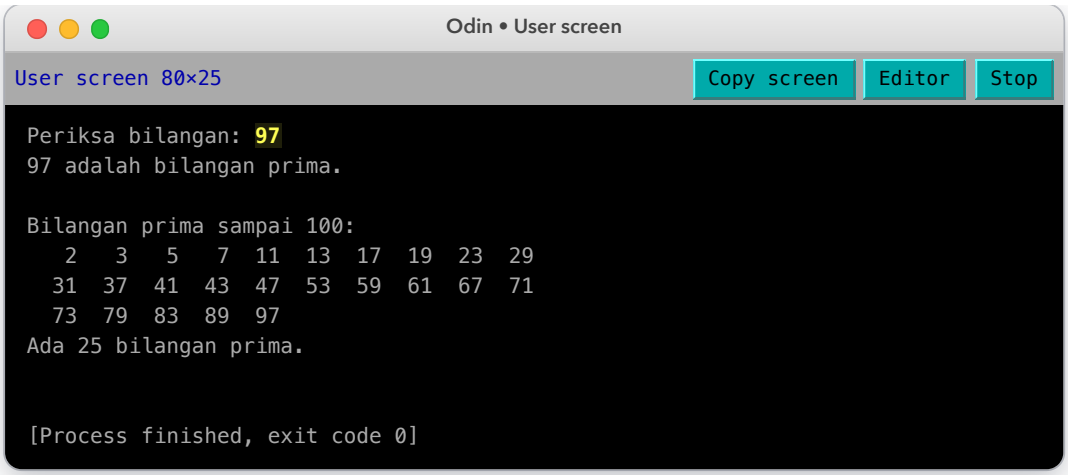
Gambar 8.2 Diagram alir pemeriksaan bilangan prima pada Program 8.1: dua syarat berhenti dan satu penanda hasil.

Program 8.1 menerapkan algoritma ini dua kali: sekali untuk bilangan yang Anda ketik, lalu untuk setiap bilangan 2 sampai 100 di dalam perulangan `for`, sehingga terbentuk daftar semua prima sampai 100.

Program 8.1 `bab08_01_bilangan_prima.pas` *Bilangan prima* File ▸ Examples…

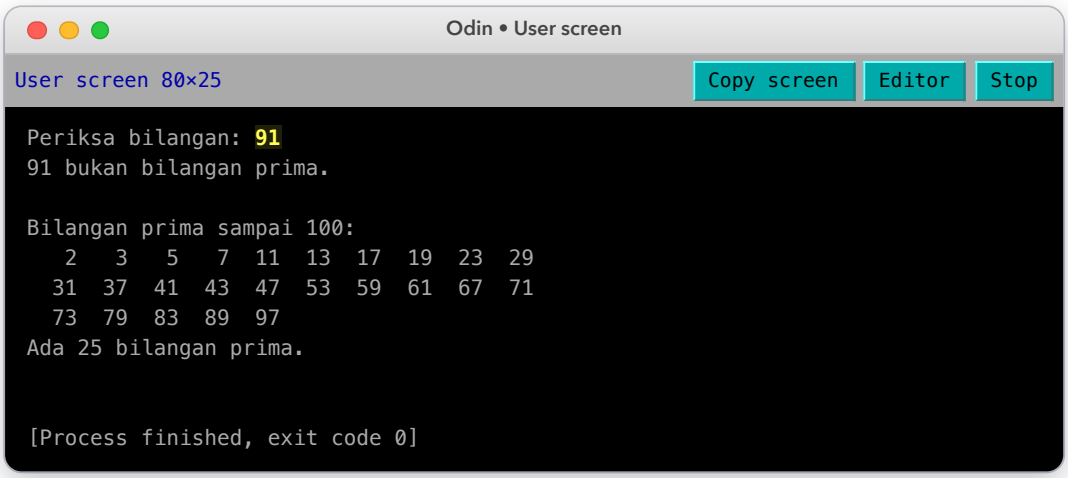
```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 8.1 — Bilangan prima
3    Bilangan prima hanya habis dibagi 1 dan dirinya sendiri (2, 3, 5,
4    7, 11, ...). Cukup periksa pembagi sampai akar bilangan itu: bila
5    N = A x B, salah satu dari A atau B pasti tidak lebih dari akar N. }
6
```

```
7  program bab08_01_bilangan_prima;
8
9  var
10   N, Calon, Pembagi, Banyak: Integer;
11   Prima: Boolean;
12
13 begin
14   Write('Periksa bilangan: ');
15   ReadLn(N);
16   Prima := N >= 2;                      { 0 dan 1 bukan prima }
17   Pembagi := 2;
18   while Prima and (Pembagi * Pembagi <= N) do
19   begin
20     if N mod Pembagi = 0 then
21       Prima := False;                   { ketemu pembagi: bukan prima }
22     Pembagi := Pembagi + 1;
23   end;
24   if Prima then
25     WriteLn(N, ' adalah bilangan prima.')
26   else
27     WriteLn(N, ' bukan bilangan prima.');
28   WriteLn;
29
30   { Cara yang sama, diulang untuk setiap bilangan 2..100 }
31   WriteLn('Bilangan prima sampai 100:');
32   Banyak := 0;
33   for Calon := 2 to 100 do
34   begin
35     Prima := True;
36     Pembagi := 2;
37     while Prima and (Pembagi * Pembagi <= Calon) do
38     begin
39       if Calon mod Pembagi = 0 then
40         Prima := False;
41       Pembagi := Pembagi + 1;
42     end;
43     if Prima then
44     begin
45       Write(Calon:4);
46       Banyak := Banyak + 1;
47       if Banyak mod 10 = 0 then WriteLn;   { 10 bilangan per baris }
48     end;
49   end;
50   WriteLn;
51   WriteLn('Ada ', Banyak, ' bilangan prima.');
52 end.
```



Gambar 8.3 Program 8.1 dengan masukan 97 (prima) dan daftar prima sampai 100

Sekarang coba bilangan komposit. Untuk 91, tidak ada pembagi kecil yang jelas (91 bukan genap dan jumlah digitnya 10), tetapi 7 x 13 = 91, jadi program harus menemukannya.



Gambar 8.4 Masukan 91: bukan prima karena habis dibagi 7

Tabel 8.1 Jejak pemeriksaan `N = 91` pada baris 18–23

`Pembagi`	`Pembagi * Pembagi`	`91 mod Pembagi`	`Prima` sesudah
2	4	1	`True`
3	9	1	`True`
4	16	3	`True`
5	25	1	`True`
6	36	1	`True`
7	49	0	`False`, perulangan berhenti

Baris	Penjelasan
10–11	`Calon` dan `Pembagi` bertipe `Integer`; `Prima` bertipe `Boolean` sebagai penanda hasil.
16	`Prima := N >= 2` memberi nilai awal sekaligus menangani 0 dan 1 (bukan prima): bila `N` kurang dari 2, `Prima` langsung `False` dan perulangan tidak dijalankan.

Baris	Penjelasan
18	Syarat `while` gabungan: `Prima` masih `True` dan `Pembagi * Pembagi <= N`. Kurung wajib di sekitar perbandingan karena `and` lebih kuat daripada operator pembanding (Bab 6).
20–21	Bila `Pembagi` membagi habis, `N` pasti bukan prima.
22	Pembagi berikutnya. Baris ini selalu dijalankan sehingga perulangan pasti berakhir (`Pembagi * Pembagi` akhirnya melewati `N`).
24–27	Hasil dilaporkan dengan `if..then..else`. Perhatikan: tidak ada titik koma sebelum `else`.
32	`Banyak` menghitung prima yang ditemukan, untuk dicetak sepuluh per baris.
33–49	Perulangan `for` terluar mencoba setiap `Calon` dari 2 sampai 100. Di dalamnya, baris 35–42 mengulang algoritma yang sama dengan `Calon` sebagai pengganti `N`. Penanda `Prima` harus diatur ulang ke `True` (baris 35) di setiap putaran luar.
43–48	Bila `Calon` prima: cetak selebar 4 karakter, tambah `Banyak`, dan setiap kelipatan 10 pindah baris.
51	Mencetak jumlah akhir, yaitu 25 bilangan prima sampai 100.

COBA SENDIRI

Ubah batas pada baris 33 menjadi 1000 dan jalankan. Berapa bilangan prima di bawah 1000? Tebak dulu, lalu bandingkan dengan keluaran program. Perhatikan juga bahwa tipe `Integer` masih aman karena hasil kali `Pembagi * Pembagi` tidak melewati 32767.

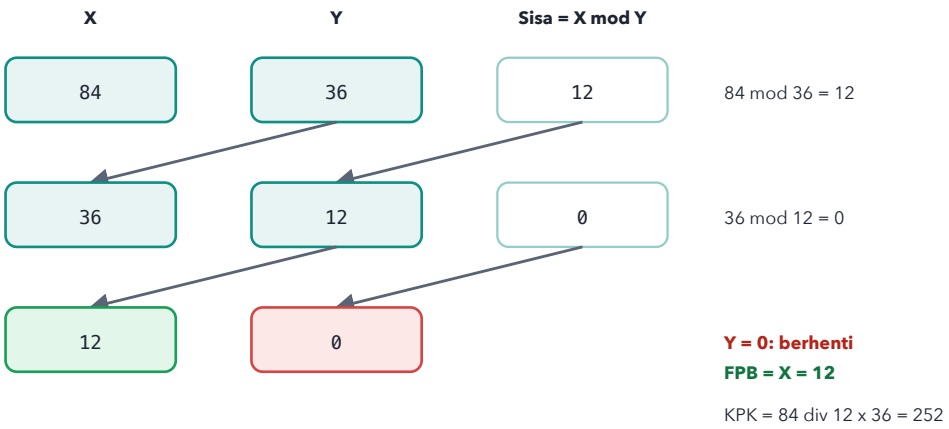
8.3 FPB dan KPK: Algoritma Euclid

Faktor Persekutuan Terbesar (FPB) dua bilangan adalah bilangan terbesar yang membagi habis keduanya; FPB(84, 36) adalah 12. **Kelipatan Persekutuan Terkecil (KPK)** adalah bilangan terkecil yang habis dibagi keduanya. Cara di sekolah memakai pohon faktor, tetapi lebih dari dua ribu tahun lalu Euclid menemukan cara yang jauh lebih singkat, dan cara itu masih dipakai komputer sampai sekarang.

Idenya: FPB dari `A` dan `B` sama dengan FPB dari `B` dan `A mod B`. Ganti terus pasangan (A, B) dengan (B, sisa), dan bilangan-bilangan itu mengecil. Ketika yang kedua menjadi 0, yang pertama adalah FPB.

BENTUK UMUM

```
selama B <> 0 lakukan:
    Sisa := A mod B
    A := B
    B := Sisa
FPB adalah A
```



Gambar 8.5 Algoritma Euclid untuk 84 dan 36: pada setiap baris, `Y` turun menjadi `X` berikutnya, dan `Sisa` turun menjadi `Y` berikutnya.

Setelah FPB diketahui, KPK mudah dihitung: KPK = (A x B) / FPB. Program 8.2 membagi lebih dulu (`A div FPB * B`) agar hasil antara tidak meluap, sebab A x B bisa jauh lebih besar dari A atau B. Program ini juga menampilkan setiap langkah Euclid supaya prosesnya terlihat.

Program 8.2 `bab08_02_fpb_kpk.pas` *FPB dan KPK* File ▸ Examples…

```
{ Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 8.2 – FPB dan KPK
  FPB (faktor persekutuan terbesar) dicari dengan algoritma Euclid:
  ganti (A, B) menjadi (B, A mod B) sampai B bernilai 0; A adalah FPB.
  KPK (kelipatan persekutuan terkecil) = A x B / FPB. }

program bab08_02_fpb_kpk;

var
  A, B, X, Y, Sisa, FPB, KPK: LongInt;

begin
  Write('Bilangan pertama: '); ReadLn(A);
  Write('Bilangan kedua  : '); ReadLn(B);

  X := A;                              { salin, agar A dan B tetap utuh }
  Y := B;
  WriteLn('Langkah algoritma Euclid:');
  while Y <> 0 do
  begin
    Sisa := X mod Y;
    WriteLn('  ', X, ' mod ', Y, ' = ', Sisa);
    X := Y;
    Y := Sisa;
  end;
  FPB := X;

  KPK := A div FPB * B;                { bagi dulu agar tidak meluap }
  WriteLn('FPB(', A, ', ', B, ') = ', FPB);
  WriteLn('KPK(', A, ', ', B, ') = ', KPK);
end.
```



Gambar 8.6 Program 8.2 untuk 84 dan 36

Bagaimana bila bilangan pertama lebih kecil? Euclid tetap bekerja tanpa penanganan khusus, karena langkah pertama menukar posisi keduanya: `36 mod 84 = 36`, sehingga pasangan menjadi (84, 36).



Gambar 8.7 Masukan dibalik (36 dan 84): langkah pertama menukar posisi

Baris	Penjelasan
10	Semua variabel bertipe `LongInt`, agar KPK dua bilangan besar masih muat.
13–14	Dua bilangan dibaca; dua pernyataan ditulis di satu baris, dipisah titik koma.
16–17	Salinan `X` dan `Y` dibuat karena algoritma merusak nilainya; `A` dan `B` masih diperlukan untuk menghitung KPK dan untuk tampilan akhir.
19–25	Perulangan `while Y <> 0`. Sisa dihitung (baris 21), ditulis agar langkahnya terlihat (22), lalu pasangan digeser (23–24). Urutan baris 23 dan 24 penting: `Sisa` sudah menyimpan hasil sebelum `Y` ditimpa.
26	Saat perulangan selesai, `Y = 0` dan `X` adalah FPB.
28	KPK dihitung dengan membagi lebih dulu. Hasilnya sama dengan A x B / FPB karena FPB pasti membagi habis `A`.

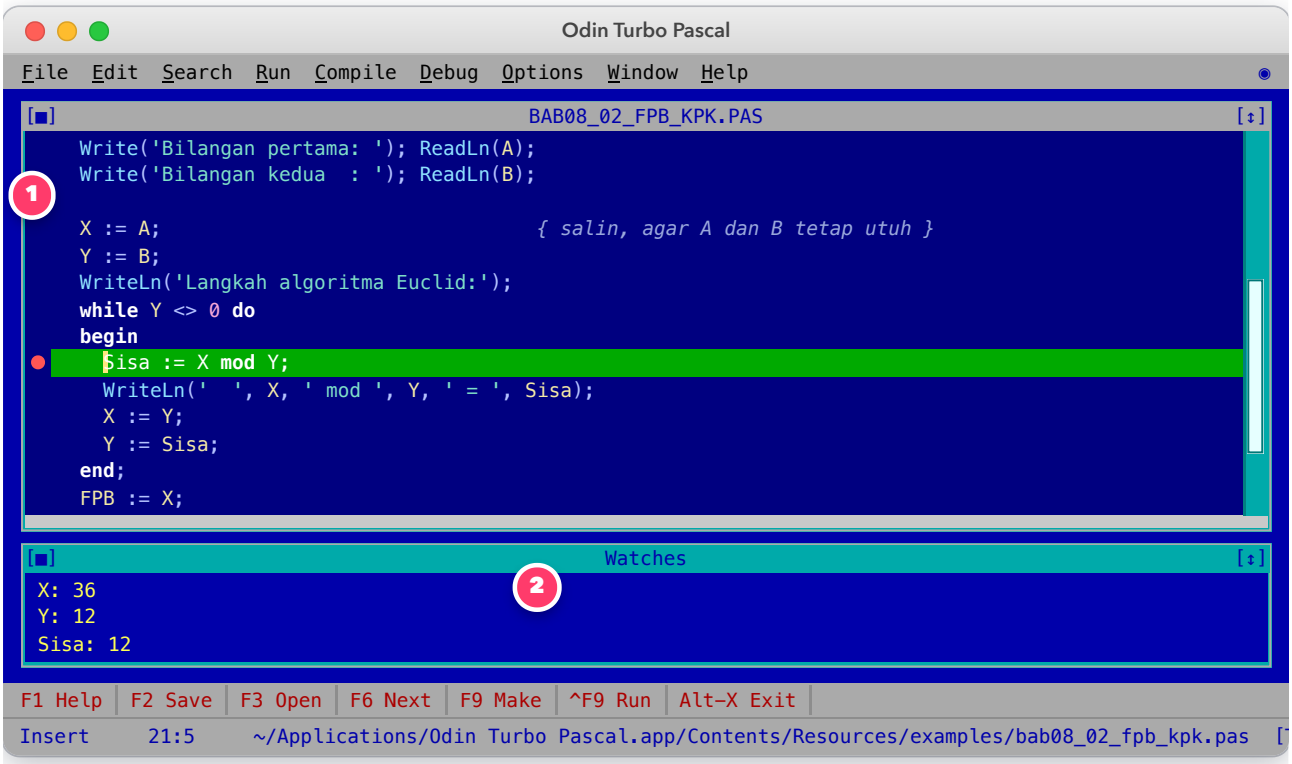
Perhatikan juga bahwa perulangan berhenti pasti terjadi: pada setiap putaran, sisa selalu lebih kecil dari pembagi, sehingga nilai `Y` menyusut sampai 0. Berikut jejak untuk 84 dan 36 sesuai gambar tadi.

Tabel 8.2 Jejak Program 8.2 untuk 84 dan 36

Putaran	X	Y	Sisa := X mod Y	Sesudah geser (X, Y)
1	84	36	12	(36, 12)
2	36	12	0	(12, 0)
(cek)	12	0	berhenti, FPB = 12, KPK = 84 div 12 * 36 = 252	

8.3.1 Melihat Euclid di debugger

Pasang breakpoint di baris 21 (`Sisa := X mod Y`), tambahkan watch untuk `X`, `Y`, dan `Sisa`, lalu mulai debugging dengan masukan 84 dan 36. Setiap kali program berhenti di baris 21, tekan `Debug ▸ Continue` untuk maju satu putaran. Pada gambar di bawah, program berhenti di putaran kedua: `X` dan `Y` sudah bergeser, tetapi `Sisa` masih menyimpan nilai putaran pertama.



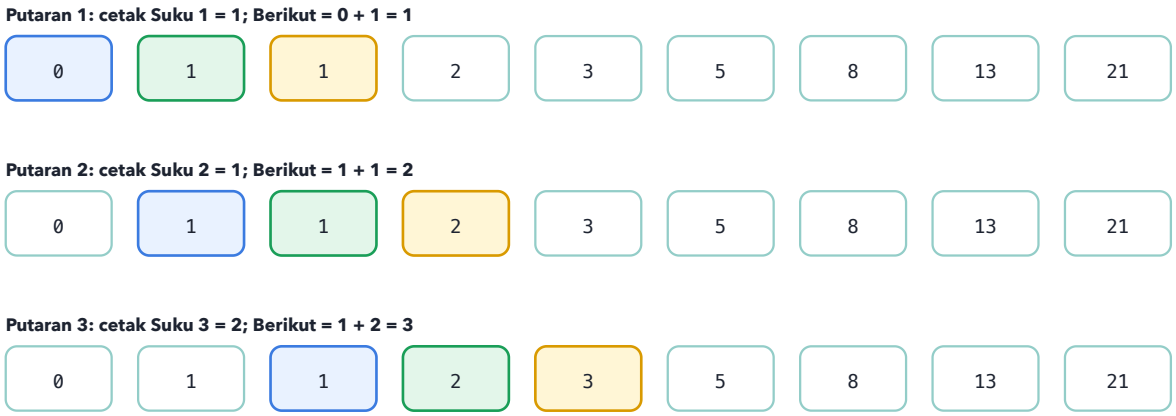
Gambar 8.8 Program 8.2 berhenti di baris 21 pada putaran kedua: X = 36, Y = 12

Nomor	Penjelasan
1	Gutter dengan breakpoint pada baris 21 (titik merah).
2	Jendela Watches: pada saat berhenti, `X = 36` dan `Y = 12`, sama dengan baris pertama tabel jejak sesudah pergeseran.

8.4 Deret Fibonacci

Deret Fibonacci dimulai dari 1, 1, lalu setiap suku adalah jumlah dua suku sebelumnya: 1, 1, 2, 3, 5, 8, 13, 21, 34, … Deret ini muncul pada banyak hal, mulai dari susunan biji bunga matahari sampai soal pertumbuhan populasi. Untuk membangkitkannya kita tidak perlu menyimpan seluruh deret; cukup **dua**

suku terakhir. Setiap putaran, kita hitung suku berikutnya, lalu "menggeser" kedua variabel satu langkah ke depan.



Setelah setiap putaran, ketiga penanda bergeser satu kotak ke kanan.

Gambar 8.9 Tiga putaran pertama Program 8.3: tiga variabel bergeser sepanjang deret. Biru = `Sebelum`, hijau = `Sekarang`, kuning = `Berikut`.

Urutan tiga baris pergeseran sangat penting: `Berikut` dihitung dari nilai lama, `Sebelum` diisi `Sekarang`, baru `Sekarang` diisi `Berikut`. Kalau urutannya terbalik, nilai lama sudah tertimpa sebelum dipakai. Program 8.3 juga memakai `const` untuk batas jumlah suku dan `repeat..until` untuk memvalidasi jumlah suku yang diminta.

Program 8.3 `bab08_03_deret_fibonacci.pas` *Deret Fibonacci* File ▸ Examples...

```
{ Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 8.3 – Deret Fibonacci
  Deret 1, 1, 2, 3, 5, 8, 13, ... : setiap suku adalah jumlah dua suku
  sebelumnya. Cukup simpan dua suku terakhir, lalu geser. }

program bab08_03_deret_fibonacci;

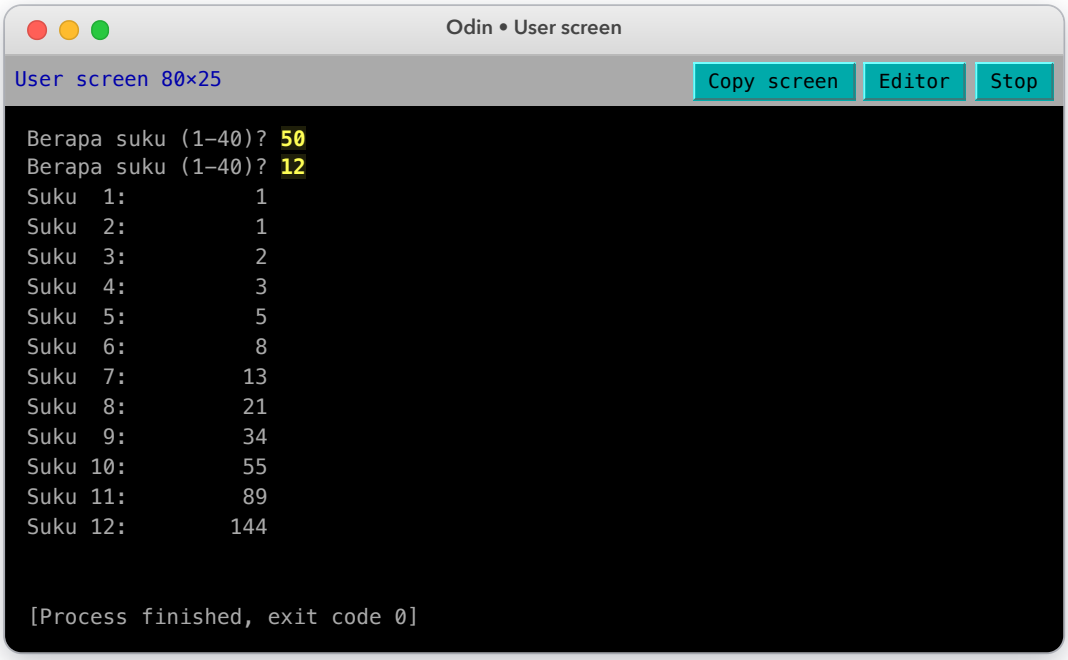
const
  BatasSuku = 40;                       { agar tetap dalam rentang LongInt }

var
  N, I: Integer;
  Sebelum, Sekarang, Berikut: LongInt;

begin
  repeat
    Write('Berapa suku (1–', BatasSuku, ')? ');
    ReadLn(N);
  until (N >= 1) and (N <= BatasSuku);

  Sebelum := 0;
  Sekarang := 1;
  for I := 1 to N do
  begin
    WriteLn('Suku ', I:2, ': ', Sekarang:10);
    Berikut := Sebelum + Sekarang;      { jumlah dua suku terakhir }
    Sebelum := Sekarang;                { geser ke depan }
```

```
28      Sekarang := Berikut;
29    end;
30  end.
```



Gambar 8.10 Masukan 50 ditolak (di atas batas 40), lalu 12 suku ditampilkan

Tabel 8.3 Jejak pergeseran pada baris 23–29 (enam putaran pertama)

`I`	`Sebelum`	`Sekarang` (dicetak)	`Berikut`	Sesudah geser (`Sebelum`, `Sekarang`)
1	0	1	1	(1, 1)
2	1	1	2	(1, 2)
3	1	2	3	(2, 3)
4	2	3	5	(3, 5)
5	3	5	8	(5, 8)
6	5	8	13	(8, 13)

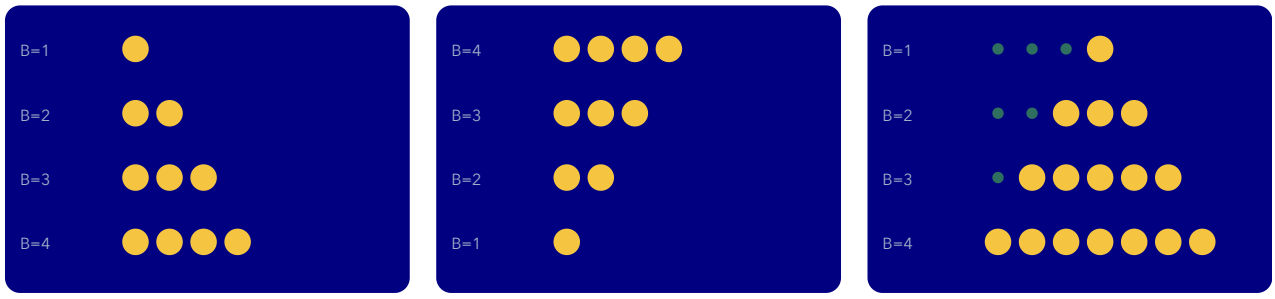
Baris	Penjelasan
8–9	Konstanta `BatasSuku = 40`. Memakai konstanta bernama membuat batas mudah diubah di satu tempat dan dipakai ulang di baris 17 dan 19.
12–13	Pencacah dan jumlah suku bertipe `Integer`; nilai deret bertipe `LongInt`.
16–19	Validasi: ulangi pertanyaan sampai `N` berada di 1..40. Syarat `until` menyatakan masukan yang sah.
21–22	Nilai awal: `Sebelum = 0` (suku ke-0), `Sekarang = 1` (suku ke-1).
23–29	Satu putaran: cetak suku ke-`I` selebar 10 (baris 25), hitung suku berikutnya (26), lalu geser (27–28).

> **AWAS**
>
> **Mengapa batasnya 40?** Suku Fibonacci tumbuh cepat. `LongInt` hanya menampung sampai 2.147.483.647, dan suku ke-47 (2.971.215.073) sudah melampauinya. Bila tipe terlalu kecil, Pascal tidak memberi peringatan, melainkan membalik ke bilangan negatif yang salah. Bagian Kesalahan yang Sering Terjadi di akhir bab ini memperlihatkannya dengan `Integer`.

8.5 Segitiga Bintang

Pola gambar dari karakter adalah latihan terbaik untuk perulangan bersarang. Rahasianya satu: **cari hubungan antara nomor baris dan jumlah karakter** di baris itu. Setelah rumusnya ketemu, kodenya hampir selalu berupa perulangan luar untuk baris dan satu atau dua perulangan dalam untuk karakter.

Program 8.4 menggambar tiga pola dengan tinggi `T`. Berikut rumusnya, dengan `B` sebagai nomor baris dari atas (1 sampai `T`).



Segitiga siku-siku
spasi 0, bintang B

Segitiga terbalik
spasi 0, bintang B (B turun)

Piramida
spasi T-B, bintang 2B-1

Piramida: spasi pengisi di kiri membuat bintang tampak di tengah.

Gambar 8.11 Tiga pola Program 8.4 untuk `T = 4`. Titik hijau muda adalah spasi, bintang digambar sebagai titik pejal.

Tabel 8.4 Rumus ketiga pola untuk tinggi `T` dan nomor baris `B`

Pola	Perulangan baris	Spasi di baris `B`	Bintang di baris `B`
Segitiga siku-siku	`B` = 1 sampai `T`	0	`B`
Segitiga terbalik	`B` = `T` turun ke 1	0	`B`
Piramida	`B` = 1 sampai `T`	`T - B`	`2B - 1`

Segitiga terbalik memakai `downto`: perulangan baris cukup dibalik, sedangkan isi barisnya sama persis dengan segitiga siku-siku. Piramida membutuhkan dua perulangan dalam berurutan: pertama spasi pengisi di kiri agar bintang terlihat di tengah, kemudian bintang dengan jumlah ganjil (1, 3, 5, ...).

Program 8.4 `bab08_04_segitiga_bintang.pas` *Segitiga bintang* File ▸ Examples...

```
1  { Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 8.4 - Segitiga bintang
3    Pola gambar dibuat dengan perulangan bersarang: perulangan luar
4    memilih baris, perulangan dalam menulis spasi dan bintang. Kuncinya
```

```
  adalah mencari hubungan nomor baris dengan jumlah bintang. }

program bab08_04_segitiga_bintang;

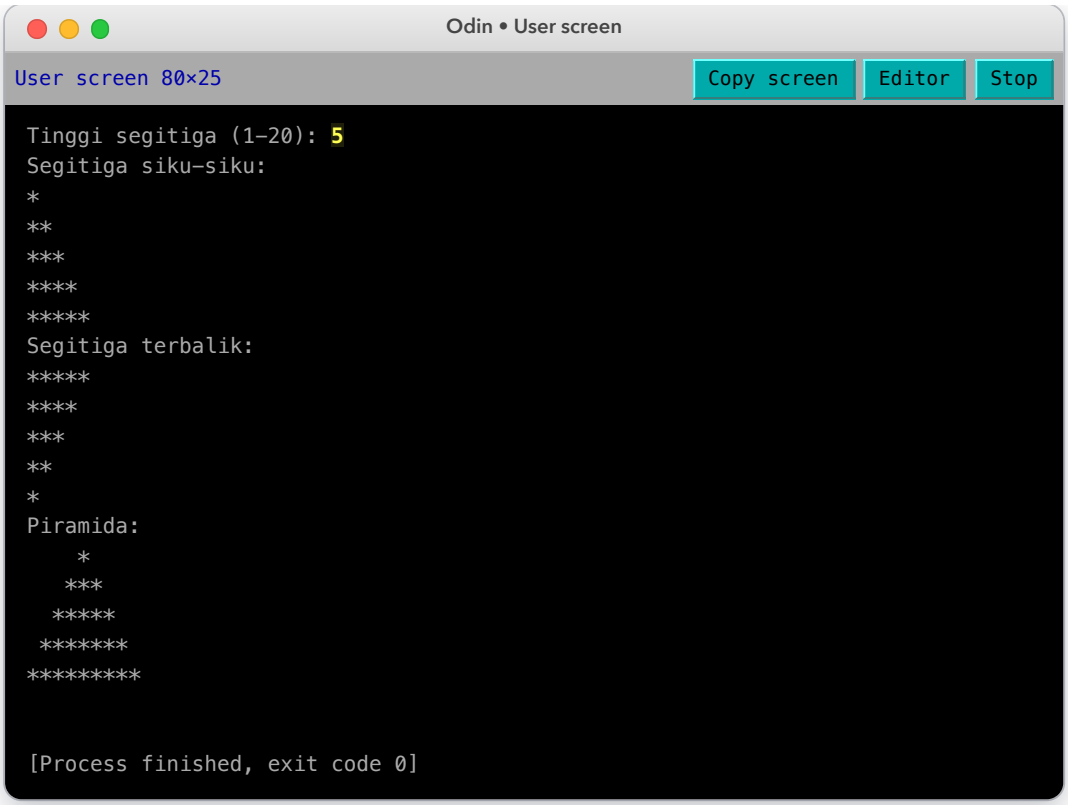
var
  Tinggi, Baris, I: Integer;

begin
  Write('Tinggi segitiga (1-20): ');
  ReadLn(Tinggi);
  if (Tinggi < 1) or (Tinggi > 20) then
    Tinggi := 5;                        { nilai bawaan bila tidak sah }

  WriteLn('Segitiga siku-siku:');       { baris ke-B berisi B bintang }
  for Baris := 1 to Tinggi do
  begin
    for I := 1 to Baris do Write('*');
    WriteLn;
  end;

  WriteLn('Segitiga terbalik:');        { bintang berkurang tiap baris }
  for Baris := Tinggi downto 1 do
  begin
    for I := 1 to Baris do Write('*');
    WriteLn;
  end;

  WriteLn('Piramida:');                 { spasi T-B, bintang 2B-1 }
  for Baris := 1 to Tinggi do
  begin
    for I := 1 to Tinggi - Baris do Write(' ');
    for I := 1 to 2 * Baris - 1 do Write('*');
    WriteLn;
  end;
end.
```



Gambar 8.12 Program 8.4 dengan tinggi 5

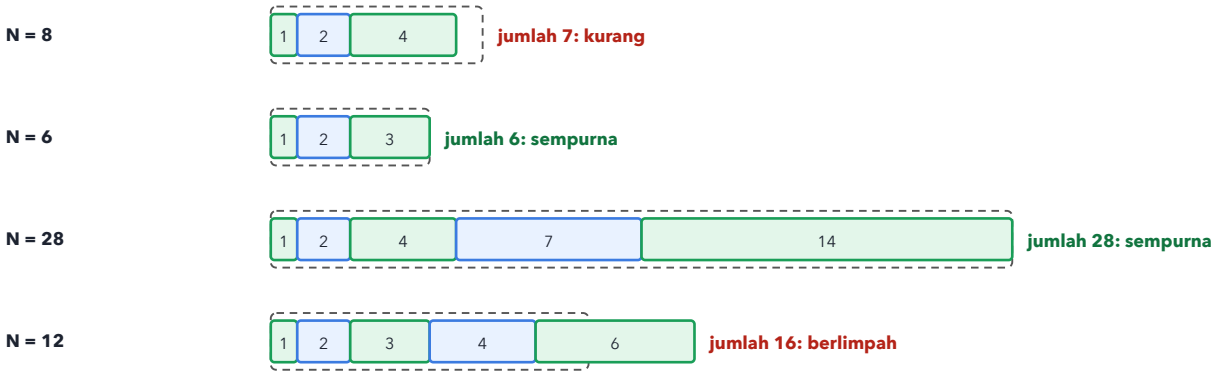
Baris	Penjelasan
13–16	Tinggi dibaca. Bila di luar 1..20, dipakai nilai bawaan 5 agar program tidak menggambar pola yang terlalu lebar atau kosong.
19–23	Segitiga siku-siku. Perulangan dalam (baris 21) menulis `Baris` buah bintang. Satu pernyataan `for` tunggal ditulis di satu baris bersama `Write('*')`. `WriteLn` di baris 22 pindah baris setelah itu.
26–30	Segitiga terbalik: `Baris` menurun dari `Tinggi` ke 1, sehingga baris pertama paling panjang.
33–38	Piramida. Baris 35 menulis `Tinggi - Baris` spasi, baris 36 menulis `2 * Baris - 1` bintang. Di baris pertama, 1 bintang berdiri di tengah; di baris terakhir tidak ada spasi dan bintangnya paling lebar.

COBA SENDIRI

Buat pola **belah ketupat**: piramida diikuti piramida terbalik. Atau gambar kotak `Tinggi x Tinggi` berongga (bintang hanya di tepi). Gambar dulu pada kertas dengan memberi nomor baris dan kolom, lalu cari rumusnya sebelum menulis kode.

8.6 Bilangan Sempurna

Sebuah bilangan disebut **sempurna** bila sama dengan jumlah semua pembaginya selain dirinya sendiri. Contoh 6 = 1 + 2 + 3, dan 28 = 1 + 2 + 4 + 7 + 14. Bilangan yang jumlah pembaginya lebih kecil disebut kurang (misalnya 8, jumlah pembaginya 1 + 2 + 4 = 7), dan yang lebih besar disebut berlimpah (misalnya 12, jumlahnya 1 + 2 + 3 + 4 + 6 = 16).



Gambar 8.13 Tiap pembagi (kecuali bilangan itu sendiri) digambar sebagai potongan berskala. Bila potongan tepat memenuhi garis putus-putus, bilangan itu sempurna.

Algoritmanya memakai dua perulangan bersarang: perulangan luar mencoba setiap `N`, perulangan dalam menjumlahkan semua pembaginya. Pembagi sebuah bilangan tidak mungkin lebih dari setengahnya (kecuali bilangan itu sendiri, yang memang tidak dihitung), sehingga perulangan dalam cukup berjalan sampai `N div 2`. Untuk bilangan yang lolos, perulangan dalam kedua mencetak rinciannya.

Program 8.5 `bab08_05_bilangan_sempurna.pas` *Bilangan sempurna* File ▸ Examples…

```
{ Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 8.5 - Bilangan sempurna
  Bilangan sempurna sama dengan jumlah semua pembaginya selain dirinya
  sendiri. Contoh: 6 = 1 + 2 + 3, dan 28 = 1 + 2 + 4 + 7 + 14. }

program bab08_05_bilangan_sempurna;

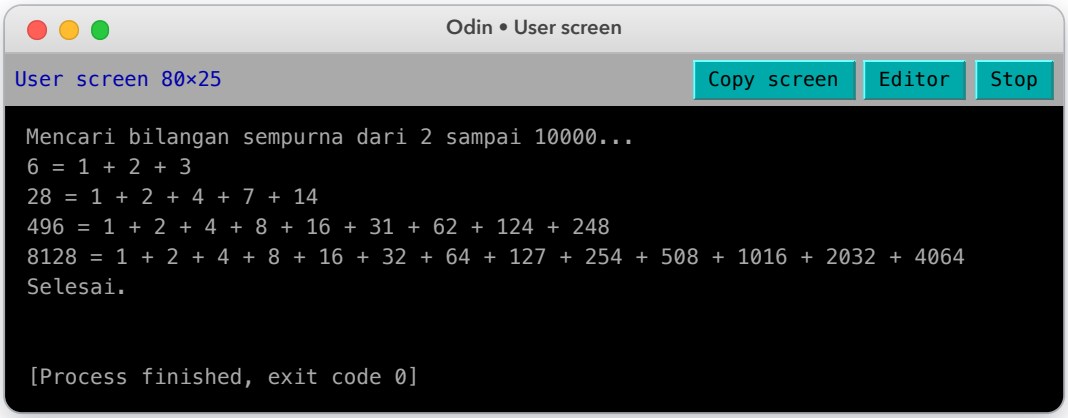
const
  Batas = 10000;

var
  N, Pembagi: Integer;
  Jumlah: LongInt;                    { bisa melebihi batas Integer }

begin
  WriteLn('Mencari bilangan sempurna dari 2 sampai ', Batas, '...');
  for N := 2 to Batas do
  begin
    Jumlah := 0;
    for Pembagi := 1 to N div 2 do       { pembagi tidak mungkin > N/2 }
      if N mod Pembagi = 0 then
        Jumlah := Jumlah + Pembagi;

    if Jumlah = N then
    begin
      Write(N, ' = 1');                  { tampilkan rinciannya }
      for Pembagi := 2 to N div 2 do
        if N mod Pembagi = 0 then
          Write(' + ', Pembagi);
      WriteLn;
    end;
  end;
  WriteLn('Selesai.');
```

```
34  end.
```



Gambar 8.14 Program 8.5: bilangan sempurna dari 2 sampai 10000

Baris	Penjelasan
8–9	Konstanta `Batas` menetapkan jangkauan pencarian.
12–13	`N` dan `Pembagi` cukup `Integer` (maksimum 10000), tetapi `Jumlah` dibuat `LongInt` karena penjumlahan semua pembagi bisa melebihi 32767 untuk bilangan yang besar.
17–32	Perulangan luar: satu putaran per bilangan `N`.
19	`Jumlah` harus dinolkan di setiap putaran luar. Bila terlupa, jumlah putaran sebelumnya ikut terbawa.
20–22	Perulangan dalam menjumlahkan pembagi. `if` sebagai badan `for` menyaring angka yang membagi habis `N`. Tidak ada `begin..end` karena badannya satu pernyataan `if` utuh.
24	Bila jumlah pembagi sama dengan `N`, bilangan itu sempurna.
26–30	Cetak rincian: mulai dari `N = 1`, lalu tambahkan `+ pembagi` untuk setiap pembagi berikutnya. Karena 1 pasti pembagi, penulisannya dimulai dengan "= 1".

INFO

Program ini melakukan sekitar 25 juta pembagian, jadi tunggu beberapa saat sebelum hasilnya muncul. Keempat bilangan sempurna yang ditemukan sampai 10000 adalah yang pertama dari deretnya; yang kelima sudah melebihi 33 juta.

8.7 Permainan Tebak Angka

Program terakhir bab ini sebuah permainan. Komputer memilih angka rahasia antara 1 dan 100, dan Anda menebaknya. Setiap tebakan dijawab "terlalu kecil" atau "terlalu besar", sampai tebakan benar. Program juga menghitung banyak percobaan. Dua hal baru muncul: `Randomize` dan `Random`.

BENTUK UMUM

```
Randomize;              { sekali di awal program }
Peubah := Random(N);    { bilangan acak 0..N-1 }
```

`Random(100)` menghasilkan bilangan bulat acak dari 0 sampai 99. Agar angkanya 1 sampai 100, hasilnya digeser dengan `+ 1`. `Randomize` mengacak "benih" pembangkit bilangan acak berdasarkan jam komputer; tanpa itu, urutan angkanya cenderung sama setiap kali program dijalankan.

> **7** **ATURAN TURBO PASCAL 7**
>
> Pemanggilan `Randomize` cukup sekali, di awal program, bukan di dalam perulangan. Fungsi `Random(N)` pada Turbo Pascal 7 memberi hasil `0..N-1`, bukan `1..N`. Itu sebabnya perlu `+ 1` pada baris 13.

Program 8.6 memakai `repeat..until` karena pemain harus menebak setidaknya sekali. Setelah salah, program memberi petunjuk dengan rantai `if..else if` yang Anda kenal dari Bab 6. Perhatikan bahwa kasus tebakan benar tidak memiliki cabang: ia ditangani oleh syarat `until`.

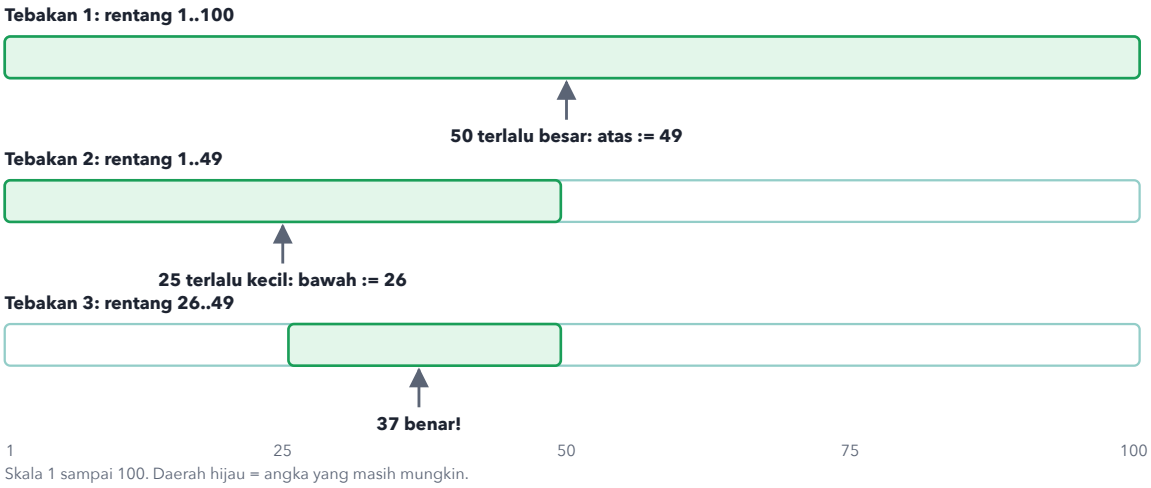
Program 8.6 `bab08_06_tebak_angka.pas` *Permainan tebak angka* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 8.6 — Permainan tebak angka
3    Komputer memilih angka acak 1..100, pemain menebak sampai benar.
4    Randomize mengacak benih bilangan acak; Random(100) memberi 0..99. }
5
6  program bab08_06_tebak_angka;
7
8  var
9    Rahasia, Tebakan, Percobaan: Integer;
10
11 begin
12   Randomize;                          { panggil sekali di awal program }
13   Rahasia := Random(100) + 1;         { 0..99 digeser menjadi 1..100 }
14   Percobaan := 0;
15
16   WriteLn('Saya memilih angka 1 sampai 100. Tebaklah!');
17   repeat
18     Write('Tebakan Anda: ');
19     ReadLn(Tebakan);
20     Percobaan := Percobaan + 1;
21     if Tebakan < Rahasia then
22       WriteLn('  Terlalu kecil.')
23     else if Tebakan > Rahasia then
24       WriteLn('  Terlalu besar.');
25   until Tebakan = Rahasia;
26
27   WriteLn('Benar! Angkanya ', Rahasia, '.');
28   WriteLn('Anda menebak dalam ', Percobaan, ' kali percobaan.');
29   if Percobaan <= 7 then
30     WriteLn('Hebat, strategi yang jitu!');  { pencarian biner: maks 7 kali }
31 end.
```

8.7.1 Strategi pencarian biner

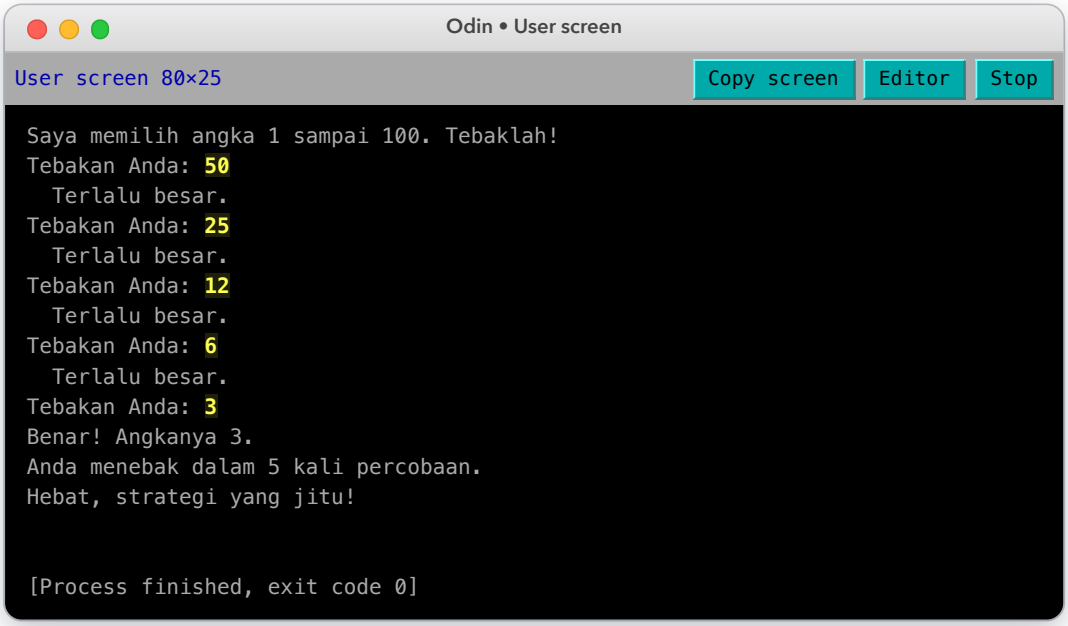
Bagaimana cara menebak paling efisien? Tebakan terbaik selalu **tepat di tengah** rentang yang masih mungkin. Bila jawabannya "terlalu besar", separuh atas rentang gugur; bila "terlalu kecil", separuh bawahnya gugur. Rentang menyusut setengah pada setiap tebakan, dan itulah **pencarian biner** (*binary*

search). Untuk 100 angka, rentang menyusut 100, 50, 25, 12, 6, 3, 1, sehingga paling banyak 7 tebakan sudah cukup (karena $2^7 = 128$, lebih dari 100). Itulah alasan komentar di baris 30 menyebut "maks 7 kali".



Gambar 8.15 Ilustrasi pencarian biner dengan angka rahasia pengandaian 37. Setiap jawaban membuang separuh rentang yang masih mungkin.

Untuk membuktikannya, buku ini **memainkan Program 8.6 secara otomatis**. Pemain otomatis membaca petunjuk yang tertulis di User screen dan mengetik tebakan di tengah rentang yang tersisa. Angka rahasia berbeda setiap kali program dijalankan (itulah gunanya `Randomize`), tetapi pemain otomatis selalu menemukannya dalam batas 7 kali percobaan, dan pesan "Hebat, strategi yang jitu!" di akhir ikut tampil. Perhatikan setiap tebakan di bawah: rentangnya selalu bergerak menuju angka rahasia.



Gambar 8.16 Program 8.6 dimainkan otomatis dengan pencarian biner (angka rahasia berbeda tiap kali dibangun)

Tabel 8.5 Cara pemain otomatis menentukan tebakan

Petunjuk program	Rentang baru	Tebakan berikutnya
(awal permainan)	1..100	`(1 + 100) div 2 = 50`

Petunjuk program	Rentang baru	Tebakan berikutnya
Terlalu kecil	bawah := tebakan + 1	tengah rentang `(bawah + atas) div 2`
Terlalu besar	atas := tebakan - 1	tengah rentang `(bawah + atas) div 2`
(tidak ada petunjuk: benar)	permainan selesai	-

Baris	Penjelasan
9	Tiga `Integer`: angka rahasia, tebakan terakhir, dan banyak percobaan.
12–14	`Randomize` dipanggil sekali; `Random(100) + 1` memilih angka 1..100; hitungan percobaan dimulai dari nol.
17–25	`repeat..until Tebakan = Rahasia`. Badan: baca tebakan, tambah `Percobaan`, beri petunjuk.
21–24	Rantai `if..else if`: dua cabang petunjuk dan tidak ada `else`, karena kasus "sama" ditangani `until`. Titik koma hanya di akhir seluruh rantai.
27–28	Setelah keluar dari perulangan: tampilkan angka dan jumlah percobaan.
29–30	Bonus pujian bila percobaan 7 kali atau kurang, batas teoretis pencarian biner untuk 100 angka.

COBA SENDIRI

Mainkan sendiri beberapa kali, dan usahakan memakai strategi tengah rentang. Lalu ubah baris 13 menjadi `Random(1000) + 1` dan sesuaikan teksnya. Berapa tebakan maksimum yang dibutuhkan sekarang? (Petunjuk: berapa kali 1000 harus dibagi dua sampai tersisa 1?)

8.8 Kesalahan yang Sering Terjadi

8.8.1 Syarat gabungan tanpa kurung

Pada Program 8.1, syarat `Prima and (Pembagi * Pembagi <= N)` memakai kurung. Tanpa kurung, `and` dikerjakan lebih dulu daripada `<=`, sehingga Pascal mencoba menghitung `Prima and Pembagi`, yaitu operasi antara Boolean dan bilangan.

tanpa_kurung.pas

```
1  program TanpaKurung;
2  var
3    N, Pembagi: Integer;
4    Prima: Boolean;
5  begin
6    N := 97;
7    Prima := True;
8    Pembagi := 2;
9    while Prima and Pembagi * Pembagi <= N do
10   begin
11     if N mod Pembagi = 0 then
12       Prima := False;
13     Pembagi := Pembagi + 1;
14   end;
15   WriteLn(Prima);
```

```
16  end.
```

PESAN KOMPILER

```
tanpa_kurung.pas(9,9) Error: Tipe operand tidak cocok dengan operator
```

Gambar 8.17 Tanpa kurung, and mengikat Prima dengan Pembagi * Pembagi lebih dulu

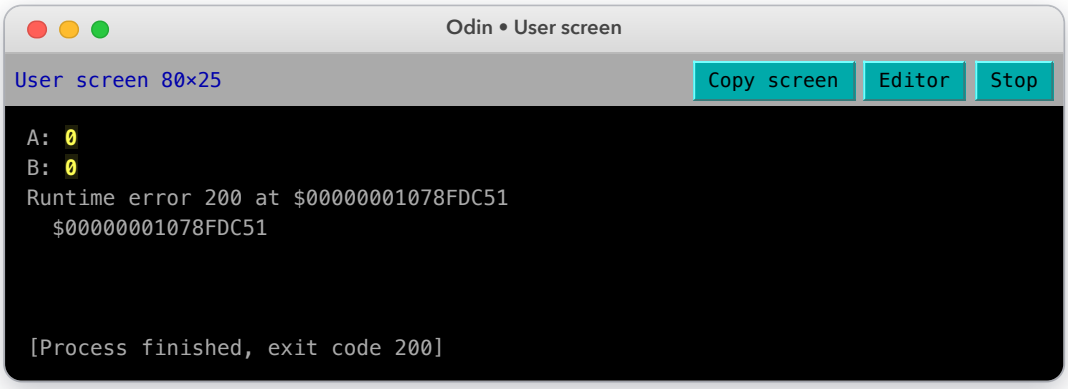
Perbaikannya: bungkus setiap perbandingan dengan kurung, seperti pada baris 18 Program 8.1.

8.8.2 FPB dari dua nol

Algoritma Euclid tidak memberikan jawaban untuk FPB(0, 0), sebab tidak ada bilangan terbesar yang membagi habis nol. Pada Program 8.2, `FPB` akan bernilai 0 dan baris 28 membaginya. Akibatnya program berhenti dengan runtime error. Program berikut meniru bagian itu.

fpb_nol.pas

```
1  program FpbNol;
2  var
3    A, B, X, Y, Sisa, FPB, KPK: LongInt;
4  begin
5    Write('A: '); ReadLn(A);
6    Write('B: '); ReadLn(B);
7    X := A;
8    Y := B;
9    while Y <> 0 do
10   begin
11     Sisa := X mod Y;
12     X := Y;
13     Y := Sisa;
14   end;
15   FPB := X;
16   KPK := A div FPB * B;
17   WriteLn('KPK = ', KPK);
18 end.
```



Gambar 8.18 Masukan 0 dan 0: FPB bernilai 0, lalu dipakai sebagai pembagi

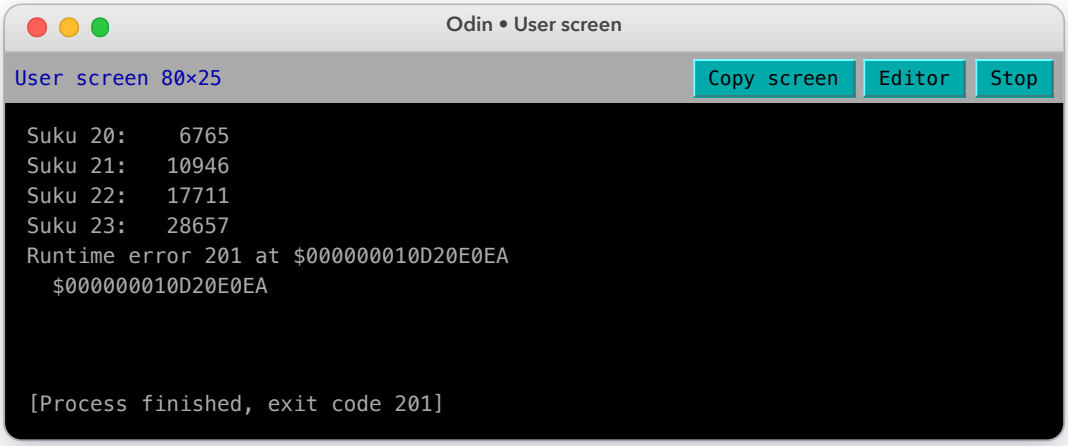
Penyelesaiannya: periksa dulu bahwa `FPB <> 0` (atau tolak masukan 0) sebelum membagi.

8.8.3 Tipe terlalu kecil: Fibonacci meluap

Program ini menghitung deret Fibonacci dengan `Integer` dan hanya menampilkan suku ke-20 sampai ke-25. Pada Turbo Pascal 7, `Integer` berukuran 16 bit (-32768..32767).

fib_integer.pas

```
1  program FibInteger;
2  var
3    I: Integer;
4    Sebelum, Sekarang, Berikut: Integer;
5  begin
6    Sebelum := 0;
7    Sekarang := 1;
8    for I := 1 to 25 do
9    begin
10     if I >= 20 then
11       WriteLn('Suku ', I:2, ': ', Sekarang:7);
12     Berikut := Sebelum + Sekarang;
13     Sebelum := Sekarang;
14     Sekarang := Berikut;
15   end;
16 end.
```



Gambar 8.19 Dengan Integer, suku yang melewati 32767 berubah menjadi bilangan negatif yang keliru

Pascal tidak memberi peringatan. Penjumlahan "melingkar" ke ujung rentang yang lain, sehingga angkanya tiba-tiba negatif. Pilihlah tipe yang cukup besar (`LongInt`) dan batasi jumlah suku, seperti di Program 8.3.

8.8.4 Lupa mengatur ulang penampung

Pada Program 8.5, bila baris `Jumlah := 0` (baris 19) dipindahkan ke luar perulangan, `Jumlah` terus bertambah dari satu `N` ke `N` berikutnya. Hasilnya, hampir tidak ada bilangan yang cocok dengan `Jumlah = N`. Aturan umum: variabel penampung (akumulator, penghitung, penanda) harus diberi nilai awal **tepat sebelum** perulangan yang memakainya, dan di dalam perulangan luar bila ia dipakai ulang di setiap putaran. Kesalahan serupa terjadi pada penanda `Prima` di baris 35 Program 8.1.

8.8.5 Pola bintang yang meleset satu

Pada piramida, rumus yang benar adalah `2 * Baris - 1`. Bila Anda menulis `2 * Baris`, setiap baris mendapat satu bintang ekstra dan baris pertama berisi dua bintang, sehingga puncak tidak lagi simetris. Selalu uji pola dengan baris pertama dan terakhir, dan gambarkan di kertas lebih dulu.

8.9 Rangkuman

- Algoritma adalah urutan langkah yang berhingga, jelas, dan benar untuk semua masukan sah. Rancang dengan contoh tangan lebih dulu, lalu tulis kode, lalu uji nilai batas.
- Bilangan prima diuji dengan pembagi 2, 3, ... selama `Pembagi * Pembagi <= N`; penanda Boolean `Prima` menghentikan perulangan begitu ketemu pembagi.
- Algoritma Euclid: ganti (A, B) dengan (B, A `mod` B) sampai B = 0; A adalah FPB. KPK = A `div` FPB * B (bagi dulu agar tidak meluap).
- Deret Fibonacci cukup menyimpan dua suku terakhir; urutan pergeseran harus benar. Pilih tipe data yang muat: suku ke-47 sudah melebihi `LongInt`.
- Pola dua dimensi dibuat dengan perulangan bersarang: cari rumus hubungan nomor baris dengan spasi dan bintang (piramida: `T - B` spasi, `2B - 1` bintang).
- Bilangan sempurna: jumlah pembagi sejati sama dengan bilangan itu (6, 28, 496, 8128 sampai 10000). Penampung jumlah harus dinolkan di setiap putaran luar.
- `Randomize` sekali di awal, `Random(N)` memberi 0..N-1. Strategi tebak tengah rentang (pencarian biner) menyelesaikan 1..100 dalam paling banyak 7 tebakan.

8.10 Latihan

1. **DASAR** Ubah Program 8.1 agar mencetak bilangan prima kembar sampai 100, yaitu pasangan prima dengan selisih 2 seperti (3, 5) dan (11, 13). Tuliskan lebih dulu daftarnya dengan tangan sebagai pembanding.
2. **DASAR** Tulis program yang membaca `N` lalu mencetak semua faktor (pembagi) `N` dan banyaknya. Gunakan perulangan `for` dari 1 sampai `N`.
3. **DASAR** Gunakan algoritma Euclid dengan tangan untuk FPB(252, 105). Buat tabel jejak seperti **Tabel 8.2**, lalu periksa dengan Program 8.2.
4. **MENENGAH** Tulis program faktorisasi prima: baca `N`, lalu tulis faktornya, misalnya `360 = 2 x 2 x 2 x 3 x 3 x 5`. Petunjuk: bagi dengan 2 selama habis, lalu 3, dan seterusnya.
5. **MENENGAH** Ubah Program 8.3 agar juga menampilkan perbandingan `Sekarang / Sebelum` tiap suku dengan lima desimal. Ke angka berapa nilai itu mendekat? (Ini disebut rasio emas.)
6. **MENENGAH** Gambar belah ketupat setinggi `2T - 1` baris (piramida disusul piramida terbalik), dan gambar juga papan catur 8 x 8 dari karakter `#` dan titik yang berselang-seling.

7. **TANTANGAN** Balik permainan: Anda memikirkan angka 1..100, dan komputer yang menebak. Setelah setiap tebakan, pemain menjawab `b` (terlalu besar), `k` (terlalu kecil), atau `s` (sama). Gunakan pencarian biner, dan tampilkan banyak tebakan yang dibutuhkan.

BAGIAN III

Prosedur, Fungsi, dan Rekursi

Memecah program menjadi bagian kecil yang bisa dipakai ulang, termasuk bagian yang memanggil dirinya sendiri.

09 Prosedur dan Fungsi

10 Rekursi

BAB 09

BAGIAN III · PROSEDUR, FUNGSI, DAN REKURSI

Prosedur dan Fungsi

1 2 3 4 5 6 7 8 **9** 10 11 12 13 14 15 16 17 18 19 20

TUJUAN PEMBELAJARAN

- Menjelaskan manfaat memecah program menjadi prosedur dan fungsi yang masing-masing punya satu tugas.
- Menulis dan memanggil `procedure` tanpa dan dengan parameter.
- Menulis `function` yang mengembalikan nilai lewat nama fungsinya, sesuai aturan Turbo Pascal 7 (tanpa `Result`).
- Membedakan parameter nilai dan parameter `var`, serta memilih yang tepat.
- Membedakan variabel global dan lokal, termasuk apa yang terjadi bila namanya sama.
- Melacak panggilan prosedur dengan debugger memakai F8 (Step over) dan F7 (Trace into).

PROGRAM DALAM BAB INI

9.1	`bab09_01_prosedur_dasar.pas`	Prosedur
9.2	`bab09_02_fungsi_dasar.pas`	Fungsi
9.3	`bab09_03_parameter_nilai.pas`	Parameter nilai
9.4	`bab09_04_parameter_var.pas`	Parameter var
9.5	`bab09_05_lokal_global.pas`	Variabel lokal dan global
9.6	`bab09_06_kalkulator_modular.pas`	Latihan: kalkulator modular

Program yang Anda tulis sejauh ini mengerjakan semuanya di satu tempat: blok `begin..end.` yang makin lama makin panjang. Program kecil masih nyaman dibaca seperti itu, tetapi program yang sebenarnya, apalagi yang dikerjakan banyak orang, harus dipecah menjadi potongan-potongan kecil yang masing-masing diberi nama. Potongan itu disebut **prosedur** dan **fungsi**. Dengan keduanya, program Anda terbaca hampir seperti kalimat, kode yang sama tidak perlu ditulis berulang, dan bila ada kesalahan Anda tahu persis ke bagian mana harus melihat.

9.1 Memecah Program Menjadi Bagian

Bayangkan dapur sebuah warung. Juru masaknya tidak menghafal ulang cara menanak nasi setiap kali ada pesanan. Ada resep yang sudah diberi nama: "nasi goreng", "mi ayam". Pelayan cukup berkata "satu nasi goreng", dan dapur tahu apa yang harus dikerjakan. Di Pascal, **subprogram** adalah resep semacam itu: sekumpulan perintah yang diberi nama, ditulis sekali, dan dijalankan kapan pun namanya disebut. Menyebut nama itu disebut **memanggil** (*call*) subprogram.

Pascal mengenal dua macam subprogram. **Prosedur** (*procedure*) hanya mengerjakan sesuatu, misalnya menampilkan garis atau menukar dua nilai. **Fungsi** (*function*) mengerjakan sesuatu lalu **menghasilkan sebuah nilai**, misalnya kuadrat suatu bilangan, sehingga pemanggilannya bisa dipakai di dalam ekspresi. Anda sudah sering memakai keduanya tanpa menyadarinya: `WriteLn` adalah prosedur bawaan, sedangkan `Sqr` dan `Length` adalah fungsi bawaan. Sekarang Anda akan membuatnya sendiri.

1 Terbaca
Program utama cukup berisi urutan nama langkah, misalnya `TampilkanJudul; TampilkanMenu;`, tanpa rincian di dalamnya.

2 Tidak berulang
Perintah yang dipakai berkali-kali ditulis sekali. Mengubahnya pun cukup di satu tempat.

3 Mudah diuji
Setiap bagian bisa dicoba sendiri. Bila hasilnya salah, Anda tahu bagian mana yang bermasalah.

9.2 Prosedur: Perintah yang Diberi Nama

Prosedur ditulis di bagian deklarasi, **di atas** blok program utama. Kata `procedure`, nama, titik koma, lalu badan prosedur berupa `begin..end` yang diakhiri titik koma (bukan titik: titik hanya untuk penutup program).

BENTUK UMUM

```
procedure NamaProsedur;
begin
  pernyataan;
end;

NamaProsedur;        { pemanggilan: cukup tulis namanya }
```

Program 9.1 menampilkan daftar menu kantin. Ada tiga prosedur. `GarisPemisah` menulis satu garis. `TampilkanJudul` memanggil `GarisPemisah` dua kali, dan `TampilkanMenu` mencetak daftar harga. Prosedur boleh memanggil prosedur lain, asalkan prosedur yang dipanggil sudah ditulis **lebih dulu** di atasnya. Buka `bab09_01_prosedur_dasar.pas` lewat `File ▸ Examples...` dan jalankan dengan Ctrl + F9.

Program 9.1 `bab09_01_prosedur_dasar.pas` *Prosedur* File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 9.1 — Prosedur
3    Prosedur adalah kumpulan perintah yang diberi nama. Ditulis sekali
4    di atas program utama, lalu dipanggil dengan namanya berkali-kali.
5    Program menjadi lebih pendek dan mudah dibaca. }
6
7  program bab09_01_prosedur_dasar;
8
9  procedure GarisPemisah;                  { deklarasi prosedur }
10 begin
11   WriteLn('----------------------------------------');
12 end;
13
14 procedure TampilkanJudul;
15 begin
16   GarisPemisah;                          { memanggil prosedur di atasnya }
17   WriteLn('       DAFTAR MENU KANTIN SEKOLAH');
18   GarisPemisah;
19 end;
20
21 procedure TampilkanMenu;
22 begin
23   WriteLn(' 1. Nasi goreng ..... Rp 12000');
24   WriteLn(' 2. Mi ayam ......... Rp 10000');
25   WriteLn(' 3. Soto banjar ..... Rp 15000');
26   WriteLn(' 4. Es teh .......... Rp  3000');
27 end;
28
29 begin                                    { program utama mulai di sini }
30   TampilkanJudul;                        { panggil cukup dengan namanya }
31   TampilkanMenu;
32   GarisPemisah;
33   WriteLn('Selamat makan!');
34 end.
```



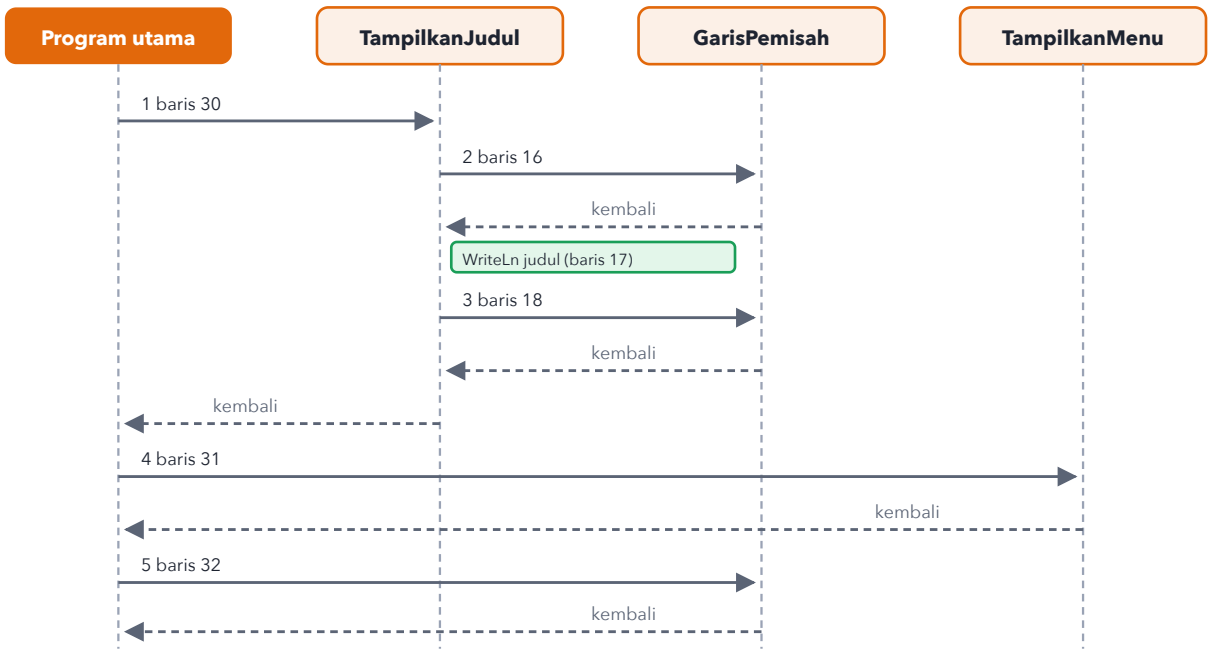
Gambar 9.1 Tiga prosedur bekerja sama menyusun daftar menu

Baris	Penjelasan
9–12	Deklarasi `GarisPemisah`. Badannya hanya satu `WriteLn`. Tidak ada yang tampil saat prosedur ini dideklarasikan; isinya baru berjalan ketika dipanggil.
14–19	`TampilkanJudul` memanggil `GarisPemisah` di baris 16 dan 18. Garis yang sama muncul di atas dan di bawah judul, padahal `WriteLn` garisnya hanya ditulis satu kali.
21–27	`TampilkanMenu` mencetak empat baris menu.
29–34	Program utama. Isinya hanya nama-nama prosedur, sehingga alur program terbaca sekilas: judul, menu, garis, ucapan penutup.

AWAS

Pascal membaca berkas dari atas ke bawah dan hanya mengenal nama yang **sudah dideklarasikan** di atasnya. Bila `TampilkanJudul` ditulis sebelum `GarisPemisah`, kompiler akan menolak panggilan di baris 16 karena nama itu belum dikenal. Karena itu urutan penulisan prosedur selalu dari yang paling dasar ke yang memakainya.

Urutan kerjanya bisa digambarkan sebagai diagram. Setiap kali sebuah prosedur dipanggil, program "melompat" ke dalam prosedur itu, mengerjakan isinya dari atas ke bawah, lalu **kembali** ke baris tepat sesudah pemanggilan. **Gambar 9.2** memperlihatkan seluruh perjalanan Program 9.1: panah penuh adalah pemanggilan, panah putus-putus adalah kembalinya kendali.



Gambar 9.2 Perjalanan kendali pada Program 9.1. Angka menunjukkan urutan pemanggilan; setiap pemanggilan diakhiri kembalinya kendali ke pemanggil.

9.3 Fungsi: Subprogram yang Menghasilkan Nilai

Fungsi sama seperti prosedur, dengan dua perbedaan. Pertama, setelah daftar parameter ditulis titik dua dan **tipe hasil**. Kedua, di dalam badan fungsi Anda harus mengisi hasilnya. Cara mengisinya di Turbo Pascal 7 adalah menugaskan nilai ke **nama fungsi itu sendiri**.

BENTUK UMUM

```
function NamaFungsi(parameter: Tipe): TipeHasil;
begin
  NamaFungsi := nilai;       { inilah hasil yang dikembalikan }
end;

Variabel := NamaFungsi(argumen);   { dipanggil di dalam ekspresi }
```

Perbedaan terpenting dengan prosedur terletak pada cara memanggilnya. Prosedur dipanggil sebagai pernyataan tersendiri. Fungsi dipanggil sebagai **bagian dari ekspresi**: disimpan ke variabel, dijumlahkan, dibandingkan, dicetak, atau dijadikan argumen fungsi lain. Tabel 9.1 merangkum perbedaannya.

Tabel 9.1 Prosedur dan fungsi dibandingkan

	Prosedur	Fungsi
Kata kunci	`procedure`	`function`
Tipe hasil	tidak ada	ditulis sesudah titik dua
Mengisi hasil	tidak perlu	`NamaFungsi := nilai;`
Cara memanggil	pernyataan sendiri: `Tukar(X, Y);`	di dalam ekspresi: `Z := Kuadrat(5) + 1;`

	Prosedur	Fungsi
Cocok untuk	melakukan tindakan (tampil, ubah, tukar)	menghitung satu nilai (kuadrat, luas, genap atau tidak)

7 **ATURAN TURBO PASCAL 7**

Di Turbo Pascal 7 **tidak ada variabel** `Result`. Bahasa lain, dan mode Delphi di Free Pascal, mengizinkan `Result := nilai`, tetapi di buku ini hasil fungsi selalu diberikan lewat nama fungsi: `Kuadrat := X * X`. Jangan sampai tertukar: di dalam fungsi, nama fungsi di **sisi kiri** `:=` berarti "tetapkan hasil", sedangkan nama fungsi yang muncul di **sisi kanan** berarti memanggil fungsi itu lagi (itulah rekursi, yang dibahas di bab berikutnya).

Program 9.2 berisi empat fungsi dengan tipe hasil berbeda: `LongInt`, `Integer`, `Boolean`, dan `Real`. Perhatikan bahwa `Genap` cukup mengisi namanya dengan hasil sebuah perbandingan, dan `Terbesar` mengisi namanya di salah satu dari dua cabang `if`.

Program 9.2 `bab09_02_fungsi_dasar.pas` *Fungsi* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 9.2 — Fungsi
  Fungsi mirip prosedur, tetapi menghasilkan sebuah nilai. Tipe nilai
  ditulis sesudah titik dua. Di dalam fungsi, nilai hasil diberikan
  dengan mengisi nama fungsinya: NamaFungsi := nilai. }

program bab09_02_fungsi_dasar;

function Kuadrat(X: LongInt): LongInt;
begin
  Kuadrat := X * X;                    { nilai yang dikembalikan }
end;

function Terbesar(A, B: Integer): Integer;
begin
  if A > B then
    Terbesar := A
  else
    Terbesar := B;
end;

function Genap(N: Integer): Boolean;
begin
  Genap := N mod 2 = 0;                { hasil perbandingan: Boolean }
end;

function LuasLingkaran(R: Real): Real;
begin
  LuasLingkaran := Pi * R * R;
end;

var
  Hasil: LongInt;

begin
  Hasil := Kuadrat(12);                { hasil fungsi disimpan }
```

```
37    WriteLn('Kuadrat 12       = ', Hasil);
38    { Fungsi juga bisa langsung dipakai di dalam ekspresi }
39    WriteLn('Kuadrat 3 + 4    = ', Kuadrat(3) + Kuadrat(4));
40    WriteLn('Terbesar(17, 42) = ', Terbesar(17, 42));
41    WriteLn('Genap(7)?          ', Genap(7));
42    WriteLn('Luas lingkaran r=7: ', LuasLingkaran(7):0:2);
43    if Genap(Terbesar(10, 3)) then        { fungsi di dalam fungsi }
44      WriteLn('Bilangan terbesar dari 10 dan 3 adalah genap.');
45  end.
```



Gambar 9.3 Hasil fungsi dipakai dalam ekspresi, penugasan, dan `WriteLn`

Baris	Penjelasan
9–12	`Kuadrat` menerima satu parameter `X` dan mengembalikan `LongInt`. Baris 11 mengisi hasilnya. Tipe `LongInt` dipilih agar bilangan besar muat.
14–20	`Terbesar` membandingkan dua `Integer`. Tepat satu dari baris 17 dan 19 dijalankan, sehingga hasilnya selalu terisi.
22–25	`Genap` mengembalikan `Boolean`. Ekspresi `N mod 2 = 0` sudah berupa `True` atau `False`, jadi tidak perlu `if`.
27–30	`LuasLingkaran` memakai konstanta bawaan `Pi`.
36–37	Hasil `Kuadrat(12)` disimpan ke variabel `Hasil` lebih dulu, baru dicetak.
39	Dua pemanggilan dalam satu ekspresi: `Kuadrat(3) + Kuadrat(4)` sama dengan `9 + 16`.
41	Nilai `Boolean` bisa dicetak oleh `WriteLn`; Turbo Pascal menuliskannya sebagai `TRUE` atau `FALSE`.
42	Hasil bertipe `Real` diformat dengan `:0:2`, dua angka di belakang titik.
43–44	Fungsi di dalam fungsi: `Terbesar(10, 3)` dihitung dulu (hasilnya 10), baru hasilnya diberikan kepada `Genap`.

COBA SENDIRI

Tambahkan fungsi `Kubik(X: LongInt): LongInt` yang mengembalikan `X * X * X`, lalu cetak `Kubik(5)`. Setelah itu hapus baris `Kuadrat := X * X;` dan jalankan lagi: fungsi yang tidak pernah mengisi namanya mengembalikan nilai sembarang, dan itu salah satu sumber kesalahan paling licin.

9.4 Parameter: Mengirim Data ke Subprogram

Subprogram yang selalu mengerjakan hal yang persis sama kurang berguna. Agar fleksibel, kita mengirim data lewat **parameter**. Ada dua istilah yang sering tertukar:

- **Parameter formal**: nama yang ditulis pada judul subprogram, misalnya `X` pada `procedure Gandakan(X: Integer)`. Ia bekerja seperti variabel lokal yang diisi saat subprogram dipanggil.
- **Argumen** (parameter aktual): nilai atau variabel yang Anda tulis saat memanggil, misalnya `Angka` pada `Gandakan(Angka)`.

Argumen dicocokkan dengan parameter berdasarkan **urutan** dan **tipenya**, bukan namanya. Beberapa parameter yang bertipe sama boleh digabung dengan koma, seperti `Terbesar(A, B: Integer)`. Pascal memiliki dua cara mengirim parameter, dan memilih yang tepat adalah inti bab ini.

9.4.1 Parameter nilai: subprogram menerima salinan

Parameter biasa disebut **parameter nilai** (*value parameter*). Saat subprogram dipanggil, nilai argumen **disalin** ke parameter. Subprogram bekerja pada salinan itu; apa pun yang dilakukannya tidak menyentuh variabel aslinya. Karena yang disalin hanya nilai, argumennya boleh berupa ekspresi, misalnya `Gandakan(10 + 3)`.

Program 9.3 memperlihatkan hal ini. `Gandakan` mengalikan `X` dengan 2 dan mencetaknya, tetapi `Angka` di program utama tetap utuh. `CetakBaris` menunjukkan dua parameter dengan tipe berbeda dan sebuah variabel lokal `I`.

Program 9.3 `bab09_03_parameter_nilai.pas` *Parameter nilai* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 9.3 — Parameter nilai
  Parameter biasa (parameter nilai) menerima SALINAN nilai dari
  pemanggil. Mengubah parameter di dalam prosedur tidak mengubah
  variabel asli di luar. Argumennya boleh berupa ekspresi. }

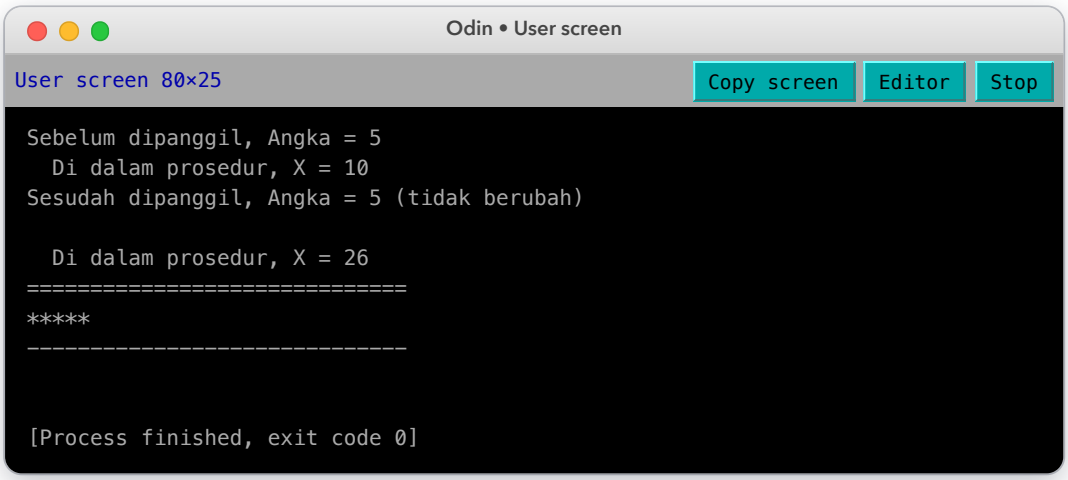
program bab09_03_parameter_nilai;

procedure Gandakan(X: Integer);          { X hanya salinan }
begin
  X := X * 2;
  WriteLn('  Di dalam prosedur, X = ', X);
end;

procedure CetakBaris(Karakter: Char; Panjang: Integer);
var
  I: Integer;
begin
  for I := 1 to Panjang do
    Write(Karakter);
  WriteLn;
end;

var
  Angka: Integer;
```

```
26
27  begin
28    Angka := 5;
29    WriteLn('Sebelum dipanggil, Angka = ', Angka);
30    Gandakan(Angka);                        { nilai 5 disalin ke X }
31    WriteLn('Sesudah dipanggil, Angka = ', Angka, ' (tidak berubah)');
32    WriteLn;
33
34    Gandakan(10 + 3);                       { argumen boleh berupa ekspresi }
35
36    { Urutan argumen harus sesuai urutan parameter: Char dulu, lalu Integer }
37    CetakBaris('=', 30);
38    CetakBaris('*', Angka);
39    CetakBaris('-', Angka * 6);
40  end.
```



Gambar 9.4 `Angka` tetap 5 walaupun `X` di dalam prosedur menjadi 10

Baris	Penjelasan
9–13	`X` adalah parameter nilai. Baris 11 mengubah `X`, dan baris 12 mencetak nilai barunya.
15–22	`CetakBaris` mencetak `Karakter` sebanyak `Panjang` kali. `I` adalah variabel lokal (baris 16–17) untuk menghitung perulangan `for`.
28–31	`Angka` bernilai 5 sebelum dan sesudah pemanggilan `Gandakan(Angka)`.
34	Argumen berupa ekspresi: `10 + 3` dihitung dulu menjadi 13, lalu disalin ke `X`.
37–39	Argumen untuk `Karakter` harus `Char` dan untuk `Panjang` harus bilangan bulat, dalam urutan itu. Baris 39 mengirim `Angka * 6` = 30.

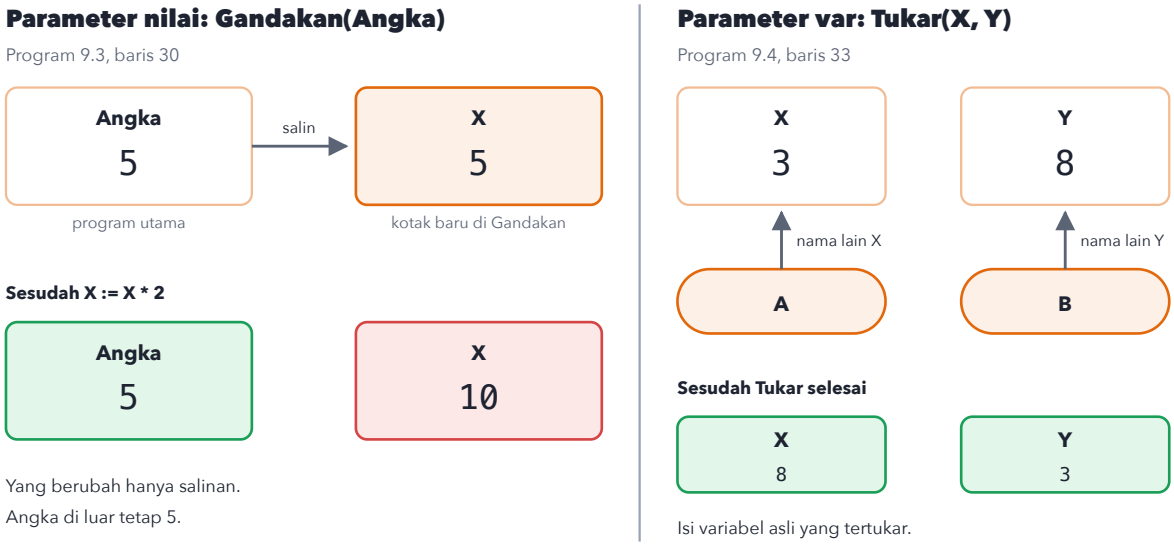
9.4.2 Parameter var: subprogram memakai variabel aslinya

Kadang kita justru **ingin** subprogram mengubah variabel pemanggil, misalnya menukar isi dua variabel. Untuk itu tuliskan kata `var` di depan parameter. Pada **parameter var** (*variable parameter*, atau *pass by reference*) tidak ada salinan. Nama parameter hanya menjadi **nama lain** bagi variabel yang dikirim, jadi perubahan di dalam subprogram langsung terjadi pada variabel aslinya.

BENTUK UMUM

```
procedure Nama(P1: Tipe; var P2, P3: Tipe);
                    { P1: nilai (salinan);  P2 dan P3: var (variabel asli) }
```

Gambar berikut membandingkan keduanya dengan kotak memori. Di kiri, `X` adalah kotak baru berisi salinan nilai `Angka`; di kanan, `A` dan `B` bukan kotak baru, melainkan tambahan label pada kotak `X` dan `Y` yang sudah ada.



Gambar 9.5 Parameter nilai bekerja pada salinan di kotak baru; parameter `var` hanyalah nama lain bagi kotak yang sudah ada.

Program 9.4 memakai parameter `var` dalam dua cara. `Tukar` menukar dua bilangan, pekerjaan klasik yang membutuhkan variabel pembantu `Sementara`. `HitungPersegi` memperlihatkan manfaat kedua: sebuah fungsi hanya bisa mengembalikan satu nilai, sedangkan prosedur dengan parameter `var` dapat mengembalikan **beberapa hasil sekaligus**.

Program 9.4 `bab09_04_parameter_var.pas` *Parameter var* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 9.4 — Parameter var
3    Parameter yang diawali var tidak menerima salinan, tetapi memakai
4    variabel aslinya. Perubahan di dalam prosedur ikut terlihat di luar.
5    Dipakai untuk mengubah variabel atau mengembalikan beberapa hasil. }
6
7  program bab09_04_parameter_var;
8
9  procedure Tukar(var A, B: Integer);      { A dan B adalah variabel asli }
10 var
11   Sementara: Integer;
12 begin
13   Sementara := A;
14   A := B;
15   B := Sementara;
16 end;
17
18 { Dua hasil sekaligus: Luas dan Keliling dikirim lewat parameter var }
```

```
19  procedure HitungPersegi(Panjang, Lebar: Real; var Luas, Keliling: Real);
20  begin
21    Luas := Panjang * Lebar;
22    Keliling := 2 * (Panjang + Lebar);
23  end;
24
25  var
26    X, Y: Integer;
27    L, K: Real;
28
29  begin
30    X := 3;
31    Y := 8;
32    WriteLn('Sebelum ditukar: X = ', X, ', Y = ', Y);
33    Tukar(X, Y);
34    WriteLn('Sesudah ditukar: X = ', X, ', Y = ', Y);
35
36    HitungPersegi(6, 4, L, K);
37    WriteLn('Persegi panjang 6 x 4: luas ', L:0:1, ', keliling ', K:0:1);
38
39    { Argumen untuk parameter var harus variabel, bukan angka atau
40      ekspresi. Tukar(3, 8) akan ditolak kompiler. }
41  end.
```



Gambar 9.6 `X` dan `Y` bertukar isi; luas dan keliling diterima lewat parameter `var`

Baris	Penjelasan
9	`var A, B: Integer` : kedua parameter adalah variabel asli milik pemanggil. Setiap pemanggilan boleh memakai variabel yang berbeda.
10–11	`Sementara` adalah variabel lokal. Ia menyimpan isi `A` agar tidak hilang saat `A` ditimpa.
13–15	Pertukaran tiga langkah: simpan `A` , isi `A` dengan `B` , isi `B` dengan simpanan tadi. Tanpa variabel bantu, salah satu nilai akan hilang.
19	Dua parameter pertama adalah parameter nilai (masukan); dua terakhir parameter `var` (keluaran). Kata `var` berlaku untuk `Luas` dan `Keliling` sekaligus.
21–22	Hasil ditulis ke parameter `var` , yang sebenarnya adalah `L` dan `K` di program utama.
33	`Tukar(X, Y)` : argumen harus berupa variabel.
36–37	Argumen `6` dan `4` boleh angka (parameter nilai), tetapi `L` dan `K` wajib variabel (parameter `var`).

> ★ **INGAT**
>
> Aturan memilih: gunakan **parameter nilai** untuk data masukan yang hanya dibaca, dan **parameter `var`** hanya bila subprogram memang harus mengubah variabel pemanggil atau mengeluarkan lebih dari satu hasil. Bila yang dikembalikan hanya satu nilai, fungsi biasanya lebih rapi daripada prosedur dengan `var`.

9.5 Variabel Lokal dan Global

Setiap nama di Pascal punya **cakupan** (*scope*): daerah program tempat nama itu dikenal. Variabel yang dideklarasikan di atas program utama (di luar semua subprogram) adalah variabel **global**: ia dikenal di seluruh program di bawahnya, dan hidup sampai program berakhir. Variabel yang dideklarasikan di dalam sebuah subprogram adalah variabel **lokal**: hanya subprogram itu yang mengenalnya, ia dibuat baru setiap kali subprogram dipanggil, dan lenyap begitu subprogram selesai. Parameter formal juga termasuk lokal.

Bila variabel lokal memakai nama yang sama dengan variabel global, di dalam subprogram nama itu menunjuk ke yang **lokal**; yang global "tertutup" dan tidak tersentuh. Program 9.5 memperlihatkan ketiga kemungkinan sekaligus.



Gambar 9.7 Cakupan pada Program 9.5: satu variabel global, satu variabel lokal yang menutupinya, dan satu variabel lokal biasa.

Program 9.5 `bab09_05_lokal_global.pas` *Variabel lokal dan global* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 9.5 — Variabel lokal dan global
3    Variabel global dideklarasikan di program utama dan dikenal di mana
4    saja sesudahnya. Variabel lokal dideklarasikan di dalam prosedur,
5    hanya dikenal di sana, dan hilang begitu prosedur selesai. }
6
7  program bab09_05_lokal_global;
8
9  var
10   Hitungan: Integer;                    { global }
11
```

```
12  procedure Tambah;
13  begin
14    Hitungan := Hitungan + 1;              { memakai variabel global }
15  end;
16
17  procedure Coba;
18  var
19    Hitungan: Integer;                     { lokal, menutupi yang global }
20  begin
21    Hitungan := 100;
22    WriteLn('  Di dalam Coba, Hitungan lokal = ', Hitungan);
23  end;
24
25  procedure Sapa;
26  var
27    Kali: Integer;                         { lokal: dibuat baru tiap dipanggil }
28  begin
29    Kali := 1;
30    WriteLn('  Halo! Variabel lokal Kali selalu mulai dari ', Kali);
31  end;
32
33  begin
34    Hitungan := 0;
35    Tambah;
36    Tambah;
37    WriteLn('Setelah dua kali Tambah, Hitungan = ', Hitungan);
38
39    Coba;                                  { tidak mengubah Hitungan global }
40    WriteLn('Kembali di program utama, Hitungan = ', Hitungan);
41
42    Sapa;
43    Sapa;
44    { Kali tidak dikenal di sini. WriteLn(Kali) akan ditolak kompiler.
45      Sebisa mungkin pakai parameter daripada variabel global. }
46  end.
```



Gambar 9.8 Hitungan global hanya berubah oleh `Tambah`; `Coba` mengubah variabel lokalnya sendiri

Baris	Penjelasan
9–10	`Hitungan` global. Baris 34 mengisinya dengan 0 secara eksplisit; jangan mengandalkan nilai awal variabel.
12–15	`Tambah` tidak mendeklarasikan `Hitungan` sendiri, sehingga baris 14 mengubah variabel global.

Baris	Penjelasan
17–23	`Coba` mendeklarasikan `Hitungan` lagi di baris 19. Baris 21 dan 22 hanya melihat yang lokal, dan nilai 100 hilang saat prosedur selesai.
25–31	`Kali` selalu mulai dari 1 pada setiap panggilan, karena variabel lokal dibuat baru dan diisi di baris 29. Nilai lama dari panggilan sebelumnya tidak bisa diandalkan.
34–37	Dua kali `Tambah` menaikkan `Hitungan` dari 0 menjadi 2.
39–40	Sesudah `Coba`, `Hitungan` global masih 2.

> **AWAS**
>
> Variabel global memang mudah dipakai, tetapi berbahaya: setiap subprogram bisa mengubahnya diam-diam, sehingga sulit melacak siapa yang membuat nilainya salah. Biasakan **mengirim data lewat parameter** dan menyimpan hasil sementara di variabel lokal. Beri nama yang berbeda bagi variabel lokal dan global, agar tidak ada yang tertutup tanpa disengaja.

9.6 Melacak Panggilan dengan Debugger

Membaca diagram panggilan itu berguna, tetapi melihatnya terjadi jauh lebih meyakinkan. Debugger Odin Turbo Pascal menjalankan program satu baris demi satu baris dan menunjukkan isi variabel. Baris yang **akan** dijalankan berikutnya ditandai hijau. Dua tombol langkahnya berbeda hanya pada baris yang berisi pemanggilan subprogram:

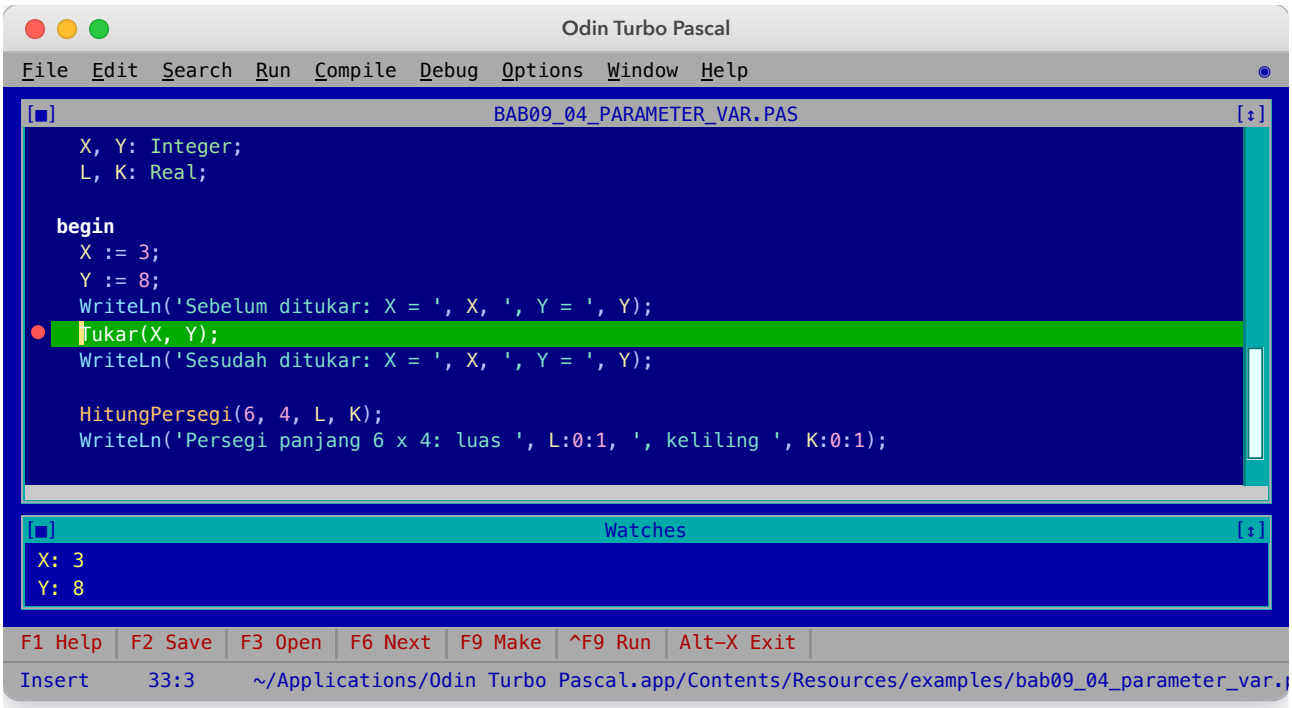
F8 Step over

Menjalankan satu baris. Bila baris itu memanggil prosedur, seluruh prosedur dijalankan sekaligus dan Anda berhenti di baris sesudah pemanggilan. Pakai bila Anda percaya isi prosedur sudah benar.

F7 Trace into

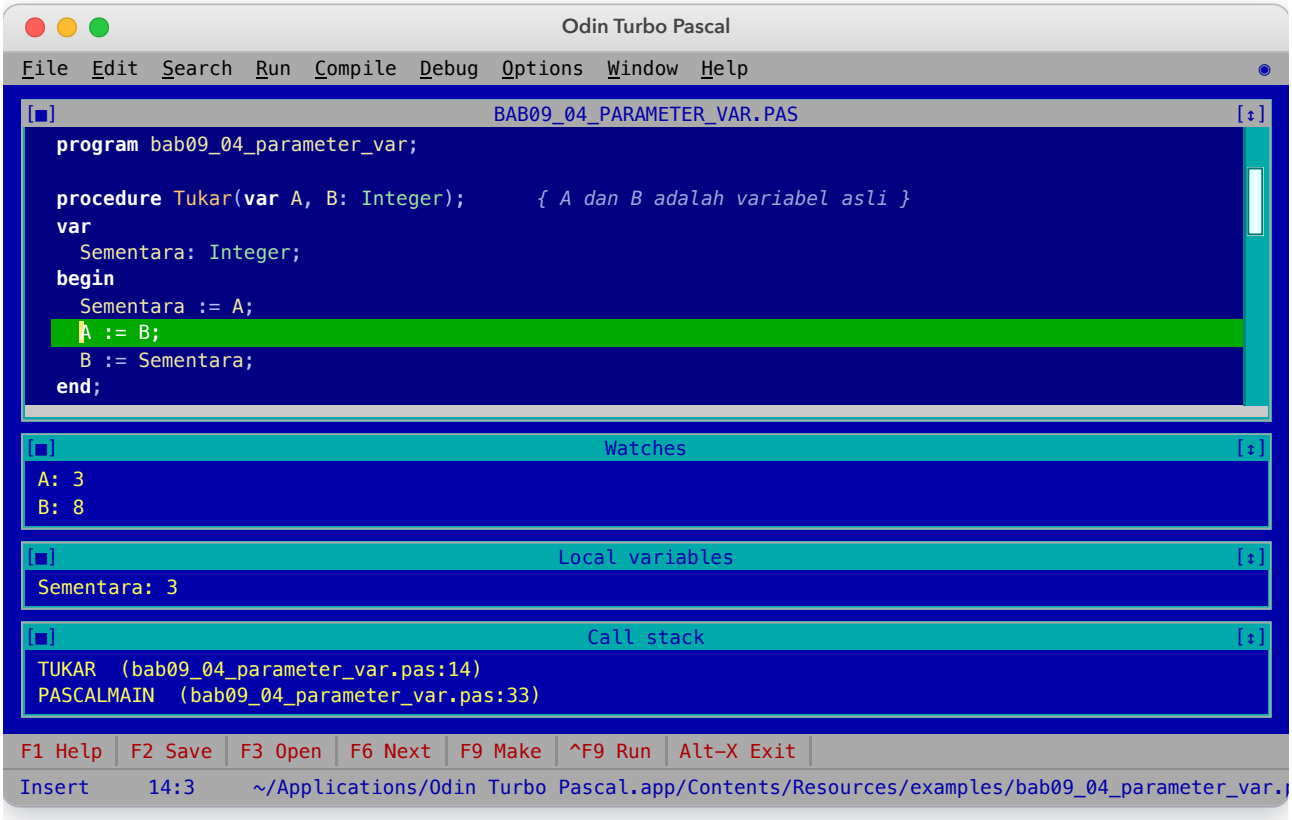
Menjalankan satu baris, tetapi bila baris itu memanggil prosedur, Anda **masuk** dan berhenti di baris pertama prosedur. Pakai bila Anda ingin melihat isinya.

Mari lacak Program 9.4. Pasang titik henti di baris 33 (`Debug ▸ Toggle breakpoint` atau Ctrl + F8), lalu mulai dengan `Debug ▸ Start debugging`. Program berhenti di baris 33, tepat sebelum `Tukar(X, Y)` dijalankan.



Gambar 9.9 Berhenti di baris 33. F8 akan menyelesaikan `Tukar` sekaligus dan berhenti di baris 34; F7 akan masuk ke dalam `Tukar`

Tekan **F7** dan program masuk ke `Tukar`. Tekan **F7** atau **F8** beberapa kali sampai baris 14 ditandai hijau. Buka `Debug ▸ Local variables` dan `Debug ▸ Call stack` untuk melihat keadaan di dalam prosedur: variabel lokalnya, dan rantai subprogram yang sedang aktif.



Gambar 9.10 Di dalam `Tukar`: `Sementara` sudah menyimpan 3, tetapi `A := B` (baris 14) belum dijalankan

Perhatikan tiga jendela di bawah editor. **Watches** memperlihatkan `A` dan `B` bernilai 3 dan 8: itulah isi `X` dan `Y`, karena `A` dan `B` hanyalah nama lain mereka. **Local variables** menampilkan `Sementara` yang baru berisi 3. **Call stack** menyebut `Tukar` di puncak dan program utama di bawahnya, beserta baris pemanggilnya (33). Tekan F8 tiga kali lagi dan Anda akan melihat `A` menjadi 8 dan `B` menjadi 3 sebelum prosedur kembali ke baris 34.

> ▶ **COBA SENDIRI**
>
> Ulangi pelacakan pada Program 9.3 dengan titik henti di baris 30. Masuk ke `Gandakan` dengan F7, lalu perhatikan Watches: setelah baris 11 dijalankan, `X` berubah menjadi 10, tetapi `Angka` tetap 5. Itulah perbedaan parameter nilai dan `var` yang Anda lihat dengan mata sendiri.

9.7 Latihan Terpadu: Kalkulator Modular

Sekarang kita gabungkan semuanya dalam program yang sedikit lebih besar. Kalkulator berikut dipecah menjadi tiga subprogram dengan satu tugas masing-masing: `TampilkanMenu` (prosedur), `BacaBilangan` (fungsi yang menampilkan pesan lalu membaca satu bilangan), dan `Hitung` (fungsi yang menghitung sekaligus melaporkan apakah operasinya berhasil). Program utamanya tinggal berupa perulangan `repeat` yang rapi.

Perhatikan rancangan `Hitung`. Hasil hitungannya bisa berupa bilangan `Real` apa saja, termasuk nol atau negatif, sehingga tidak ada nilai istimewa yang bisa menandakan "gagal". Maka fungsi mengembalikan `Boolean` sebagai status (berhasil atau tidak), sedangkan angka hasilnya dikeluarkan lewat parameter `var Hasil`. Pola "status lewat fungsi, data lewat parameter `var`" sangat lazim di Pascal.

Program 9.6 `bab09_06_kalkulator_modular.pas` *Latihan: kalkulator modular* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 9.6 — Latihan: kalkulator modular
3    Kalkulator dipecah menjadi prosedur dan fungsi kecil. Setiap bagian
4    punya satu tugas, sehingga program utama ringkas dan mudah dibaca. }
5
6  program bab09_06_kalkulator_modular;
7
8  procedure TampilkanMenu;
9  begin
10   WriteLn;
11   WriteLn('=== KALKULATOR ===');
12   WriteLn('1. Tambah   2. Kurang   3. Kali   4. Bagi   0. Keluar');
13   Write('Pilihan: ');
14 end;
15
16 function BacaBilangan(Pesan: string): Real;
17 var
18   X: Real;
19 begin
20   Write(Pesan);
21   ReadLn(X);
22   BacaBilangan := X;
23 end;
24
```

```
{ Mengembalikan False bila operasi tidak bisa dihitung }
function Hitung(A, B: Real; Operasi: Integer; var Hasil: Real): Boolean;
begin
  Hitung := True;
  case Operasi of
    1: Hasil := A + B;
    2: Hasil := A - B;
    3: Hasil := A * B;
    4: if B = 0 then
         Hitung := False                { pembagian dengan nol ditolak }
       else
         Hasil := A / B;
  end;
end;

var
  Pilihan: Integer;
  A, B, Hasil: Real;

begin
  repeat
    TampilkanMenu;
    ReadLn(Pilihan);
    if (Pilihan >= 1) and (Pilihan <= 4) then
    begin
      A := BacaBilangan('Bilangan pertama: ');
      B := BacaBilangan('Bilangan kedua  : ');
      if Hitung(A, B, Pilihan, Hasil) then
        WriteLn('Hasil: ', Hasil:0:2)
      else
        WriteLn('Pembagi tidak boleh nol.');
    end
    else if Pilihan <> 0 then
      WriteLn('Pilihan tidak tersedia.');
  until Pilihan = 0;
  WriteLn('Sampai jumpa!');
end.
```

Odin • User screen

User screen 80×25 | Copy screen | Editor | Stop

```
=== KALKULATOR ===
1. Tambah   2. Kurang   3. Kali   4. Bagi   0. Keluar
Pilihan: 4
Bilangan pertama: 12
Bilangan kedua  : 0
Pembagi tidak boleh nol.

=== KALKULATOR ===
1. Tambah   2. Kurang   3. Kali   4. Bagi   0. Keluar
Pilihan: 4
Bilangan pertama: 12
Bilangan kedua  : 4
Hasil: 3.00

=== KALKULATOR ===
1. Tambah   2. Kurang   3. Kali   4. Bagi   0. Keluar
Pilihan: 0
Sampai jumpa!


[Process finished, exit code 0]
```

Gambar 9.11 Pembagian 12 dengan 0 ditolak, lalu 12 dibagi 4 menghasilkan 3.00

Baris	Penjelasan
8–14	`TampilkanMenu` hanya mencetak. Baris 13 memakai `Write`, bukan `WriteLn`, agar kursor tinggal di sebelah `Pilihan:`.
16–23	`BacaBilangan` menerima teks pesan sebagai parameter nilai bertipe `string`, membaca ke variabel lokal `X`, dan mengembalikannya lewat nama fungsi di baris 22.
26	Judul `Hitung`: dua masukan `Real`, kode operasi, satu keluaran `var`, dan hasil `Boolean` sebagai status.
28	Hasil status diisi `True` lebih dulu (anggap berhasil). Hanya kasus yang gagal yang mengubahnya.
29–37	`case` memilih operasi. Pada operasi 4, pembagi nol diperiksa lebih dulu di baris 33; bila nol, status menjadi `False` dan `Hasil` tidak disentuh.
40–42	Variabel global program utama. Semua kerja di subprogram memakai parameter, bukan variabel ini.
45–59	Perulangan `repeat` menampilkan menu terus sampai `Pilihan = 0`.
48–56	Pilihan 1 sampai 4 membaca dua bilangan (baris 50–51) lalu memanggil `Hitung` di dalam syarat `if` (baris 52): hasil `True` / `False` fungsi langsung menjadi syaratnya.
57–58	Pilihan lain selain 0 ditolak dengan pesan.

COBA SENDIRI

Tambahkan operasi 5 untuk pangkat dua: buat fungsi `Kuadrat(X: Real): Real`, perluas menu di `TampilkanMenu`, dan tambahkan satu cabang di `case`. Perhatikan bahwa program utama nyaris tidak berubah; itulah keuntungan pemecahan menjadi subprogram.

9.8 Kesalahan yang Sering Terjadi

9.8.1 Argumen angka untuk parameter var

Parameter `var` membutuhkan variabel yang bisa diubah. Angka, ekspresi, atau hasil fungsi bukan variabel, jadi kompiler menolaknya. Ini yang dimaksud komentar di akhir Program 9.4.

tukar.pas

```
1  program Tukar;
2  procedure Tukar(var A, B: Integer);
3  var
4    T: Integer;
5  begin
6    T := A;
7    A := B;
8    B := T;
9  end;
10
11 begin
12   Tukar(3, 8);
13 end.
```

PESAN KOMPILER

```
tukar.pas(12,9) Error: Identifier variabel diharapkan
```

Gambar 9.12 Angka 3 dan 8 tidak bisa menjadi argumen untuk parameter var

Perbaikannya: simpan angka itu dulu ke variabel, lalu kirim variabelnya.

9.8.2 Memakai Result di Turbo Pascal 7

Pemrogram yang terbiasa dengan bahasa lain sering menulis `Result := ...`. Di Turbo Pascal 7, nama `Result` tidak dikenal, sehingga dianggap variabel yang belum dideklarasikan.

result.pas

```
1  program Hasil;
2  function Dua(X: Integer): Integer;
3  begin
4    Result := X * 2;
5  end;
6
7  begin
8    WriteLn(Dua(4));
9  end.
```

PESAN KOMPILER

```
result.pas(4,3) Error: Identifier tidak dikenal: Result
```

Gambar 9.13 Nilai fungsi harus diberikan lewat nama fungsi, bukan Result

Perbaikannya: ganti `Result` dengan nama fungsinya, `Dua := X * 2;`.

9.8.3 Memakai variabel lokal di luar subprogramnya

Variabel lokal hanya dikenal di dalam subprogram tempat ia dideklarasikan. Memakainya dari program utama, seperti komentar di akhir Program 9.5 mengingatkan, ditolak kompiler.

lingkup.pas

```
1  program Lingkup;
2  procedure Hitung;
3  var
4    Total: Integer;
5  begin
6    Total := 10;
7  end;
8
9  begin
10   Hitung;
11   WriteLn(Total);
12 end.
```

PESAN KOMPILER

```
lingkup.pas(11,11) Error: Identifier tidak dikenal: Total
```

Gambar 9.14 Total hanya dikenal di dalam prosedur Hitung

Perbaikannya: kirim hasilnya keluar lewat parameter `var` atau ubah `Hitung` menjadi fungsi. Jangan "memperbaiki" dengan menjadikan `Total` global tanpa pikir panjang.

9.8.4 Kesalahan yang tidak ditolak kompiler

- **Fungsi yang tidak mengisi namanya** pada salah satu jalur. Misalnya `if A > B then Terbesar := A;` tanpa `else`: bila `A` tidak lebih besar, hasil fungsi tidak terdefinisi. Pastikan setiap jalur mengisi hasil.
- **Lupa kata** `var` sehingga perubahan "hilang" begitu prosedur selesai. Bila variabel pemanggil tidak berubah padahal prosedur sudah dijalankan, periksa judul prosedurnya.
- **Argumen tertukar urutannya** walaupun tipenya cocok, misalnya dua bilangan bulat. Kompiler tidak bisa menebak maksud Anda; hasil hitungannya saja yang akan aneh.

9.9 Rangkuman

- Subprogram memecah program menjadi bagian bernama: lebih terbaca, tidak berulang, dan mudah diuji. Subprogram harus ditulis **sebelum** dipakai.
- `procedure` mengerjakan tindakan dan dipanggil sebagai pernyataan; `function` menghasilkan nilai dan dipanggil di dalam ekspresi.
- Di Turbo Pascal 7 hasil fungsi diberikan lewat **nama fungsi** (`NamaFungsi := nilai`). Tidak ada `Result`.
- **Parameter nilai** menerima salinan: perubahan di dalam tidak memengaruhi pemanggil, dan argumennya boleh ekspresi.

- **Parameter `var`** adalah nama lain bagi variabel pemanggil: perubahan terlihat di luar, dan argumennya wajib variabel.
- Variabel **lokal** hanya dikenal dan hidup di dalam subprogramnya; variabel **global** dikenal di seluruh program. Nama lokal menutupi nama global yang sama.
- Utamakan parameter daripada variabel global. Bila perlu beberapa hasil, pakai prosedur dengan parameter `var` atau fungsi berstatus `Boolean` plus `var`.
- Di debugger, F8 melewati pemanggilan subprogram, F7 masuk ke dalamnya; Watches, Local variables, dan Call stack menunjukkan keadaan di dalamnya.

9.10 Latihan

1. **DASAR** Buat prosedur `Salam` tanpa parameter yang mencetak "Selamat datang di Odin Turbo Pascal!" lalu panggil tiga kali dari program utama.
2. **DASAR** Buat fungsi `Tengah(A, B: Integer): Integer` yang mengembalikan rata-rata bulat dua bilangan (`(A + B) div 2`). Tulis juga fungsi `Positif(N: Integer): Boolean`.
3. **MENENGAH** Tulis prosedur `CetakSegitiga(Tinggi: Integer)` yang mencetak segitiga bintang. Pakai kembali ide `CetakBaris` pada Program 9.3.
4. **MENENGAH** Buat prosedur `Urutkan2(var A, B: Integer)` yang memastikan `A <= B`, menukar bila perlu. Panggil tiga kali berurutan untuk mengurutkan tiga bilangan.
5. **MENENGAH** Buat fungsi `NilaiHuruf(Nilai: Integer): Char` memakai `case` dengan rentang (ingat Bab 6). Program utama membaca nilai dan mencetak hurufnya.
6. **TANTANGAN** Buat prosedur `PecahDetik(Total: LongInt; var Jam, Menit, Detik: Integer)` yang menguraikan total detik. Uji dengan 3725 detik, lalu telusuri dengan F7 untuk memastikan nilai tiap parameter `var`.
7. **TANTANGAN** Ubah Program 9.5 agar `Tambah` menerima parameter `var Cacah: Integer` alih-alih memakai variabel global. Apa saja yang harus diubah pada pemanggilnya, dan apa keuntungannya?

BAB 10

Rekursi

TUJUAN PEMBELAJARAN

- Menjelaskan apa itu rekursi dan mengapa setiap fungsi rekursif membutuhkan kasus dasar dan kasus rekursif.
- Menelusuri tumpukan pemanggilan (*call stack*) saat fungsi rekursif memanggil dirinya sendiri dan saat hasilnya kembali.
- Menulis fungsi rekursif untuk faktorial, deret Fibonacci, dan pangkat.
- Membandingkan banyaknya pemanggilan antara dua rancangan rekursif, dan mengenali mengapa Fibonacci rekursif lambat.
- Memecahkan teka-teki Menara Hanoi dengan satu prosedur rekursif yang pendek.
- Mengamati Call stack di debugger dan mengenali kesalahan khas rekursi: kasus dasar hilang dan tumpukan penuh.

PROGRAM DALAM BAB INI

10.1	`bab10_01_faktorial_rekursif.pas`	Faktorial secara rekursif
10.2	`bab10_02_fibonacci_rekursif.pas`	Fibonacci secara rekursif
10.3	`bab10_03_pangkat_rekursif.pas`	Pangkat secara rekursif
10.4	`bab10_04_menara_hanoi.pas`	Menara Hanoi

Di Bab 9 Anda belajar bahwa sebuah subprogram boleh memanggil subprogram lain. Bab ini mengajukan pertanyaan yang tampak aneh: bolehkah sebuah subprogram memanggil **dirinya sendiri**? Boleh, dan teknik itu disebut **rekursi**. Rekursi bukan sekadar trik. Banyak masalah, mulai dari menghitung faktorial sampai menjelajahi folder di komputer, berbentuk "masalah besar yang tersusun dari masalah yang sama tetapi lebih kecil", dan rekursi adalah cara paling alami untuk menuliskannya. Anda akan mempelajarinya lewat empat program, dari yang paling sederhana sampai teka-teki Menara Hanoi.

10.1 Berpikir Secara Rekursif

Bayangkan Anda berdiri di antrean panjang dan ingin tahu Anda urutan ke berapa. Anda bertanya kepada orang di depan, "Kamu urutan berapa?", lalu ia bertanya kepada orang di depannya dengan pertanyaan yang persis sama, dan seterusnya sampai pertanyaan itu tiba di orang pertama. Orang pertama tidak perlu bertanya: ia langsung menjawab "satu". Jawaban itu lalu merambat ke belakang, ditambah satu oleh setiap orang, sampai tiba di Anda. Itulah rekursi: masalah yang sama diteruskan ke versi yang **lebih kecil**, sampai tiba pada kasus yang begitu kecil sehingga jawabannya langsung diketahui.

Setiap pemecahan rekursif memiliki dua bagian yang tidak boleh hilang:

1 Kasus dasar (*base case*)

Kasus paling kecil yang jawabannya diberikan langsung, tanpa memanggil diri sendiri. Inilah rem yang menghentikan rekursi. Pada antrean tadi: orang pertama menjawab "satu".

2 Kasus rekursif

Memecah masalah menjadi masalah yang **lebih kecil** dan memanggil diri sendiri untuknya, lalu menggabungkan hasilnya. Pada antrean tadi: "posisiku = posisi orang di depanku + 1".



AWAS

Bila kasus dasar tidak ada, atau masalahnya tidak pernah mengecil menuju kasus dasar, fungsi akan memanggil dirinya tanpa henti sampai memori tumpukan habis dan program dihentikan. Setiap kali Anda menulis fungsi rekursif, tanyakan dua hal: "Di mana kasus dasarnya?" dan "Apakah parameter pada pemanggilan berikutnya lebih dekat ke kasus dasar?"

10.2 Faktorial: Rekursi yang Paling Klasik

Faktorial sebuah bilangan bulat tak negatif *N*, ditulis *N*!, adalah hasil kali semua bilangan bulat dari 1 sampai *N*. Jadi 4! = 4 × 3 × 2 × 1 = 24. Perhatikan bahwa 4! sama dengan 4 × 3!, dan 3! sama dengan 3 × 2!. Setiap faktorial tersusun dari faktorial yang lebih kecil, dan itu persis bentuk rekursif:

BENTUK UMUM

```
N! = N × (N − 1)!      untuk N > 1      { kasus rekursif }
0! = 1! = 1                             { kasus dasar }
```

Penerjemahannya ke Pascal hampir kata per kata. Ingat aturan Turbo Pascal 7 dari Bab 9: hasil fungsi diberikan lewat nama fungsi. Namun nama fungsi punya dua peran yang harus dibedakan. Di **sisi kiri** `:=`, `Faktorial := ...` menetapkan hasil. Di **sisi kanan**, `Faktorial(N − 1)` **memanggil** fungsi itu lagi.

7 ATURAN TURBO PASCAL 7

Tidak ada `Result` di Turbo Pascal 7, jadi tidak ada cara lain untuk menetapkan hasil selain lewat nama fungsi. Di dalam fungsi rekursif kedua peran nama itu bertemu dalam satu baris: `Faktorial := N * Faktorial(N - 1)`. Baca dari kanan ke kiri: panggil dulu `Faktorial(N − 1)`, kalikan dengan `N`, baru hasilnya ditetapkan.

Program 10.1 berisi dua versi. `Faktorial` adalah versi ringkas. `FaktorialJejak` adalah versi yang sama, tetapi mencetak pesan setiap kali masuk dan keluar, dengan jorokan sesuai kedalaman pemanggilan, sehingga Anda dapat **melihat** rekursi berlangsung. Program ini memanggil versi jejak untuk 4, lalu mencetak 0! sampai 12!. Buka `bab10_01_faktorial_rekursif.pas` lewat `File ▸ Examples...` dan jalankan.

Program 10.1 `bab10_01_faktorial_rekursif.pas` *Faktorial secara rekursif* File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 10.1 — Faktorial secara rekursif
3    Fungsi rekursif memanggil dirinya sendiri dengan masalah yang lebih
4    kecil. Wajib ada kasus dasar yang menghentikan pemanggilan.
5    N! = N x (N-1)!, dan 0! = 1! = 1. }
6
7  program bab10_01_faktorial_rekursif;
8
9  function Faktorial(N: Integer): LongInt;
10 begin
11   if N <= 1 then
12     Faktorial := 1                    { kasus dasar: berhenti di sini }
13   else
14     Faktorial := N * Faktorial(N - 1);   { kasus rekursif }
15 end;
16
17 { Versi yang sama, tetapi mencetak jejak setiap pemanggilan.
18   Tingkat dipakai untuk menjorokkan tulisan sesuai kedalaman. }
19 function FaktorialJejak(N, Tingkat: Integer): LongInt;
20 var
21   Hasil: LongInt;
22 begin
```

```
23    WriteLn(''':Tingkat * 2, 'masuk Faktorial(', N, ')');
24    if N <= 1 then
25      Hasil := 1
26    else
27      Hasil := N * FaktorialJejak(N - 1, Tingkat + 1);
28    WriteLn(''':Tingkat * 2, 'Faktorial(', N, ') = ', Hasil);
29    FaktorialJejak := Hasil;
30  end;
31
32  var
33    I: Integer;
34    Hasil: LongInt;
35
36  begin
37    Hasil := FaktorialJejak(4, 0);
38    WriteLn;
39    for I := 0 to 12 do                    { 13! sudah melebihi LongInt }
40      WriteLn(I:2, '! = ', Faktorial(I):10);
41  end.
```

Odin • User screen

User screen 80×25 | Copy screen | Editor | Stop

```
masuk Faktorial(4)
  masuk Faktorial(3)
    masuk Faktorial(2)
      masuk Faktorial(1)
      Faktorial(1) = 1
    Faktorial(2) = 2
  Faktorial(3) = 6
Faktorial(4) = 24

 0! =          1
 1! =          1
 2! =          2
 3! =          6
 4! =         24
 5! =        120
 6! =        720
 7! =       5040
 8! =      40320
 9! =     362880
10! =    3628800
11! =   39916800
12! =  479001600


[Process finished, exit code 0]
```

Gambar 10.1 Jejak `FaktorialJejak(4, 0)` disusul tabel 0! sampai 12!

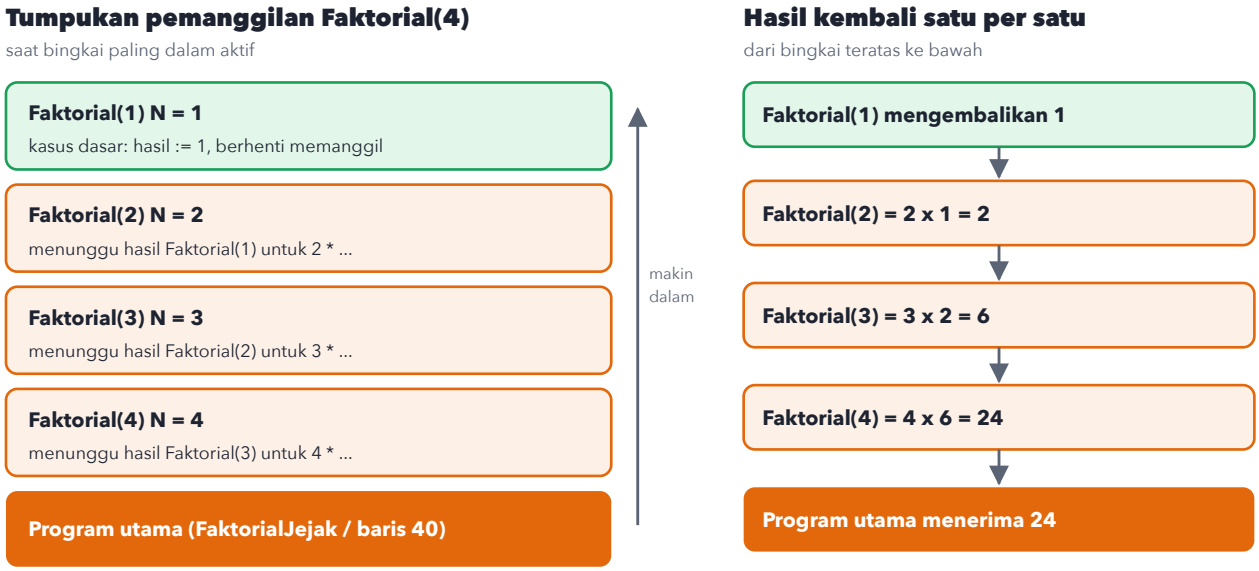
Baris	Penjelasan
9	Parameter `N` bertipe `Integer`, tetapi hasilnya `LongInt`, karena faktorial cepat membesar.
11–12	Kasus dasar: untuk `N` sama dengan 0 atau 1 hasilnya 1 dan fungsi **tidak** memanggil dirinya lagi. Memakai `N <= 1` (bukan `N = 1`) juga menangani 0! dan melindungi dari masukan negatif.
14	Kasus rekursif. `N - 1` lebih kecil dari `N`, jadi setiap pemanggilan selangkah lebih dekat ke kasus dasar.

Baris	Penjelasan
19–21	`FaktorialJejak` punya parameter kedua `Tingkat` (kedalaman) dan variabel lokal `Hasil`. Karena `Hasil` lokal, setiap pemanggilan memiliki `Hasil` miliknya sendiri.
23	`WriteLn('':Tingkat * 2, ...)` mencetak teks kosong dengan lebar `Tingkat * 2`, yaitu cara membuat jorokan. Pesan "masuk" dicetak **sebelum** memanggil diri sendiri.
24–27	Kasus dasar dan kasus rekursif, kali ini hasilnya disimpan ke `Hasil`. Pada baris 27 pemanggilan berikutnya memakai `Tingkat + 1` agar jorokannya bertambah.
28	Pesan "keluar" dicetak **sesudah** pemanggilan rekursif kembali. Itu sebabnya pesan ini muncul dalam urutan terbalik.
29	Hasil dikembalikan lewat nama fungsi.
37	Memanggil versi jejak dengan `N = 4` pada tingkat 0.
39–40	Perulangan mencetak `I` ! untuk 0 sampai 12 dengan lebar kolom 10 agar angkanya rata kanan.

10.2.1 Apa yang terjadi di balik layar: tumpukan pemanggilan

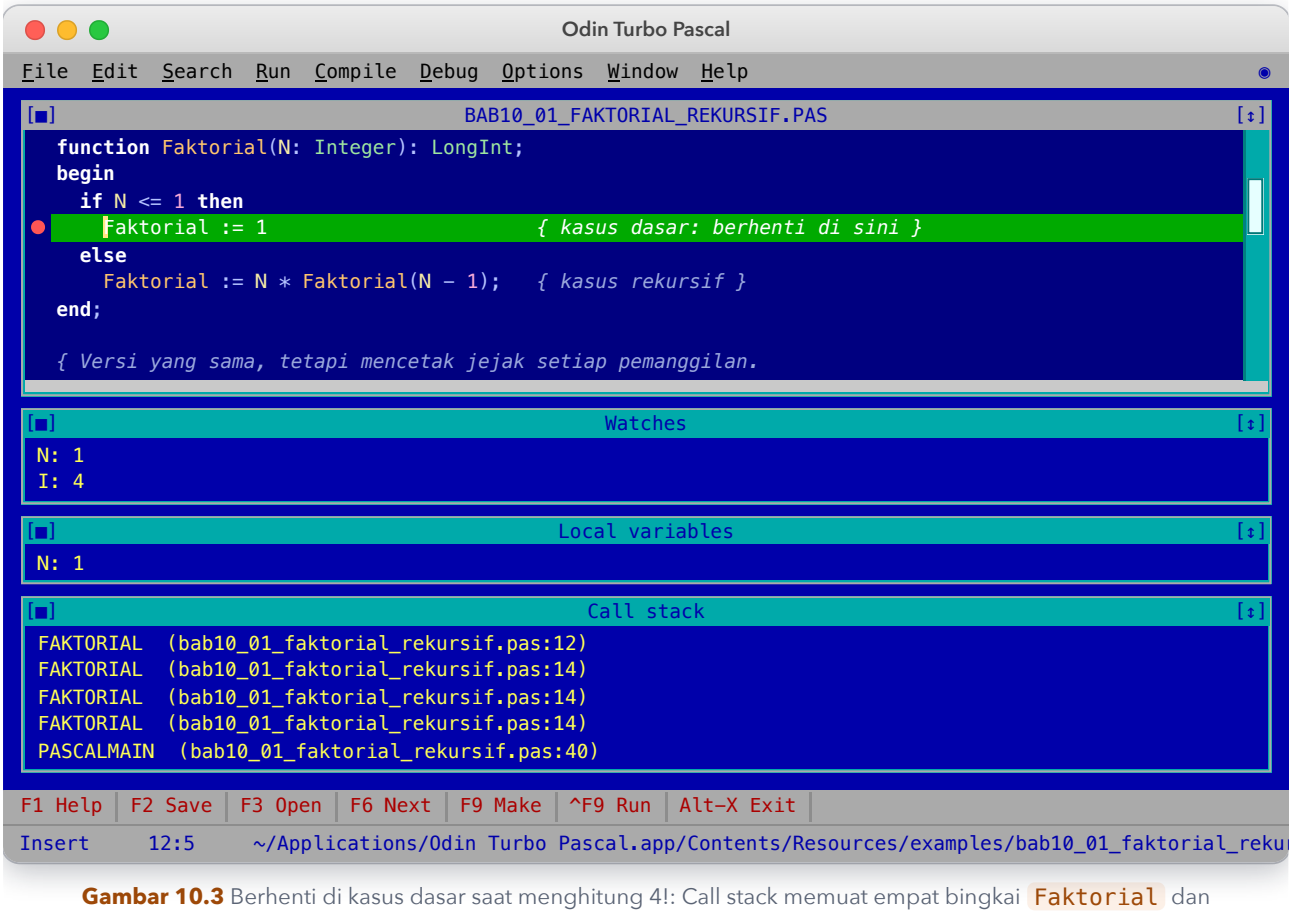
Mengapa pesan "masuk" tertulis dari 4 turun ke 1, sedangkan pesan "keluar" dari 1 naik ke 4? Jawabannya adalah **tumpukan pemanggilan** (*call stack*). Setiap kali sebuah fungsi dipanggil, komputer menyiapkan satu **bingkai** (*frame*) di tumpukan memori untuk menyimpan parameter, variabel lokal, dan tempat untuk kembali. Bingkai baru diletakkan di atas bingkai pemanggilnya, seperti piring yang ditumpuk. Fungsi yang sedang bekerja selalu yang paling atas. Saat sebuah fungsi selesai, bingkainya dibuang dan pekerjaan dilanjutkan oleh bingkai di bawahnya.

Karena setiap pemanggilan punya bingkainya sendiri, `Faktorial(4)` dan `Faktorial(3)` memiliki `N` yang berbeda walaupun namanya sama dan keduanya aktif bersamaan. Tidak ada yang saling menimpa. Di kiri **Gambar 10.2** memperlihatkan tumpukan pada saat paling dalam, ketika `Faktorial(1)` dipanggil; di kanan terlihat bagaimana hasil kembali satu per satu, dari bingkai teratas ke bawah.



Gambar 10.2 Kiri: bingkai-bingkai menumpuk saat `Faktorial` memanggil dirinya. Kanan: setiap bingkai menyelesaikan perkaliannya begitu hasil dari atasnya tiba, lalu dibuang.

Anda bisa melihat tumpukan itu langsung di debugger. Pasang titik henti di baris 12 (kasus dasar), pilih `Debug ▸ Start debugging`, lalu tekan `Debug ▸ Continue` (atau Ctrl + F9) sampai `I` di jendela Watches bernilai 4. Program berhenti di dalam pemanggilan yang paling dalam. Buka `Debug ▸ Call stack`.



Gambar 10.3 Berhenti di kasus dasar saat menghitung 4!: Call stack memuat empat bingkai `Faktorial` dan satu program utama

Empat baris `FAKTORIAL` di Call stack adalah `Faktorial(1)` di puncak, lalu `Faktorial(2)`, `(3)`, dan `(4)`. Tiga yang bawah berhenti di baris 14, karena sedang menunggu pemanggilan di dalam ekspresi perkaliannya selesai. Yang paling bawah adalah program utama, yang memanggil dari baris 40. Local variables hanya menampilkan milik bingkai teratas (`N = 1`). Klik dua kali sebuah bingkai di Call stack untuk memilihnya; Watches dan Local variables lalu menampilkan `N` milik bingkai itu.

10.2.2 Batas LongInt

Perhatikan komentar di baris 39: "13! sudah melebihi `LongInt`". Faktorial membesar sangat cepat, dan 12! = 479001600 adalah faktorial terbesar yang masih muat di `LongInt` (maksimum 2147483647). 13! kira-kira 6,2 miliar, sehingga tidak muat. Anda akan melihat akibatnya di bagian kesalahan umum di akhir bab.

24
4! = 4 x 3 x 2 x 1

479001600
12!, faktorial terbesar yang muat di LongInt

4
kedalaman tumpukan untuk menghitung 4!

COBA SENDIRI

Ubah `FaktorialJejak(4, 0)` di baris 37 menjadi `FaktorialJejak(6, 0)`. Berapa baris "masuk" yang tampil, dan berapa kedalaman jorokan terbesar? Lalu tebak lebih dulu: bila baris 28 dipindah ke atas baris 24, bagaimana urutan pesannya? Cobalah.

10.3 Fibonacci: Rumus Indah yang Lambat

Deret Fibonacci dimulai dari 1, 1, lalu setiap suku berikutnya adalah jumlah dua suku sebelumnya: 1, 1, 2, 3, 5, 8, 13, 21, ... Rumusnya langsung berbentuk rekursif, dengan **dua** kasus dasar dan **dua** pemanggilan rekursif:

BENTUK UMUM

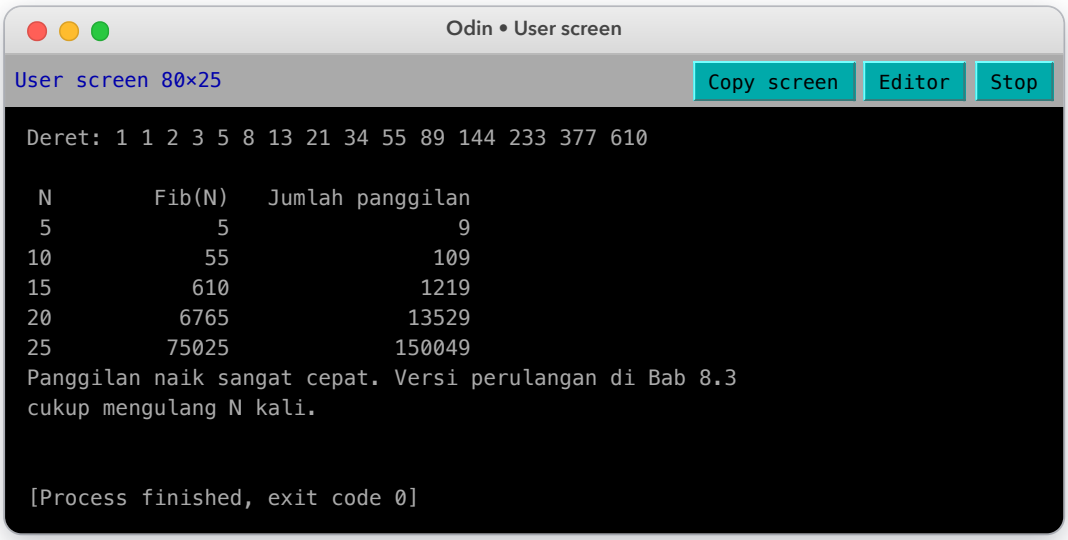
```
Fib(1) = Fib(2) = 1                    { dua kasus dasar }
Fib(N)  = Fib(N − 1) + Fib(N − 2)      untuk N > 2
```

Program 10.2 menulis deret itu dan sekaligus mengukur biayanya. Variabel global `Panggilan` bertambah satu setiap kali `Fib` dipanggil, sehingga kita bisa menghitung berapa banyak pemanggilan yang dibutuhkan untuk setiap *N*. (Variabel global dipakai di sini karena `Fib` perlu "melapor" tanpa mengubah parameternya; di Bab 9 sudah dibahas bahwa ini pengecualian, bukan kebiasaan.)

Program 10.2 `bab10_02_fibonacci_rekursif.pas` *Fibonacci secara rekursif* File ▸ Examples...

```
1  { Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 10.2 – Fibonacci secara rekursif
3    Fib(1) = Fib(2) = 1, dan Fib(N) = Fib(N–1) + Fib(N–2).
4    Rumusnya pendek, tetapi lambat: suku yang sama dihitung berulang
5    kali. Variabel Panggilan menghitung berapa kali fungsi dipanggil. }
6
7  program bab10_02_fibonacci_rekursif;
8
9  var
10   Panggilan: LongInt;                  { global: dihitung oleh Fib }
11
12 function Fib(N: Integer): LongInt;
13 begin
14   Panggilan := Panggilan + 1;
15   if N <= 2 then
16     Fib := 1                           { dua kasus dasar }
17   else
18     Fib := Fib(N - 1) + Fib(N - 2);    { dua pemanggilan rekursif }
19 end;
20
21 var
22   N: Integer;
23   Hasil: LongInt;
24
25 begin
26   Write('Deret: ');
27   for N := 1 to 15 do
28     Write(Fib(N), ' ');
```

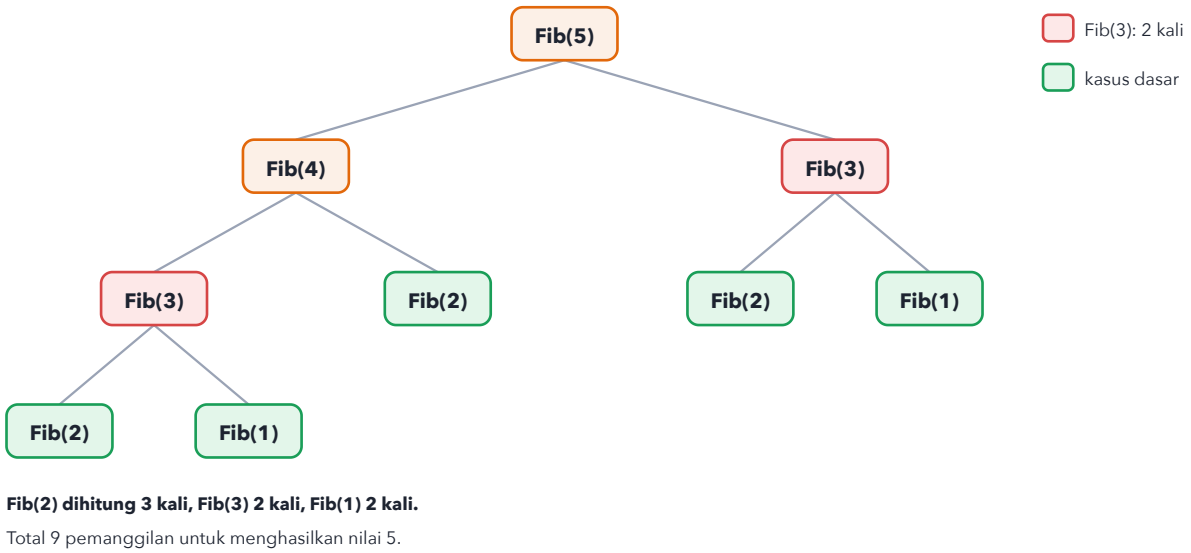
```
29    WriteLn;
30    WriteLn;
31
32    WriteLn(' N        Fib(N)   Jumlah panggilan');
33    N := 5;
34    while N <= 25 do
35    begin
36      Panggilan := 0;
37      Hasil := Fib(N);
38      WriteLn(N:2, Hasil:14, Panggilan:19);
39      N := N + 5;
40    end;
41    WriteLn('Panggilan naik sangat cepat. Versi perulangan di Bab 8.3');
42    WriteLn('cukup mengulang N kali.');
43  end.
```



Gambar 10.4 Deret Fibonacci, lalu jumlah pemanggilan untuk *N* = 5, 10, 15, 20, dan 25

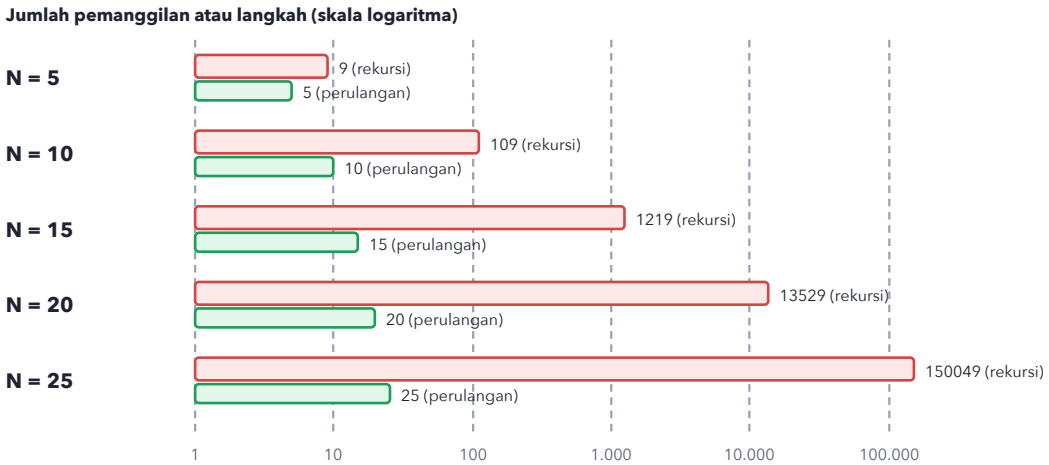
Baris	Penjelasan
10	`Panggilan` adalah variabel global bertipe `LongInt` yang menghitung pemanggilan.
14	Baris pertama `Fib`: menambah penghitung pada **setiap** pemanggilan, termasuk yang berakhir di kasus dasar.
15–16	Dua kasus dasar sekaligus: untuk `N` sama dengan 1 atau 2, hasilnya 1.
18	Dua pemanggilan rekursif dalam satu ekspresi, `Fib(N - 1)` dan `Fib(N - 2)`, lalu dijumlahkan.
26–29	Mencetak 15 suku pertama pada satu baris. Pemanggilan `Fib(N)` dilakukan di dalam `Write`.
33–40	Untuk `N` = 5, 10, ..., 25: penghitung dikosongkan (baris 36), `Fib(N)` dihitung, lalu `N`, hasil, dan jumlah pemanggilan dicetak. Lebar kolom `:2`, `:14`, `:19` membuat angkanya rata kanan.

Lihat kolom terakhir pada keluaran. Menghitung `Fib(25)` yang bernilai 75025 ternyata membutuhkan 150049 pemanggilan, dua kali lipat dari nilainya. Mengapa sebanyak itu? Karena `Fib(N - 1)` dan `Fib(N - 2)` bekerja sendiri-sendiri tanpa saling tahu, sehingga suku yang sama dihitung berulang-ulang. **Gambar 10.5** menggambar pohon pemanggilan untuk `Fib(5)`, yang hanya butuh sembilan pemanggilan.



Gambar 10.5 Pohon pemanggilan `Fib(5)`: sembilan simpul. Suku yang sama (merah dan hijau) dihitung berulang kali.

Karena setiap pemanggilan di atas kasus dasar melahirkan dua pemanggilan baru, jumlah pemanggilan hampir **berlipat ganda** setiap kali N naik satu. Untuk `Fib(N)` jumlahnya persis 2 × `Fib(N)` - 1, dan karena `Fib(N)` sendiri tumbuh secara eksponensial, waktu hitungnya meledak. Pada $N = 25$ masih tidak terasa, tetapi untuk $N = 45$ komputer harus melakukan miliaran pemanggilan. Bandingkan dengan perulangan di Bab 8.3, yang cukup mengulang N kali. Diagram batang berikut memakai skala logaritma, artinya setiap garis vertikal putus-putus mewakili jumlah yang **sepuluh kali lebih besar** dari garis sebelumnya.



Gambar 10.6 Pemanggilan `Fib(N)` secara rekursif (merah) dibandingkan dengan langkah perulangan (hijau). Selisihnya melebar cepat.

COBA SENDIRI

Ubah batas perulangan `while` pada baris 34 menjadi `N <= 35`, tambahkan tampilan untuk $N = 30$ dan 35, lalu jalankan. Ukur dengan jam tangan berapa lama program berpikir. Ini contoh nyata bahwa program yang **benar** belum tentu program yang **baik**.

10.4 Pangkat: Memecah Separuh demi Separuh

Rekursi tidak selalu mengurangi masalah satu demi satu. Kadang masalah bisa **dibelah dua**, dan itu jauh lebih hemat. Menghitung A^N ada dua cara. Cara biasa mengalikan A sebanyak N kali, jadi $A^N = A \times A^{N-1}$, dengan kasus dasar $A^0 = 1$. Cara cepat memakai fakta bahwa $A^N = (A^{N \text{ div } 2})^2$, dikali A sekali lagi bila N ganjil. Karena pangkatnya dibagi dua setiap langkah, banyaknya pemanggilan hanya sekitar $2 \log N$.

Program 10.3 mengimplementasikan keduanya dan menghitung berapa kali masing-masing dipanggil, memakai variabel global `Langkah`. Satu hal penting di `PangkatCepat`: hasil separuhnya dihitung **sekali** dan disimpan di variabel lokal `Separuh`, lalu dikalikan dengan dirinya. Bila ditulis `PangkatCepat(A, N div 2) * PangkatCepat(A, N div 2)`, kita kembali ke dua pemanggilan dan seluruh penghematannya hilang.

Program 10.3 `bab10_03_pangkat_rekursif.pas` *Pangkat secara rekursif* File ▸ Examples…

```
{ Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 10.3 - Pangkat secara rekursif
  Cara biasa : A^N = A x A^(N-1), kasus dasar A^0 = 1.
  Cara cepat : A^N = (A^(N div 2))^2, dikali A sekali lagi bila N
  ganjil. Pangkat dibagi dua setiap langkah, jadi jauh lebih sedikit
  pemanggilan. }

program bab10_03_pangkat_rekursif;

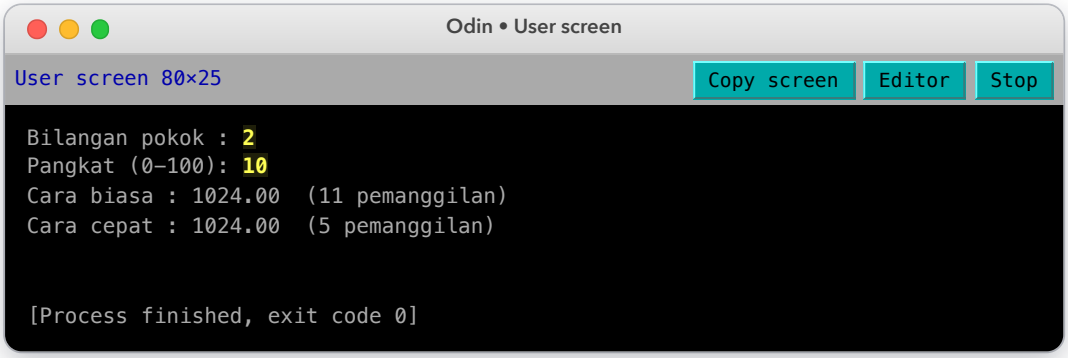
var
  Langkah: Integer;                    { menghitung pemanggilan }

function Pangkat(A: Real; N: Integer): Real;
begin
  Langkah := Langkah + 1;
  if N = 0 then
    Pangkat := 1                       { kasus dasar }
  else
    Pangkat := A * Pangkat(A, N - 1);
end;

function PangkatCepat(A: Real; N: Integer): Real;
var
  Separuh: Real;
begin
  Langkah := Langkah + 1;
  if N = 0 then
    PangkatCepat := 1
  else
  begin
    Separuh := PangkatCepat(A, N div 2); { dihitung sekali saja }
    if N mod 2 = 0 then
      PangkatCepat := Separuh * Separuh
    else
      PangkatCepat := Separuh * Separuh * A;
  end;
end;

var
  Basis, Hasil: Real;
```

```
41    N: Integer;
42
43  begin
44    Write('Bilangan pokok : '); ReadLn(Basis);
45    repeat
46      Write('Pangkat (0-100): '); ReadLn(N);
47    until (N >= 0) and (N <= 100);
48
49    Langkah := 0;
50    Hasil := Pangkat(Basis, N);            { hitung dulu, Langkah ikut terisi }
51    WriteLn('Cara biasa : ', Hasil:0:2, '  (', Langkah, ' pemanggilan)');
52
53    Langkah := 0;
54    Hasil := PangkatCepat(Basis, N);
55    WriteLn('Cara cepat : ', Hasil:0:2, '  (', Langkah, ' pemanggilan)');
56  end.
```



Gambar 10.7 2 pangkat 10: cara biasa 11 pemanggilan, cara cepat hanya 5

Coba pangkat yang lebih besar untuk melihat jurang antara keduanya.

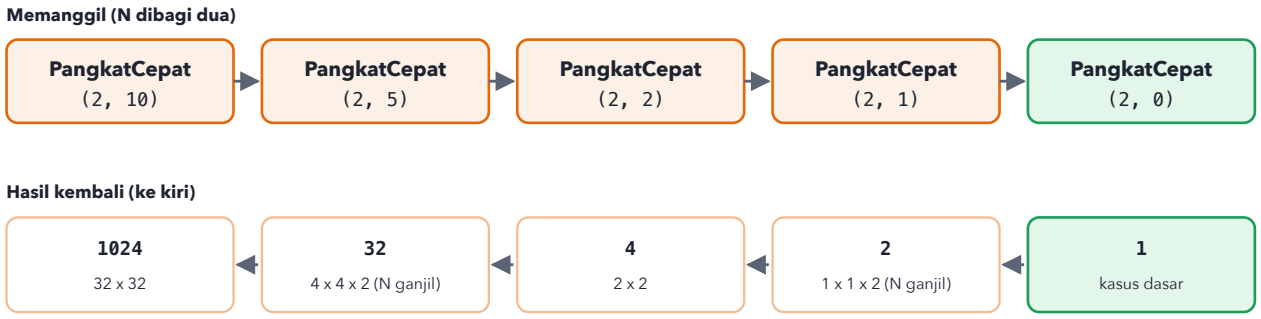


Gambar 10.8 2 pangkat 30: cara biasa 31 pemanggilan, cara cepat 6

Baris	Penjelasan
11	`Langkah` menghitung jumlah pemanggilan, dikosongkan sebelum setiap cara dijalankan (baris 49 dan 53).
13–20	`Pangkat` : cara biasa. Kasus dasar `N = 0` memberi 1. Kasus rekursif memanggil dengan `N - 1` : pangkat turun satu demi satu, sehingga untuk N ada $N + 1$ pemanggilan.
22–37	`PangkatCepat` : cara cepat. Kasus dasar yang sama di baris 27–28.
31	Memanggil dengan `N div 2` dan menyimpan hasilnya di `Separuh` . `div` adalah pembagian bulat, jadi 5 div 2 = 2.

Baris	Penjelasan
32–35	Bila `N` genap, hasilnya `Separuh * Separuh`. Bila ganjil, ada satu faktor `A` tersisa yang harus dikalikan lagi.
44–47	Membaca bilangan pokok, lalu pangkat 0 sampai 100 dengan `repeat..until` yang mengulang pertanyaan bila masukan di luar rentang.
49–55	Kedua cara dijalankan, dan hasil serta jumlah pemanggilan dicetak berdampingan. Hasil `Real` diformat `:0:2`.

Gambar 10.9 menelusuri `PangkatCepat(2, 10)`. Rantai pemanggilan menjalar ke kanan: 10, 5, 2, 1, lalu 0 (kasus dasar). Hanya lima pemanggilan, dibanding sebelas pada cara biasa. Hasilnya lalu kembali ke kiri sambil dikuadratkan, dan dikalikan 2 pada pangkat ganjil.



Lima pemanggilan menghasilkan 2^10 = 1024; cara biasa membutuhkan sebelas.

Gambar 10.9 Penelusuran `PangkatCepat(2, 10)`: pangkat terus dibelah dua sampai 0, lalu hasil dikuadratkan kembali.

Tabel 10.1 Banyaknya pemanggilan untuk menghitung 2^N (dari keluaran Program 10.3)

Pangkat *N*	Cara biasa (`Pangkat`)	Cara cepat (`PangkatCepat`)
10	11 pemanggilan	5 pemanggilan
30	31 pemanggilan	6 pemanggilan
100	101 pemanggilan	8 pemanggilan

> ★ **INGAT**
>
> Dua program, dua pelajaran. Pada **Fibonacci** rekursi membuat pekerjaan berlipat ganda karena masalah yang sama dihitung berulang. Pada **pangkat** rekursi justru menghemat pekerjaan karena masalah dibelah dua dan hasil separuhnya dipakai ulang. Rekursi bukan otomatis lambat atau cepat; yang menentukan adalah bagaimana masalah dipecah.

10.5 Menara Hanoi: Teka-teki yang Dipecahkan Rekursi

Menara Hanoi adalah teka-teki dengan tiga tiang dan sejumlah cakram berukuran berbeda yang bertumpuk di tiang pertama, yang terbesar di dasar. Tujuannya memindahkan seluruh tumpukan ke tiang lain dengan dua aturan: **satu cakram** per langkah, dan cakram besar **tidak boleh** diletakkan di atas

cakram yang lebih kecil. Tanpa rekursi, teka-teki ini membuat pemula kebingungan. Dengan rekursi, pemecahannya hanya tiga langkah. Untuk memindahkan *N* cakram dari tiang **Asal** ke tiang **Tujuan** dengan bantuan tiang **Bantu**:

1. **Pindahkan *N* - 1 cakram atas** dari Asal ke Bantu (masalah yang lebih kecil, diserahkan ke rekursi).
2. **Pindahkan cakram terbesar** dari Asal ke Tujuan (satu langkah nyata).
3. **Pindahkan *N* - 1 cakram** dari Bantu ke Tujuan (sekali lagi masalah yang lebih kecil).

Kasus dasarnya adalah nol cakram: tidak ada yang perlu dipindahkan. Kuncinya adalah **percaya** bahwa pemanggilan rekursif akan memindahkan *N* - 1 cakram dengan benar, tanpa Anda perlu membayangkan bagaimana. Program 10.4 menuliskan ketiga langkah itu dan mencetak setiap pemindahan.

7 **ATURAN TURBO PASCAL 7**

Prosedur `Hanoi` memakai `Exit` sebagai kasus dasar (baris 17–18). Di Turbo Pascal 7 `Exit` tidak membawa nilai: ia hanya mengakhiri prosedur atau fungsi yang sedang berjalan dan kembali ke pemanggilnya. Pada fungsi, nilai hasilnya tetap harus diberikan lewat nama fungsi sebelum `Exit`.

Program 10.4 `bab10_04_menara_hanoi.pas` *Menara Hanoi* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 10.4 — Menara Hanoi
3    Pindahkan N cakram dari tiang A ke tiang C dengan bantuan tiang B.
4    Hanya satu cakram dipindah per langkah, dan cakram besar tidak boleh
5    berada di atas cakram kecil. Pemecahan rekursifnya:
6      1. pindahkan N-1 cakram atas dari Asal ke Bantu
7      2. pindahkan cakram terbesar dari Asal ke Tujuan
8      3. pindahkan N-1 cakram dari Bantu ke Tujuan }
9
10 program bab10_04_menara_hanoi;
11
12 var
13   JumlahLangkah: LongInt;
14
15 procedure Hanoi(N: Integer; Asal, Tujuan, Bantu: Char);
16 begin
17   if N = 0 then
18     Exit;                              { kasus dasar: keluar dari prosedur }
19   Hanoi(N - 1, Asal, Bantu, Tujuan);     { langkah 1 }
20   JumlahLangkah := JumlahLangkah + 1;
21   WriteLn(JumlahLangkah:4, '. Cakram ', N, ': ', Asal, ' -> ', Tujuan);
22   Hanoi(N - 1, Bantu, Tujuan, Asal);     { langkah 3 }
23 end;
24
25 var
26   N: Integer;
27
28 begin
29   repeat
30     Write('Jumlah cakram (1-8): ');
31     ReadLn(N);
```

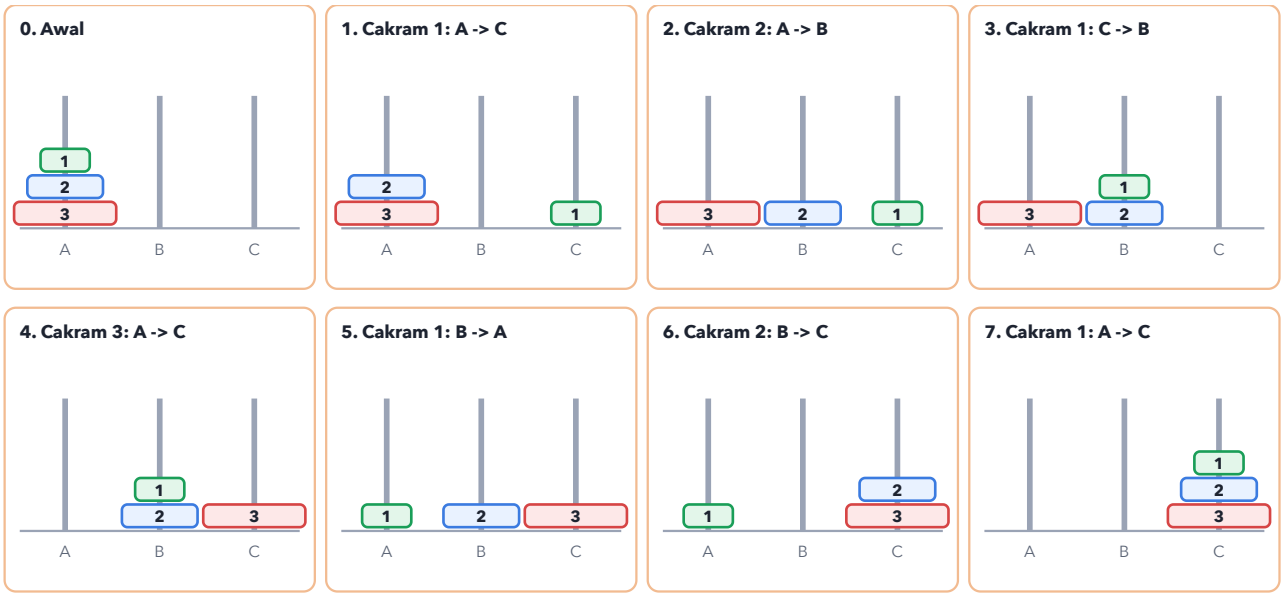
```
32    until (N >= 1) and (N <= 8);
33
34    JumlahLangkah := 0;
35    Hanoi(N, 'A', 'C', 'B');
36    WriteLn('Selesai dalam ', JumlahLangkah, ' langkah (2^N - 1).');
37  end.
```



Gambar 10.10 Tiga cakram dipindahkan dari A ke C dalam tujuh langkah

Baris	Penjelasan
13	`JumlahLangkah` adalah penghitung global pemindahan cakram.
15	Parameter `N` adalah banyak cakram, dan tiga `Char` adalah nama tiang. Perhatikan urutan **Asal, Tujuan, Bantu**.
17–18	Kasus dasar: bila tidak ada cakram, `Exit` segera mengakhiri prosedur.
19	Langkah 1: pindahkan `N - 1` cakram dari `Asal` ke `Bantu`. Di sini peran tiang **bertukar**: yang tadinya Tujuan kini menjadi Bantu.
20–21	Langkah 2, satu-satunya pemindahan nyata: penghitung bertambah dan baris pemindahan dicetak.
22	Langkah 3: pindahkan `N - 1` cakram dari `Bantu` ke `Tujuan`. Perannya bertukar lagi.
29–32	Meminta jumlah cakram 1 sampai 8 dan mengulang pertanyaan selama masukan di luar rentang. Batas 8 menjaga keluaran tetap pendek (255 langkah).
34–36	Penghitung dikosongkan, `Hanoi` dipanggil dengan A sebagai asal, C tujuan, B bantu, lalu total langkah dicetak.

Gambar 10.11 menggambar keadaan tiang setelah setiap langkah pada keluaran di atas. Perhatikan panel 4: cakram terbesar baru berpindah setelah dua cakram di atasnya menyingkir ke tiang B. Perhatikan juga panel 7: dua cakram yang tadi disingkirkan dipindahkan lagi ke atas cakram terbesar.



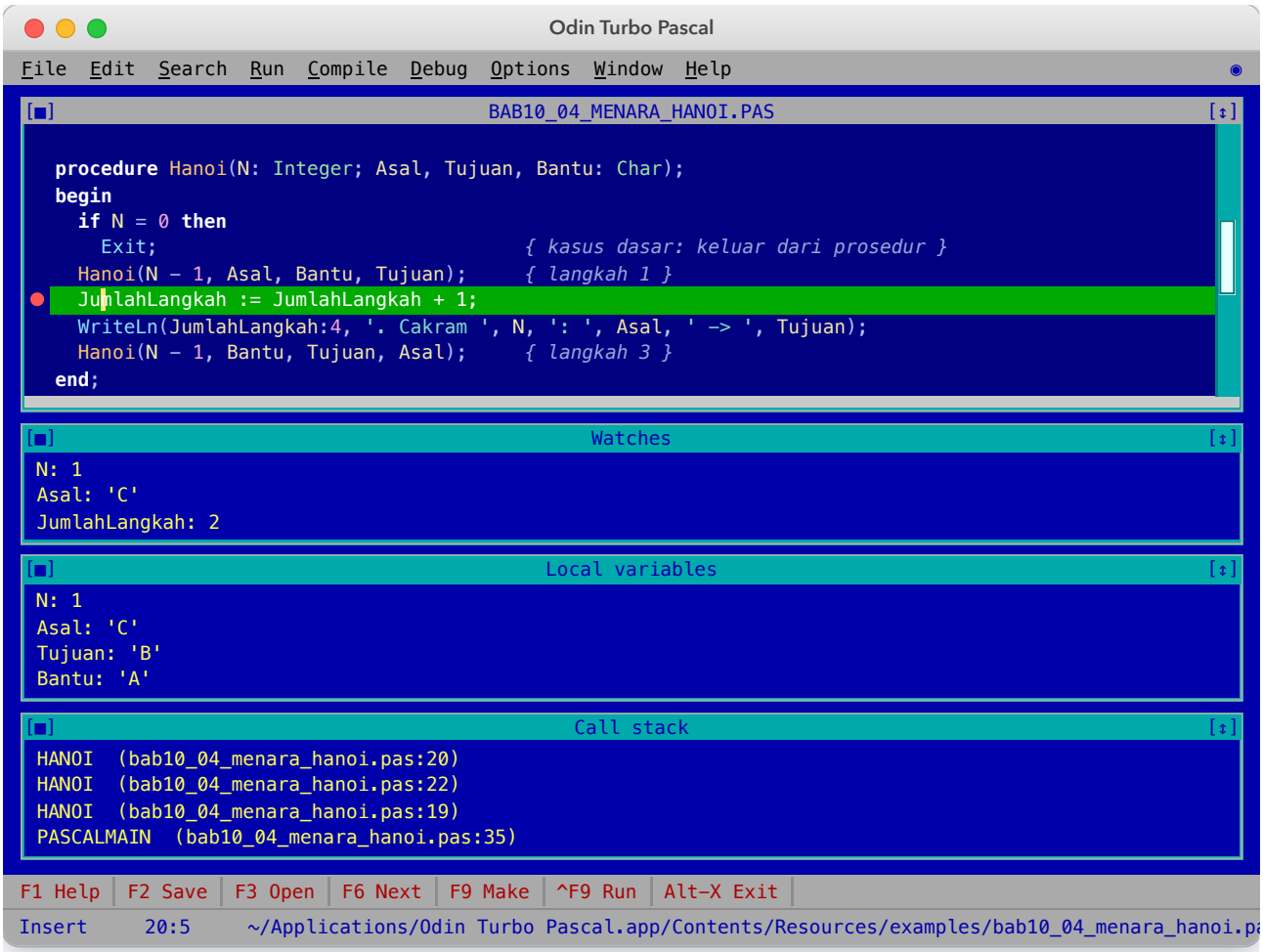
Gambar 10.11 Menara Hanoi dengan tiga cakram: keadaan awal dan tujuh langkah. Angka pada cakram adalah ukurannya (3 terbesar).

Karena setiap cakram ekstra menggandakan pekerjaan lalu menambah satu, jumlah langkah minimum untuk *N* cakram adalah $2^N - 1$ (itulah komentar di baris 36).

Tabel 10.2 Jumlah langkah Menara Hanoi menurut banyak cakram

Cakram *N*	1	2	3	4	5	6	7	8
Langkah $2^N - 1$	1	3	7	15	31	63	127	255

Selain itu, tumpukan pemanggilan Hanoi cukup menarik untuk dilihat. Pasang titik henti di baris 20, mulai debugging dengan masukan 3, dan lanjutkan sampai `JumlahLangkah` bernilai 2. Di sini program berhenti tepat sebelum mencetak langkah ketiga.



Gambar 10.12 Berhenti di dalam `Hanoi(1, 'C', 'B', 'A')`: tiga bingkai `Hanoi` aktif, dan baris pemanggil di setiap bingkai memberi tahu langkah mana yang sedang berjalan

Cara membaca Call stack ini: bingkai teratas adalah `Hanoi(1, 'C', 'B', 'A')` yang akan memindahkan cakram 1 dari C ke B. Di bawahnya, `Hanoi(2, 'A', 'B', 'C')` sedang berada di baris 22, yaitu langkah 3 miliknya (memindahkan cakram kecil dari Bantu ke Tujuan). Di bawahnya lagi, `Hanoi(3, 'A', 'C', 'B')` berada di baris 19, yaitu langkah 1 miliknya (menyingkirkan dua cakram atas). Tidak ada tumpukan kertas catatan: tumpukan pemanggilan itulah "ingatan" program tentang bagian pekerjaan yang masih tertunda.

COBA SENDIRI

Jalankan Program 10.4 dengan 4 cakram dan hitung baris keluarannya. Cocokkah dengan $2^4 - 1 = 15$? Lalu ubah urutan pemanggilan di baris 35 menjadi `Hanoi(N, 'A', 'B', 'C')` dan lihat ke tiang mana tumpukan akhirnya berpindah.

10.6 Rekursi atau Perulangan?

Semua yang dikerjakan rekursi bisa juga dikerjakan perulangan, dan sebaliknya. Pilih berdasarkan bentuk masalahnya, bukan selera. Tabel 16.6 merangkum pertimbangan yang praktis.

Tabel 10.3 Rekursi dibandingkan dengan perulangan

	Rekursi	Perulangan
Cocok untuk	masalah yang tersusun dari masalah sejenis yang lebih kecil: Hanoi, pohon, folder, pencarian bercabang	mengulang pekerjaan yang sama dalam jumlah yang diketahui: menjumlah, mencacah, membaca data
Kejelasan	sangat ringkas bila rumusnya sendiri rekursif	sangat jelas untuk pekerjaan berurutan sederhana
Memori	setiap pemanggilan memakai satu bingkai tumpukan; kedalaman besar bisa menghabiskan tumpukan	memori tetap, berapa pun jumlah pengulangan
Risiko khas	kasus dasar hilang; pekerjaan berulang (Fibonacci)	kondisi berhenti salah sehingga perulangan tak berujung

Faktorial dan Fibonacci di bab ini sebenarnya lebih cocok ditulis dengan perulangan di program sungguhan. Keduanya dipakai sebagai contoh karena rumusnya mudah dipahami, bukan karena rekursi pilihan terbaik. Hanoi lain cerita: tanpa rekursi, program pemecahnya jauh lebih rumit.

10.7 Kesalahan yang Sering Terjadi

10.7.1 Lupa argumen pada pemanggilan rekursif

Pada pemanggilan rekursif, nama fungsi di sisi kanan tetap harus diberi argumen. Menulisnya tanpa argumen membuat kompiler menganggapnya pemanggilan dengan parameter yang kurang.

tanpaarg.pas

```
1  program TanpaArg;
2  function Faktorial(N: Integer): LongInt;
3  begin
4    if N <= 1 then
5      Faktorial := 1
6    else
7      Faktorial := N * Faktorial;
8  end;
9
10 begin
11   WriteLn(Faktorial(5));
12 end.
```

PESAN KOMPILER

```
tanpaarg.pas(7,22) Error: Jumlah parameter tidak cocok: diharapkan 1, diberikan 0
```

Gambar 10.13 Pemanggilan rekursif tanpa argumen ditolak kompiler

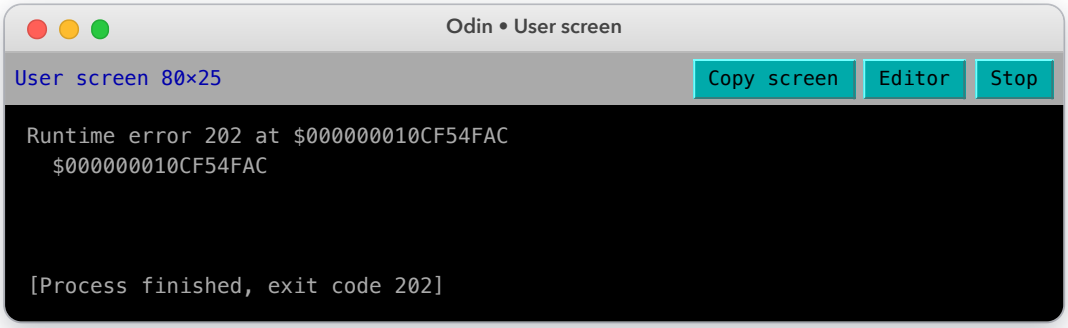
Perbaikannya: tulis `Faktorial(N - 1)`.

10.7.2 Kasus dasar hilang: tumpukan penuh

Program berikut lolos kompilasi, tetapi memanggil dirinya tanpa pernah berhenti. Setiap pemanggilan menambah satu bingkai sampai ruang tumpukan habis. Pada keluaran di bawah, program dihentikan dengan *runtime error* 202, kode untuk tumpukan penuh (*stack overflow*).

tanpadasar.pas

```
1  program TanpaDasar;
2  function Hitung(N: LongInt): LongInt;
3  begin
4    Hitung := N + Hitung(N - 1);
5  end;
6
7  begin
8    WriteLn(Hitung(5));
9  end.
```



Gambar 10.14 Tanpa kasus dasar, bingkai terus menumpuk sampai tumpukan habis

Perbaikannya: tambahkan kasus dasar, misalnya `if N <= 0 then Hitung := 0 else ...`. Pastikan juga parameter pada pemanggilan berikutnya benar-benar bergerak menuju kasus dasar itu.

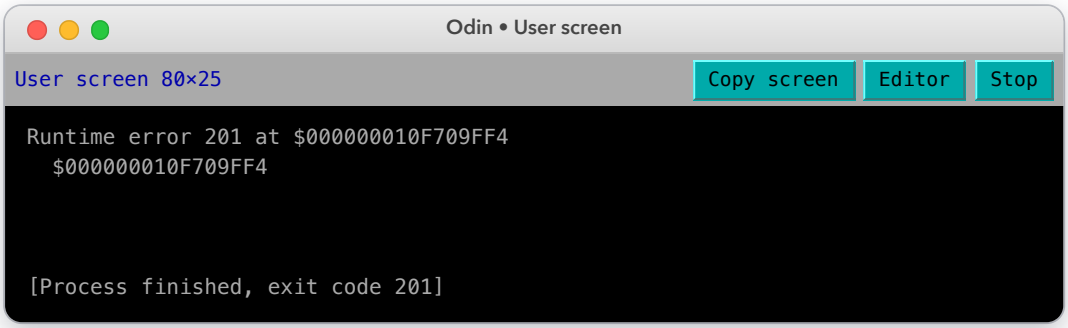
10.7.3 Hasil melebihi tipe data

Faktorial 13 tidak muat di `LongInt`. Perhatikan apa yang terjadi saat program mencoba menghitungnya: pemeriksaan jangkauan menghentikan program dengan *runtime error* 201, bukan menghasilkan angka yang salah diam-diam.

lebihbesar.pas

```
1  program LebihBesar;
2  function Faktorial(N: Integer): LongInt;
3  begin
4    if N <= 1 then
5      Faktorial := 1
6    else
7      Faktorial := N * Faktorial(N - 1);
8  end;
9
10 begin
11   WriteLn(Faktorial(13));
```

```
12  end.
```



Gambar 10.15 13! lebih besar dari batas LongInt sehingga program dihentikan

Perbaikannya: pakai tipe yang lebih besar (misalnya `Real` dengan format `:0:0`, yang menampung bilangan jauh lebih besar tetapi tidak persis untuk digit yang sangat banyak), atau batasi masukan seperti yang dilakukan Program 10.1 dan Program 10.4.

10.7.4 Kesalahan yang tidak ditolak kompiler

- **Masalah tidak mengecil**: memanggil `Faktorial(N)` alih-alih `Faktorial(N - 1)` di kasus rekursif. Kompilasi lolos, tetapi hasilnya sama seperti tumpukan penuh di atas.
- **Kasus dasar terlewat**: untuk `N = 0` pada `if N = 1 then` yang tidak ada cabang untuk nol, rekursi menembus ke bilangan negatif. Pakai `N <= 1` bila memang bisa terjadi.
- **Pemanggilan ganda yang tidak perlu**, seperti menulis `PangkatCepat` dua kali dalam satu ekspresi. Simpan hasilnya di variabel lokal.

10.8 Rangkuman

- **Rekursi** adalah fungsi atau prosedur yang memanggil dirinya sendiri untuk versi masalah yang lebih kecil.
- Setiap rekursi wajib punya **kasus dasar** yang berhenti tanpa memanggil diri, dan **kasus rekursif** yang membuat parameter bergerak menuju kasus dasar.
- Setiap pemanggilan mendapat **bingkai** sendiri di tumpukan pemanggilan (parameter dan variabel lokal sendiri). Bingkai menumpuk saat masuk dan dibuang satu per satu saat kembali; Call stack di debugger memperlihatkannya.
- Di Turbo Pascal 7, hasil fungsi rekursif ditetapkan lewat nama fungsi di sisi kiri `:=`; nama yang sama di sisi kanan adalah pemanggilan rekursif. `Exit` tanpa nilai mengakhiri subprogram.
- **Fibonacci** rekursif menghitung suku yang sama berulang kali, sehingga pemanggilannya berlipat ganda (2 x `Fib(N)` - 1). Rekursi tidak otomatis efisien.
- **Pangkat cepat** membelah pangkat setiap langkah sehingga pemanggilannya sekitar 2 log N, jauh lebih sedikit daripada $N + 1$; hasil separuh harus dihitung sekali.
- **Menara Hanoi** memecahkan N cakram dalam $2^N - 1$ langkah lewat tiga langkah rekursif yang bertukar peran tiang.

✓ Kesalahan khas: kasus dasar hilang atau tidak dicapai (tumpukan penuh), argumen lupa ditulis, dan hasil melampaui batas tipe seperti `LongInt`.

10.9 Latihan

1. **DASAR** Tulis fungsi rekursif `JumlahSampai(N: Integer): LongInt` yang menghitung 1 + 2 + ... + *N*. Tentukan kasus dasarnya, lalu telusuri `JumlahSampai(4)` di kertas seperti **Gambar 10.2**.

2. **DASAR** Ubah Program 10.1 agar mencetak jejak untuk 6 dan hitung banyak pesan "masuk" yang muncul. Berapa banyak bingkai yang aktif bersamaan pada saat terdalam?

3. **MENENGAH** Tulis prosedur rekursif `HitungMundur(N: Integer)` yang mencetak *N*, *N* - 1, ..., 1, lalu "Selesai!". Kemudian ubah agar mencetak 1 sampai *N* (hanya memindahkan satu baris). Jelaskan mengapa urutannya berubah.

4. **MENENGAH** Tulis fungsi rekursif `JumlahDigit(N: LongInt): Integer` yang menjumlahkan angka-angka penyusun bilangan (misalnya 4527 menjadi 18) dengan `mod 10` dan `div 10`.

5. **MENENGAH** Tulis fungsi rekursif `FPB(A, B: Integer): Integer` berdasarkan algoritma Euclid: FPB(*A*, 0) = *A*, dan FPB(*A*, *B*) = FPB(*B*, *A* mod *B*). Uji dengan 48 dan 36.

6. **TANTANGAN** Tulis `Fib` yang efisien tanpa pemanggilan berulang, misalnya dengan fungsi bantu rekursif `FibBantu(N, A, B)` yang membawa dua suku terakhir sebagai parameter. Bandingkan jumlah pemanggilannya dengan Program 10.2 untuk *N* = 25.

7. **TANTANGAN** Ubah Program 10.4 agar hanya mencetak total langkah (tanpa daftar), lalu ukur kapan komputer mulai terasa lambat bila batas cakram dinaikkan. Berapa langkah yang dibutuhkan 20 cakram menurut rumusnya?

BAGIAN IV

Struktur Data

Menyimpan dan mengolah banyak data dengan array, string, record, enumerasi, subrange, dan set.

11 Array

12 Mengolah String

13 Record

14 Enumerasi, Subrange, dan Set

BAGIAN IV · STRUKTUR DATA

BAB 11

Array

TUJUAN PEMBELAJARAN

- Menjelaskan mengapa sederet data sejenis lebih baik disimpan dalam satu *array* daripada dalam banyak variabel.
- Mendeklarasikan array satu dimensi, mengisinya, membacanya, dan mengubah elemennya lewat indeks.
- Mengolah array dengan perulangan `for`: mencetak, membalik urutan, mencari nilai terbesar, terkecil, dan rata-rata.
- Mengenali runtime error 201 yang muncul saat indeks keluar dari batas array.
- Memakai array dua dimensi (tabel baris dan kolom) dan konstanta bertipe untuk mengisinya.
- Menjumlahkan dan mengalikan matriks, serta meneruskan array ke prosedur lewat tipe buatan sendiri.

PROGRAM DALAM BAB INI

11.1	`bab11_01_array_satu_dimensi.pas`	Array satu dimensi
11.2	`bab11_02_maks_min_rata.pas`	Nilai terbesar, terkecil, dan rata-rata
11.3	`bab11_03_array_dua_dimensi.pas`	Array dua dimensi
11.4	`bab11_04_penjumlahan_matriks.pas`	Penjumlahan matriks
11.5	`bab11_05_perkalian_matriks.pas`	Perkalian matriks

Bayangkan program yang harus mengolah nilai ujian tiga puluh siswa. Dengan cara yang sudah Anda pelajari, program itu membutuhkan tiga puluh variabel, masing-masing dengan nama sendiri, dan tiga puluh perintah `ReadLn` yang hampir sama. Pekerjaan semacam ini membosankan dan mudah salah. Pascal menyediakan jalan keluar yang rapi: **array**, satu nama untuk sederet variabel sejenis. Dengan array dan perulangan, mengolah tiga puluh data sama mudahnya dengan mengolah tiga data.

11.1 Satu Nama, Banyak Kotak

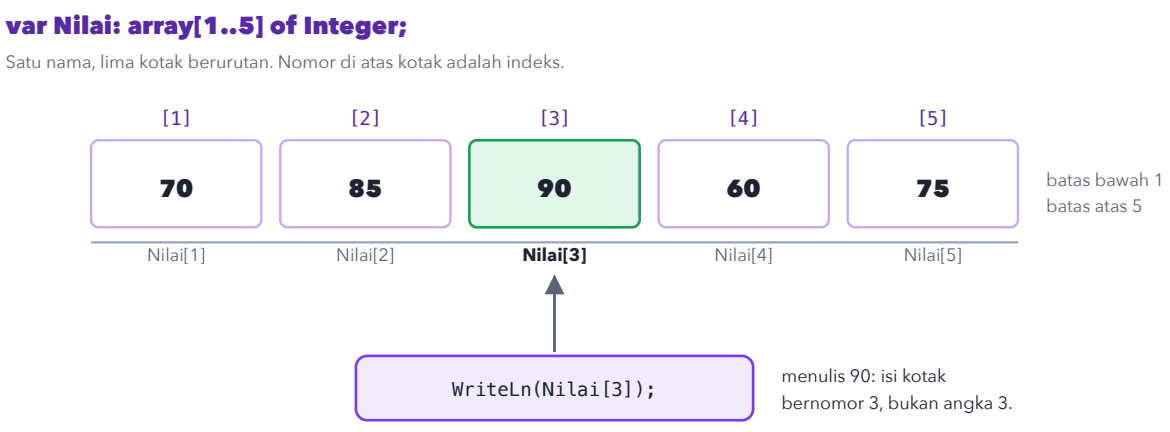
Array (*array*, larik) adalah kumpulan variabel bertipe sama yang disimpan berurutan di memori dan diberi satu nama bersama. Setiap anggotanya disebut **elemen** dan dibedakan oleh nomor urut yang disebut **indeks** (*index*). Analoginya deretan loker di sekolah: seluruh deretan punya satu nama ("loker kelas X"), dan setiap loker punya nomor sendiri. Untuk mengambil isi loker nomor 3, Anda cukup menyebut deretannya dan nomornya. Di Pascal, nomor itu ditulis di dalam kurung siku: `Nilai[3]`.

BENTUK UMUM

```
var
  NamaArray: array[indeksAwal..indeksAkhir] of TipeElemen;

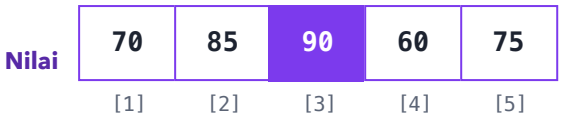
NamaArray[indeks]            { satu elemen: boleh dibaca maupun diisi }
```

Contoh `Nilai: array[1..5] of Integer;` membuat lima variabel `Integer` sekaligus, dengan indeks 1, 2, 3, 4, dan 5. Rentang indeks `1..5` menentukan **batas bawah** dan **batas atas**. Di Pascal, Anda bebas memilih batas itu, jadi `array[0..4]` atau `array[10..14]` juga sah, selama batas bawah tidak lebih besar dari batas atas. Buku ini memakai indeks mulai dari 1 supaya cocok dengan cara kita menghitung sehari-hari. Banyak elemen dihitung dengan rumus *batas atas - batas bawah + 1*.



Gambar 11.1 Array `Nilai` berisi lima `Integer`. Nomor indeks menunjuk kotak, nilai di dalamnya yang diolah.

Bedakan dua angka yang sama-sama hadir di `Nilai[3] = 90`: angka dalam kurung siku adalah **indeks** (letak kotak), sedangkan 90 adalah **isi** kotak itu. Gambar berikut menunjukkan hal yang sama dengan komponen sederhana yang akan dipakai di sepanjang bab ini.



Kekuatan sebenarnya muncul karena indeks boleh berupa **variabel**. Tulisan `Nilai[I]` dengan `I` yang berubah dalam perulangan `for` berarti "elemen yang nomornya sedang ditunjuk `I`". Satu rangkaian perintah yang sama dapat mengolah seluruh elemen, berapa pun jumlahnya.

1 Deklarasi

`array[1..5] of Integer` memesan lima kotak `Integer` sekaligus di bawah satu nama.

2 Akses

`Nilai[3]` menunjuk satu kotak. Boleh dibaca (`WriteLn`) dan boleh diisi (`:=`, `ReadLn`).

3 Perulangan

`for I := 1 to 5` membuat `Nilai[I]` menyentuh semua kotak satu per satu.

11.2 Array Satu Dimensi

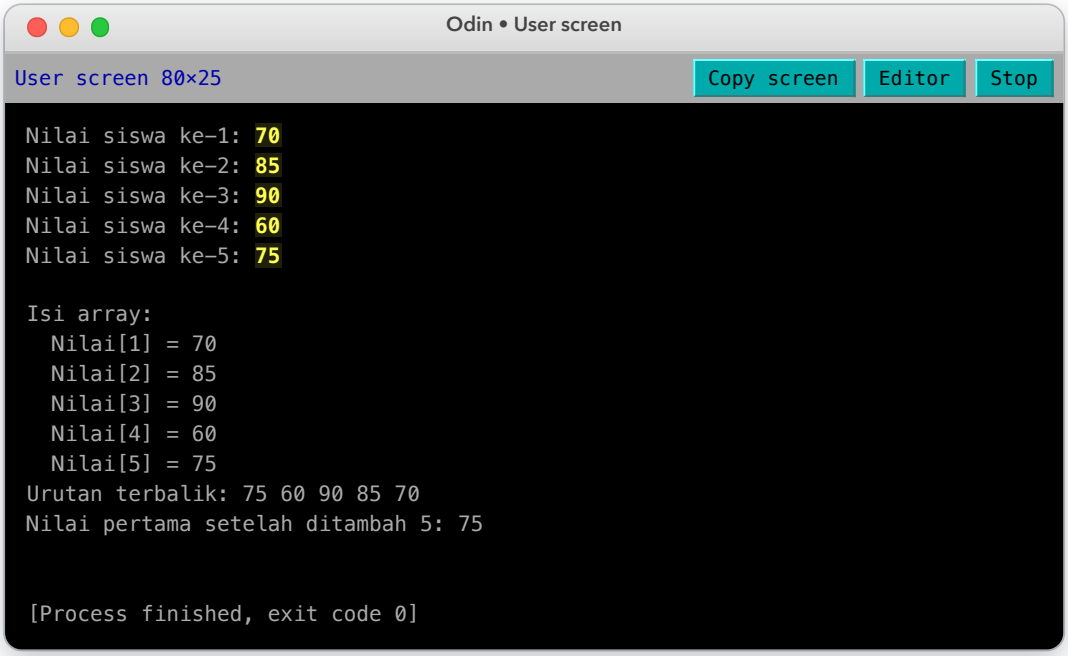
Array dengan satu deret indeks, seperti `Nilai` di atas, disebut array **satu dimensi**. Program 11.1 memakainya untuk empat pekerjaan dasar: mengisi array dari keyboard, mencetak isinya, mencetaknya dari belakang, dan mengubah satu elemen. Perhatikan bahwa jumlah elemen ditulis sekali saja sebagai konstanta `Banyak`. Bila suatu hari datanya menjadi sepuluh, Anda hanya mengubah satu angka di baris 10.

Buka `bab11_01_array_satu_dimensi.pas` lewat `File ▸ Examples...`, lalu jalankan dengan Ctrl+F9 dan ketik lima nilai, masing-masing diakhiri Enter.

Program 11.1 `bab11_01_array_satu_dimensi.pas` *Array satu dimensi* — File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 11.1 — Array satu dimensi
3    Array adalah sederet variabel sejenis dengan satu nama. Setiap
4    elemen diakses dengan indeks di dalam kurung siku: Nilai[1],
5    Nilai[2], dan seterusnya. Perulangan for cocok untuk mengolahnya. }
6
7  program bab11_01_array_satu_dimensi;
8
9  const
10   Banyak = 5;
11
12 var
13   Nilai: array[1..Banyak] of Integer;    { 5 elemen, indeks 1 sampai 5 }
14   I: Integer;
15
16 begin
17   { Mengisi array dari keyboard }
18   for I := 1 to Banyak do
19   begin
20     Write('Nilai siswa ke-', I, ': ');
21     ReadLn(Nilai[I]);                    { elemen ke-I diisi }
22   end;
23
```

```
24    WriteLn;
25    WriteLn('Isi array:');
26    for I := 1 to Banyak do
27      WriteLn('  Nilai[', I, '] = ', Nilai[I]);
28
29    Write('Urutan terbalik: ');
30    for I := Banyak downto 1 do
31      Write(Nilai[I], ' ');
32    WriteLn;
33
34    { Elemen bisa diubah satu per satu }
35    Nilai[1] := Nilai[1] + 5;
36    WriteLn('Nilai pertama setelah ditambah 5: ', Nilai[1]);
37
38    { Indeks di luar 1..5, misalnya Nilai[6], menghentikan program
39      dengan runtime error 201 karena range checking aktif. }
40  end.
```



Gambar 11.2 Masukan 70, 85, 90, 60, dan 75

Baris	Penjelasan
9–10	Konstanta `Banyak = 5`. Memakai konstanta untuk ukuran array adalah kebiasaan baik: batas atas array dan batas perulangan selalu sama, dan perubahan cukup dilakukan di satu tempat.
13	Deklarasi array: lima `Integer` dengan indeks 1 sampai 5. Batas atas boleh berupa konstanta, tidak boleh variabel.
18–22	Perulangan pengisian. Pada putaran ke-`I`, baris 20 menulis "Nilai siswa ke-`I`" dan baris 21 menyimpan angka yang diketik ke dalam `Nilai[I]`.
26–27	Perulangan pencetakan dari indeks 1 ke 5. Satu `WriteLn` menulis nomor indeks dan isi elemennya.
30–31	`for I := Banyak downto 1` menghitung mundur sehingga elemen terakhir tercetak lebih dulu. Itulah urutan terbalik yang tampak di layar: `75 60 90 85 70`.
35	Elemen diperlakukan seperti variabel biasa: `Nilai[1]` dibaca di sisi kanan, ditambah 5, lalu disimpan kembali ke `Nilai[1]`. Nilai 70 menjadi 75.

Baris	Penjelasan
38–39	Komentar yang mengingatkan bahwa `Nilai[6]` tidak ada. Bagian berikut menunjukkan apa yang terjadi bila Anda tetap menulisnya.

7 **ATURAN TURBO PASCAL 7**

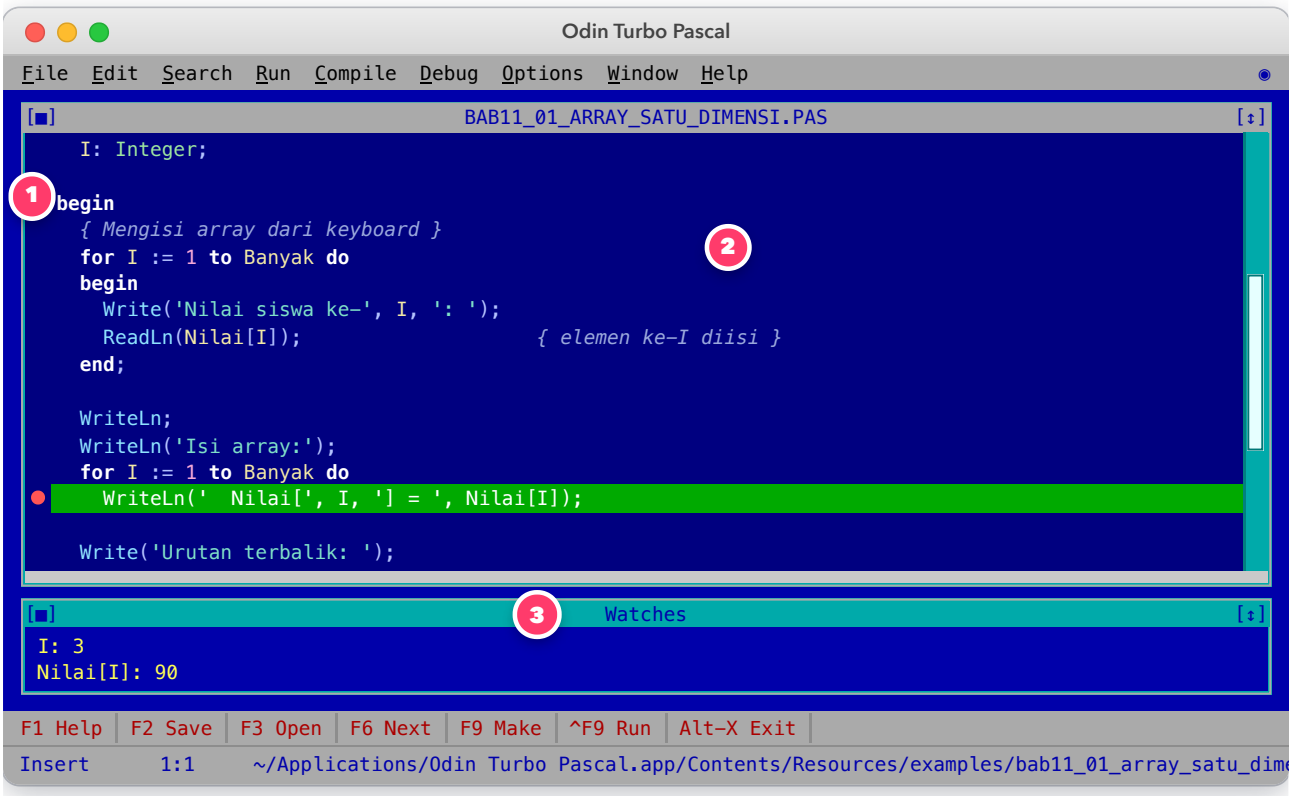
Indeks array boleh bertipe ordinal apa saja, bukan hanya angka mulai 1. Karena `Integer` di Turbo Pascal 7 hanya 16 bit (-32768 sampai 32767), variabel indeks dan penghitung yang nilainya mungkin melebihi 32767 harus bertipe `LongInt`. Contoh buku ini selalu memakai array kecil, jadi `Integer` sudah cukup.

COBA SENDIRI

Ubah `Banyak` pada baris 10 menjadi 3. Jalankan kembali dan perhatikan bahwa seluruh perulangan menyesuaikan diri tanpa Anda menyentuh baris lainnya. Lalu ubah baris 30 menjadi `for I := 1 to Banyak do` dan lihat bahwa "urutan terbalik" kini menjadi urutan biasa.

11.2.1 Memeriksa isi array dengan debugger

Untuk benar-benar melihat array bekerja, jalankan program langkah demi langkah. Pasang *breakpoint* pada baris 27 dengan `Debug ▸ Toggle breakpoint`, jalankan dengan `Debug ▸ Start debugging`, lalu tambahkan *watch* pada `I` dan `Nilai[I]` lewat `Debug ▸ Add watch...`. Setiap kali Anda menekan `F8` (`Step over`), nilai `I` naik satu dan `Nilai[I]` menunjukkan isi kotak berikutnya.



Gambar 11.3 Debugger berhenti di baris 27 pada putaran ketiga: `I` bernilai 3 dan `Nilai[I]` adalah 90

Nomor	Penjelasan
1	**Gutter.** Titik merah dan sorotan hijau menandai baris 27 sebagai tempat program berhenti.
2	**Editor.** Baris di dalam perulangan yang dieksekusi berikutnya.
3	**Jendela Watches.** Menampilkan nilai `I` dan elemen yang sedang ditunjuknya.

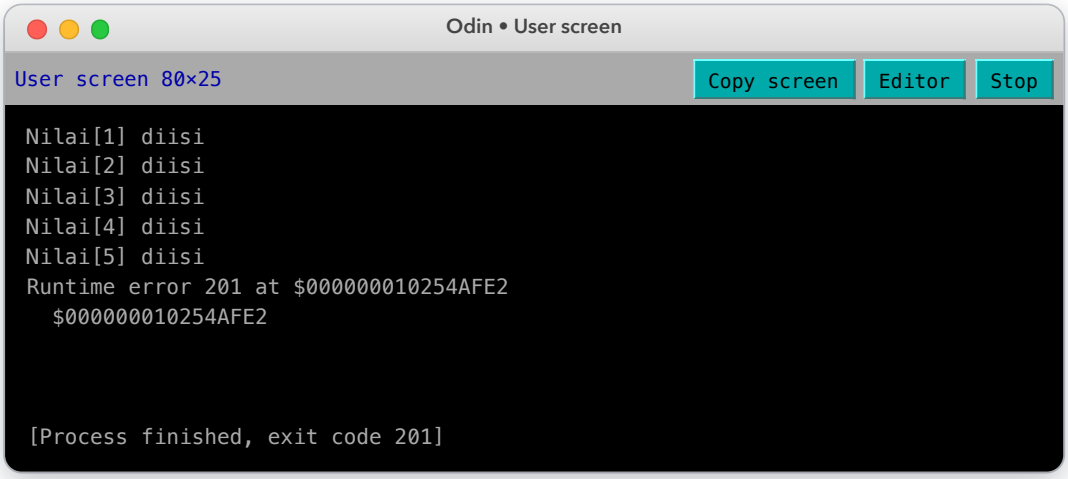
11.3 Indeks di Luar Batas

Array `Nilai` hanya punya kotak nomor 1 sampai 5. Apa yang terjadi bila program meminta `Nilai[6]` atau `Nilai[0]`? Kotak itu tidak ada, dan menulis ke sana berarti menimpa memori milik variabel lain, kesalahan yang sangat sulit dilacak. Karena itu Pascal menyediakan **pemeriksaan batas** (*range checking*): setiap kali program memakai indeks, nilainya dicocokkan dengan batas array. Bila keluar dari batas, program langsung dihentikan dengan **runtime error 201** (*range check error*).

Program berikut sengaja memakai perulangan yang terlalu panjang: array berisi lima elemen, tetapi perulangan berjalan sampai 6.

batas.pas

```
1  program Batas;
2  var
3    Nilai: array[1..5] of Integer;
4    I: Integer;
5  begin
6    for I := 1 to 6 do
7    begin
8      Nilai[I] := I * 10;
9      WriteLn('Nilai[', I, '] diisi');
10   end;
11   WriteLn('Selesai');
12 end.
```



Gambar 11.4 Perulangan sampai 6 untuk array berindeks 1..5

Lima putaran pertama berjalan normal. Pada putaran keenam, `I` bernilai 6, indeks itu di luar `1..5`, dan program berhenti dengan galat 201 sebelum sempat menulis "Selesai". Galat ini adalah *teman* Anda: ia menunjukkan bahwa ada kesalahan hitung di perulangan. Perbaikannya selalu pada logika program, misalnya mengubah batas perulangan menjadi konstanta yang sama dengan ukuran array.

> **AWAS**
>
> Runtime error 201 hanya muncul bila pemeriksaan batas aktif. Di Odin Turbo Pascal pemeriksaan ini aktif sehingga kesalahan indeks tertangkap. Jangan berharap bahwa kompiler lain atau pengaturan lain akan menolong Anda dengan cara yang sama: pastikan indeks Anda benar sejak awal.

V Indeks aman

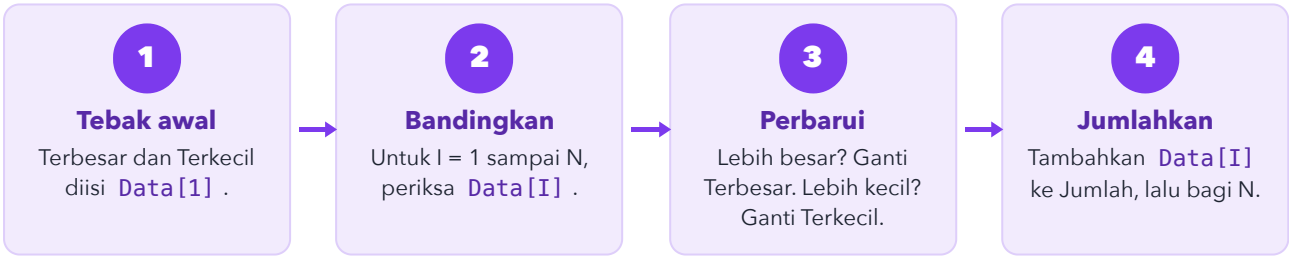
`for I := 1 to Banyak do` dengan `Banyak` sama dengan batas atas array.

X Indeks berbahaya

Angka batas ditulis langsung berkali-kali (`to 5`, `to 6`), sehingga mudah tidak sinkron dengan deklarasi.

11.4 Nilai Terbesar, Terkecil, dan Rata-rata

Salah satu pola paling berguna pada array adalah mencari nilai terbesar dan terkecil. Caranya mirip menentukan siswa tertinggi di barisan: tunjuk siswa pertama sebagai "tertinggi sementara", lalu bandingkan dengan setiap siswa berikutnya. Bila ada yang lebih tinggi, dia menggantikan juara sementara. Setelah semua dibandingkan, juara sementara adalah juara sebenarnya. Pola yang sama dipakai untuk terkecil, dan sambil lewat kita bisa menjumlahkan semua nilai untuk menghitung rata-rata.



Program 11.2 menerapkannya pada data yang banyaknya ditentukan pengguna. Array dipesan untuk 20 elemen (`MaksData`), tetapi hanya sebanyak `N` elemen yang dipakai. Ini cara umum bila jumlah data belum diketahui saat program ditulis: pesan kapasitas maksimum, lalu catat berapa yang terpakai. Perulangan `repeat` pada baris 18–21 memastikan `N` tidak melebihi kapasitas, sehingga indeks tidak akan keluar batas.

Program 11.2 `bab11_02_maks_min_rata.pas` *Nilai terbesar, terkecil, dan rata-rata* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 11.2 — Nilai terbesar, terkecil, dan rata-rata
3    Pola umum: anggap elemen pertama sebagai yang terbesar/terkecil,
4    lalu bandingkan dengan elemen berikutnya satu per satu. }
5
6  program bab11_02_maks_min_rata;
7
```

```
const
  MaksData = 20;                        { kapasitas array }

var
  Data: array[1..MaksData] of Integer;
  N, I: Integer;                        { N = banyak data yang dipakai }
  Terbesar, Terkecil, PosisiBesar: Integer;
  Jumlah: LongInt;

begin
  repeat
    Write('Banyak data (1-', MaksData, '): ');
    ReadLn(N);
  until (N >= 1) and (N <= MaksData);

  for I := 1 to N do
  begin
    Write('Data ke-', I, ': ');
    ReadLn(Data[I]);
  end;

  Terbesar := Data[1];                  { mulai dari elemen pertama }
  Terkecil := Data[1];
  PosisiBesar := 1;
  Jumlah := 0;
  for I := 1 to N do
  begin
    if Data[I] > Terbesar then
    begin
      Terbesar := Data[I];
      PosisiBesar := I;                 { ingat posisinya juga }
    end;
    if Data[I] < Terkecil then
      Terkecil := Data[I];
    Jumlah := Jumlah + Data[I];
  end;

  WriteLn;
  WriteLn('Terbesar  : ', Terbesar, ' (data ke-', PosisiBesar, ')');
  WriteLn('Terkecil  : ', Terkecil);
  WriteLn('Jumlah    : ', Jumlah);
  WriteLn('Rata-rata : ', Jumlah / N:0:2);
end.
```



Gambar 11.5 Empat data: 12, 45, 7, dan 30

Gambar berikut melacak isi variabel untuk keempat data tersebut. Sel hijau menandai saat nilai berubah.

Jejak perulangan baris 33-43 untuk Data = 12, 45, 7, 30

Putaran	I = 1	I = 2	I = 3	I = 4
Data[I]	12	45	7	30
Terbesar	12	45	45	45
Terkecil	12	12	7	7
Jumlah	12	57	64	94

Hasil akhir: Terbesar 45 (data ke-2), Terkecil 7, Jumlah 94, rata-rata 94 / 4 = 23.50.

Gambar 11.6 Isi variabel pada setiap putaran. Terbesar berubah pada putaran 2, Terkecil berubah pada putaran 3.

Baris	Penjelasan
8–9	`MaksData = 20` adalah kapasitas array. Array dideklarasikan satu kali di baris 12 dan tidak bisa membesar saat program berjalan.
13	`N` mencatat banyaknya data yang benar-benar dipakai (1 sampai 20).
15	`Jumlah` bertipe `LongInt` karena jumlah banyak data `Integer` dapat melebihi 32767.
18–21	Meminta `N` berulang kali sampai nilainya sah. Selama `N` di luar 1..20, pertanyaan diulang.
23–27	Mengisi `Data[1]` sampai `Data[N]`. Elemen sesudah `N` tidak disentuh.
29–32	Persiapan: `Terbesar` dan `Terkecil` diisi elemen pertama, posisi terbesar diisi 1, jumlah dimulai dari 0.
35–39	Bila elemen lebih besar dari `Terbesar`, simpan nilainya dan juga posisinya (baris 38), supaya program dapat melaporkan "data ke-2".
40–41	Bila elemen lebih kecil dari `Terkecil`, ganti `Terkecil`.

Baris	Penjelasan
42	Setiap elemen ditambahkan ke `Jumlah`, tanpa syarat.
49	`Jumlah / N` memakai operator `/` yang selalu menghasilkan bilangan real, lalu `:0:2` memformatnya dengan dua angka di belakang koma: 23.50.

7 **ATURAN TURBO PASCAL 7**

Hasil operator `/` bertipe `Real` walaupun kedua operandnya bulat, dan di Turbo Pascal 7 bilangan `Real` harus diformat dengan `:lebar:desimal` bila dicetak (`:0:2` berarti lebar secukupnya, dua desimal). Untuk pembagian bulat gunakan `div`.

COBA SENDIRI

Jalankan Program 11.2 dengan data yang semuanya sama, misalnya 5, 5, 5. Data ke berapa yang dilaporkan sebagai terbesar, dan mengapa? Lalu ubah `>` pada baris 35 menjadi `>=` dan coba lagi.

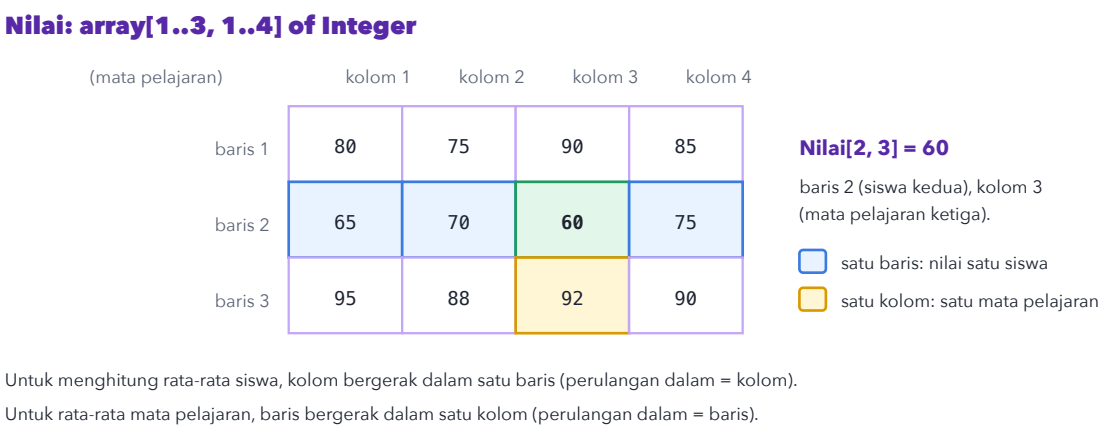
11.5 Array Dua Dimensi

Sebagian data lebih alami berbentuk tabel daripada deretan: nilai tiga siswa untuk empat mata pelajaran, denah kursi, papan catur. Untuk itu Pascal menyediakan array **dua dimensi**, yaitu array yang elemennya ditunjuk oleh dua indeks: indeks pertama untuk **baris**, indeks kedua untuk **kolom**. Seperti menyebut posisi kursi di bioskop: "baris C, nomor 7".

BENTUK UMUM

```
var
  Tabel: array[barisAwal..barisAkhir, kolomAwal..kolomAkhir] of TipeElemen;

Tabel[baris, kolom]
```



Gambar 11.7 Array dua dimensi sebagai tabel. Indeks pertama memilih baris, indeks kedua memilih kolom.

Untuk menelusuri seluruh tabel, dipakai dua perulangan `for` yang **bersarang**: perulangan luar memilih satu baris, dan untuk setiap baris itu perulangan dalam berjalan melewati semua kolomnya. Urutan penelusuran adalah baris 1 kolom 1, baris 1 kolom 2, dan seterusnya sampai ujung baris, lalu pindah ke baris 2.

Program 11.3 menampilkan daftar nilai tiga siswa dalam empat mata pelajaran beserta rata-rata tiap siswa dan tiap mata pelajaran. Karena mengetik dua belas angka akan melelahkan, isi tabel ditulis langsung di program sebagai **konstanta bertipe** (*typed constant*): konstanta yang tipenya ditentukan, sehingga boleh berupa array. Setiap pasang kurung bagian dalam adalah satu baris tabel.

Program 11.3 `bab11_03_array_dua_dimensi.pas` *Array dua dimensi* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 11.3 — Array dua dimensi
  Array dua dimensi berbentuk tabel: indeks pertama untuk baris,
  indeks kedua untuk kolom, ditulis Nilai[Baris, Kolom].
  Di sini datanya diisi lewat konstanta bertipe agar tidak perlu
  diketik satu per satu. }

program bab11_03_array_dua_dimensi;

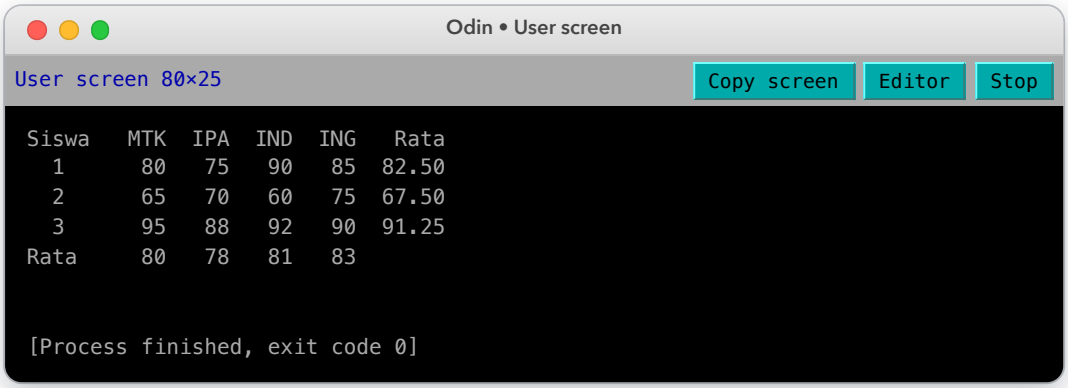
const
  JumlahSiswa = 3;
  JumlahMapel = 4;
  { Konstanta bertipe: setiap baris dalam kurung adalah satu siswa }
  Nilai: array[1..JumlahSiswa, 1..JumlahMapel] of Integer = (
    (80, 75, 90, 85),
    (65, 70, 60, 75),
    (95, 88, 92, 90));

var
  Baris, Kolom: Integer;
  Total: Integer;

begin
  WriteLn('Siswa   MTK  IPA  IND  ING   Rata');
  for Baris := 1 to JumlahSiswa do
  begin
    Write('  ', Baris, '   ');
    Total := 0;
    for Kolom := 1 to JumlahMapel do
    begin
      Write(Nilai[Baris, Kolom]:5);      { elemen baris B, kolom K }
      Total := Total + Nilai[Baris, Kolom];
    end;
    WriteLn(Total / JumlahMapel:7:2);
  end;

  { Kini kolom di luar, baris di dalam: rata-rata per mapel }
  Write('Rata  ');
  for Kolom := 1 to JumlahMapel do
  begin
    Total := 0;
    for Baris := 1 to JumlahSiswa do
      Total := Total + Nilai[Baris, Kolom];
    Write(Total / JumlahSiswa:5:0);
```

```
45    end;
46    WriteLn;
47  end.
```



Gambar 11.8 Tabel nilai dan rata-ratanya

Baris	Penjelasan
14–17	Konstanta bertipe dengan format `(baris1), (baris2), (baris3)`. Jumlah angka di tiap baris harus sama dengan banyak kolom, dan jumlah barisnya sama dengan banyak siswa.
24	Judul kolom, ditulis dengan `WriteLn` biasa.
25–35	Perulangan luar (`Baris`) untuk setiap siswa. Baris 27 mencetak nomor siswa dan baris 28 mengosongkan `Total` sebelum setiap siswa dijumlahkan.
29–33	Perulangan dalam (`Kolom`) berjalan sepanjang satu baris. `Nilai[Baris, Kolom]:5` mencetak setiap nilai rata kanan dalam lima kolom layar, dan nilainya ditambahkan ke `Total`.
34	Setelah kolom habis, `Total / JumlahMapel` dicetak dengan lebar 7 dan dua desimal, lalu `WriteLn` berpindah baris.
38–46	Urutan perulangan dibalik: kolom di luar, baris di dalam. Sekarang yang dijumlahkan adalah satu kolom (satu mata pelajaran) untuk semua siswa, dan `:5:0` mencetak rata-ratanya tanpa desimal.

7 ATURAN TURBO PASCAL 7

Konstanta bertipe adalah fasilitas khas Turbo Pascal: sintaks `Nama: Tipe = nilai;` di bagian `const`. Isinya dapat diubah saat program berjalan seperti variabel yang sudah punya nilai awal. Pada Program 11.3 isinya hanya dibaca. Format `:5:0` pada baris 44 mencetak bilangan real tanpa desimal dengan pembulatan.

★ INGAT

Urutan indeks selalu **[baris, kolom]**. Menukarnya, misalnya `Nilai[Kolom, Baris]`, tidak ditolak bila batasnya kebetulan cocok, tetapi hasilnya salah. Bila batas tidak cocok, Anda akan bertemu galat 201.

11.6 Matriks dan Penjumlahannya

Dalam matematika, tabel angka berbentuk persegi panjang disebut **matriks**, dan array dua dimensi adalah wadah yang tepat untuknya. Dua matriks berukuran sama dijumlahkan dengan menjumlahkan elemen yang letaknya sama: `C[i, j] = A[i, j] + B[i, j]`. Matriks 3 x 3 berarti tiga baris dan tiga kolom, jadi hasilnya pun 3 x 3.



Program 11.4 memperkenalkan satu hal baru: **tipe buatan sendiri**. Deklarasi `TMatriks = array[1..Ukuran, 1..Ukuran] of Integer` di bagian `type` memberi nama pada bentuk array tertentu. Nama itu kemudian dipakai untuk konstanta `A` dan `B`, variabel `C`, dan juga **parameter prosedur**. Ini penting: di Pascal, parameter prosedur harus berupa nama tipe, sehingga `procedure Tampil(M: array[1..3, 1..3] of Integer)` tidak diperbolehkan.

BENTUK UMUM

```
type
  NamaTipe = array[batas] of TipeElemen;

procedure Nama(P: NamaTipe);
```

Program 11.4 `bab11_04_penjumlahan_matriks.pas` *Penjumlahan matriks* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 11.4 — Penjumlahan matriks
  Dua matriks berukuran sama dijumlahkan elemen demi elemen:
  C[i, j] = A[i, j] + B[i, j]. Tipe TMatriks dibuat sekali, lalu
  dipakai untuk variabel dan parameter prosedur. }

program bab11_04_penjumlahan_matriks;

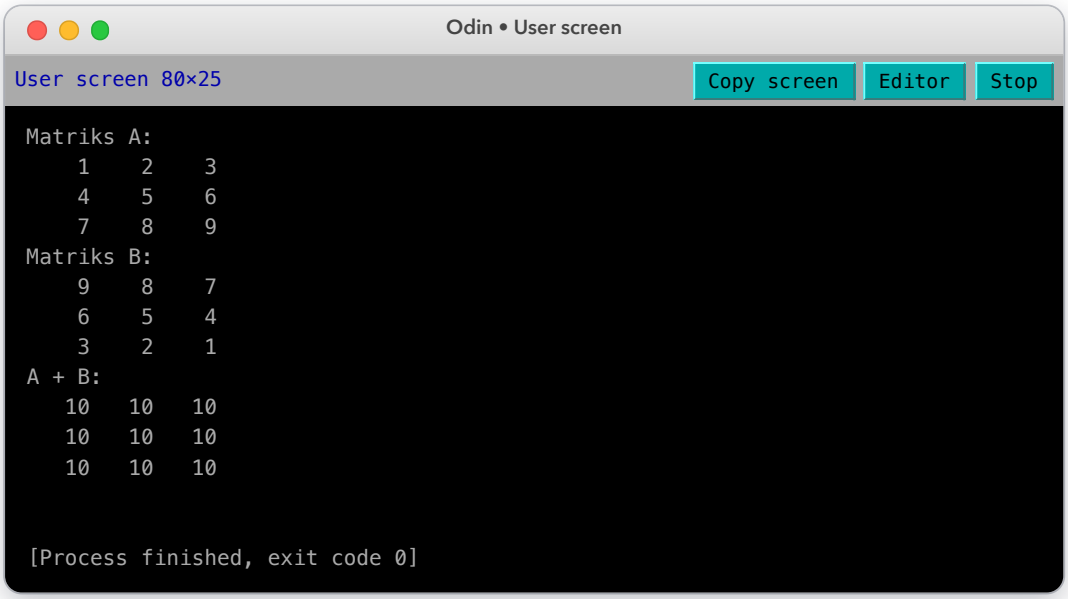
const
  Ukuran = 3;

type
  TMatriks = array[1..Ukuran, 1..Ukuran] of Integer;

const
  A: TMatriks = ((1, 2, 3),
                 (4, 5, 6),
                 (7, 8, 9));
  B: TMatriks = ((9, 8, 7),
                 (6, 5, 4),
                 (3, 2, 1));

procedure Tampilkan(Judul: string; M: TMatriks);
var
  I, J: Integer;
begin
```

```
27    WriteLn(Judul);
28    for I := 1 to Ukuran do
29    begin
30      for J := 1 to Ukuran do
31        Write(M[I, J]:5);
32      WriteLn;
33    end;
34  end;
35
36  var
37    C: TMatriks;
38    I, J: Integer;
39
40  begin
41    for I := 1 to Ukuran do                { setiap baris }
42      for J := 1 to Ukuran do              { setiap kolom }
43        C[I, J] := A[I, J] + B[I, J];
44
45    Tampilkan('Matriks A:', A);
46    Tampilkan('Matriks B:', B);
47    Tampilkan('A + B:', C);
48  end.
```



Gambar 11.9 Matriks A, matriks B, dan jumlahnya

Baris	Penjelasan
9–10	Konstanta `Ukuran = 3` dipakai di semua tempat sehingga ukuran matriks dapat diubah dari satu titik.
12–13	Bagian `type` : `TMatriks` adalah nama untuk array 3 x 3 berisi `Integer`. Awalan `T` adalah kebiasaan penamaan tipe.
15–21	Dua konstanta bertipe `TMatriks` dengan isi yang langsung diberikan. Dengan `A` dan `B` terisi, program tidak perlu meminta masukan.
23–34	Prosedur `Tampilkan` menerima judul dan sebuah `TMatriks`. Parameter `M` adalah salinan matriks, jadi perubahan di dalam prosedur tidak mengubah aslinya. Dua perulangan bersarang mencetak tiap elemen dalam lima kolom layar, dan baris 32 pindah baris di akhir setiap baris matriks.

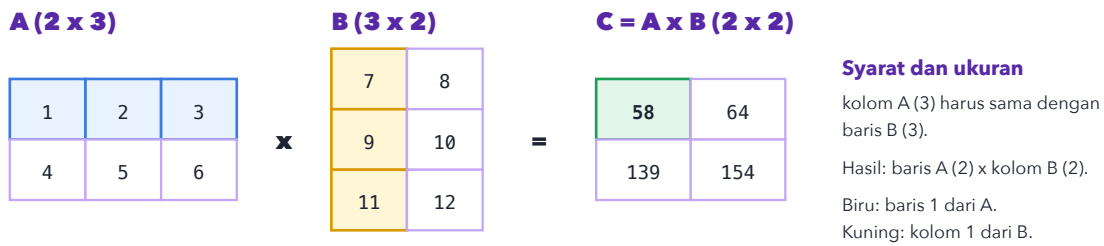
Baris	Penjelasan
36–38	Variabel global `C` untuk hasil, serta `I` dan `J` sebagai penghitung baris dan kolom.
41–43	Inti penjumlahan. Untuk setiap baris `I` dan kolom `J`, `C[I, J]` diisi jumlah elemen yang bersesuaian. Perintah baris 43 dijalankan sembilan kali.
45–47	Prosedur yang sama dipanggil tiga kali dengan matriks berbeda. Satu prosedur menggantikan tiga pasang perulangan.

Perhatikan hasil `A + B`: setiap elemennya sama dengan 10. Itu bukan kebetulan. Elemen `A` naik dari 1 sampai 9, elemen `B` turun dari 9 sampai 1, sehingga pasangan yang sama letaknya selalu berjumlah 10.

> **COBA SENDIRI**
>
> Tambahkan matriks ketiga `D` dan hitung `A - B` dengan mengubah tanda penjumlahan di baris 43. Apakah hasil pengurangan juga bersifat simetris seperti penjumlahan?

11.7 Perkalian Matriks

Perkalian matriks lebih menarik daripada penjumlahan, dan bukan perkalian elemen demi elemen. Matriks A berukuran 2 x 3 (dua baris, tiga kolom) boleh dikalikan dengan matriks B berukuran 3 x 2 karena **jumlah kolom A sama dengan jumlah baris B**. Hasilnya, matriks C, berukuran **baris A x kolom B**, yaitu 2 x 2. Setiap elemen `C[i, j]` dihitung dari **baris i** matriks A dan **kolom j** matriks B: kalikan pasangan elemen yang berurutan, lalu jumlahkan semuanya.



C[i, j] = A[i,1] x B[1,j] + A[i,2] x B[2,j] + A[i,3] x B[3,j]

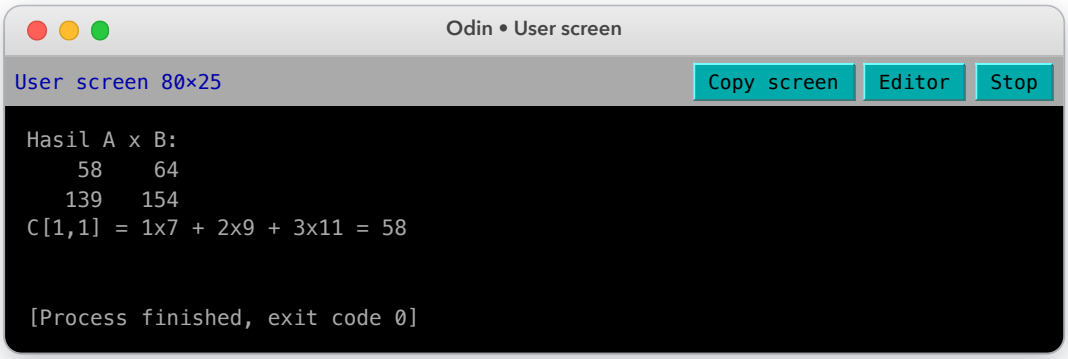
```
C[1,1] = 1x7 + 2x9 + 3x11 = 7 + 18 + 33 = 58
C[1,2] = 1x8 + 2x10 + 3x12 = 8 + 20 + 36 = 64
C[2,1] = 4x7 + 5x9 + 6x11 = 28 + 45 + 66 = 139
C[2,2] = 4x8 + 5x10 + 6x12 = 32 + 50 + 72 = 154
```

Gambar 11.10 Setiap elemen C adalah jumlah perkalian satu baris A dengan satu kolom B. Elemen C[1,1] ditandai hijau.

Dalam program, rumus itu membutuhkan **tiga** perulangan bersarang. Perulangan `I` memilih baris A, perulangan `J` memilih kolom B, dan perulangan terdalam `K` berjalan sepanjang baris A sekaligus sepanjang kolom B, menjumlahkan `A[I, K] * B[K, J]`. Sebelum menjumlahkan, `C[I, J]` harus dikosongkan lebih dulu agar jumlah elemen sebelumnya tidak ikut terbawa.

Program 11.5 `bab11_05_perkalian_matriks.pas` *Perkalian matriks* File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 11.5 — Perkalian matriks
3    A (2 x 3) dikali B (3 x 2) menghasilkan C (2 x 2). Syaratnya:
4    jumlah kolom A sama dengan jumlah baris B. Setiap C[i, j] adalah
5    jumlah perkalian baris i dari A dengan kolom j dari B. }
6
7  program bab11_05_perkalian_matriks;
8
9  const
10   BarisA = 2;
11   KolomA = 3;                          { sama dengan baris B }
12   KolomB = 2;
13   A: array[1..BarisA, 1..KolomA] of Integer = ((1, 2, 3),
14                                                (4, 5, 6));
15   B: array[1..KolomA, 1..KolomB] of Integer = ((7, 8),
16                                                (9, 10),
17                                                (11, 12));
18
19 var
20   C: array[1..BarisA, 1..KolomB] of Integer;
21   I, J, K: Integer;
22
23 begin
24   for I := 1 to BarisA do
25     for J := 1 to KolomB do
26     begin
27       C[I, J] := 0;
28       for K := 1 to KolomA do          { baris I dari A x kolom J dari B }
29         C[I, J] := C[I, J] + A[I, K] * B[K, J];
30     end;
31
32   WriteLn('Hasil A x B:');
33   for I := 1 to BarisA do
34   begin
35     for J := 1 to KolomB do
36       Write(C[I, J]:6);
37     WriteLn;
38   end;
39
40   { Rincian satu elemen agar polanya terlihat }
41   WriteLn('C[1,1] = 1x7 + 2x9 + 3x11 = ', C[1, 1]);
42 end.
```



Gambar 11.11 Hasil perkalian dan rincian satu elemen

Baris	Penjelasan
10–12	Ukuran matriks ditulis sebagai konstanta: `KolomA` dipakai bersama sebagai banyak kolom A sekaligus banyak baris B, sehingga syarat perkalian otomatis terpenuhi.
13–17	Dua konstanta bertipe, masing-masing langsung menyebut bentuk arraynya (tanpa nama tipe, karena tidak dipakai sebagai parameter).
20	Matriks hasil `C` berukuran `BarisA x KolomB`, yaitu 2 x 2.
24–26	Dua perulangan luar: untuk setiap elemen `C[I, J]` yang akan dihitung. Karena isi perulangan `J` lebih dari satu perintah, dibungkus `begin..end`.
27	Mengosongkan elemen hasil sebelum dijumlahkan. Tanpa baris ini, hasilnya bergantung pada isi awal `C`.
28–29	Perulangan `K` mengumpulkan perkalian `A[I, K] * B[K, J]` ke `C[I, J]`. Indeks `K` bergerak di kolom A dan di baris B pada saat yang sama.
33–38	Mencetak matriks hasil, enam kolom layar per elemen, satu baris matriks per baris layar.
41	Mencetak rincian `C[1,1]` sebagai pembuktian: `1x7 + 2x9 + 3x11` memang 58.

INFO

Perkalian matriks tidak bersifat komutatif: pada umumnya `A x B` tidak sama dengan `B x A`. Pada contoh ini `B x A` bahkan menghasilkan matriks 3 x 3, bukan 2 x 2.

COBA SENDIRI

Ubah `KolomB` menjadi 3 dan tambahkan satu kolom pada konstanta `B` (misalnya `13, 14, 15` pada baris-barisnya). Hasil kini 2 x 3. Perhatikan bahwa kode perulangan tidak perlu diubah sama sekali.

11.8 Kesalahan yang Sering Terjadi

11.8.1 Indeks melewati batas

Kesalahan terbesar pada array sudah Anda lihat: perulangan yang kelebihan satu putaran, atau indeks yang dimulai dari 0 padahal array dimulai dari 1. Gejalanya runtime error 201. Cara menghindarinya: tulis batas perulangan dengan konstanta yang sama dengan deklarasi array, dan periksa batas awal serta akhir.

11.8.2 Tipe elemen tidak cocok

Setiap elemen array bertipe sama dengan yang disebut setelah `of`. Mengisi elemen dengan nilai bertipe lain ditolak kompiler.

tipe.pas

```
1  program Tipe;
2  var
3    Nilai: array[1..3] of Integer;
```

```
4  begin
5    Nilai[1] := 'tujuh puluh';
6    WriteLn(Nilai[1]);
7  end.
```

PESAN KOMPILER

```
tipe.pas(5,15) Error: Tipe tidak cocok: String dan Integer
```

Gambar 11.12 Array berisi Integer, tetapi diisi teks

11.8.3 Indeks salah jumlah

Array dua dimensi memerlukan dua indeks, dan array satu dimensi memerlukan satu. Salah menghitung kurung siku adalah kesalahan ketik yang umum.

dimensi.pas

```
1  program Dimensi;
2  var
3    Nilai: array[1..3] of Integer;
4  begin
5    Nilai[1, 2] := 70;
6    WriteLn(Nilai[1]);
7  end.
```

PESAN KOMPILER

```
dimensi.pas(5,3) Error: Qualifier tidak valid
```

Gambar 11.13 Array satu dimensi dipanggil dengan dua indeks

11.8.4 Lupa mengosongkan penampung

Pada perkalian matriks, lupa menulis `C[I, J] := 0` sebelum perulangan `K` adalah kesalahan logika yang tidak ditolak kompiler. Program menghasilkan angka, tetapi angka yang salah, terutama bila array sudah pernah dipakai untuk hal lain. Aturan umumnya: variabel yang dipakai sebagai penjumlah harus diberi nilai awal tepat sebelum dipakai, seperti `Jumlah := 0` pada Program 11.2 dan `Total := 0` pada Program 11.3.

11.9 Rangkuman

- Array adalah sederet variabel sejenis dengan satu nama. Elemennya ditunjuk oleh indeks dalam kurung siku, misalnya `Nilai[3]`.
- Deklarasinya `array[batasBawah..batasAtas] of Tipe`. Banyak elemen adalah batas atas dikurangi batas bawah ditambah 1.
- Indeks boleh berupa variabel, sehingga perulangan `for` dapat mengolah seluruh elemen dengan satu kumpulan perintah.
- Indeks di luar batas menghentikan program dengan runtime error 201. Gunakan konstanta yang sama untuk ukuran array dan batas perulangan.

- Mencari nilai terbesar atau terkecil: mulai dari elemen pertama, bandingkan dengan elemen lain, dan simpan juga posisinya bila diperlukan. Rata-rata adalah jumlah dibagi banyak data.
- Array dua dimensi `array[1..B, 1..K]` ditulis `Tabel[baris, kolom]` dan ditelusuri dengan dua perulangan bersarang.
- Konstanta bertipe memberi nilai awal pada array; tipe buatan sendiri (`type`) diperlukan agar array dapat menjadi parameter prosedur.
- Penjumlahan matriks bekerja elemen demi elemen; perkalian matriks memerlukan tiga perulangan, dan jumlah kolom A harus sama dengan jumlah baris B.

11.10 Latihan

1. **DASAR** Buat program yang membaca 7 suhu harian ke dalam array, lalu menampilkannya dengan format `Hari ke-1: 31` dan seterusnya.
2. **DASAR** Ubah Program 11.1 agar membaca 8 nilai dan menampilkan jumlah seluruh elemen. Pastikan hanya konstanta `Banyak` yang berubah.
3. **MENENGAH** Tambahkan ke Program 11.2 perhitungan banyaknya data yang nilainya di atas rata-rata. Petunjuk: rata-rata baru diketahui setelah perulangan pertama selesai, jadi dibutuhkan perulangan kedua.
4. **MENENGAH** Buat program yang membaca 10 bilangan, lalu menukar elemen pertama dengan terakhir, kedua dengan kedua terakhir, dan seterusnya, sehingga seluruh array terbalik di tempat. Tampilkan hasilnya.
5. **MENENGAH** Ubah Program 11.3 agar juga menampilkan nilai tertinggi tiap siswa di kolom terakhir tabel.
6. **TANTANGAN** Tulis prosedur yang menerima matriks 3 x 3 dan menampilkan **transposnya** (baris menjadi kolom, kolom menjadi baris). Gunakan tipe `TMatriks` seperti pada Program 11.4.
7. **TANTANGAN** Buat program yang membaca matriks 3 x 3 dari keyboard dan menghitung jumlah elemen pada **diagonal utama** (elemen yang indeks baris dan kolomnya sama).

BAGIAN IV · STRUKTUR DATA

BAB 12

Mengolah String

TUJUAN PEMBELAJARAN

- Memahami string sebagai deretan karakter berindeks yang dimulai dari 1, dan membaca atau mengubah satu karakternya lewat `S[I]`.
- Memakai `Length`, `Copy`, `Pos`, `Insert`, `Delete`, `UpCase`, dan penyambungan string.
- Menulis fungsi yang mengembalikan string, misalnya untuk membalik huruf dan membalik urutan kata.
- Memeriksa palindrom dengan membandingkan karakter dari kedua ujung string.
- Menghitung vokal, konsonan, angka, dan spasi dengan `case` pada karakter.
- Mengubah bilangan menjadi string dengan `Str` dan sebaliknya dengan `Val`, serta memeriksa masukan yang salah lewat kode galat.

PROGRAM DALAM BAB INI

12.1 `bab12_01_fungsi_string.pas`
Fungsi dan prosedur string

12.2 `bab12_02_balik_kata.pas`
Membalik kata

12.3 `bab12_03_palindrom.pas`
Palindrom

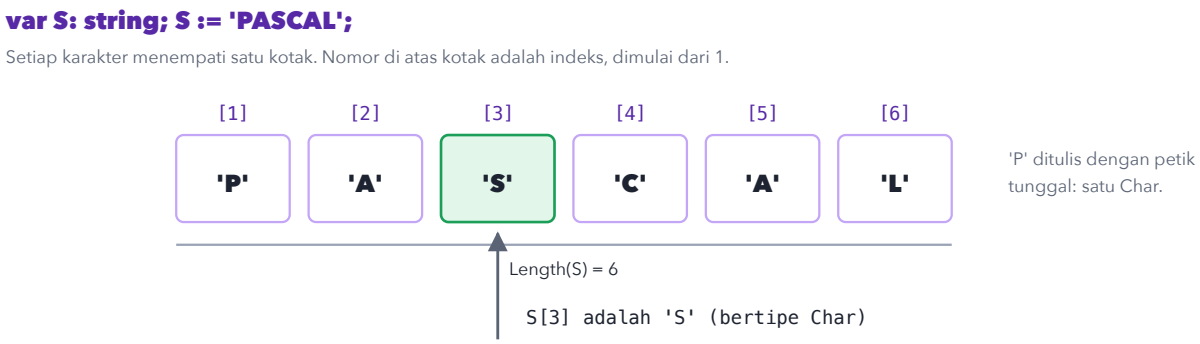
12.4 `bab12_04_hitung_vokal.pas`
Menghitung vokal

12.5 `bab12_05_konversi_angka_string.pas`
Konversi angka dan string

Hampir setiap program yang berguna berurusan dengan teks: nama, alamat, kalimat, kode pos, nomor telepon. Anda sudah memakai string sejak Bab 3 untuk menyimpan dan menampilkan teks. Sekarang saatnya membuka isinya. Di Pascal, string tidak lain adalah deretan karakter yang masing-masing punya nomor urut, sama seperti elemen array pada Bab 11. Dengan perulangan dan beberapa fungsi bawaan, Anda dapat memotong, mencari, menyisipkan, membalik, dan memeriksa teks.

12.1 String sebagai Deretan Karakter

Sebuah nilai bertipe `string` terdiri dari nol atau lebih karakter. Karakter pertama bernomor **1**, karakter kedua bernomor 2, dan seterusnya. Untuk mengambil satu karakter, tulis nama string diikuti nomornya di dalam kurung siku, persis seperti elemen array: `S[3]` adalah karakter ketiga dari `S`, bertipe `Char`. Banyaknya karakter disebut **panjang** string, dan diperoleh dari fungsi `Length(S)`. String yang panjangnya nol, ditulis `''`, disebut **string kosong**.



Gambar 12.1 String `'PASCAL'`: enam karakter dengan indeks 1 sampai 6. `Length` memberi tahu jumlahnya.

Karena karakter-karakternya berindeks, hampir semua pola yang Anda pelajari pada array berlaku juga untuk string: perulangan `for I := 1 to Length(S)` menelusuri teks dari awal sampai akhir, dan `for I := Length(S) downto 1` menelusurinya dari belakang. Bedanya, string memiliki satu set fungsi bawaan khusus untuk pekerjaan yang sering dilakukan pada teks.

1 Membaca

`S[I]` mengambil satu karakter. `Length(S)` memberi panjangnya.

2 Memotong dan mencari

`Copy` mengambil potongan, `Pos` mencari letak sebuah teks dalam string.

3 Mengubah

`Insert` menyisipkan, `Delete` menghapus, `UpCase` membesarkan huruf, dan `+` menyambung.

7 **ATURAN TURBO PASCAL 7**

Di Turbo Pascal 7, `string` panjangnya paling banyak **255 karakter**. Anda juga dapat membatasi panjang maksimum dengan `string[20]`. Nilai yang lebih panjang dari batasnya dipotong diam-diam. Karakter dalam string bertipe `Char`, dan untuk `Char` yang dipakai adalah petik tunggal: `'A'`.

12.2 Fungsi dan Prosedur String

Tabel berikut merangkum enam alat yang dipakai Program 12.1, dengan hasilnya untuk string `'Belajar Pascal itu menyenangkan'`.

Tabel 12.1 Fungsi dan prosedur string bawaan

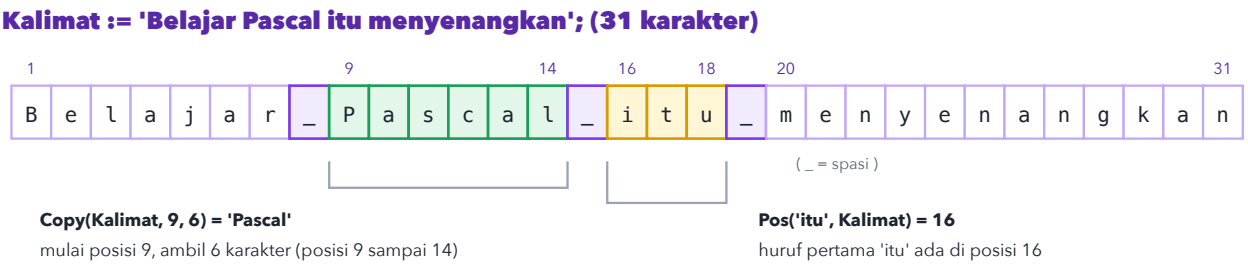
Bentuk	Jenis	Fungsinya	Contoh hasil
`Length(S)`	fungsi	banyak karakter	31
`Copy(S, Awal, N)`	fungsi	mengambil N karakter mulai posisi Awal	`Copy(S, 9, 6)` = `'Pascal'`
`Pos(Cari, S)`	fungsi	posisi pertama Cari di S; 0 bila tidak ada	`Pos('itu', S)` = 16
`Insert(T, S, P)`	prosedur	menyisipkan T ke dalam S mulai posisi P	mengubah S
`Delete(S, P, N)`	prosedur	menghapus N karakter mulai posisi P	mengubah S
`UpCase(C)`	fungsi	mengubah satu `Char` huruf kecil menjadi kapital	`UpCase('a')` = `'A'`

BENTUK UMUM

```
Hasil := Length(S);
Potongan := Copy(S, posisiAwal, banyak);
Letak := Pos(yangDicari, S);        { 0 bila tidak ditemukan }

Insert(teksSisipan, S, posisi);       { mengubah S }
Delete(S, posisi, banyak);          { mengubah S }
Besar := UpCase(karakter);
```

Perhatikan perbedaan **fungsi** dan **prosedur**. Fungsi (`Length`, `Copy`, `Pos`, `UpCase`) menghasilkan nilai dan **tidak mengubah** string asalnya, sehingga dipakai di dalam ekspresi. Prosedur (`Insert`, `Delete`) **mengubah** variabel string yang diberikan, sehingga ditulis sebagai perintah tersendiri. Gambar berikut memperlihatkan keempat operasi itu pada string contoh.



Insert('sangat ', Kalimat, 20)
Posisi 20 adalah awal kata 'menyenangkan'. Teks sisipan masuk di posisi itu, dan sisa kalimat bergeser ke kanan.

Delete(Kalimat, 1, 8)
Menghapus 8 karakter mulai posisi 1: 'Belajar' dan spasinya. Sisa karakter bergeser ke kiri.

Gambar 12.2 Posisi karakter pada `Kalimat`. `Copy` dan `Pos` bekerja dengan nomor posisi yang sama seperti indeks `Kalimat[I]`.

Program 12.1 mencoba semuanya berurutan. Buka `bab12_01_fungsi_string.pas` lewat `File ▸ Examples...` dan jalankan. Program ini tidak meminta masukan.

Program 12.1 `bab12_01_fungsi_string.pas` *Fungsi dan prosedur string* File ▸ Examples...

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 12.1 — Fungsi dan prosedur string
  Length(S)          panjang string
  Copy(S, Awal, N)   mengambil N karakter mulai posisi Awal
  Pos(Cari, S)       posisi pertama Cari di S, 0 bila tidak ada
  Insert(T, S, P)    menyisipkan T ke S pada posisi P
  Delete(S, P, N)    menghapus N karakter mulai posisi P
  UpCase(C)          huruf kecil menjadi huruf besar (satu Char) }

program bab12_01_fungsi_string;

var
  Kalimat, Kata: string;
  I, Posisi: Integer;

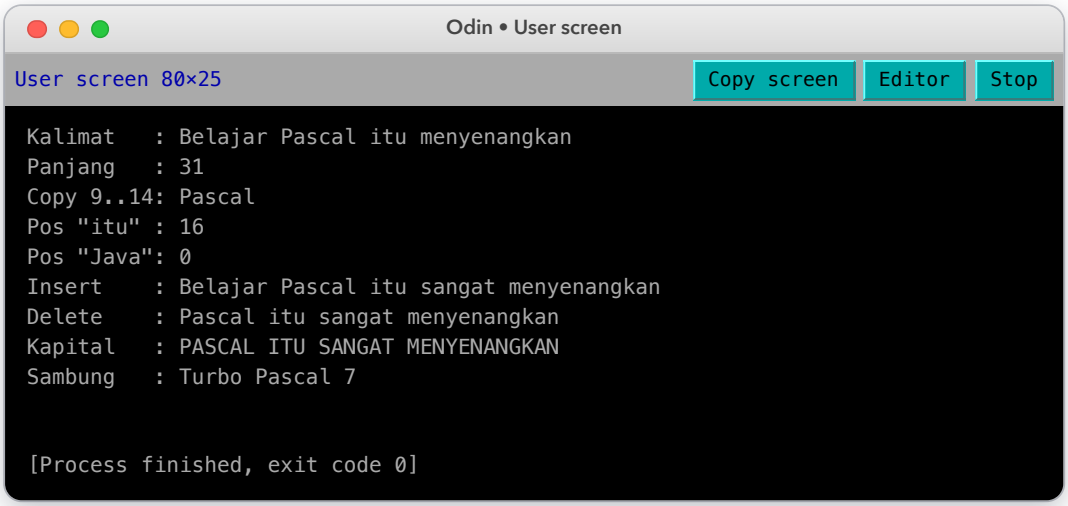
begin
  Kalimat := 'Belajar Pascal itu menyenangkan';
  WriteLn('Kalimat   : ', Kalimat);
  WriteLn('Panjang   : ', Length(Kalimat));
  WriteLn('Copy 9..14: ', Copy(Kalimat, 9, 6));     { 'Pascal' }

  Posisi := Pos('itu', Kalimat);
  WriteLn('Pos "itu" : ', Posisi);
  WriteLn('Pos "Java": ', Pos('Java', Kalimat));    { 0: tidak ditemukan }

  { Insert dan Delete mengubah variabelnya langsung }
  Insert('sangat ', Kalimat, Pos('menyenangkan', Kalimat));
  WriteLn('Insert    : ', Kalimat);
  Delete(Kalimat, 1, 8);                            { buang 'Belajar ' }
  WriteLn('Delete    : ', Kalimat);

  { Mengubah seluruh huruf menjadi kapital, satu per satu }
  Kata := Kalimat;
  for I := 1 to Length(Kata) do
```

```
35      Kata[I] := UpCase(Kata[I]);
36    WriteLn('Kapital   : ', Kata);
37
38    { Menyambung string: operator + atau fungsi Concat }
39    WriteLn('Sambung   : ', Concat('Turbo', ' ', 'Pascal'), ' ' + '7');
40  end.
```



Gambar 12.3 Keluaran Program 12.1

Baris	Penjelasan
12–14	Dua variabel string, `Kalimat` dan `Kata`, serta dua `Integer` untuk indeks dan posisi.
17–19	Mengisi `Kalimat` dengan literal string, lalu mencetaknya beserta panjangnya, yaitu 31 karakter.
20	`Copy(Kalimat, 9, 6)` mengambil enam karakter mulai dari posisi 9: `Pascal`. Perhatikan bahwa argumen ketiga adalah *banyak karakter*, bukan posisi akhir.
22–24	`Pos('itu', Kalimat)` memberi posisi 16. `Pos('Java', Kalimat)` memberi 0 karena kata itu tidak ada. Nilai 0 adalah cara `Pos` mengatakan "tidak ditemukan", karena posisi karakter selalu dimulai dari 1.
27–28	`Insert` menyisipkan `'sangat '` (dengan spasi di akhir) sebelum kata `menyenangkan`. Posisi sisipan didapat dari `Pos` agar tidak perlu menghitung manual. `Kalimat` sendiri berubah.
29–30	`Delete(Kalimat, 1, 8)` membuang delapan karakter pertama, yaitu `Belajar` dan satu spasi. Keluaran mulai dari `Pascal`.
33–36	Salinan kalimat dibesarkan satu karakter demi satu: `UpCase` menerima satu `Char`, jadi harus dipanggil di dalam perulangan. Hasilnya disimpan kembali ke `Kata[I]`. Mengubah satu karakter lewat indeks diperbolehkan.
39	Dua cara menyambung string: fungsi `Concat` dan operator `+`. Keduanya menghasilkan string baru.

AWAS

`Insert` dan `Delete` mengubah variabelnya, jadi nilai lama hilang. Bila Anda masih membutuhkan string aslinya, salin dahulu ke variabel lain, seperti baris 33 menyalin `Kalimat` ke `Kata` sebelum diubah. Perhatikan pula urutan argumen `Pos`: yang dicari lebih dulu, baru string tempat mencarinya.

COBA SENDIRI

Tambahkan baris yang memakai `Pos(' ', Kalimat)` untuk menemukan spasi pertama, lalu `Copy` untuk mencetak kata pertama saja. Petunjuk: kata pertama adalah `Copy(Kalimat, 1, Posisi - 1)`.

12.3 Membalik Huruf dan Membalik Kata

Membalik string adalah latihan klasik yang melatih Anda menelusuri string dan membangun string baru. Ada dua versi yang berbeda sifatnya. **Membalik huruf** menjadikan `pascal` menjadi `lacsap`. **Membalik urutan kata** menjadikan `saya suka pascal` menjadi `pascal suka saya`, dengan huruf di dalam kata tetap utuh.

Cara membalik huruf: mulai dari string hasil yang kosong, baca string asal dari karakter terakhir ke pertama, dan tempelkan tiap karakter ke belakang hasil. Karena yang dibaca paling akhir menjadi yang tertempel paling awal, hasilnya terbalik. Membalik urutan kata lebih rumit: program memotong satu kata dari depan string, lalu menaruhnya di **depan** hasil, dan mengulanginya sampai string asal habis.

Kedua pekerjaan ini ditulis sebagai **fungsi yang mengembalikan string**. Di Turbo Pascal, hasil fungsi diberikan dengan menugaskan nilai ke nama fungsinya sendiri (`BalikHuruf := Hasil;`), bukan dengan `Result`.

Program 12.2 `bab12_02_balik_kata.pas` *Membalik kata* File ▸ Examples…

```
{ Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 12.2 - Membalik kata
  Dua latihan: membalik urutan huruf, dan membalik urutan kata.
  Fungsi boleh mengembalikan string. }

program bab12_02_balik_kata;

{ Membaca huruf dari belakang ke depan, lalu menyambungnya }
function BalikHuruf(S: string): string;
var
  I: Integer;
  Hasil: string;
begin
  Hasil := '';                          { mulai dari string kosong }
  for I := Length(S) downto 1 do
    Hasil := Hasil + S[I];
  BalikHuruf := Hasil;
end;

{ Memotong kata pertama berulang kali, lalu menaruhnya di depan hasil }
function BalikUrutanKata(S: string): string;
var
  Spasi: Integer;
  Hasil, Kata: string;
begin
  Hasil := '';
  while S <> '' do
  begin
    Spasi := Pos(' ', S);
```

```
30      if Spasi = 0 then                    { kata terakhir }
31      begin
32        Kata := S;
33        S := '';
34      end
35      else
36      begin
37        Kata := Copy(S, 1, Spasi - 1);
38        Delete(S, 1, Spasi);               { buang kata dan spasinya }
39      end;
40      if Kata <> '' then                   { lewati spasi ganda }
41        if Hasil = '' then
42          Hasil := Kata
43        else
44          Hasil := Kata + ' ' + Hasil;
45    end;
46    BalikUrutanKata := Hasil;
47  end;
48
49  var
50    Kalimat: string;
51
52  begin
53    Write('Ketik sebuah kalimat: ');
54    ReadLn(Kalimat);
55    WriteLn('Huruf dibalik : ', BalikHuruf(Kalimat));
56    WriteLn('Kata dibalik  : ', BalikUrutanKata(Kalimat));
57  end.
```



Gambar 12.4 Kalimat dengan dua spasi di tengah: huruf dibalik, lalu kata dibalik

Pada keluaran, `Huruf dibalik` menampilkan `lacsap akus ayas`: dua spasi tetap ada, karena membalik huruf memperlakukan spasi seperti karakter lain. Pada `Kata dibalik`, spasi ganda dirapikan menjadi satu spasi. Gambar berikut menelusuri `BalikUrutanKata` untuk masukan yang sama, `saya suka pascal` (dengan dua spasi setelah `saya`).

Jejak BalikUrutanKata('saya suka pascal') (_ = spasi)

Putaran	S sebelum	Kata yang dipotong	Hasil sesudah
1	`saya__suka_pascal`	`saya`	`saya`
2	`_suka_pascal`	`'' (kosong)`	`saya (tidak berubah)`
3	`suka_pascal`	`suka`	`suka_saya`
4	`pascal`	`pascal`	`pascal_suka_saya`

Putaran 2 memotong kata kosong (spasi ganda), sehingga dilewati. Putaran 4: tidak ada spasi lagi, S habis.
Kata baru selalu ditaruh di depan Hasil, sehingga kata yang dipotong paling awal berakhir paling belakang.

Gambar 12.5 Empat putaran `while S <> ''`. Spasi ganda menghasilkan kata kosong yang dilewati.

Baris	Penjelasan
9–18	Fungsi `BalikHuruf`. Baris 14 mulai dari string kosong. Perulangan `downto` pada baris 15–16 menempelkan `S[I]` dari karakter terakhir ke karakter pertama. Baris 17 mengembalikan hasilnya lewat nama fungsi.
21–24	Fungsi `BalikUrutanKata`. `S` adalah salinan parameter, sehingga boleh dirusak (dipotong sampai habis) tanpa mengubah variabel milik pemanggil.
27	Perulangan berjalan selama `S` belum kosong.
29–34	`Pos(' ', S)` mencari spasi pertama. Bila 0, sisa string adalah kata terakhir: ambil semuanya dan kosongkan `S`.
35–39	Bila ada spasi, `Copy(S, 1, Spasi - 1)` mengambil kata di depannya, dan `Delete(S, 1, Spasi)` membuang kata beserta spasinya.
40–44	Kata yang kosong (akibat spasi ganda) dilewati. Kata pertama langsung menjadi hasil, kata berikutnya ditaruh *di depan* hasil dengan satu spasi pemisah.
46	Hasil diberikan ke nama fungsi.
53–56	Program utama membaca satu kalimat dan memanggil kedua fungsi di dalam `WriteLn`.

7 ATURAN TURBO PASCAL 7

Fungsi di Turbo Pascal 7 tidak memiliki variabel `Result`. Hasil fungsi diberikan dengan menulis `NamaFungsi := nilai`. Fungsi boleh mengembalikan `string`, tetapi tidak boleh mengembalikan array atau record.
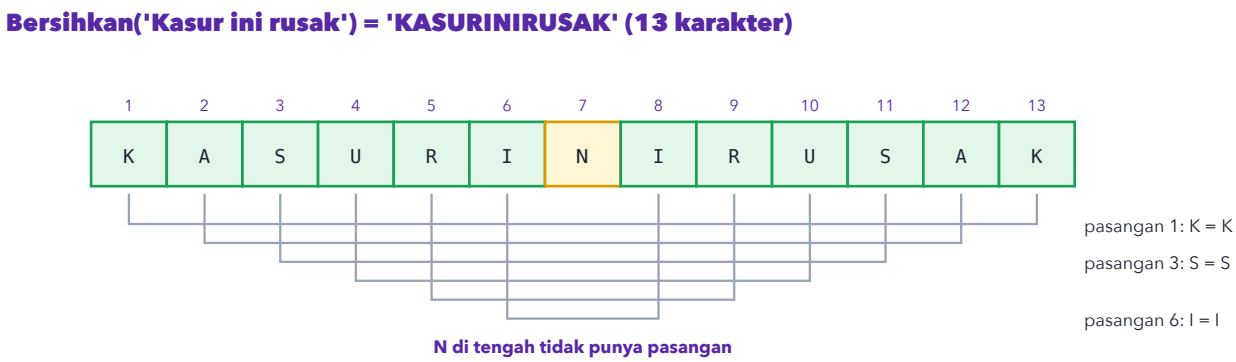
COBA SENDIRI

Jalankan Program 12.2 dengan kalimat `a b c`. Lalu coba kalimat yang diawali dan diakhiri spasi. Apakah hasil `Kata dibalik` memiliki spasi di tepinya? Telusuri bagian mana dari baris 40–44 yang mencegahnya.

12.4 Palindrom

Palindrom adalah kata atau kalimat yang terbaca sama dari depan maupun dari belakang, seperti `kakak`, `katak`, atau `Kasur ini rusak`. Untuk kalimat, spasi dan perbedaan huruf besar dan kecil diabaikan. Program harus mengerjakan dua hal: membersihkan teks (membuang selain huruf dan angka, lalu menjadikan semuanya kapital) dan memeriksa apakah teks bersih itu palindrom.

Memeriksa palindrom tidak perlu membuat string terbalik. Cukup dua penunjuk: `Kiri` mulai dari karakter pertama dan `Kanan` mulai dari karakter terakhir. Keduanya dibandingkan, lalu bergerak ke tengah satu langkah demi satu langkah. Begitu ada pasangan yang berbeda, teks bukan palindrom. Bila kedua penunjuk bertemu tanpa menemukan perbedaan, teks adalah palindrom.



Putaran ke-k membandingkan karakter ke-k dari kiri dengan karakter ke-k dari kanan. Kiri dan Kanan bertemu di karakter 7, perulangan berhenti.

Gambar 12.6 Pasangan karakter yang dibandingkan pada teks bersih. Semua pasangan sama, jadi teks ini palindrom.

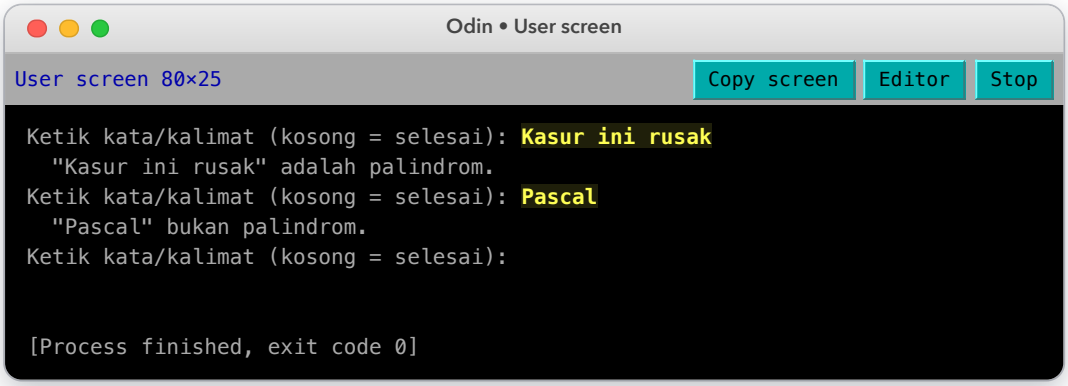
Program 12.3 `bab12_03_palindrom.pas` *Palindrom* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 12.3 — Palindrom
  Palindrom terbaca sama dari depan dan dari belakang, misalnya
  "kakak", "katak", atau "Kasur ini rusak". Spasi dan huruf besar/
  kecil diabaikan, jadi teks dibersihkan dulu sebelum diperiksa. }

program bab12_03_palindrom;

{ Hanya menyimpan huruf dan angka, semuanya dijadikan kapital }
function Bersihkan(S: string): string;
var
  I: Integer;
  C: Char;
  Hasil: string;
begin
  Hasil := '';
  for I := 1 to Length(S) do
  begin
    C := UpCase(S[I]);
    if ((C >= 'A') and (C <= 'Z')) or ((C >= '0') and (C <= '9')) then
      Hasil := Hasil + C;
  end;
  Bersihkan := Hasil;
end;
```

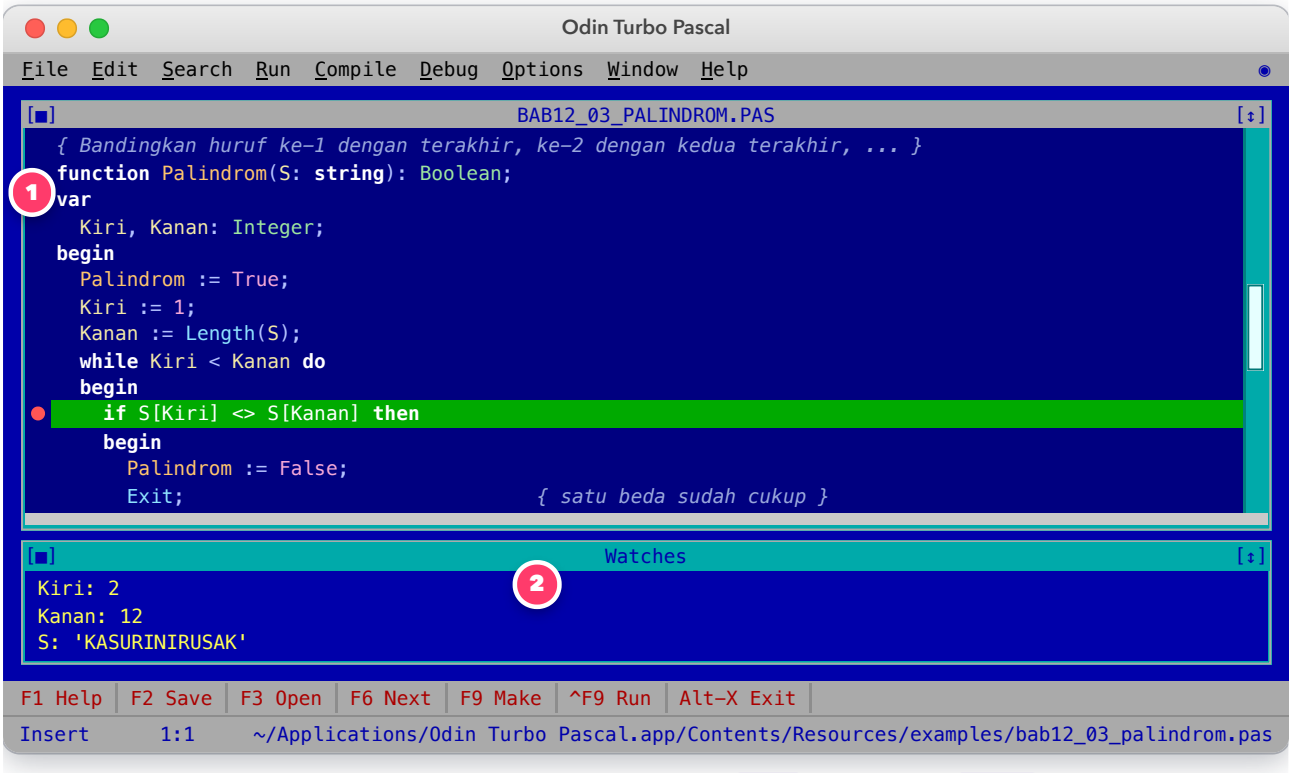
```
25
26  { Bandingkan huruf ke-1 dengan terakhir, ke-2 dengan kedua terakhir, ... }
27  function Palindrom(S: string): Boolean;
28  var
29    Kiri, Kanan: Integer;
30  begin
31    Palindrom := True;
32    Kiri := 1;
33    Kanan := Length(S);
34    while Kiri < Kanan do
35    begin
36      if S[Kiri] <> S[Kanan] then
37      begin
38        Palindrom := False;
39        Exit;                              { satu beda sudah cukup }
40      end;
41      Kiri := Kiri + 1;
42      Kanan := Kanan - 1;
43    end;
44  end;
45
46  var
47    Teks, Bersih: string;
48
49  begin
50    repeat
51      Write('Ketik kata/kalimat (kosong = selesai): ');
52      ReadLn(Teks);
53      if Teks <> '' then
54      begin
55        Bersih := Bersihkan(Teks);
56        if Palindrom(Bersih) then
57          WriteLn('  "', Teks, '" adalah palindrom.')
58        else
59          WriteLn('  "', Teks, '" bukan palindrom.');
60      end;
61    until Teks = '';
62  end.
```



Gambar 12.7 Dua kalimat, lalu **Enter** pada baris kosong untuk mengakhiri

Baris	Penjelasan
10–24	Fungsi `Bersihkan`. Setiap karakter dikapitalkan dengan `UpCase` (baris 19) lalu diuji: hanya huruf `'A'..'Z'` atau angka `'0'..'9'` yang ditempelkan ke `Hasil` (baris 20–21). Spasi dan tanda baca terbuang.
27–33	Fungsi `Palindrom` bertipe `Boolean`. Ia memulai dengan anggapan `True`, `Kiri` di awal, dan `Kanan` di akhir (`Length(S)`).
34	Perulangan hanya berjalan selama `Kiri < Kanan`: pemeriksaan berhenti di tengah, karena membandingkan separuh pertama dengan separuh kedua sudah cukup.
36–40	Bila sepasang karakter berbeda, hasil menjadi `False` dan `Exit` langsung keluar dari fungsi. Di Turbo Pascal, `Exit` tidak membawa nilai; nilai yang sudah ditugaskan ke nama fungsi (baris 38) yang dipakai.
41–42	Kedua penunjuk bergerak satu langkah ke tengah.
50–61	Perulangan `repeat` membaca teks terus-menerus. Baris kosong (`Teks = ''`) mengakhirinya. Teks dibersihkan dulu pada baris 55, lalu `Bersih` diperiksa, tetapi yang ditulis ke layar tetap `Teks` asli.

Untuk melihat penunjuk bergerak, jalankan program di bawah debugger. Pasang *breakpoint* di baris 36 dan tambahkan *watch* untuk `Kiri` dan `Kanan`. Setiap putaran, `Kiri` naik dan `Kanan` turun.



Gambar 12.8 Berhenti di baris 36 pada pasangan kedua: `Kiri` bernilai 2 dan `Kanan` bernilai 12

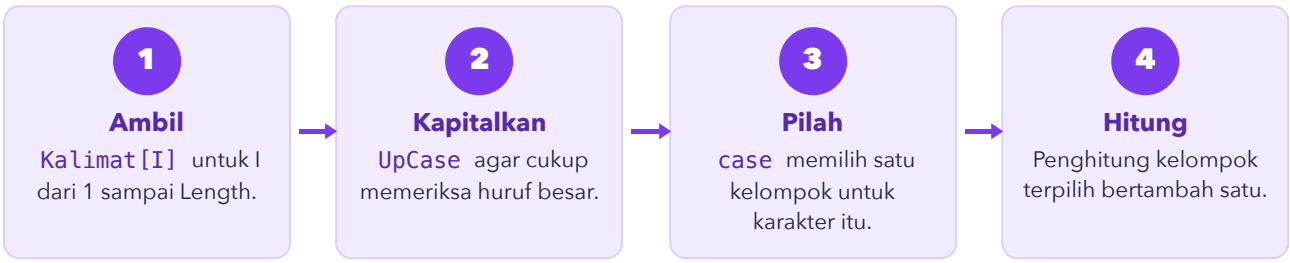
Nomor	Penjelasan
1	**Gutter.** Titik merah dan sorotan hijau pada baris 36, yaitu pembandingan pasangan karakter.
2	**Watches.** `Kiri` dan `Kanan` bergerak ke tengah; `S` adalah teks yang sudah dibersihkan.

COBA SENDIRI

Coba masukan `Ibu Ratna antar ubi`, lalu `Kasur Rusak`, `12321`, dan satu huruf saja. Satu karakter dan string kosong dianggap palindrom oleh fungsi ini. Mengapa? Perhatikan kapan perulangan baris 34 berhenti.

12.5 Menghitung Vokal dan Jenis Karakter Lain

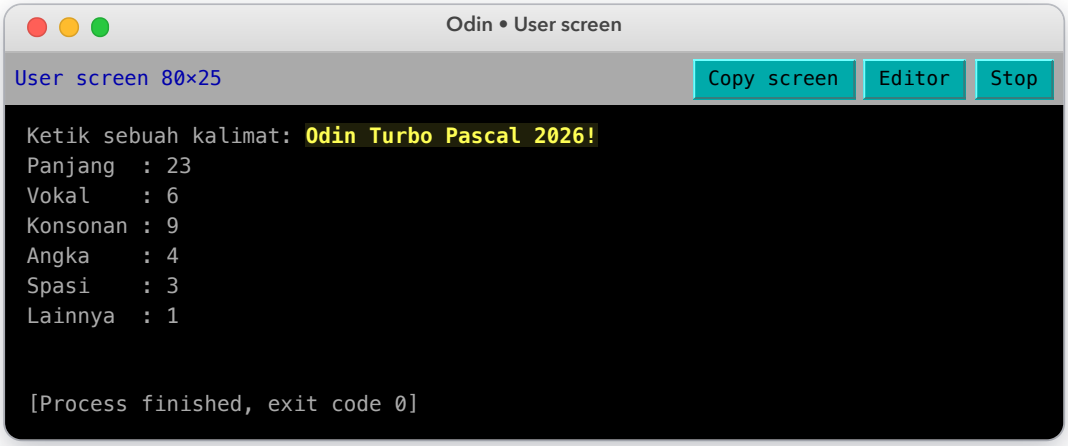
Karena string adalah deretan `Char`, Anda dapat menyortir tiap karakter ke dalam kelompok. Program 12.4 menghitung lima kelompok: vokal, konsonan, angka, spasi, dan lainnya (tanda baca dan simbol). Pekerjaan memilih kelompok ini cocok dengan `case` dari Bab 6, karena label `case` boleh berupa nilai tunggal (`'A'`), daftar (`'A', 'I'`), maupun rentang (`'0'..'9'`).



Program 12.4 `bab12_04_hitung_vokal.pas` *Menghitung vokal* File ▸ Examples…

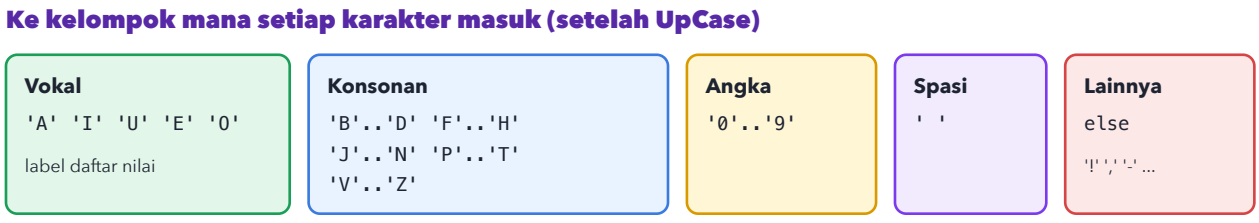
```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 12.4 — Menghitung vokal
3    Setiap karakter diperiksa satu per satu, lalu dihitung menurut
4    jenisnya: vokal, konsonan, angka, spasi, atau lainnya. }
5
6  program bab12_04_hitung_vokal;
7
8  var
9    Kalimat: string;
10   I: Integer;
11   C: Char;
12   Vokal, Konsonan, Angka, Spasi, Lainnya: Integer;
13
14 begin
15   Write('Ketik sebuah kalimat: ');
16   ReadLn(Kalimat);
17
18   Vokal := 0; Konsonan := 0; Angka := 0; Spasi := 0; Lainnya := 0;
19   for I := 1 to Length(Kalimat) do
20   begin
21     C := UpCase(Kalimat[I]);            { cukup periksa huruf besar }
22     case C of
23       'A', 'I', 'U', 'E', 'O':
24         Vokal := Vokal + 1;
25       'B'..'D', 'F'..'H', 'J'..'N', 'P'..'T', 'V'..'Z':
26         Konsonan := Konsonan + 1;
27       '0'..'9':
28         Angka := Angka + 1;
29       ' ':
30         Spasi := Spasi + 1;
```

```
31      else
32        Lainnya := Lainnya + 1;           { tanda baca dan simbol }
33      end;
34    end;
35
36    WriteLn('Panjang  : ', Length(Kalimat));
37    WriteLn('Vokal    : ', Vokal);
38    WriteLn('Konsonan : ', Konsonan);
39    WriteLn('Angka    : ', Angka);
40    WriteLn('Spasi    : ', Spasi);
41    WriteLn('Lainnya  : ', Lainnya);
42  end.
```



Gambar 12.9 Kalimat `Odin Turbo Pascal 2026!`

Mari periksa hasilnya dengan tangan. Kalimat `Odin Turbo Pascal 2026!` terdiri atas 23 karakter. Vokalnya `O`, `i`, `u`, `o`, `a`, `a` (enam); konsonannya `d`, `n`, `T`, `r`, `b`, `P`, `s`, `c`, `l` (sembilan); angkanya 2, 0, 2, 6 (empat); spasinya tiga; dan tanda seru satu. Jumlahnya 6 + 9 + 4 + 3 + 1 = 23, sama dengan panjang kalimat. Penjumlahan semacam ini adalah cara cepat untuk menguji bahwa tidak ada karakter yang terlewat atau terhitung dua kali.



Gambar 12.10 Pembagian kelompok pada `case` baris 22–33. Label tidak boleh bertumpuk.

Baris	Penjelasan
8–12	Satu `string`, satu `Char` penampung, dan lima penghitung.
15–16	Membaca seluruh kalimat dengan `ReadLn` ke dalam variabel string. Spasi di dalam kalimat ikut terbaca.

Baris	Penjelasan
18	Lima penghitung diberi nilai awal 0 sekaligus pada satu baris, dipisah titik koma.
19–21	Perulangan menelusuri semua karakter. `UpCase` menjadikannya huruf besar sehingga label `case` cukup ditulis untuk `'A'..'Z'`.
22–24	Label daftar: karakter vokal menaikkan `Vokal`.
25–26	Label rentang: lima rentang yang menutupi semua huruf konsonan.
27–30	Rentang angka dan label spasi.
31–32	`else` menampung semuanya: tanda baca dan simbol. Di sini tanda seru dihitung sebagai "lainnya".
36–41	Mencetak panjang dan kelima hitungan dengan perataan yang rapi.

7 **ATURAN TURBO PASCAL 7**

`UpCase` di Turbo Pascal hanya mengenal huruf Latin a sampai z. Huruf bersimbol (misalnya huruf beraksen) tidak diubah, sehingga jatuh ke cabang `else`. Karena `UpCase` menerima satu `Char` bukan seluruh string, mengapitalkan string utuh selalu memerlukan perulangan seperti pada Program 12.1.

COBA SENDIRI

Ubah program agar menghitung huruf besar dan huruf kecil secara terpisah. Petunjuk: periksa `Kalimat[I]` langsung (`('A' <= C) and (C <= 'Z')`) sebelum `UpCase` dipakai.

12.6 Mengubah Angka dan String

Angka `1250` dan string `'1250'` terlihat sama di layar, tetapi bagi komputer keduanya berbeda jenis: yang satu bilangan yang bisa dijumlahkan, yang lain deretan empat karakter. Untuk berpindah dari satu bentuk ke bentuk lain, Pascal menyediakan dua prosedur:

BENTUK UMUM

```
Str(bilangan, S);                 { bilangan menjadi string }
Str(bilangan:lebar:desimal, S);   { dengan format, untuk Real }
Val(S, bilangan, Kode);            { string menjadi bilangan }
```

`Str` bekerja seperti `WriteLn` yang hasilnya disimpan ke variabel string, bukan ditampilkan. `Val` memiliki parameter ketiga yang unik, yaitu **kode galat**. Bila seluruh string berhasil dibaca sebagai bilangan, `Kode` bernilai **0**. Bila tidak, `Kode` berisi **nomor posisi karakter pertama yang salah**, dan bilangannya tidak boleh dipercaya.

V Val: kode 0

`Val('1250', Harga, Kode)` menghasilkan `Harga = 1250` dan `Kode = 0`. Aman dipakai.

X Val: kode bukan 0

`Val('12a50', Harga, Kode)` menghasilkan `Kode = 3`, karena `a` adalah karakter ke-3. `Harga` tidak boleh dipakai.

Mengapa tidak langsung memakai `ReadLn(Harga)`? Karena bila pengguna mengetik huruf, program akan berhenti dengan runtime error. Dengan membaca masukan sebagai **string** lalu memeriksanya dengan `Val`, program Anda dapat memberi pesan yang ramah dan meminta pengguna mengulang. Pola ini, *baca sebagai teks, validasi, baru pakai*, dipakai di hampir semua program yang menerima ketikan pengguna.

Program 12.5 menerapkannya pada harga barang. Fungsi `FormatRibuan` memakai `Str` dan `Insert` untuk menyisipkan titik pemisah ribuan, misalnya mengubah `1250000` menjadi `1.250.000`. Program juga menghitung PPN 11% dan menampilkannya dengan `Str` berformat.

FormatRibuan(1250000): titik disisipkan dari belakang, setiap tiga digit

Langkah	P	Perintah	S sesudahnya
Str(N, S); P := 5	5	`Length(S) - 2`	`1250000`
putaran 1	5	`Insert('.', S, 5)`	`1250.000`
putaran 2	2	`Insert('.', S, 2)`	`1.250.000`

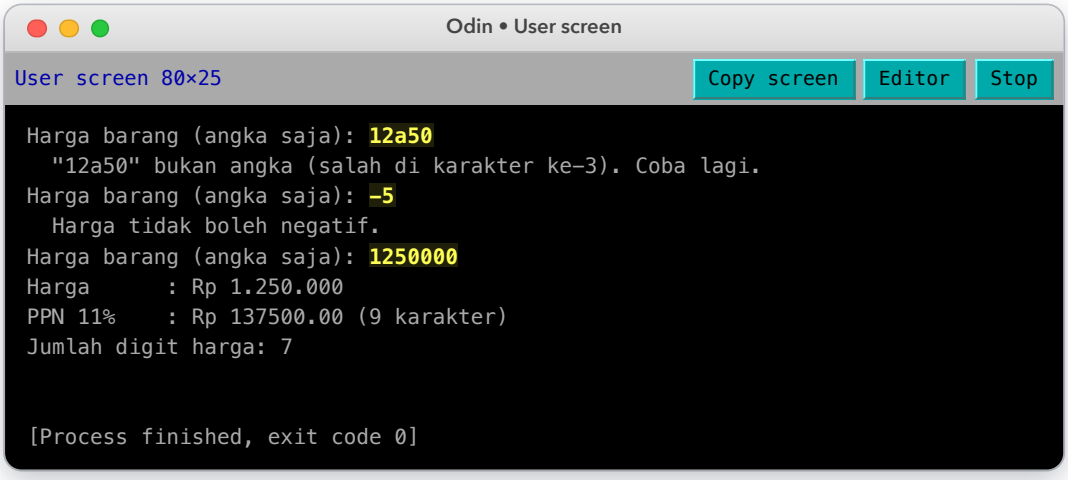
Sesudah putaran 2, P menjadi -1 sehingga perulangan while P > 1 berhenti. Hasilnya 1.250.000.
Menyisipkan dari belakang menjaga posisi digit di sebelah kiri tetap sama, jadi P tidak perlu dihitung ulang.

Gambar 12.11 Dua penyisipan titik oleh `FormatRibuan`. Langkah dimulai dari kelompok tiga digit paling kanan.

Program 12.5 `bab12_05_konversi_angka_string.pas` *Konversi angka dan string* File ▸ Examples…

```
1  { Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 12.5 - Konversi angka dan string
3    Str(Angka, S)        mengubah bilangan menjadi string (boleh berformat)
4    Val(S, Angka, Kode)  mengubah string menjadi bilangan. Kode = 0 bila
5                         berhasil, selain itu posisi karakter yang salah.
6    Dengan Val, masukan yang salah tidak lagi menghentikan program. }
7
8  program bab12_05_konversi_angka_string;
9
10 { Menyisipkan titik setiap tiga digit: 1250000 menjadi 1.250.000.
11   Hanya untuk bilangan positif. }
12 function FormatRibuan(N: LongInt): string;
13 var
14   S: string;
15   P: Integer;
16 begin
17   Str(N, S);                          { bilangan -> string }
18   P := Length(S) - 2;
19   while P > 1 do
20   begin
21     Insert('.', S, P);
22     P := P - 3;
```

```
23    end;
24    FormatRibuan := S;
25  end;
26
27  var
28    Teks, Hasil: string;
29    Harga: LongInt;
30    Kode: Integer;
31    Pajak: Real;
32
33  begin
34    { Membaca sebagai string dulu, lalu diperiksa dengan Val }
35    repeat
36      Write('Harga barang (angka saja): ');
37      ReadLn(Teks);
38      Val(Teks, Harga, Kode);              { string -> bilangan }
39      if Kode <> 0 then
40        WriteLn('  "', Teks, '" bukan angka (salah di karakter ke-',
41          Kode, '). Coba lagi.')
42      else if Harga < 0 then
43        WriteLn('  Harga tidak boleh negatif.');
44    until (Kode = 0) and (Harga >= 0);
45
46    WriteLn('Harga      : Rp ', FormatRibuan(Harga));
47
48    Pajak := Harga * 0.11;
49    Str(Pajak:0:2, Hasil);                 { Str juga menerima format }
50    WriteLn('PPN 11%    : Rp ', Hasil, ' (', Length(Hasil), ' karakter)');
51
52    Str(Harga, Hasil);
53    WriteLn('Jumlah digit harga: ', Length(Hasil));
54  end.
```



Gambar 12.12 Masukan `12a50` (huruf), `-5` (negatif), lalu `1250000` (sah)

Perhatikan bagaimana program menolak dua masukan pertama dengan pesan yang berbeda dan baru menerima yang ketiga. Kode galat 3 berarti karakter ketiga, yaitu `a`, tidak dapat dibaca sebagai angka.

Baris	Penjelasan
12–25	Fungsi `FormatRibuan` menerima `LongInt` dan mengembalikan string. Baris 17 mengubah angka menjadi string dengan `Str`.

Baris	Penjelasan
18–23	Titik pertama disisipkan tiga karakter dari kanan (`Length(S) - 2`). Selama posisinya masih lebih dari 1, titik disisipkan dan posisi mundur tiga langkah. Syarat `P > 1` mencegah titik di awal angka seperti `.250` .
28–31	`Teks` menampung ketikan mentah, `Harga` bilangannya, `Kode` kode galat `Val` , dan `Pajak` bertipe `Real` .
35–38	Masukan dibaca sebagai string, lalu `Val` mencoba mengubahnya menjadi `Harga` .
39–41	Bila `Kode` tidak nol, tampilkan pesan lengkap dengan posisi karakter yang salah.
42–43	Bila bilangan terbaca tetapi negatif, tampilkan pesan lain. Perhatikan `else if` yang memeriksa syarat kedua hanya bila syarat pertama `False` .
44	Perulangan diulang sampai `Kode = 0` *dan* `Harga >= 0` .
46	Harga dicetak lewat `FormatRibuan` .
48–50	`Harga * 0.11` adalah `Real` . `Str(Pajak:0:2, Hasil)` menyimpannya sebagai teks dengan dua desimal, lalu `Length(Hasil)` menghitung panjang teks itu, termasuk titik desimal.
52–53	Memakai `Str` lagi untuk menghitung jumlah digit harga.

7 **ATURAN TURBO PASCAL 7**

Pada Turbo Pascal 7, `Val` menerima bilangan bulat maupun real sebagai tujuan, dan jenis tujuan menentukan format string yang dianggap sah. Untuk `Real` yang dicetak ke string, selalu beri format `:lebar:desimal` seperti pada baris 49.

COBA SENDIRI

Jalankan Program 12.5 dengan masukan `abc` , lalu `12 34` (ada spasi), lalu string kosong (langsung Enter). Di karakter keberapa `Val` menunjuk kesalahan pada tiap kasus?

12.7 Kesalahan yang Sering Terjadi

12.7.1 Menugaskan string ke Char

Variabel `Char` hanya menampung **satu** karakter. Memberinya teks dua huruf atau lebih ditolak oleh kompiler.

chardua.pas

```
1  program CharDua;
2  var
3    C: Char;
4  begin
5    C := 'AB';
6    WriteLn(C);
7  end.
```

PESAN KOMPILER

```
chardua.pas(5,8) Error: Tipe tidak cocok: String dan Char
```

Gambar 12.13 Char tidak bisa menampung dua karakter

Bila yang Anda inginkan adalah teks, deklarasikan variabelnya sebagai `string`. Bila yang Anda inginkan adalah karakter pertama, tulis `C := S[1];`.

12.7.2 Menyimpan angka bukan di tempat yang tepat

Keliru memakai variabel string untuk menerima hasil `Length`, atau sebaliknya, adalah kesalahan tipe yang umum. `Length` selalu menghasilkan bilangan.

panjang.pas

```
1  program Panjang;
2  var
3    Teks, Banyak: string;
4  begin
5    Teks := 'Pascal';
6    Banyak := Length(Teks);
7    WriteLn('Panjang: ', Banyak);
8  end.
```

PESAN KOMPILER

```
panjang.pas(6,13) Error: Tipe tidak cocok: LongInt dan String
```

Gambar 12.14 Hasil Length adalah bilangan, bukan string

Perbaikannya: deklarasikan `Banyak` sebagai `Integer`. Bila Anda memang membutuhkan bilangan itu sebagai teks, gunakan `Str(Length(Teks), Banyak)`.

12.7.3 Val dengan kode galat yang salah tipe

Parameter ketiga `Val` harus bertipe bilangan bulat. Memberinya variabel bertipe lain ditolak.

valsalah.pas

```
1  program ValSalah;
2  var
3    Angka: Integer;
4    Kode: Real;
5  begin
6    Val('123', Angka, Kode);
7    WriteLn(Angka);
8  end.
```

PESAN KOMPILER

```
valsalah.pas(6,14) Error: Identifier variabel diharapkan
```

Gambar 12.15 Kode galat Val harus Integer, bukan Real

12.7.4 Indeks di luar panjang string

Seperti array, string yang diakses di luar rentangnya bermasalah. Akses karakter nomor 0 atau nomor yang lebih besar dari `Length` mengambil data yang bukan bagian dari teks. Pada perulangan selalu batasi indeks dengan `Length(S)`, dan pada `Copy`, `Insert`, `Delete` pastikan posisi awal berada di dalam string. Ingat pula bahwa `Pos` bernilai 0 bila teks tidak ditemukan, sehingga hasilnya harus diperiksa sebelum dipakai sebagai posisi.

12.8 Rangkuman

- String adalah deretan karakter yang berindeks mulai dari 1. `S[I]` adalah karakter ke- `I` bertipe `Char`, dan `Length(S)` adalah banyaknya karakter.
- `Copy(S, Awal, N)` mengambil potongan, `Pos(Cari, S)` mencari posisi (0 bila tidak ada), `Insert` menyisipkan, dan `Delete` menghapus. `Insert` dan `Delete` mengubah variabelnya.
- `UpCase` bekerja pada satu `Char`; untuk string utuh dibutuhkan perulangan. String disambung dengan `+` atau `Concat`.
- Fungsi boleh mengembalikan string. Di Turbo Pascal hasilnya ditugaskan ke nama fungsi, bukan ke `Result`.
- Membalik huruf: baca dari belakang ke depan dan tempelkan ke hasil. Membalik kata: potong kata di depan, taruh di depan hasil.
- Palindrom diperiksa dengan membandingkan karakter dari kedua ujung bergerak ke tengah, setelah teks dibersihkan dari spasi dan dikapitalkan.
- Menghitung jenis karakter memakai `case` dengan label daftar dan rentang; `else` menampung sisanya.
- `Str` mengubah bilangan menjadi string (boleh berformat), `Val` mengubah string menjadi bilangan dengan kode galat 0 bila sukses atau posisi karakter yang salah. Baca masukan sebagai string, validasi dengan `Val`, baru pakai.

12.9 Latihan

1. **DASAR** Baca sebuah kata, lalu tampilkan karakter pertama, karakter terakhir, dan panjangnya.
2. **DASAR** Baca sebuah kalimat dan tampilkan kata pertamanya saja (petunjuk: `Pos` dan `Copy`).
3. **MENENGAH** Ubah Program 12.4 agar juga menampilkan persentase vokal terhadap seluruh huruf, dengan dua desimal. Hati-hati dengan pembagian oleh nol bila kalimat tidak mengandung huruf.
4. **MENENGAH** Buat program yang membaca satu kalimat dan mengubah huruf pertama setiap kata menjadi kapital, sedangkan huruf lainnya tetap.
5. **MENENGAH** Buat program yang menghitung banyaknya kata dalam kalimat, dengan memperhitungkan spasi ganda dan spasi di awal atau akhir.

6. **TANTANGAN** Tulis fungsi `Anagram(A, B: string): Boolean` yang memeriksa apakah dua kata memiliki huruf yang sama persis dengan urutan berbeda (misalnya `kasur` dan `rusak`), mengabaikan huruf besar dan kecil.

7. **TANTANGAN** Tulis program kalkulator sederhana: baca ekspresi seperti `12+30` sebagai string, cari tanda `+` dengan `Pos`, pecah menjadi dua bagian dengan `Copy`, ubah keduanya dengan `Val`, dan tampilkan jumlahnya. Tolak masukan yang salah.

BAGIAN IV · STRUKTUR DATA

BAB 13

Record

TUJUAN PEMBELAJARAN

- Menjelaskan perbedaan *array* (banyak data sejenis) dan *record* (beberapa data berbeda jenis yang satu kesatuan).
- Mendeklarasikan tipe `record`, mengisi dan membaca *field* dengan notasi titik.
- Memakai pernyataan `with` agar penulisan field lebih ringkas, dan menyalin record utuh dengan satu penugasan.
- Membuat tabel data dengan *array of record* dan mengolahnya dengan perulangan.
- Menyusun record bersarang (record di dalam record) dan mengaksesnya dengan titik berantai.
- Memantau isi record dengan jendela *Watches* pada debugger.

PROGRAM DALAM BAB INI

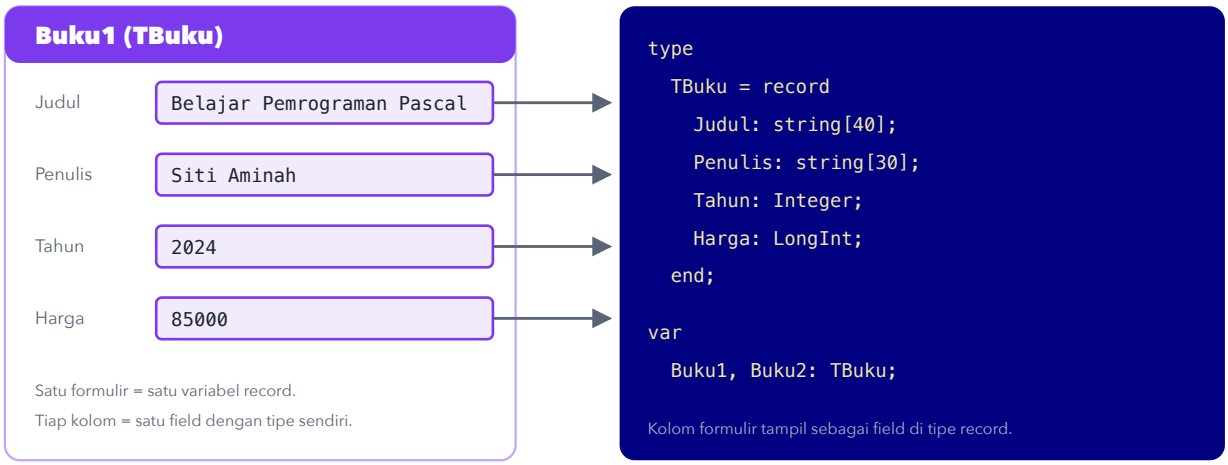
13.1	`bab13_01_record_dasar.pas`	Record
13.2	`bab13_02_array_of_record.pas`	Array of record
13.3	`bab13_03_record_bersarang.pas`	Record bersarang
13.4	`bab13_04_data_mahasiswa.pas`	Latihan: data mahasiswa

Data di dunia nyata jarang berdiri sendiri. Sebuah buku punya judul, penulis, tahun terbit, dan harga; seorang siswa punya nama, kelas, dan nilai. Bab-bab sebelumnya hanya memberi Anda variabel tunggal dan *array* yang isinya seragam. Bab ini memperkenalkan **record**, wadah yang menyatukan beberapa data berbeda jenis menjadi satu kesatuan yang bisa diberi nama, disalin, dan disusun menjadi tabel.

13.1 Mengapa Perlu Record?

Bayangkan Anda mencatat tiga buku. Tanpa record, Anda harus membuat variabel terpisah untuk setiap bagian data: `JudulBuku1`, `PenulisBuku1`, `TahunBuku1`, `HargaBuku1`, lalu hal yang sama untuk buku kedua dan ketiga. Dua belas variabel berserakan, dan tak ada yang menjamin keempat bagian milik buku yang sama tetap berpasangan. *Array* tidak membantu, karena semua elemen array harus bertipe sama, sedangkan judul adalah teks dan tahun adalah bilangan.

Record memecahkan masalah ini. Anggap record sebagai **formulir**: selembar kertas dengan kolom-kolom berlabel. Setiap kolom punya nama dan jenis isian sendiri, tetapi seluruh kolom itu tetap satu lembar. Pada Pascal, kolom itu disebut **field** (medan), dan lembar kosongnya disebut **tipe record**. Satu lembar yang sudah terisi adalah satu **variabel record**.



Gambar 13.1 Record sebagai formulir: tiap kolom berlabel adalah satu field, dan kode di kanan mendeklarasikan formulir yang sama.

13.2 Mendeklarasikan Record dan Memakai Field

Tipe record ditulis di bagian `type`, dengan kata tercadang `record` dan diakhiri `end` (tanpa `begin`). Di dalamnya, daftar field ditulis seperti deklarasi variabel: nama, titik dua, tipe.

BENTUK UMUM

```
type
  NamaTipe = record
    Field1: Tipe1;
    Field2, Field3: Tipe2;
  end;

var
  Variabel: NamaTipe;

Variabel.Field1 := nilai;      { notasi titik }
```

Untuk mengakses satu field, tulis nama variabel, sebuah **titik**, lalu nama field: `Buku1.Judul`. Field diperlakukan persis seperti variabel biasa bertipe field itu. Anda dapat menugaskan, membandingkan, mencetak, atau memakainya dalam ekspresi. Nama field hanya perlu unik *di dalam* recordnya, jadi dua record berbeda boleh sama-sama punya field bernama `Nama`.

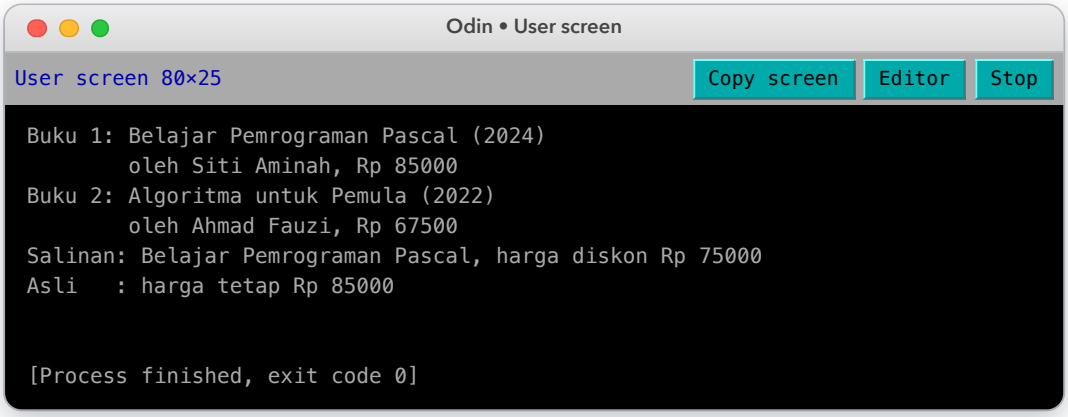
13.2.1 Program pertama: data buku

Buka `bab13_01_record_dasar.pas` lewat `File ▸ Examples...`. Program ini mendeklarasikan `TBuku`, mengisi dua buku dengan dua cara, mencetaknya, lalu menyalin sebuah record.

Program 13.1 `bab13_01_record_dasar.pas` *Record* File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 13.1 — Record
3    Record mengelompokkan beberapa data yang berbeda tipe menjadi satu
4    kesatuan, misalnya data sebuah buku. Setiap bagian disebut field
5    dan diakses dengan titik: Buku.Judul. }
6
7  program bab13_01_record_dasar;
8
9  type
10   TBuku = record
11     Judul: string[40];                { field bertipe string }
12     Penulis: string[30];
13     Tahun: Integer;                   { field bertipe Integer }
14     Harga: LongInt;
15   end;
16
17 var
18   Buku1, Buku2: TBuku;
19
20 begin
21   { Mengisi field satu per satu }
22   Buku1.Judul := 'Belajar Pemrograman Pascal';
23   Buku1.Penulis := 'Siti Aminah';
24   Buku1.Tahun := 2024;
25   Buku1.Harga := 85000;
26
27   { with: tidak perlu menulis "Buku2." berulang-ulang }
28   with Buku2 do
29   begin
30     Judul := 'Algoritma untuk Pemula';
```

```
31      Penulis := 'Ahmad Fauzi';
32      Tahun := 2022;
33      Harga := 67500;
34    end;
35
36    WriteLn('Buku 1: ', Buku1.Judul, ' (', Buku1.Tahun, ')');
37    WriteLn('        oleh ', Buku1.Penulis, ', Rp ', Buku1.Harga);
38    with Buku2 do
39    begin
40      WriteLn('Buku 2: ', Judul, ' (', Tahun, ')');
41      WriteLn('        oleh ', Penulis, ', Rp ', Harga);
42    end;
43
44    { Record bisa disalin utuh dengan satu penugasan }
45    Buku2 := Buku1;
46    Buku2.Harga := Buku1.Harga - 10000;    { ubah salinannya saja }
47    WriteLn('Salinan: ', Buku2.Judul, ', harga diskon Rp ', Buku2.Harga);
48    WriteLn('Asli   : harga tetap Rp ', Buku1.Harga);
49  end.
```



Gambar 13.2 Keluaran `bab13_01_record_dasar.pas` di User screen

Baris	Penjelasan
9–15	Deklarasi tipe `TBuku` dengan empat field. `Judul` dan `Penulis` bertipe `string` berbatas panjang (maksimal 40 dan 30 karakter), `Tahun` bertipe `Integer`, `Harga` bertipe `LongInt` agar muat harga di atas 32767. Tidak ada memori yang dipesan di sini; ini baru cetakan formulir.
17–18	Dua variabel `Buku1` dan `Buku2` bertipe `TBuku`. Sekarang ada dua formulir kosong di memori.
22–25	Mengisi `Buku1` field demi field dengan notasi titik. Urutan pengisian bebas.
28–34	Pernyataan `with Buku2 do`. Di dalam `begin..end`, nama `Judul`, `Penulis`, `Tahun`, dan `Harga` otomatis berarti field milik `Buku2`.
36–37	Mencetak `Buku1` dengan notasi titik lengkap.
38–42	Mencetak `Buku2` di dalam `with`, sehingga awalan `Buku2.` tidak perlu ditulis.
45	`Buku2 := Buku1` menyalin **seluruh** isi formulir dalam satu penugasan. Semua field `Buku2` kini sama dengan `Buku1`.
46	Hanya salinan yang diubah: harga `Buku2` dikurangi 10000, yaitu 85000 menjadi 75000.
47–48	Hasilnya terlihat pada keluaran: baris `Salinan` menampilkan 75000, sedangkan baris `Asli` tetap 85000. Setelah disalin, kedua record berdiri sendiri dan saling tidak memengaruhi.

13.2.2 Pernyataan with

Menulis `Buku2.` berulang-ulang melelahkan. Pernyataan `with` memberi tahu kompiler: "di dalam blok ini, nama field yang tidak dikenali berarti field milik record ini."

BENTUK UMUM

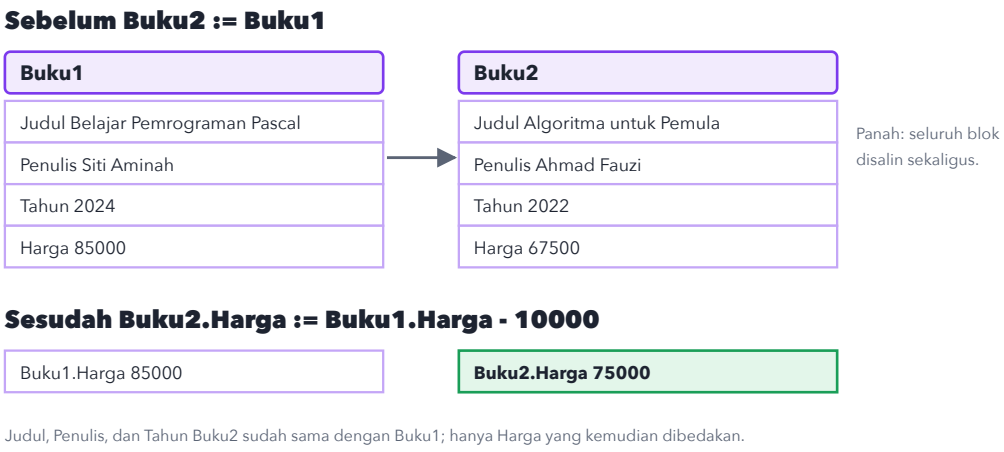
```
with Variabel do
begin
  Field1 := nilai;      { sama dengan Variabel.Field1 := nilai }
  Field2 := nilai;
end;
```

AWAS

Hati-hati dengan nama yang bentrok. Bila di luar `with` ada variabel bernama sama dengan sebuah field, nama itu di dalam `with` diartikan sebagai **field**, bukan variabel luar. Bila Anda perlu variabel luar itu, beri tahu dengan menulis nama lengkap record yang lain, atau jangan pakai `with`. Untuk blok yang panjang, notasi titik lengkap sering lebih jelas dibaca.

13.2.3 Isi memori dan penyalinan

Sebuah variabel record menempati satu blok memori yang berurutan; field-fieldnya tersusun di dalam blok itu sesuai urutan deklarasi. Itu sebabnya satu penugasan `Buku2 := Buku1` cukup untuk menyalin semuanya: kompiler menyalin satu blok utuh, bukan field satu per satu.



Gambar 13.3 Penyalinan record: satu penugasan menyalin semua field, lalu kedua record berdiri sendiri.

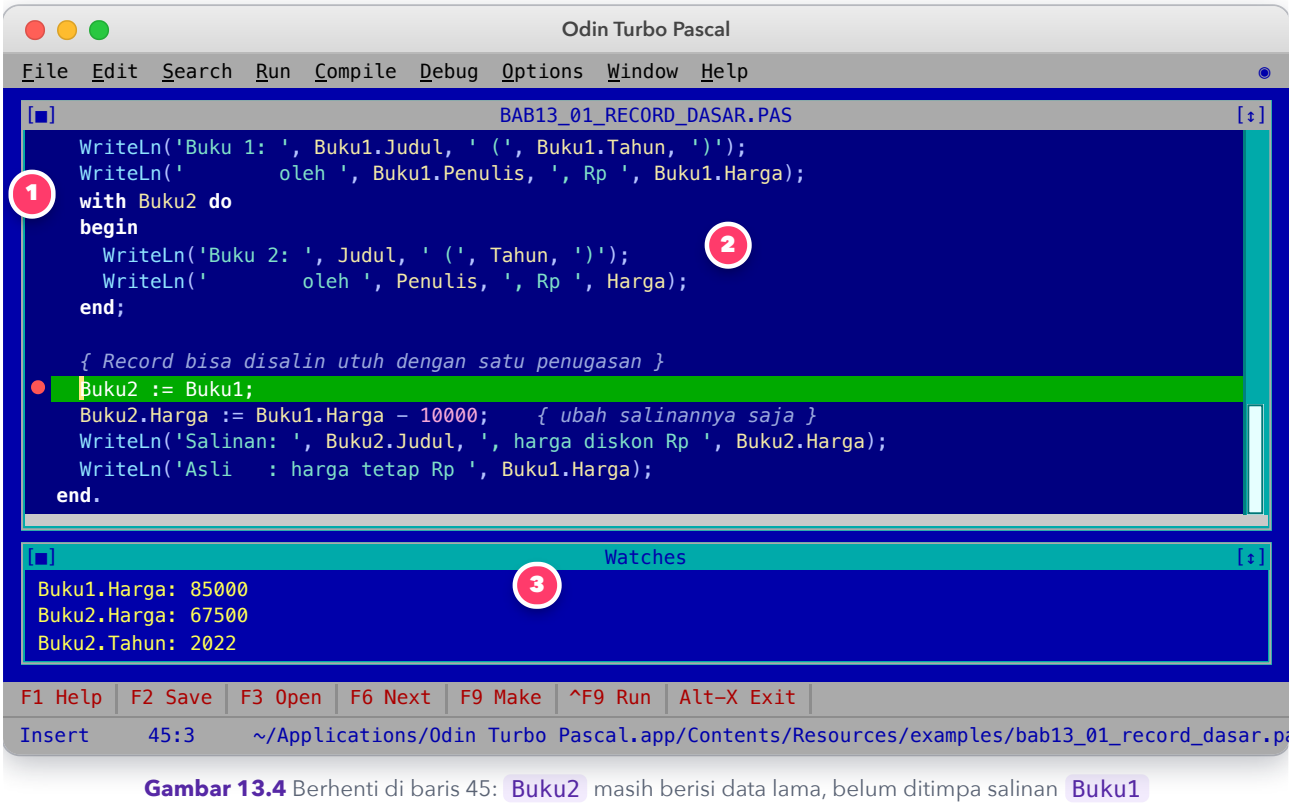
Dua variabel record bisa saling ditugaskan hanya bila **tipenya sama persis** (sama-sama `TBuku`). Record dengan isi serupa tetapi dideklarasikan dengan tipe berbeda tidak dapat disalin langsung.

ATURAN TURBO PASCAL 7

Aturan Turbo Pascal 7. Record tidak dapat dibandingkan dengan `=`, dan tidak dapat dicetak utuh dengan `Write`. Keduanya harus dikerjakan **per field**. Satu-satunya operasi yang berlaku untuk seluruh record adalah penugasan (`:=`) antara dua record bertipe sama.

13.2.4 Memantau record dengan debugger

Jendela *Watches* sangat berguna untuk record. Anda dapat menambahkan watch untuk tiap field (`Buku1.Harga`) lewat `Debug ▸ Add watch...`. Taruh titik henti pada baris 45, mulai debugging dengan `Debug ▸ Start debugging`, lalu perhatikan nilai watch ketika Anda melangkah dengan `Debug ▸ Step over`.



Gambar 13.4 Berhenti di baris 45: `Buku2` masih berisi data lama, belum ditimpa salinan `Buku1`

1. **Pasang titik henti.** Pindahkan kursor ke baris 45, lalu pilih `Debug ▸ Toggle breakpoint`. Titik merah (1) muncul di gutter, baris yang akan dijalankan disorot hijau (2).
2. **Tambah watch.** Pilih `Debug ▸ Add watch...` untuk `Buku1.Harga`, `Buku2.Harga`, dan `Buku2.Tahun`. Jendela Watches (3) menampilkan nilainya.
3. **Langkahi satu baris.** Pilih `Debug ▸ Step over` sekali. Nilai `Buku2.Harga` dan `Buku2.Tahun` berubah mengikuti `Buku1`, bukti bahwa semua field ikut tersalin.

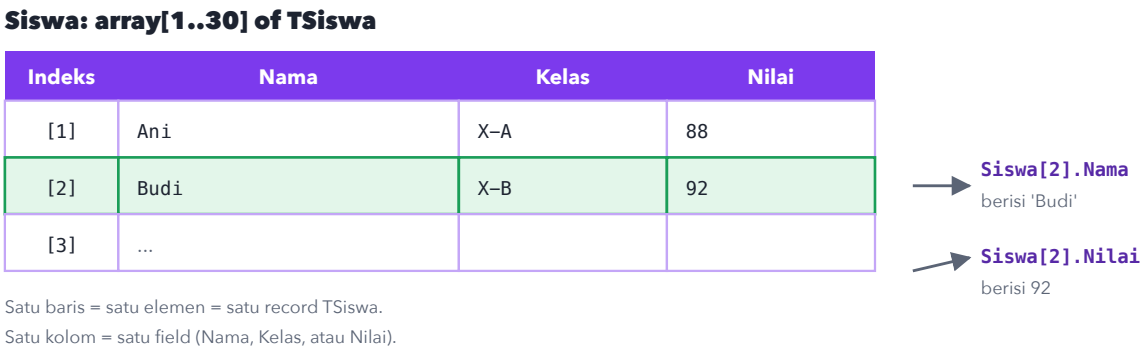
13.3 Array of Record: Record sebagai Tabel

Satu record cocok untuk satu buku, satu siswa, satu pegawai. Namun daftar nilai kelas berisi banyak siswa. Gabungkan dua gagasan yang sudah Anda kenal: buat **array yang setiap elemennya berupa record**. Hasilnya adalah tabel seperti lembar kerja: **satu elemen array = satu baris**, **satu field = satu kolom**.

BENTUK UMUM

```
var
  Tabel: array[1..Maks] of TRecord;

Tabel[I].Field := nilai;      { field milik elemen ke-I }
```



Gambar 13.5 Array of record sebagai tabel: baris dipilih dengan indeks, kolom dipilih dengan nama field. Baris yang disorot hijau adalah `Siswa[2]`.

13.3.1 Program kedua: daftar nilai siswa

Program `bab13_02_array_of_record.pas` membaca sejumlah siswa dari papan ketik, menyimpannya di tabel, mencetak tabel berkolom rapi, dan mencari nilai tertinggi. Contoh di bawah mengetikkan dua siswa.

Program 13.2 `bab13_02_array_of_record.pas` *Array of record* File ▸ Examples...

```
{ Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 13.2 - Array of record
  Array yang setiap elemennya berupa record, cocok untuk tabel data:
  satu elemen array = satu baris, satu field = satu kolom.
  Field elemen ke-I diakses dengan Siswa[I].Nama. }

program bab13_02_array_of_record;

const
  MaksSiswa = 30;

type
  TSiswa = record
    Nama: string[20];
    Kelas: string[5];
    Nilai: Integer;
  end;

var
  Siswa: array[1..MaksSiswa] of TSiswa;
  N, I, Terbaik: Integer;

begin
  repeat
    Write('Jumlah siswa (1-', MaksSiswa, '): ');
    ReadLn(N);
```

```
27    until (N >= 1) and (N <= MaksSiswa);
28
29    for I := 1 to N do
30    begin
31      WriteLn('Siswa ke-', I);
32      Write('  Nama  : '); ReadLn(Siswa[I].Nama);
33      Write('  Kelas : '); ReadLn(Siswa[I].Kelas);
34      Write('  Nilai : '); ReadLn(Siswa[I].Nilai);
35    end;
36
37    WriteLn;
38    WriteLn('No  Nama                 Kelas  Nilai');
39    WriteLn('---------------------------------------');
40    Terbaik := 1;
41    for I := 1 to N do
42    begin
43      { '':N menulis N spasi, agar teks rata kiri dalam kolomnya }
44      with Siswa[I] do                    { with juga berlaku untuk elemen }
45        WriteLn(I:2, '  ', Nama, '':22 - Length(Nama), Kelas,
46          '':7 - Length(Kelas), Nilai:5);
47      if Siswa[I].Nilai > Siswa[Terbaik].Nilai then
48        Terbaik := I;
49    end;
50    WriteLn('---------------------------------------');
51    WriteLn('Nilai tertinggi: ', Siswa[Terbaik].Nama,
52      ' (', Siswa[Terbaik].Nilai, ')');
53  end.
```

Odin • User screen

User screen 80×25 | Copy screen | Editor | Stop

```
Jumlah siswa (1–30): 2
Siswa ke-1
  Nama  : Ani
  Kelas : X-A
  Nilai : 88
Siswa ke-2
  Nama  : Budi
  Kelas : X-B
  Nilai : 92

No  Nama                 Kelas  Nilai
---------------------------------------
 1  Ani                  X-A       88
 2  Budi                 X-B       92
---------------------------------------
Nilai tertinggi: Budi (92)


[Process finished, exit code 0]
```

Gambar 13.6 Dua siswa dimasukkan: Ani (X-A, 88) dan Budi (X-B, 92)

Baris	Penjelasan
9–10	Konstanta `MaksSiswa` menetapkan kapasitas tabel, 30 baris. Mengubah angka ini cukup di satu tempat.

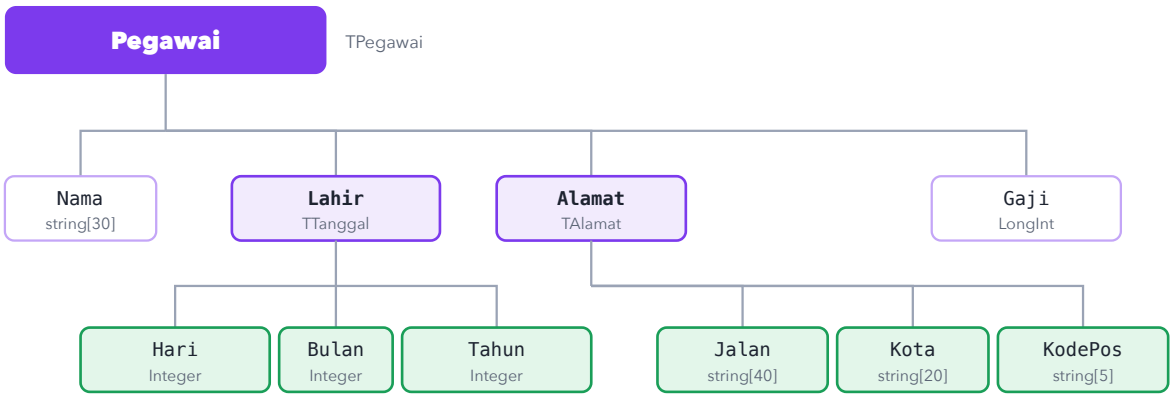
Baris	Penjelasan
12–17	Tipe `TSiswa` dengan tiga field: `Nama`, `Kelas`, `Nilai`.
20	`Siswa` adalah array dengan 30 elemen, masing-masing satu `TSiswa`. Itulah tabelnya.
24–27	Perulangan `repeat..until` meminta jumlah siswa sampai nilainya antara 1 dan 30, agar indeks tidak melewati batas array.
29–35	Untuk tiap baris `I` dari 1 sampai `N`, tiga `ReadLn` mengisi tiga field: `Siswa[I].Nama`, `Siswa[I].Kelas`, `Siswa[I].Nilai`.
37–39	Mencetak baris kosong, judul kolom, dan garis pemisah.
40	Menganggap siswa nomor 1 sebagai yang terbaik sementara.
43–46	Bentuk `'':N` menulis tepat *N* spasi. Dengan `'':22 - Length(Nama)`, kolom nama selalu lebar 22 karakter berapa pun panjang namanya, sehingga kolom `Kelas` sejajar. Pernyataan `with Siswa[I] do` memperpendek penulisan field elemen ke-`I`.
47–48	Bila nilai baris ini melebihi nilai calon terbaik, catat `I` sebagai yang terbaik. Ini pola "cari maksimum" yang sama dengan pencarian pada array biasa, tetapi membandingkan *field* `Nilai`.
51–52	Mencetak nama dan nilai milik baris terbaik. Pada contoh ini Budi (92) mengalahkan Ani (88).

COBA SENDIRI

Jalankan program dengan 3 siswa dan beri dua siswa nilai yang sama tertinggi. Siapa yang dipilih sebagai nilai tertinggi, siswa pertama atau yang kedua? Jelaskan dengan melihat operator pada baris 47.

13.4 Record Bersarang

Field sebuah record boleh bertipe record lain. Seorang pegawai punya tanggal lahir yang terdiri atas hari, bulan, tahun, dan alamat yang terdiri atas jalan, kota, kode pos. Daripada memipihkan semuanya menjadi satu daftar panjang, kita kelompokkan: `Lahir` adalah record `TTanggal`, `Alamat` adalah record `TAlamat`. Susunannya mirip formulir yang memuat lampiran berupa formulir lain.



Pegawai.Lahir.Tahun berarti: dari Pegawai ambil Lahir, dari Lahir ambil Tahun.
Pegawai.Alamat.Kota berarti: dari Pegawai ambil Alamat, dari Alamat ambil Kota.

Gambar 13.7 Pohon `TPegawai`: dua field (`Lahir`, `Alamat`) adalah record lagi. Daun hijau adalah data tunggal.

Untuk mencapai daun, ikuti pohon dari akar dengan **titik berantai**: `Pegawai.Lahir.Tahun`. Setiap titik berarti "masuk satu tingkat". Tipe yang dipakai sebagai field harus dideklarasikan **lebih dulu**, sebab Pascal membaca dari atas ke bawah.

13.4.1 Program ketiga: data pegawai

Program `bab13_03_record_bersarang.pas` mengisi sebuah `TPegawai` dan mencetak hasilnya, termasuk umur yang dihitung dari tahun lahir.

Program 13.3 `bab13_03_record_bersarang.pas` *Record bersarang* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 13.3 — Record bersarang
  Field sebuah record boleh berupa record lain. Pegawai punya tanggal
  lahir (record TTanggal) dan alamat (record TAlamat). Aksesnya
  memakai titik berantai: Pegawai.Lahir.Tahun. }

program bab13_03_record_bersarang;

type
  TTanggal = record
    Hari, Bulan, Tahun: Integer;
  end;

  TAlamat = record
    Jalan: string[40];
    Kota: string[20];
    KodePos: string[5];
  end;

  TPegawai = record
    Nama: string[30];
    Lahir: TTanggal;                    { record di dalam record }
    Alamat: TAlamat;
    Gaji: LongInt;
  end;

const
```

```
28    TahunIni = 2026;
29
30  var
31    Pegawai: TPegawai;
32
33  begin
34    Pegawai.Nama := 'Rina Maharani';
35    Pegawai.Lahir.Hari := 17;              { titik berantai }
36    Pegawai.Lahir.Bulan := 8;
37    Pegawai.Lahir.Tahun := 1995;
38
39    with Pegawai.Alamat do                 { with untuk record bagian dalam }
40    begin
41      Jalan := 'Jl. Pangeran Antasari No. 12';
42      Kota := 'Kandangan';
43      KodePos := '71211';
44    end;
45    Pegawai.Gaji := 4750000;
46
47    with Pegawai do
48    begin
49      WriteLn('Nama      : ', Nama);
50      WriteLn('Lahir     : ', Lahir.Hari, '-', Lahir.Bulan, '-', Lahir.Tahun);
51      WriteLn('Umur      : sekitar ', TahunIni - Lahir.Tahun, ' tahun');
52      WriteLn('Alamat    : ', Alamat.Jalan);
53      WriteLn('            ', Alamat.Kota, ' ', Alamat.KodePos);
54      WriteLn('Gaji      : Rp ', Gaji);
55    end;
56  end.
```

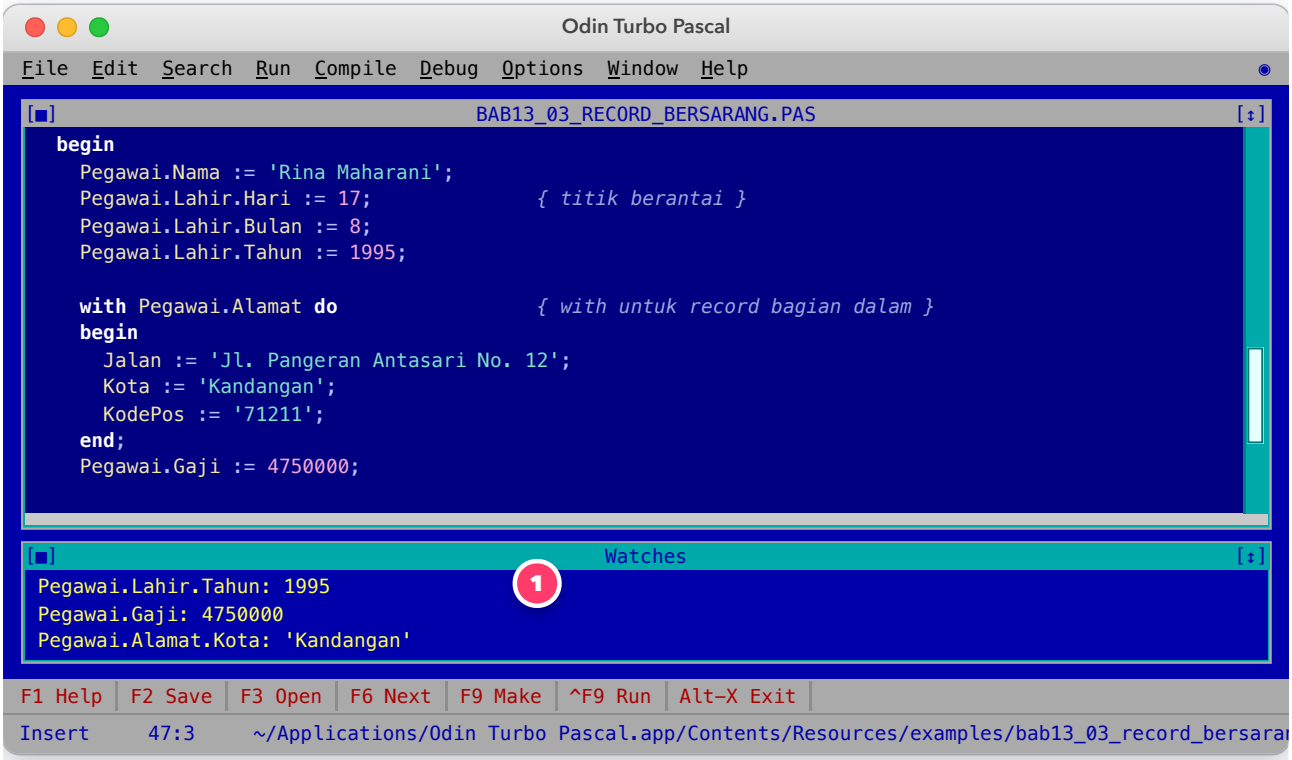


Gambar 13.8 Keluaran `bab13_03_record_bersarang.pas` di User screen

Baris	Penjelasan
9–12	`TTanggal` berisi tiga `Integer` sekaligus: `Hari, Bulan, Tahun`. Beberapa field bertipe sama boleh dideklarasikan dalam satu baris, dipisah koma.
14–18	`TAlamat` dengan tiga field string. `KodePos` bertipe string karena kode pos bukan bilangan yang dihitung.
20–25	`TPegawai` memakai kedua tipe di atas sebagai tipe field `Lahir` dan `Alamat`. Itulah sarangnya.
27–28	Konstanta `TahunIni = 2026` dipakai untuk menghitung umur.
34–37	Mengisi nama, lalu tiga field `Lahir` dengan titik berantai.

Baris	Penjelasan
39–44	`with Pegawai.Alamat do` membuka record bagian dalam saja. Di dalamnya cukup menulis `Jalan`, `Kota`, `KodePos`.
45	Mengisi `Gaji`, field paling luar.
47–55	`with Pegawai do` membuka record luar. Karena itu `Lahir.Hari` dan `Alamat.Kota` cukup ditulis dari tingkat kedua. Baris 51 menghitung `TahunIni - Lahir.Tahun`, yaitu 2026 dikurangi 1995, sekitar 31 tahun.

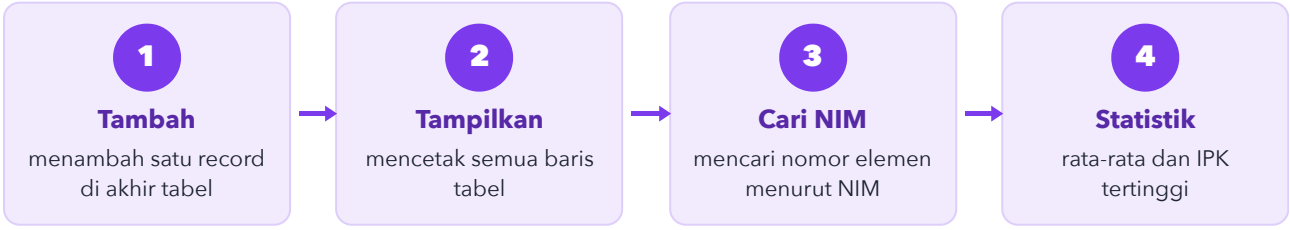
Debugger juga dapat menampilkan field bersarang. Tambahkan watch `Pegawai.Lahir.Tahun` dan `Pegawai.Gaji` lalu langkahi program.



Gambar 13.9 Watches untuk field bersarang: titik berantai dipakai juga pada `Debug ▸ Add watch...`

13.5 Menyatukan Semuanya: Data Mahasiswa

Program terakhir bab ini memakai array of record sebagai "basis data mini" dengan menu. Setiap tugas dipisah menjadi prosedur atau fungsi, sehingga program mudah dibaca. Susunan programnya demikian.



Variabel `Jumlah` mencatat berapa baris yang sudah terisi, sehingga perulangan cukup berjalan dari 1 sampai `Jumlah`, bukan sampai `MaksData`. Buka `bab13_04_data_mahasiswa.pas`. Program ini panjang, jadi penjelasannya dibagi per bagian.

Program 13.4 `bab13_04_data_mahasiswa.pas` *Latihan: data mahasiswa* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 13.4 — Latihan: data mahasiswa
  Menu sederhana untuk mengelola data mahasiswa di dalam array of
  record: tambah, tampilkan, cari menurut NIM, dan statistik IPK.
  Tiga data contoh sudah diisi saat program mulai. }

program bab13_04_data_mahasiswa;

const
  MaksData = 50;

type
  TMahasiswa = record
    NIM: string[10];
    Nama: string[25];
    IPK: Real;
  end;

var
  Data: array[1..MaksData] of TMahasiswa;
  Jumlah: Integer;                      { banyak data yang sudah terisi }

procedure Tambah(NIM, Nama: string; IPK: Real);
begin
  if Jumlah >= MaksData then
    WriteLn('Data penuh.')
  else
  begin
    Jumlah := Jumlah + 1;
    Data[Jumlah].NIM := NIM;
    Data[Jumlah].Nama := Nama;
    Data[Jumlah].IPK := IPK;
  end;
end;

procedure TampilkanSatu(I: Integer);
begin
  with Data[I] do
    WriteLn(NIM, '':12 - Length(NIM), Nama, '':27 - Length(Nama), IPK:4:2);
end;

procedure TampilkanSemua;
var
  I: Integer;
begin
  WriteLn('NIM         Nama                          IPK');
  for I := 1 to Jumlah do
    TampilkanSatu(I);
  WriteLn('(', Jumlah, ' mahasiswa)');
end;

{ Mengembalikan nomor elemen, atau 0 bila NIM tidak ditemukan }
```

```
function CariNIM(NIM: string): Integer;
var
  I: Integer;
begin
  CariNIM := 0;
  for I := 1 to Jumlah do
    if Data[I].NIM = NIM then
      CariNIM := I;
end;

procedure Statistik;
var
  I, Terbaik: Integer;
  Total: Real;
begin
  if Jumlah = 0 then
  begin
    WriteLn('Belum ada data.');
    Exit;
  end;
  Total := 0;
  Terbaik := 1;
  for I := 1 to Jumlah do
  begin
    Total := Total + Data[I].IPK;
    if Data[I].IPK > Data[Terbaik].IPK then
      Terbaik := I;
  end;
  WriteLn('Rata-rata IPK : ', Total / Jumlah:0:2);
  WriteLn('IPK tertinggi : ', Data[Terbaik].Nama, ' (',
    Data[Terbaik].IPK:0:2, ')');
end;

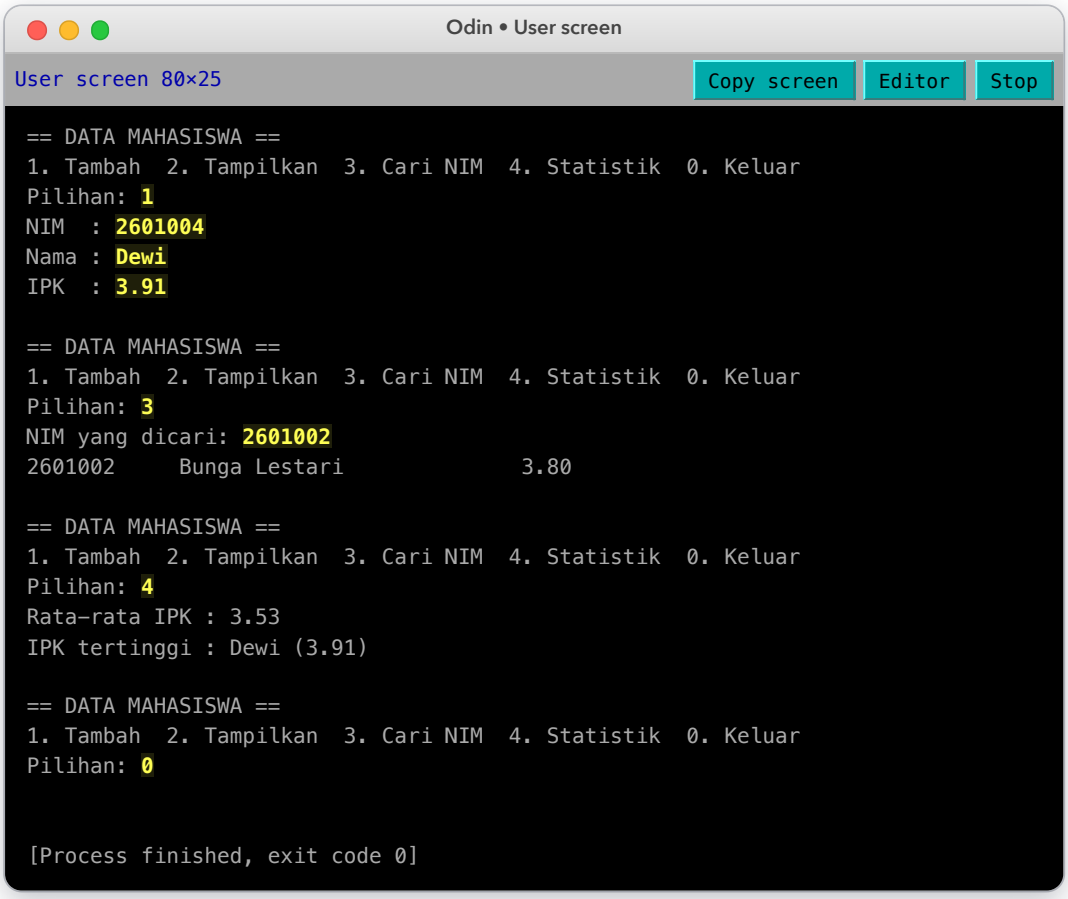
var
  Pilihan, Posisi: Integer;
  NIM, Nama: string;
  IPK: Real;

begin
  Jumlah := 0;
  Tambah('2601001', 'Andi Pratama', 3.45);     { data contoh }
  Tambah('2601002', 'Bunga Lestari', 3.80);
  Tambah('2601003', 'Candra Wijaya', 2.95);

  repeat
    WriteLn;
    WriteLn('== DATA MAHASISWA ==');
    WriteLn('1. Tambah  2. Tampilkan  3. Cari NIM  4. Statistik  0. Keluar');
    Write('Pilihan: ');
    ReadLn(Pilihan);
    case Pilihan of
      1: begin
           Write('NIM  : '); ReadLn(NIM);
           if CariNIM(NIM) > 0 then
             WriteLn('NIM sudah terdaftar.')
           else
           begin
             Write('Nama : '); ReadLn(Nama);
             Write('IPK  : '); ReadLn(IPK);
```

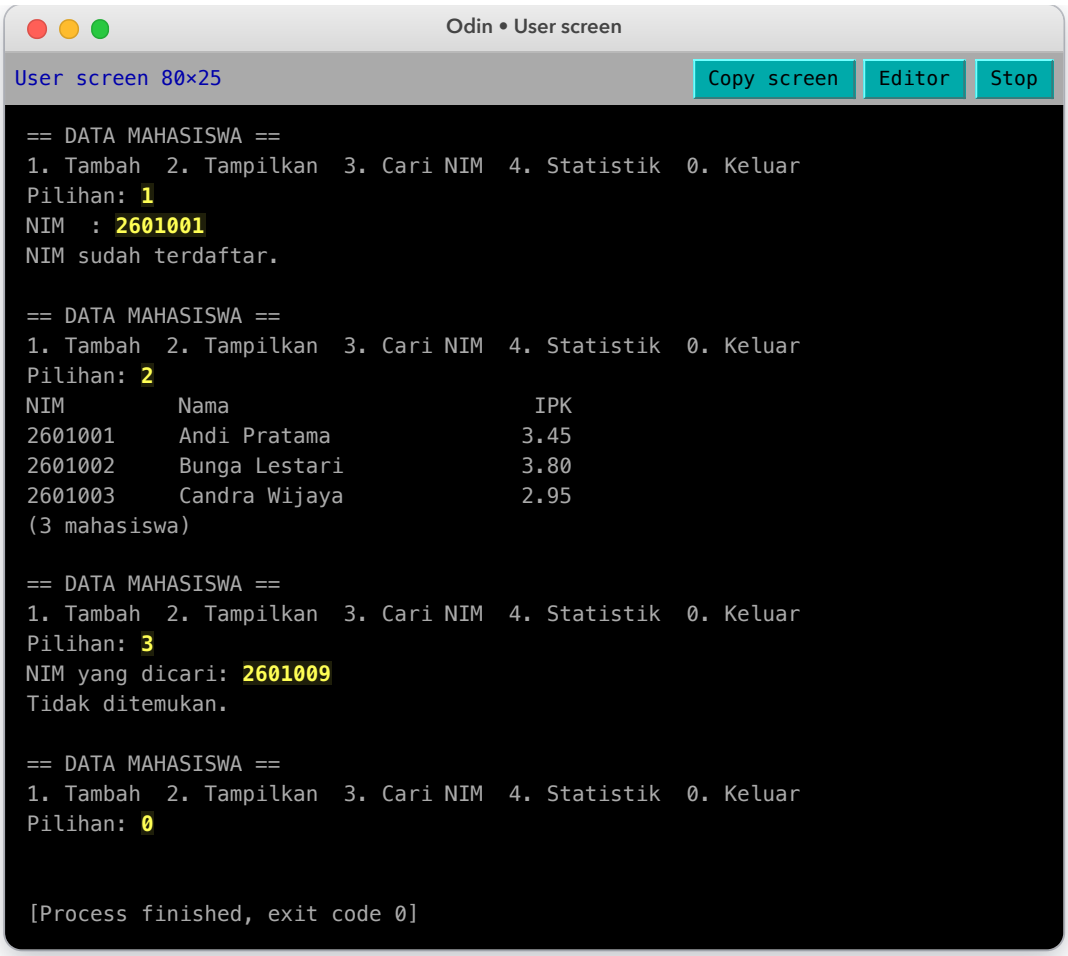
```
112               Tambah(NIM, Nama, IPK);
113             end;
114           end;
115         2: TampilkanSemua;
116         3: begin
117              Write('NIM yang dicari: '); ReadLn(NIM);
118              Posisi := CariNIM(NIM);
119              if Posisi = 0 then
120                WriteLn('Tidak ditemukan.')
121              else
122                TampilkanSatu(Posisi);
123           end;
124         4: Statistik;
125       end;
126     until Pilihan = 0;
127   end.
```

Masukan bawaan menambah mahasiswa baru bernama Dewi (IPK 3.91), mencari NIM 2601002, menampilkan statistik, lalu keluar. Menu tampil ulang setelah setiap perintah sampai Anda memilih 0.



Gambar 13.10 Menambah Dewi, mencari NIM 2601002, lalu statistik

Satu jalannya lagi memperlihatkan pemeriksaan NIM ganda dan tampilan seluruh tabel:



Gambar 13.11 NIM ganda ditolak, lalu seluruh tabel ditampilkan, dan pencarian NIM yang tidak ada

Baris	Penjelasan
9–17	Kapasitas 50 baris dan tipe `TMahasiswa` dengan `NIM`, `Nama`, dan `IPK` bertipe `Real`.
20–21	Tabel `Data` sebagai variabel global, dan `Jumlah` sebagai penunjuk baris terakhir yang terisi.
23–34	`Tambah` menolak penambahan bila tabel penuh (baris 25). Bila tidak, `Jumlah` dinaikkan lebih dulu, lalu tiga field baris baru diisi dari parameter.
36–40	`TampilkanSatu` mencetak satu baris. Dengan `with Data[I] do` nama field langsung dipakai, dan `IPK:4:2` memformat `Real` dengan dua desimal sesuai aturan TP7.
42–50	`TampilkanSemua` mencetak judul kolom, memanggil `TampilkanSatu` untuk tiap baris, lalu menuliskan banyak mahasiswa.
52–61	`CariNIM` adalah fungsi pencari. Nilai awal 0 berarti “tidak ketemu” (baris 57). Bila `Data[I].NIM` sama dengan NIM yang dicari, hasil diganti `I`. Karena NIM unik, hanya satu baris yang cocok.
63–84	`Statistik` menjumlahkan IPK dan mencari IPK tertinggi sekaligus dalam satu perulangan. Bagian awal (68–72) keluar lebih cepat dengan `Exit` bila tabel kosong, supaya tidak membagi dengan nol.
81–83	Rata-rata dicetak dengan `Total / Jumlah:0:2`; nama dan IPK tertinggi dengan `:0:2`.
92–95	Program mulai dengan `Jumlah := 0` lalu menambah tiga data contoh lewat `Tambah`.
97–126	Menu berulang sampai pilihan 0. Pilihan 1 (baris 104–114) memeriksa dulu dengan `CariNIM` bahwa NIM belum ada. Pilihan 3 (116–123) memanggil `CariNIM` lalu `TampilkanSatu` bila hasilnya bukan 0.

> **INFO**
>
> **Mengapa dipisah menjadi subprogram?** Karena `Data` dan `Jumlah` adalah variabel global, setiap prosedur langsung dapat memakainya. Untuk program kecil seperti ini cara itu sederhana. Pada program besar, lebih baik data dilewatkan lewat parameter agar tiap prosedur jelas apa yang ia ubah.

13.6 Kesalahan yang Sering Terjadi

13.6.1 Menulis field yang tidak ada

Salah ketik nama field adalah kekeliruan paling umum. Kompiler langsung menolaknya.

salah1.pas

```
1  program Salah1;
2  type
3    TBuku = record
4      Judul: string[40];
5      Penulis: string[30];
6    end;
7  var
8    B: TBuku;
9  begin
10   B.Judul := 'Pascal';
11   B.Pengarang := 'Budi';
12 end.
```

PESAN KOMPILER

```
salah1.pas(11,5) Error: Identifier tidak dikenal: field Pengarang
```

Gambar 13.12 Field bernama `Pengarang` tidak ada pada `TBuku`

Perbaikannya: gunakan nama field yang dideklarasikan, di sini `Penulis`. Bila ragu, lihat kembali deklarasi tipe record.

13.6.2 Mencetak record utuh

salah2.pas

```
1  program Salah2;
2  type
3    TTitik = record
4      X, Y: Integer;
5    end;
6  var
7    P: TTitik;
8  begin
9    P.X := 3;
10   P.Y := 4;
11   WriteLn(P);
12 end.
```

PESAN KOMPILER

```
salah2.pas(11,11) Error: Variabel bertipe ini tidak bisa dibaca atau ditulis
```

Gambar 13.13 `WriteLn` tidak bisa mencetak record

Cetak field satu per satu: `WriteLn(P.X, ' ', P.Y)`.

13.6.3 Membandingkan dua record dengan =

salah3.pas

```
1  program Salah3;
2  type
3    TTitik = record
4      X, Y: Integer;
5    end;
6  var
7    A, B: TTitik;
8  begin
9    A.X := 1; A.Y := 2;
10   B := A;
11   if A = B then
12     WriteLn('sama');
13 end.
```

PESAN KOMPILER

```
salah3.pas(11,6) Error: Tipe operand tidak cocok dengan operator
```

Gambar 13.14 Operator `=` tidak berlaku untuk record

Bandingkan per field: `(A.X = B.X) and (A.Y = B.Y)`.

13.6.4 Lupa indeks pada array of record

Pada tabel, menulis `Siswa.Nama` tanpa indeks tidak bermakna, karena `Siswa` adalah array, bukan satu record. Tulis `Siswa[I].Nama`. Dan jangan memakai indeks di luar `1..MaksSiswa`; indeks di luar batas merusak data lain atau menghentikan program saat pemeriksaan batas aktif.

13.7 Rangkuman

- Record mengelompokkan beberapa data **berbeda tipe** dalam satu kesatuan; array mengelompokkan banyak data **sama tipe**.
- Tipe record dideklarasikan dengan `record ... end` di bagian `type`; variabelnya dideklarasikan di `var`.
- Field diakses dengan notasi titik: `Variabel.Field`; bentuk `with Variabel do` menghemat penulisan.
- Dua record bertipe sama dapat disalin dengan satu penugasan; perbandingan dan pencetakan harus per field.
- Array of record membentuk tabel: indeks memilih baris, nama field memilih kolom (`Tabel[I].Field`).

- Record dapat bersarang; akses memakai titik berantai (`Pegawai.Lahir.Tahun`), dan tipe bagian dalam dideklarasikan lebih dulu.
- Variabel penghitung seperti `Jumlah` memisahkan kapasitas tabel dari banyak data yang benar-benar terisi.
- Jendela Watches memantau field record, termasuk field bersarang, selama debugging.

13.8 Latihan

1. **DASAR** Buat record `TSiswa` dengan field `Nama`, `Umur`, dan `Kelas`. Isi satu variabel lalu cetak isinya dalam satu kalimat.
2. **DASAR** Ubah Program 13.1 agar menyimpan satu buku lagi (`Buku3`) dan mencetaknya dengan `with`.
3. **MENENGAH** Tambahkan field `Kelas` bertipe `string[5]` ke Program 13.2. Setelah tabel dicetak, hitung dan cetak rata-rata nilai seluruh siswa.
4. **MENENGAH** Buat record `TTitik` (X, Y bertipe `Real`) dan record `TGaris` yang berisi dua titik, `Awal` dan `Akhir`. Hitung panjang garis dengan rumus jarak.
5. **MENENGAH** Pada Program 13.4, tambahkan pilihan 5 untuk menghapus mahasiswa menurut NIM. Petunjuk: geser semua baris sesudahnya satu tempat ke depan, lalu kurangi `Jumlah`.
6. **TANTANGAN** Urutkan tabel mahasiswa Program 13.4 menurut IPK dari tertinggi ke terendah dengan metode gelembung. Petunjuk: Anda dapat menukar **seluruh record** dengan variabel bantu bertipe `TMahasiswa`.
7. **TANTANGAN** Buat buku harian: record `TCatatan` berisi `Tanggal` (`TTanggal` bersarang) dan `Isi`. Simpan sampai 20 catatan, lalu cetak catatan yang tanggalnya sama dengan tanggal yang diketik pengguna.

BAGIAN IV · STRUKTUR DATA

BAB 14

Enumerasi, Subrange, dan Set

TUJUAN PEMBELAJARAN

- Mendefinisikan tipe enumerasi dan memakainya pada `for`, `case`, dan perbandingan.
- Memakai `Ord`, `Succ`, `Pred`, `Low`, dan `High` pada tipe ordinal.
- Mendefinisikan tipe subrange, dan memeriksa masukan agar tidak melampaui rentangnya.
- Membentuk himpunan (`set`) dan memakai operator `+`, `*`, `-`, `in`, `=`, `<=`.
- Memakai `Include` dan `Exclude`, serta `set of Char` untuk memeriksa jenis karakter.
- Menerapkan aturan Turbo Pascal 7: `Write` tidak dapat mencetak nilai enumerasi.

PROGRAM DALAM BAB INI

14.1 `bab14_01_tipe_enumerasi.pas`
Tipe enumerasi

14.2 `bab14_02_tipe_subrange.pas`
Tipe subrange

14.3 `bab14_03_tipe_set.pas`
Tipe set (himpunan)

14.4 `bab14_04_validasi_huruf_set.pas`
Latihan: validasi dengan set

Hari dalam seminggu, bulan, jenis huruf, nilai ujian: semua ini sebenarnya dapat ditulis sebagai bilangan bulat, tetapi program menjadi sulit dibaca dan mudah salah. Pascal menyediakan tiga alat untuk membuat tipe data yang sesuai dengan masalahnya: **enumerasi** (daftar nama buatan sendiri), **subrange** (potongan rentang), dan **set** (himpunan). Ketiganya berbagi satu kesamaan: semuanya berdasar pada **tipe ordinal**.

14.1 Tipe Ordinal: Nilai yang Berurutan

Tipe **ordinal** adalah tipe yang nilainya dapat diurutkan dan dihitung satu per satu, seperti anak tangga: setiap nilai (kecuali yang pertama) punya nilai sebelumnya, dan setiap nilai (kecuali yang terakhir) punya nilai sesudahnya. `Integer`, `Char`, `Boolean`, enumerasi, dan subrange semuanya ordinal. `Real` bukan ordinal, sebab di antara dua bilangan real selalu ada bilangan real lain.

Tabel 14.1 Fungsi bawaan untuk tipe ordinal

Fungsi	Arti	Contoh
`Ord(x)`	nomor urut nilai `x` (mulai dari 0 untuk enumerasi)	`Ord(Rabu)` bernilai 2
`Succ(x)`	nilai sesudah `x`	`Succ(Rabu)` bernilai `Kamis`
`Pred(x)`	nilai sebelum `x`	`Pred(Rabu)` bernilai `Selasa`
`Low(T)`	nilai terkecil tipe `T`	`Low(THari)` bernilai `Senin`
`High(T)`	nilai terbesar tipe `T`	`High(THari)` bernilai `Minggu`

14.2 Tipe Enumerasi

Dengan enumerasi, Anda **mendaftar sendiri** semua nilai yang mungkin, lengkap dengan nama. Daripada menulis “hari ke-3” sebagai angka 3, Anda menulis `Rabu`. Program menjadi dokumentasi dirinya sendiri, dan kompiler menolak nilai di luar daftar.

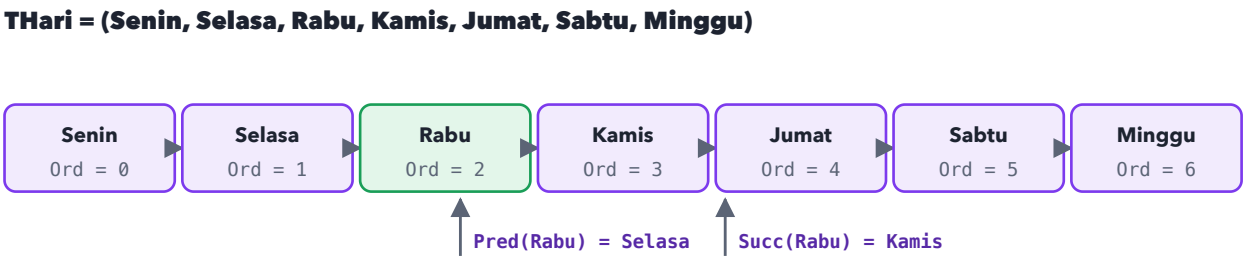
BENTUK UMUM

```
type
  NamaTipe = (Nilai1, Nilai2, Nilai3);

var
  V: NamaTipe;

V := Nilai2;
```

Kompiler memberi nomor urut 0, 1, 2, ... sesuai urutan penulisan. Nama-nama itu adalah **konstanta** yang tidak berubah, bukan string, sehingga ditulis tanpa tanda kutip.



Gambar 14.1 Enumerasi sebagai deretan bernomor: `Ord` memberi nomor urut, `Succ` dan `Pred` bergeser satu langkah.

7 **ATURAN TURBO PASCAL 7**

Aturan Turbo Pascal 7. `Write` dan `WriteLn` **tidak dapat mencetak nilai enumerasi.** `WriteLn(Hari)` ditolak kompiler. Dua cara umum: cetak nomor urut dengan `Ord(Hari)`, atau simpan nama dalam **array berindeks enumerasi** lalu cetak `NamaHari[Hari]`. Cara kedua dipakai di program berikut. Demikian pula `ReadLn` tidak dapat membaca enumerasi.

14.2.1 Program pertama: nama hari

Buka `bab14_01_tipe_enumerasi.pas`. Program ini mendefinisikan `THari`, memakainya sebagai pencacah `for` dan pemilih `case`, lalu mencoba `Succ`, `Pred`, dan `Ord`.

Program 14.1 `bab14_01_tipe_enumerasi.pas` *Tipe enumerasi* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 14.1 — Tipe enumerasi
  Enumerasi adalah tipe yang nilainya kita daftar sendiri, misalnya
  nama hari. Program jadi lebih mudah dibaca daripada memakai angka
  1..7. Urutannya bernomor 0, 1, 2, ... dan bisa dipakai di for/case. }

program bab14_01_tipe_enumerasi;

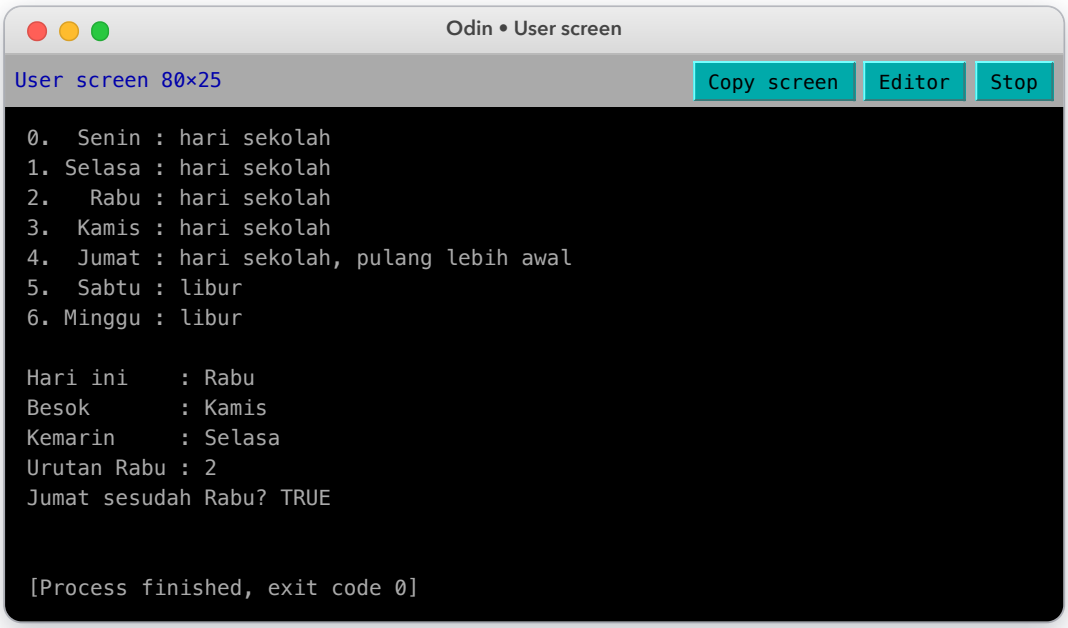
type
  THari = (Senin, Selasa, Rabu, Kamis, Jumat, Sabtu, Minggu);

const
  { Turbo Pascal tidak bisa menulis nilai enumerasi langsung dengan
    WriteLn, jadi namanya disimpan dalam array berindeks THari }
  NamaHari: array[THari] of string[6] =
    ('Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat', 'Sabtu', 'Minggu');

var
  Hari: THari;

begin
  for Hari := Senin to Minggu do        { enumerasi sebagai pencacah }
  begin
    Write(Ord(Hari), '. ', NamaHari[Hari]:6, ' : ');
    case Hari of
```

```
26        Senin..Kamis: WriteLn('hari sekolah');
27        Jumat:        WriteLn('hari sekolah, pulang lebih awal');
28        Sabtu, Minggu: WriteLn('libur');
29      end;
30    end;
31    WriteLn;
32
33    Hari := Rabu;
34    WriteLn('Hari ini    : ', NamaHari[Hari]);
35    WriteLn('Besok       : ', NamaHari[Succ(Hari)]);   { nilai sesudahnya }
36    WriteLn('Kemarin     : ', NamaHari[Pred(Hari)]);   { nilai sebelumnya }
37    WriteLn('Urutan Rabu : ', Ord(Rabu));               { dihitung dari 0 }
38    WriteLn('Jumat sesudah Rabu? ', Jumat > Rabu);      { bisa dibandingkan }
39    { Succ(Minggu) tidak ada: range checking menghentikan program }
40  end.
```



Gambar 14.2 Keluaran `bab14_01_tipe_enumerasi.pas` di User screen

Baris	Penjelasan
9–10	Mendefinisikan `THari` dengan tujuh nilai. `Senin` bernomor 0 dan `Minggu` bernomor 6.
12–16	Konstanta bertipe array dengan indeks `THari`: elemen pertama untuk `Senin`, dan seterusnya. Inilah cara mencetak nama karena `Write` tidak bisa mencetak enumerasi. Ada tujuh nilai di daftar, sesuai tujuh elemen indeks.
19	`Hari` adalah variabel bertipe `THari`; hanya tujuh nilai itu yang boleh ia simpan.
22	`for Hari := Senin to Minggu`: tipe ordinal boleh menjadi pencacah. Hari bergerak dari `Senin` sampai `Minggu`.
24	Mencetak nomor urut dengan `Ord(Hari)` dan namanya dengan `NamaHari[Hari]:6` (lebar 6 agar sejajar).
25–29	`case Hari of` memilih keterangan. `Senin..Kamis` adalah rentang nilai enumerasi, `Sabtu, Minggu` adalah daftar nilai.
33	`Hari := Rabu` menetapkan satu nilai.

Baris	Penjelasan
35–36	`Succ(Hari)` menghasilkan `Kamis`, `Pred(Hari)` menghasilkan `Selasa`. Hasilnya dipakai sebagai indeks array nama.
37	`Ord(Rabu)` bernilai 2 karena penomoran mulai dari 0.
38	Nilai enumerasi dapat dibandingkan menurut urutan daftar: `Jumat > Rabu` bernilai benar. Hasil Boolean ini dicetak langsung, sebab Boolean boleh ditulis.
39	Komentar peringatan: `Succ(Minggu)` tidak mempunyai nilai. Dengan pemeriksaan jangkauan aktif, program berhenti dengan galat.

COBA SENDIRI

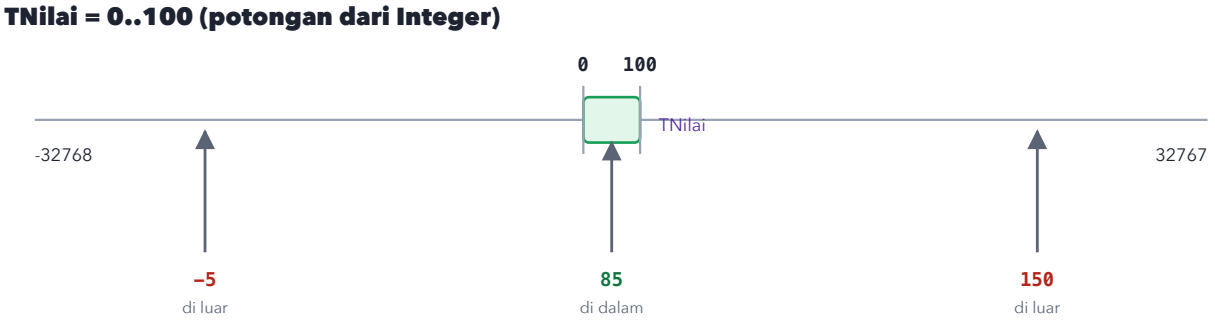
Tambahkan nilai baru di tengah daftar, misalnya `Rabu, Libur, Kamis` tanpa mengubah array `NamaHari`. Apa yang dikatakan kompiler? Apa hubungannya dengan jumlah elemen array dan nomor urut?

14.3 Tipe Subrange

Nilai ujian tidak mungkin negatif atau lebih dari 100, dan nomor bulan hanya 1 sampai 12. **Subrange** menyatakan batas itu di dalam tipenya: sebuah potongan rentang dari tipe ordinal lain yang disebut **tipe dasar** (*host type*). Variabelnya diperlakukan seperti tipe dasarnya, hanya saja hanya boleh berisi nilai di dalam rentang.

BENTUK UMUM

```
type
  NamaTipe = Batas bawah .. Batas atas;
  TNilai = 0..100;
  TAwal  = 'A'..'Z';
  THariKerja = Senin..Jumat;      { dari enumerasi }
```



Gambar 14.3 Garis bilangan subrange: nilai 85 sah untuk `TNilai`, sedangkan -5 dan 150 tidak.

Subrange punya dua manfaat. Pertama, **dokumentasi**: pembaca langsung tahu bahwa `TBulan` berarti 1 sampai 12. Kedua, **pengaman**: dengan pemeriksaan jangkauan (*range checking*) aktif, memberi nilai di luar rentang menghentikan program dengan *runtime error* 201, daripada diam-diam menyimpan data yang salah. Subrange juga dapat menjadi indeks array, seperti `array[TBulan] of Integer`.

> **INFO**
>
> **Periksa dulu, baru simpan.** Pemeriksaan jangkauan adalah jaring pengaman, bukan cara menerima masukan pengguna. Masukan dari papan ketik harus diperiksa dengan `if` sebelum disimpan ke variabel subrange, agar pengguna mendapat pesan yang ramah, bukan program yang berhenti mendadak.

14.3.1 Program kedua: bulan, nilai, dan inisial

Program `bab14_02_tipe_subrange.pas` membaca tiga masukan lalu menyimpannya ke variabel subrange hanya setelah lolos pemeriksaan. Masukan bawaan di bawah sengaja memuat nilai yang salah: 150 dan karakter ‘5’.

Program 14.2 `bab14_02_tipe_subrange.pas` *Tipe subrange* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 14.2 — Tipe subrange
  Subrange adalah potongan rentang tipe lain, misalnya 0..100 atau
  'A'..'Z'. Variabelnya hanya boleh berisi nilai dalam rentang itu.
  Dengan range checking aktif, nilai di luar rentang menghentikan
  program (runtime error 201), jadi periksa masukan sebelum disimpan. }

program bab14_02_tipe_subrange;

type
  TNilai = 0..100;                    { potongan dari Integer }
  TBulan = 1..12;
  THurufBesar = 'A'..'Z';             { potongan dari Char }
  THari = (Senin, Selasa, Rabu, Kamis, Jumat, Sabtu, Minggu);
  THariKerja = Senin..Jumat;          { potongan dari enumerasi }

const
  JumlahHari: array[TBulan] of Integer =
    (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31);

var
  Nilai: TNilai;
  Bulan: TBulan;
  Inisial: THurufBesar;
  Masukan: Integer;
  Huruf: Char;

begin
  Write('Nomor bulan (1–12): ');
  ReadLn(Masukan);
  if (Masukan >= 1) and (Masukan <= 12) then
  begin
    Bulan := Masukan;                 { aman: sudah diperiksa }
    WriteLn('Bulan ', Bulan, ' punya ', JumlahHari[Bulan],
      ' hari (tahun biasa).');
```

```
  end
  else
    WriteLn('Bulan tidak sah.');

  Write('Nilai ujian (0–100): ');
  ReadLn(Masukan);
  if (Masukan >= Low(TNilai)) and (Masukan <= High(TNilai)) then
  begin
    Nilai := Masukan;
    WriteLn('Nilai ', Nilai, ' disimpan.');
  end
  else
    WriteLn('Nilai harus ', Low(TNilai), ' sampai ', High(TNilai), '.');

  Write('Huruf awal nama: ');
  ReadLn(Huruf);
  Huruf := UpCase(Huruf);
  if (Huruf >= 'A') and (Huruf <= 'Z') then
  begin
    Inisial := Huruf;
    WriteLn('Inisial: ', Inisial);
  end
  else
    WriteLn('Inisial harus berupa huruf.');

  { Low dan High juga berlaku untuk subrange dari enumerasi }
  WriteLn('Hari kerja: ', Ord(High(THariKerja)) - Ord(Low(THariKerja)) + 1,
    ' hari');
end.
```



Gambar 14.4 Bulan 2 sah; nilai 150 dan inisial '5' ditolak oleh pemeriksaan `if`

Sekarang masukan yang sah semuanya: bulan 12, nilai 85, dan huruf kecil 'b' yang diubah oleh `UpCase`.



Gambar 14.5 Semua masukan sah

Baris	Penjelasan
10–15	Lima tipe: `TNilai` (0..100) dan `TBulan` (1..12) dari `Integer`, `THurufBesar` ('A'..'Z') dari `Char`, enumerasi `THari`, dan `THariKerja` (`Senin..Jumat`) sebagai subrange dari enumerasi.
17–19	Konstanta array dengan indeks `TBulan`: tepat 12 elemen, satu per bulan. Februari (elemen ke-2) berisi 28.
21–26	Variabel subrange `Nilai`, `Bulan`, `Inisial`, serta `Masukan` (`Integer` biasa) dan `Huruf` (`Char` biasa) sebagai penampung sementara masukan mentah.
29–38	Masukan dibaca ke `Integer` biasa dulu. Bila di antara 1 dan 12 (baris 31), barulah disalin ke `Bulan` (baris 33) dan dipakai sebagai indeks `JumlahHari[Bulan]`. Bila tidak, pesan galat ramah (baris 38).
40–48	Batas diperiksa dengan `Low(TNilai)` dan `High(TNilai)`, bukan angka 0 dan 100 yang ditulis ulang. Bila batas tipe diubah, pemeriksaan ikut menyesuaikan.
50–59	`UpCase` mengubah huruf kecil menjadi besar (baris 52); setelah itu `Huruf` dibandingkan dengan 'A' dan 'Z'. Huruf yang lolos disalin ke `Inisial`.
61–63	`Low` dan `High` juga bekerja pada `THariKerja`. Selisih nomor urutnya ditambah 1 memberi banyaknya nilai dalam rentang, yaitu jumlah hari kerja.

14.4 Tipe Set (Himpunan)

Himpunan adalah kumpulan nilai **tanpa urutan** dan **tanpa kembar**. Pikirkan daftar hari latihan seorang atlet: yang penting adalah hari mana saja yang termasuk, bukan urutan menuliskannya. Pascal mengungkapkan ini dengan `set of` disertai tipe dasar ordinal yang kecil.

BENTUK UMUM

```
type
  THimpunan = set of TipeDasar;

var
  H: THimpunan;

H := [nilai1, nilai2];      { himpunan berisi dua anggota }
H := [];                     { himpunan kosong }
H := [awal..akhir];        { rentang anggota }
```

Tipe dasar set harus ordinal dan berjumlah paling banyak 256 nilai (nomor urut 0 sampai 255). Karena itu `set of THari`, `set of Char`, dan `set of 0..100` sah, tetapi `set of Integer` tidak. Di memori, himpunan disimpan sebagai deretan bit, satu bit per kemungkinan anggota.

Satu bit per anggota yang mungkin; 1 berarti hari itu termasuk dalam himpunan.

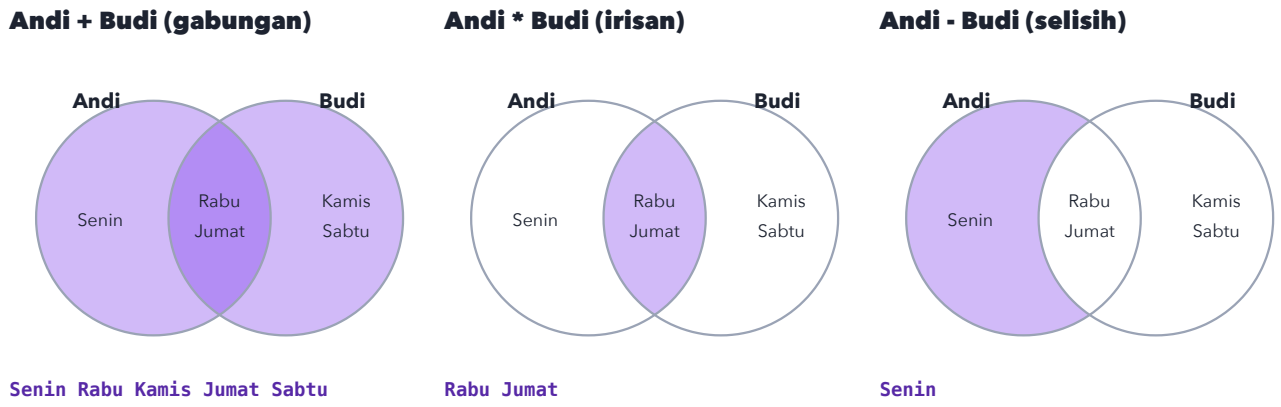
	Sen	Sel	Rab	Kam	Jum	Sab	Min
Andi	1	0	1	0	1	0	0
Budi	0	0	1	1	1	1	0
Andi + Budi (or)	1	0	1	1	1	1	0
Andi * Budi (and)	0	0	1	0	1	0	0
Andi - Budi (and not)	1	0	0	0	0	0	0

Gambar 14.6 Himpunan sebagai deretan bit: gabungan sama dengan OR, irisan sama dengan AND, selisih sama dengan AND NOT per kolom.

14.4.1 Operator himpunan

Tabel 14.2 Operator pada himpunan

Operator	Nama	Hasil
`A + B`	gabungan (*union*)	himpunan anggota A atau B
`A * B`	irisan (*intersection*)	himpunan anggota A dan sekaligus B
`A - B`	selisih (*difference*)	anggota A yang bukan anggota B
`x in A`	keanggotaan	`Boolean` : apakah x anggota A
`A = B`, `A <> B`	sama, tidak sama	`Boolean`
`A <= B`, `A >= B`	himpunan bagian, himpunan induk	`Boolean`



Bagian berwarna adalah hasil operasi. Teks di bawah diagram adalah isi himpunan hasil, seperti yang dicetak Program 14.3.

Gambar 14.7 Diagram Venn untuk `Andi = [Senin, Rabu, Jumat]` dan `Budi = [Rabu, Kamis, Jumat, Sabtu]`.

Selain operator, ada dua prosedur untuk mengubah satu anggota: `Include(H, x)` menambahkan `x` ke `H`, dan `Exclude(H, x)` membuangnya. Keduanya aman dipanggil berulang: menambah anggota yang sudah ada, atau membuang yang tidak ada, tidak menimbulkan galat.

> **AWAS**
>
> Himpunan **tidak dapat dicetak** dengan `Write`, dan anggotanya tidak dapat diambil dengan indeks. Untuk menampilkannya, jalankan perulangan atas semua nilai tipe dasar dan periksa dengan `in`, persis seperti prosedur `Tampilkan` pada program berikut.

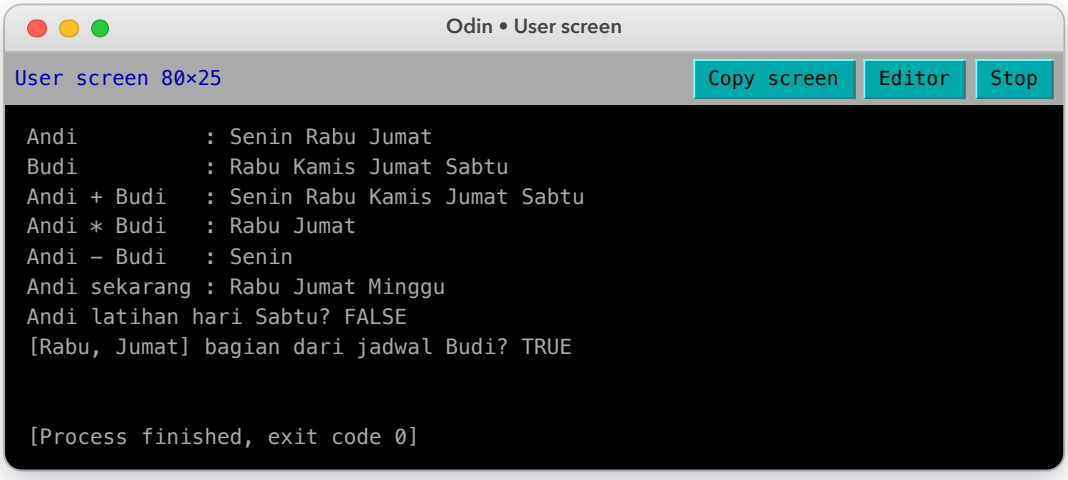
14.4.2 Program ketiga: jadwal latihan

Buka `bab14_03_tipe_set.pas`. Andi berlatih pada Senin, Rabu, dan Jumat; Budi pada Rabu, Kamis, Jumat, dan Sabtu. Program menampilkan gabungan, irisan, dan selisih jadwal keduanya, lalu mengubah jadwal Andi.

Program 14.3 `bab14_03_tipe_set.pas` *Tipe set (himpunan)* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 14.3 — Tipe set (himpunan)
3    Set menyimpan kumpulan nilai ordinal tanpa urutan dan tanpa kembar.
4    Operasi himpunan:  +  gabungan     *  irisan     -  selisih
5                       in  anggota?    =  sama?      <=  himpunan bagian?
6    Include dan Exclude menambah atau membuang satu anggota. }
7
8  program bab14_03_tipe_set;
9
10 type
11   THari = (Senin, Selasa, Rabu, Kamis, Jumat, Sabtu, Minggu);
12   TJadwal = set of THari;
13
14 const
15   NamaHari: array[THari] of string[6] =
16     ('Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat', 'Sabtu', 'Minggu');
17
18 procedure Tampilkan(Judul: string; Jadwal: TJadwal);
```

```
19  var
20    H: THari;
21  begin
22    Write(Judul, ': ');
23    if Jadwal = [] then                    { [] adalah himpunan kosong }
24      Write('(kosong)');
25    for H := Senin to Minggu do
26      if H in Jadwal then                  { periksa keanggotaan }
27        Write(NamaHari[H], ' ');
28    WriteLn;
29  end;
30
31  var
32    Andi, Budi: TJadwal;
33
34  begin
35    Andi := [Senin, Rabu, Jumat];          { jadwal latihan Andi }
36    Budi := [Rabu, Kamis, Jumat, Sabtu];   { jadwal latihan Budi }
37
38    Tampilkan('Andi          ', Andi);
39    Tampilkan('Budi          ', Budi);
40    Tampilkan('Andi + Budi   ', Andi + Budi);   { salah satu latihan }
41    Tampilkan('Andi * Budi   ', Andi * Budi);   { latihan bersama }
42    Tampilkan('Andi - Budi   ', Andi - Budi);   { hanya Andi }
43
44    Include(Andi, Minggu);                 { Andi menambah hari Minggu }
45    Exclude(Andi, Senin);                  { dan berhenti hari Senin }
46    Tampilkan('Andi sekarang ', Andi);
47
48    WriteLn('Andi latihan hari Sabtu? ', Sabtu in Andi);
49    WriteLn('[Rabu, Jumat] bagian dari jadwal Budi? ', [Rabu, Jumat] <= Budi);
50  end.
```



Gambar 14.8 Keluaran `bab14_03_tipe_set.pas` di User screen

Baris	Penjelasan
10–12	`TJadwal` adalah himpunan dengan anggota yang mungkin berupa nilai `THari`, tujuh bit.
14–16	Array nama hari, seperti Program 14.1, untuk mencetak nama.

Baris	Penjelasan
18–29	Prosedur `Tampilkan` menerima judul dan satu `TJadwal` (set boleh menjadi parameter). Baris 23 membandingkan dengan `[]`, himpunan kosong, lalu perulangan baris 25–27 mencetak tiap hari yang `H in Jadwal` (baris 26).
35–36	Mengisi dua himpunan dengan kurung siku. Urutan penulisan anggota tidak berpengaruh.
38–39	Mencetak himpunan awal Andi dan Budi.
40	`Andi + Budi`: hari yang dipakai salah satunya.
41	`Andi * Budi`: hari latihan bersama, yaitu Rabu dan Jumat.
42	`Andi - Budi`: hari yang hanya dipakai Andi, yaitu Senin.
44–46	`Include` menambah Minggu ke jadwal Andi, `Exclude` membuang Senin; hasilnya dicetak ulang.
48	`Sabtu in Andi` menghasilkan `Boolean`. Setelah perubahan di baris 44–45, Sabtu memang tidak ada di jadwal Andi.
49	Himpunan literal `[Rabu, Jumat]` dibandingkan dengan `<=`: apakah ia himpunan bagian jadwal Budi.

14.4.3 Set of Char: memeriksa jenis karakter

Himpunan paling sering dipakai pada `Char`. Tanpa set, memeriksa huruf besar berarti menulis `(C >= 'A') and (C <= 'Z')`. Dengan set, cukup `C in ['A'..'Z']`. Set dapat disimpan sebagai konstanta bernama sehingga kondisi terbaca seperti kalimat: `Sandi[I] in HurufBesar`.

```
const
  HurufBesar = ['A'..'Z'];
  Angka      = ['0'..'9'];
  Simbol     = ['!', '@', '#', '$'];

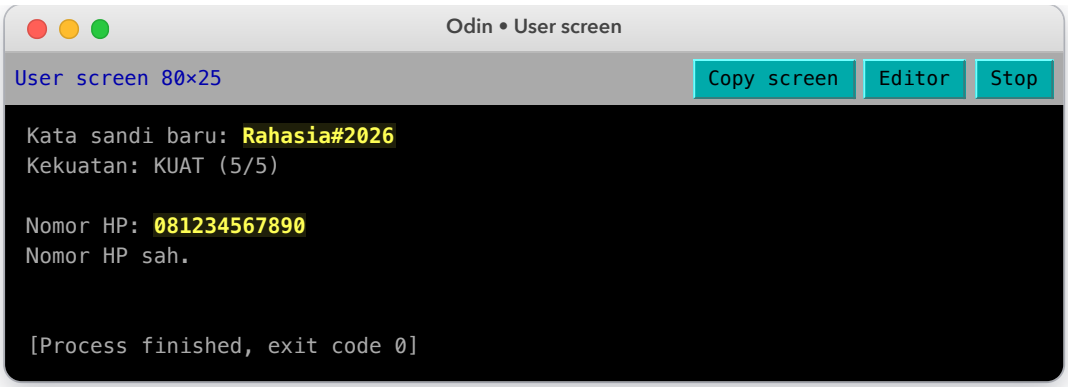
if Sandi[I] in HurufBesar then ...
```

Program `bab14_04_validasi_huruf_set.pas` menerapkannya untuk menilai kekuatan kata sandi (satu poin untuk tiap syarat dari lima) dan memeriksa nomor HP. Masukan pertama adalah kata sandi yang memenuhi semua syarat.

Program 14.4 `bab14_04_validasi_huruf_set.pas` *Latihan: validasi dengan set* File ▸ Examples…

```
1  { Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 14.4 – Latihan: validasi dengan set
3    Set of Char memudahkan pemeriksaan jenis karakter. Daripada menulis
4    (C >= 'A') and (C <= 'Z'), cukup tulis C in HurufBesar.
5    Program ini menilai kekuatan kata sandi dan memeriksa nomor HP. }
6
7  program bab14_04_validasi_huruf_set;
8
9  const
10   HurufBesar = ['A'..'Z'];
11   HurufKecil = ['a'..'z'];
12   Angka = ['0'..'9'];
13   Simbol = ['!', '@', '#', '$', '%', '&', '*', '-', '_', '?'];
```

```
14
15  var
16    Sandi, NomorHP: string;
17    I, Skor: Integer;
18    AdaBesar, AdaKecil, AdaAngka, AdaSimbol, Sah: Boolean;
19
20  begin
21    Write('Kata sandi baru: ');
22    ReadLn(Sandi);
23    AdaBesar := False; AdaKecil := False;
24    AdaAngka := False; AdaSimbol := False;
25    for I := 1 to Length(Sandi) do
26    begin
27      if Sandi[I] in HurufBesar then AdaBesar := True;
28      if Sandi[I] in HurufKecil then AdaKecil := True;
29      if Sandi[I] in Angka then AdaAngka := True;
30      if Sandi[I] in Simbol then AdaSimbol := True;
31    end;
32
33    Skor := 0;                              { satu poin untuk tiap syarat }
34    if Length(Sandi) >= 8 then Skor := Skor + 1;
35    if AdaBesar then Skor := Skor + 1;
36    if AdaKecil then Skor := Skor + 1;
37    if AdaAngka then Skor := Skor + 1;
38    if AdaSimbol then Skor := Skor + 1;
39    case Skor of
40      0..2: WriteLn('Kekuatan: LEMAH (', Skor, '/5)');
41      3..4: WriteLn('Kekuatan: SEDANG (', Skor, '/5)');
42      5:    WriteLn('Kekuatan: KUAT (', Skor, '/5)');
43    end;
44    if not AdaAngka then WriteLn('  Saran: tambahkan angka.');
45    if not AdaSimbol then WriteLn('  Saran: tambahkan simbol seperti ! atau #.');
46    WriteLn;
47
48    { Nomor HP: diawali 08, panjang 10–13, hanya berisi angka }
49    Write('Nomor HP: ');
50    ReadLn(NomorHP);
51    Sah := (Length(NomorHP) >= 10) and (Length(NomorHP) <= 13)
52           and (Copy(NomorHP, 1, 2) = '08');
53    for I := 1 to Length(NomorHP) do
54      if not (NomorHP[I] in Angka) then    { kurung: not lalu in }
55        Sah := False;
56    if Sah then
57      WriteLn('Nomor HP sah.')
58    else
59      WriteLn('Nomor HP tidak sah.');
60  end.
```



Gambar 14.9 Kata sandi kuat dan nomor HP sah

Pada contoh berikutnya kata sandi lemah dan nomor HP mengandung huruf.



Gambar 14.10 Kata sandi lemah dan nomor HP tidak sah

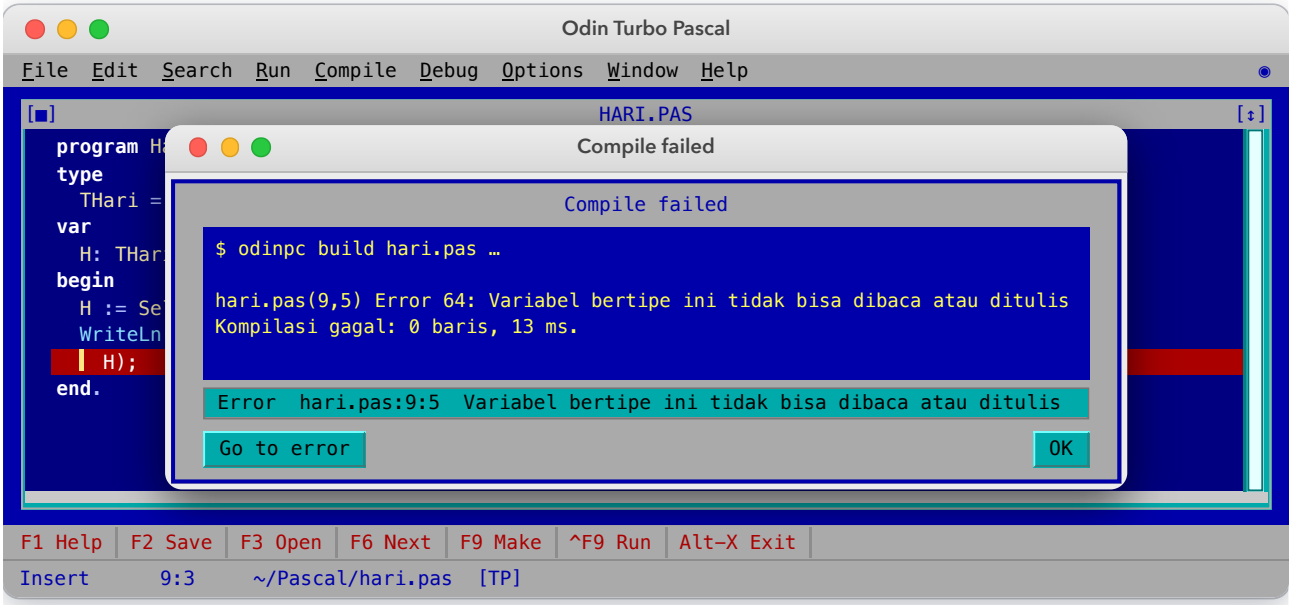
Baris	Penjelasan
9–13	Empat konstanta himpunan karakter. Konstanta `Simbol` mendaftar sepuluh karakter satu per satu, sedangkan tiga lainnya memakai rentang `'A'..'Z'`.
15–18	Penampung kata sandi, nomor HP, skor, dan lima bendera `Boolean`.
23–24	Semua bendera dimulai dari `False`.
25–31	Perulangan membaca kata sandi karakter demi karakter. Tiap `if` memakai `in` untuk menyalakan bendera yang sesuai. Satu karakter bisa menyalakan paling banyak satu bendera karena himpunan-himpunannya tidak saling beririsan.
33–38	Menghitung skor: satu poin bila panjang minimal 8, satu poin untuk tiap bendera yang menyala. Skor maksimum 5.
39–43	`case Skor of` menerjemahkan skor menjadi LEMAH (0..2), SEDANG (3..4), atau KUAT (5).
44–45	Memberi saran bila angka atau simbol belum ada.
49–52	Nomor HP dianggap sah bila panjangnya 10 sampai 13 dan diawali `08` (`Copy(NomorHP, 1, 2)`).
53–55	Setiap karakter harus anggota `Angka`. Perhatikan kurung pada `not (NomorHP[I] in Angka)`: tanpa kurung, `not` dikerjakan lebih dulu dan kompiler menolaknya.
56–59	Hasil akhir dicetak sesuai bendera `Sah`.

> **7** **ATURAN TURBO PASCAL 7**
>
> **Aturan Turbo Pascal 7.** Set hanya boleh berisi paling banyak **256** anggota dengan nomor urut 0..255. `set of Integer` ditolak karena `Integer` berisi 65536 nilai. Gunakan subrange kecil seperti `set of 0..100` atau `set of Byte`.

14.5 Kesalahan yang Sering Terjadi

14.5.1 Mencetak nilai enumerasi

Kesalahan paling sering pada bab ini adalah mencetak enumerasi langsung.



Gambar 14.11 `WriteLn(Hari)` pada enumerasi ditolak kompiler

Perbaikannya sudah Anda lihat di Program 14.1: tampilkan `Ord(H)` atau gunakan array nama `NamaHari[H]`.

14.5.2 Nilai di luar rentang

luar.pas

```
1  program Luar;
2  type
3    TNilai = 0..100;
4  var
5    N: TNilai;
6  begin
7    N := 150;
8    WriteLn(N);
9  end.
```

PESAN KOMPILER

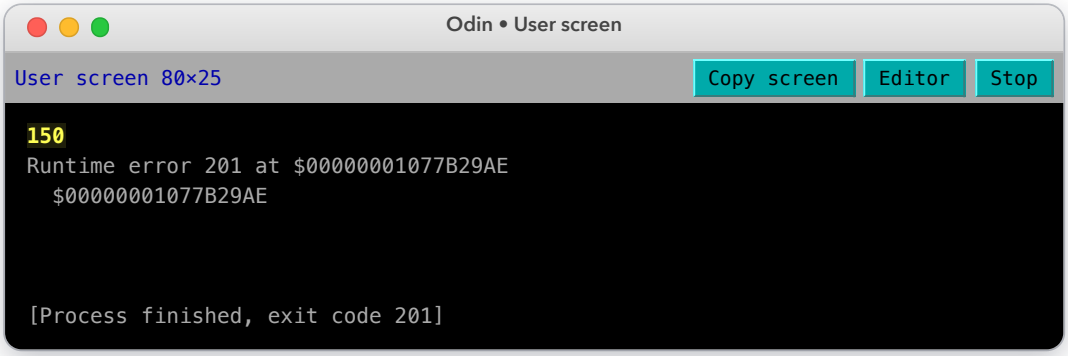
```
luar.pas(7,8) Error: Konstanta di luar jangkauan: 150 tidak di dalam 0..100
```

Gambar 14.12 Konstanta 150 berada di luar rentang `TNilai` (0..100)

Bila nilai berasal dari masukan, kompiler tidak dapat mengetahuinya, sehingga yang terjadi adalah galat saat program berjalan:

luar2.pas

```
1  program Luar2;
2  {$R+}
3  type
4    TNilai = 0..100;
5  var
6    N: TNilai;
7    M: Integer;
8  begin
9    ReadLn(M);
10   N := M;
11   WriteLn(N);
12 end.
```



Gambar 14.13 Masukan 150 disalin ke variabel subrange tanpa diperiksa

Cegah dengan memeriksa `(M >= Low(TNilai)) and (M <= High(TNilai))` sebelum menyalin.

14.5.3 Set dengan tipe dasar terlalu besar

besar.pas

```
1  program Besar;
2  var
3    S: set of Integer;
4  begin
5    S := [1, 2, 3];
6  end.
```

PESAN KOMPILER

```
besar.pas(3,13) Error: Tipe dasar set di luar jangkauan
```

Gambar 14.14 `Integer` terlalu besar untuk tipe dasar set

Pakai tipe dasar yang kecil, misalnya `set of Byte` atau subrange `set of 1..100`.

14.5.4 Memakai ReadLn untuk enumerasi

Sama seperti `Write`, `ReadLn` tidak dapat membaca enumerasi. Bacalah nomor atau teks, lalu terjemahkan sendiri dengan `case` atau perulangan yang mencocokkan nama pada array.

14.6 Rangkuman

- Tipe ordinal (`Integer`, `Char`, `Boolean`, enumerasi, subrange) berurutan dan mendukung `Ord`, `Succ`, `Pred`, `Low`, `High`.
- Enumerasi `(A, B, C)` membuat daftar nama buatan sendiri bernomor 0, 1, 2, ...; dapat dipakai pada `for`, `case`, dan perbandingan.
- Turbo Pascal tidak dapat `Write` atau `ReadLn` enumerasi; gunakan `Ord` atau array nama berindeks enumerasi.
- Subrange `Awal..Akhir` membatasi nilai suatu tipe ordinal; periksa masukan dengan `if` sebelum menyimpannya, sebab nilai di luar rentang menimbulkan runtime error 201.
- Set `set of T` menyimpan himpunan nilai tanpa urutan dan tanpa kembar, paling banyak 256 anggota.
- Operator set: `+` gabungan, `*` irisan, `-` selisih, `in` keanggotaan, `=` sama, `<=` himpunan bagian.
- `Include` dan `Exclude` menambah dan membuang satu anggota; set tidak dapat dicetak langsung.
- `set of Char` menyederhanakan pemeriksaan jenis karakter: `C in ['0'..'9']`.

14.7 Latihan

1. **DASAR** Definisikan `TBulan = (Jan, Feb, Mar, Apr, Mei, Jun)` dan cetak nomor urut tiap bulan dengan `for` serta `Ord`.
2. **DASAR** Ubah Program 14.1 agar hari Sabtu dan Minggu dicetak sebagai “akhir pekan” dan hari lain sebagai “hari kerja”, memakai `case` dengan rentang.
3. **MENENGAH** Buat tipe `TUmur = 0..120`. Baca umur, periksa dengan `Low` dan `High`, lalu tampilkan golongannya (anak, remaja, dewasa, lansia).
4. **MENENGAH** Program 14.3 hanya mencetak hari. Tambahkan jadwal Citra, lalu tampilkan hari ketika **ketiga** orang sama-sama berlatih (irisan tiga himpunan).
5. **MENENGAH** Dengan `set of Char`, hitung berapa huruf vokal pada sebuah kalimat yang diketik pengguna. Petunjuk: `Vokal = ['a','e','i','o','u','A','E','I','O','U']`.
6. **TANTANGAN** Buat program menghitung `Succ` secara melingkar: setelah `Minggu` kembali ke `Senin`. Tulis fungsi `HariBerikut(H: THari): THari` yang tidak menimbulkan galat.
7. **TANTANGAN** Perluas Program 14.4: tambahkan pemeriksaan bahwa kata sandi tidak mengandung spasi (`' '`) dan tampilkan semua jenis karakter yang belum ada dalam kata sandi sebagai saran.

BAGIAN V

Pencarian dan Pengurutan

Algoritma klasik untuk menemukan dan menyusun data, lengkap dengan perbandingan efisiensinya.

15 Pencarian Data

16 Pengurutan Data

BAB 15

BAGIAN V · PENCARIAN DAN PENGURUTAN

Pencarian Data

TUJUAN PEMBELAJARAN

- Menjelaskan apa itu pencarian data, kunci pencarian, dan mengapa data yang banyak membutuhkan algoritma yang tepat.
- Menulis pencarian beruntun (*sequential search*) dengan perulangan `while` dan penanda `Ketemu`.
- Menulis pencarian biner (*binary search*) dengan variabel `Kiri`, `Kanan`, dan `Tengah`, serta menyebutkan syaratnya: data harus terurut.
- Menghitung dan membandingkan jumlah langkah pencarian beruntun (sebanding dengan n) dan pencarian biner (sebanding dengan $\log_2 n$).
- Mencari data di *array of record* menurut kode yang harus sama persis dan menurut sebagian nama.

PROGRAM DALAM BAB INI

15.1 `bab15_01_sequential_search.pas`
Pencarian beruntun (sequential search)

15.2 `bab15_02_binary_search.pas`
Pencarian biner (binary search)

15.3 `bab15_03_cari_data_record.pas`
Mencari data di array of record

Kita mencari sesuatu setiap hari: nomor telepon di daftar kontak, nama siswa di daftar hadir, satu kata di kamus. Komputer pun terus mencari, dan karena komputer cepat, kita jarang menyadari bahwa cara mencari menentukan seberapa cepat jawaban muncul. Bab ini memperkenalkan dua algoritma pencarian klasik: pencarian beruntun yang sederhana, dan pencarian biner yang jauh lebih hemat langkah tetapi menuntut data terurut. Anda akan menjalankan keduanya, menelusuri langkahnya satu per satu, lalu menerapkannya pada data berbentuk *record*.

15.1 Apa yang Dimaksud Mencari Data

Pencarian (*searching*) adalah proses menemukan satu atau beberapa elemen dalam sekumpulan data yang cocok dengan sesuatu yang kita inginkan. Sesuatu yang kita inginkan itu disebut **kunci pencarian** (*search key*). Pada program bab ini, kunci pencarian berupa bilangan (untuk array `Integer`), kode barang, atau sepotong nama (untuk array *record*). Hasil pencarian selalu salah satu dari dua kemungkinan: data **ditemukan**, biasanya bersama **posisinya** (nomor indeks array), atau data **tidak ditemukan**.

Bayangkan guru yang mencari lembar jawaban milik seorang siswa di tumpukan 40 lembar yang tercampur. Ia membuka lembar pertama, kedua, ketiga, dan seterusnya sampai bertemu nama yang tepat. Itulah pencarian beruntun. Sekarang bayangkan Anda mencari kata "*komputer*" di kamus. Tidak seorang pun membuka kamus dari halaman pertama: kita membuka bagian tengah, melihat huruf apa yang muncul, lalu melompat ke kiri atau ke kanan. Itulah pencarian biner, dan ia hanya mungkin karena kata di kamus tersusun menurut abjad.

1 Pencarian beruntun

Memeriksa elemen satu per satu dari depan. Berlaku untuk data apa pun, terurut atau tidak. Pada data n elemen, paling banyak n perbandingan.

2 Pencarian biner

Membandingkan dengan elemen tengah lalu membuang separuh data. **Data wajib terurut.** Pada data n elemen, hanya sekitar $\log_2 n$ perbandingan.

Semua program di bab ini memakai array dengan indeks mulai dari 1 (Bab 11) dan menyimpan hasil pencarian dalam sebuah variabel posisi atau penanda `Boolean`. Perulangan yang dipakai adalah `while` (Bab 7), karena kita tidak tahu sebelumnya berapa kali perulangan harus berjalan: ia boleh berhenti begitu data ditemukan.

15.2 Pencarian Beruntun

Pencarian beruntun (*sequential search* atau *linear search*) bekerja persis seperti guru tadi. Mulai dari elemen pertama, bandingkan elemen dengan kunci. Bila sama, berhenti dan laporkan posisinya. Bila berbeda, pindah ke elemen berikutnya. Bila elemen terakhir sudah diperiksa dan tidak ada yang sama, kunci itu tidak ada dalam data.

BENTUK UMUM

```
I := 1;
Ketemu := False;
while (I <= N) and not Ketemu do
  if Data[I] = Kunci then
    Ketemu := True
  else
    I := I + 1;
{ Ketemu = True: posisinya I.   Ketemu = False: I = N + 1 }
```

Dua hal penting pada pola ini. Pertama, syarat perulangan punya **dua alasan berhenti**: elemen ditemukan (`Ketemu` menjadi `True`) atau data habis (`I` melewati `N`). Kedua, ketika elemen ditemukan, `I` sengaja **tidak** ditambah, sehingga setelah perulangan `I` tetap menunjuk posisi data yang cocok. Gambar berikut memperlihatkan jalannya pencarian bilangan 32 pada data yang dipakai Program 15.1.



Gambar 15.1 Pencarian 32 pada data Program 15.1: empat perbandingan, lalu berhenti

15.2.1 Program 15.1: Mencari Bilangan di Dalam Array

Program ini menyimpan sepuluh bilangan pada konstanta bertipe `Data`, meminta satu bilangan, lalu mencarinya dengan pola di atas sambil menghitung berapa perbandingan yang dilakukan. Buka lewat `File ▸ Examples...`, pilih `bab15_01_sequential_search.pas`, lalu tekan Ctrl+F9.

Program 15.1 `bab15_01_sequential_search.pas` *Pencarian beruntun (sequential search)* File ▸ Examples...

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 15.1 — Pencarian beruntun (sequential search)
3    Periksa elemen satu per satu dari depan sampai yang dicari ketemu
4    atau data habis. Sederhana, dan data tidak perlu terurut. }
5
6  program bab15_01_sequential_search;
7
8  const
9    N = 10;
10   Data: array[1..N] of Integer = (34, 7, 23, 32, 5, 62, 32, 18, 9, 41);
11
12 var
13   Dicari, I, Langkah: Integer;
14   Ketemu: Boolean;
```

```
15
16  begin
17    Write('Data: ');
18    for I := 1 to N do
19      Write(Data[I], ' ');
20    WriteLn;
21    Write('Bilangan yang dicari: ');
22    ReadLn(Dicari);
23
24    I := 1;
25    Ketemu := False;
26    Langkah := 0;
27    while (I <= N) and not Ketemu do       { berhenti bila ketemu atau habis }
28    begin
29      Langkah := Langkah + 1;
30      if Data[I] = Dicari then
31        Ketemu := True
32      else
33        I := I + 1;
34    end;
35
36    if Ketemu then
37      WriteLn(Dicari, ' ditemukan di posisi ', I, ' setelah ', Langkah,
38        ' perbandingan.')
39    else
40      WriteLn(Dicari, ' tidak ada. Semua ', Langkah, ' data sudah diperiksa.');
41
42    { Variasi: cari SEMUA posisi, jadi perulangan tidak berhenti di awal }
43    Write('Semua posisi: ');
44    for I := 1 to N do
45      if Data[I] = Dicari then
46        Write(I, ' ');
47    WriteLn;
48  end.
```



Gambar 15.2 Mencari 32: ditemukan di posisi 4 setelah 4 perbandingan

Tabel 15.1 Penjelasan Program 15.1

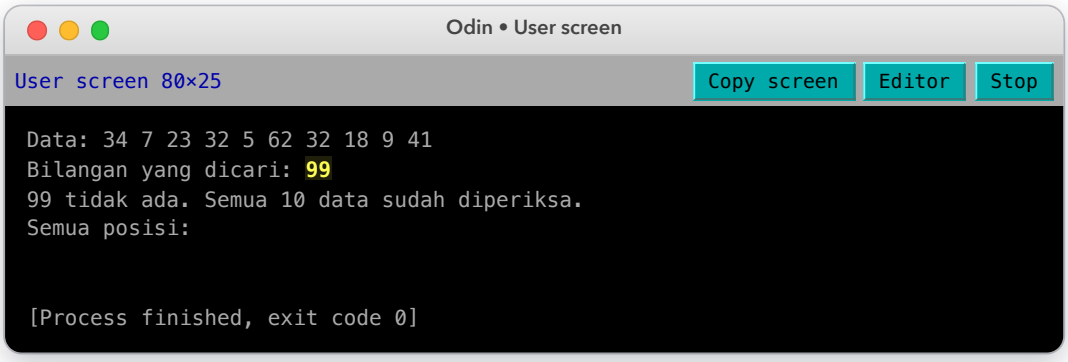
Baris	Penjelasan
9–10	`N` adalah banyaknya data. `Data` adalah array konstanta berisi sepuluh `Integer` yang sengaja tidak terurut, dan **bilangan 32 muncul dua kali** (posisi 4 dan 7).
13–14	`Dicari` menyimpan kunci, `I` adalah penunjuk posisi, `Langkah` menghitung perbandingan, `Ketemu` adalah penanda.

Baris	Penjelasan
17–22	Menampilkan seluruh data dengan `for`, lalu meminta kunci dengan `ReadLn(Dicari)`.
24–26	Keadaan awal: mulai dari posisi 1, belum ketemu, belum ada perbandingan.
27	Syarat dua alasan berhenti: `(I <= N) and not Ketemu`. Kurung wajib (lihat bagian kesalahan).
29	Setiap putaran melakukan satu perbandingan, sehingga `Langkah` naik satu.
30–33	Bila `Data[I]` sama dengan kunci, `Ketemu := True` dan `I` dibiarkan. Bila tidak, `I` maju satu. Perhatikan tidak ada titik koma sebelum `else`.
36–40	Melaporkan hasil: posisi dan jumlah perbandingan bila ketemu, atau pemberitahuan bahwa semua data sudah diperiksa.
42–47	Variasi: perulangan `for` yang memeriksa **semua** elemen dan mencetak setiap posisi yang cocok.

Dengan kunci 32, program berhenti di posisi 4 setelah 4 perbandingan: elemen 34, 7, dan 23 gagal, elemen ke-4 berhasil. Baris terakhir keluaran, `Semua posisi: 4 7`, datang dari perulangan variasi (baris 42–47) yang tidak berhenti di temuan pertama, sehingga posisi 7 ikut tercetak. Dua kasus lain memperlihatkan batas atas dan batas bawah jumlah langkah.



Gambar 15.3 Kasus terbaik: 34 ada di posisi pertama, cukup satu perbandingan



Gambar 15.4 Kasus terburuk: 99 tidak ada, sehingga kesepuluh data diperiksa

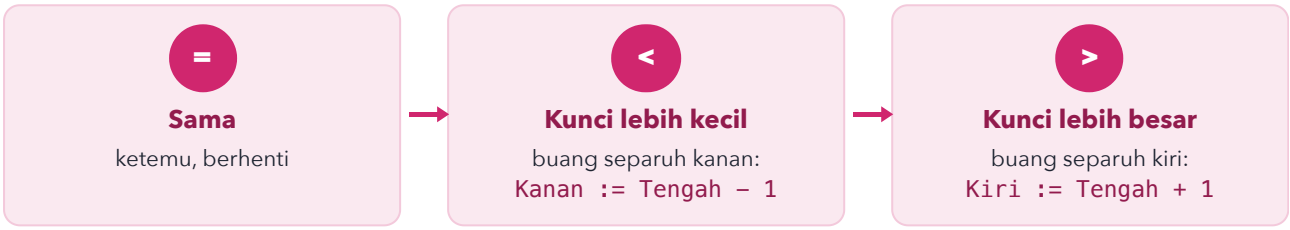
Kasus terbaik hanya satu perbandingan, kasus terburuk n perbandingan (di sini 10), yaitu ketika data berada di posisi terakhir atau tidak ada sama sekali. Perhatikan bahwa pada kasus 99 baris `Semua posisi:` kosong, karena tidak ada posisi yang cocok.

COBA SENDIRI

Coba Sendiri. Jalankan program dengan kunci 62, 9, dan 5, lalu catat jumlah perbandingannya. Apakah jumlahnya selalu sama dengan posisi data? Lalu ubah isi `Data` agar 5 berada di depan, dan jalankan lagi.

15.3 Pencarian Biner

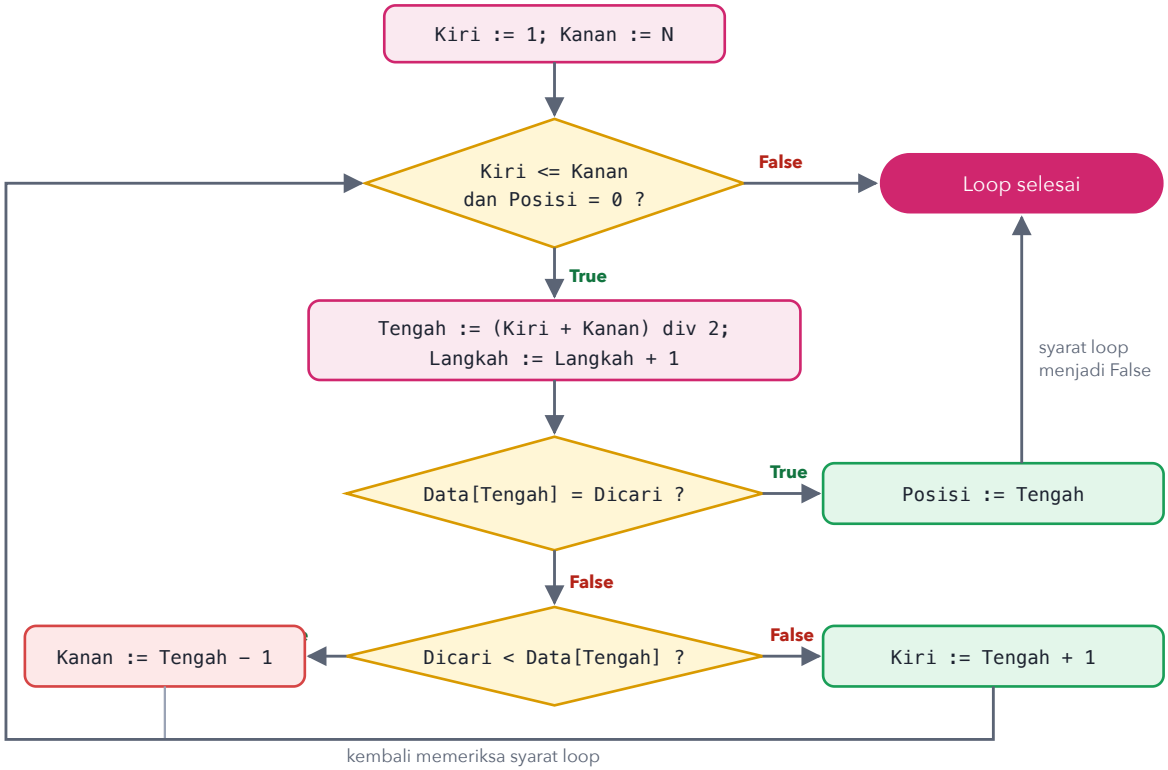
Pencarian beruntun jujur tetapi boros: untuk sejuta data, kasus terburuknya sejuta perbandingan. Bila data sudah **terurut**, ada cara yang jauh lebih cerdik. Seperti membuka kamus, kita membandingkan kunci dengan elemen di **tengah** rentang. Ada tiga kemungkinan:



Rentang yang masih mungkin berisi kunci dibatasi dua penunjuk: `Kiri` (awal rentang) dan `Kanan` (akhir rentang). Pada awalnya `Kiri = 1` dan `Kanan = N`, yaitu seluruh array. Elemen tengah dihitung dengan `Tengah := (Kiri + Kanan) div 2`. Setiap putaran, rentang menyusut sekitar separuh; bila `Kiri` melampaui `Kanan`, rentang kosong dan kunci pasti tidak ada.

BENTUK UMUM

```
Kiri := 1;  Kanan := N;  Posisi := 0;
while (Kiri <= Kanan) and (Posisi = 0) do
begin
  Tengah := (Kiri + Kanan) div 2;
  if Data[Tengah] = Kunci then Posisi := Tengah
  else if Kunci < Data[Tengah] then Kanan := Tengah - 1
  else Kiri := Tengah + 1;
end;
```



Gambar 15.5 Diagram alir pencarian biner pada Program 15.2: setiap putaran membuang separuh rentang

AWAS

Syarat mutlak: data terurut. Seluruh logika "buang separuh" bersandar pada satu fakta, yaitu elemen di kiri `Tengah` pasti lebih kecil dan elemen di kanannya pasti lebih besar. Bila data tidak terurut, program membuang separuh yang salah dan melaporkan "tidak ditemukan" untuk data yang sebenarnya ada. Contohnya dibuktikan di bagian kesalahan.

15.3.1 Program 15.2: Pencarian Biner dengan Jejak Langkah

Program ini mencari bilangan di array 15 elemen yang sudah terurut naik. Ia mencetak **setiap langkah** (rentang dan elemen tengahnya), sehingga Anda bisa mengikuti penyusutan rentang. Buka `bab15_02_binary_search.pas` dari `File ▸ Examples...`.

Program 15.2 `bab15_02_binary_search.pas` *Pencarian biner (binary search)* File ▸ Examples...

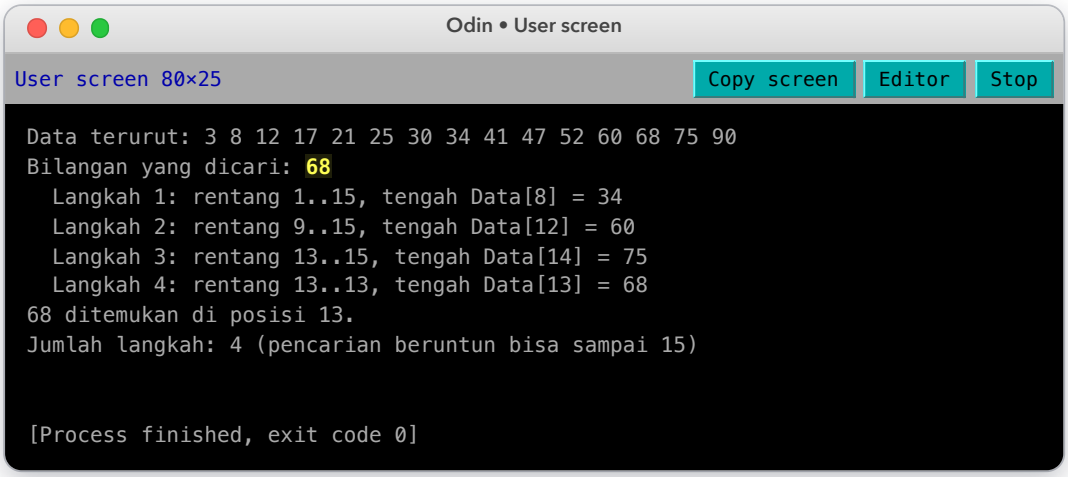
```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 15.2 — Pencarian biner (binary search)
3    Syarat: data TERURUT. Bandingkan dengan elemen tengah; bila yang
4    dicari lebih kecil, buang separuh kanan, bila lebih besar buang
5    separuh kiri. Setiap langkah membuang separuh data, jadi 1000 data
6    cukup diperiksa sekitar 10 kali. }
7
8  program bab15_02_binary_search;
9
10 const
11   N = 15;
12   Data: array[1..N] of Integer =
13     (3, 8, 12, 17, 21, 25, 30, 34, 41, 47, 52, 60, 68, 75, 90);
14
```

```
15  var
16    Dicari, Kiri, Kanan, Tengah, Langkah, Posisi, I: Integer;
17
18  begin
19    Write('Data terurut: ');
20    for I := 1 to N do
21      Write(Data[I], ' ');
22    WriteLn;
23    Write('Bilangan yang dicari: ');
24    ReadLn(Dicari);
25
26    Kiri := 1;
27    Kanan := N;
28    Posisi := 0;                          { 0 berarti belum ketemu }
29    Langkah := 0;
30    while (Kiri <= Kanan) and (Posisi = 0) do
31    begin
32      Tengah := (Kiri + Kanan) div 2;
33      Langkah := Langkah + 1;
34      WriteLn('  Langkah ', Langkah, ': rentang ', Kiri, '..', Kanan,
35        ', tengah Data[', Tengah, '] = ', Data[Tengah]);
36      if Data[Tengah] = Dicari then
37        Posisi := Tengah
38      else if Dicari < Data[Tengah] then
39        Kanan := Tengah - 1               { cari di separuh kiri }
40      else
41        Kiri := Tengah + 1;               { cari di separuh kanan }
42    end;
43
44    if Posisi > 0 then
45      WriteLn(Dicari, ' ditemukan di posisi ', Posisi, '.')
46    else
47      WriteLn(Dicari, ' tidak ada dalam data.');
48    WriteLn('Jumlah langkah: ', Langkah, ' (pencarian beruntun bisa sampai ',
49      N, ')');
50  end.
```

Tabel 15.2 Penjelasan Program 15.2

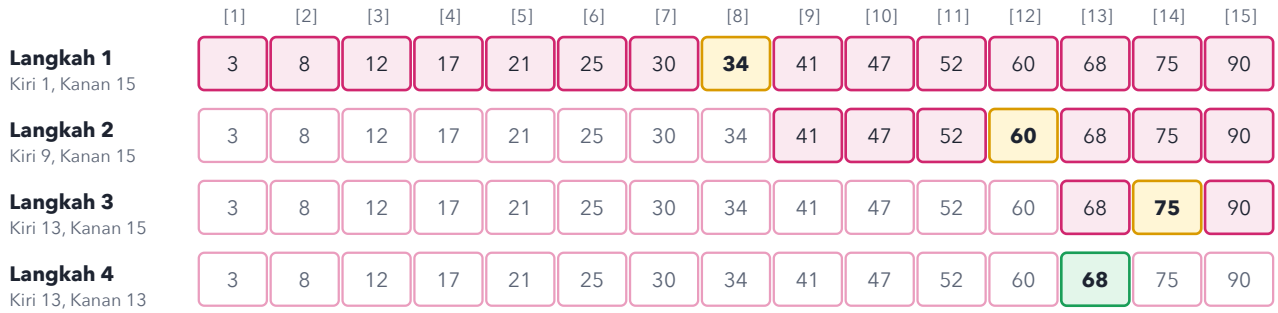
Baris	Penjelasan
11–13	Lima belas bilangan **terurut naik** dari 3 sampai 90. Banyak data 15 dipilih karena $15 = 2^4 - 1$: rentang habis dibagi dua tepat empat kali.
16	Semua penunjuk dan penghitung bertipe `Integer`. `Posisi` menyimpan hasil, `Tengah` posisi elemen yang sedang dibandingkan.
26–29	Rentang awal adalah seluruh array. `Posisi := 0` berarti belum ketemu; angka 0 aman dipakai karena indeks array dimulai dari 1.
30	Perulangan berjalan selama rentang masih berisi elemen (`Kiri <= Kanan`) dan data belum ditemukan.
32–33	Menghitung posisi tengah dengan `div` (pembagian bulat), lalu menambah `Langkah`.
34–35	Mencetak jejak: nomor langkah, rentang `Kiri..Kanan`, posisi tengah dan isinya.
36–41	Tiga kemungkinan: sama (simpan posisi), lebih kecil (`Kanan := Tengah - 1`), atau lebih besar (`Kiri := Tengah + 1`). Tanda `-1` dan `+1` membuang elemen tengah yang sudah diperiksa.

Baris	Penjelasan
44–49	Melaporkan hasil, lalu membandingkan jumlah langkah dengan batas atas pencarian beruntun, yaitu `N` perbandingan.



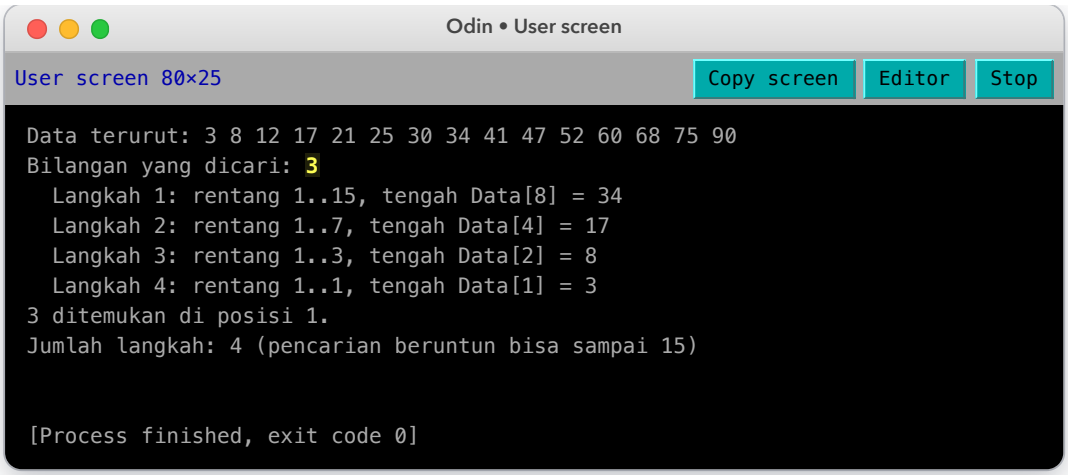
Gambar 15.6 Mencari 68: empat langkah, ditemukan di posisi 13

Telusuri jejak untuk kunci 68. Langkah 1: rentang 1..15, tengah `Data[8]` = 34. Karena 68 > 34, `Kiri` menjadi 9. Langkah 2: rentang 9..15, tengah `Data[12]` = 60; 68 > 60, `Kiri` menjadi 13. Langkah 3: rentang 13..15, tengah `Data[14]` = 75; 68 < 75, `Kanan` menjadi 13. Langkah 4: rentang 13..13, tengah `Data[13]` = 68, ketemu. Dari lima belas elemen, hanya empat yang dibandingkan.

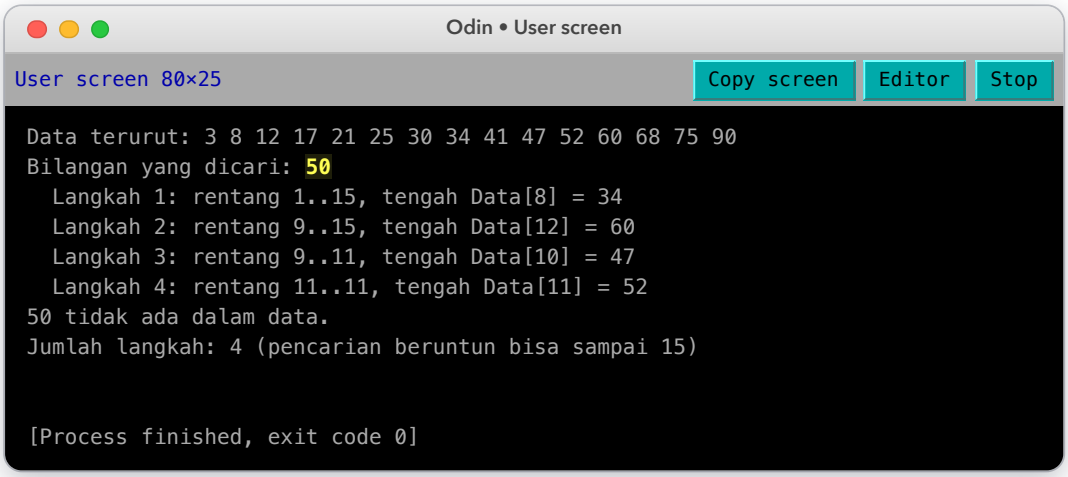


Gambar 15.7 Penyusutan rentang saat mencari 68: biru muda = masih mungkin, putih = sudah dibuang, kuning = elemen tengah, hijau = ketemu

Kunci yang berada di ujung kiri pun hanya butuh empat langkah, karena setiap langkah selalu memotong rentang menjadi sekitar separuh. Kasus kunci yang **tidak ada** diperlihatkan oleh dua run berikut.



Gambar 15.8 Mencari 3, elemen paling kiri: tetap empat langkah



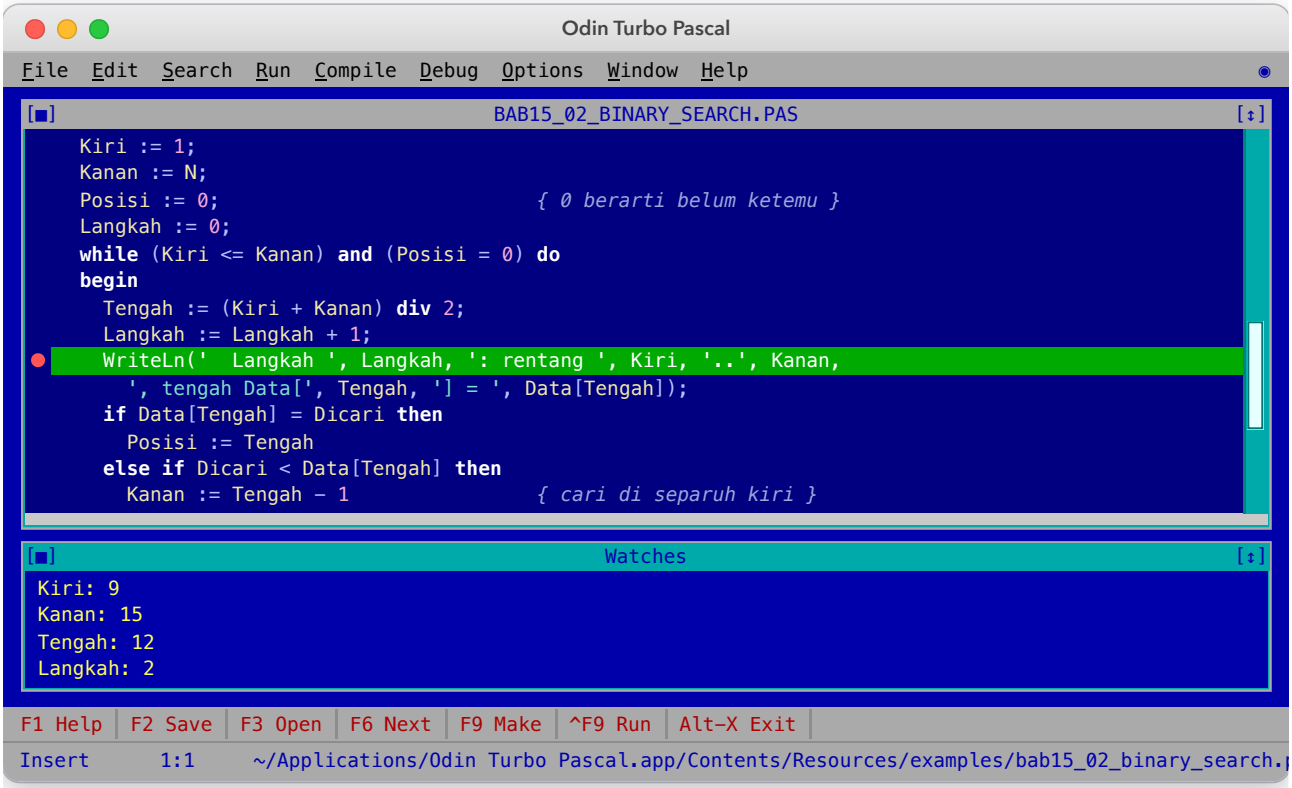
Gambar 15.9 Mencari 50, yang tidak ada: rentang menyusut sampai kosong

Untuk kunci 50, langkah ke-4 memeriksa `Data[11]` = 52. Karena 50 < 52, `Kanan` menjadi 10 sementara `Kiri` tetap 11. Syarat `Kiri <= Kanan` kini salah, perulangan berhenti, dan `Posisi` masih 0, sehingga program melaporkan "tidak ada dalam data". Gambar berikut menunjukkan akhir yang sama.

	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]
Langkah 1 Kiri 1, Kanan 15	3	8	12	17	21	25	30	**34**	41	47	52	60	68	75	90
Langkah 2 Kiri 9, Kanan 15	3	8	12	17	21	25	30	34	41	47	52	**60**	68	75	90
Langkah 3 Kiri 9, Kanan 11	3	8	12	17	21	25	30	34	41	**47**	52	60	68	75	90
Langkah 4 Kiri 11, Kanan 11	3	8	12	17	21	25	30	34	41	47	**52**	60	68	75	90
Berhenti Kiri 11, Kanan 10	3	8	12	17	21	25	30	34	41	47	52	60	68	75	90

Gambar 15.10 Pencarian 50: setelah langkah 4, Kiri = 11 melampaui Kanan = 10, jadi tidak ada elemen yang tersisa

Pencarian ini bisa dilacak langsung di debugger Odin. Pasang titik henti (*breakpoint*) pada baris 34 dengan Ctrl + F8, jalankan lewat `Debug ▸ Start debugging`, dan buka jendela `Debug ▸ Watch window` untuk mengamati `Kiri`, `Kanan`, `Tengah`, dan `Langkah` setiap putaran.



Gambar 15.11 Debugger berhenti di baris 34 pada putaran ke-2 saat mencari 68: rentang sudah menyempit menjadi 9..15

15.4 Membandingkan Jumlah Langkah

Mengapa repot mengurutkan data dan menulis pencarian biner? Karena selisih jumlah langkahnya membesar dengan cepat. Pada n elemen, pencarian beruntun paling banyak melakukan n perbandingan. Pencarian biner paling banyak melakukan $\lfloor \log_2 n \rfloor + 1$ perbandingan, yaitu berapa kali n dapat dibagi dua sampai tersisa satu elemen, ditambah satu. Untuk $n = 15$ hasilnya 4, sama dengan "Jumlah langkah: 4" pada keluaran Program 15.2.

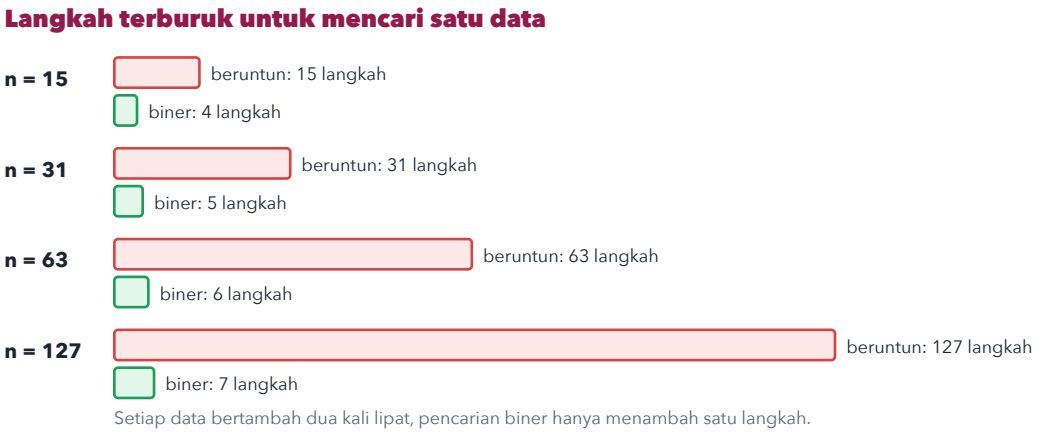
Tabel 15.3 Langkah terburuk pencarian beruntun dan pencarian biner (hasil hitungan matematika)

Jumlah data (n)	Beruntun (paling banyak n)	Biner ($\lfloor \log_2 n \rfloor + 1$)
15	15	4
100	100	7
1.000	1.000	10
1.000.000	1.000.000	20
1.000.000.000	1.000.000.000	30

10
langkah biner untuk 1.000 data

20
langkah biner untuk 1 juta data

30
langkah biner untuk 1 miliar data



Gambar 15.12 Pencarian beruntun tumbuh sebanding dengan jumlah data, pencarian biner hanya bertambah satu langkah setiap data berlipat dua

Perbandingan ini tidak berarti pencarian biner selalu lebih baik. Data harus terurut dulu, dan mengurutkan data itu sendiri memakan langkah (Bab 16 membahasnya). Bila data hanya dicari sekali, lalu dibuang, pencarian beruntun yang sederhana sudah cukup. Bila data dicari berulang kali, mengurutkan satu kali lalu memakai pencarian biner berkali-kali jauh lebih hemat.

Tabel 15.4 Memilih algoritma pencarian

Keadaan	Pilihan yang masuk akal
Data sedikit, atau hanya dicari sekali	Pencarian beruntun
Data tidak terurut dan tidak akan diurutkan	Pencarian beruntun
Data besar, sudah terurut, dicari berulang kali	Pencarian biner
Ingin semua kemunculan, bukan satu	Pencarian beruntun dengan `for` seperti baris 42–47 Program 15.1

15.5 Mencari Data di Array of Record

Data nyata jarang berupa bilangan polos. Daftar barang punya kode, nama, harga, dan stok; daftar siswa punya nama dan nilai. Bab 13 memperkenalkan *record* untuk menggabungkan beberapa *field* menjadi satu satuan, dan array of record menyimpan banyak satuan seperti itu. Prinsip pencarian tidak berubah: yang dibandingkan adalah **satu field** sebagai kunci, sedangkan hasilnya adalah satu record utuh.

Program 15.3 mengelola enam barang alat tulis dan menawarkan dua jenis pencarian yang berbeda sifatnya:

K **Menurut kode**

Kecocokan **persis**. Kode itu unik, jadi pencarian cukup menemukan satu record lalu berhenti. Pola sama dengan Program 15.1.

N **Menurut nama**

Kecocokan **sebagian**. Banyak barang bisa memuat kata yang sama, jadi semua record diperiksa dan setiap yang cocok ditampilkan.

Kecocokan sebagian memakai fungsi `Pos(Cari, Teks)`, yang mengembalikan posisi pertama `Cari` di dalam `Teks`, atau 0 bila tidak ada. Karena huruf besar dan kecil dianggap berbeda oleh `Pos`, program lebih dulu mengubah kedua string menjadi huruf kapital dengan fungsi `Kapital` buatannya sendiri, memakai `UpCase` pada tiap karakter (Bab 12).



Gambar 15.13 Dua cara mencari di array of record pada Program 15.3: satu hasil lalu berhenti, atau semua hasil

15.5.1 Program 15.3: Mencari Barang di Gudang

Buka `bab15_03_cari_data_record.pas`. Program meminta kode barang, lalu kata kunci nama.

Program 15.3 `bab15_03_cari_data_record.pas` *Mencari data di array of record* File ▸ Examples…

```
1  { Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 15.3 – Mencari data di array of record
3    Pencarian beruntun pada data barang: menurut kode (harus sama
4    persis) dan menurut nama (cukup sebagian, huruf besar/kecil
5    diabaikan). }
6
7  program bab15_03_cari_data_record;
8
9  type
10   TBarang = record
11     Kode: string[4];
12     Nama: string[20];
13     Harga: LongInt;
14     Stok: Integer;
15   end;
16
17 const
18   N = 6;
19   Gudang: array[1..N] of TBarang = (
20     (Kode: 'A001'; Nama: 'Buku tulis';    Harga: 5000;  Stok: 120),
```

```
    (Kode: 'A002'; Nama: 'Pensil 2B';     Harga: 3500;  Stok: 200),
    (Kode: 'A003'; Nama: 'Penghapus';     Harga: 2000;  Stok: 75),
    (Kode: 'B001'; Nama: 'Buku gambar';   Harga: 8000;  Stok: 40),
    (Kode: 'B002'; Nama: 'Pensil warna';  Harga: 25000; Stok: 15),
    (Kode: 'C001'; Nama: 'Penggaris 30';  Harga: 4500;  Stok: 60));

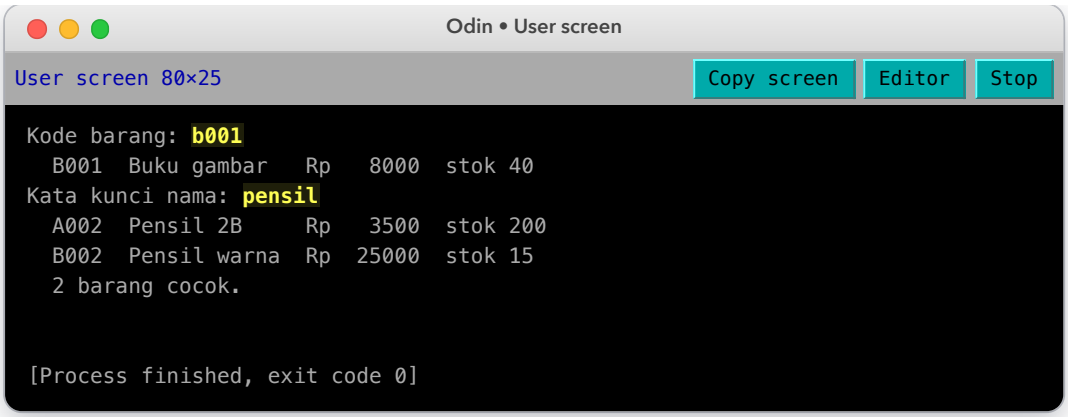
function Kapital(S: string): string;
var
  I: Integer;
begin
  for I := 1 to Length(S) do
    S[I] := UpCase(S[I]);                { S salinan, aslinya aman }
  Kapital := S;
end;

procedure Tampilkan(I: Integer);
begin
  with Gudang[I] do
    WriteLn('  ', Kode, '  ', Nama, '':14 - Length(Nama), 'Rp', Harga:7,
      '  stok ', Stok);
end;

var
  Kode, Kata: string;
  I, Posisi, Banyak: Integer;

begin
  { Cari menurut kode: berhenti begitu ketemu }
  Write('Kode barang: ');
  ReadLn(Kode);
  Kode := Kapital(Kode);
  Posisi := 0;
  I := 1;
  while (I <= N) and (Posisi = 0) do
  begin
    if Gudang[I].Kode = Kode then
      Posisi := I;
    I := I + 1;
  end;
  if Posisi > 0 then
    Tampilkan(Posisi)
  else
    WriteLn('  Kode ', Kode, ' tidak ditemukan.');

  { Cari menurut nama: tampilkan semua yang mengandung kata kunci }
  Write('Kata kunci nama: ');
  ReadLn(Kata);
  Banyak := 0;
  for I := 1 to N do
    if Pos(Kapital(Kata), Kapital(Gudang[I].Nama)) > 0 then
    begin
      Tampilkan(I);
      Banyak := Banyak + 1;
    end;
  WriteLn('  ', Banyak, ' barang cocok.');
end.
```

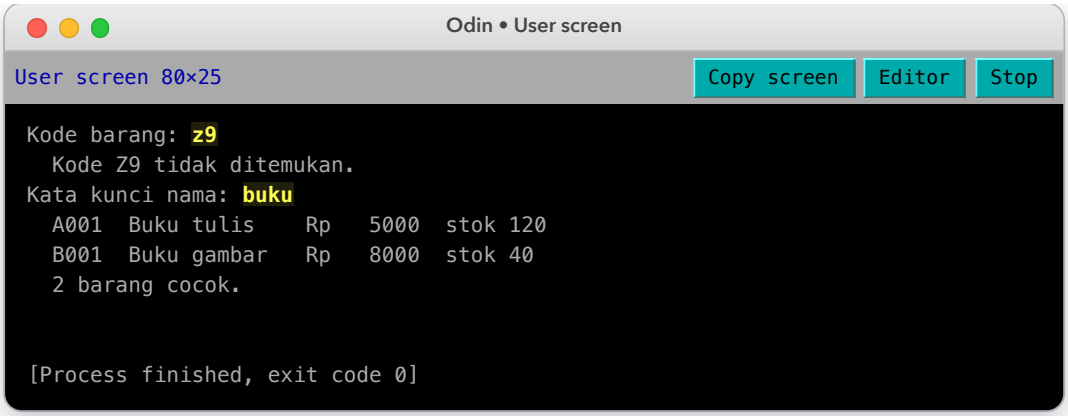


Gambar 15.14 Kode b001 (ditulis huruf kecil) tetap ditemukan; kata kunci pensil cocok dengan dua barang

Tabel 15.5 Penjelasan Program 15.3

Baris	Penjelasan
9–15	Tipe `TBarang` adalah *record* dengan empat field: `Kode` (maksimal 4 karakter), `Nama` (maksimal 20), `Harga` (`LongInt` karena bisa melebihi 32767), dan `Stok`.
18–25	Konstanta array enam `TBarang`. Setiap record ditulis dengan pasangan `Field: nilai` yang dipisah titik koma.
27–34	Fungsi `Kapital` mengubah semua huruf string menjadi kapital. Parameter `S` adalah salinan (parameter nilai), sehingga mengubahnya tidak merusak string aslinya.
36–41	Prosedur `Tampilkan` mencetak satu record. `with Gudang[I] do` membuat nama field bisa ditulis tanpa awalan. Ekspresi `':14 - Length(Nama)` mencetak string kosong selebar sisa kolom, sehingga kolom harga rata.
49–51	Membaca kode dan menyeragamkannya menjadi kapital, jadi `b001` sama dengan `B001`.
52–59	Pencarian beruntun menurut kode: `Posisi` terisi ketika `Gudang[I].Kode = Kode`, dan perulangan berhenti karena `Posisi = 0` tidak benar lagi.
60–63	Bila `Posisi > 0`, record ditampilkan; bila tidak, ada pesan tidak ditemukan.
66–75	Pencarian menurut nama memakai `for` penuh. Setiap record yang namanya memuat kata kunci (`Pos(...) > 0`) ditampilkan dan dihitung di `Banyak`.

Kode `b001` ditemukan sebagai "Buku gambar", sedangkan kata kunci `pensil` cocok dengan "Pensil 2B" dan "Pensil warna", sehingga laporan akhirnya `2 barang cocok.` Run berikut menunjukkan kode yang tidak ada, dan kata kunci yang memuat dua barang berbeda.



Gambar 15.15 Kode z9 tidak ada; kata kunci buku cocok dengan Buku tulis dan Buku gambar

COBA SENDIRI

Coba Sendiri. Tambahkan record ketujuh (jangan lupa mengubah `N` menjadi 7), lalu cari kodenya. Kemudian cari kata kunci `an` dan perhatikan barang mana saja yang cocok, termasuk yang kata itu berada di tengah nama.

15.6 Kesalahan yang Sering Terjadi

15.6.1 Lupa kurung pada syarat gabungan

Dalam Pascal, `and` memiliki prioritas lebih tinggi daripada operator pembanding. Syarat `I <= N and not Ketemu` tanpa kurung dibaca sebagai `I <= (N and not Ketemu)`, yaitu membandingkan bilangan dengan hasil operasi `and` antara bilangan dan `Boolean`, dan kompiler menolaknya.

kurung.pas

```
1  program Kurung;
2  const
3    N = 5;
4  var
5    I: Integer;
6    Ketemu: Boolean;
7  begin
8    I := 1;
9    Ketemu := False;
10   while I <= N and not Ketemu do
11     I := I + 1;
12 end.
```

PESAN KOMPILER

```
kurung.pas(10,14) Error: Tipe operand tidak cocok dengan operator
```

Gambar 15.16 Syarat tanpa kurung: and dihitung lebih dulu daripada <=

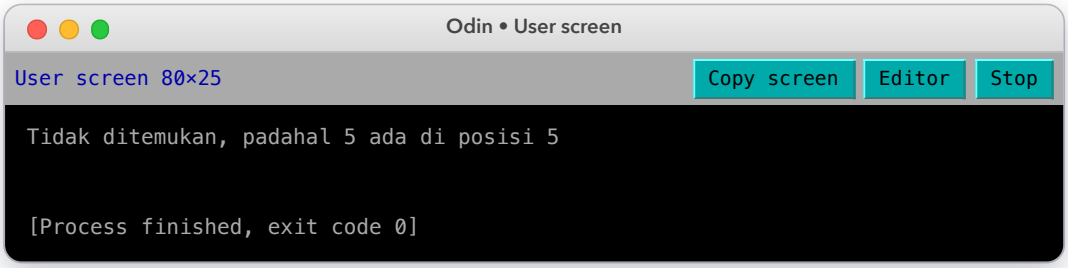
Perbaikannya: bungkus setiap pembanding dengan kurung, `(I <= N) and not Ketemu`, seperti baris 27 Program 15.1.

15.6.2 Pencarian biner pada data yang tidak terurut

Ini kesalahan logika, bukan kesalahan sintaks: program lolos kompilasi tetapi memberi jawaban salah. Data di bawah ini berisi 5 di posisi terakhir, tetapi tidak terurut.

takurut.pas

```
program TakUrut;
const
  N = 5;
  Data: array[1..N] of Integer = (9, 3, 7, 1, 5);
var
  Kiri, Kanan, Tengah, Posisi: Integer;
begin
  Kiri := 1;
  Kanan := N;
  Posisi := 0;
  while (Kiri <= Kanan) and (Posisi = 0) do
  begin
    Tengah := (Kiri + Kanan) div 2;
    if Data[Tengah] = 5 then
      Posisi := Tengah
    else if 5 < Data[Tengah] then
      Kanan := Tengah - 1
    else
      Kiri := Tengah + 1;
  end;
  if Posisi > 0 then
    WriteLn('Ditemukan di posisi ', Posisi)
  else
    WriteLn('Tidak ditemukan, padahal 5 ada di posisi 5');
end.
```



Gambar 15.17 5 ada di posisi 5, tetapi pencarian biner melaporkan tidak ditemukan

Langkah pertama membandingkan 5 dengan `Data[3]` = 7; karena 5 lebih kecil, separuh kanan (yang justru berisi 5) dibuang. Langkah kedua membandingkan dengan 9 dan membuang lagi, hingga rentang kosong. Pencarian biner percaya bahwa data terurut; bila kepercayaan itu salah, hasilnya salah tanpa peringatan apa pun. Urutkan dulu datanya, atau pakai pencarian beruntun.

15.6.3 Lupa memberi indeks pada array of record

Pada array of record, nama field tidak berdiri sendiri: ia milik satu elemen tertentu. Menulis `Gudang.Kode` tanpa `[I]` berarti meminta field milik seluruh array, dan kompiler menolak.

tanpaindeks.pas

```
1  program TanpaIndeks;
2  type
3    TBarang = record
4      Kode: string[4];
5      Harga: LongInt;
6    end;
7  var
8    Gudang: array[1..3] of TBarang;
9  begin
10   Gudang[1].Kode := 'A001';
11   if Gudang.Kode = 'A001' then
12     WriteLn('Ada');
13 end.
```

PESAN KOMPILER

```
tanpaindeks.pas(11,6) Error: Variabel record atau object diharapkan
```

Gambar 15.18 Gudang adalah array; field Kode hanya ada pada elemennya, misalnya Gudang[1].Kode

7 ATURAN TURBO PASCAL 7

Dua batasan Turbo Pascal 7 yang terlihat di bab ini: `Integer` hanya 16 bit (−32768..32767), sehingga harga barang memakai `LongInt`; dan string pendek `string[4]` memotong teks yang lebih panjang dari 4 karakter tanpa pemberitahuan. Selain itu, tidak ada fungsi bawaan untuk mencari tanpa membedakan huruf besar/kecil: itulah sebabnya Program 15.3 membuat fungsi `Kapital` sendiri.

15.7 Rangkuman

- Pencarian menemukan elemen yang cocok dengan **kunci pencarian**; hasilnya berupa posisi atau pernyataan bahwa data tidak ada.
- Pencarian beruntun memeriksa elemen satu per satu dengan `while (I <= N) and not Ketemu`; data tidak perlu terurut; paling banyak n perbandingan.
- Pada pencarian beruntun, `I` tidak ditambah ketika data ditemukan, sehingga `I` menunjuk posisi hasil.
- Pencarian biner memakai `Kiri`, `Kanan`, dan `Tengah := (Kiri + Kanan) div 2`, lalu membuang separuh rentang pada setiap langkah.
- Pencarian biner **hanya benar bila data terurut**; pada data tak terurut ia dapat melaporkan "tidak ada" untuk data yang ada.
- Jumlah langkah terburuk: beruntun n, biner $\lfloor \log_2 n \rfloor + 1$ (15 data: 4 langkah; sejuta data: 20 langkah).
- Pada array of record, satu field menjadi kunci dan hasilnya satu record utuh; kunci unik cukup satu hasil, kunci sebagian perlu memeriksa semua record.
- Selalu beri kurung pada syarat gabungan dan indeks pada elemen array of record.

15.8 Latihan

1. **DASAR** Jalankan Program 15.1 dengan kunci 41, 18, dan 100. Berapa perbandingan untuk masing-masing? Jelaskan mengapa jawabannya berbeda.

2. **DASAR** Pada data Program 15.2, tuliskan di buku catatan Anda urutan nilai `Kiri`, `Kanan`, dan `Tengah` untuk kunci 90, lalu cocokkan dengan hasil menjalankan program.

3. **DASAR** Berapa langkah terburuk pencarian biner untuk 255 data? Untuk 1.023 data? (Petunjuk: bagi dua berulang kali.)

4. **MENENGAH** Ubah Program 15.1 agar mencetak *berapa kali* kunci muncul, bukan daftar posisinya. Tambahkan variabel penghitung.

5. **MENENGAH** Ubah Program 15.2 agar array berisi 15 bilangan genap yang terurut, lalu cari bilangan ganjil. Berapa langkah yang dipakai sebelum program menyimpulkan data tidak ada?

6. **MENENGAH** Pada Program 15.3, tambahkan pencarian menurut `Stok`: tampilkan semua barang yang stoknya di bawah 50.

7. **TANTANGAN** Tulis fungsi `CariBiner(Kunci: Integer): Integer` yang mengembalikan posisi atau 0, lalu panggil dari program utama. Setelah itu tambahkan penghitung langkah global dan bandingkan dengan pencarian beruntun untuk setiap elemen array.

BAB 16

Pengurutan Data

TUJUAN PEMBELAJARAN

- Menjelaskan mengapa data perlu diurutkan dan membedakan urutan naik (*ascending*) dari urutan turun (*descending*).
- Menukar dua elemen dengan variabel bantu dan menjelaskan mengapa dua penugasan saja tidak cukup.
- Menulis dan menelusuri tiga algoritma pengurutan dasar: *bubble sort*, *selection sort*, dan *insertion sort*.
- Mengurutkan *array of record* menurut satu field dengan aturan pembanding sendiri, termasuk pemecah seri.
- Menghitung jumlah perbandingan dan pemindahan, lalu membandingkan ketiga algoritma pada data acak, terurut, dan terbalik.

PROGRAM DALAM BAB INI

16.1 `bab16_01_bubble_sort.pas` — Bubble sort

16.2 `bab16_02_selection_sort.pas` — Selection sort

16.3 `bab16_03_insertion_sort.pas` — Insertion sort

16.4 `bab16_04_urut_record.pas` — Mengurutkan array of record

16.5 `bab16_05_banding_jumlah_langkah.pas` — Membandingkan jumlah langkah

Daftar nilai yang tercetak dari yang tertinggi, kontak telepon yang tersusun menurut abjad, hasil pencarian yang diurutkan dari termurah: semuanya adalah hasil **pengurutan**. Bab ini mengajak Anda membongkar cara komputer menyusun data, dengan tiga algoritma klasik yang bisa dibayangkan seperti menyusun kartu atau botol. Anda akan menelusuri setiap putaran dengan mata, lalu menghitung berapa langkah yang dibutuhkan masing-masing. Algoritma di bab ini juga menyiapkan syarat pencarian biner pada Bab 15: data harus terurut.

16.1 Mengapa Data Perlu Diurutkan

Pengurutan (*sorting*) adalah menyusun ulang elemen menurut suatu aturan urutan. Bila elemen disusun dari kecil ke besar, urutannya **naik** (*ascending*); dari besar ke kecil, **turun** (*descending*). Untuk teks, urutan mengikuti abjad. Data terurut lebih enak dibaca, lebih mudah dicari (pencarian biner hanya bekerja pada data terurut), dan mempermudah pekerjaan lain seperti mencari nilai tertinggi, median, atau elemen kembar.

Pascal Turbo 7 tidak menyediakan perintah pengurutan bawaan, jadi Anda sendiri yang menulisnya. Ini justru kesempatan baik untuk belajar: tiga algoritma di bab ini hanya memakai perulangan, perbandingan, dan satu trik penukaran. Ketiganya dijalankan pada data yang sama, delapan bilangan **29, 10, 14, 37, 13, 5, 42, 21**, supaya cara kerjanya bisa dibandingkan langsung.

Data awal

29	10	14	37	13	5	42	21
[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

B Bubble sort

Bandingkan dua elemen bersebelahan dan tukar bila salah urut. Elemen terbesar "mengapung" ke ujung kanan.

S Selection sort

Pilih elemen terkecil dari bagian yang belum terurut, lalu tukar ke depan. Satu pertukaran per putaran.

I Insertion sort

Ambil satu elemen, geser yang lebih besar ke kanan, lalu sisipkan seperti menyusun kartu di tangan.

16.2 Menukar Dua Elemen

Ketiga algoritma bertumpu pada satu operasi: **menukar** isi dua tempat. Tampaknya sederhana, tetapi ada jebakan. Bila Anda menulis `A := B; B := A;`, nilai awal `A` tertimpa sebelum sempat disimpan, sehingga kedua variabel akhirnya berisi nilai `B`. Seperti menukar isi dua gelas, Anda memerlukan gelas ketiga sebagai tempat singgah: sebuah **variabel bantu** (*temporary variable*).

BENTUK UMUM

```
Bantu := A;       { 1. amankan nilai A }
A := B;           { 2. A boleh ditimpa }
B := Bantu;       { 3. B menerima nilai A yang lama }
```

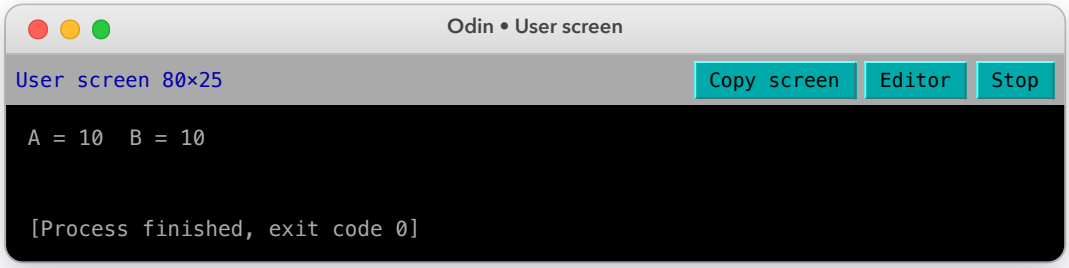


Gambar 16.1 Menukar dua variabel membutuhkan variabel bantu: tanpa itu, nilai pertama tertimpa dan hilang

Pada program bab ini variabel bantu bernama `Sementara` (atau `T`), dan elemen yang ditukar adalah `Data[I]` dengan `Data[I + 1]`. Bila Anda penasaran, kesalahan penukaran dua langkah mudah dibuktikan:

tukarsalah.pas

```
1  program TukarSalah;
2  var
3    A, B: Integer;
4  begin
5    A := 29;
6    B := 10;
7    A := B;
8    B := A;
9    WriteLn('A = ', A, '  B = ', B);
10 end.
```

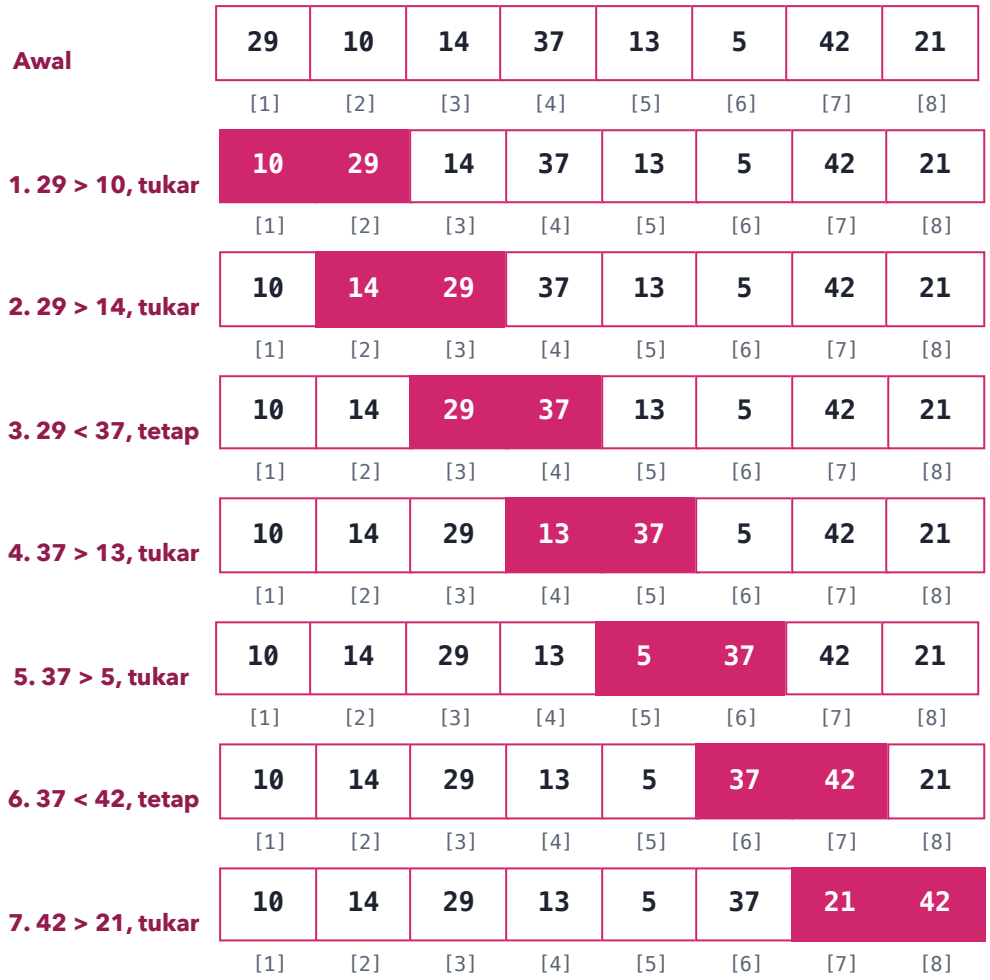


Gambar 16.2 Tanpa variabel bantu: kedua variabel berakhir bernilai 10

16.3 Bubble Sort

Nama *bubble sort* (pengurutan gelembung) berasal dari cara elemen besar bergerak: seperti gelembung udara di air, elemen terbesar naik ke ujung kanan putaran demi putaran. Dalam satu **putaran**, algoritma menyusuri array dari kiri dan membandingkan setiap **pasangan bersebelahan**. Bila pasangan itu salah urut (kiri lebih besar daripada kanan), keduanya ditukar. Setelah satu putaran penuh, elemen terbesar pasti sudah berada di posisi paling kanan. Putaran berikutnya tidak perlu menyentuhnya lagi.

Berikut putaran pertama pada data kita, perbandingan demi perbandingan. Sel berwarna pekat adalah pasangan yang sedang diperiksa, dengan hasil sesudah diperiksa.



Gambar 16.3 Putaran 1 bubble sort: tujuh perbandingan, lima pertukaran, dan 42 sampai di ujung kanan (angka 42 dibawa naik sejak perbandingan ke-7)

Perhatikan bahwa **42** baru dibandingkan pada langkah ke-7, sedangkan **37** terus terbawa naik dari langkah ke-4 sampai ke-6, karena setiap kali ia lebih besar daripada tetangga kanannya. Putaran berikutnya hanya perlu memeriksa sampai elemen ke-7, karena elemen ke-8 sudah final. Karena itu `for I := 1 to N - Putaran`: batas atasnya mengecil satu setiap putaran.

Bila suatu putaran tidak melakukan satu pun pertukaran, berarti semua pasangan sudah benar dan data sudah terurut, jadi perulangan boleh berhenti. Penanda `AdaTukar` (`Boolean`) mencatat apakah putaran itu menukar sesuatu.

BENTUK UMUM

```
repeat
  AdaTukar := False;
  Putaran := Putaran + 1;
  for I := 1 to N - Putaran do
    if Data[I] > Data[I + 1] then
    begin
      { tukar Data[I] dan Data[I + 1] dengan variabel bantu }
      AdaTukar := True;
    end;
until not AdaTukar;
```

16.3.1 Program 16.1: Bubble Sort Putaran demi Putaran

Program ini menyalin data awal ke array kerja `Data`, lalu mengurutkannya dengan bubble sort dan mencetak isi array sesudah **setiap putaran**. Buka `bab16_01_bubble_sort.pas` dari `File ▸ Examples...`.

Program 16.1 `bab16_01_bubble_sort.pas` *Bubble sort* File ▸ Examples...

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 16.1 — Bubble sort
  Bandingkan dua elemen bersebelahan, tukar bila urutannya salah.
  Setiap putaran membawa elemen terbesar ke ujung kanan, seperti
  gelembung yang naik. Bila satu putaran tanpa pertukaran, data
  sudah terurut dan proses boleh berhenti lebih awal. }

program bab16_01_bubble_sort;

const
  N = 8;
  Awal: array[1..N] of Integer = (29, 10, 14, 37, 13, 5, 42, 21);

type
  TData = array[1..N] of Integer;

procedure Tampilkan(Judul: string; D: TData);
var
  I: Integer;
begin
  Write(Judul);
  for I := 1 to N do
    Write(D[I]:4);
  WriteLn;
end;

var
  Data: TData;
  I, Putaran, Sementara: Integer;
  AdaTukar: Boolean;

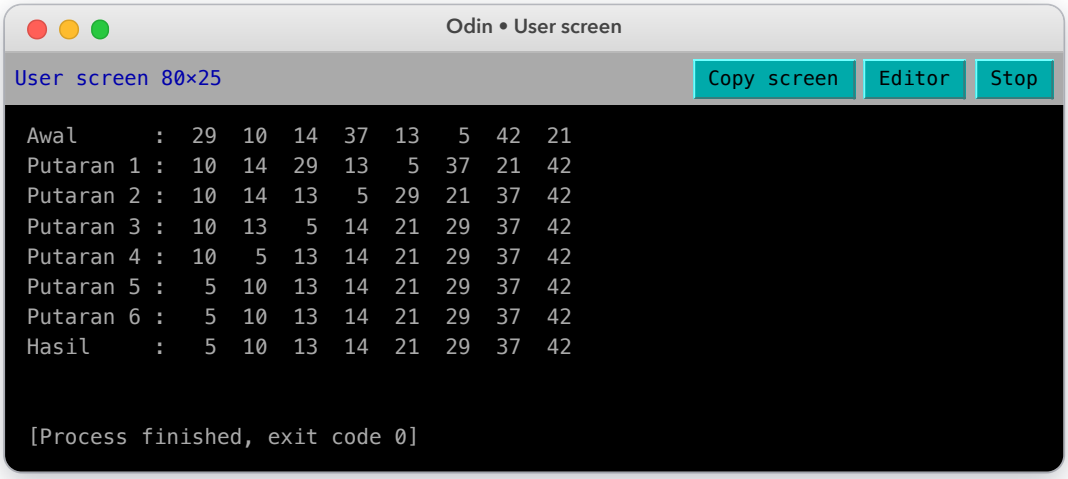
begin
  for I := 1 to N do
    Data[I] := Awal[I];                 { salin data awal }
  Tampilkan('Awal      :', Data);

  Putaran := 0;
  repeat
    AdaTukar := False;
    Putaran := Putaran + 1;
    for I := 1 to N - Putaran do        { ujung kanan sudah terurut }
      if Data[I] > Data[I + 1] then
      begin
        Sementara := Data[I];           { tukar dua elemen }
        Data[I] := Data[I + 1];
        Data[I + 1] := Sementara;
        AdaTukar := True;
      end;
    Write('Putaran ', Putaran, ' :');
    Tampilkan('', Data);
  until not AdaTukar;
```

```
52
53    Tampilkan('Hasil     :', Data);
54  end.
```

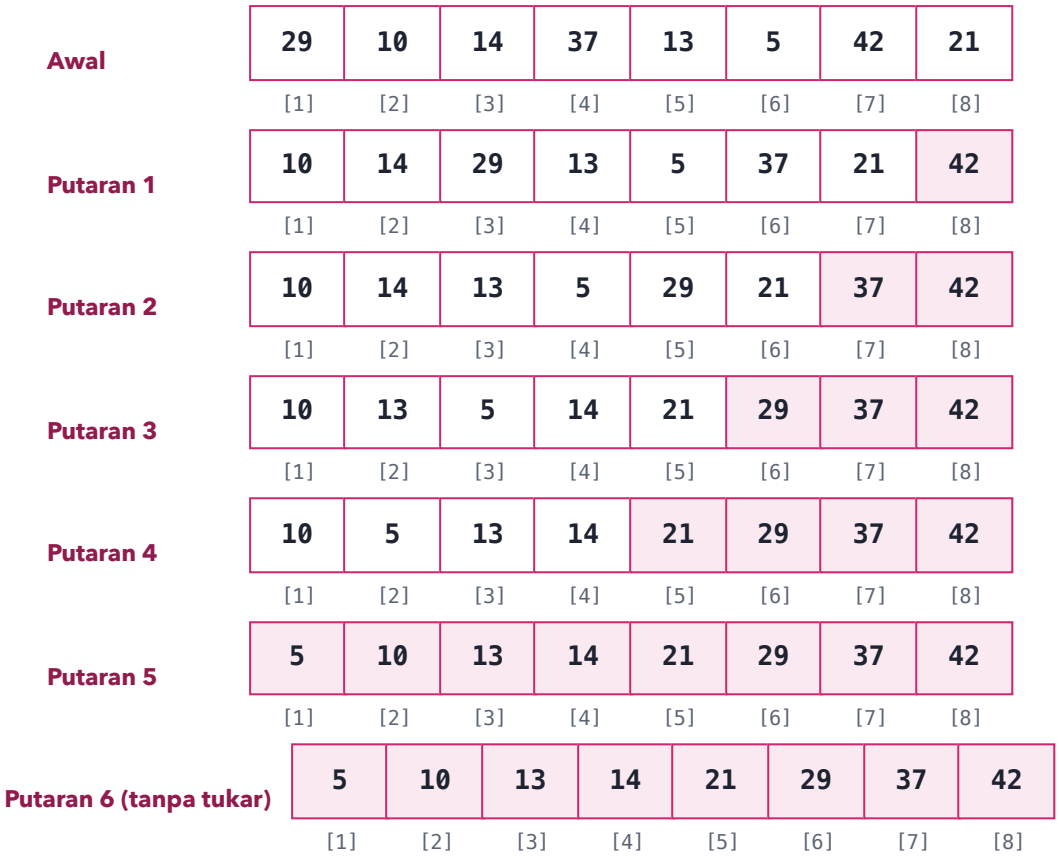
Tabel 16.1 Penjelasan Program 16.1

Baris	Penjelasan
10–12	`N` adalah banyak data, `Awal` menyimpan data yang tidak terurut sebagai konstanta (tetap tidak berubah selama program berjalan).
14–15	Tipe `TData` adalah array `1..N` berisi `Integer`. Tipe bernama ini diperlukan agar array dapat dipakai sebagai parameter prosedur.
17–25	Prosedur `Tampilkan` mencetak `Judul`, lalu semua elemen dengan lebar 4 kolom (`D[I]:4`) agar angka satu dan dua digit rata ke kanan. Parameter `D` adalah salinan, jadi array asli tidak berubah.
33–35	Menyalin `Awal` ke `Data` elemen demi elemen (karena `Awal` konstanta, yang diurutkan adalah salinannya), lalu mencetak keadaan awal.
37–40	Awal setiap putaran: `Putaran` naik satu, `AdaTukar` direset menjadi `False`.
41–42	Menyusuri pasangan `I` dan `I + 1` sampai `N - Putaran`, sehingga ujung kanan yang sudah terurut tidak disentuh lagi, dan memeriksa apakah pasangan itu salah urut.
43–48	Penukaran tiga langkah dengan `Sementara` (baris 44–46), lalu `AdaTukar := True`. Blok `begin..end` perlu karena ada empat pernyataan di bawah `if`.
49–50	Mencetak isi array setelah putaran: `Write('Putaran ', Putaran, ' :')` lalu `Tampilkan` dengan judul kosong.
51	`until not AdaTukar`: ulangi sampai ada satu putaran tanpa pertukaran.
53	Mencetak hasil akhir.



Gambar 16.4 Bubble sort pada delapan bilangan: enam putaran, yang terakhir tanpa pertukaran

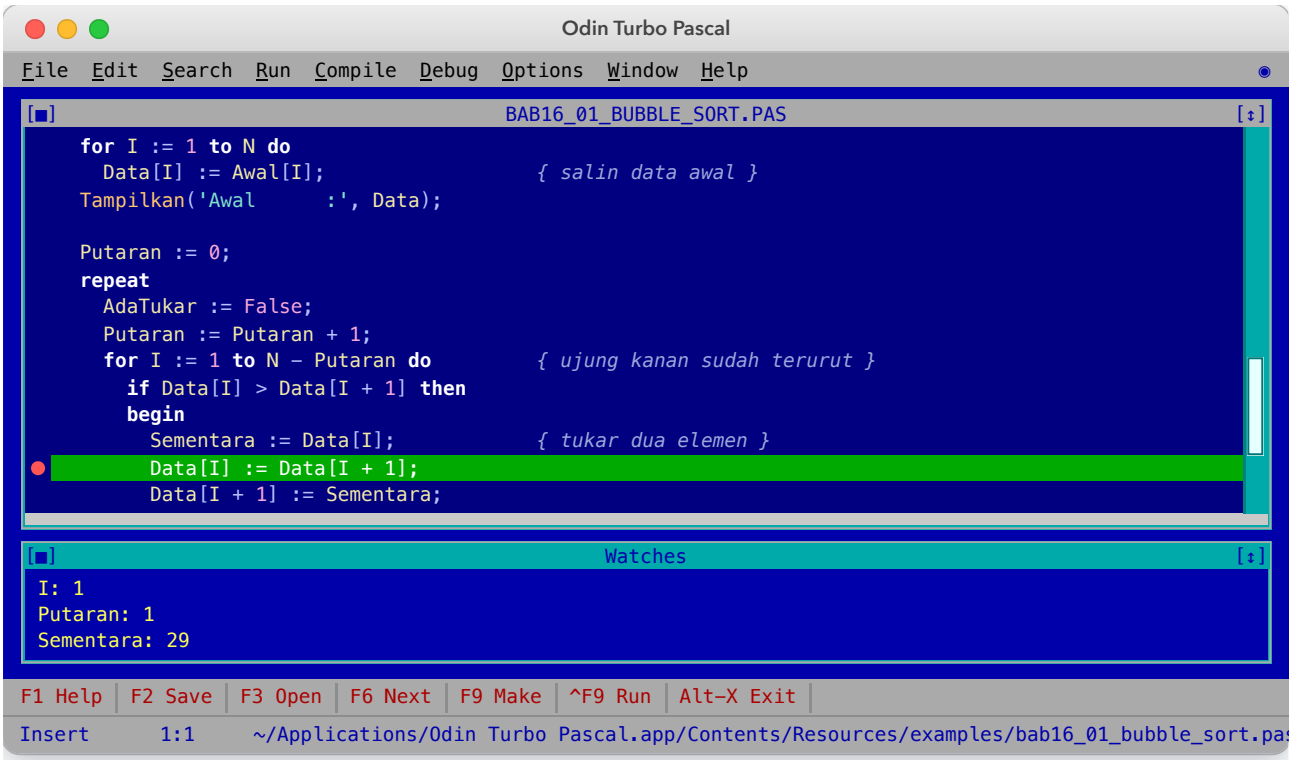
Bandingkan keluaran dengan gambar di atas: putaran 1 memang berakhir dengan `10 14 29 13 5 37 21 42`. Gambar berikut menyusun seluruh keluaran, dengan sel muda menandai bagian ujung kanan yang sudah menempati tempat akhirnya.



Gambar 16.5 Isi array sesudah tiap putaran bubble sort (sama dengan keluaran Program 16.1); sel muda adalah bagian yang sudah di tempat akhirnya

Putaran 5 menghasilkan array yang sudah terurut, tetapi program belum tahu itu: putaran 5 masih menukar (10 dan 5), jadi `AdaTukar` bernilai `True`. Hanya putaran 6, yang tidak menukar apa pun, yang memberi tahu program bahwa pekerjaan selesai. Itulah sebabnya keluaran menampilkan putaran 6 yang isinya sama dengan putaran 5.

Bubble sort juga dapat ditelusuri di debugger. Pasang titik henti pada baris 45 (`Data[I] := Data[I + 1]`), mulai lewat `Debug ▸ Start debugging`, dan pantau `I`, `Putaran`, dan `Sementara` di jendela Watches. Berikut keadaan saat penukaran pertama.



Gambar 16.6 Debugger berhenti di baris 45 pada penukaran pertama: Sementara sudah menyimpan 29, nilai Data[1] yang akan ditimpa

> **7** **ATURAN TURBO PASCAL 7**
>
> Di Turbo Pascal 7, parameter prosedur harus berupa **nama tipe**: Anda tidak dapat menulis `procedure Tampilkan(D: array[1..8] of Integer)` langsung di daftar parameter. Itulah mengapa program mendefinisikan `TData` lebih dulu dan memakai `D: TData`. Kebiasaan ini membuat program tetap berjalan di kompiler Pascal lain.

16.4 Selection Sort

Bayangkan Anda ingin mengurutkan sebelas kartu di meja dari yang terkecil. Cara lain selain menukar tetangga terus-menerus adalah: cari kartu terkecil di antara semua kartu, letakkan di tempat pertama; cari yang terkecil dari sisanya, letakkan di tempat kedua; dan seterusnya. Itulah *selection sort* (pengurutan seleksi). Pada **putaran ke-I**, algoritma memindai bagian `I..N` (yang belum terurut) untuk menemukan **posisi** elemen terkecil, lalu menukarnya dengan elemen ke-I. Setelah putaran ke-I, elemen `1..I` sudah final.

BENTUK UMUM

```
for I := 1 to N - 1 do
begin
  PosisiMin := I;                          { anggap elemen ke-I terkecil }
  for J := I + 1 to N do
    if Data[J] < Data[PosisiMin] then
      PosisiMin := J;                      { ketemu yang lebih kecil }
  if PosisiMin <> I then
    { tukar Data[I] dan Data[PosisiMin] }
end;
```

Algoritma ini menyimpan **indeks** elemen terkecil (`PosisiMin`), bukan nilainya, karena yang dibutuhkan nanti adalah *tempat* untuk bertukar. Hanya ada satu penukaran per putaran, dan tidak ada sama sekali bila elemen ke-I sudah yang terkecil.

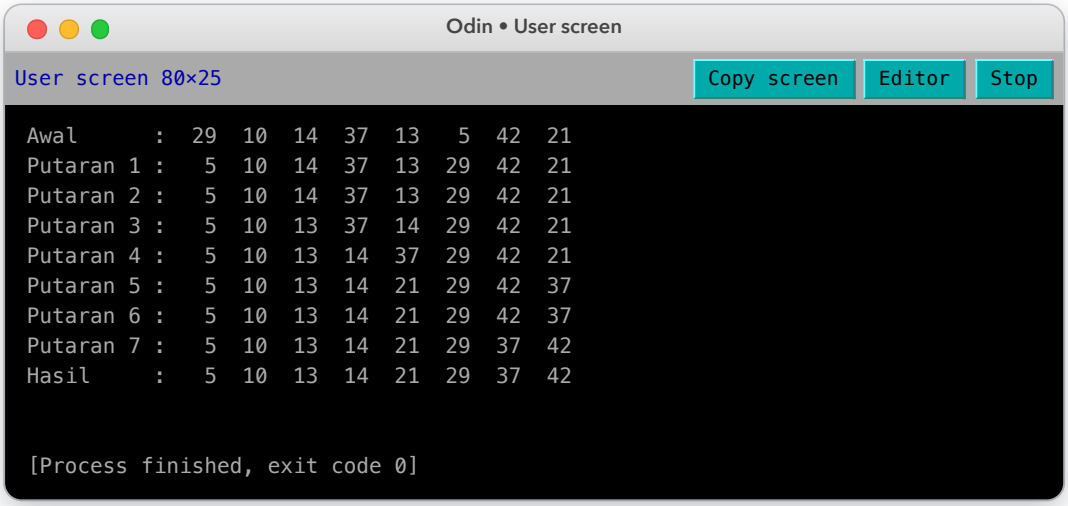
16.4.1 Program 16.2: Selection Sort

Buka `bab16_02_selection_sort.pas`. Data dan prosedur `Tampilkan` sama dengan Program 16.1, sehingga hasil kedua algoritma dapat dibandingkan baris demi baris.

Program 16.2 `bab16_02_selection_sort.pas` *Selection sort* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 16.2 — Selection sort
3    Pada putaran ke-I, cari elemen terkecil di bagian yang belum
4    terurut (I..N), lalu tukar dengan elemen ke-I. Hanya satu
5    pertukaran per putaran. }
6
7  program bab16_02_selection_sort;
8
9  const
10   N = 8;
11   Awal: array[1..N] of Integer = (29, 10, 14, 37, 13, 5, 42, 21);
12
13 type
14   TData = array[1..N] of Integer;
15
16 procedure Tampilkan(Judul: string; D: TData);
17 var
18   I: Integer;
19 begin
20   Write(Judul);
21   for I := 1 to N do
22     Write(D[I]:4);
23   WriteLn;
24 end;
25
26 var
27   Data: TData;
28   I, J, PosisiMin, Sementara: Integer;
29
30 begin
31   for I := 1 to N do
32     Data[I] := Awal[I];
```

```
33    Tampilkan('Awal      :', Data);
34
35    for I := 1 to N - 1 do
36    begin
37      PosisiMin := I;                    { anggap elemen ke-I terkecil }
38      for J := I + 1 to N do
39        if Data[J] < Data[PosisiMin] then
40          PosisiMin := J;                { ketemu yang lebih kecil }
41      if PosisiMin <> I then
42      begin
43        Sementara := Data[I];
44        Data[I] := Data[PosisiMin];
45        Data[PosisiMin] := Sementara;
46      end;
47      Write('Putaran ', I, ' :');
48      Tampilkan('', Data);
49    end;
50
51    Tampilkan('Hasil     :', Data);
52  end.
```

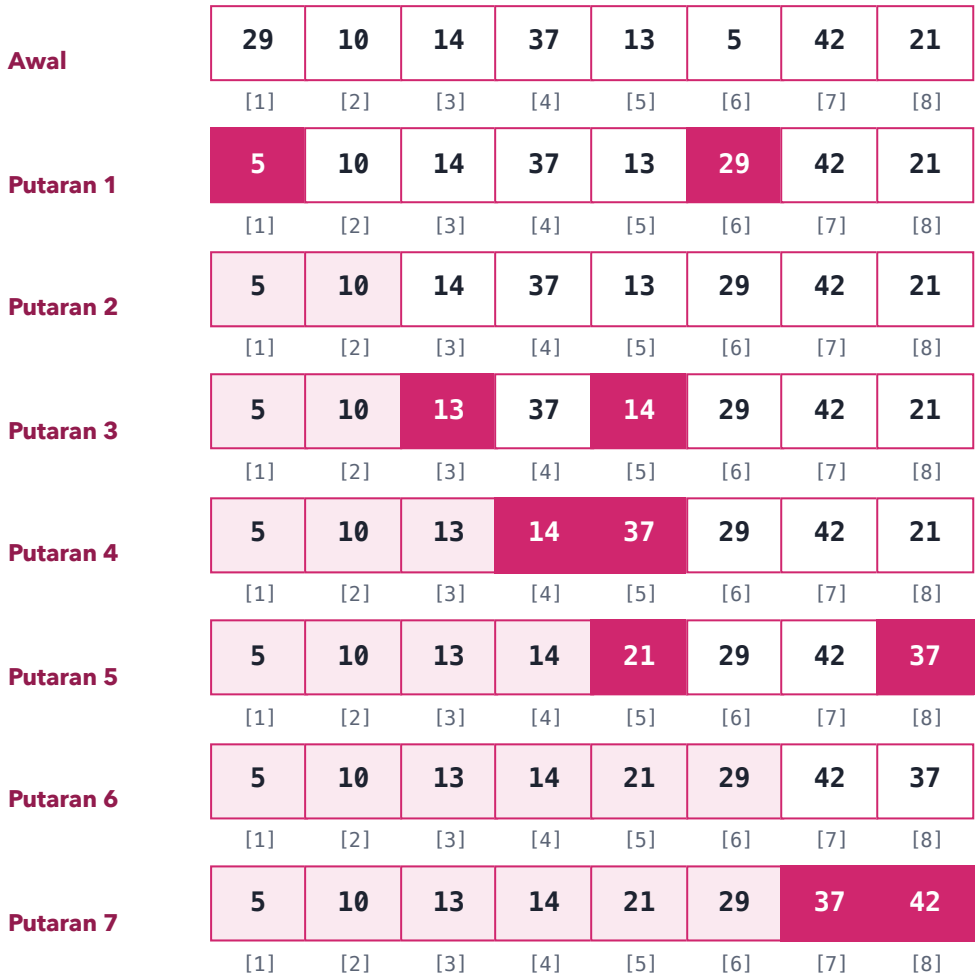


Gambar 16.7 Selection sort: tujuh putaran, putaran ke-2 dan ke-6 tanpa pertukaran

Tabel 16.2 Penjelasan Program 16.2

Baris	Penjelasan
26–28	Dua penunjuk perulangan `I` (batas bagian terurut) dan `J` (pemindai), `PosisiMin` (indeks terkecil sementara), dan `Sementara` untuk penukaran.
31–33	Menyalin data awal dan mencetaknya, sama seperti Program 16.1.
35	Putaran luar `I` berjalan dari 1 sampai `N - 1`: elemen terakhir otomatis benar bila yang lain sudah benar.
37	Mulai dengan anggapan bahwa elemen ke-I adalah yang terkecil.
38–40	Putaran dalam memindai `I + 1` sampai `N`. Setiap elemen yang lebih kecil daripada `Data[PosisiMin]` menjadi calon baru.
41–46	Tukar hanya bila `PosisiMin <> I`, yaitu bila ada yang lebih kecil daripada elemen ke-I.
47–48	Mencetak isi array sesudah putaran.

Selection sort selalu berjalan tepat `N - 1` putaran, tujuh untuk data kita. Berikut pergerakannya: sel pekat adalah dua posisi yang bertukar, sel muda adalah bagian kiri yang sudah final.



Gambar 16.8 Putaran selection sort pada Program 16.2. Putaran 2 dan 6 tidak menukar karena elemen ke-I sudah terkecil di bagian sisa

Putaran 1 menemukan 5 (di posisi 6) dan menukarnya dengan 29 di posisi 1, sehingga 29 berpindah ke posisi 6. Putaran 2 menemukan bahwa 10 di posisi 2 sudah yang terkecil di antara sisa elemen, sehingga tidak ada pertukaran. Total pertukaran pada data ini adalah lima (putaran 1, 3, 4, 5, dan 7), jauh lebih sedikit daripada bubble sort. Perbandingan pemindai tetap banyak, seperti terlihat di bagian perbandingan langkah nanti.

COBA SENDIRI

Coba Sendiri. Ubah baris 39 dari `<` menjadi `>` lalu jalankan. Apa yang terjadi pada urutan hasilnya? Mengapa satu tanda perbandingan cukup untuk membalik arah urutan?

16.5 Insertion Sort

Saat bermain kartu, kebanyakan orang menyusun kartu di tangan seperti ini: ambil kartu baru dari meja, bandingkan dari kanan ke kiri dengan kartu yang sudah tersusun, **geser** kartu yang lebih besar ke kanan, lalu **sisipkan** kartu baru di celah yang terbuka. Itulah *insertion sort* (pengurutan sisip). Bagian kiri array

selalu terurut, dan setiap putaran memperbesar bagian itu satu elemen. Putaran dimulai dari `I = 2`, karena satu elemen pertama dengan sendirinya sudah terurut.

BENTUK UMUM

```
for I := 2 to N do
begin
  Kunci := Data[I];                    { kartu yang akan disisipkan }
  J := I - 1;
  while (J >= 1) and (Data[J] > Kunci) do
  begin
    Data[J + 1] := Data[J];            { geser ke kanan }
    J := J - 1;
  end;
  Data[J + 1] := Kunci;                { sisipkan di tempat kosong }
end;
```

Di sini tidak ada penukaran: elemen digeser satu per satu menimpa tempat yang kosong, dan `Kunci` yang tadinya diambil (disimpan di variabel) diletakkan di akhir. Gambar berikut menunjukkan satu putaran lengkap, yaitu menyisipkan 13.



Gambar 16.9 Satu langkah insertion sort (I = 5, Kunci = 13): ambil, geser elemen yang lebih besar, sisipkan di tempat kosong

16.5.1 Program 16.3: Insertion Sort

Buka `bab16_03_insertion_sort.pas`. Program mencetak isi array setelah setiap penyisipan, dan label tiap baris menyebut nilai `Kunci` yang baru disisipkan.

Program 16.3 `bab16_03_insertion_sort.pas` *Insertion sort* File ▸ Examples…

```
1  { Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 16.3 - Insertion sort
3    Seperti menyusun kartu di tangan: ambil elemen berikutnya, geser
4    elemen yang lebih besar ke kanan, lalu sisipkan elemen itu di
5    tempat yang kosong. Bagian kiri selalu sudah terurut. }
6
7  program bab16_03_insertion_sort;
8
9  const
```

```
10    N = 8;
11    Awal: array[1..N] of Integer = (29, 10, 14, 37, 13, 5, 42, 21);
12
13  type
14    TData = array[1..N] of Integer;
15
16  procedure Tampilkan(Judul: string; D: TData);
17  var
18    I: Integer;
19  begin
20    Write(Judul);
21    for I := 1 to N do
22      Write(D[I]:4);
23    WriteLn;
24  end;
25
26  var
27    Data: TData;
28    I, J, Kunci: Integer;
29
30  begin
31    for I := 1 to N do
32      Data[I] := Awal[I];
33    Tampilkan('Awal      :', Data);
34
35    for I := 2 to N do
36    begin
37      Kunci := Data[I];                    { kartu yang akan disisipkan }
38      J := I - 1;
39      while (J >= 1) and (Data[J] > Kunci) do
40      begin
41        Data[J + 1] := Data[J];            { geser ke kanan }
42        J := J - 1;
43      end;
44      Data[J + 1] := Kunci;                { sisipkan di tempat kosong }
45      Write('Sisip ', Kunci:2, '  :');
46      Tampilkan('', Data);
47    end;
48
49    Tampilkan('Hasil     :', Data);
50  end.
```

Odin • User screen

User screen 80×25 | Copy screen | Editor | Stop

```
Awal      :  29  10  14  37  13   5  42  21
Sisip 10  :  10  29  14  37  13   5  42  21
Sisip 14  :  10  14  29  37  13   5  42  21
Sisip 37  :  10  14  29  37  13   5  42  21
Sisip 13  :  10  13  14  29  37   5  42  21
Sisip  5  :   5  10  13  14  29  37  42  21
Sisip 42  :   5  10  13  14  29  37  42  21
Sisip 21  :   5  10  13  14  21  29  37  42
Hasil     :   5  10  13  14  21  29  37  42

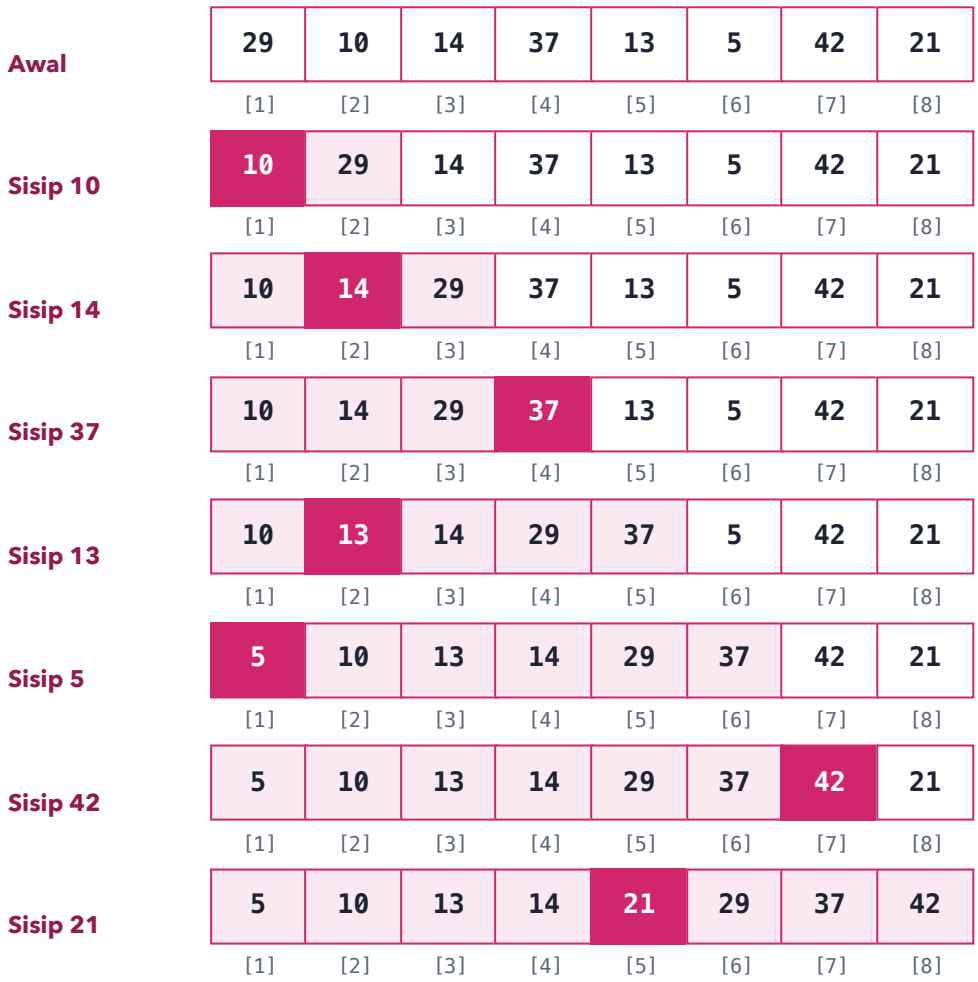

[Process finished, exit code 0]
```

Gambar 16.10 Insertion sort: tujuh penyisipan, dari Kunci 10 sampai Kunci 21

Tabel 16.3 Penjelasan Program 16.3

Baris	Penjelasan
28	`Kunci` menyimpan elemen yang sedang disisipkan, `J` menunjuk elemen yang diperiksa di bagian kiri.
35	Mulai dari elemen kedua: elemen pertama sendiri sudah "terurut".
37–38	Mengambil `Kunci := Data[I]`, lalu mulai membandingkan dari elemen tepat di kiri.
39–43	Selama masih ada elemen di kiri (`J >= 1`) dan elemen itu lebih besar daripada `Kunci`, geser elemen itu satu tempat ke kanan dan mundur satu langkah.
44	Tempat kosong ada di `J + 1`; di sanalah `Kunci` diletakkan.
45–46	Mencetak keadaan array, dengan `Kunci:2` agar nilai satu digit tidak menggeser kolom.

Kondisi `(J >= 1) and (Data[J] > Kunci)` mempunyai dua alasan berhenti: bertemu elemen yang tidak lebih besar (tempat sudah ketemu), atau mencapai awal array (`J = 0`, `Kunci` adalah yang terkecil dan menempati posisi 1). Penyisipan 5 menunjukkan kasus kedua: 5 menggeser semua elemen di kirinya dan menempati posisi pertama. Penyisipan 37 dan 42 menunjukkan kebalikannya: keduanya sudah lebih besar daripada semua elemen di kiri, sehingga tidak ada yang digeser.



Gambar 16.11 Isi array sesudah tiap penyisipan (Program 16.3): sel pekat adalah Kunci yang baru disisipkan, sel muda bagian kiri yang terurut

16.6 Mengurutkan Array of Record

Data nyata jarang berupa bilangan polos. Daftar siswa berisi nama *dan* nilai. Saat mengurutkan, ada dua aturan emas: **yang dibandingkan** hanya satu field (atau kombinasi aturan), tetapi **yang dipindahkan** adalah record utuh. Bila hanya field `Nilai` yang dipindahkan, nilai berpisah dari pemiliknya.

terpisah.pas

```
program Terpisah;
const
  N = 3;
  Nama: array[1..N] of string[8] = ('Dimas', 'Ayu', 'Fajar');
var
  Nilai: array[1..N] of Integer;
  I, J, T: Integer;
begin
  Nilai[1] := 78;
  Nilai[2] := 92;
  Nilai[3] := 85;
  for I := 1 to N - 1 do
    for J := 1 to N - I do
      if Nilai[J] < Nilai[J + 1] then
      begin
        T := Nilai[J];
        Nilai[J] := Nilai[J + 1];
        Nilai[J + 1] := T;
      end;
  for I := 1 to N do
    WriteLn(Nama[I], ' ', Nilai[I]);
end.
```



Gambar 16.12 Hanya nilai yang diurutkan: Dimas tiba-tiba mendapat 92, padahal nilai aslinya 78

Solusinya: simpan semua field dalam satu *record* (Bab 13), urutkan array of record itu, dan tukar atau geser **record utuh**. Penugasan `Siswa[J + 1] := Siswa[J]` menyalin seluruh field sekaligus.

Aturan urutan yang lebih rumit pun mudah ditulis bila dikemas dalam sebuah fungsi pembanding. Program 16.4 mendefinisikan `LebihDulu(A, B)`, yang bernilai `True` bila `A` harus berada di depan `B`, dengan dua aturan: nilai yang lebih tinggi di depan, dan bila nilai sama, urut abjad menurut nama. Aturan kedua disebut **pemecah seri** (*tie-breaker*).

✓ Pindahkan record utuh

`Siswa[J + 1] := Siswa[J];` Nama dan nilai selalu berpindah bersama.

✗ Pindahkan satu field

`Nilai[J + 1] := Nilai[J];` Nama tinggal di tempatnya, dan pasangan nama-nilai menjadi salah.

16.6.1 Program 16.4: Peringkat Siswa

Buka `bab16_04_urut_record.pas`. Program mengurutkan tujuh siswa dengan insertion sort (Program 16.3) memakai `LebihDulu` sebagai pembanding, lalu mencetak daftar peringkat. Dua pasang siswa mempunyai nilai yang sama, 92 dan 85, sehingga program harus menangani peringkat kembar.

Program 16.4 `bab16_04_urut_record.pas` *Mengurutkan array of record* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 16.4 — Mengurutkan array of record
  Yang dibandingkan adalah satu field (misalnya Nilai), tetapi yang
  dipindahkan adalah record utuh, supaya nama tetap bersama nilainya.
  Urutan: nilai tertinggi dulu; bila nilainya sama, urut nama A–Z. }

program bab16_04_urut_record;

type
  TSiswa = record
    Nama: string[15];
    Nilai: Integer;
  end;

const
  N = 7;
  Awal: array[1..N] of TSiswa = (
    (Nama: 'Dimas'; Nilai: 78), (Nama: 'Ayu';   Nilai: 92),
    (Nama: 'Fajar'; Nilai: 85), (Nama: 'Citra'; Nilai: 92),
    (Nama: 'Bayu';  Nilai: 70), (Nama: 'Eka';   Nilai: 85),
    (Nama: 'Gita';  Nilai: 88));

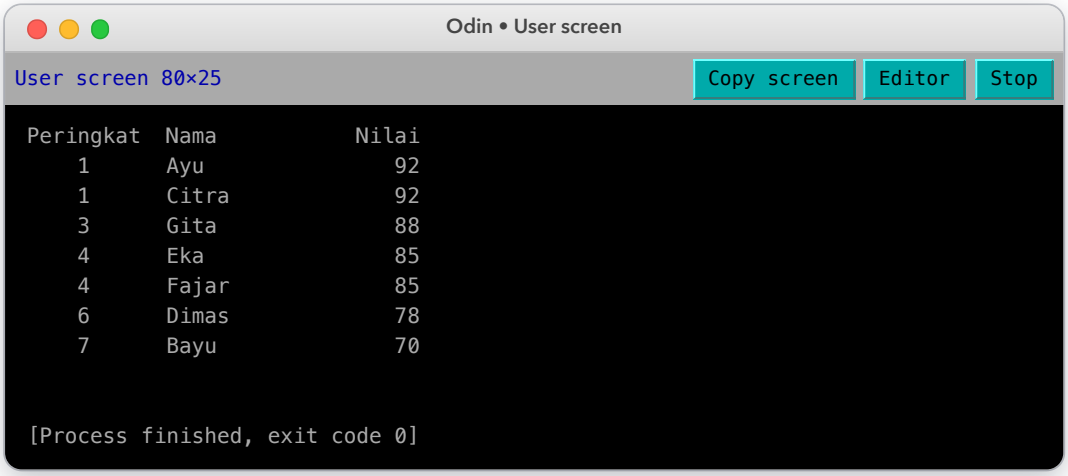
{ True bila A harus berada di depan B }
function LebihDulu(A, B: TSiswa): Boolean;
begin
  if A.Nilai <> B.Nilai then
    LebihDulu := A.Nilai > B.Nilai       { nilai besar di depan }
  else
    LebihDulu := A.Nama < B.Nama;        { nilai sama: urut abjad }
end;

var
  Siswa: array[1..N] of TSiswa;
  Kunci: TSiswa;
  I, J, Peringkat: Integer;

begin
  for I := 1 to N do
    Siswa[I] := Awal[I];                 { salin record utuh }

  { Insertion sort dari Bab 16.3, memakai LebihDulu sebagai pembanding }
  for I := 2 to N do
  begin
```

```
44      Kunci := Siswa[I];
45      J := I - 1;
46      while (J >= 1) and LebihDulu(Kunci, Siswa[J]) do
47      begin
48        Siswa[J + 1] := Siswa[J];
49        J := J - 1;
50      end;
51      Siswa[J + 1] := Kunci;
52    end;
53
54    WriteLn('Peringkat  Nama           Nilai');
55    for I := 1 to N do
56    begin
57      { Nilai sama mendapat peringkat yang sama }
58      if (I = 1) or (Siswa[I].Nilai <> Siswa[I - 1].Nilai) then
59        Peringkat := I;
60      WriteLn(Peringkat:5, '      ', Siswa[I].Nama,
61        '':15 - Length(Siswa[I].Nama), Siswa[I].Nilai:5);
62    end;
63  end.
```



Gambar 16.13 Daftar peringkat: nilai sama mendapat peringkat yang sama, urut abjad di dalamnya

Tabel 16.4 Penjelasan Program 16.4

Baris	Penjelasan
9–13	Tipe `TSiswa`: nama (maksimal 15 karakter) dan nilai.
15–21	Konstanta `Awal` berisi tujuh siswa dalam urutan acak: Dimas 78, Ayu 92, Fajar 85, Citra 92, Bayu 70, Eka 85, Gita 88.
23–30	Fungsi `LebihDulu`. Bila nilai berbeda, nilai yang lebih besar di depan. Bila nilai sama, nama yang lebih kecil menurut abjad (`A.Nama < B.Nama`) di depan.
33–35	Array kerja `Siswa`, dan variabel `Kunci` bertipe `TSiswa` untuk menyimpan satu record yang sedang disisipkan.
38–39	Menyalin tiap record utuh dengan satu penugasan.
41–52	Insertion sort dengan syarat `LebihDulu(Kunci, Siswa[J])`: `Kunci` terus bergeser ke kiri selama ia harus lebih dulu daripada elemen di kirinya.

Baris	Penjelasan
54–62	Mencetak daftar peringkat. Peringkat naik ke posisi `I` hanya bila nilai berbeda dari elemen sebelumnya (baris 58–59); bila sama, peringkat sebelumnya dipertahankan.

Ayu dan Citra sama-sama 92, dan Ayu mendahului Citra karena "Ayu" lebih awal menurut abjad; keduanya mendapat peringkat 1. Peringkat berikutnya adalah 3, bukan 2, karena dua siswa sudah menempati posisi 1 dan 2: itu cara peringkat kembar biasa dihitung. Eka dan Fajar sama-sama 85 dan berperingkat 4, dengan Eka (E) lebih dulu daripada Fajar (F). Dimas peringkat 6 dan Bayu peringkat 7.

TIPS

Memisahkan **aturan urutan** ke dalam fungsi `LebihDulu` membuat algoritma pengurutannya tidak perlu diubah bila aturannya berganti. Ingin urutan menurut nama saja? Cukup ganti isi fungsi. Dalam proyek besar, teknik ini lazim dipakai.

16.7 Membandingkan Jumlah Langkah

Ketiga algoritma menghasilkan hasil akhir yang sama, tetapi jumlah pekerjaannya berbeda. Ada dua macam pekerjaan yang bisa dihitung: **perbandingan** (berapa kali dua elemen dibandingkan) dan **pemindahan** (berapa kali elemen ditukar atau digeser). Program 16.5 menjalankan ketiganya pada 20 data dalam tiga keadaan, yaitu **acak**, **sudah terurut**, dan **terurut terbalik**, lalu mencatat kedua hitungan lewat dua variabel global `Banding` dan `Pindah`.

Program ini lebih panjang karena berisi tiga prosedur pengurutan, satu prosedur `Tukar`, dan satu prosedur `SiapkanData`. Buka `bab16_05_banding_jumlah_langkah.pas` dan bacalah per bagian.

Program 16.5 `bab16_05_banding_jumlah_langkah.pas` *Membandingkan jumlah langkah* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 16.5 — Membandingkan jumlah langkah
3    Tiga algoritma pengurutan dijalankan pada data yang sama, lalu
4    dihitung perbandingan dan pemindahannya. Dicoba tiga keadaan data:
5    acak, sudah terurut, dan terurut terbalik. }
6
7  program bab16_05_banding_jumlah_langkah;
8
9  const
10   N = 20;
11   Acak: array[1..N] of Integer = (57, 12, 89, 3, 44, 71, 25, 96, 38, 60,
12                                   18, 83, 7, 52, 66, 30, 91, 14, 77, 41);
13
14 type
15   TData = array[1..N] of Integer;
16
17 var
18   Banding, Pindah: LongInt;              { penghitung langkah }
19
20 procedure Tukar(var A, B: Integer);
21 var
22   T: Integer;
23 begin
```

```
  T := A; A := B; B := T;
  Pindah := Pindah + 1;
end;

procedure BubbleSort(var D: TData);
var
  I, Putaran: Integer;
  AdaTukar: Boolean;
begin
  Putaran := 0;
  repeat
    AdaTukar := False;
    Putaran := Putaran + 1;
    for I := 1 to N - Putaran do
    begin
      Banding := Banding + 1;
      if D[I] > D[I + 1] then
      begin
        Tukar(D[I], D[I + 1]);
        AdaTukar := True;
      end;
    end;
  until not AdaTukar;
end;

procedure SelectionSort(var D: TData);
var
  I, J, PosisiMin: Integer;
begin
  for I := 1 to N - 1 do
  begin
    PosisiMin := I;
    for J := I + 1 to N do
    begin
      Banding := Banding + 1;
      if D[J] < D[PosisiMin] then
        PosisiMin := J;
    end;
    if PosisiMin <> I then
      Tukar(D[I], D[PosisiMin]);
  end;
end;

procedure InsertionSort(var D: TData);
var
  I, J, Kunci: Integer;
  Lanjut: Boolean;
begin
  for I := 2 to N do
  begin
    Kunci := D[I];
    J := I - 1;
    Lanjut := True;
    while (J >= 1) and Lanjut do
    begin
      Banding := Banding + 1;
      if D[J] > Kunci then
      begin
        D[J + 1] := D[J];
```

```
83          Pindah := Pindah + 1;
84          J := J - 1;
85        end
86        else
87          Lanjut := False;
88      end;
89      D[J + 1] := Kunci;
90    end;
91  end;
92
93  { Menyiapkan data: 1 = acak, 2 = terurut, 3 = terbalik }
94  procedure SiapkanData(Jenis: Integer; var D: TData);
95  var
96    I: Integer;
97  begin
98    for I := 1 to N do
99      case Jenis of
100       1: D[I] := Acak[I];
101       2: D[I] := I * 5;
102       3: D[I] := (N - I + 1) * 5;
103     end;
104 end;
105
106 var
107   D: TData;
108   Algoritma, Jenis: Integer;
109
110 begin
111   WriteLn('Jumlah data: ', N);
112   WriteLn('Isi kolom: jumlah perbandingan / jumlah pemindahan');
113   WriteLn('Algoritma        Acak    Terurut   Terbalik');
114   for Algoritma := 1 to 3 do
115   begin
116     case Algoritma of
117       1: Write('Bubble sort   ');
118       2: Write('Selection sort');
119       3: Write('Insertion sort');
120     end;
121     for Jenis := 1 to 3 do
122     begin
123       SiapkanData(Jenis, D);
124       Banding := 0;
125       Pindah := 0;
126       case Algoritma of
127         1: BubbleSort(D);
128         2: SelectionSort(D);
129         3: InsertionSort(D);
130       end;
131       Write(Banding:5, '/', Pindah:4);
132     end;
133     WriteLn;
134   end;
135   WriteLn;
136   WriteLn('Perhatikan: pada data terurut, bubble dan insertion sort');
137   WriteLn('selesai sangat cepat, selection sort tetap membandingkan semua.');
138 end.
```

Tabel 16.5 Penjelasan Program 16.5

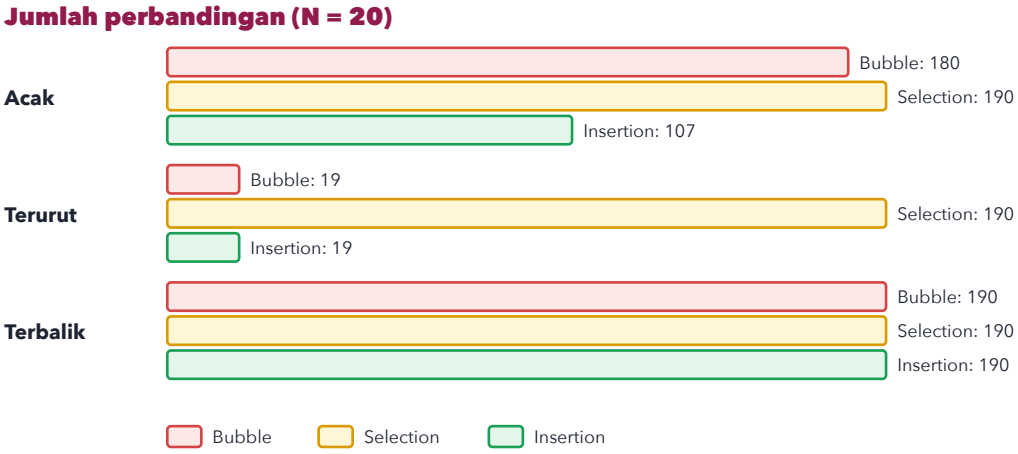
Baris	Penjelasan
9–15	`N = 20` dan satu set data `Acak` tetap supaya hasil selalu sama. `TData` adalah array 20 `Integer`.
18	`Banding` dan `Pindah` bertipe `LongInt` : variabel global yang dinaikkan oleh prosedur pengurutan.
20–26	`Tukar(var A, B)` menukar dua variabel lewat parameter `var` (Bab 9) dan sekaligus menaikkan `Pindah`.
28–47	`BubbleSort` : sama dengan Program 16.1, hanya setiap perbandingan menaikkan `Banding` dan setiap penukaran memanggil `Tukar`.
49–65	`SelectionSort` : `Banding` naik pada setiap perbandingan pemindai (baris 58), dan `Tukar` hanya dipanggil bila perlu (baris 62–63).
67–91	`InsertionSort` : bentuk `while` dengan penanda `Lanjut` agar `Banding` dihitung tepat satu kali per pemeriksaan elemen (baris 79). Setiap geseran menaikkan `Pindah` (baris 83).
93–104	`SiapkanData` mengisi `D` sesuai `Jenis` dengan `case` : 1 acak, 2 terurut (`I * 5` : 5, 10, ..., 100), 3 terbalik.
114–134	Tiga algoritma dalam `case` di dalam tiga keadaan data. Sebelum setiap pengurutan, hitungan direset menjadi 0, lalu hasil dicetak dengan `Write(Banding:5, '/', Pindah:4)`.



Gambar 16.14 Perbandingan / pemindahan untuk 20 data: tiga algoritma, tiga keadaan data

Tabel 16.6 Hasil Program 16.5 (perbandingan / pemindahan, N = 20)

Algoritma	Acak	Terurut	Terbalik
Bubble sort	180 / 90	19 / 0	190 / 190
Selection sort	190 / 18	190 / 0	190 / 10
Insertion sort	107 / 90	19 / 0	190 / 190



Gambar 16.15 Perbandingan hasil Program 16.5: selection sort selalu 190, bubble dan insertion sort sangat cepat pada data yang sudah terurut

Beberapa hal yang dapat dibaca dari tabel dan grafik:

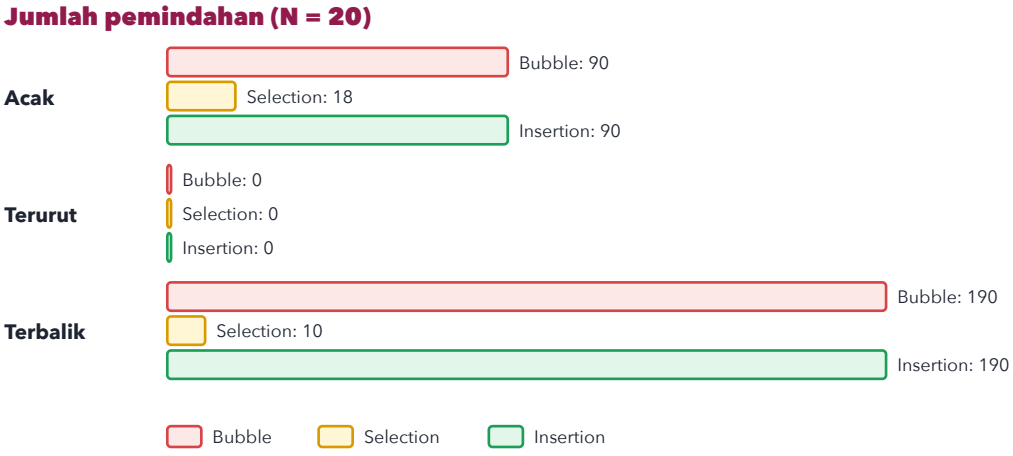
1. **Selection sort tidak peduli keadaan data.** Perbandingannya selalu 190, karena pemindai tetap memeriksa seluruh sisa array di setiap putaran. Nilai 190 sama dengan 20 × 19 / 2, yaitu jumlah semua pasangan elemen.

2. **Data terurut adalah impian bubble dan insertion sort.** Keduanya hanya 19 perbandingan (N − 1) dan nol pemindahan: bubble sort melihat tidak ada pertukaran pada putaran pertama lalu berhenti, insertion sort mendapati tiap elemen sudah lebih besar dari elemen di kirinya.

3. **Data terbalik adalah mimpi buruk.** Bubble sort dan insertion sort perlu 190 perbandingan dan 190 pemindahan: tiap pasangan elemen harus diputar.

4. **Selection sort hemat pemindahan.** Dengan paling banyak satu penukaran per putaran, pemindahannya hanya 18 pada data acak dan 10 pada data terbalik. Bila memindahkan elemen itu mahal (misalnya record besar), ini keunggulan nyata.

5. **Bubble dan insertion sort memindahkan sama banyak pada data acak (90).** Itu bukan kebetulan: keduanya hanya membetulkan pasangan yang urutannya terbalik, dan jumlah pasangan terbalik adalah angka yang sama. Satuannya agak berbeda (penukaran melawan geseran), tetapi angkanya sama.



Gambar 16.16 Pemindahan: selection sort paling hemat, sedangkan bubble dan insertion sort memindahkan banyak pada data terbalik

INFO

Ketiga algoritma bekerja dalam waktu yang tumbuh seperti **kuadrat** jumlah data: dua kali lipat data berarti sekitar empat kali lipat pekerjaan (20 data punya 190 pasangan, 40 data punya 780). Untuk puluhan ribu data, Anda memerlukan algoritma yang lebih cerdas seperti *quicksort*, tetapi ketiga algoritma di bab ini adalah fondasi yang tepat untuk memahaminya, dan untuk data beberapa puluh sampai beberapa ratus elemen mereka sudah memadai.

COBA SENDIRI

Coba Sendiri. Ubah `N` menjadi 10 dan data `Acak` menjadi sepuluh bilangan pertama, atau tambahkan keadaan data keempat (misalnya data sudah terurut kecuali elemen terakhir). Perhatikan algoritma mana yang paling cepat untuk keadaan baru itu.

16.8 Kesalahan yang Sering Terjadi

16.8.1 Menukar tanpa variabel bantu

Telah ditunjukkan pada bagian awal bab: `A := B; B := A;` membuat kedua variabel sama. Pastikan setiap penukaran memakai tiga langkah dengan variabel bantu, atau memakai prosedur seperti `Tukar` pada Program 16.5.

16.8.2 Memindahkan satu field, bukan record utuh

Telah dibuktikan pada program `terpisah.pas` di bagian record: nilai berpisah dari nama. Urutkan array of record dan pindahkan record, jangan field.

16.8.3 Lupa mendeklarasikan variabel bantu

Penukaran memerlukan variabel bantu yang harus dideklarasikan di bagian `var` seperti variabel lain. Bila terlupa, kompiler menolak nama itu begitu ia dipakai pertama kali.

lupabantu.pas

```
1  program LupaBantu;
2  var
3    A, B: Integer;
4  begin
5    A := 29;
6    B := 10;
7    Sementara := A;
8    A := B;
9    B := Sementara;
10 end.
```

PESAN KOMPILER

```
lupabantu.pas(7,3) Error: Identifier tidak dikenal: Sementara
```

Gambar 16.17 Sementara dipakai tetapi tidak pernah dideklarasikan

Perbaikannya: tambahkan `Sementara: Integer;` pada bagian `var`, sejajar dengan `A` dan `B`.

16.9 Rangkuman

- Pengurutan menyusun elemen menurut aturan urutan, naik atau turun; data terurut adalah syarat pencarian biner.
- Menukar dua elemen memerlukan **variabel bantu**: `Bantu := A; A := B; B := Bantu`.
- **Bubble sort** menukar pasangan bersebelahan yang salah urut; setiap putaran membawa elemen terbesar ke kanan, dan `AdaTukar` memungkinkan berhenti lebih awal.
- **Selection sort** memilih elemen terkecil bagian sisa dan menukarnya ke depan; paling banyak `N - 1` pertukaran.
- **Insertion sort** mengambil `Kunci`, menggeser elemen yang lebih besar, dan menyisipkannya; sangat cepat pada data yang hampir terurut.
- Pada array of record, bandingkan satu field (atau aturan dalam fungsi `LebihDulu`) tetapi pindahkan **record utuh**; pemecah seri menangani nilai kembar.
- Pada 20 data, selection sort selalu 190 perbandingan; bubble dan insertion sort hanya 19 pada data terurut, tetapi 190 perbandingan dan 190 pemindahan pada data terbalik.
- Variabel bantu harus dideklarasikan; parameter array sebaiknya memakai tipe bernama seperti `TData`.

16.10 Latihan

1. **DASAR** Dengan pensil dan kertas, jalankan bubble sort pada data `4, 2, 5, 1`. Tulis isi array setiap putaran, lalu cocokkan dengan program Anda.
2. **DASAR** Mengapa batas atas perulangan dalam bubble sort adalah `N - Putaran` dan bukan `N`?

3. **DASAR** Ubah tanda perbandingan pada Program 16.1 agar data terurut turun (terbesar di depan) lalu jalankan.

4. **MENENGAH** Tambahkan penghitung perbandingan dan pertukaran pada Program 16.1. Berapa totalnya untuk data awal pada Program 16.1?

5. **MENENGAH** Ubah Program 16.4 agar nilai dari yang terendah (urutan naik), dengan nama menurut abjad bila seri.

6. **MENENGAH** Pakai data terurut pada Program 16.2. Apakah ada pertukaran? Apakah putaran tetap tujuh? Jelaskan.

7. **TANTANGAN** Buat program yang mengisi array 12 bilangan, mengurutkannya dengan insertion sort, lalu meminta bilangan yang dicari dan mencarinya dengan pencarian biner dari Bab 15.

BAGIAN VI

File, Pointer, Unit, dan Proyek Akhir

Menyimpan data ke disk, memakai memori dinamis, membuat unit sendiri, dan merangkai semuanya dalam proyek.

17 Bekerja dengan File

18 Pointer dan Linked List

19 Unit dan Layar Crt

20 Proyek Akhir

BAB 17

BAGIAN VI · FILE, POINTER, UNIT, DAN PROYEK AKHIR

Bekerja dengan File

TUJUAN PEMBELAJARAN

- Menjelaskan mengapa data perlu disimpan di file, dan apa bedanya dengan variabel yang hilang saat program berhenti.
- Mengikuti alur kerja file: `Assign`, buka (`Rewrite`, `Reset`, atau `Append`), baca atau tulis, lalu `Close`.
- Menulis, membaca, dan menambah isi file teks memakai `WriteLn(F, ...)`, `ReadLn(F, ...)`, dan `Eof`.
- Memeriksa keberhasilan membuka file dengan `{$I-}` dan `IOResult`, serta mengenali galat *runtime* 2 dan 103.
- Membuat file bertipe (`file of`) dan memakai `Seek`, `FilePos`, `FileSize` untuk mengakses elemen mana pun.
- Menyimpan dan membaca `record` utuh sehingga data tetap ada setelah program ditutup.

PROGRAM DALAM BAB INI

17.1 `bab17_01_tulis_file_teks.pas`
Menulis file teks

17.2 `bab17_02_baca_file_teks.pas`
Membaca file teks

17.3 `bab17_03_tambah_file_teks.pas`
Menambah isi file teks

17.4 `bab17_04_file_bertipe.pas`
File bertipe

17.5 `bab17_05_simpan_baca_record.pas`
Menyimpan dan membaca record

Semua variabel yang Anda pakai sampai bab ini hidup di memori, dan memori itu dikosongkan begitu program berhenti. Nilai yang diketik pengguna, hasil hitungan, daftar nama: semuanya lenyap. Bab ini mengajarkan cara menyimpan data ke **file** di disk, sehingga program hari ini bisa membaca data yang ditulis program kemarin. Dengan file, program Anda mulai terasa seperti aplikasi sungguhan: punya ingatan.

17.1 Mengapa Perlu File?

Bayangkan Anda mencatat nilai siswa di papan tulis. Selama papan itu belum dihapus, catatannya ada; begitu dihapus, semuanya hilang. Variabel adalah papan tulis itu: ia berguna selama program berjalan, tetapi tidak ada yang tersisa sesudahnya. **File** adalah buku catatan. Tulisan di buku tetap ada setelah lampu dimatikan, dan bisa dibuka kembali besok, oleh program yang sama atau program lain.

V Variabel

Disimpan di memori kerja (RAM). Cepat, tetapi **hilang** saat program selesai atau komputer dimatikan.

F File

Disimpan di disk. Lebih lambat, tetapi **awet**: tetap ada setelah program ditutup dan bisa dibaca lagi kapan saja.

Pascal mengenal dua jenis file yang akan Anda pakai di bab ini. **File teks** (`Text`) berisi tulisan yang tersusun dalam baris-baris, persis seperti yang tampil di layar; Anda bisa membukanya dengan editor teks mana pun. **File bertipe** (`file of` sebuah tipe) berisi deretan data biner berukuran sama, misalnya bilangan atau `record`; isinya tidak enak dibaca manusia, tetapi cepat diakses dan rapi.

Tabel 17.1 Perbandingan file teks dan file bertipe

	File teks	File bertipe
Deklarasi	`F: Text;`	`F: file of LongInt;`
Isi	Baris-baris tulisan	Deretan elemen bertipe sama (biner)
Dibaca manusia?	Ya, dengan editor teks	Tidak, tampak seperti karakter acak
Menulis	`Write`, `WriteLn`	`Write(F, X)` (satu elemen)
Membaca	`Read`, `ReadLn`	`Read(F, X)` (satu elemen)
Akses	Berurutan dari awal ke akhir	Berurutan atau langsung ke elemen mana pun (`Seek`)
Cocok untuk	Catatan, laporan, data yang ingin dibuka di editor	Daftar bilangan, data siswa, basis data kecil

17.2 Alur Kerja File

Berbeda dengan variabel biasa, file tidak bisa langsung dipakai. Ada urutan baku yang harus diikuti, dan urutan ini sama untuk file teks maupun file bertipe. Hafalkan empat langkah berikut.



Gambar 17.1 Empat langkah wajib setiap kali memakai file, dan tiga pilihan cara membukanya.

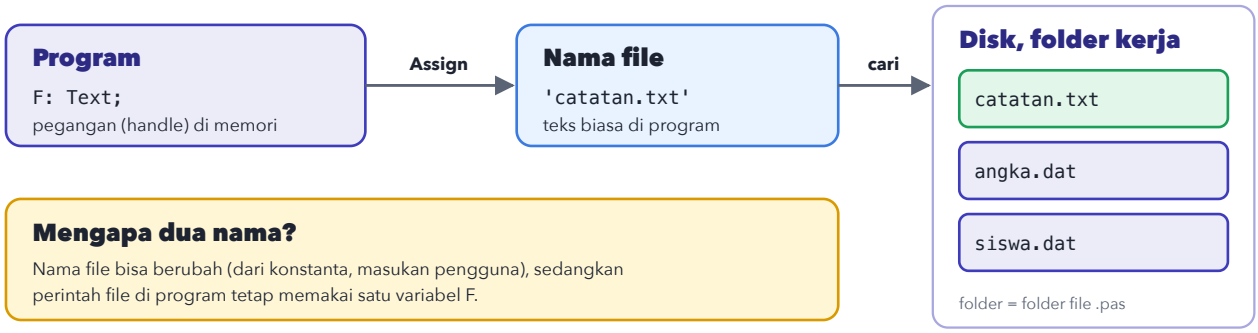
1. **Assign.** `Assign(F, 'catatan.txt')` menghubungkan variabel file `F` dengan sebuah nama file di disk. Sesudah ini, semua perintah file memakai `F`, bukan nama file lagi. `Assign` belum membuka apa-apa.
2. **Buka.** Pilih salah satu: `Rewrite(F)` untuk membuat file baru, `Reset(F)` untuk membuka file yang sudah ada, atau `Append(F)` untuk menambah di akhir file teks.
3. **Pakai.** Tulis dengan `Write` / `WriteLn`, baca dengan `Read` / `ReadLn`. Perintahnya sama dengan yang sudah Anda kenal, hanya diberi `F` sebagai argumen pertama.
4. **Close.** `Close(F)` menutup file. Langkah ini wajib: isi yang ditulis baru dijamin sampai ke disk setelah `Close`.

BENTUK UMUM

```
var
  F: Text;                    { variabel file teks }

Assign(F, 'nama.txt');        { 1. hubungkan }
Rewrite(F);                   { 2. buka (atau Reset / Append) }
WriteLn(F, data);             { 3. tulis (atau ReadLn) }
Close(F);                     { 4. tutup }
```

Bagaimana hubungan antara variabel `F`, nama file, dan isi disk? Variabel `F` bukan file itu sendiri; ia hanya **pegangan** (*handle*) yang dibuat Pascal untuk mengendalikan file. Mirip nomor antrean di loket: nomor itu bukan orangnya, tetapi lewat nomor itulah petugas tahu siapa yang dilayani.



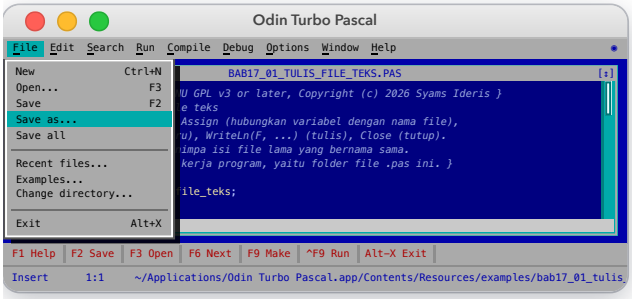
Gambar 17.2 Variabel file `F` hanyalah pegangan; `Assign` menautkannya ke nama file di folder kerja.

INFO

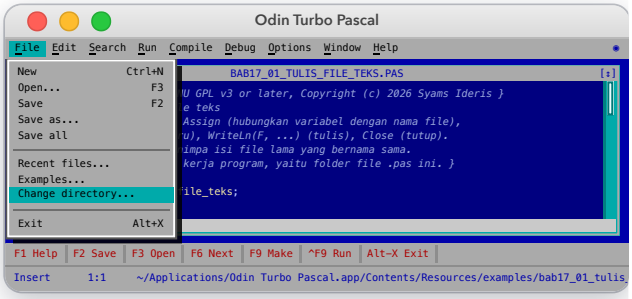
Program Anda dijalankan dengan **folder file** `.pas` sebagai folder kerja. Nama file tanpa alamat, seperti `'catatan.txt'`, dibuat dan dicari di folder itu. Karena itu semua contoh bab ini meninggalkan file di folder tempat program sumbernya berada.

AWAS

Contoh bawaan Odin dianggap *template*: bila Anda membukanya lewat `File ▸ Examples...` lalu menekan `File ▸ Save`, IDE akan menawarkan `File ▸ Save as...`. Sebaiknya simpan dulu ke **folder Anda sendiri** (misalnya `Dokumen/Pascal`) sebelum menjalankan program file, supaya berkas data yang dibuat tidak bercampur dengan berkas bawaan aplikasi. Bila perlu, ganti folder kerja lewat `File ▸ Change directory...`.



Gambar 17.3 Simpan salinan contoh ke folder sendiri lewat File ▸ Save as...



Gambar 17.4 Atau pindahkan folder kerja lewat File ▸ Change directory...

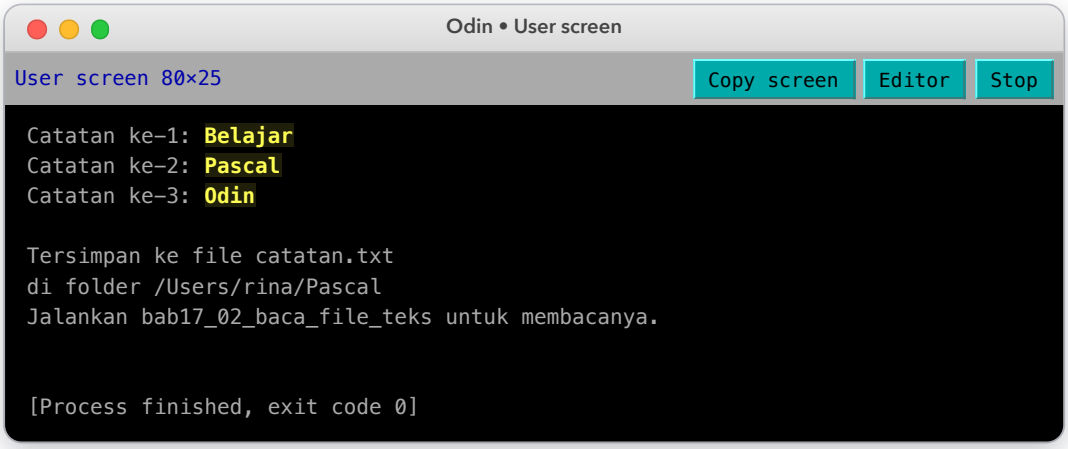
17.3 Menulis File Teks

Program pertama membuat file baru bernama `catatan.txt` dan mengisinya dengan judul serta tiga catatan yang diketik pengguna. Buka `bab17_01_tulis_file_teks.pas` lewat `File ▸ Examples...` (simpan dulu ke folder Anda, lihat kotak di atas), lalu jalankan dengan **Ctrl** + **F9**. Ketikkan tiga catatan, masing-masing diakhiri **Enter**.

Program 17.1 `bab17_01_tulis_file_teks.pas` *Menulis file teks* — File ▸ Examples...

```
1  { Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 17.1 - Menulis file teks
3    Langkah menulis file: Assign (hubungkan variabel dengan nama file),
```

```
4    Rewrite (buat file baru), WriteLn(F, ...) (tulis), Close (tutup).
5    PERHATIAN: Rewrite menimpa isi file lama yang bernama sama.
6    File dibuat di folder kerja program, yaitu folder file .pas ini. }
7
8  program bab17_01_tulis_file_teks;
9
10 const
11   NamaFile = 'catatan.txt';              { satu tempat untuk nama file }
12
13 var
14   F: Text;                               { variabel file teks }
15   Folder, Catatan: string;
16   I: Integer;
17
18 begin
19   Assign(F, NamaFile);
20   Rewrite(F);                            { buat baru (atau kosongkan) }
21   WriteLn(F, 'CATATAN HARIAN');          { sama seperti WriteLn biasa, }
22   WriteLn(F, '==============');          { tetapi ditulis ke file F    }
23
24   for I := 1 to 3 do
25   begin
26     Write('Catatan ke-', I, ': ');
27     ReadLn(Catatan);
28     WriteLn(F, I, '. ', Catatan);
29   end;
30   Close(F);                              { wajib: memastikan isi tersimpan }
31
32   GetDir(0, Folder);                     { folder kerja saat ini }
33   WriteLn;
34   WriteLn('Tersimpan ke file ', NamaFile);
35   WriteLn('di folder ', Folder);
36   WriteLn('Jalankan bab17_02_baca_file_teks untuk membacanya.');
37 end.
```



Gambar 17.5 Tiga catatan diketik di User screen; file `catatan.txt` dibuat di folder kerja

Baris	Penjelasan
10–11	Nama file ditaruh di konstanta `NamaFile`. Bila suatu hari nama berubah, cukup satu tempat yang diganti.
14	`F: Text` mengumumkan variabel file teks. Nama `Folder` dan `Catatan` (baris 15) bertipe string biasa.

Baris	Penjelasan
19	`Assign` menghubungkan `F` dengan `'catatan.txt'`. File belum disentuh.
20	`Rewrite` membuat file baru. Bila `catatan.txt` sudah ada, isinya **dikosongkan** lebih dulu.
21–22	Dua baris judul ditulis ke file. Perhatikan `F` sebagai argumen pertama: tanpa `F`, tulisan akan muncul di layar.
24–29	Perulangan tiga kali. Baris 26 menulis prompt ke **layar**, baris 27 membaca ketikan dari **papan ketik**, dan baris 28 menulis nomor serta catatan ke **file**. Tiga arah data dalam satu perulangan.
30	`Close(F)` menutup file sehingga semua tulisan pasti tersimpan.
32–36	`GetDir(0, Folder)` mengisi `Folder` dengan folder kerja saat ini, lalu program menampilkannya. Dari sini Anda tahu persis di mana `catatan.txt` berada.

Sesudah program selesai, buka folder yang ditampilkan dan periksa `catatan.txt` dengan editor teks. Isinya lima baris: dua baris judul, lalu tiga catatan bernomor yang Anda ketik.

AWAS

`Rewrite` **menimpa** file lama yang bernama sama tanpa bertanya. Menjalankan Program 17.1 untuk kedua kalinya akan menghapus catatan dari kali pertama. Bila Anda ingin menambah isi, gunakan `Append` (subbab 17.4).

COBA SENDIRI

Coba sendiri. Jalankan Program 17.1 dua kali dengan catatan yang berbeda, lalu buka `catatan.txt`. Hanya catatan dari kali kedua yang tersisa. Lalu ubah baris 11 menjadi `NamaFile = 'harian.txt';` dan jalankan lagi: sekarang terbentuk dua file berbeda.

17.4 Membaca File Teks

Menulis tanpa membaca tidak ada gunanya. Program kedua membuka `catatan.txt` yang baru dibuat dan menampilkan isinya baris demi baris, lengkap dengan nomor baris. Dua hal baru muncul: `Reset` untuk membuka file yang sudah ada, dan fungsi `Eof(F)` (*end of file*) yang bernilai `True` bila pembacaan sudah sampai ujung file.

BENTUK UMUM

```
Reset(F);                       { buka untuk dibaca }
while not Eof(F) do
begin
  ReadLn(F, variabelString);    { baca satu baris }
  ...olah baris itu...
end;
Close(F);
```

Pola `while not Eof(F) do ReadLn(F, ...)` adalah cara baku membaca file teks sampai habis tanpa perlu tahu berapa banyak barisnya. Eksekusinya digambarkan di bawah ini: kursor baca bergerak satu baris setiap `ReadLn`, dan `Eof` menjadi `True` setelah baris terakhir.



Gambar 17.6 Kursor baca maju satu baris setiap `ReadLn`; perulangan berhenti ketika `Eof(F)` bernilai `True`.

Buka `bab17_02_baca_file_teks.pas` dan jalankan. Karena kedua program dijalankan dari folder yang sama, program ini menemukan `catatan.txt` milik Program 17.1.

Program 17.2 `bab17_02_baca_file_teks.pas` *Membaca file teks* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 17.2 — Membaca file teks
  Reset membuka file untuk dibaca. Eof(F) bernilai True bila sudah di
  akhir file. Membuka file yang tidak ada biasanya menghentikan
  program; dengan $I- pemeriksaan otomatis dimatikan sementara, lalu
  hasilnya dibaca lewat IOResult (0 berarti berhasil). }

program bab17_02_baca_file_teks;

const
  NamaFile = 'catatan.txt';              { dibuat oleh bab17_01 }

var
  F: Text;
  Baris: string;
  NomorBaris: Integer;

begin
  Assign(F, NamaFile);
  {$I-}                                  { matikan pemeriksaan galat I/O }
  Reset(F);
  {$I+}                                  { nyalakan lagi }
  if IOResult <> 0 then
  begin
    WriteLn('File ', NamaFile, ' belum ada.');
    WriteLn('Jalankan bab17_01_tulis_file_teks lebih dulu.');
  end
  else
  begin
    NomorBaris := 0;
    while not Eof(F) do                  { ulangi sampai akhir file }
    begin
```

```
33        ReadLn(F, Baris);                { baca satu baris dari file }
34        NomorBaris := NomorBaris + 1;
35        WriteLn(NomorBaris:3, ' | ', Baris);
36      end;
37      Close(F);
38      WriteLn('Jumlah baris: ', NomorBaris);
39    end;
40  end.
```



Gambar 17.7 Isi `catatan.txt` dibaca baris demi baris dan diberi nomor

Baris	Penjelasan
19	`Assign` menautkan `F` ke `catatan.txt`, nama yang sama dengan yang dipakai Program 17.1.
20–22	Direktif `{$I-}` mematikan pemeriksaan galat I/O otomatis sementara `Reset` dikerjakan, lalu `{$I+}` menyalakannya lagi. Dengan begitu, file yang tidak ada tidak menghentikan program.
23	`IOResult` mengembalikan 0 bila operasi file terakhir berhasil, dan kode galat bila gagal. Nilai `<> 0` berarti file tidak bisa dibuka.
25–26	Cabang gagal: pesan ramah yang menyuruh pengguna menjalankan Program 17.1 lebih dulu.
30	Penghitung baris dimulai dari nol.
31–36	Inti pembacaan: ulangi selama belum `Eof`, baca satu baris ke `Baris`, naikkan penghitung, tulis dengan nomor berlebar 3 (`NomorBaris:3`).
37–38	File ditutup, lalu jumlah baris dilaporkan. Jumlahnya 5: dua baris judul ditambah tiga catatan.

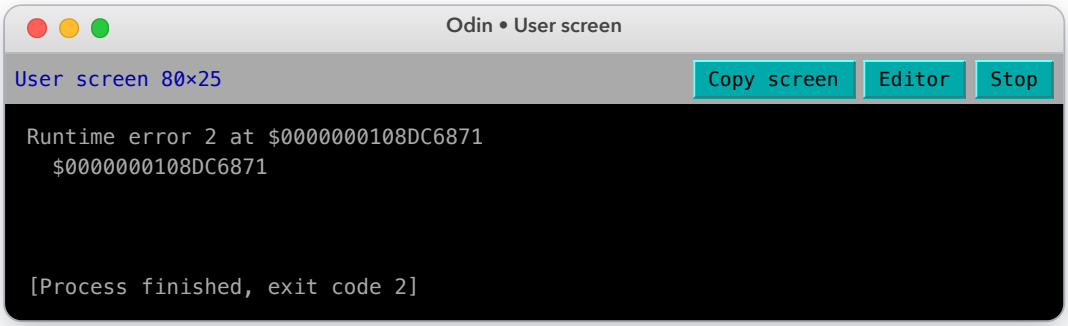
7 **ATURAN TURBO PASCAL 7**

Dalam Turbo Pascal, membuka file yang tidak ada dengan `Reset` menghentikan program dengan galat *runtime* 2 ("File not found"). Cara menghindarinya adalah pola tiga langkah di baris 20–23: matikan pemeriksaan dengan `{$I-}`, buka file, hidupkan kembali dengan `{$I+}`, lalu periksa `IOResult`. Perhatikan bahwa `IOResult` harus dibaca **sekali** setelah operasi; nilainya dikosongkan begitu dibaca.

Apa yang terjadi bila pengaman itu tidak ada? Program kecil berikut membuka file yang tidak pernah dibuat, tanpa `{$I-}`. Ia dikompilasi tanpa masalah, tetapi berhenti di tengah jalan.

tidakada.pas

```
1  program TidakAda;
2  var
3    F: Text;
4    Baris: string;
5  begin
6    Assign(F, 'tidak_ada.txt');
7    Reset(F);
8    ReadLn(F, Baris);
9    WriteLn(Baris);
10   Close(F);
11 end.
```

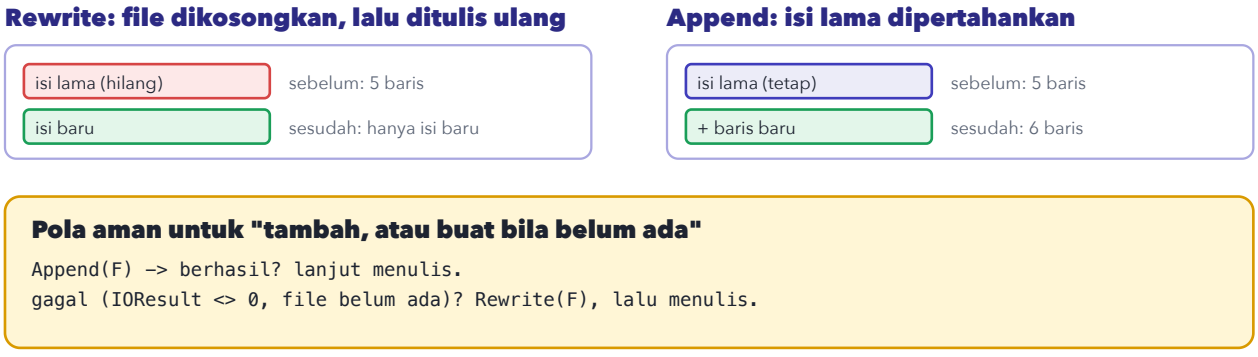


Gambar 17.8 Reset pada file yang tidak ada menghentikan program dengan *runtime error* 2

Galat 2 berarti *file not found*: file yang diminta tidak ada di folder kerja. Kompiler tidak bisa mencegahnya, karena ada tidaknya file baru diketahui saat program berjalan. Karena itu pembukaan file yang berisiko selalu dijaga dengan `{$I-}` dan `IOResult`, seperti di Program 17.2.

17.5 Menambah Isi File Teks

Bagaimana bila Anda ingin menambah satu catatan ke `catatan.txt` tanpa menghapus yang lama? Gunakan `Append`. Perintah ini membuka file teks dan meletakkan kursor tulis di **akhir**, sehingga `WriteLn` berikutnya menyambung isi lama.



Gambar 17.9 Perbedaan `Rewrite` dan `Append` pada file yang sudah berisi lima baris.

Masalahnya: `Append` gagal bila file belum ada. Program berikut menanganinya dengan pola "coba `Append`; bila gagal, buat dengan `Rewrite`". Setelah menulis, program membuka file lagi dengan `Reset` dan menampilkan seluruh isinya agar Anda melihat hasilnya.

Program 17.3 `bab17_03_tambah_file_teks.pas` *Menambah isi file teks* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 17.3 — Menambah isi file teks
  Append membuka file dan menaruh tulisan baru di akhir, tanpa
  menghapus isi lama. Append gagal bila file belum ada, jadi periksa
  dulu dengan IOResult; bila belum ada, buat dengan Rewrite. }

program bab17_03_tambah_file_teks;

const
  NamaFile = 'catatan.txt';

var
  F: Text;
  Catatan, Baris: string;
  Jumlah: Integer;

begin
  Write('Catatan baru: ');
  ReadLn(Catatan);

  Assign(F, NamaFile);
  {$I-}
  Append(F);                          { buka untuk menambah di akhir }
  {$I+}
  if IOResult <> 0 then
  begin
    WriteLn('(', NamaFile, ' belum ada, dibuat baru)');
    Rewrite(F);
  end;
  WriteLn(F, '+ ', Catatan);
  Close(F);

  { Baca ulang untuk melihat hasilnya }
  Reset(F);
  Jumlah := 0;
  while not Eof(F) do
  begin
    ReadLn(F, Baris);
    WriteLn('  ', Baris);
    Jumlah := Jumlah + 1;
  end;
  Close(F);
  WriteLn(NamaFile, ' kini berisi ', Jumlah, ' baris.');
end.
```



Gambar 17.10 Catatan baru disambung di akhir; `catatan.txt` kini berisi enam baris

Baris	Penjelasan
18–19	Catatan baru dibaca dari papan ketik lebih dulu, sebelum file disentuh.
21–24	`Assign`, lalu `Append` dalam kurung `{$I-}` dan `{$I+}` agar kegagalan tidak menghentikan program.
25–29	Bila `IOResult` bukan 0, file belum ada: program memberi tahu pengguna dan membuatnya dengan `Rewrite`.
30–31	Baris baru dengan awalan `+` ditulis di akhir file, lalu file ditutup. Dua jalur di atas bertemu di sini.
34	`Reset` membuka file yang sama untuk dibaca dari awal. Sebuah file harus ditutup dulu sebelum dibuka dengan mode lain.
35–41	Pola baca sampai `Eof` seperti Program 17.2, dengan penghitung `Jumlah`.
42–43	File ditutup dan jumlah barisnya dilaporkan: lima baris lama ditambah satu baris baru.

COBA SENDIRI

Coba sendiri. Jalankan Program 17.3 beberapa kali dan perhatikan daftarnya memanjang. Lalu hapus `catatan.txt` dari folder (atau ganti `NamaFile` dengan nama baru) dan jalankan lagi: pesan "belum ada, dibuat baru" muncul dan jumlah barisnya menjadi 1.

17.6 File Bertipe

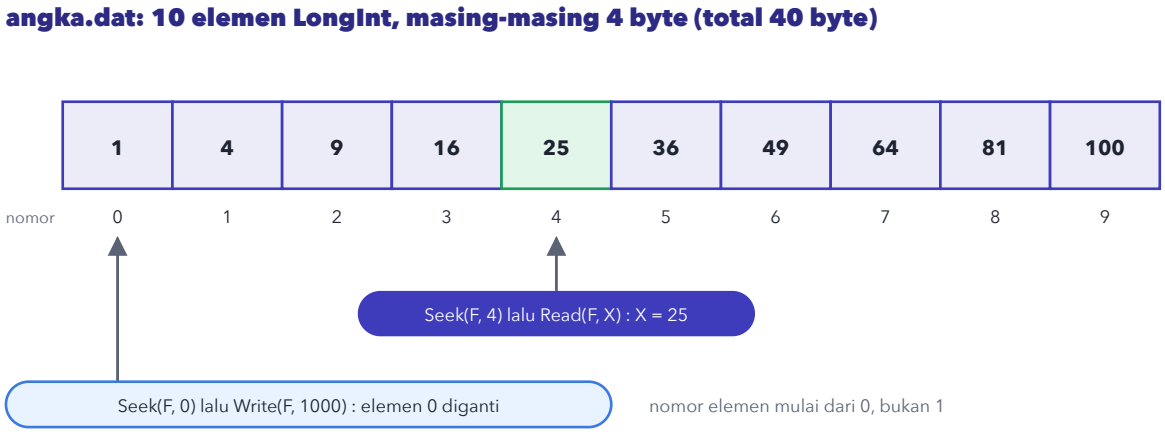
File teks menyimpan semuanya sebagai tulisan. Untuk menyimpan bilangan atau data terstruktur dalam jumlah besar, cara itu boros dan lambat: angka 1000 harus diubah menjadi karakter `'1'`, `'0'`, `'0'`, `'0'` dan dibaca balik. **File bertipe** menyimpan data sebagaimana adanya di memori. Setiap elemen memiliki ukuran yang sama, misalnya 4 byte untuk `LongInt`.

BENTUK UMUM

```
var
  F: file of Tipe;          { setiap elemen bertipe Tipe }

Write(F, X);                 { simpan satu elemen }
Read(F, X);                  { baca satu elemen }
Seek(F, N);                  { pindah ke elemen nomor N (mulai 0) }
FilePos(F)                   { nomor elemen saat ini }
FileSize(F)                  { banyak elemen dalam file }
```

Karena semua elemen berukuran sama, Pascal dapat menghitung letak elemen mana pun: elemen nomor *N* berada *N* × ukuran elemen byte dari awal file. Itulah sebabnya `Seek` bisa langsung melompat, tidak perlu membaca dari awal. Gambar berikut memperlihatkan file `angka.dat` yang berisi kuadrat 1 sampai 10.



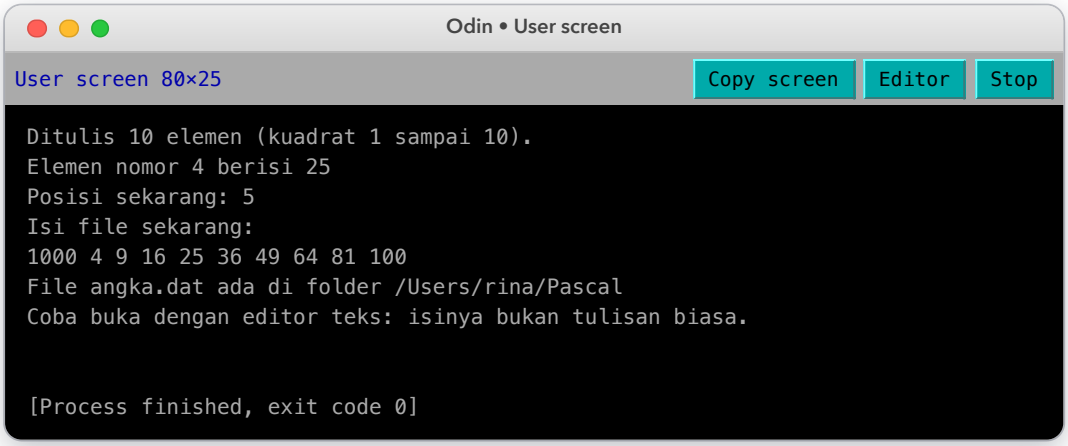
Gambar 17.11 Elemen file bertipe diberi nomor mulai 0. `Seek(F, 4)` mendatangi elemen kelima, yang berisi 25.

Program berikut menulis sepuluh kuadrat ke `angka.dat`, melompat ke elemen nomor 4 untuk membacanya, mengubah elemen pertama menjadi 1000, lalu membaca seluruh file. Perhatikan bahwa file tidak ditutup di antara menulis dan membaca: file bertipe yang dibuka dengan `Rewrite` bisa ditulis dan dibaca, dan `Seek` mengatur posisinya.

Program 17.4 `bab17_04_file_bertipe.pas` *File bertipe* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 17.4 — File bertipe
3    File bertipe (file of ...) menyimpan data dalam bentuk biner, bukan
4    teks. Setiap elemen berukuran sama, jadi elemen mana pun bisa
5    langsung didatangi dengan Seek. Nomor elemen dimulai dari 0.
6      FileSize(F)  banyak elemen     FilePos(F)  posisi saat ini
7      Seek(F, N)   pindah ke elemen N }
8
9  program bab17_04_file_bertipe;
10
11 const
12   NamaFile = 'angka.dat';
13
14 var
```

```
15    F: file of LongInt;                    { setiap elemen satu LongInt }
16    X: LongInt;
17    I: Integer;
18    Folder: string;
19
20  begin
21    Assign(F, NamaFile);
22    Rewrite(F);
23    for I := 1 to 10 do
24    begin
25      X := I * I;
26      Write(F, X);                         { file bertipe memakai Write }
27    end;
28    WriteLn('Ditulis ', FileSize(F), ' elemen (kuadrat 1 sampai 10).');
29
30    Seek(F, 4);                            { elemen ke-5, karena mulai 0 }
31    Read(F, X);
32    WriteLn('Elemen nomor 4 berisi ', X);
33    WriteLn('Posisi sekarang: ', FilePos(F));
34
35    Seek(F, 0);                            { ubah elemen pertama }
36    X := 1000;
37    Write(F, X);
38
39    WriteLn('Isi file sekarang:');
40    Seek(F, 0);
41    while not Eof(F) do
42    begin
43      Read(F, X);
44      Write(X, ' ');
45    end;
46    WriteLn;
47    Close(F);
48
49    GetDir(0, Folder);
50    WriteLn('File ', NamaFile, ' ada di folder ', Folder);
51    WriteLn('Coba buka dengan editor teks: isinya bukan tulisan biasa.');
52  end.
```



Gambar 17.12 Isi `angka.dat` setelah elemen pertama diubah menjadi 1000

Baris	Penjelasan
15	`F: file of LongInt`: setiap elemen file adalah satu `LongInt` (4 byte).

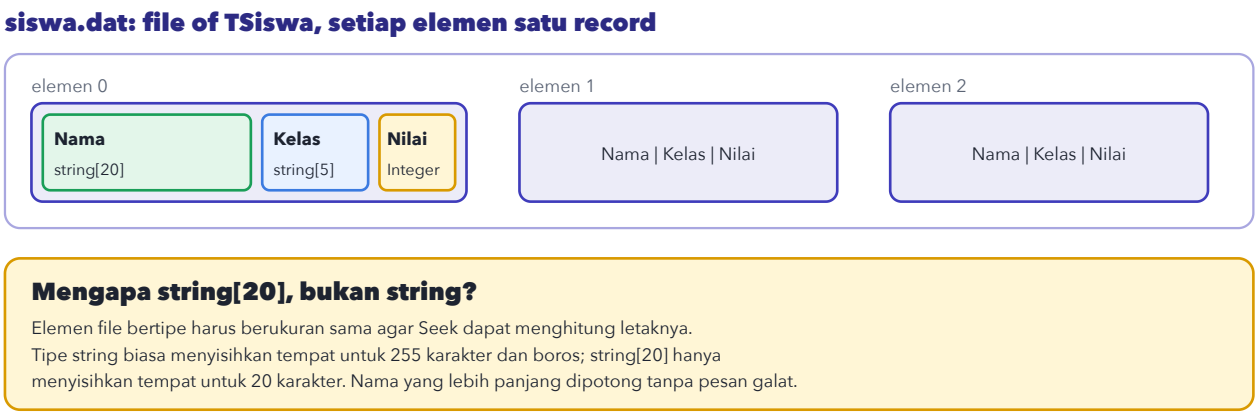
Baris	Penjelasan
21–22	`Assign` dan `Rewrite` membuat `angka.dat` baru. Berbeda dengan file teks, `Rewrite` pada file bertipe tetap memungkinkan membaca.
23–27	Sepuluh kuadrat (1, 4, 9, ... 100) ditulis dengan `Write(F, X)`. Setiap `Write` menyimpan satu elemen dan memajukan posisi.
28	`FileSize(F)` mengembalikan banyak elemen: 10.
30–33	`Seek(F, 4)` melompat ke elemen nomor 4 (yang kelima). Setelah `Read`, posisi maju satu elemen sehingga `FilePos(F)` bernilai 5.
35–37	Kembali ke elemen 0 dan menimpanya dengan 1000. Hanya satu elemen yang berubah; sembilan lainnya tidak disentuh.
39–46	`Seek(F, 0)` memulai pembacaan dari awal, lalu pola `while not Eof` membaca semua elemen dan mencetaknya dalam satu baris.
47–52	File ditutup dan lokasinya ditampilkan. Coba buka `angka.dat` dengan editor teks: yang tampak hanya karakter aneh, bukan angka.

7 **ATURAN TURBO PASCAL 7**

Di Turbo Pascal 7 mode, `Integer` hanya 2 byte, sedangkan `LongInt` 4 byte. Ukuran elemen file bertipe mengikuti tipenya, sehingga `file of Integer` dan `file of LongInt` tidak saling kompatibel. File biner buatan DOS dengan tipe lain atau perataan `record` yang berbeda juga tidak otomatis terbaca di macOS 64-bit; lihat `docs/COMPATIBILITY.txt`.

17.7 Menyimpan Record ke File

Kekuatan sebenarnya file bertipe tampak saat elemennya adalah `record`. Satu elemen dapat memuat nama, kelas, dan nilai seorang siswa sekaligus. Dengan `file of TSiswa`, Anda menyimpan dan membaca seluruh catatan siswa dengan satu perintah `Write` atau `Read`. Inilah dasar program basis data sederhana, yang akan Anda kembangkan di bab proyek akhir.



Gambar 17.13 Setiap elemen `siswa.dat` adalah satu `TSiswa` utuh; semuanya berukuran sama.

Program 17.5 membaca data yang sudah ada (bila ada), meminta siswa baru sampai nama dikosongkan, menyimpan setiap siswa dengan `Write(F, S)`, lalu menampilkan seluruh isi file sebagai tabel. Karena datanya disimpan di file, **jalankan program ini beberapa kali** dan Anda akan melihat daftarnya terus bertambah. Pada jalankan pertama di bawah ini file belum ada, jadi jumlah data tersimpan 0.

Program 17.5 `bab17_05_simpan_baca_record.pas` *Menyimpan dan membaca record* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 17.5 — Menyimpan dan membaca record
  file of TSiswa menyimpan record utuh. Data tetap ada walau program
  ditutup, jadi jalankan program ini beberapa kali dan lihat datanya
  bertambah. Field string memakai panjang tetap, misalnya string[20]. }

program bab17_05_simpan_baca_record;

const
  NamaFile = 'siswa.dat';

type
  TSiswa = record
    Nama: string[20];
    Kelas: string[5];
    Nilai: Integer;
  end;

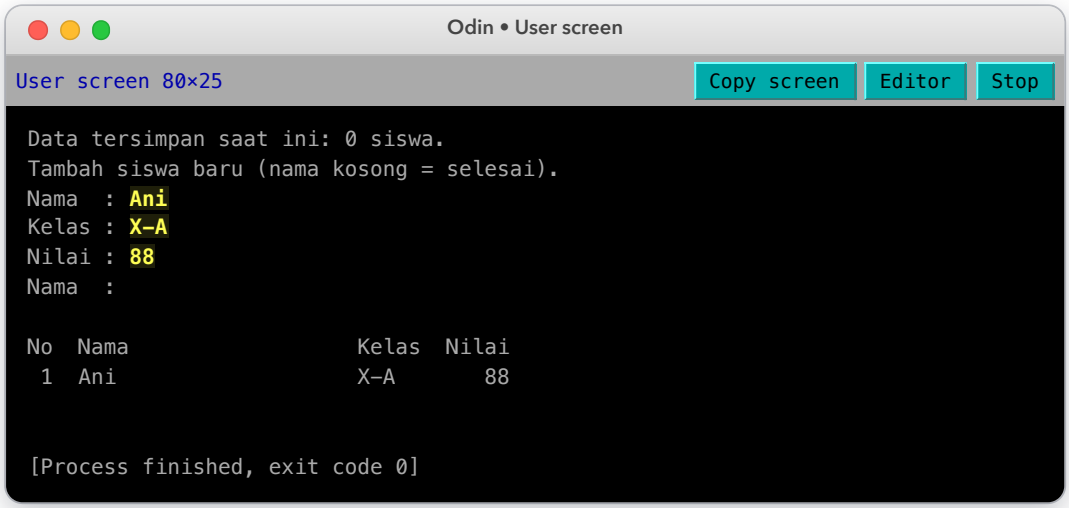
var
  F: file of TSiswa;
  S: TSiswa;
  Nama: string;
  Nomor: Integer;

begin
  Assign(F, NamaFile);
  {$I-}
  Reset(F);                          { buka file yang sudah ada }
  {$I+}
  if IOResult <> 0 then
    Rewrite(F);                      { belum ada: buat baru }
  WriteLn('Data tersimpan saat ini: ', FileSize(F), ' siswa.');

  Seek(F, FileSize(F));              { pindah ke akhir untuk menambah }
  WriteLn('Tambah siswa baru (nama kosong = selesai).');
  repeat
    Write('Nama  : ');
    ReadLn(Nama);
    if Nama <> '' then
    begin
      S.Nama := Nama;
      Write('Kelas : '); ReadLn(S.Kelas);
      Write('Nilai : '); ReadLn(S.Nilai);
      Write(F, S);                   { simpan satu record }
    end;
  until Nama = '';

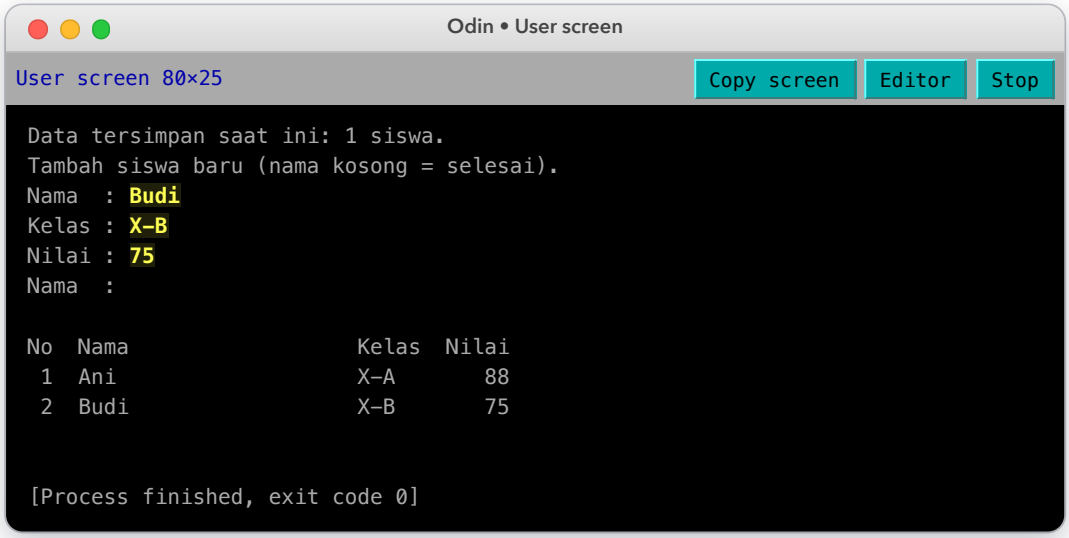
  WriteLn;
  WriteLn('No  Nama                 Kelas  Nilai');
  Seek(F, 0);                        { kembali ke awal untuk membaca }
```

```
51    Nomor := 0;
52    while not Eof(F) do
53    begin
54      Read(F, S);
55      Nomor := Nomor + 1;
56      WriteLn(Nomor:2, '  ', S.Nama, '':22 - Length(S.Nama), S.Kelas,
57        '':7 - Length(S.Kelas), S.Nilai:5);
58    end;
59    Close(F);
60  end.
```



Gambar 17.14 Jalankan pertama: file belum ada, satu siswa ditambah, lalu nama kosong mengakhiri masukan

Sekarang jalankan sekali lagi dan tambahkan siswa kedua. Daftar lama tidak hilang: program melaporkan satu siswa tersimpan, lalu menampilkan dua siswa sesudah penambahan.



Gambar 17.15 Jalankan kedua: data Ani dari jalankan pertama masih ada, Budi ditambahkan

Baris	Penjelasan
12–17	Tipe `TSiswa` adalah `record` berisi dua string pendek dan satu bilangan. Bentuknya menentukan ukuran setiap elemen file.
20	`F: file of TSiswa` mengumumkan file yang elemennya `TSiswa`.

Baris	Penjelasan
26–31	Mencoba `Reset` (file sudah ada) dengan pengaman `{$I-}`. Bila gagal, `Rewrite` membuat file kosong. Dengan begitu program bekerja pada jalankan pertama maupun berikutnya.
32	`FileSize(F)` melaporkan banyak siswa yang sudah tersimpan.
34	`Seek(F, FileSize(F))` memindah posisi ke tepat sesudah elemen terakhir, sehingga `Write` berikutnya **menambah**, bukan menimpa.
36–46	Perulangan `repeat..until` meminta nama. Bila nama tidak kosong, kelas dan nilai dibaca ke field `S.Kelas` dan `S.Nilai` (baris 42–43), lalu seluruh record disimpan dengan `Write(F, S)` (baris 44). Nama kosong mengakhiri perulangan.
50	`Seek(F, 0)` kembali ke awal agar seluruh data dapat dibaca.
52–58	Membaca satu record per putaran dan mencetak barisnya. Kolom diratakan dengan `'':22 - Length(S.Nama)`, yaitu menulis string kosong selebar sisa kolom sebagai spasi pengisi. Nilai ditulis selebar 5.
59	File ditutup.

> ★ **INGAT**
>
> Satu program, satu *file variable*, satu urutan: `Assign`, buka, pakai, `Close`. Tidak peduli apakah isinya teks, bilangan, atau record, rangka program file selalu sama. Yang berubah hanyalah `Text` atau `file of ...` pada deklarasi dan pilihan perintah baca-tulisnya.

17.8 Kesalahan yang Sering Terjadi

17.8.1 Membuka file yang tidak ada (galat 2)

Sudah dibahas pada Program 17.2: `Reset` atau `Append` pada file yang tidak ada di folder kerja menghentikan program dengan galat *runtime* 2. Penyebab tersering adalah salah nama, salah folder (program dijalankan dari folder lain), atau file memang belum dibuat. Periksalah dengan `{$I-}` dan `IOResult`, dan pastikan Anda tahu folder kerja dengan `GetDir`.

17.8.2 Menulis ke file yang belum dibuka (galat 103)

Lupa memanggil `Rewrite` atau `Reset` setelah `Assign` adalah kekeliruan klasik. Kompiler menerimanya, tetapi saat dijalankan file belum "terbuka".

lupabuka.pas

```
1  program LupaBuka;
2  var
3    F: Text;
4  begin
5    Assign(F, 'x.txt');
6    WriteLn(F, 'halo');
7    Close(F);
8  end.
```



Gambar 17.16 Menulis tanpa Rewrite: file belum dibuka (*runtime error* 103)

Galat 103 berarti *file not open*. Perbaikannya: sisipkan `Rewrite(F);` (untuk menulis) atau `Reset(F);` (untuk membaca) di antara `Assign` dan perintah baca-tulis.

17.8.3 Mencampur tipe pada file bertipe

File bertipe hanya menerima nilai bertipe elemennya. Menulis string ke `file of LongInt` ditolak saat kompilasi.

tipesalah.pas

```
1  program TipeSalah;
2  var
3    F: file of LongInt;
4  begin
5    Assign(F, 'a.dat');
6    Rewrite(F);
7    Write(F, 'abc');
8    Close(F);
9  end.
```

PESAN KOMPILER

```
tipesalah.pas(7,12) Error: Tipe tidak cocok: String dan LongInt
```

Gambar 17.17 File of LongInt tidak menerima string

17.8.4 Lupa Close, menimpa data dengan Rewrite

Dua kesalahan ini tidak menghasilkan pesan apa pun, sehingga lebih berbahaya. **Lupa** `Close` dapat membuat sebagian isi tidak sempat tersimpan. Biasakan menulis `Close(F)` segera setelah menulis `Rewrite` atau `Reset`, lalu isi bagian tengahnya. **Memakai** `Rewrite` **untuk menambah data** menghapus semua data lama; untuk menambah, pakai `Append` (file teks) atau `Seek(F, FileSize(F))` (file bertipe) seperti pada Program 17.5.

17.9 Rangkuman

- Variabel hilang saat program berhenti; **file** menyimpan data di disk sehingga tetap ada.
- Alur baku: `Assign` (hubungkan variabel file dengan nama file), buka dengan `Rewrite`, `Reset`, atau `Append`, baca atau tulis, lalu `Close`.

- Berkas dibuat di **folder kerja** program, yaitu folder file `.pas`. Simpan contoh ke folder sendiri dengan `File ▸ Save as...` sebelum menjalankan program file.
- `Rewrite` membuat file baru dan **menghapus** isi lama; `Append` menambah di akhir file teks dan gagal bila file belum ada.
- Membaca file teks sampai habis: `while not Eof(F) do ReadLn(F, ...)`.
- File yang tidak ada ditangani dengan `{$I-}`, `Reset`, `{$I+}`, lalu memeriksa `IOResult <> 0`; tanpa itu program berhenti dengan galat 2.
- File bertipe (`file of`) menyimpan elemen biner berukuran sama. `Seek` (nomor elemen mulai 0), `FilePos`, dan `FileSize` memungkinkan akses langsung.
- `file of record` menyimpan data terstruktur; pakai `string[n]` agar ukuran elemen tetap, dan `Seek(F, FileSize(F))` untuk menambah di akhir.

17.10 Latihan

1. **DASAR** Buat program yang menulis lima nama teman ke file `teman.txt`, satu nama per baris. Periksa hasilnya dengan editor teks.
2. **DASAR** Buat program yang membaca `teman.txt` dan menampilkannya, dengan pesan ramah bila file belum ada (pakai `{$I-}` dan `IOResult`).
3. **MENENGAH** Ubah Program 17.2 agar menanyakan nama file kepada pengguna, bukan memakai konstanta. Uji dengan nama yang ada dan yang tidak ada.
4. **MENENGAH** Buat program yang membaca file teks berisi bilangan bulat, satu per baris, lalu menampilkan jumlah dan rata-ratanya. Siapkan filenya dengan program penulis lebih dulu.
5. **MENENGAH** Tambahkan ke Program 17.4 sebuah perintah yang membaca nomor elemen dari pengguna dan menampilkan isinya dengan `Seek`. Tolak nomor yang di luar `0..FileSize(F)-1`.
6. **TANTANGAN** Kembangkan Program 17.5: setelah menampilkan daftar, hitung nilai rata-rata kelas dan tampilkan siswa dengan nilai tertinggi.
7. **TANTANGAN** Buat program yang menyalin sebuah file teks ke file lain baris demi baris. Setelah itu tambahkan nomor baris di setiap baris hasil salinan.

Pointer dan Linked List

TUJUAN PEMBELAJARAN

- Membedakan variabel biasa dari *pointer*: yang satu menyimpan data, yang lain menyimpan alamat data.
- Memakai tipe pointer `^T`, operator alamat `@`, dereferensi `P^`, dan nilai `nil`.
- Memesan dan mengembalikan memori selama program berjalan dengan `New` dan `Dispose`.
- Menjelaskan struktur simpul (*node*) berisi `Data` dan `Berikut`, serta rantai simpul yang disebut *linked list*.
- Menambah simpul di depan dan di belakang list, serta menelusuri list dengan perulangan `while P <> nil`.
- Menghapus simpul di awal, tengah, atau akhir list dengan menyambung simpul sebelumnya ke simpul sesudahnya.

PROGRAM DALAM BAB INI

18.1 `bab18_01_pointer_dasar.pas`
Pointer

18.2 `bab18_02_new_dispose.pas`
New dan Dispose

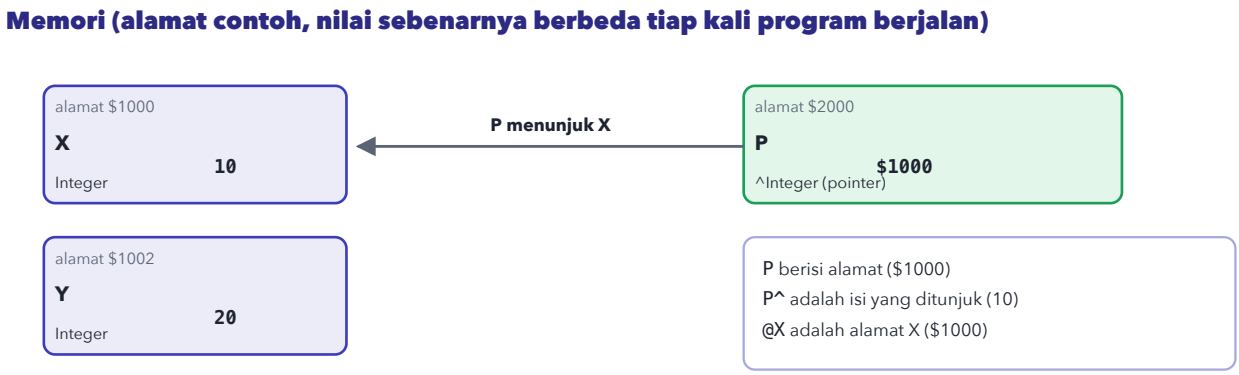
18.3 `bab18_03_linked_list_tambah.pas`
Linked list: menambah simpul

18.4 `bab18_04_linked_list_hapus.pas`
Linked list: menghapus simpul

Sampai bab ini, setiap variabel sudah ditentukan jumlah dan ukurannya sejak program ditulis. Array dengan 100 elemen akan selalu 100 elemen, walau yang terpakai hanya tiga. Bab ini memperkenalkan **pointer**, alat untuk meminta memori saat program berjalan dan menyambung data satu sama lain. Dengan pointer Anda dapat membangun *linked list*, struktur data yang tumbuh dan menyusut mengikuti kebutuhan. Konsepnya terasa abstrak pada awalnya, karena itu bab ini banyak memakai gambar. Pelan-pelan saja, dan gambarlah sendiri setiap langkah di kertas.

18.1 Variabel Menyimpan Data, Pointer Menyimpan Alamat

Setiap variabel menempati tempat tertentu di memori, dan setiap tempat itu punya **alamat**, yaitu nomor yang mengidentifikasi letaknya, seperti nomor rumah di sebuah jalan. Variabel biasa menyimpan **isi** di rumah itu. **Pointer** (penunjuk) menyimpan **alamat** rumah lain. Analogi yang berguna: sebuah kartu nama berisi alamat sebuah toko. Kartu itu bukan tokonya, tetapi dengan kartu itu Anda tahu ke mana harus pergi, dan bila alamat di kartu diganti, Anda diarahkan ke toko lain.



Gambar 18.1 Variabel `X` menyimpan angka 10. Pointer `P` menyimpan alamat `X`, bukan angka 10.

Sebuah pointer selalu **bertipe**: pointer ke `Integer`, pointer ke `record`, dan seterusnya. Tipe ini memberi tahu kompiler bagaimana memperlakukan data di alamat itu. Pascal memakai tiga notasi yang harus dihafalkan.

Tabel 18.1 Notasi pointer di Pascal

Notasi	Dibaca	Arti
`^Integer`	"pointer ke Integer"	Tipe pointer; sering diberi nama, mis. `PInteger = ^Integer;`
`@X`	"alamat X"	Operator yang menghasilkan alamat variabel `X`
`P^`	"isi yang ditunjuk P"	Dereferensi: mengakses data di alamat yang disimpan `P`
`nil`	"nihil"	Nilai pointer yang tidak menunjuk ke mana pun

BENTUK UMUM

```
type
  PInteger = ^Integer;          { tipe pointer, nama diawali P }
var
  P: PInteger;

P := @X;                      { P berisi alamat X }
P^ := 5;                      { ubah data yang ditunjuk P }
P := nil;                     { P tidak menunjuk apa pun }
```

7 **ATURAN TURBO PASCAL 7**

Dalam Odin Turbo Pascal di macOS 64-bit, sebuah pointer berukuran **8 byte**, sedangkan `Integer` tetap 2 byte. Jangan berasumsi ukuran pointer sama dengan ukuran `Integer` seperti di DOS 16-bit. Nilai alamat yang tampil di debugger juga berbeda di setiap komputer dan setiap kali program dijalankan.

18.2 Pointer ke Variabel yang Ada

Program pertama memakai pointer dengan cara yang paling sederhana: menunjuk ke variabel biasa yang sudah ada. Buka `bab18_01_pointer_dasar.pas` lewat `File ▸ Examples...` dan jalankan. Perhatikan bagaimana nilai `X` berubah padahal tidak ada pernyataan yang menulis ke `X` secara langsung.

Program 18.1 `bab18_01_pointer_dasar.pas` *Pointer* — File ▸ Examples...

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 18.1 — Pointer
  Pointer menyimpan ALAMAT sebuah data di memori, bukan datanya.
    ^Integer   tipe "pointer ke Integer"
    @X         alamat variabel X
    P^         data yang ditunjuk P
    nil        pointer yang tidak menunjuk ke mana pun }

program bab18_01_pointer_dasar;

type
  PInteger = ^Integer;                  { nama tipe pointer, diawali P }

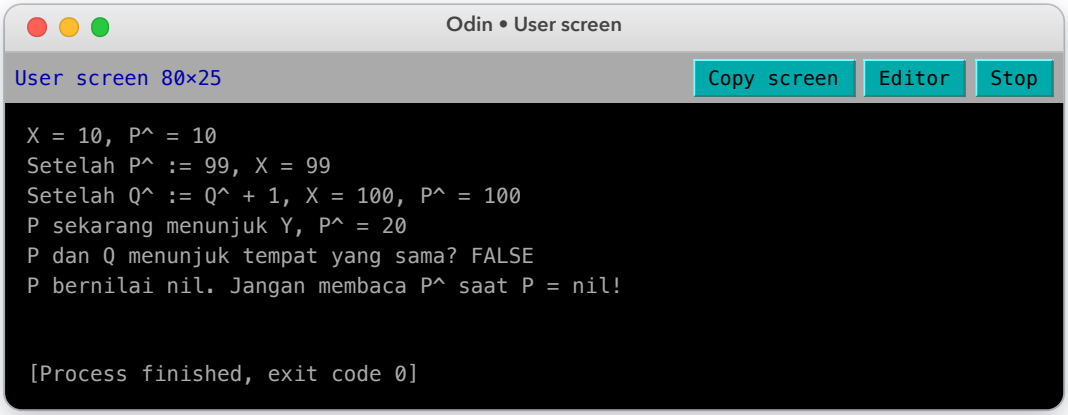
var
  X, Y: Integer;
  P, Q: PInteger;

begin
  X := 10;
  Y := 20;
  P := @X;                              { P menunjuk ke X }
  WriteLn('X = ', X, ', P^ = ', P^);    { P^ membaca isi X }

  P^ := 99;                             { mengubah X lewat pointer }
  WriteLn('Setelah P^ := 99, X = ', X);

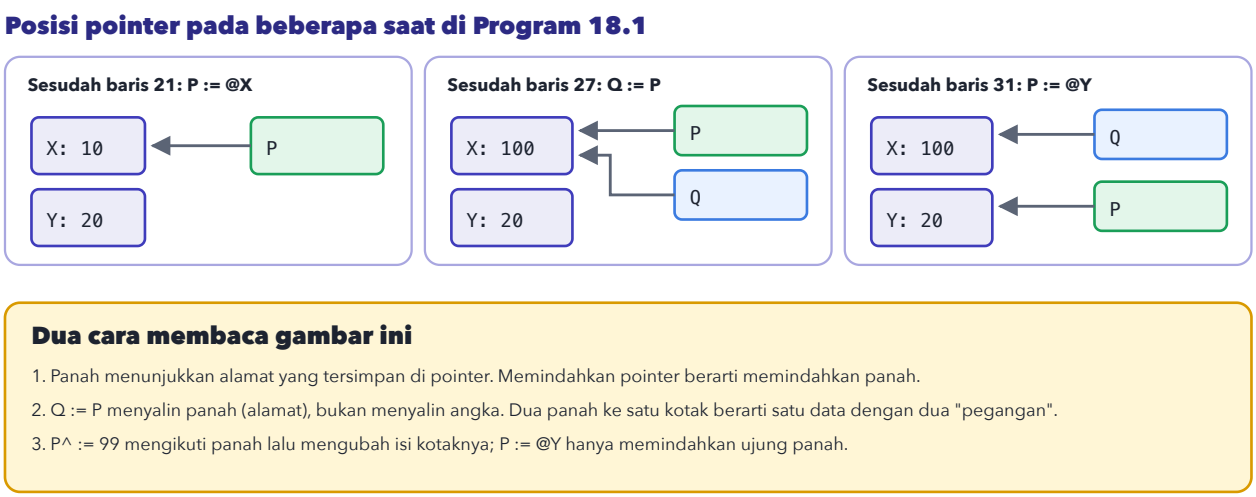
  Q := P;                               { Q menunjuk ke tempat yang sama }
```

```
28    Q^ := Q^ + 1;
29    WriteLn('Setelah Q^ := Q^ + 1, X = ', X, ', P^ = ', P^);
30
31    P := @Y;                              { P pindah menunjuk ke Y }
32    WriteLn('P sekarang menunjuk Y, P^ = ', P^);
33    WriteLn('P dan Q menunjuk tempat yang sama? ', P = Q);
34
35    P := nil;                             { tidak menunjuk ke mana pun }
36    if P = nil then
37      WriteLn('P bernilai nil. Jangan membaca P^ saat P = nil!');
38  end.
```



Gambar 18.2 Mengubah `P^` sama dengan mengubah `X`; dua pointer boleh menunjuk tempat yang sama

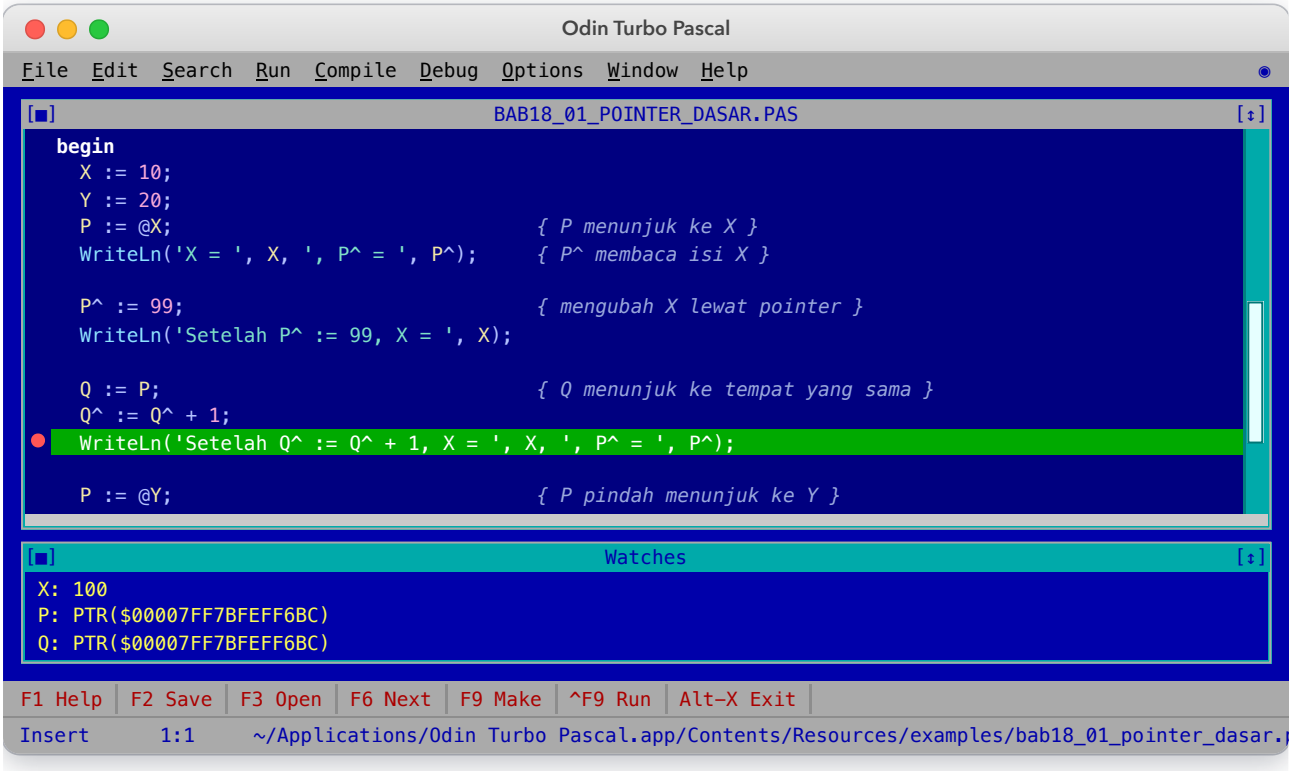
Baris	Penjelasan
11–12	`PInteger = ^Integer` membuat tipe baru: pointer ke `Integer`. Memberi nama tipe pointer (diawali `P`) adalah kebiasaan umum.
15–16	Dua variabel `Integer` (`X`, `Y`) dan dua pointer (`P`, `Q`).
21–22	`P := @X` mengisi `P` dengan alamat `X`. Karena itu `P^` sama dengan isi `X`, yaitu 10.
24–25	`P^ := 99` menulis 99 ke tempat yang ditunjuk `P`, yaitu ke `X`. Nilai `X` berubah menjadi 99.
27–29	`Q := P` menyalin **alamat** (bukan data): kini `P` dan `Q` menunjuk `X`. `Q^ := Q^ + 1` menaikkan `X` menjadi 100, dan `P^` ikut bernilai 100 karena sama-sama menunjuk `X`.
31–33	`P := @Y` memindahkan `P` ke `Y` (`P^` kini 20). `Q` masih menunjuk `X`, jadi pembandingan `P = Q` bernilai `False`: yang dibandingkan adalah alamatnya.
35–37	`P := nil` melepaskan `P` dari `Y`. Memeriksa `P = nil` sebelum memakai `P^` adalah pengaman penting.



Gambar 18.3 Dari kiri ke kanan: `P` menunjuk `X`; `Q` ikut menunjuk `X`; `P` berpindah ke `Y`.

18.2.1 Melihat alamat di debugger

Anda dapat melihat isi pointer yang sebenarnya lewat debugger. Pasang titik henti pada baris yang diinginkan dengan `Debug ▸ Toggle breakpoint`, jalankan dengan `Debug ▸ Start debugging`, lalu tambahkan `X`, `P`, dan `Q` lewat `Debug ▸ Add watch...`. Nilai pointer ditampilkan sebagai alamat bentuk `PTR($...)`. Gambar berikut memperlihatkan bentuk tampilannya; **angka alamat hanyalah contoh**, alamat di komputer Anda akan berbeda.



Gambar 18.4 Debugger berhenti di baris 29: `P` dan `Q` berisi alamat yang sama (angka alamat hanya contoh)

Titik merah di gutter menandai baris yang menghentikan program, dan baris hijau adalah baris yang akan dijalankan berikutnya. Pada jendela **Watches**, nilai `X` berupa angka biasa, sedangkan `P` dan `Q` berupa `PTR($...)`. Dua pointer yang menunjuk tempat yang sama menampilkan alamat yang sama.

Gunakan `Debug ▸ Step over` (`F8`) untuk maju satu baris dan perhatikan alamat di `P` berubah ketika baris 31 selesai, sementara alamat di `Q` tetap. Ini cara terbaik "melihat" panah pointer yang tadi digambar.

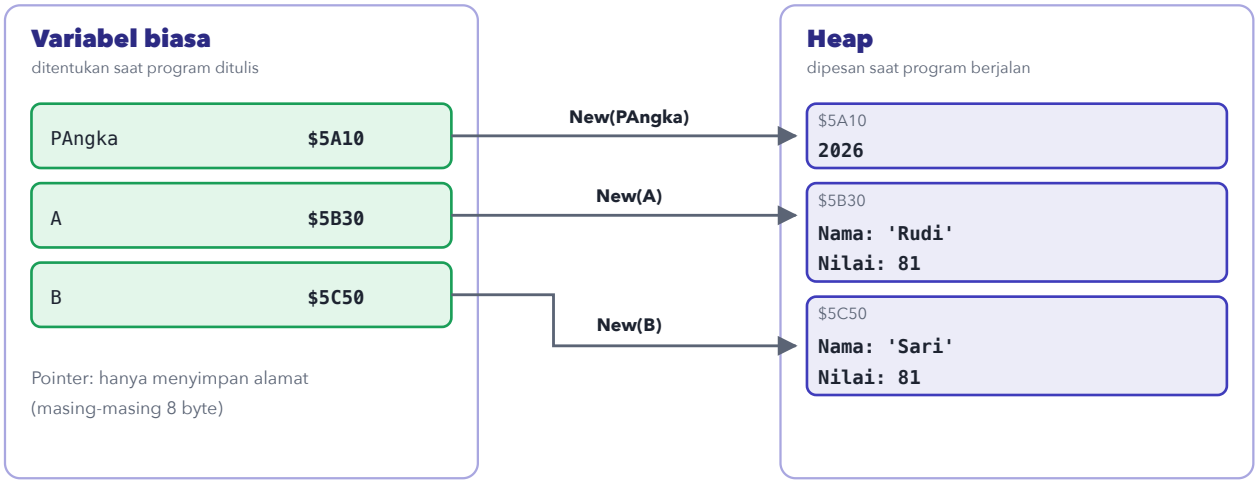
> **AWAS**
>
> Jangan pernah mengakses `P^` saat `P` bernilai `nil` atau berisi alamat sembarang. Program akan berhenti dengan galat (di Turbo Pascal: *runtime error* 216, *general protection fault*) atau, lebih buruk, merusak data lain tanpa pesan. Selalu pastikan pointer menunjuk ke tempat yang sah sebelum diikuti.

18.3 Memori Dinamis: New dan Dispose

Pointer baru benar-benar berguna bila ia menunjuk ke memori yang **dipesan sendiri saat program berjalan**. Memori program terbagi menjadi dua wilayah. Variabel yang Anda umumkan di bagian `var` berada di wilayah yang diatur otomatis. Wilayah lain disebut ***heap*** (tumpukan bebas): tempat memori bisa dipesan kapan saja dengan `New` dan dikembalikan kapan saja dengan `Dispose`. Analoginya penyewaan: `New` menyewa kamar dan memberi Anda kuncinya (pointer); `Dispose` mengembalikan kamar itu ke pengelola.

BENTUK UMUM

```
New(P);        { pesan memori seukuran tipe yang ditunjuk P; P menunjuk ke sana }
P^ := nilai;    { isi memori itu }
Dispose(P);    { kembalikan memori ke heap }
```



Gambar 18.5 Setiap `New` membuat kotak baru di heap dan mengisi pointernya dengan alamat kotak itu (alamat hanya contoh).

Program kedua memakai `New` untuk satu `Integer` dan untuk dua `record`. Untuk pointer ke `record`, field diakses dengan `P^.Field`: ikuti panah dahulu (`P^`), lalu ambil field-nya. Buka `bab18_02_new_dispose.pas` dan jalankan.

Program 18.2 `bab18_02_new_dispose.pas` *New dan Dispose* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 18.2 — New dan Dispose
  New(P) meminta memori baru selama program berjalan (alokasi
  dinamis) dan membuat P menunjuk ke sana. Dispose(P) mengembalikan
  memori itu. Setiap New harus punya pasangan Dispose. }

program bab18_02_new_dispose;

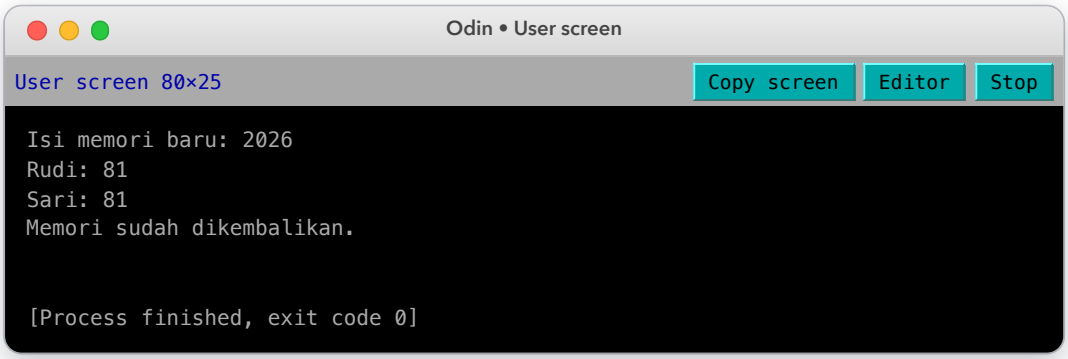
type
  PSiswa = ^TSiswa;
  TSiswa = record
    Nama: string[20];
    Nilai: Integer;
  end;

var
  PAngka: ^Integer;
  A, B: PSiswa;

begin
  New(PAngka);                          { pesan tempat untuk satu Integer }
  PAngka^ := 2026;
  WriteLn('Isi memori baru: ', PAngka^);
  Dispose(PAngka);                      { kembalikan memorinya }
  PAngka := nil;                        { tandai sudah tidak dipakai }

  { Pointer ke record: field diakses dengan P^.Field }
  New(A);
  A^.Nama := 'Rudi';
  A^.Nilai := 81;
  New(B);
  B^ := A^;                             { salin isi record }
  B^.Nama := 'Sari';
  WriteLn(A^.Nama, ': ', A^.Nilai);
  WriteLn(B^.Nama, ': ', B^.Nilai);

  Dispose(A);
  Dispose(B);
  { Setelah Dispose, A dan B masih berisi alamat lama, tetapi memorinya
    bukan milik kita lagi. Jangan memakai A^ atau B^ lagi. }
  WriteLn('Memori sudah dikembalikan.');
end.
```



Gambar 18.6 Dua record di heap: salinan `B` diubah tanpa memengaruhi `A`

Baris	Penjelasan
9–14	`PSiswa = ^TSiswa` ditulis **sebelum** `TSiswa` dideklarasikan. Pascal membolehkan ini khusus untuk tipe pointer; cara ini kelak dipakai untuk simpul yang menunjuk ke tipenya sendiri.
17	`PAngka: ^Integer` mengumumkan pointer tanpa nama tipe terpisah, bentuk lain yang setara.
21–23	`New(PAngka)` memesan tempat satu `Integer` di heap dan mengisi `PAngka` dengan alamatnya. Isi memori itu masih sembarang sampai baris 22 menaruh 2026.
24–25	`Dispose` mengembalikan memori ke heap. `PAngka := nil` menandai bahwa pointer ini tidak lagi menunjuk apa pun; tanpa baris ini, `PAngka` masih berisi alamat lama yang sudah tidak sah.
28–30	`New(A)` memesan satu `TSiswa`. Field diisi lewat `A^.Nama` dan `A^.Nilai`.
31–33	`New(B)` memesan record kedua. `B^ := A^` menyalin **isi** record (berbeda dengan `B := A` yang hanya menyalin alamat). Lalu nama di `B` diganti menjadi `'Sari'`.
34–35	Keluaran membuktikan dua record itu terpisah: `A` tetap Rudi, `B` menjadi Sari, nilai keduanya 81 hasil salinan.
37–38	Setiap `New` punya pasangan `Dispose`: dua `New` untuk record, dua `Dispose`.

AWAS

Aturan emas heap: setiap `New` harus punya pasangan `Dispose`. Memori yang dipesan tetapi tidak pernah dikembalikan disebut *memory leak* (kebocoran memori): program makin besar dan akhirnya kehabisan tempat. Sebaliknya, memakai pointer sesudah `Dispose` disebut *dangling pointer*: isinya tidak lagi milik Anda. Setelah `Dispose(P)`, biasakan menulis `P := nil`.

COBA SENDIRI

Coba sendiri. Ganti baris 32 menjadi `B := A;` (tanpa `^`), lalu jalankan. Kini `A` dan `B` menunjuk record yang sama, sehingga mengubah nama lewat `B` juga mengubah `A^.Nama`. Catatan: record kedua yang dipesan di baris 31 menjadi tidak terjangkau (bocor), dan `Dispose(B)` di baris 38 membebaskan record yang sama dua kali. Ini contoh mengapa `B := A` dan `B^ := A^` jangan dicampuradukkan.

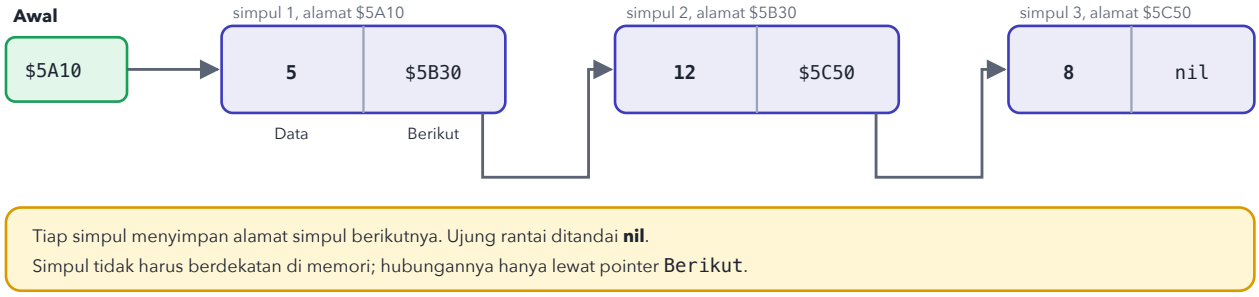
18.4 Simpul dan Linked List

Bagaimana bila Anda perlu menyimpan daftar bilangan tetapi tidak tahu berapa banyaknya? Array mengharuskan Anda menetapkan ukuran lebih dulu; terlalu kecil gagal, terlalu besar boros. ***Linked list*** (senarai berantai) menyelesaikannya dengan merangkai **simpul** (*node*) satu per satu. Setiap simpul berisi dua bagian: **data**, dan **pointer ke simpul berikutnya**. Simpul terakhir menunjuk `nil` sebagai tanda ujung rantai. Satu pointer tambahan, `Awal`, menunjuk simpul pertama; hilangkan pointer ini dan seluruh rantai tidak dapat dicapai lagi.

BENTUK UMUM

```
type
  PSimpul = ^TSimpul;          { boleh ditulis sebelum TSimpul }
  TSimpul = record
    Data: Integer;
    Berikut: PSimpul;          { pointer ke simpul selanjutnya }
  end;
```

`TSimpul` berisi pointer ke `TSimpul` lain. Definisi yang menunjuk dirinya sendiri ini tidak bermasalah, karena yang disimpan hanya alamat (8 byte), bukan seluruh simpul berikutnya.



Gambar 18.7 Linked list `5 → 12 → 8`: `Awal` menunjuk simpul pertama dan simpul terakhir menunjuk `nil`.

Tabel 18.2 Array dan linked list

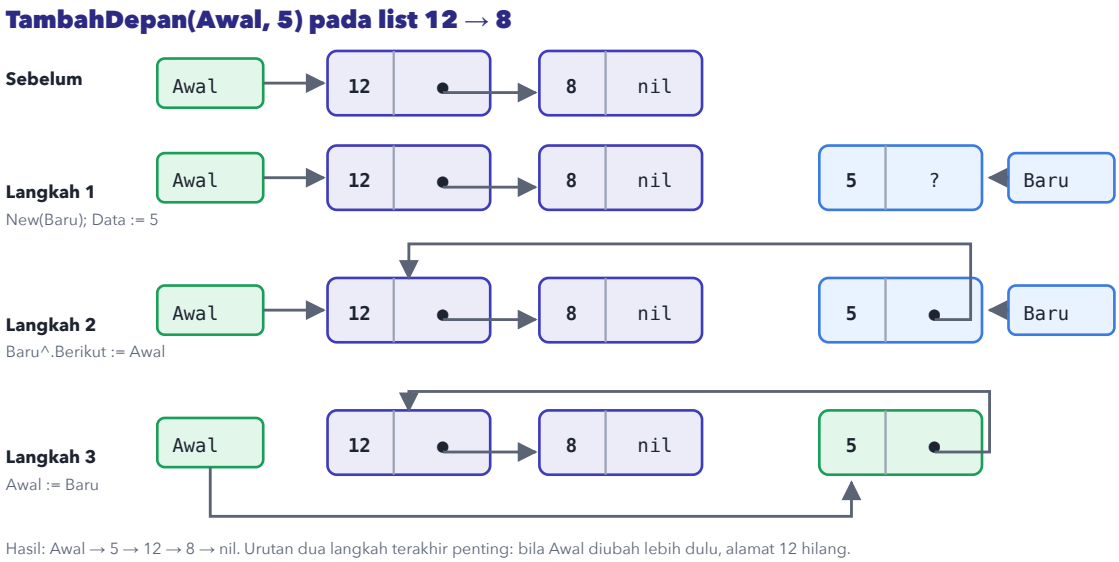
	Array	Linked list
Ukuran	Tetap sejak program ditulis	Tumbuh dan menyusut saat berjalan
Akses elemen ke-*n*	Langsung: `A[n]`	Harus menelusuri dari awal
Menyisip atau menghapus di tengah	Menggeser banyak elemen	Cukup mengubah pointer
Memori	Hanya data	Data ditambah satu pointer per simpul

18.5 Menambah Simpul

Ada dua tempat lazim untuk menambah simpul: di **depan** dan di **belakang** list. Menambah di depan sangat cepat, hanya tiga langkah tanpa menelusuri apa pun. Menambah di belakang harus berjalan dulu sampai simpul terakhir.

Perhatikan prosedur `TambahDepan` dari Program 18.3. Setiap baris di dalamnya adalah satu langkah pada gambar berikut, dan urutannya tidak boleh ditukar.

```
procedure TambahDepan(var Awal: PSimpul; Nilai: Integer);
var
  Baru: PSimpul;
begin
  New(Baru);                    { langkah 1: simpul baru di heap }
  Baru^.Data := Nilai;
  Baru^.Berikut := Awal;        { langkah 2: tunjuk awal lama }
  Awal := Baru;                 { langkah 3: jadi awal baru }
end;
```



Gambar 18.8 Tiga langkah menambah simpul di depan; lingkaran kecil melambangkan pointer `Berikut`.

Prosedur `TambahBelakang` berbeda dari `TambahDepan`: simpul baru harus dipasang ke simpul *terakhir*, yang hanya bisa ditemukan dengan menelusuri rantai sampai `Berikut = nil`. Program ini juga memiliki prosedur `Tampilkan` untuk menelusuri list dan `HapusSemua` untuk mengembalikan seluruh memori. Buka `bab18_03_linked_list_tambah.pas` dan jalankan.

Program 18.3 `bab18_03_linked_list_tambah.pas` *Linked list: menambah simpul* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 18.3 — Linked list: menambah simpul
3    Linked list adalah rantai simpul. Setiap simpul berisi data dan
4    pointer ke simpul berikutnya; simpul terakhir menunjuk nil. Berbeda
5    dengan array, panjangnya bisa bertambah selama program berjalan.
6      Awal -> [5| ] -> [12| ] -> [8|nil] }
7
8  program bab18_03_linked_list_tambah;
9
10 type
11   PSimpul = ^TSimpul;                  { boleh ditulis sebelum TSimpul }
12   TSimpul = record
13     Data: Integer;
14     Berikut: PSimpul;                  { pointer ke simpul selanjutnya }
15   end;
16
17 procedure TambahDepan(var Awal: PSimpul; Nilai: Integer);
18 var
19   Baru: PSimpul;
```

```
20  begin
21    New(Baru);
22    Baru^.Data := Nilai;
23    Baru^.Berikut := Awal;                { simpul baru menunjuk awal lama }
24    Awal := Baru;                         { simpul baru menjadi awal }
25  end;
26
27  procedure TambahBelakang(var Awal: PSimpul; Nilai: Integer);
28  var
29    Baru, P: PSimpul;
30  begin
31    New(Baru);
32    Baru^.Data := Nilai;
33    Baru^.Berikut := nil;                 { akan menjadi simpul terakhir }
34    if Awal = nil then
35      Awal := Baru                        { list masih kosong }
36    else
37    begin
38      P := Awal;
39      while P^.Berikut <> nil do          { jalan sampai simpul terakhir }
40        P := P^.Berikut;
41      P^.Berikut := Baru;
42    end;
43  end;
44
45  procedure Tampilkan(Awal: PSimpul);
46  var
47    P: PSimpul;
48  begin
49    Write('Awal -> ');
50    P := Awal;
51    while P <> nil do
52    begin
53      Write(P^.Data, ' -> ');
54      P := P^.Berikut;                    { pindah ke simpul berikutnya }
55    end;
56    WriteLn('nil');
57  end;
58
59  procedure HapusSemua(var Awal: PSimpul);
60  var
61    P: PSimpul;
62  begin
63    while Awal <> nil do
64    begin
65      P := Awal;
66      Awal := Awal^.Berikut;              { simpan sambungan dulu }
67      Dispose(P);                         { baru simpulnya dibuang }
68    end;
69  end;
70
71  var
72    Daftar: PSimpul;
73
74  begin
75    Daftar := nil;                        { list kosong }
76    Tampilkan(Daftar);
77    TambahBelakang(Daftar, 12);
78    Tampilkan(Daftar);
```

```
79    TambahBelakang(Daftar, 8);
80    Tampilkan(Daftar);
81    TambahDepan(Daftar, 5);
82    Tampilkan(Daftar);
83    TambahDepan(Daftar, 1);
84    Tampilkan(Daftar);
85    HapusSemua(Daftar);                    { kembalikan semua memori }
86    Tampilkan(Daftar);
87  end.
```



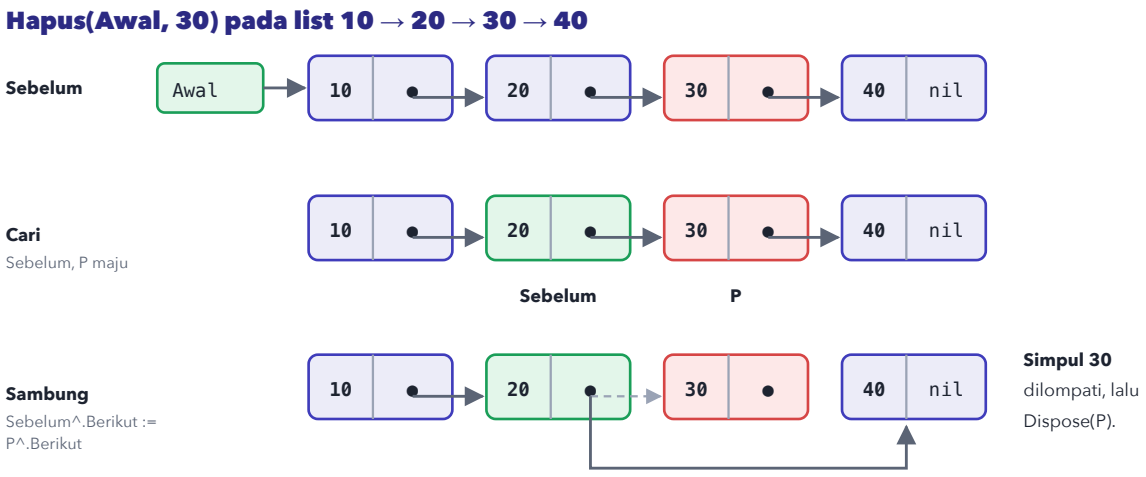
Gambar 18.9 List tumbuh satu simpul tiap kali; pada akhirnya `HapusSemua` mengosongkannya

Baris	Penjelasan
10–15	Tipe simpul seperti pada bentuk umum: `Data` dan pointer `Berikut`.
17	`Awal` adalah parameter `var` karena prosedur akan **mengubah** pointer awal milik pemanggil.
21–24	Tiga langkah dari Gambar di atas: pesan simpul (21), isi data (22), sambungkan ke awal lama (23), jadikan awal baru (24). Berlaku juga untuk list kosong, karena `Awal` yang `nil` menjadi `Berikut` simpul pertama.
31–33	Simpul baru dibuat dan `Berikut`-nya `nil`, karena ia akan menjadi simpul terakhir.
34–35	Kasus khusus: bila list kosong, simpul baru langsung menjadi `Awal`.
37–41	Selain itu, pointer bantu `P` berjalan dari awal selama `P^.Berikut <> nil` sampai berhenti di simpul terakhir, lalu simpul baru dikaitkan di sana.
45–57	`Tampilkan` menelusuri list. Awal `P := Awal`, lalu selama `P <> nil` data dicetak dan `P := P^.Berikut` memindahkannya ke simpul berikutnya. Karena `Awal` di sini parameter nilai, pointer aslinya tidak terganggu.
59–69	`HapusSemua` membuang simpul satu per satu. Urutan di baris 65–67 penting: simpan simpul di `P`, geser `Awal` ke simpul berikutnya, *baru* `Dispose(P)`. Bila `Dispose` lebih dulu, sambungan ke simpul berikutnya ikut hilang.
75–86	Program utama membuat list kosong, lalu bergantian menambah dan menampilkan. `TambahBelakang` 12 dan 8, kemudian `TambahDepan` 5 dan 1 membuat urutan akhir 1, 5, 12, 8. Pemanggilan terakhir `HapusSemua` mengembalikan list ke `nil`.

> ★ **INGAT**
>
> Tiga pola penelusuran yang akan sering Anda tulis: (1) `P := Awal;` (2) ulangi `while P <> nil do` (3) di akhir putaran `P := P^.Berikut`. Bila Anda lupa baris (3), perulangan tidak pernah berhenti.

18.6 Menghapus Simpul

Menghapus simpul dari tengah list tidak boleh sembarangan: simpul sebelum dan sesudahnya harus tetap tersambung. Triknya adalah mengingat **simpul sebelumnya** selagi menelusuri, lalu menyambungkannya langsung ke simpul **sesudahnya** sehingga simpul yang dihapus "dilompati". Setelah itu barulah simpul itu dikembalikan dengan `Dispose`. Simpul pertama adalah kasus khusus, karena tidak ada simpul sebelumnya; yang diubah adalah `Awal` itu sendiri.



Gambar 18.10 Menghapus simpul 30: simpul sebelumnya (20) disambung langsung ke simpul sesudahnya (40), baru simpul 30 dikembalikan.

Fungsi `Hapus` pada program berikut menerapkannya dan mengembalikan `True` bila nilai ditemukan. Program utama membuat list 10, 20, 30, 40, 50, lalu menghapus nilai di tengah, di awal, di akhir, nilai yang tidak ada, dan akhirnya mengosongkan list. Buka `bab18_04_linked_list_hapus.pas` dan jalankan.

Program 18.4 `bab18_04_linked_list_hapus.pas` *Linked list: menghapus simpul* File ▸ Examples…

```
{ Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 18.4 – Linked list: menghapus simpul
  Untuk membuang simpul, sambungkan simpul SEBELUMNYA langsung ke
  simpul SESUDAHNYA, lalu Dispose simpul itu. Simpul pertama
  ditangani khusus karena tidak punya simpul sebelumnya. }

program bab18_04_linked_list_hapus;

type
  PSimpul = ^TSimpul;
  TSimpul = record
    Data: Integer;
```

```
    Berikut: PSimpul;
  end;

procedure TambahBelakang(var Awal: PSimpul; Nilai: Integer);
var
  Baru, P: PSimpul;
begin
  New(Baru);
  Baru^.Data := Nilai;
  Baru^.Berikut := nil;
  if Awal = nil then
    Awal := Baru
  else
  begin
    P := Awal;
    while P^.Berikut <> nil do
      P := P^.Berikut;
    P^.Berikut := Baru;
  end;
end;

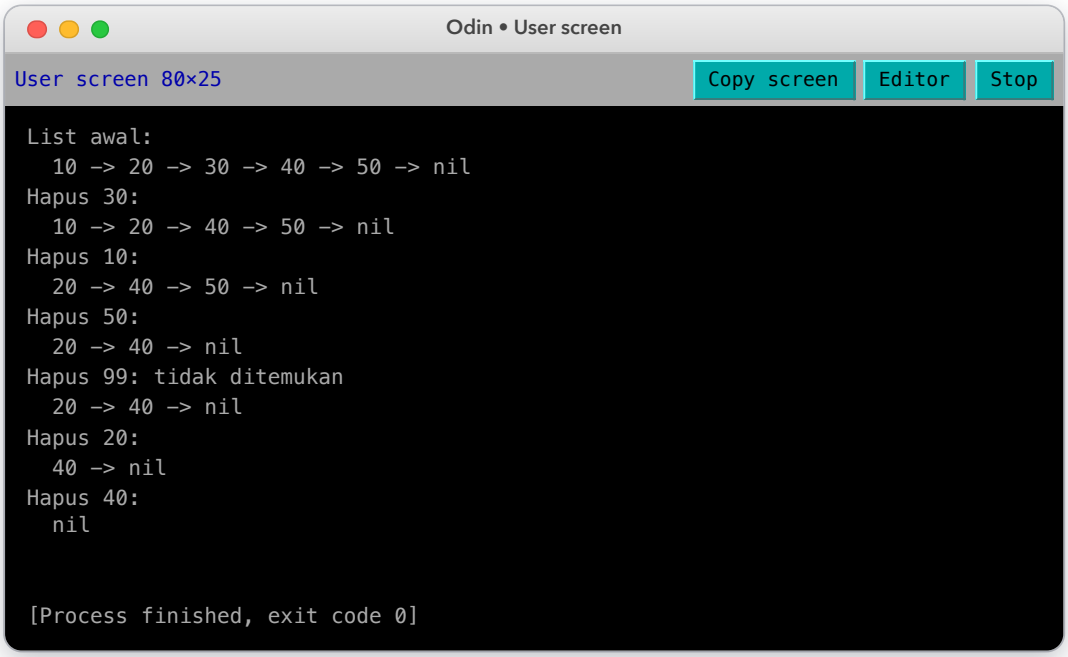
procedure Tampilkan(Awal: PSimpul);
begin
  Write('  ');
  while Awal <> nil do                    { Awal di sini hanya salinan }
  begin
    Write(Awal^.Data, ' -> ');
    Awal := Awal^.Berikut;
  end;
  WriteLn('nil');
end;

{ Menghapus simpul pertama yang berisi Nilai; False bila tidak ada }
function Hapus(var Awal: PSimpul; Nilai: Integer): Boolean;
var
  Sebelum, P: PSimpul;
begin
  Sebelum := nil;
  P := Awal;
  while (P <> nil) and (P^.Data <> Nilai) do
  begin
    Sebelum := P;                         { ingat simpul sebelumnya }
    P := P^.Berikut;
  end;

  if P = nil then
    Hapus := False                        { tidak ditemukan }
  else
  begin
    if Sebelum = nil then
      Awal := P^.Berikut                  { yang dihapus simpul pertama }
    else
      Sebelum^.Berikut := P^.Berikut;     { lompati simpul P }
    Dispose(P);
    Hapus := True;
  end;
end;

procedure Coba(var Awal: PSimpul; Nilai: Integer);
```

```
72  begin
73    if Hapus(Awal, Nilai) then
74      WriteLn('Hapus ', Nilai, ':')
75    else
76      WriteLn('Hapus ', Nilai, ': tidak ditemukan');
77    Tampilkan(Awal);
78  end;
79
80  var
81    Daftar: PSimpul;
82    I: Integer;
83
84  begin
85    Daftar := nil;
86    for I := 1 to 5 do
87      TambahBelakang(Daftar, I * 10);
88    WriteLn('List awal:');
89    Tampilkan(Daftar);
90
91    Coba(Daftar, 30);                    { simpul di tengah }
92    Coba(Daftar, 10);                    { simpul pertama }
93    Coba(Daftar, 50);                    { simpul terakhir }
94    Coba(Daftar, 99);                    { tidak ada }
95    Coba(Daftar, 20);
96    Coba(Daftar, 40);                    { list menjadi kosong }
97  end.
```



Gambar 18.11 Enam kali penghapusan: tengah, awal, akhir, nilai yang tidak ada, lalu sampai list kosong

Baris	Penjelasan
34–43	`Tampilkan` kali ini memakai parameternya sendiri (`Awal`) sebagai penelusur. Aman karena parameter nilai hanyalah salinan pointer; pointer asli milik pemanggil tidak ikut bergeser.
46–48	`Hapus` menerima `var Awal` karena penghapusan simpul pertama mengubah `Awal`. Ada dua pointer bantu: `Sebelum` dan `P`.

Baris	Penjelasan
50–56	Penelusuran: `Sebelum` dimulai `nil`, `P` dari awal. Selama belum ketemu, `Sebelum` mengikuti `P` satu langkah di belakangnya. Syarat `(P <> nil) and (...)` mengamankan akses `P^.Data`.
58–59	Bila `P` sampai `nil`, nilai tidak ditemukan dan fungsi mengembalikan `False`.
62–65	Kasus pertama: `Sebelum = nil` berarti yang dihapus simpul awal, jadi `Awal := P^.Berikut`. Kasus kedua: simpul `Sebelum` disambung ke `P^.Berikut`, melompati `P`.
66–67	Barulah `Dispose(P)` dan hasil `True` diberikan.
71–78	`Coba` membungkus pemanggilan `Hapus` dengan pesan hasilnya dan menampilkan list sesudahnya.
84–96	Program utama mengisi list 10..50 dengan perulangan, lalu menjalankan enam skenario berurutan. Perhatikan hasil `Hapus 99` yang tidak ditemukan dan hasil terakhir saat list menjadi `nil`.

7 **ATURAN TURBO PASCAL 7**

Fungsi `Hapus` mengembalikan nilai dengan memberi nilai ke **nama fungsinya** (`Hapus := True`), bukan ke `Result`, karena `Result` tidak ada di Turbo Pascal 7.

18.7 Kesalahan yang Sering Terjadi

18.7.1 Mengisi pointer dengan nilai, bukan alamat

ptrangka.pas

```
1  program PtrAngka;
2  var
3    P: ^Integer;
4  begin
5    P := 5;
6    WriteLn(P^);
7  end.
```

PESAN KOMPILER

```
ptrangka.pas(5,8) Error: Tipe tidak cocok: ShortInt dan ^Integer
```

Gambar 18.12 Pointer tidak boleh diisi bilangan; isilah dengan alamat atau New

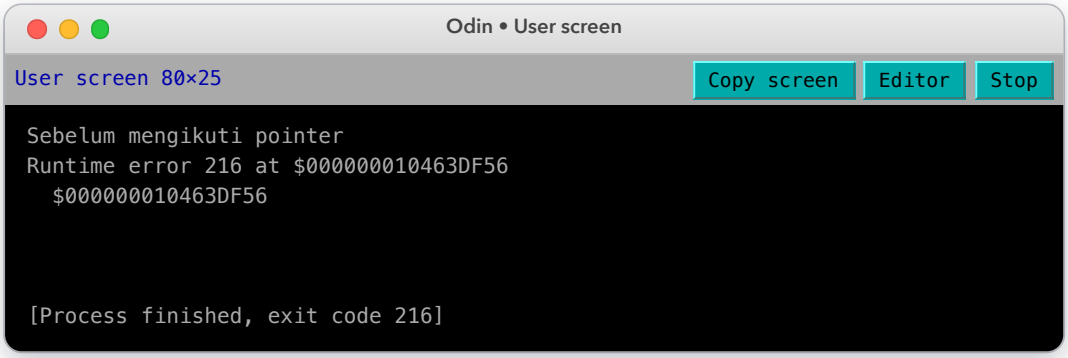
Kompiler menolak karena `5` adalah bilangan bulat biasa, sedangkan `P` adalah pointer. Bila maksud Anda mengisi data yang ditunjuk, tulis `P^ := 5` setelah `P` menunjuk tempat yang sah (`New(P)` atau `P := @Sesuatu`).

18.7.2 Mengikuti pointer nil

Ini kesalahan paling sering saat bekerja dengan list: menelusuri sampai melewati ujung, atau memakai pointer yang belum diisi. Program berikut memakai pointer yang bernilai `nil`.

nilptr.pas

```
1  program NilPtr;
2  var
3    P: ^Integer;
4  begin
5    P := nil;
6    WriteLn('Sebelum mengikuti pointer');
7    WriteLn(P^);
8    WriteLn('Tidak akan sampai sini');
9  end.
```



Gambar 18.13 Mengikuti pointer nil menghentikan program (*runtime error* 216)

Kompilasinya lancar, tetapi program berhenti pada baris 7 dengan galat *runtime* 216. Kode galat ini berarti program mengakses alamat yang tidak sah. Pada list, sumber galat yang sama adalah `P^.Berikut` ketika `P` sudah `nil`. Pengamannya: periksa `P <> nil` sebelum mengikuti panah, seperti pada perulangan `while P <> nil` di Program 18.3 dan `(P <> nil) and (P^.Data <> Nilai)` di Program 18.4.

18.7.3 Kesalahan yang diam: bocor dan menggantung

Dua kesalahan berikut tidak menghasilkan pesan galat sehingga sulit ditemukan.

! Memori bocor

Mengubah `Awal := nil` atau `P := Baru` tanpa `Dispose` membuat simpul lama tak terjangkau. Memorinya tidak bisa dikembalikan lagi. Hapus list dengan `HapusSemua` sebelum program selesai.

! Pointer menggantung

Memakai `P^` setelah `Dispose(P)` membaca memori yang bukan milik Anda. Hasilnya tidak terduga. Tulis `P := nil` sesudah `Dispose`, dan jangan menyimpan salinan pointer yang sudah dibuang.

18.7.4 Urutan langkah yang terbalik

Pada `TambahDepan`, bila `Awal := Baru` ditulis *sebelum* `Baru^.Berikut := Awal`, simpul baru menunjuk dirinya sendiri dan seluruh list lama terputus. Pada `HapusSemua`, `Dispose(P)` sebelum `Awal := Awal^.Berikut` membuat Anda mengikuti pointer ke memori yang sudah dibuang. Gambarlah panah di kertas sebelum menulis kodenya; urutan yang benar biasanya tampak jelas dari gambar.

18.8 Rangkuman

- Variabel biasa menyimpan **data**; **pointer** menyimpan **alamat** data. Tipe pointer ditulis `^Tipe` dan ukurannya 8 byte di macOS 64-bit.
- `@X` menghasilkan alamat `X`; `P^` mengakses data yang ditunjuk `P`; `nil` berarti tidak menunjuk apa pun.
- `Q := P` menyalin alamat (dua penunjuk, satu data); `Q^ := P^` menyalin isi.
- `New(P)` memesan memori di *heap* dan `Dispose(P)` mengembalikannya. Setiap `New` harus punya `Dispose`; sesudahnya isi `P := nil`.
- Simpul linked list adalah `record` berisi data dan pointer `Berikut`. Pointer `Awal` menunjuk simpul pertama; simpul terakhir menunjuk `nil`.
- Menambah di depan: simpul baru menunjuk `Awal` lama, lalu menjadi `Awal`. Menambah di belakang: telusuri sampai simpul terakhir dan kaitkan di sana.
- Menghapus: ingat simpul `Sebelum`, sambungkan ke `P^.Berikut`, baru `Dispose(P)`. Simpul pertama dihapus dengan mengubah `Awal`.
- Parameter yang akan mengubah pointer awal list harus `var`. Debugger menampilkan pointer sebagai `PTR($...)`.

18.9 Latihan

1. **DASAR** Deklarasikan `X: Integer` dan pointer `P` ke `Integer`. Buat `P` menunjuk `X`, ubah `X` lewat `P^`, lalu tampilkan keduanya. Gambar kotak dan panah untuk setiap langkah.
2. **DASAR** Pesan satu `Real` dengan `New`, isi dengan hasil perhitungan (mis. luas lingkaran), tampilkan dengan format `:0:2`, lalu `Dispose`.
3. **MENENGAH** Tambahkan fungsi `Hitung(Awal): Integer` pada Program 18.3 yang mengembalikan banyak simpul, dan fungsi `Jumlah(Awal): Integer` untuk menjumlahkan data.
4. **MENENGAH** Buat fungsi `Cari(Awal, Nilai): Boolean` yang menelusuri list dan mengembalikan `True` bila nilai ada. Pastikan aman untuk list kosong.
5. **MENENGAH** Tulis prosedur `SisipUrut(var Awal; Nilai)` yang menyisipkan simpul pada tempat yang benar agar list selalu terurut naik. Pertimbangkan tiga kasus: list kosong, nilai paling kecil (disisip di depan), dan nilai di tengah atau akhir.
6. **TANTANGAN** Tulis prosedur `Balik(var Awal)` yang membalik urutan list tanpa membuat simpul baru, hanya mengubah pointer `Berikut`. Gunakan tiga pointer bantu.
7. **TANTANGAN** Gabungkan Bab 17 dan 18: simpan isi list ke file bertipe `file of Integer`, lalu bacalah kembali ke list baru. Mengapa pointer `Berikut` tidak boleh ikut disimpan ke file?

BAB 19

Unit dan Layar Crt

TUJUAN PEMBELAJARAN

- Menjelaskan fungsi *unit* dan perbedaan bagian `interface` dengan `implementation`.
- Menulis unit sendiri, menyimpannya sebagai berkas yang namanya sama dengan nama unit, lalu memakainya dengan `uses`.
- Mengatur Primary file di Odin Turbo Pascal agar program dan unit dikompilasi bersama.
- Memakai unit `Crt`: `ClrScr`, `GotoXY`, `WhereX` / `WhereY`, `TextColor`, `TextBackground`, `Delay`, dan `ReadKey`.
- Membaca koordinat layar teks 80 kolom x 25 baris dan 16 warna Crt.
- Membangun menu berwarna yang dioperasikan dengan tombol panah, **Enter**, dan **Esc**.

PROGRAM DALAM BAB INI

19.1 `bab19_01_pakai_unit_sendiri.pas`
Memakai unit buatan sendiri

19.2 `bab19_02_unit_crt.pas`
Unit Crt

19.3 `bab19_03_menu_berwarna.pas`
Menu berwarna

Unit `matematika.pas`
Unit buatan sendiri: matematika

Selama ini seluruh program Anda berada dalam satu berkas, dari baris pertama sampai `end.`. Untuk program kecil itu cukup. Namun begitu Anda punya fungsi yang berguna di banyak program, menyalin dan menempelkannya berulang kali jelas bukan cara yang sehat. Pascal menyediakan **unit**: berkas terpisah berisi kumpulan konstanta, prosedur, dan fungsi yang dapat dipakai program mana saja. Bab ini juga memperkenalkan unit bawaan yang paling populer pada masa Turbo Pascal, yaitu `Crt`, untuk mengatur layar teks dan keyboard.

19.1 Mengapa Memakai Unit

Bayangkan sebuah dapur restoran. Koki tidak membuat tepung dari gandum setiap kali memasak; ia mengambilnya dari gudang. Unit adalah gudang itu: tempat menyimpan "bahan jadi" (fungsi dan prosedur yang sudah teruji) yang bisa diambil oleh banyak program. Dengan unit, Anda mendapat tiga keuntungan.

1 Dipakai ulang
Fungsi ditulis dan diuji sekali, lalu dipanggil dari program mana pun dengan satu baris `uses`.

2 Program lebih pendek
Program utama hanya berisi alur kerja. Rincian hitungan tersembunyi di dalam unit.

3 Terbagi rapi
Setiap unit mengurus satu tema, misalnya matematika atau tampilan layar. Mencari dan memperbaiki kode menjadi mudah.

Anda sebenarnya sudah memakai unit sejak Bab 1. Fungsi `WriteLn`, `Length`, dan `Sqrt` berasal dari unit `System` yang otomatis disertakan dalam setiap program. Yang baru di bab ini: membuat unit sendiri, dan memakai unit lain yang harus disebut dengan `uses`.

19.2 Anatomi Sebuah Unit

Sebuah unit selalu terbagi menjadi dua bagian besar. Bagian **`interface`** adalah "etalase": daftar nama yang boleh dilihat dan dipakai program lain. Bagian **`implementation`** adalah "dapur": isi lengkap setiap prosedur dan fungsi. Program yang memakai unit hanya bisa melihat etalase; dapurnya tertutup.

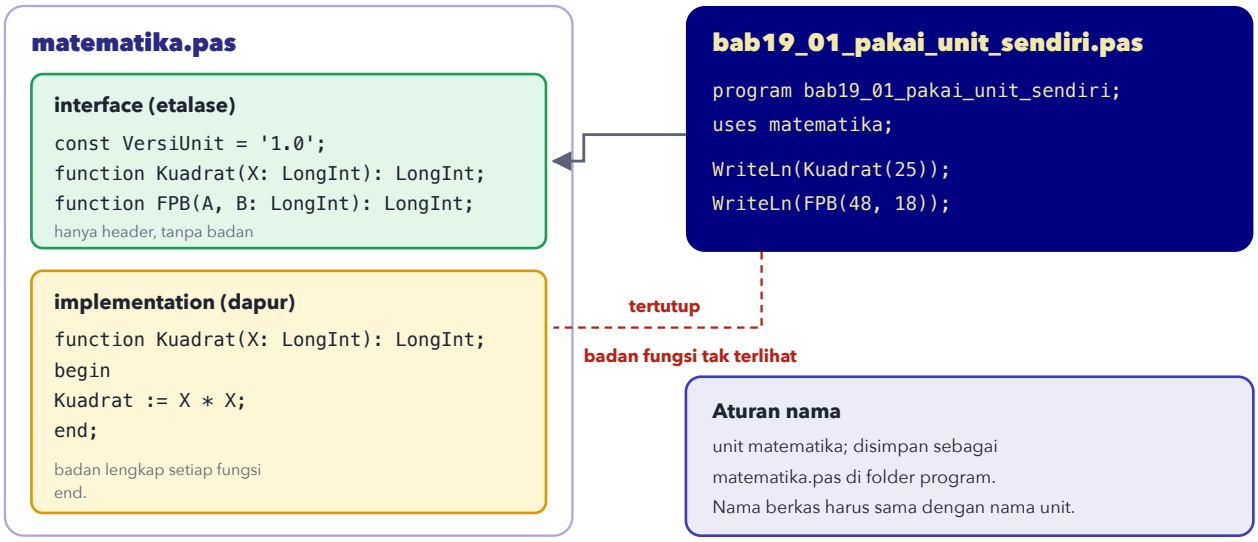
BENTUK UMUM

```
unit NamaUnit;

interface
  { const, type, var, dan HEADER prosedur/fungsi yang boleh dipakai pihak luar }

implementation
  { badan lengkap setiap prosedur/fungsi yang diumumkan di interface }
  { boleh juga ada prosedur pribadi yang tidak diumumkan }

end.
```



Gambar 19.1 Unit `matematika` : program hanya memakai apa yang diumumkan di `interface` .

Alasan pemisahan ini mirip merek dagang: pembeli cukup tahu *apa* yang bisa dilakukan sebuah mesin kopi (menekan tombol, keluar kopi), tanpa perlu tahu *bagaimana* bagian dalamnya bekerja. Pembuat unit bebas mengubah isi `implementation` selama header di `interface` tidak berubah, dan program yang memakainya tetap berjalan.

> ★ **INGAT**
>
> Setiap prosedur atau fungsi yang diumumkan di `interface` **wajib** punya badan di `implementation` , dengan nama dan daftar parameter yang sama. Hal ini serupa dengan `forward` yang Anda kenal pada Bab 9, hanya saja untuk seluruh berkas.

19.3 Membuat dan Memakai Unit matematika

Unit pertama kita menyimpan fungsi matematika yang sudah Anda tulis di bab-bab sebelumnya: kuadrat, pangkat, faktorial, bilangan prima, FPB, dan KPK. Buka `matematika.pas` lewat `File ▸ Examples...` untuk melihat isinya.

Unit `matematika.pas` *Unit buatan sendiri: matematika* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 19 — Unit buatan sendiri: matematika
3    Unit adalah kumpulan konstanta, prosedur, dan fungsi yang bisa
4    dipakai banyak program dengan "uses matematika;". Nama file harus
5    sama dengan nama unit. Unit tidak bisa dijalankan sendiri: buka dan
6    jalankan bab19_01_pakai_unit_sendiri.pas. }
7
8  unit matematika;
9
10 interface   { bagian umum: daftar yang boleh dipakai program lain }
11
12 const
13   VersiUnit = '1.0';
14
```

```
15  function Kuadrat(X: LongInt): LongInt;
16  function Pangkat(A: LongInt; N: Integer): LongInt;
17  function Faktorial(N: Integer): LongInt;
18  function Prima(N: LongInt): Boolean;
19  function FPB(A, B: LongInt): LongInt;
20  function KPK(A, B: LongInt): LongInt;
21
22  implementation  { bagian dalam: isi lengkap setiap fungsi }
23
24  function Kuadrat(X: LongInt): LongInt;
25  begin
26    Kuadrat := X * X;
27  end;
28
29  function Pangkat(A: LongInt; N: Integer): LongInt;
30  var
31    I: Integer;
32    Hasil: LongInt;
33  begin
34    Hasil := 1;
35    for I := 1 to N do
36      Hasil := Hasil * A;
37    Pangkat := Hasil;
38  end;
39
40  function Faktorial(N: Integer): LongInt;
41  begin
42    if N <= 1 then
43      Faktorial := 1
44    else
45      Faktorial := N * Faktorial(N - 1);
46  end;
47
48  function Prima(N: LongInt): Boolean;
49  var
50    Pembagi: LongInt;
51  begin
52    Prima := N >= 2;
53    Pembagi := 2;
54    while Pembagi * Pembagi <= N do
55    begin
56      if N mod Pembagi = 0 then
57      begin
58        Prima := False;
59        Exit;
60      end;
61      Pembagi := Pembagi + 1;
62    end;
63  end;
64
65  function FPB(A, B: LongInt): LongInt;
66  var
67    Sisa: LongInt;
68  begin
69    while B <> 0 do                    { algoritma Euclid, Bab 8.2 }
70    begin
71      Sisa := A mod B;
72      A := B;
73      B := Sisa;
```

```
74    end;
75    FPB := A;
76  end;
77
78  function KPK(A, B: LongInt): LongInt;
79  begin
80    KPK := A div FPB(A, B) * B;            { memakai fungsi lain di unit ini }
81  end;
82
83  end.
```

Perhatikan hal-hal berikut dalam listing di atas.

Baris	Penjelasan
8	`unit matematika;` menggantikan `program ...;`. Nama unit harus sama persis (tanpa memedulikan huruf besar/kecil) dengan nama berkas tanpa `.pas`: berkasnya `matematika.pas`.
10	Awal bagian `interface`. Semua yang ditulis sampai kata `implementation` terlihat oleh program pemakai.
12–13	Konstanta `VersiUnit` ikut diumumkan, jadi program pemakai bisa membacanya.
15–20	Enam header fungsi. Hanya baris pertama deklarasi, diakhiri titik koma, tanpa `begin..end`.
22	Awal bagian `implementation`: dari sini sampai `end.` adalah isi lengkap.
24–27	Badan `Kuadrat`. Header diulang lengkap, lalu hasil diberikan dengan `Kuadrat := X * X`.
29–38	`Pangkat` mengalikan `Hasil` dengan `A` sebanyak `N` kali. Variabel `I` dan `Hasil` bersifat lokal.
40–46	`Faktorial` memakai rekursi seperti di Bab 10.
48–63	`Prima` mencoba pembagi dari 2 selama `Pembagi * Pembagi <= N`. Begitu ada yang habis membagi, hasil diisi `False` dan `Exit` menghentikan fungsi.
65–76	`FPB` dengan algoritma Euclid (Bab 8.2).
78–81	`KPK` memanggil `FPB`, yaitu fungsi lain di unit yang sama. Dalam `implementation`, fungsi yang sudah ditulis lebih dulu boleh dipanggil fungsi sesudahnya.
83	`end.` dengan titik menutup unit. Tidak ada blok `begin` untuk program utama.

AWAS

Unit tidak punya program utama, sehingga **tidak dapat dijalankan sendiri**. Menekan Ctrl + F9 ketika `matematika.pas` aktif hanya mengompilasinya menjadi berkas objek, tanpa executable. Yang dijalankan adalah program yang memakainya.

19.3.1 Memanggil unit dengan uses

Program yang ingin memakai unit menulis `uses` sesudah judul program dan sebelum deklarasi lain. Setelah itu semua nama di bagian `interface` unit dapat dipakai seolah-olah ditulis di program itu sendiri.

BENTUK UMUM

```
program NamaProgram;

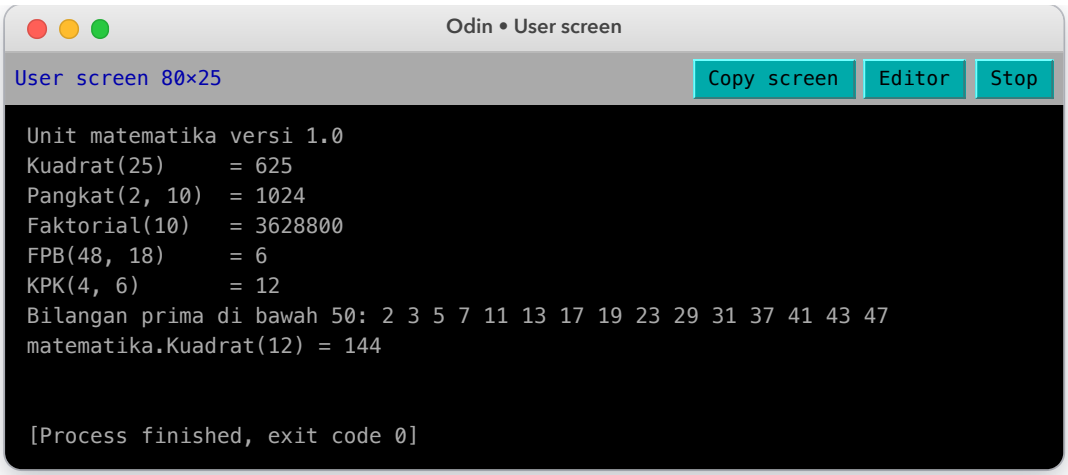
uses
  Unit1, Unit2;       { boleh lebih dari satu, dipisah koma }

begin
  ...
end.
```

Buka `bab19_01_pakai_unit_sendiri.pas`. Program ini memanggil semua fungsi unit, lalu mencetak bilangan prima di bawah 50.

Program 19.1 **bab19_01_pakai_unit_sendiri.pas** *Memakai unit buatan sendiri* File ▸ Examples…

```
1  { Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
2  { Bab 19.1 — Memakai unit buatan sendiri
3    "uses matematika;" membuat semua yang ada di bagian interface unit
4    matematika.pas bisa dipakai di sini. Kompiler mencari file unit di
5    folder yang sama dengan program ini. }
6
7  program bab19_01_pakai_unit_sendiri;
8
9  uses
10   matematika;                          { unit dari file matematika.pas }
11
12 var
13   I: Integer;
14
15 begin
16   WriteLn('Unit matematika versi ', VersiUnit);   { konstanta dari unit }
17   WriteLn('Kuadrat(25)     = ', Kuadrat(25));
18   WriteLn('Pangkat(2, 10)  = ', Pangkat(2, 10));
19   WriteLn('Faktorial(10)   = ', Faktorial(10));
20   WriteLn('FPB(48, 18)     = ', FPB(48, 18));
21   WriteLn('KPK(4, 6)       = ', KPK(4, 6));
22
23   Write('Bilangan prima di bawah 50: ');
24   for I := 1 to 50 do
25     if Prima(I) then
26       Write(I, ' ');
27   WriteLn;
28
29   { Nama unit boleh ditulis di depan, berguna bila ada nama yang sama }
30   WriteLn('matematika.Kuadrat(12) = ', matematika.Kuadrat(12));
31 end.
```

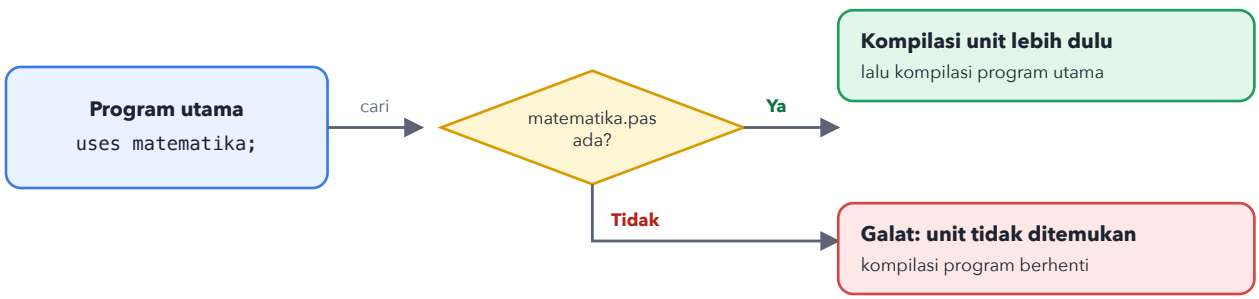


Gambar 19.2 Program memakai unit `matematika`: semua hasil dihitung oleh fungsi di `matematika.pas`

Baris	Penjelasan
9–10	`uses matematika;` memberi tahu kompiler untuk mencari berkas `matematika.pas`, yaitu di folder yang sama dengan program ini, lalu mengompilasinya lebih dulu.
16	`VersiUnit` adalah konstanta dari unit. Program ini tidak mendeklarasikannya, tetapi tetap bisa memakainya.
17–21	Pemanggilan fungsi unit. `Kuadrat(25)` = 625, `Pangkat(2, 10)` = 1024, `Faktorial(10)` = 3628800, `FPB(48, 18)` = 6, `KPK(4, 6)` = 12, sesuai keluaran di atas.
23–27	Perulangan memeriksa 1 sampai 50 dengan `Prima(I)`. Bilangan 1 tidak dicetak karena `Prima` mulai dari syarat `N >= 2`.
30	Nama unit boleh ditulis di depan sebagai awalan, `matematika.Kuadrat(12)`. Berguna bila program dan unit memiliki nama yang sama: awalan memastikan fungsi milik unit yang dipakai.

19.3.2 Bagaimana kompiler menemukan unit

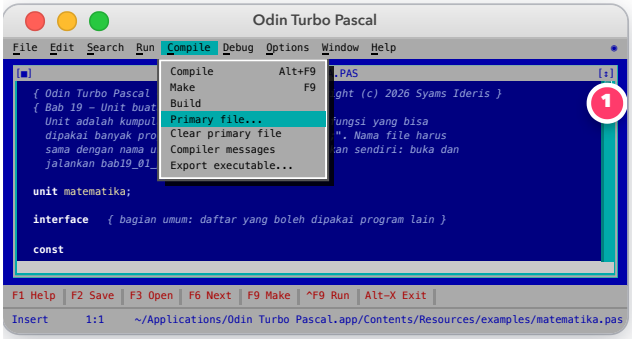
Saat membaca baris `uses`, kompiler mencari berkas unit di folder yang sama dengan program, lalu di folder tambahan yang Anda atur lewat `Options ▸ Directories...`. Setiap unit dikompilasi lebih dulu, baru program utama. Jika berkas unit tidak ditemukan, kompilasi berhenti dengan galat.



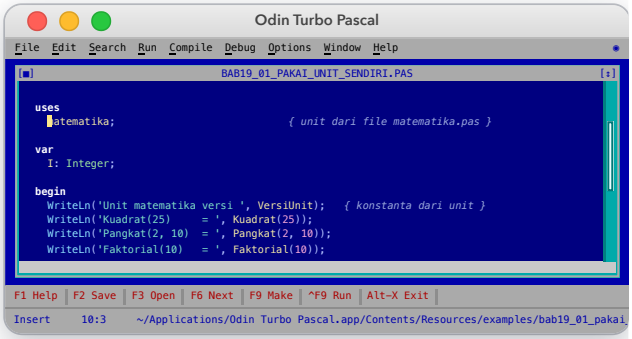
Gambar 19.3 Kompiler memproses `uses`: unit dicari, dikompilasi lebih dulu, baru program utama.

19.3.3 Primary file

Odin Turbo Pascal mengompilasi berkas yang sedang aktif di editor. Bila Anda sedang mengedit `matematika.pas` lalu menekan Ctrl+F9, yang dikompilasi adalah unit itu saja, dan tidak ada program yang berjalan. Untuk proyek yang terdiri atas program dan beberapa unit, tetapkan berkas program sebagai **Primary file**: pilih `Compile ▸ Primary file...` lalu tunjuk berkas program. Sesudah itu Ctrl+F9 selalu mengompilasi dan menjalankan program tersebut, apa pun jendela yang aktif. `Compile ▸ Clear primary file` mengembalikan mode "berkas aktif".



Gambar 19.4 Langkah 1: menu Compile ▸ Primary file... saat unit terbuka



Gambar 19.5 Langkah 2: tunjuk berkas program sebagai Primary file; unit tetap dicari otomatis

1. **Salin folder.** Contoh bawaan tidak bisa disimpan di tempatnya. Salin `matematika.pas` dan `bab19_01_pakai_unit_sendiri.pas` ke satu folder kerja Anda.
2. **Buka kedua berkas** dengan `File ▸ Open...`.
3. **Tetapkan Primary file** ke `bab19_01_pakai_unit_sendiri.pas`.
4. **Jalankan** dengan Ctrl+F9. Ubah isi `matematika.pas` (misalnya `VersiUnit` menjadi `'1.1'`), jalankan lagi, dan perhatikan keluaran berubah.

> **COBA SENDIRI**
>
> Tambahkan fungsi `Kubik(X: LongInt): LongInt` ke unit. Ingat dua langkah: tulis header di `interface`, tulis badan di `implementation`. Lalu panggil `Kubik(5)` dari program. Apa yang terjadi bila Anda hanya menulis salah satunya?

19.4 Galat Seputar Unit

Dua galat paling sering: unit tidak ditemukan (nama berkas tidak cocok atau berkas tidak berada di folder program), dan memakai nama yang tidak diumumkan di `interface`. Berikut contoh keduanya, dikompilasi sungguhan.

tanpa_unit.pas

```
1  program TanpaUnit;
2
3  uses
4    alatku;
```

```
5
6  begin
7    WriteLn(Kuadrat(5));
8  end.
```

PESAN KOMPILER

```
tanpa_unit.pas(4,3) Error: Berkas tidak ditemukan: unit alatku
```

Gambar 19.6 Berkas unit tidak ada di folder program: kompiler tidak menemukan unit `alatku`

Contoh kedua adalah program yang lupa menyebutkan unit `Crt` yang akan dibahas di bagian berikut. Pesannya berbeda: kompiler menemukan semua unit, tetapi tidak mengenal nama `ClrScr`.

lupa_crt.pas

```
1  program LupaCrt;
2
3  begin
4    ClrScr;
5    GotoXY(10, 5);
6    Write('Halo');
7  end.
```

PESAN KOMPILER

```
lupa_crt.pas(4,3) Error: Identifier tidak dikenal: ClrScr
```

Gambar 19.7 Lupa menulis `uses Crt`: `ClrScr` dan `GotoXY` tidak dikenal

19.5 Unit Crt: Mengatur Layar Teks

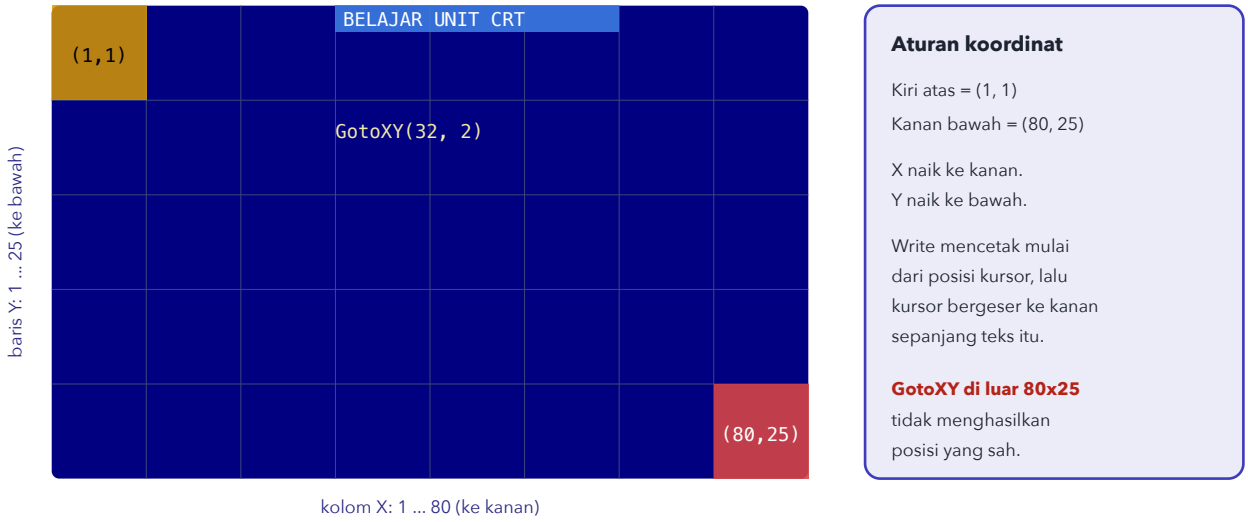
Unit `Crt` (*cathode ray tube*, tabung sinar katoda: layar lama) adalah unit bawaan Turbo Pascal yang paling terkenal. Dengannya program dapat membersihkan layar, memindahkan kursor ke posisi mana pun, mewarnai tulisan dan latar, menunggu sebentar, dan membaca tombol tanpa menunggu **Enter**. Hampir semua program antarmuka teks di era DOS, dari editor sampai permainan, dibangun dari lima-enam prosedur ini. Karena tidak menjadi bagian `System`, unit ini harus disebutkan: `uses Crt;`.

Tabel 19.1 Rutin unit Crt yang dipakai di bab ini

Rutin	Jenis	Fungsi
`ClrScr`	prosedur	menghapus layar dengan warna latar saat ini, kursor ke (1, 1)
`GotoXY(X, Y)`	prosedur	memindahkan kursor ke kolom `X`, baris `Y`
`WhereX`, `WhereY`	fungsi	kolom dan baris kursor saat ini
`TextColor(W)`	prosedur	warna tulisan berikutnya, 0 sampai 15
`TextBackground(W)`	prosedur	warna latar berikutnya, 0 sampai 7
`Delay(ms)`	prosedur	menunda program sekian milidetik
`ReadKey`	fungsi `Char`	menunggu satu tombol dan mengembalikannya, tanpa **Enter**, tanpa tampil di layar

19.5.1 Koordinat layar: 80 kolom x 25 baris

Layar teks adalah kisi 80 kolom dan 25 baris. Setiap petak memuat tepat satu karakter. Posisi dinyatakan dengan **(X, Y)**: X adalah kolom dihitung dari kiri, Y adalah baris dihitung dari atas. Titik pojok kiri atas adalah `(1, 1)` dan pojok kanan bawah `(80, 25)`. Kebiasaan dari pelajaran matematika perlu dibalik di sini: Y bertambah ke **bawah**, bukan ke atas.



Gambar 19.8 Kisi layar teks. Kisi digambar berskala (satu petak gambar mewakili 8 kolom dan 5 baris), pojok-pojoknya diberi koordinat.

19.5.2 Enam belas warna Crt

Crt mengenal 16 warna bernomor 0 sampai 15, masing-masing juga punya konstanta bernama (`Black`, `Blue`, `Yellow`, dan seterusnya). Warna tulisan (`TextColor`) boleh memakai semuanya, sedangkan warna latar (`TextBackground`) hanya 0 sampai 7 pada layar DOS asli. Warna 8 sampai 15 adalah versi "terang" dari 0 sampai 7.



Gambar 19.9 Enam belas warna Crt dengan nomor dan nama konstantanya.

7 ATURAN TURBO PASCAL 7

Konstanta warna adalah `Black` (0) sampai `White` (15), sehingga `TextColor(Yellow)` sama dengan `TextColor(14)`. Menambahkan `Blink` (128) pada `TextColor` membuat tulisan berkedip di DOS asli. Latar baru berlaku mulai dari karakter yang dicetak sesudahnya, dan `ClrScr` mengecat seluruh layar dengan warna latar saat itu: itu sebabnya program mengatur latar dahulu, baru memanggil `ClrScr`.

19.5.3 Program contoh: tur Crt

Program 19.2 memperagakan hampir semua rutin Crt dalam satu layar: judul berwarna, 16 warna, posisi kursor, animasi dengan `Delay`, dan `ReadKey`. Program berjalan di User screen yang berukuran 80x25, jadi hasilnya sama dengan layar DOS. Buka `bab19_02_unit_crt.pas` dan jalankan; tekan sembarang tombol saat diminta.

Program 19.2 `bab19_02_unit_crt.pas` *Unit Crt* File ▸ Examples...

```
{ Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 19.2 - Unit Crt
  Crt adalah unit bawaan untuk mengatur layar teks dan keyboard:
    ClrScr           bersihkan layar      GotoXY(X, Y)  pindah kursor
    TextColor(W)     warna tulisan        TextBackground(W)  warna latar
    Delay(ms)        jeda sekian milidetik
    ReadKey          tunggu satu tombol, tanpa perlu Enter
  Layar berukuran 80 kolom x 25 baris; posisi (1, 1) di kiri atas. }

program bab19_02_unit_crt;

uses
  Crt;

var
  I, X, Y: Integer;
  Tombol: Char;

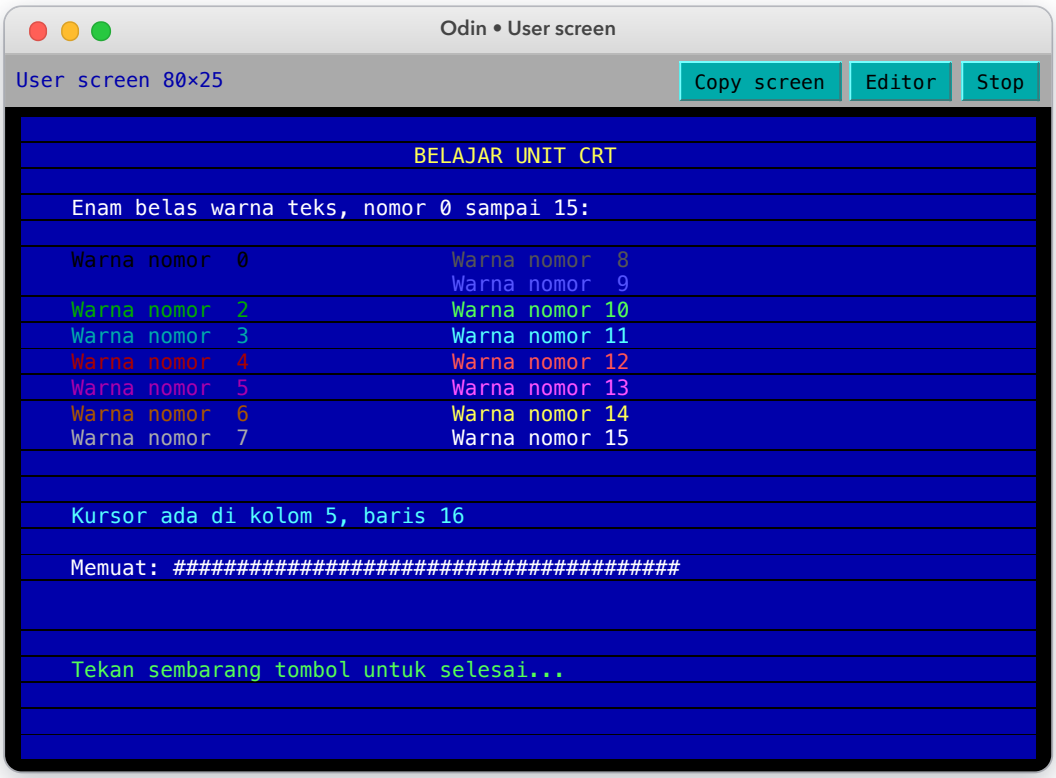
begin
  TextBackground(Blue);                 { latar biru... }
  ClrScr;                               { ...berlaku setelah ClrScr }
  TextColor(Yellow);
  GotoXY(32, 2);                        { kolom 32, baris 2 }
  Write('BELAJAR UNIT CRT');

  TextColor(White);
  GotoXY(5, 4);
  Write('Enam belas warna teks, nomor 0 sampai 15:');
  for I := 0 to 15 do
  begin
    TextColor(I);                       { 1 = Blue: sewarna latar }
    GotoXY(5 + (I div 8) * 30, 6 + I mod 8);
    Write('Warna nomor ', I:2);
  end;

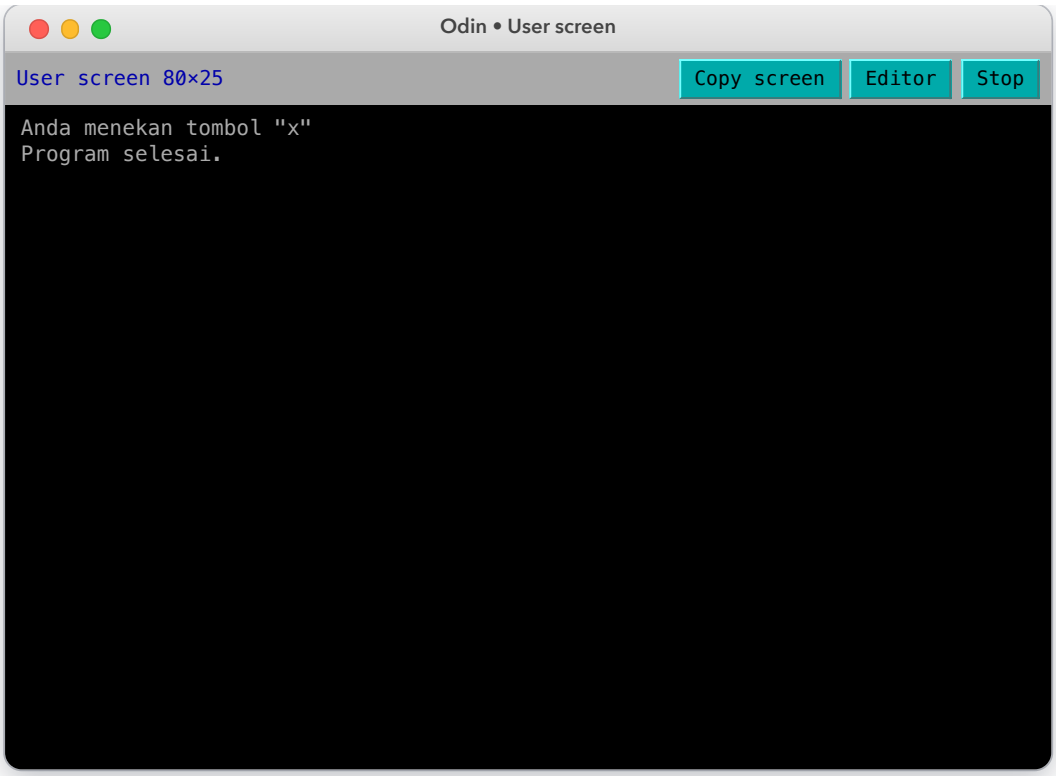
  TextColor(LightCyan);
  GotoXY(5, 16);
  X := WhereX;                          { posisi kursor saat ini }
  Y := WhereY;
  Write('Kursor ada di kolom ', X, ', baris ', Y);

  TextColor(White);                     { animasi sederhana dengan Delay }
  GotoXY(5, 18);
  Write('Memuat: ');
  for I := 1 to 40 do
  begin
    Write('#');
    Delay(25);
  end;
```

```
50
51    TextColor(LightGreen);
52    GotoXY(5, 22);
53    Write('Tekan sembarang tombol untuk selesai...');
54    Tombol := ReadKey;                    { tidak tampil di layar }
55
56    TextColor(LightGray);                 { kembalikan warna semula }
57    TextBackground(Black);
58    ClrScr;
59    if Tombol = #0 then                   { tombol khusus: F1, panah, ... }
60      WriteLn('Anda menekan tombol khusus, kode ', Ord(ReadKey))
61    else
62      WriteLn('Anda menekan tombol "', Tombol, '"');
63    WriteLn('Program selesai.');
64  end.
```



Gambar 19.10 Layar tepat sebelum tombol ditekan: judul, 16 warna, posisi kursor, dan baris kemajuan



Gambar 19.11 Setelah menekan `x`: layar dibersihkan dan program melaporkan tombolnya

Baris	Penjelasan
12–13	`uses Crt;` membuka seluruh rutin layar dan keyboard.
20–21	Latar diatur biru **lebih dulu**, baru `ClrScr`, sehingga seluruh layar menjadi biru. Kalau urutannya terbalik, layar tetap hitam.
22–24	Judul kuning dicetak mulai kolom 32 baris 2. Teks sepanjang 16 karakter, jadi kira-kira berada di tengah layar 80 kolom: $(80 - 16) / 2 = 32$.
26–28	Teks putih di (5, 4).
29–34	Perulangan 16 warna. `TextColor(I)` memakai nomor warna langsung. Posisinya `GotoXY(5 + (I div 8) * 30, 6 + I mod 8)`: `I div 8` bernilai 0 untuk warna 0 sampai 7 (kolom 5) dan 1 untuk 8 sampai 15 (kolom 35), sedangkan `I mod 8` menurunkan baris dari 6 sampai 13. Jadi dua kolom berisi delapan warna.
31, 33	Warna 1 (`Blue`) sama dengan warna latar, sehingga tulisan "Warna nomor 1" tidak terlihat. Itu contoh mengapa kombinasi warna perlu dipilih hati-hati.
36–40	`WhereX` dan `WhereY` dibaca setelah `GotoXY(5, 16)`, jadi nilainya 5 dan 16, lalu dicetak.
42–49	Animasi sederhana: 40 kali mencetak `#` dengan jeda `Delay(25)` (25 milidetik), total sekitar satu detik.
51–54	`ReadKey` menghentikan program sampai ada tombol; karakter tombol disimpan di `Tombol` tanpa tampil di layar.
56–58	Warna dikembalikan ke abu-abu terang di atas hitam, layar dihapus.
59–62	Tombol biasa dilaporkan karakternya. Tombol khusus (panah, F1) menghasilkan `#0` disusul satu `ReadKey` kedua yang berisi kodenya; bagian ini diuraikan lebih lanjut berikut.

19.5.4 Membaca tombol dengan ReadKey

`ReadLn` menunggu Anda menekan **Enter** dan menampilkan huruf yang diketik. `ReadKey` berbeda: ia langsung mengembalikan satu karakter begitu sebuah tombol ditekan, dan tidak menampilkannya. Karena itulah `ReadKey` dipakai untuk menu, permainan, dan perintah "tekan sembarang tombol".

Tombol yang punya karakter ASCII dikembalikan apa adanya: **Enter** menjadi `#13`, **Esc** menjadi `#27`. Tombol **khusus** (panah, F1 sampai F12, **Home**, dan sebagainya) tidak punya karakter. Untuk itu `ReadKey` memberikan **dua kali**: pertama `#0` sebagai tanda "tombol khusus", lalu panggilan kedua mengembalikan kode tombolnya.

Tabel 19.2 Hasil ReadKey untuk tombol yang sering dipakai

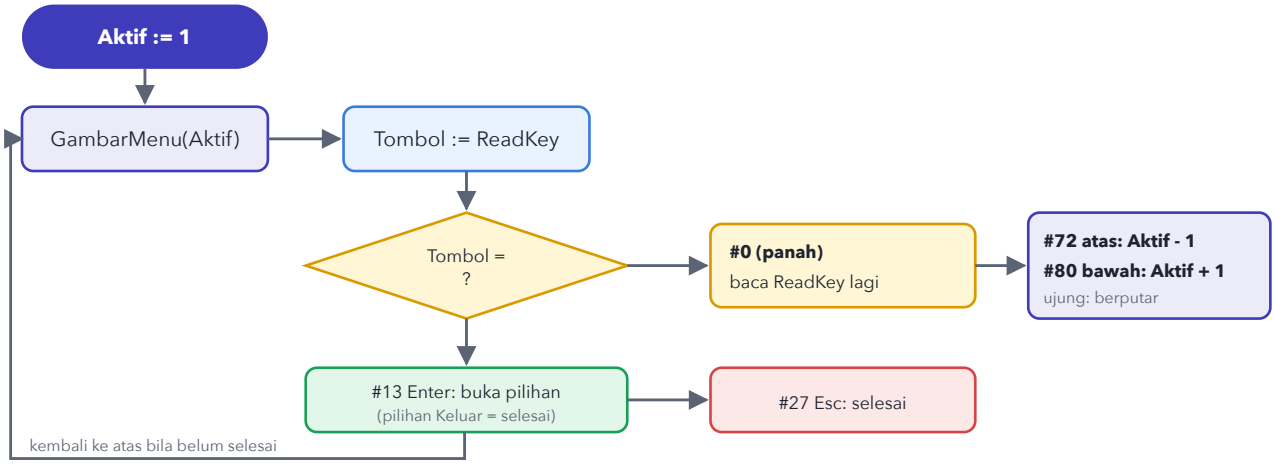
Tombol	Panggilan pertama	Panggilan kedua
huruf **x**	`'x'`	(tidak ada)
Enter	`#13`	(tidak ada)
Esc	`#27`	(tidak ada)
panah atas	`#0`	`#72`
panah bawah	`#0`	`#80`
panah kiri	`#0`	`#75`
panah kanan	`#0`	`#77`

COBA SENDIRI

Ubah baris 54 pada program 19.2 sehingga `ReadKey` dipanggil sekali, lalu jalankan dan tekan panah kanan. Program akan melaporkan tombol khusus dengan kode 77. Untuk apa baris 60 memanggil `ReadKey` sekali lagi?

19.6 Proyek Kecil: Menu Berwarna

Dengan bekal `GotoXY`, warna, dan `ReadKey`, kita bisa membuat menu yang dioperasikan dengan panah seperti di aplikasi sungguhan. Rancangannya: sebuah bingkai karakter, daftar empat pilihan, pilihan aktif ditandai warna latar berbeda, panah atas/bawah memindahkan sorotan, **Enter** membuka, dan **Esc** keluar. Alur utamanya sebuah perulangan `repeat..until` yang menggambar menu, membaca satu tombol, lalu memutuskan.



Gambar 19.12 Alur `repeat..until` pada menu berwarna: gambar, baca tombol, lalu ubah keadaan.

Buka `bab19_03_menu_berwarna.pas`. Program ini lebih panjang, jadi kita bahas per blok setelah melihat tampilannya.

Program 19.3 `bab19_03_menu_berwarna.pas` *Menu berwarna* File ▸ Examples…

```
{ Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 19.3 - Menu berwarna
  Menu dipilih dengan tombol panah atas/bawah lalu Enter. Tombol
  panah mengirim dua karakter: #0 lalu kode tombolnya (atas = 72,
  bawah = 80). Pilihan aktif ditandai dengan warna latar berbeda. }

program bab19_03_menu_berwarna;

uses
  Crt;

const
  JumlahMenu = 4;
  Menu: array[1..JumlahMenu] of string[20] =
    ('Profil sekolah', 'Jadwal pelajaran', 'Pengumuman', 'Keluar');

procedure Kotak(X1, Y1, X2, Y2: Integer);
var
  I: Integer;
begin
  GotoXY(X1, Y1); Write('+');
  for I := X1 + 1 to X2 - 1 do Write('-');
  Write('+');
  for I := Y1 + 1 to Y2 - 1 do
  begin
    GotoXY(X1, I); Write('|');
    GotoXY(X2, I); Write('|');
  end;
  GotoXY(X1, Y2); Write('+');
  for I := X1 + 1 to X2 - 1 do Write('-');
  Write('+');
end;

procedure GambarMenu(Aktif: Integer);
```

```
35  var
36    I: Integer;
37  begin
38    for I := 1 to JumlahMenu do
39    begin
40      GotoXY(30, 7 + I * 2);
41      if I = Aktif then
42      begin
43        TextBackground(Cyan);              { pilihan aktif: latar cyan }
44        TextColor(Black);
45      end
46      else
47      begin
48        TextBackground(Blue);
49        TextColor(White);
50      end;
51      Write(' ', Menu[I], '':20 - Length(Menu[I]), ' ');
52    end;
53    TextBackground(Blue);
54  end;
55
56  procedure TampilkanIsi(Nomor: Integer);
57  begin
58    TextColor(Yellow);
59    GotoXY(10, 20);
60    case Nomor of
61      1: Write('SMA Negeri Pelita, berdiri tahun 1985.            ');
62      2: Write('Senin–Jumat 07.30–14.00, Sabtu kegiatan ekstra.   ');
63      3: Write('Ujian semester dimulai tanggal 1 Desember.        ');
64    end;
65  end;
66
67  var
68    Aktif: Integer;
69    Tombol: Char;
70    Selesai: Boolean;
71
72  begin
73    TextBackground(Blue);
74    ClrScr;
75    TextColor(White);
76    Kotak(25, 4, 56, 18);
77    TextColor(Yellow);
78    GotoXY(32, 6);
79    Write('MENU UTAMA SEKOLAH');
80    TextColor(LightGray);
81    GotoXY(14, 23);
82    Write('Panah atas/bawah: pilih    Enter: buka    Esc: keluar');
83
84    Aktif := 1;
85    Selesai := False;
86    repeat
87      GambarMenu(Aktif);
88      Tombol := ReadKey;
89      if Tombol = #0 then                  { tombol khusus: baca kodenya }
90      begin
91        Tombol := ReadKey;
92        case Tombol of
93          #72: if Aktif > 1 then Aktif := Aktif - 1    { panah atas }
```

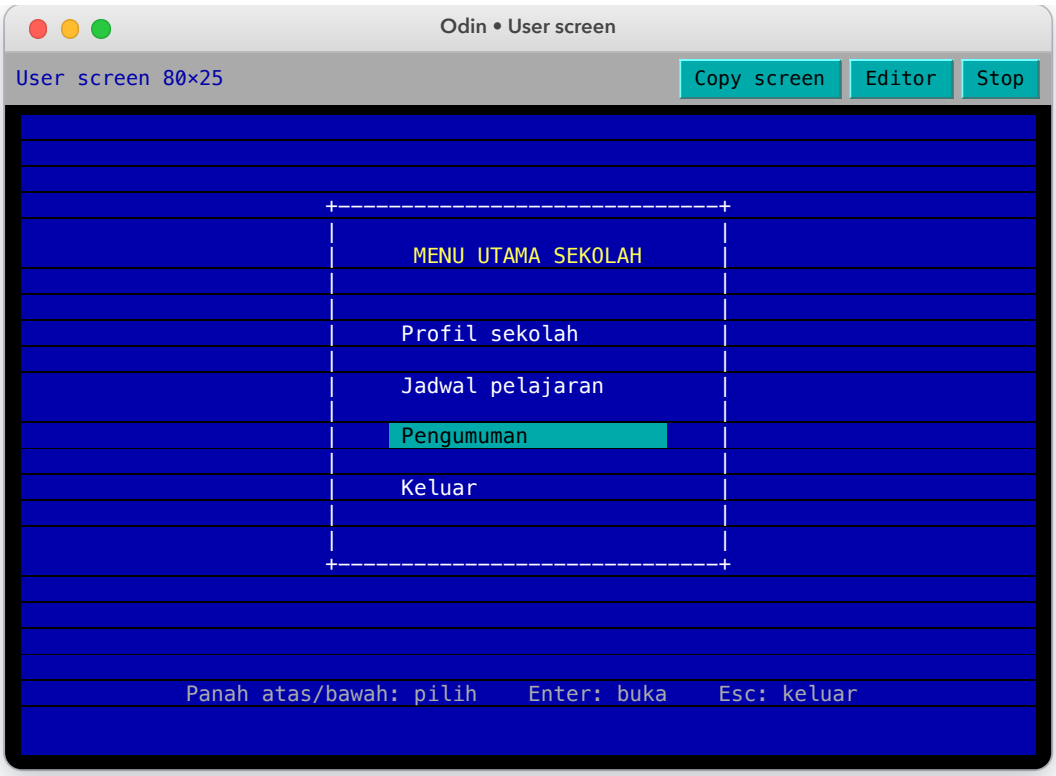
```
94              else Aktif := JumlahMenu;               { dari atas ke bawah }
95          #80: if Aktif < JumlahMenu then Aktif := Aktif + 1  { panah bawah }
96              else Aktif := 1;
97        end;
98      end
99      else if Tombol = #13 then            { Enter }
100     begin
101       if Aktif = JumlahMenu then
102         Selesai := True
103       else
104         TampilkanIsi(Aktif);
105     end
106     else if Tombol = #27 then            { Esc }
107       Selesai := True;
108   until Selesai;
109
110   TextColor(LightGray);
111   TextBackground(Black);
112   ClrScr;
113   WriteLn('Terima kasih telah memakai menu.');
114 end.
```

Layar pertama muncul begitu program mulai. Bingkai dan judul digambar sekali; pilihan pertama, "Profil sekolah", berlatar cyan karena `Aktif = 1`.



Gambar 19.13 Layar awal: pilihan *Profil sekolah* aktif (latar cyan)

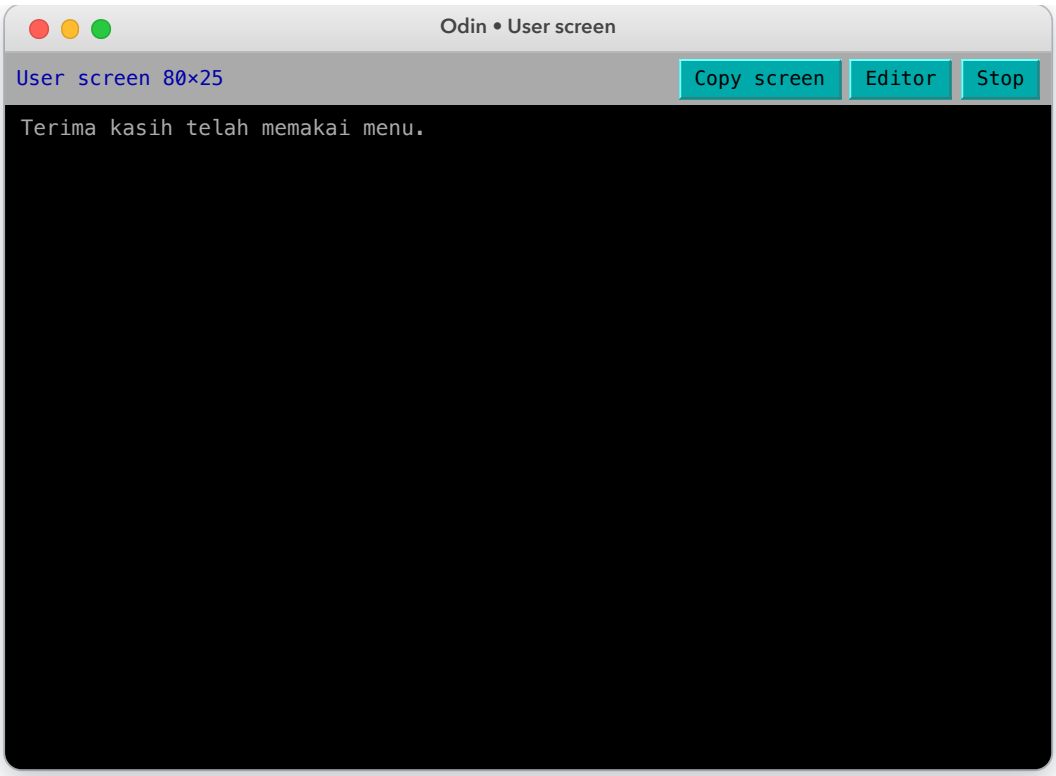
Tekan panah bawah dua kali dan sorotan pindah ke "Pengumuman" (pilihan ketiga). Enter kemudian memanggil `TampilkanIsi(3)` yang menulis kalimat kuning di baris 20.



Gambar 19.14 Setelah ↓ ↓: sorotan berpindah ke *Pengumuman*



Gambar 19.15 Setelah Enter: isi pengumuman tampil di baris 20



Gambar 19.16 Setelah Esc: layar dibersihkan, program menutup dengan pesan terima kasih

Sekarang uji perilaku berputar: dari pilihan pertama, panah **atas** melompat ke pilihan terakhir ("Keluar"). Tekan Enter di situ untuk menutup program.



Gambar 19.17 Panah atas dari pilihan pertama: sorotan berputar ke *Keluar*

19.6.1 Penjelasan per blok

Baris	Penjelasan
12–15	Konstanta `JumlahMenu` dan konstanta bertipe larik `Menu` berisi empat teks. Larik bertipe `string[20]` sehingga setiap teks muat di maksimal 20 karakter. Jika menu bertambah, cukup ubah dua baris ini.
17–32	`Kotak(X1, Y1, X2, Y2)` menggambar bingkai dari karakter `+`, `-`, `|`. Baris atas dari (X1, Y1) ke (X2, Y1); sisi kiri dan kanan dengan `GotoXY` per baris (baris 24–28); baris bawah di Y2 (29–31). Pemanggilan `Kotak(25, 4, 56, 18)` pada baris 76 menghasilkan bingkai selebar 32 kolom dan setinggi 15 baris.
34–54	`GambarMenu(Aktif)` menggambar keempat pilihan di kolom 30, baris 9, 11, 13, dan 15 (`7 + I * 2`). Pilihan aktif memakai latar `Cyan` dengan tulisan hitam, yang lain latar `Blue` dengan tulisan putih (baris 41–50).
51	Teks dicetak dengan satu spasi di kiri dan kanan serta `'':20 - Length(Menu[I])` sebagai bantalan spasi. Tiap pilihan jadi selebar 22 kolom, sehingga bilah sorotan sama panjang.
53	Latar dikembalikan ke biru supaya tulisan sesudahnya tidak ikut berlatar cyan.
56–65	`TampilkanIsi(Nomor)` memakai `case` untuk mencetak kalimat di (10, 20). Kalimat dipadatkan hingga sama panjang agar sisa kalimat sebelumnya tertimpa. Pilihan 4 (Keluar) tidak memiliki cabang karena ditangani di program utama.
73–82	Persiapan layar: latar biru, `ClrScr`, bingkai, judul di (32, 6), dan petunjuk di baris 23.
84–85	`Aktif` bermula di 1 dan `Selesai` bernilai `False`.
86–88	Awal perulangan: gambar menu dengan sorotan terbaru, lalu tunggu satu tombol.
89–98	Bila tombol `#0`, panggilan `ReadKey` kedua mengambil kodenya. `#72` (atas) mengurangi `Aktif`, atau melompat ke `JumlahMenu` bila sudah di atas; `#80` (bawah) menambah atau kembali ke 1.
99–105	`Enter` (`#13`): bila yang aktif pilihan terakhir, `Selesai := True`, selain itu isinya ditampilkan.
106–108	`Esc` (`#27`) juga mengakhiri perulangan lewat `until Selesai`.
110–113	Warna dan layar dikembalikan, lalu pesan penutup dicetak.

TIPS

Perhatikan dua gaya `if` bertingkat di baris 89 sampai 107: `if ... then begin ... end else if ... then begin ... end else if ...`. Tidak ada titik koma sebelum setiap `else`. Ini rantai `else if` yang Anda kenal pada Bab 6, hanya saja kali ini menyaring jenis tombol.

COBA SENDIRI

Tambahkan pilihan kelima "Kontak" ke menu. Anda perlu mengubah `JumlahMenu`, isi larik `Menu`, dan satu cabang `case` di `TampilkanIsi`. Apakah bingkai `Kotak` masih cukup tinggi? Hitung baris terakhir pilihan: `7 + 5 * 2`.

19.7 Kesalahan yang Sering Terjadi

- **Nama berkas tidak sama dengan nama unit.** `unit matematika;` yang disimpan sebagai `mat.pas` tidak akan ditemukan oleh `uses matematika;`.
- **Header ada di `interface` tetapi badan tidak ada** (atau sebaliknya). Setiap rutin yang diumumkan harus dilengkapi di `implementation`.
- **Lupa `uses Crt;`** ketika memakai `ClrScr`, `GotoXY`, atau `ReadKey`.
- **Mengira unit bisa dijalankan.** Jalankan program yang memakainya, atau gunakan Primary file.
- **Mengatur latar setelah `ClrScr`.** Latar baru memengaruhi teks sesudahnya, bukan layar yang sudah dihapus.
- **Menekan panah tetapi program tidak membedakannya dari huruf.** Tombol khusus selalu dua kali `ReadKey`: `#0` lalu kodenya.

badan_hilang.pas

```
1  unit badan_hilang;
2
3  interface
4
5  function Dua(X: Integer): Integer;
6
7  implementation
8
9  end.
```

PESAN KOMPILER

```
badan_hilang.pas(5,10) Error: Deklarasi forward tidak pernah didefinisikan: Badan_hilang.Dua
```

Gambar 19.18 Contoh unit yang mengumumkan fungsi di `interface` tetapi lupa menulis badannya

19.8 Rangkuman

- **Unit** adalah berkas berisi konstanta, tipe, prosedur, dan fungsi yang dipakai ulang oleh banyak program.
- Unit terbagi dua: `interface` (daftar yang boleh dipakai pihak luar, hanya header) dan `implementation` (badan lengkap), diakhiri `end.` tanpa blok utama.
- Nama berkas harus sama dengan nama unit; program memakainya dengan `uses NamaUnit;`, dan nama unit boleh dipakai sebagai awalan (`matematika.Kuadrat`).
- Unit tidak bisa dijalankan sendiri. Gunakan `Compile ▸ Primary file...` untuk menetapkan program yang dijalankan bersama unitnya.
- Unit `Crt` menyediakan `ClrScr`, `GotoXY`, `WhereX` / `WhereY`, `TextColor`, `TextBackground`, `Delay`, dan `ReadKey`.
- Layar teks berukuran 80 kolom x 25 baris; (1, 1) di kiri atas, X ke kanan, Y ke bawah. Ada 16 warna bernomor 0 sampai 15.

- `ReadKey` membaca satu tombol tanpa **Enter**; tombol khusus menghasilkan `#0` diikuti kode tombol (atas 72, bawah 80, kiri 75, kanan 77).
- Menu interaktif = perulangan *gambar, baca tombol, ubah keadaan* sampai `Selesai`.

19.9 Latihan

1. **DASAR** Tulis unit `suhu` dengan fungsi `CelsiusKeFahrenheit(C: Real): Real`. Kemudian buat program yang memanggilnya untuk 0, 37, dan 100 derajat Celsius. Simpan unit dan program di satu folder, lalu tetapkan Primary file.

2. **DASAR** Dengan `GotoXY` dan `TextColor`, tampilkan nama Anda di tengah baris ke-12 layar 80 kolom dengan warna kuning di atas latar merah. Hitung kolom awalnya dari panjang nama.

3. **DASAR** Cetak tabel warna 0 sampai 15 seperti program 19.2, tetapi tulis nama konstanta tiap warna (`'Black'`, `'Blue'`, dan seterusnya) memakai `case`.

4. **MENENGAH** Tambahkan ke unit `matematika` fungsi `JumlahDigit(N: LongInt): Integer` (memakai `mod` dan `div`), lengkap dengan header di `interface`. Uji untuk 2026.

5. **MENENGAH** Buat program kotak yang bergerak: sebuah karakter `#` dapat digeser ke kiri, kanan, atas, dan bawah dengan tombol panah (kode 75, 77, 72, 80), dan berhenti saat **Esc**. Hapus posisi lama dengan spasi sebelum menggambar posisi baru, dan jangan biarkan karakter keluar dari 80x25.

6. **TANTANGAN** Ubah menu 19.3 supaya pilihan bisa juga dipilih dengan angka **1** sampai **4** dan menu ditampilkan dari data yang dibaca dari berkas teks.

7. **TANTANGAN** Pecah program menu menjadi dua unit: `tampilan` (fungsi `Kotak`) dan `menu` (fungsi `GambarMenu`). Program utama hanya berisi alur menunya. Pikirkan: apa yang harus diumumkan di `interface` masing-masing?

BAB 20

BAGIAN VI · FILE, POINTER, UNIT, DAN PROYEK AKHIR

Proyek Akhir

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

TUJUAN PEMBELAJARAN

- Menjalani siklus kecil pengembangan program: analisis kebutuhan, rancangan, penulisan kode, dan pengujian dengan skenario.
- Membaca dan menjelaskan program 100 sampai 200 baris dengan memecahnya menjadi blok: konstanta dan tipe, fungsi bantu, prosedur, dan program utama.
- Menggabungkan *record*, *array*, string, fungsi, validasi masukan dengan `Val`, serta file teks dan file bertipe dalam satu aplikasi.
- Memakai satu prosedur untuk mencetak ke layar sekaligus ke file dengan parameter `var F: Text`.
- Menyimpan data agar bertahan antar-sesi dan memuatnya kembali saat program mulai.
- Meninjau kembali seluruh materi buku lewat peta konsep, lalu merancang pengembangan Anda sendiri.

PROGRAM DALAM BAB INI

20.1 `bab20_01_kasir_sederhana.pas`
Proyek: kasir sederhana

20.2 `bab20_02_nilai_siswa.pas`
Proyek: pengolah nilai siswa

20.3 `bab20_03_buku_telepon.pas`
Proyek: buku telepon

Sembilan belas bab telah membekali Anda dengan semua bahan: tipe data, percabangan, perulangan, prosedur dan fungsi, array, string, record, pengurutan, pencarian, file, dan unit. Bab terakhir ini merangkai semuanya menjadi tiga program yang berguna: kasir toko, pengolah nilai siswa, dan buku telepon. Ketiganya bukan lagi contoh yang menjelaskan satu fitur, tetapi aplikasi kecil yang punya masukan, pengolahan, keluaran, dan penyimpanan. Bacalah dengan tujuan mengenali kembali potongan-potongan yang sudah Anda pelajari, lalu kembangkan menjadi milik Anda sendiri.

20.1 Dari Masalah ke Program

Program yang baik jarang bermula dari mengetik kode. Seorang tukang yang membangun rumah memulai dari kebutuhan pemilik, menggambar rancangan, baru memotong kayu, dan memeriksa hasilnya setiap tahap. Pengembangan perangkat lunak memiliki empat langkah yang sama, dan ketiga proyek di bab ini disusun dengan urutan tersebut.



Setiap proyek di bab ini dibahas dengan bagian yang sama: **analisis kebutuhan** (apa yang harus bisa dilakukan program), **rancangan** (diagram struktur dan alur), **listing lengkap**, **penjelasan per blok**, **skenario pengujian** yang dijalankan sungguhan, dan **ide pengembangan**. Tabel berikut menunjukkan materi buku yang dipakai masing-masing proyek.

Tabel 20.1 Materi buku yang digabungkan dalam tiga proyek

Materi	20.1 Kasir	20.2 Nilai siswa	20.3 Buku telepon
Record (Bab 13)	barang dan belanja	data siswa	kontak
Array (Bab 11)	daftar barang, keranjang	daftar siswa	daftar kontak
String (Bab 12)	format Rupiah, kode barang	nama siswa	pencarian, huruf kapital
Prosedur dan fungsi (Bab 9)	6 rutin	5 rutin	10 rutin
Pengurutan (Bab 16)	-	insertion sort peringkat	penyisipan urut nama
Pencarian (Bab 15)	kode barang	-	pencarian sebagian nama
File teks (Bab 17)	struk.txt	rekap_nilai.txt	-
File bertipe (Bab 17)	-	-	kontak.dat

Materi	20.1 Kasir	20.2 Nilai siswa	20.3 Buku telepon
Percabangan dan perulangan (Bab 6-8)	semua	semua	semua

20.2 Proyek 1: Kasir Sederhana

20.2.1 Analisis kebutuhan

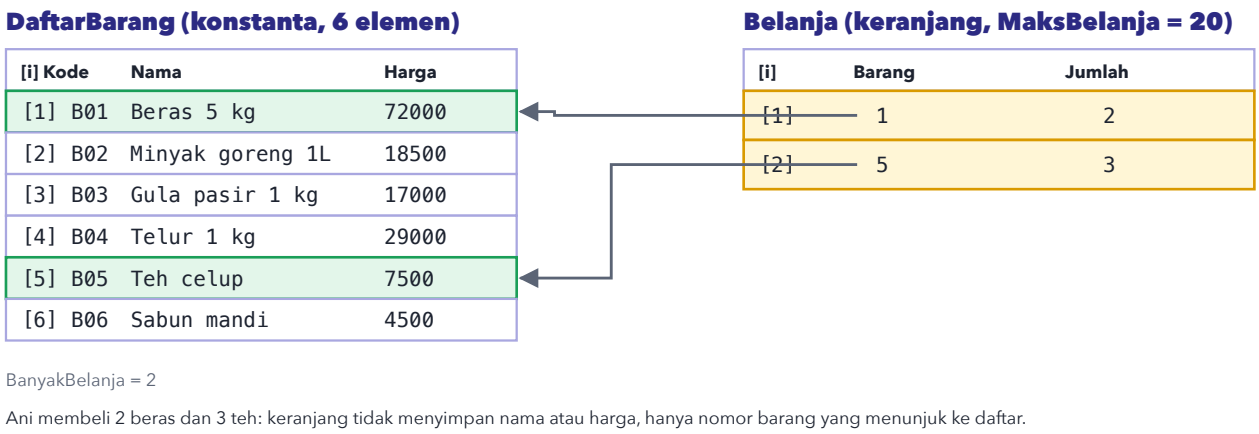
Sebuah toko kelontong ingin program kasir sederhana. Dari percakapan dengan pemilik toko, kebutuhannya dirumuskan sebagai berikut.

Tabel 20.2 Kebutuhan program kasir

No	Kebutuhan	Aturan
1	Menampilkan daftar barang dan harga	6 barang, dengan kode seperti `B01`
2	Mencatat belanjaan	kode barang dan jumlah; kode tidak peduli huruf besar atau kecil; barang yang sama digabung
3	Menghitung total dan diskon	diskon 10% bila belanja minimal Rp 100.000
4	Menerima uang tunai	harus cukup; bila kurang, ditanya ulang
5	Mencetak struk	ke layar dan ke `struk.txt`, isinya sama persis
6	Tahan salah ketik	angka yang bukan angka tidak membuat program berhenti

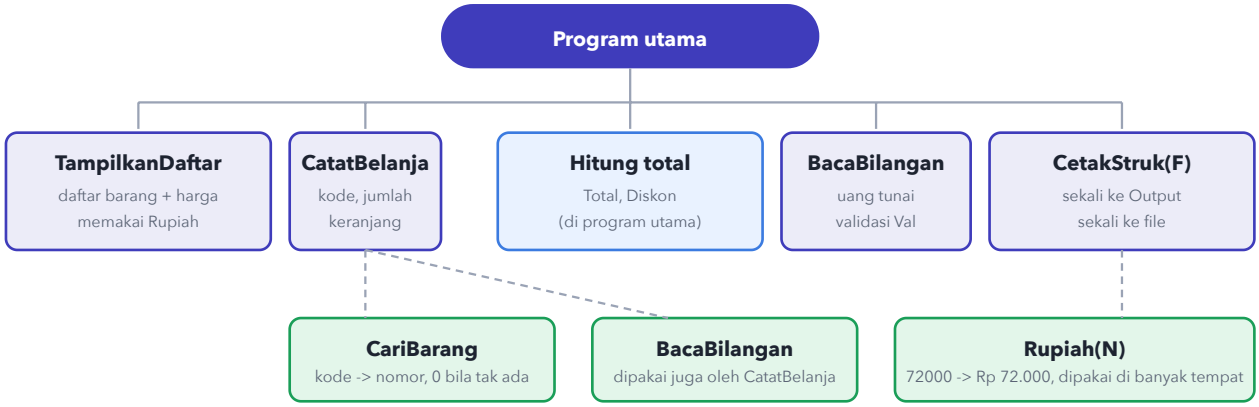
20.2.2 Rancangan

Data barang bersifat tetap, jadi disimpan sebagai **konstanta bertipe larik** berisi *record* `TBarang` (kode, nama, harga). Keranjang belanja adalah larik *record* `TBelanja` yang hanya menyimpan **nomor barang** dan **jumlah**. Tidak perlu menyalin nama dan harga: cukup rujuk ke daftar barang lewat nomornya. Cara ini menghindari data ganda yang bisa saling bertentangan.



Gambar 20.1 Dua struktur data kasir: daftar barang (tetap) dan keranjang (berubah) yang merujuk daftar lewat nomor.

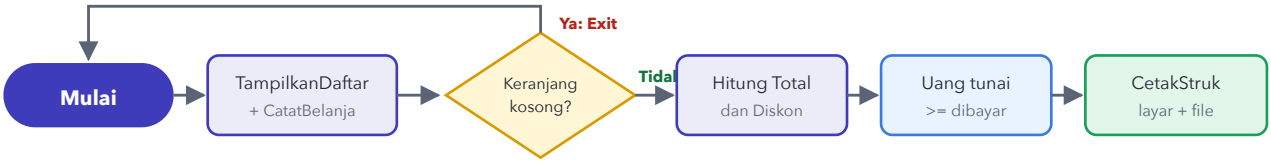
Program dipecah menjadi enam rutin kecil. Setiap rutin mengurus satu hal, dan program utama hanya menyusunnya dalam urutan yang benar.



Garis tegas: dipanggil dari program utama. Garis putus-putus: dipanggil oleh rutin di atasnya.
Fungsi bantu (hijau) ditulis lebih dulu dalam kode, rutin yang memanggilnya menyusul.

Gambar 20.2 Struktur rutin program kasir dan siapa memanggil siapa.

Alur kerja program utama sangat lurus: tampilkan daftar, catat belanja, hitung, bayar, cetak. Perhatikan bahwa bila tidak ada belanjaan, program berhenti lebih cepat tanpa mencetak struk kosong.



Gambar 20.3 Alur program utama kasir.

20.2.3 Listing lengkap

Buka `bab20_01_kasir_sederhana.pas` lewat `File ▸ Examples...`. Listing ini cukup panjang; penjelasannya dibagi per blok di bawahnya. Kenali bagian yang sudah Anda pelajari: `record` (Bab 13), konstanta larik, `with`, `Val` (Bab 12), `Insert`, dan `Assign` / `Rewrite` / `Close` (Bab 17).

Program 20.1 `bab20_01_kasir_sederhana.pas` *Proyek: kasir sederhana* File ▸ Examples...

```
{ Odin Turbo Pascal - GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 20.1 - Proyek: kasir sederhana
  Gabungan materi: record, array, fungsi, validasi dengan Val, dan
  file teks. Kasir mencatat belanjaan, menghitung diskon dan kembalian,
  lalu mencetak struk ke layar sekaligus menyimpannya ke file.
  Trik: prosedur CetakStruk menerima variabel Text, jadi bisa diberi
  Output (layar) atau variabel file. }

program bab20_01_kasir_sederhana;

const
  NamaFileStruk = 'struk.txt';
```

```
  BatasDiskon = 100000;                  { belanja minimal untuk diskon }
  PersenDiskon = 10;
  JumlahBarang = 6;
  MaksBelanja = 20;

type
  TBarang = record
    Kode: string[3];
    Nama: string[18];
    Harga: LongInt;
  end;
  TBelanja = record
    Barang: Integer;                     { nomor barang di DaftarBarang }
    Jumlah: Integer;
  end;

const
  DaftarBarang: array[1..JumlahBarang] of TBarang = (
    (Kode: 'B01'; Nama: 'Beras 5 kg';      Harga: 72000),
    (Kode: 'B02'; Nama: 'Minyak goreng 1L'; Harga: 18500),
    (Kode: 'B03'; Nama: 'Gula pasir 1 kg'; Harga: 17000),
    (Kode: 'B04'; Nama: 'Telur 1 kg';      Harga: 29000),
    (Kode: 'B05'; Nama: 'Teh celup';       Harga: 7500),
    (Kode: 'B06'; Nama: 'Sabun mandi';     Harga: 4500));

var
  Belanja: array[1..MaksBelanja] of TBelanja;
  BanyakBelanja: Integer;
  Total, Diskon, Bayar: LongInt;

function Rupiah(N: LongInt): string;     { 72000 -> 'Rp 72.000' }
var
  S: string;
  P: Integer;
begin
  Str(N, S);
  P := Length(S) - 2;
  while P > 1 do
  begin
    Insert('.', S, P);
    P := P - 3;
  end;
  Rupiah := 'Rp ' + S;
end;

{ Membaca bilangan dengan aman: diulang sampai sah dan >= Minimal }
function BacaBilangan(Pesan: string; Minimal: LongInt): LongInt;
var
  Teks: string;
  Nilai: LongInt;
  Kode: Integer;
begin
  repeat
    Write(Pesan);
    ReadLn(Teks);
    Val(Teks, Nilai, Kode);
    if (Kode <> 0) or (Nilai < Minimal) then
      WriteLn('  Masukkan angka minimal ', Minimal, '.');
  until (Kode = 0) and (Nilai >= Minimal);
```

```
72    BacaBilangan := Nilai;
73  end;
74
75  function CariBarang(Kode: string): Integer;   { 0 bila tidak ada }
76  var
77    I: Integer;
78  begin
79    CariBarang := 0;
80    for I := 1 to Length(Kode) do
81      Kode[I] := UpCase(Kode[I]);
82    for I := 1 to JumlahBarang do
83      if DaftarBarang[I].Kode = Kode then
84        CariBarang := I;
85  end;
86
87  procedure TampilkanDaftar;
88  var
89    I: Integer;
90  begin
91    WriteLn('Kode  Barang                 Harga');
92    for I := 1 to JumlahBarang do
93      with DaftarBarang[I] do
94        WriteLn(Kode, '   ', Nama, '':18 - Length(Nama), Rupiah(Harga):11);
95  end;
96
97  procedure CatatBelanja;
98  var
99    Kode: string;
100   Nomor, I, Ada: Integer;
101   Jumlah: LongInt;
102 begin
103   BanyakBelanja := 0;
104   repeat
105     Write('Kode barang (kosong = selesai): ');
106     ReadLn(Kode);
107     if Kode <> '' then
108     begin
109       Nomor := CariBarang(Kode);
110       if Nomor = 0 then
111         WriteLn('  Kode tidak dikenal.')
112       else
113       begin
114         Jumlah := BacaBilangan('  Jumlah ' + DaftarBarang[Nomor].Nama +
115           ': ', 1);
116         Ada := 0;                       { barang sudah ada di keranjang? }
117         for I := 1 to BanyakBelanja do
118           if Belanja[I].Barang = Nomor then
119             Ada := I;
120         if Ada > 0 then
121           Belanja[Ada].Jumlah := Belanja[Ada].Jumlah + Jumlah
122         else if BanyakBelanja < MaksBelanja then
123         begin
124           BanyakBelanja := BanyakBelanja + 1;
125           Belanja[BanyakBelanja].Barang := Nomor;
126           Belanja[BanyakBelanja].Jumlah := Jumlah;
127         end
128         else
129           WriteLn('  Keranjang penuh.');
130       end;
```

```
131       end;
132     until Kode = '';
133   end;
134
135   procedure CetakStruk(var F: Text);     { F: layar (Output) atau file }
136   var
137     I: Integer;
138     Subtotal: LongInt;
139   begin
140     WriteLn(F, '========================================');
141     WriteLn(F, '          TOKO SERBA ADA PELITA');
142     WriteLn(F, '========================================');
143     for I := 1 to BanyakBelanja do
144       with DaftarBarang[Belanja[I].Barang] do
145       begin
146         Subtotal := Harga * Belanja[I].Jumlah;
147         WriteLn(F, Nama);
148         WriteLn(F, '  ', Belanja[I].Jumlah:3, ' x ', Rupiah(Harga):11,
149           Rupiah(Subtotal):20);
150       end;
151     WriteLn(F, '----------------------------------------');
152     WriteLn(F, 'Total    ', Rupiah(Total):31);
153     if Diskon > 0 then
154       WriteLn(F, 'Diskon ', PersenDiskon, '%', Rupiah(Diskon):30);
155     WriteLn(F, 'Dibayar  ', Rupiah(Total - Diskon):31);
156     WriteLn(F, 'Tunai    ', Rupiah(Bayar):31);
157     WriteLn(F, 'Kembali  ', Rupiah(Bayar - (Total - Diskon)):31);
158     WriteLn(F, '========================================');
159     WriteLn(F, '      Terima kasih atas kunjungan Anda');
160   end;
161
162   var
163     I: Integer;
164     FileStruk: Text;
165     Folder: string;
166
167   begin
168     TampilkanDaftar;
169     WriteLn;
170     CatatBelanja;
171     if BanyakBelanja = 0 then
172     begin
173       WriteLn('Tidak ada belanjaan.');
174       Exit;                                { Exit di program utama: selesai }
175     end;
176
177     Total := 0;
178     for I := 1 to BanyakBelanja do
179       Total := Total + DaftarBarang[Belanja[I].Barang].Harga *
180         Belanja[I].Jumlah;
181     if Total >= BatasDiskon then
182       Diskon := Total * PersenDiskon div 100
183     else
184       Diskon := 0;
185
186     WriteLn('Yang harus dibayar: ', Rupiah(Total - Diskon));
187     Bayar := BacaBilangan('Uang tunai        : ', Total - Diskon);
188
189     WriteLn;
```

```
190    CetakStruk(Output);                    { ke layar }
191    Assign(FileStruk, NamaFileStruk);
192    Rewrite(FileStruk);
193    CetakStruk(FileStruk);                 { ke file, isi sama persis }
194    Close(FileStruk);
195    GetDir(0, Folder);
196    WriteLn;
197    WriteLn('Struk disimpan ke ', NamaFileStruk, ' di folder ', Folder);
198  end.
```

20.2.4 Penjelasan per blok

Baris	Penjelasan
11–16	Konstanta pengaturan: nama berkas struk, batas dan persen diskon, jumlah barang, dan kapasitas keranjang. Mengubah kebijakan toko (misalnya diskon 15%) cukup di sini.
18–27	Dua tipe *record*. `TBarang` berisi kode (`string[3]`), nama, dan harga `LongInt` (harga bisa melebihi 32767). `TBelanja` hanya berisi nomor barang dan jumlah.
29–36	`DaftarBarang` : konstanta larik dari record, diisi dengan sintaks `(Kode: ...; Nama: ...; Harga: ...)` . Data awal tertanam di program.
38–41	Variabel global: keranjang, jumlah isi keranjang, serta `Total` , `Diskon` , `Bayar` , semuanya `LongInt` .
43–56	`Rupiah` mengubah bilangan menjadi teks bertitik ribuan. `Str` menghasilkan teks angka, lalu dari belakang setiap tiga digit disisipkan titik dengan `Insert` : untuk `72000` , posisi `Length - 2` = 3 sehingga hasilnya `72.000` ; untuk `144000` posisi 4 menghasilkan `144.000` . Perulangan berhenti saat `P` tidak lebih dari 1.
58–73	`BacaBilangan` membaca *string* lebih dulu, lalu `Val` mengubahnya. Bila `Kode <> 0` (bukan angka) atau nilainya kurang dari `Minimal` , pesan galat tampil dan perulangan `repeat` mengulang. Inilah cara membuat program tidak jatuh bila pemakai mengetik huruf.
75–85	`CariBarang` membandingkan kode dengan setiap barang. Parameter `Kode` adalah parameter nilai, jadi perubahan huruf besar (baris 80–81) hanya berlaku di salinan lokal. Hasil 0 berarti tidak ditemukan.
87–95	`TampilkanDaftar` memakai `with` agar `Kode` , `Nama` , `Harga` bisa ditulis tanpa awalan. Bantalan `'':18 - Length(Nama)` meratakan kolom, dan `Rupiah(Harga):11` merapatkan harga ke kanan.
97–133	`CatatBelanja` : perulangan membaca kode sampai kosong. Kode dicari (baris 109); bila 0, tampil pesan. Bila ada, jumlah dibaca dengan `BacaBilangan` minimal 1. Baris 116–119 mencari apakah barang sudah ada di keranjang; bila ya, jumlahnya ditambah (baris 121), bila tidak, ditambahkan sebagai baris baru (122–127) selama keranjang belum penuh.
135–160	`CetakStruk(var F: Text)` menulis struk ke `F` . Karena `Output` juga berjenis `Text` , prosedur ini dipanggil dua kali dengan sasaran berbeda. Setiap `WriteLn(F, ...)` memakai format lebar (`:11` , `:20` , `:31`) agar kolom rupiah sejajar.
153–154	Baris diskon hanya dicetak bila `Diskon > 0` .
168–175	Program utama: daftar, catat belanja. Bila keranjang kosong, `Exit` di program utama menghentikan program dengan rapi.
177–184	Total dijumlahkan dengan mengalikan harga (diambil dari `DaftarBarang` lewat nomor) dengan jumlah. Diskon dihitung dengan `Total * PersenDiskon div 100` : dikali dulu baru dibagi supaya tidak ada pembulatan.

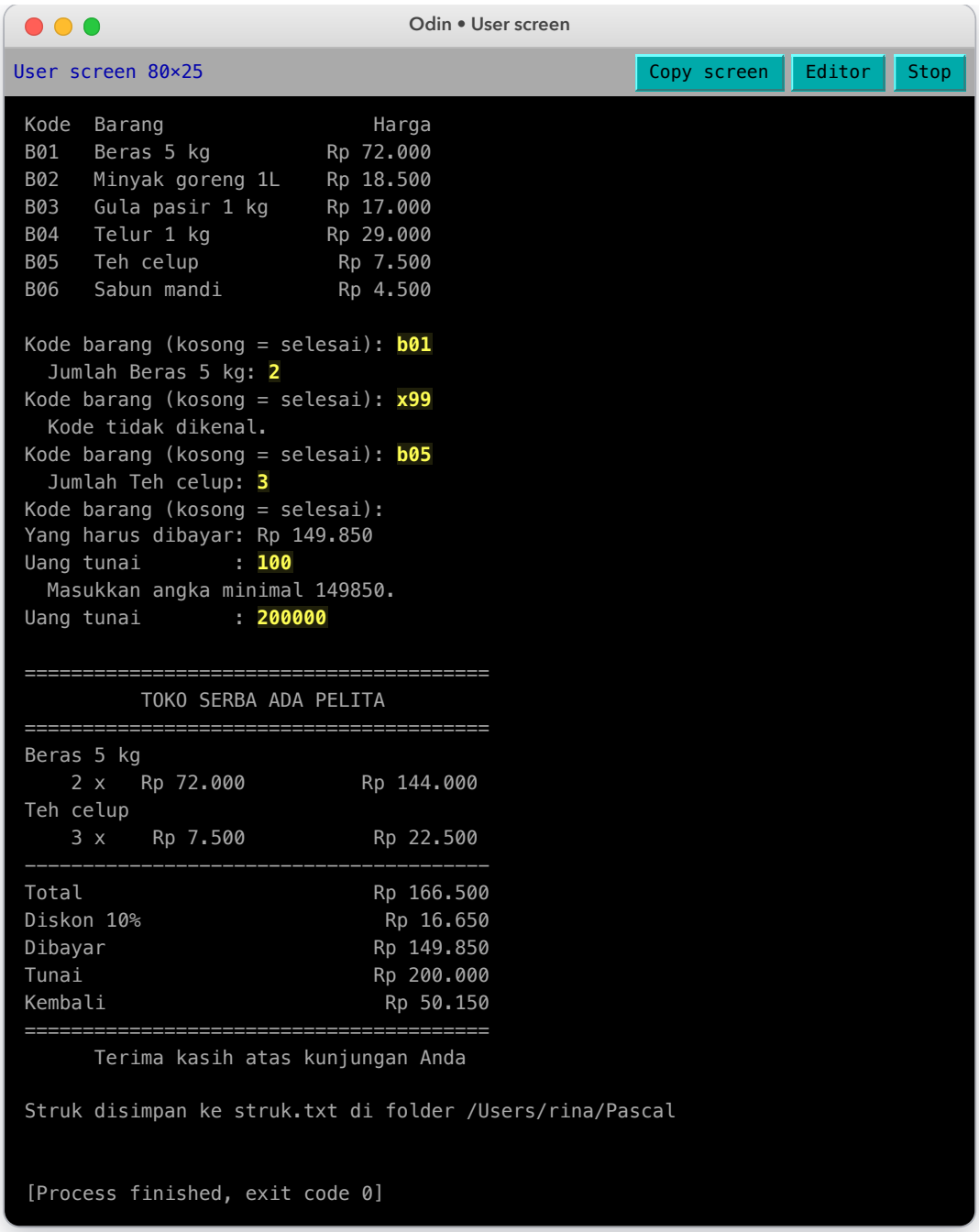
Baris	Penjelasan
186–187	Jumlah yang harus dibayar ditampilkan, lalu uang tunai dibaca dengan `BacaBilangan` yang batas minimalnya adalah `Total - Diskon`: uang kurang otomatis ditolak.
189–197	Struk dicetak ke layar, lalu file `struk.txt` dibuat dengan `Assign`, `Rewrite`, dan diisi ulang dengan prosedur yang sama, kemudian ditutup. `GetDir(0, Folder)` memberi tahu pemakai letak berkasnya.

TIPS

Pola `procedure Cetak(var F: Text)` sangat berguna: satu rutin untuk dua tujuan. Tidak ada dua versi rumus yang harus dijaga kembaran. Hal ini juga menjamin isi layar dan isi berkas selalu sama.

20.2.5 Skenario uji

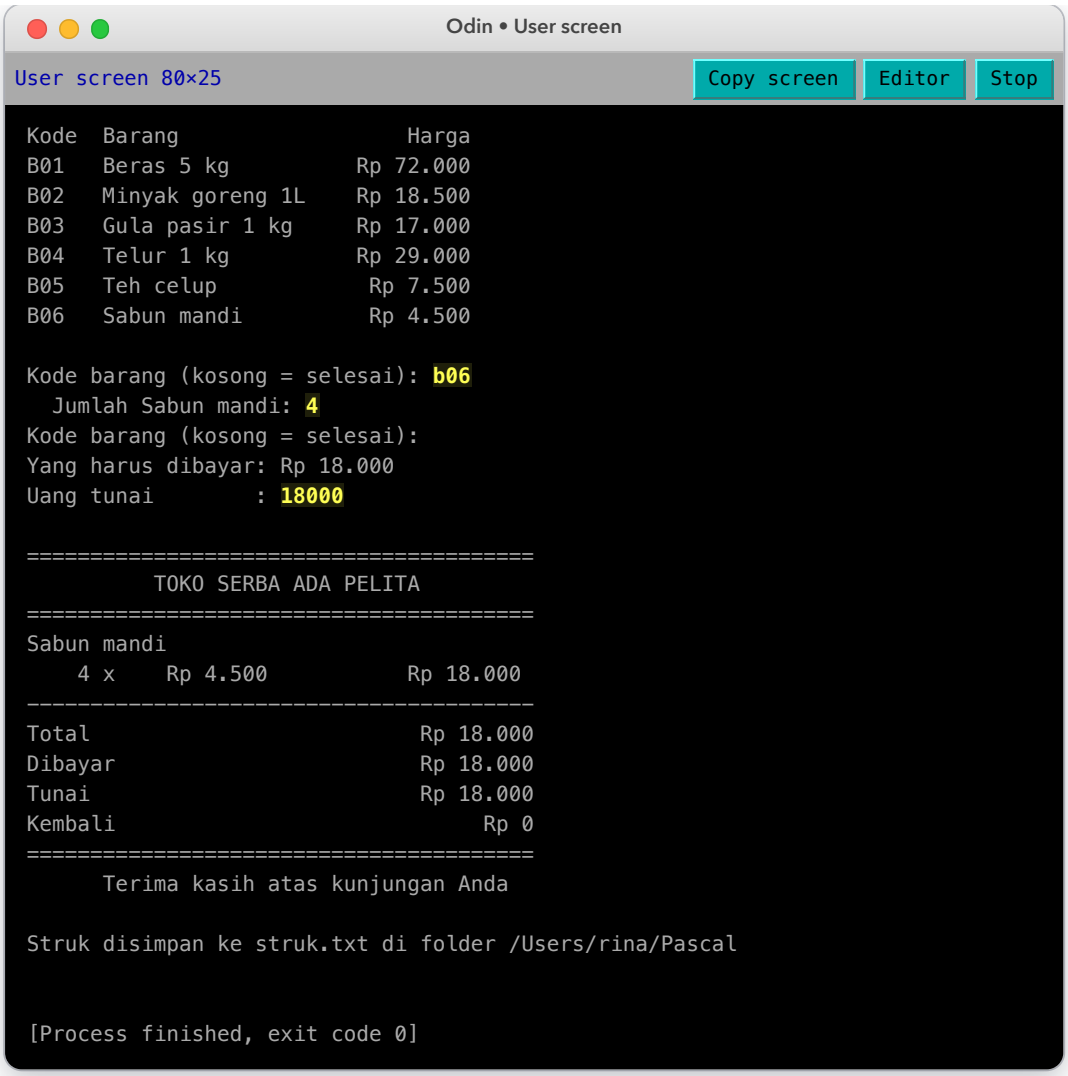
Skenario 1: belanja normal dengan diskon. Pelanggan membeli 2 beras (`b01`, huruf kecil sengaja), salah mengetik kode `x99`, membeli 3 teh (`B05`), lalu mengosongkan kode untuk selesai. Total `2 x 72000 + 3 x 7500 = 166500`, melewati batas diskon. Uang yang diketik pertama (`100`) kurang dan ditolak, lalu `200000` diterima.



Gambar 20.4 Skenario 1: kode huruf kecil, kode salah, uang kurang, lalu diskon 10%

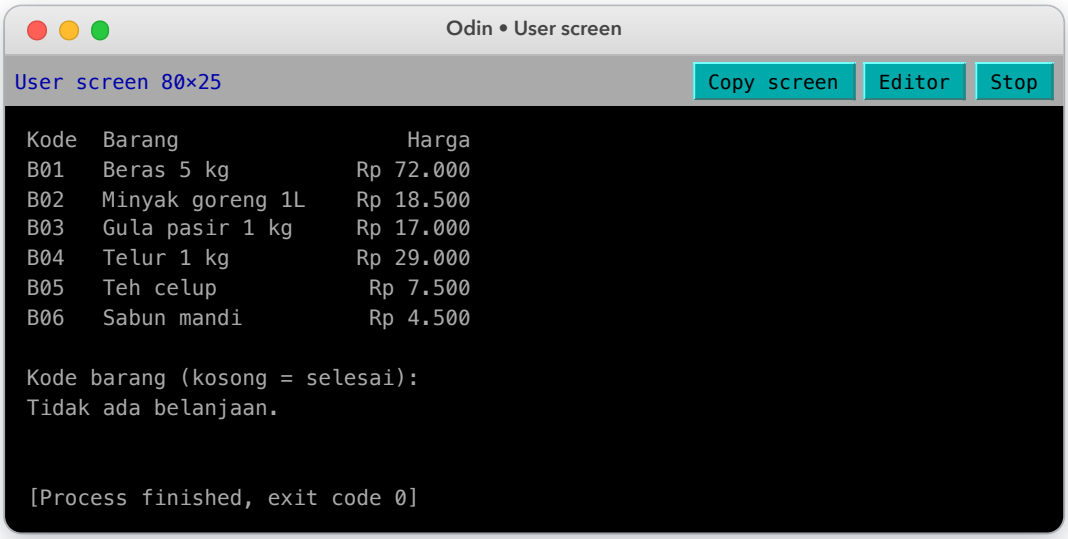
Periksa hitungannya: diskon `166500 * 10 div 100 = 16650`, dibayar `166500 - 16650 = 149850`, dan kembalian `200000 - 149850 = 50150`. Angka di struk sesuai dengan hitungan tangan. Itulah cara menguji program: hitung dulu hasil yang benar, baru bandingkan dengan hasil program.

Skenario 2: belanja kecil, tanpa diskon. Satu sabun mandi (`b06`) kali 4 = 18000, di bawah batas 100000, sehingga baris diskon tidak muncul. Uang pas.



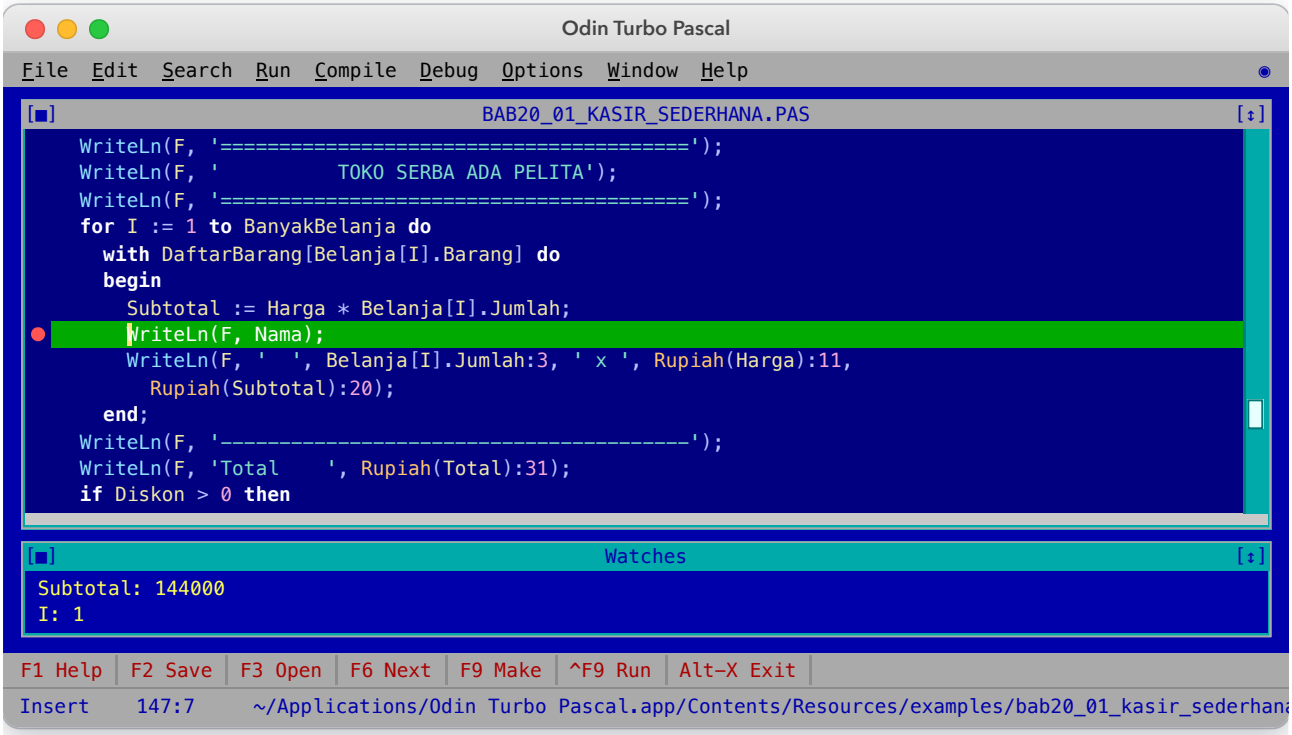
Gambar 20.5 Skenario 2: 4 sabun, total di bawah batas diskon, uang pas

Skenario 3: tidak ada belanjaan. Kasir langsung menekan **Enter** pada pertanyaan kode.



Gambar 20.6 Skenario 3: keranjang kosong, program berhenti dengan `Exit`

Anda juga dapat memakai debugger untuk mengamati hitungan di `CetakStruk`. Pasang *breakpoint* pada baris 147 dan jalankan dengan `Debug ▸ Start debugging`; jendela *watch* menampilkan `Subtotal` untuk setiap barang di keranjang.



Gambar 20.7 Debugger pada baris 147: barang pertama (beras) sudah dihitung subtotalnya 2 x 72000 = 144000

20.2.6 Ide pengembangan

- Baca daftar barang dari file teks sehingga pemilik toko dapat mengubah harga tanpa mengubah program.
- Tambahkan nomor struk yang naik otomatis dan disimpan di file, serta tanggal dari unit `Dos`.
- Tambahkan menu *batalkan barang* yang mengurangi jumlah di keranjang.
- Simpan seluruh transaksi hari itu dan buat laporan penjualan per barang.

20.3 Proyek 2: Pengolah Nilai Siswa

20.3.1 Analisis kebutuhan

Seorang guru ingin program yang membantunya merekap nilai satu kelas. Ia mencatat tiga nilai setiap siswa: tugas, ujian tengah semester (UTS), dan ujian akhir semester (UAS), dengan bobot masing-masing 30%, 30%, dan 40%.

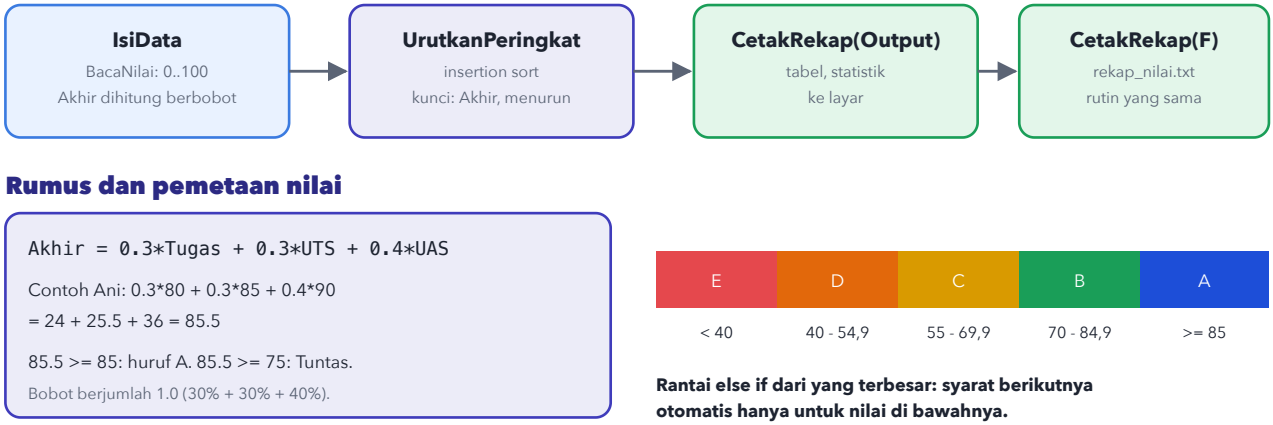
Tabel 20.3 Kebutuhan program nilai siswa

No	Kebutuhan	Aturan
1	Memasukkan data hingga 40 siswa	nama dan tiga nilai bulat 0 sampai 100
2	Menghitung nilai akhir berbobot	0,3 tugas + 0,3 UTS + 0,4 UAS
3	Memberi nilai huruf	A (85 ke atas), B (70), C (55), D (40), E

No	Kebutuhan	Aturan
4	Menyusun peringkat	dari nilai akhir tertinggi
5	Menentukan ketuntasan	KKM 75: tuntas atau remedial
6	Statistik kelas	rata-rata, tertinggi, terendah, jumlah tuntas
7	Menyimpan rekap ke berkas	`rekap_nilai.txt`, isi sama dengan layar

20.3.2 Rancangan

Datanya sebuah larik *record* `TSiswa`. Alurnya tiga tahap berurutan: data dibaca, diurutkan, lalu dicetak dua kali. Kunci urutannya bukan nama, tetapi `Akhir` (nilai akhir), sehingga setelah pengurutan `Siswa[1]` adalah yang tertinggi dan `Siswa[N]` yang terendah. Statistik tertinggi dan terendah karenanya cukup membaca dua ujung larik.



Gambar 20.8 Alur data program nilai siswa, rumus nilai akhir, dan pemetaan nilai huruf.

20.3.3 Listing lengkap

Buka `bab20_02_nilai_siswa.pas`. Perhatikan bahwa `CetakRekap` memakai trik yang sama seperti `CetakStruk`, dan `UrutkanPeringkat` adalah *insertion sort* dari Bab 16, hanya saja elemen yang diurutkan adalah seluruh *record*.

Program 20.2 `bab20_02_nilai_siswa.pas` *Proyek: pengolah nilai siswa* File ▸ Examples…

```
{ Odin Turbo Pascal – GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 20.2 – Proyek: pengolah nilai siswa
  Memasukkan nilai tugas, UTS, dan UAS, menghitung nilai akhir
  berbobot dan nilai huruf, mengurutkan peringkat, menghitung
  statistik kelas, lalu menyimpan rekapnya ke file teks. }

program bab20_02_nilai_siswa;

const
  NamaFileRekap = 'rekap_nilai.txt';
  MaksSiswa = 40;
  KKM = 75;                          { kriteria ketuntasan minimal }
  BobotTugas = 0.3;                  { bobot berjumlah 1.0 }
  BobotUTS = 0.3;
  BobotUAS = 0.4;
```

```
16
17  type
18    TSiswa = record
19      Nama: string[20];
20      Tugas, UTS, UAS: Integer;
21      Akhir: Real;
22    end;
23
24  var
25    Siswa: array[1..MaksSiswa] of TSiswa;
26    N: Integer;
27
28  { Membaca bilangan bulat dalam rentang Min..Max, diulang bila salah }
29  function BacaNilai(Pesan: string; Min, Max: Integer): Integer;
30  var
31    Teks: string;
32    Nilai, Kode: Integer;
33  begin
34    repeat
35      Write(Pesan);
36      ReadLn(Teks);
37      Val(Teks, Nilai, Kode);
38      if (Kode <> 0) or (Nilai < Min) or (Nilai > Max) then
39      begin
40        WriteLn('  Harus angka ', Min, ' sampai ', Max, '.');
41        Kode := 1;                          { paksa ulang }
42      end;
43    until Kode = 0;
44    BacaNilai := Nilai;
45  end;
46
47  function Huruf(Nilai: Real): Char;
48  begin
49    if Nilai >= 85 then Huruf := 'A'
50    else if Nilai >= 70 then Huruf := 'B'
51    else if Nilai >= 55 then Huruf := 'C'
52    else if Nilai >= 40 then Huruf := 'D'
53    else Huruf := 'E';
54  end;
55
56  procedure IsiData;
57  var
58    I: Integer;
59  begin
60    N := BacaNilai('Jumlah siswa (1-40): ', 1, MaksSiswa);
61    for I := 1 to N do
62      with Siswa[I] do
63      begin
64        WriteLn('Siswa ke-', I);
65        Write('  Nama  : '); ReadLn(Nama);
66        Tugas := BacaNilai('  Tugas : ', 0, 100);
67        UTS := BacaNilai('  UTS   : ', 0, 100);
68        UAS := BacaNilai('  UAS   : ', 0, 100);
69        Akhir := BobotTugas * Tugas + BobotUTS * UTS + BobotUAS * UAS;
70      end;
71  end;
72
73  procedure UrutkanPeringkat;             { insertion sort, tertinggi dulu }
74  var
```

```
  I, J: Integer;
  Kunci: TSiswa;
begin
  for I := 2 to N do
  begin
    Kunci := Siswa[I];
    J := I - 1;
    while (J >= 1) and (Siswa[J].Akhir < Kunci.Akhir) do
    begin
      Siswa[J + 1] := Siswa[J];
      J := J - 1;
    end;
    Siswa[J + 1] := Kunci;
  end;
end;

procedure CetakRekap(var F: Text);
var
  I, Lulus: Integer;
  Jumlah: Real;
begin
  WriteLn(F, 'REKAP NILAI KELAS');
  WriteLn(F, 'Bobot: tugas 30%, UTS 30%, UAS 40%.  KKM ', KKM);
  WriteLn(F, 'No  Nama                Tugas  UTS  UAS  Akhir  Huruf  Ket');
  WriteLn(F, '-------------------------------------------------------------');
  Jumlah := 0;
  Lulus := 0;
  for I := 1 to N do
    with Siswa[I] do
    begin
      Write(F, I:2, '  ', Nama, '':20 - Length(Nama), Tugas:6, UTS:5,
        UAS:5, Akhir:7:1, '    ', Huruf(Akhir), '    ');
      if Akhir >= KKM then
      begin
        WriteLn(F, 'Tuntas');
        Lulus := Lulus + 1;
      end
      else
        WriteLn(F, 'Remedial');
      Jumlah := Jumlah + Akhir;
    end;
  WriteLn(F, '-------------------------------------------------------------');
  WriteLn(F, 'Rata-rata kelas : ', Jumlah / N:0:1);
  WriteLn(F, 'Tertinggi       : ', Siswa[1].Akhir:0:1,
    ' (', Siswa[1].Nama, ')');
  WriteLn(F, 'Terendah        : ', Siswa[N].Akhir:0:1,
    ' (', Siswa[N].Nama, ')');
  WriteLn(F, 'Tuntas          : ', Lulus, ' dari ', N, ' siswa');
end;

var
  FileRekap: Text;
  Folder: string;

begin
  IsiData;
  UrutkanPeringkat;                    { sesudah diurut, Siswa[1] terbaik }
  WriteLn;
  CetakRekap(Output);
```

```
134
135   Assign(FileRekap, NamaFileRekap);
136   Rewrite(FileRekap);
137   CetakRekap(FileRekap);
138   Close(FileRekap);
139   GetDir(0, Folder);
140   WriteLn;
141   WriteLn('Rekap disimpan ke ', NamaFileRekap, ' di folder ', Folder);
142 end.
```

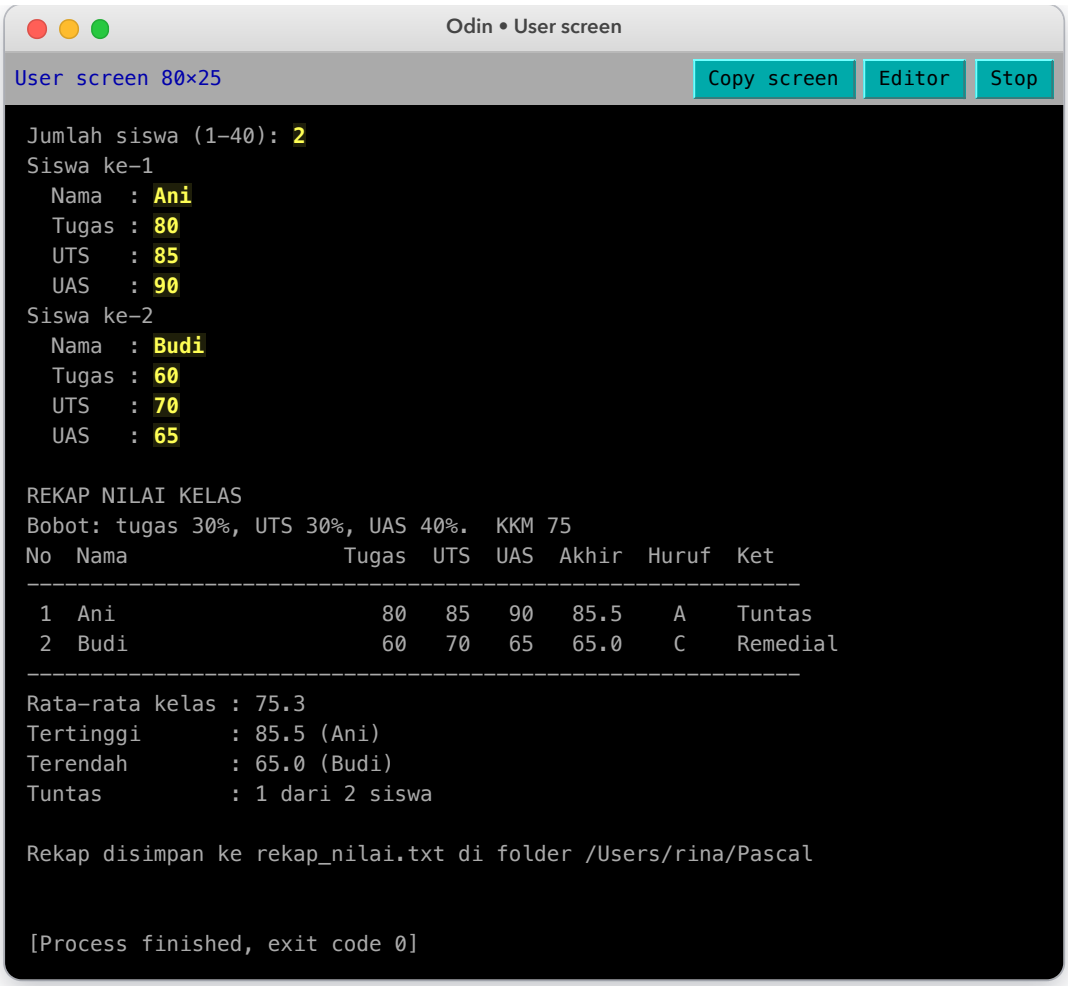
20.3.4 Penjelasan per blok

Baris	Penjelasan
9–15	Konstanta. Bobot ditulis sebagai konstanta bernama, bukan angka tersebar, sehingga guru dapat mengubah kebijakan di satu tempat. `KKM` adalah batas ketuntasan 75.
17–22	`TSiswa` berisi nama, tiga nilai bulat, dan `Akhir` bertipe `Real`. Nilai akhir disimpan, bukan dihitung ulang setiap kali, karena dipakai oleh pengurutan, nilai huruf, dan statistik.
28–45	`BacaNilai(Pesan, Min, Max)` adalah versi umum dari `BacaBilangan` di proyek kasir. Batas atas dan bawah dilewatkan sebagai parameter, jadi fungsi yang sama dipakai untuk jumlah siswa (1–40) dan untuk nilai (0–100). Baris 41 memaksa `Kode` menjadi 1 bila di luar rentang supaya perulangan mengulang.
47–54	`Huruf` mengembalikan `Char` lewat rantai `else if` dari batas terbesar. Karena dicek dari atas, tidak perlu menulis batas atas setiap rentang.
56–71	`IsiData` membaca jumlah siswa, lalu untuk setiap siswa (dengan `with Siswa[I]`) membaca nama dan tiga nilai, serta menghitung `Akhir` (baris 69).
73–89	`UrutkanPeringkat`: untuk setiap `Siswa[I]` (kunci), elemen yang nilainya **lebih kecil** digeser ke kanan, lalu kunci disisipkan (baris 82–87). Tanda `<` pada baris 82 menghasilkan urutan menurun. Karena yang digeser seluruh *record*, nama dan nilai tetap bersama.
91–123	`CetakRekap(var F: Text)`: judul dan kepala tabel, lalu satu baris per siswa dengan format lebar (`Tugas:6`, `Akhir:7:1`). Keterangan `Tuntas` atau `Remedial` bergantung `Akhir >= KKM` (baris 107–113). Sambil berjalan, `Jumlah` dan `Lulus` dikumpulkan.
117	`Jumlah / N:0:1`: pembagian dengan `/` menghasilkan `Real`, lalu diformat satu desimal. Tanpa format, `Real` akan tampil dalam notasi ilmiah.
118–121	Setelah diurutkan, tertinggi dan terendah ada di `Siswa[1]` dan `Siswa[N]`.
129–141	Program utama: isi data, urutkan, cetak ke layar, lalu tulis ke berkas dengan rutin yang sama.

> **7** **ATURAN TURBO PASCAL 7**
>
> Semua `Real` dicetak dengan format `:lebar:desimal` (`Akhir:7:1`, `Jumlah / N:0:1`). Tanpa format, Turbo Pascal 7 dan Odin menampilkan notasi ilmiah seperti `8.5500000000E+01`.

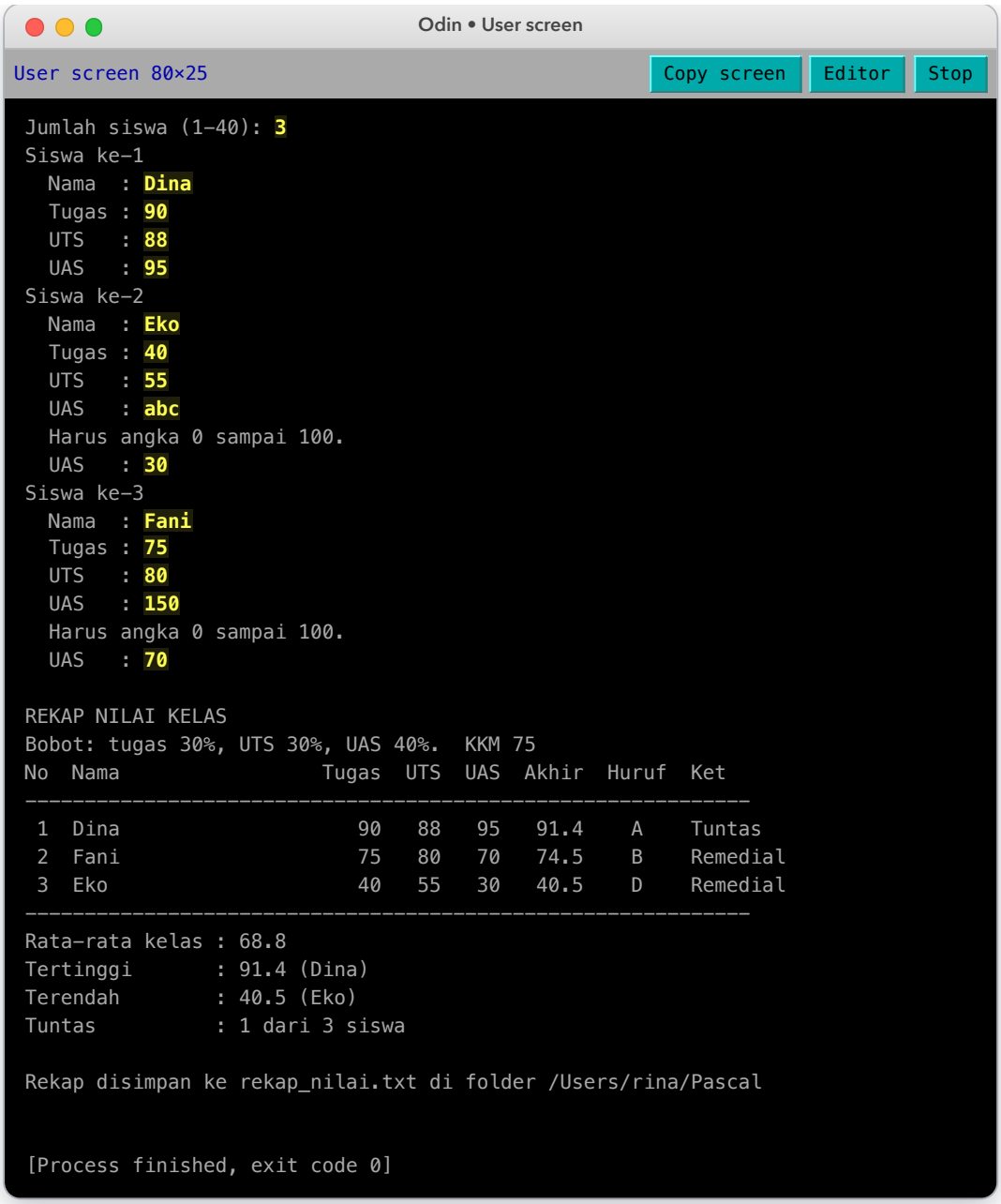
20.3.5 Skenario uji

Skenario 1: dua siswa. Masukan standar: Ani (80, 85, 90) dan Budi (60, 70, 65). Ani memperoleh 85,5 (huruf A, tuntas), Budi 65,0 (huruf C, remedial), sehingga Ani berperingkat pertama.



Gambar 20.9 Skenario 1: dua siswa, satu tuntas dan satu remedial

Skenario 2: tiga siswa dengan masukan salah. Nilai diketik tidak berurutan dari terbaik ke terburuk, dan beberapa masukan salah sengaja diberikan: UAS Eko diketik `abc`, UAS Fani diketik `150` (di luar 0–100). Program menolak keduanya dan meminta ulang. Eko dan Fani dimasukkan setelah Dina, tetapi hasilnya diurutkan ulang berdasarkan nilai akhir.



Gambar 20.10 Skenario 2: validasi menolak `abc` dan `150`; tabel tetap berurutan menurun

Hitung beberapa baris dengan tangan. Dina: `0,3 x 90 + 0,3 x 88 + 0,4 x 95 = 27 + 26,4 + 38 = 91,4`. Eko: `12 + 16,5 + 12 = 40,5`, tepat di atas batas D. Fani: `22,5 + 24 + 28 = 74,5`, hanya setengah poin di bawah KKM, sehingga remedial. Contoh ini menunjukkan mengapa pengujian dengan nilai yang dekat batas penting.

COBA SENDIRI

Jalankan skenario dengan satu siswa saja, lalu dengan nilai semua 100. Apakah tertinggi dan terendah menunjuk siswa yang sama? Bagaimana rata-ratanya? Bila Anda mengetik jumlah siswa `0`, apa yang terjadi?

20.3.6 Ide pengembangan

- Minta guru memilih urutan: peringkat nilai atau abjad nama (dua rutin pengurutan, atau satu rutin dengan parameter).
- Tambahkan histogram sederhana: berapa siswa mendapat A, B, C, D, dan E, ditampilkan dengan baris bintang.
- Simpan data mentah (bukan hanya rekap) di file bertipe sehingga nilai dapat dikoreksi di lain hari.
- Hitung median dan simpangan baku sebagai statistik tambahan.

20.4 Proyek 3: Buku Telepon

20.4.1 Analisis kebutuhan

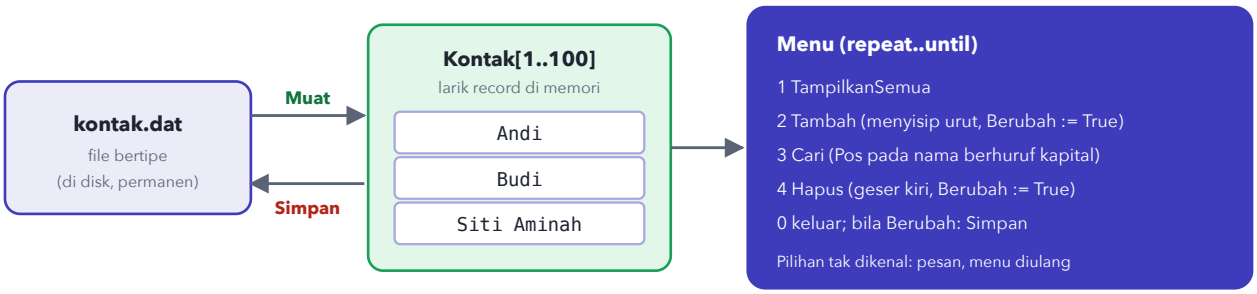
Dua proyek sebelumnya menyimpan hasil ke berkas teks yang hanya dibaca manusia. Proyek ketiga lebih menantang: data harus **bertahan** setelah program ditutup dan **dimuat kembali** pada pemakaian berikutnya, seperti aplikasi sungguhan. Buku telepon menyimpan nama, nomor telepon, dan surel setiap kontak.

Tabel 20.4 Kebutuhan program buku telepon

No	Kebutuhan	Aturan
1	Menampilkan semua kontak	rapi dalam kolom, bernomor
2	Menambah kontak	nama wajib; telepon hanya angka, +, -, spasi, minimal 6 karakter; surel boleh kosong
3	Menjaga urutan nama	kontak selalu terurut abjad, tanpa memedulikan huruf besar/kecil
4	Mencari kontak	sebagian nama cukup; tidak peduli huruf besar/kecil
5	Menghapus kontak	dengan konfirmasi Y/T
6	Menyimpan permanen	dimuat saat mulai, ditulis saat keluar bila ada perubahan

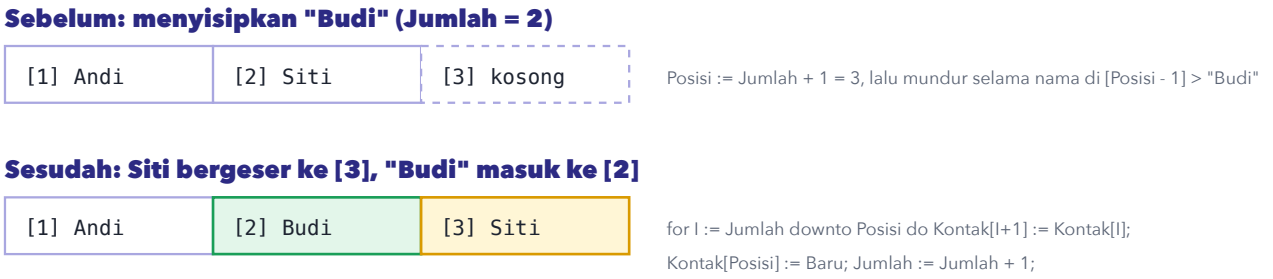
20.4.2 Rancangan

Ada dua tempat data: **berkas** `kontak.dat` (permanen) dan **larik** `Kontak` di memori (sementara). Program membaca isi berkas ke larik saat mulai (`Muat`), semua pekerjaan terjadi di larik, dan isi larik ditulis kembali ke berkas hanya bila ada perubahan (`Simpan`). Penanda `Berubah` mencatat apakah penyimpanan diperlukan.



Gambar 20.11 Siklus data buku telepon: dimuat ke larik, dikerjakan di memori, ditulis kembali saat keluar.

Menjaga larik selalu terurut dilakukan saat **menyisipkan**: posisi yang benar dicari dari belakang, elemen lebih besar digeser ke kanan satu langkah, lalu kontak baru ditaruh pada celah. Ini bagian dalam *insertion sort*. Penghapusan sebaliknya: elemen di kanan digeser ke kiri menutup celah.



Gambar 20.12 Menyisipkan kontak pada posisi urut: geser ke kanan, lalu isi celah.

20.4.3 Listing lengkap

Buka `bab20_03_buku_telepon.pas`. Ini program terpanjang dalam buku. Berkas data berupa `file of TKontak`, yaitu berkas bertipe: setiap elemen adalah satu *record* utuh dengan ukuran tetap (Bab 17), sehingga tidak perlu mengurai teks baris demi baris.

Program 20.3 `bab20_03_buku_telepon.pas` *Proyek: buku telepon* File ▸ Examples…

```
{ Odin Turbo Pascal — GNU GPL v3 or later, Copyright (c) 2026 Syams Ideris }
{ Bab 20.3 — Proyek: buku telepon
  Data kontak dibaca dari file saat program mulai, diolah di array
  (tampil, tambah, cari, hapus), lalu ditulis kembali ke file saat
  keluar. Kontak selalu disimpan urut nama, jadi tampilannya rapi.
  Jalankan beberapa kali: kontak yang disimpan tidak hilang. }

program bab20_03_buku_telepon;

const
  NamaFileData = 'kontak.dat';
  MaksKontak = 100;
  KarakterTelepon = ['0'..'9', '+', '-', ' '];

type
  TKontak = record
```

```
    Nama: string[25];
    Telepon: string[15];
    Email: string[30];
  end;

var
  Kontak: array[1..MaksKontak] of TKontak;
  Jumlah: Integer;
  Berubah: Boolean;                    { ada perubahan yang belum disimpan }

function Kapital(S: string): string;
var
  I: Integer;
begin
  for I := 1 to Length(S) do
    S[I] := UpCase(S[I]);
  Kapital := S;
end;

procedure Muat;
var
  F: file of TKontak;
begin
  Jumlah := 0;
  Assign(F, NamaFileData);
  {$I-}
  Reset(F);
  {$I+}
  if IOResult <> 0 then
    Exit;                              { belum ada file: mulai kosong }
  while not Eof(F) and (Jumlah < MaksKontak) do
  begin
    Jumlah := Jumlah + 1;
    Read(F, Kontak[Jumlah]);
  end;
  Close(F);
end;

procedure Simpan;
var
  F: file of TKontak;
  I: Integer;
begin
  Assign(F, NamaFileData);
  Rewrite(F);                          { tulis ulang seluruh isi }
  for I := 1 to Jumlah do
    Write(F, Kontak[I]);
  Close(F);
  Berubah := False;
end;

procedure TampilkanSatu(I: Integer);
begin
  with Kontak[I] do
    WriteLn(I:3, '. ', Nama, '':26 - Length(Nama), Telepon,
      '':16 - Length(Telepon), Email);
end;

procedure TampilkanSemua;
```

```
var
  I: Integer;
begin
  if Jumlah = 0 then
    WriteLn('Buku telepon masih kosong.')
  else
  begin
    WriteLn('No.  Nama                      Telepon         Email');
    for I := 1 to Jumlah do
      TampilkanSatu(I);
  end;
end;

function TeleponSah(T: string): Boolean;
var
  I: Integer;
begin
  TeleponSah := Length(T) >= 6;
  for I := 1 to Length(T) do
    if not (T[I] in KarakterTelepon) then
      TeleponSah := False;
end;

procedure Tambah;
var
  Baru: TKontak;
  Nama, Telepon: string;
  Posisi, I: Integer;
begin
  if Jumlah = MaksKontak then
  begin
    WriteLn('Buku telepon penuh.');
    Exit;
  end;
  Write('Nama    : '); ReadLn(Nama);
  if Nama = '' then Exit;
  repeat
    Write('Telepon : '); ReadLn(Telepon);
    if not TeleponSah(Telepon) then
      WriteLn('  Hanya angka, +, -, spasi; minimal 6 karakter.');
  until TeleponSah(Telepon);
  Baru.Nama := Nama;
  Baru.Telepon := Telepon;
  Write('Email   : '); ReadLn(Baru.Email);

  { Cari tempat agar urutan nama tetap terjaga, lalu geser ke kanan }
  Posisi := Jumlah + 1;
  while (Posisi > 1) and
        (Kapital(Kontak[Posisi - 1].Nama) > Kapital(Baru.Nama)) do
    Posisi := Posisi - 1;
  for I := Jumlah downto Posisi do
    Kontak[I + 1] := Kontak[I];
  Kontak[Posisi] := Baru;
  Jumlah := Jumlah + 1;
  Berubah := True;
  WriteLn('Kontak ditambahkan.');
end;

procedure Cari;
```

```
var
  Kata: string;
  I, Ketemu: Integer;
begin
  Write('Cari nama: '); ReadLn(Kata);
  Ketemu := 0;
  for I := 1 to Jumlah do
    if Pos(Kapital(Kata), Kapital(Kontak[I].Nama)) > 0 then
    begin
      TampilkanSatu(I);
      Ketemu := Ketemu + 1;
    end;
  WriteLn(Ketemu, ' kontak ditemukan.');
end;

procedure Hapus;
var
  Nomor, I: Integer;
  Teks: string;
  Kode: Integer;
  Yakin: Char;
begin
  TampilkanSemua;
  if Jumlah = 0 then Exit;
  Write('Nomor yang dihapus: '); ReadLn(Teks);
  Val(Teks, Nomor, Kode);
  if (Kode <> 0) or (Nomor < 1) or (Nomor > Jumlah) then
  begin
    WriteLn('Nomor tidak ada.');
    Exit;
  end;
  Write('Hapus ', Kontak[Nomor].Nama, ' (Y/T)? '); ReadLn(Yakin);
  if UpCase(Yakin) = 'Y' then
  begin
    for I := Nomor to Jumlah - 1 do      { geser ke kiri menutup celah }
      Kontak[I] := Kontak[I + 1];
    Jumlah := Jumlah - 1;
    Berubah := True;
    WriteLn('Kontak dihapus.');
  end;
end;

var
  Pilihan: string;
  Folder: string;

begin
  Muat;
  Berubah := False;
  GetDir(0, Folder);
  WriteLn('BUKU TELEPON - ', Jumlah, ' kontak dimuat dari ', NamaFileData);
  WriteLn('Folder data: ', Folder);
  repeat
    WriteLn;
    WriteLn('1. Tampilkan  2. Tambah  3. Cari  4. Hapus  0. Simpan & keluar');
    Write('Pilihan: '); ReadLn(Pilihan);
    if Pilihan = '1' then TampilkanSemua
    else if Pilihan = '2' then Tambah
    else if Pilihan = '3' then Cari
```

```
194       else if Pilihan = '4' then Hapus
195       else if Pilihan <> '0' then WriteLn('Pilihan tidak tersedia.');
196     until Pilihan = '0';
197
198     if Berubah then
199     begin
200       Simpan;
201       WriteLn(Jumlah, ' kontak disimpan ke ', NamaFileData, '.');
202     end
203     else
204       WriteLn('Tidak ada perubahan.');
205   end.
```

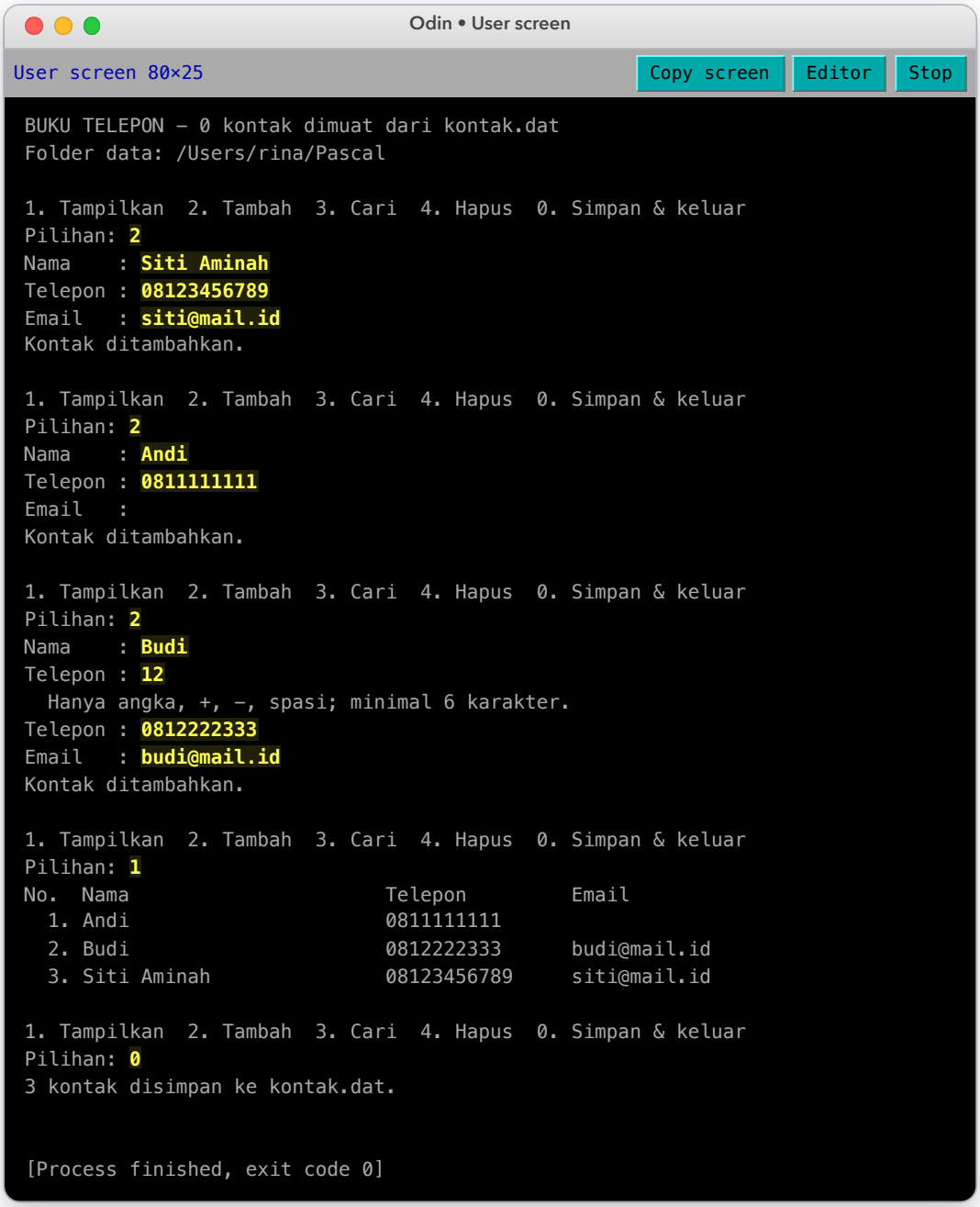
20.4.4 Penjelasan per blok

Baris	Penjelasan
10–13	Nama berkas, kapasitas, dan himpunan karakter telepon yang sah: `['0'..'9', '+', '-', ' ']`, yaitu *set* dari Bab 14.
15–25	`TKontak` dengan tiga *string* berpanjang tetap. Karena *record* berukuran tetap, ia dapat ditulis ke file bertipe. `Berubah` adalah penanda "belum disimpan".
27–34	`Kapital` mengubah seluruh *string* ke huruf besar dengan `UpCase`. Dipakai untuk membandingkan nama tanpa memedulikan huruf besar atau kecil, baik saat mengurutkan maupun mencari.
36–53	`Muat`: `Assign`, lalu `Reset` dengan `{$I-}` / `{$I+}` (baris 42–44) mematikan pemeriksaan galat otomatis. Bila berkas belum ada, `IOResult` bukan nol dan program mulai dengan daftar kosong (baris 45–46), bukan terhenti dengan galat. Berkas yang ada dibaca *record* demi *record* sampai `Eof` (baris 47–51).
55–66	`Simpan`: `Rewrite` menimpa seluruh isi, lalu setiap kontak ditulis, dan penanda `Berubah` dikembalikan ke `False`.
68–87	`TampilkanSatu` dan `TampilkanSemua`: kolom dirapikan dengan bantalan `'':26 - Length(Nama)`. Daftar kosong mendapat pesan sendiri.
89–97	`TeleponSah` memeriksa panjang minimal 6 dan setiap karakter dengan operator `in` terhadap himpunan `KarakterTelepon`.
99–132	`Tambah`: baca nama dan telepon (diulang sampai sah), siapkan `Baru`, cari posisi dari belakang (baris 122–125), geser ke kanan (126–127), lalu sisipkan (128–130). Nama kosong membatalkan penambahan (baris 111).
134–148	`Cari`: `Pos` mencari potongan nama (keduanya berhuruf kapital) pada setiap kontak, menampilkan yang cocok dan menghitungnya.
150–175	`Hapus`: tampilkan daftar, baca nomor dengan `Val` agar aman, minta konfirmasi, lalu geser elemen ke kiri menutup celah (baris 169–170).
181–205	Program utama: `Muat`, tampilkan banyaknya kontak dan folder data, perulangan menu dengan rantai `if..else if`, lalu `Simpan` hanya bila `Berubah`.

20.4.5 Skenario uji berurutan

Karena data tersimpan di berkas, ketiga skenario berikut dijalankan **berurutan di folder yang sama**. Hasil skenario sebelumnya menjadi masukan skenario berikutnya. Ini cara menguji persistensi.

Skenario 1: berkas belum ada. Program mulai dengan 0 kontak. Kita menambahkan tiga kontak dalam urutan sembarang (Siti, Andi, Budi), termasuk satu telepon yang salah (`12`, kurang dari 6 karakter), tampilkan, lalu keluar. Perhatikan bahwa daftar tampil terurut abjad walaupun dimasukkan tidak berurutan.



Gambar 20.13 Skenario 1: berkas kosong; tiga kontak ditambahkan, daftar terurut, lalu disimpan

Skenario 2: memuat, mencari, menghapus. Program mulai kembali dan **memuat 3 kontak** dari `kontak.dat`: bukti data bertahan. Lalu mencari `andi` (huruf kecil), mencari `zzz` (tidak ada), menghapus kontak nomor 2 (Budi) setelah konfirmasi, menampilkan hasilnya, dan keluar.

Odin • User screen

User screen 80×25 Copy screen Editor Stop

```
BUKU TELEPON – 3 kontak dimuat dari kontak.dat
Folder data: /Users/rina/Pascal

1. Tampilkan  2. Tambah  3. Cari  4. Hapus  0. Simpan & keluar
Pilihan: 3
Cari nama: andi
  1. Andi                    0811111111
1 kontak ditemukan.

1. Tampilkan  2. Tambah  3. Cari  4. Hapus  0. Simpan & keluar
Pilihan: 3
Cari nama: zzz
0 kontak ditemukan.

1. Tampilkan  2. Tambah  3. Cari  4. Hapus  0. Simpan & keluar
Pilihan: 4
No.  Nama                    Telepon         Email
  1. Andi                    0811111111
  2. Budi                    0812222333      budi@mail.id
  3. Siti Aminah             08123456789     siti@mail.id
Nomor yang dihapus: 2
Hapus Budi (Y/T)? Y
Kontak dihapus.

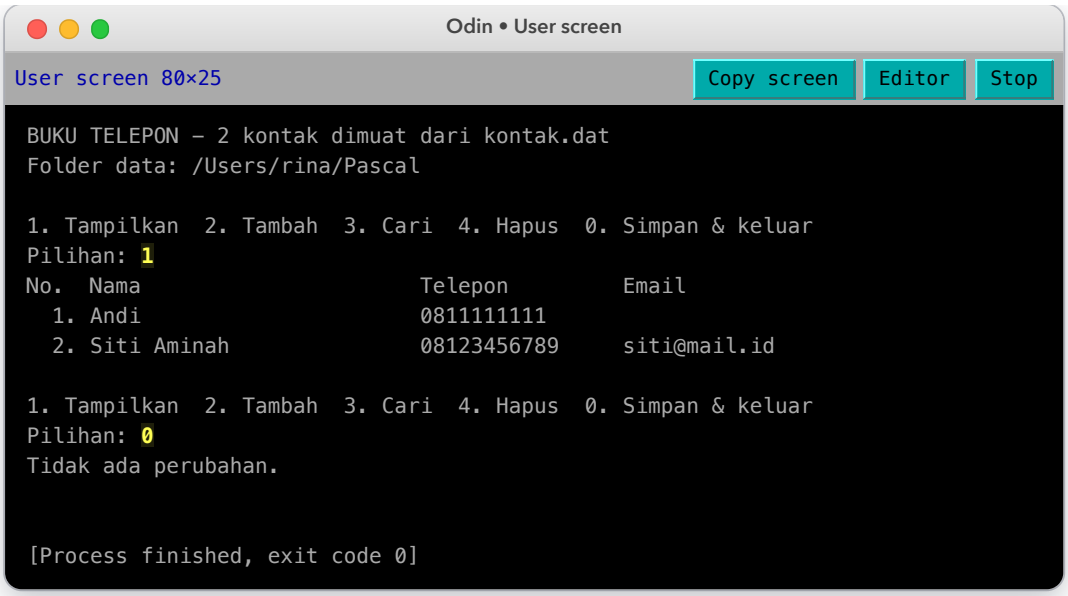
1. Tampilkan  2. Tambah  3. Cari  4. Hapus  0. Simpan & keluar
Pilihan: 1
No.  Nama                    Telepon         Email
  1. Andi                    0811111111
  2. Siti Aminah             08123456789     siti@mail.id

1. Tampilkan  2. Tambah  3. Cari  4. Hapus  0. Simpan & keluar
Pilihan: 0
2 kontak disimpan ke kontak.dat.


[Process finished, exit code 0]
```

Gambar 20.14 Skenario 2: kontak dimuat dari berkas; pencarian tak peduli huruf besar; penghapusan dengan konfirmasi

Skenario 3: memuat tanpa perubahan. Hanya melihat daftar lalu keluar. Karena `Berubah` tetap `False`, berkas tidak ditulis ulang, dan program melaporkan *Tidak ada perubahan*.



Gambar 20.15 Skenario 3: dua kontak yang tersisa dimuat; tidak ada perubahan, tidak ada penyimpanan

20.4.6 Mengapa {$I-} diperlukan

Pada pemakaian pertama, `kontak.dat` belum ada. Tanpa `{$I-}`, `Reset` pada berkas yang tidak ada langsung menghentikan program dengan *runtime error*. Contoh berikut menunjukkannya.

tanpa_io.pas

```
1  program TanpaIO;
2  var
3    F: Text;
4    Baris: string;
5  begin
6    Assign(F, 'tidak_ada.dat');
7    Reset(F);
8    ReadLn(F, Baris);
9    WriteLn(Baris);
10 end.
```



Gambar 20.16 `Reset` pada berkas yang tidak ada tanpa `{$I-}`: program berhenti dengan runtime error 2

Dengan `{$I-}`, kegagalan tidak menghentikan program; kode galatnya disimpan dan dibaca dengan `IOResult`. Setelah dibaca, `{$I+}` menyalakan kembali pemeriksaan otomatis. Pola inilah yang dipakai prosedur `Muat`.

20.4.7 Ide pengembangan

- Tambahkan menu *ubah kontak* dengan menampilkan nilai lama dan membiarkan Enter berarti "tidak diubah".
- Tambahkan kolom alamat atau tanggal lahir. Karena berkas bertipe, ukuran *record* berubah dan berkas lama tidak lagi cocok: pikirkan cara migrasi data lama.
- Tulis cadangan `kontak.bak` sebelum menimpa `kontak.dat` di `Simpan`.
- Pisahkan menjadi dua unit (`data_kontak` dan `tampilan`) dengan teknik Bab 19.

20.5 Kesalahan yang Sering Terjadi di Proyek

- **Menyalin data yang sama ke banyak tempat.** Keranjang kasir hanya menyimpan nomor barang, bukan harga, sehingga harga hanya hidup di satu tempat.
- **Lupa memvalidasi masukan.** Pembaca yang mengetik huruf pada pertanyaan angka akan menjatuhkan program bila memakai `ReadLn` ke bilangan langsung. Gunakan *string* + `Val`.
- **Menulis dua versi kode cetak.** Pakai satu rutin dengan parameter `var F: Text`.
- **Melewatkan kasus pinggir:** daftar kosong, satu elemen, penuh, nilai tepat di batas (75, 100000).
- **Tidak menguji persistensi:** jalankan program dua kali dan periksa data tidak hilang.

Dua contoh galat kompilasi khas di proyek berukuran besar: memakai `div` pada bilangan `Real` saat menghitung rata-rata, dan menulis nilai yang tipenya tidak cocok ke file bertipe.

rata_rata.pas

```
1  program RataRata;
2  var
3    Jumlah, Rata: Real;
4    N: Integer;
5  begin
6    Jumlah := 255.5;
7    N := 3;
8    Rata := Jumlah div N;
9    WriteLn(Rata:0:1);
10 end.
```

PESAN KOMPILER

```
rata_rata.pas(8,11) Error: Tipe operand tidak cocok dengan operator
```

Gambar 20.17 `div` hanya untuk bilangan bulat: untuk `Real` pakai `/`

tipe_file.pas

```
1  program TipeFile;
2  type
3    TKontak = record
4      Nama: string[25];
5    end;
6  var
7    F: file of TKontak;
8    Kontak: array[1..3] of TKontak;
```

```
 9  begin
10    Assign(F, 'k.dat');
11    Rewrite(F);
12    Write(F, Kontak);
13    Close(F);
14  end.
```

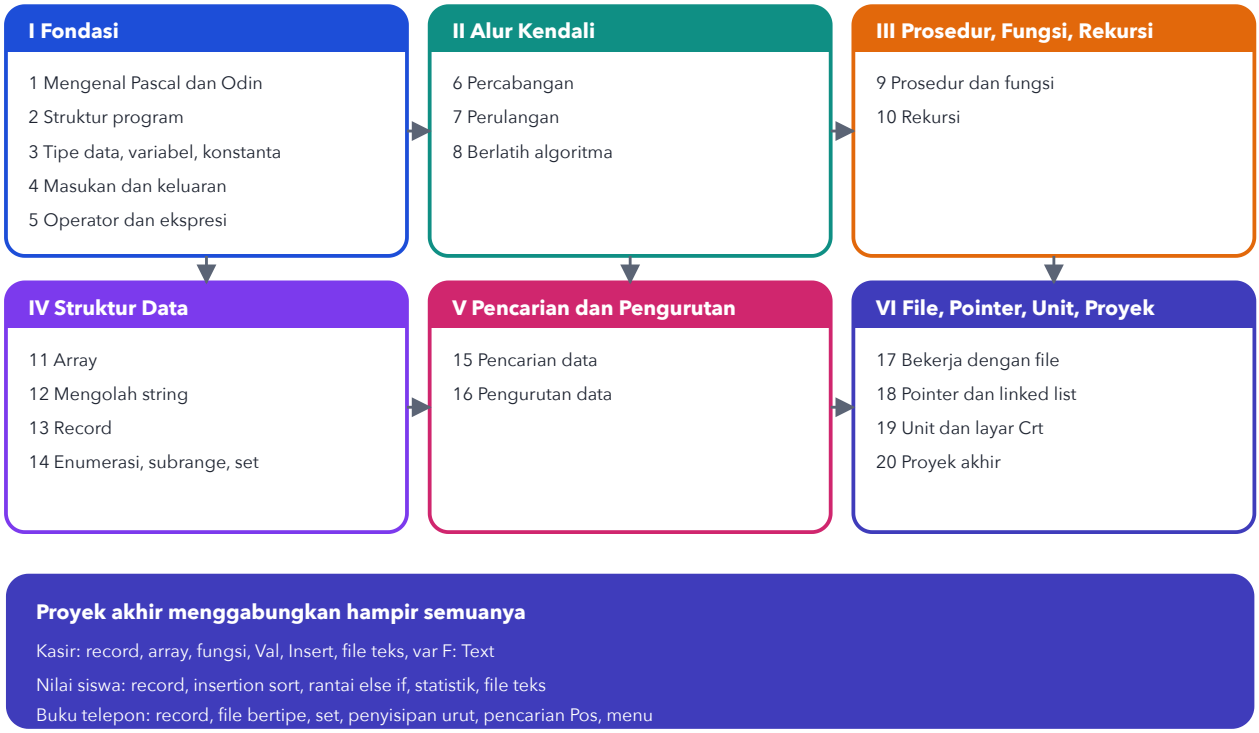
PESAN KOMPILER

```
tipe_file.pas(12,12) Error: Tipe tidak cocok: Array[1..3] of TKontak dan TKontak
```

Gambar 20.18 Menulis larik utuh ke file yang bertipe satu record: tipe tidak cocok

20.6 Peta Konsep: Perjalanan Anda

Anda telah menempuh seluruh buku. Peta berikut merangkum enam bagian dan semua bab yang telah Anda pelajari. Bagian bawahnya menunjukkan bagaimana ketiga proyek akhir menggabungkan hampir semuanya.



Gambar 20.19 Peta konsep seluruh buku: enam bagian dari fondasi sampai proyek akhir.

Tiap panah dalam peta mewakili ketergantungan: fondasi diperlukan untuk alur kendali, dan alur kendali untuk prosedur dan seterusnya. Bila Anda tersangkut pada satu bab, kembali ke bagian sebelumnya sering menjawab masalahnya. Berikut kamus singkat gagasan inti tiap bagian.

Tabel 20.5 Gagasan inti setiap bagian buku

Bagian	Pertanyaan yang dijawab	Kunci
I Fondasi	Bagaimana program menyimpan dan menampilkan data?	variabel bertipe, `Write` / `ReadLn`, operator

Bagian	Pertanyaan yang dijawab	Kunci
II Alur Kendali	Bagaimana program memilih dan mengulang?	`if`, `case`, `for`, `while`, `repeat`
III Prosedur dan Fungsi	Bagaimana memecah masalah?	parameter, nilai kembali, rekursi
IV Struktur Data	Bagaimana menyimpan banyak data?	`array`, `string`, `record`, `set`
V Pencarian dan Pengurutan	Bagaimana menemukan dan menyusun?	pencarian linear dan biner, urutan, efisiensi
VI File, Pointer, Unit, Proyek	Bagaimana membuat program yang utuh?	file, memori dinamis, unit, Crt, proyek

20.6.1 Langkah berikutnya

- **Kerjakan proyek sendiri.** Pilih masalah dari lingkungan Anda (inventaris, jadwal piket, nilai ekstrakurikuler) dan kerjakan dengan empat langkah: analisis, rancangan, kode, uji.
- **Baca kode orang lain.** Bandingkan cara orang lain menyelesaikan masalah yang sama.
- **Pelajari Free Pascal dan Lazarus** untuk aplikasi berjendela, atau lanjutkan ke bahasa lain: konsep yang Anda pelajari di sini (variabel, alur kendali, fungsi, struktur data, berkas) hadir di hampir semua bahasa pemrograman.

20.7 Rangkuman

- Program yang baik berawal dari **analisis kebutuhan**, dilanjutkan **rancangan**, **kode**, dan **pengujian** dengan skenario normal dan skenario salah.
- Pecah program menjadi rutin kecil yang masing-masing mengurus satu hal; program utama hanya menyusunnya dalam urutan yang benar.
- Simpan data sekali dan rujuk lewat nomor atau kunci (keranjang kasir menyimpan nomor barang), supaya tidak ada data ganda yang bertentangan.
- `procedure Cetak(var F: Text)` memungkinkan satu rutin mencetak ke layar (`Output`) dan ke berkas.
- Masukan pemakai harus divalidasi: baca sebagai *string*, ubah dengan `Val`, dan ulangi sampai sah dan berada dalam rentang.
- Data yang harus bertahan disimpan ke berkas (teks untuk laporan, bertipe untuk data) dan dimuat saat program mulai; penanda `Berubah` menghindari penulisan yang tidak perlu.
- `{$I-}` dan `IOResult` memungkinkan program menangani berkas yang belum ada tanpa berhenti.
- Uji dengan nilai di batas (75, 100000, 0, 100), daftar kosong, dan pemakaian berulang untuk persistensi.

20.8 Latihan

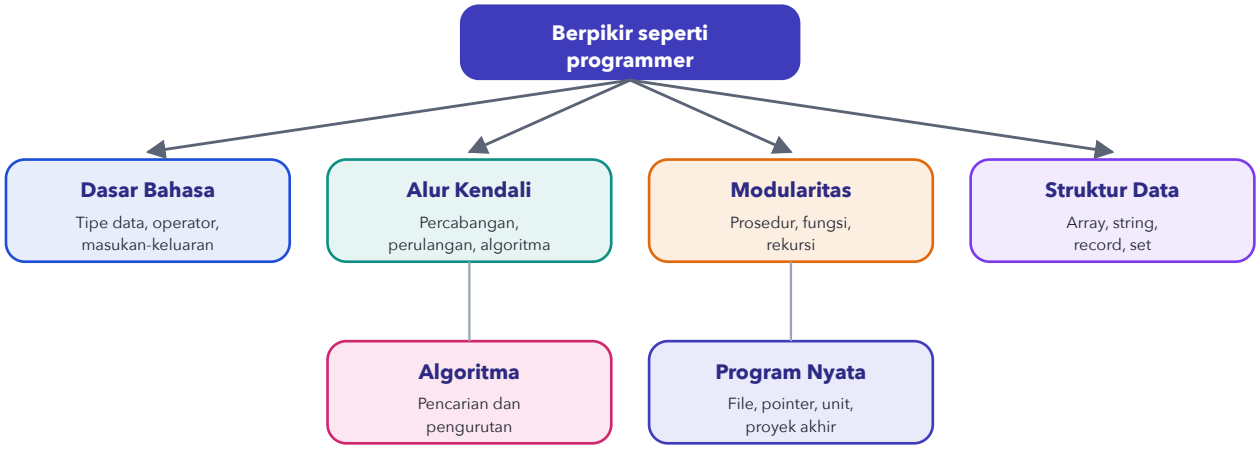
1. **DASAR** Ubah kasir agar diskon berlaku bila belanja minimal Rp 200.000 dan besarnya 5%. Berapa baris yang perlu Anda ubah? Mengapa jumlah baris itu sedikit?
2. **DASAR** Tambahkan dua barang ke `DaftarBarang` (ingat mengubah `JumlahBarang`). Jalankan dan beli barang baru itu.
3. **DASAR** Pada program nilai, ubah bobot menjadi 20%, 30%, 50%, lalu hitung dengan tangan nilai akhir Ani (80, 85, 90) dan bandingkan dengan hasil program.
4. **MENENGAH** Tambahkan ke program nilai statistik jumlah siswa untuk setiap nilai huruf (A sampai E) dan tampilkan di akhir rekap.
5. **MENENGAH** Tambahkan menu *Ubah* pada buku telepon: pilih nomor kontak, lalu ubah telepon atau surelnya. Bila nama berubah, kontak harus dipindahkan agar urutan abjad terjaga. Bagaimana caranya dengan rutin yang sudah ada?
6. **TANTANGAN** Ubah kasir agar daftar barang dibaca dari berkas teks `barang.txt` (satu baris per barang: kode, nama, harga). Bagaimana bila berkas tidak ada?
7. **TANTANGAN** Rancang dan buat proyek Anda sendiri (misalnya inventaris perpustakaan kelas): tuliskan analisis kebutuhan, gambarkan diagram struktur, kerjakan, dan siapkan minimal lima skenario uji termasuk yang salah.

Penutup

Selamat! Bila Anda sampai di halaman ini setelah menyelesaikan kedua puluh bab, Anda telah menempuh perjalanan yang panjang: dari menampilkan "Halo, Dunia!" sampai membangun program kasir, pengolah nilai, dan buku telepon yang menyimpan datanya ke disk. Itu pencapaian yang patut dibanggakan.

Apa yang Sudah Anda Pelajari

Di sepanjang buku ini Anda tidak sekadar menghafal perintah Pascal. Anda membangun cara berpikir seorang programmer: memecah masalah menjadi langkah-langkah kecil, memilih tipe data yang tepat, menyusun program dari prosedur dan fungsi, menyimpan data dalam struktur yang sesuai, lalu menguji dan memperbaiki. Berikut peta perjalanan Anda.



Gambar Peta konsep: enam kelompok keterampilan yang membangun cara berpikir seorang programmer.

Langkah Selanjutnya

Satu bahasa saja tidak cukup untuk menjadi programmer yang mahir, tetapi satu bahasa yang dikuasai dengan baik adalah pondasi yang kuat. Berikut beberapa arah yang bisa Anda tempuh.

1 Perdalam Pascal

Pelajari tipe `object`, Odin Vision (Turbo Vision) untuk antarmuka teks, serta unit `Dos` dan `Strings`. Rujukan ada di `Help ▸ Pascal reference` dan `docs/COMPATIBILITY.txt`.

2 Latih Algoritma

Cari soal pemrograman (misalnya olimpiade komputer atau situs latihan), dan selesaikan dengan Pascal. Kecepatan dan ketelitian tumbuh dari latihan rutin.

3 Bahasa Lain

Konsep yang Anda kuasai, variabel, kondisi, perulangan, fungsi, array, dan record, ada di C, Python, Java, dan lainnya. Perpindahan akan terasa mudah.

Odin Turbo Pascal sendiri adalah perangkat lunak bebas yang kode sumbernya terbuka di `github.com/Syamsuddin/ODIN-TURBO-PASCAL`. Di sana Anda bisa mengunduh versi terbaru, melaporkan kesalahan atau mengusulkan perbaikan lewat **Issues**, dan mempelajari bagaimana sebuah kompiler Pascal dibuat.

Pesan Penutup

Pemrograman adalah keterampilan yang tumbuh dengan latihan, bukan sekadar bacaan. Teruslah menulis program, meski kecil. Setiap kesalahan yang Anda perbaiki menambah pemahaman. Bila program Anda akhirnya berjalan seperti yang diinginkan, nikmati kepuasan itu; itulah yang membuat banyak orang jatuh cinta pada pemrograman.

Terima kasih telah menjadikan buku ini dan Odin Turbo Pascal sebagai teman belajar Anda. Semoga ini menjadi awal perjalanan yang panjang dan menyenangkan. Bila ada pertanyaan, koreksi, atau cerita keberhasilan, saya senang mendengarnya di **syamsuddin.ideris@gmail.com**.

Selamat melanjutkan, dan tetaplah penasaran.

Syams Ideris

LAMPIRAN A

Memasang Odin Turbo Pascal

Odin Turbo Pascal berjalan di macOS, Windows, dan Linux. Berikut langkah pemasangan lengkap untuk setiap sistem. Versi yang dibahas adalah Odin Turbo Pascal 2.3.0. Petunjuk terbaru selalu ada di berkas `README` yang menyertai aplikasi.

> **INFO**
> Paket dan kode sumber Odin Turbo Pascal ada di repositori GitHub `github.com/Syamsuddin/ODIN-TURBO-PASCAL`. Daftar rilis beserta catatan perubahannya ada di `github.com/Syamsuddin/ODIN-TURBO-PASCAL/releases`.

A.1 Yang Diperlukan

Tabel A.1 Persyaratan sistem Odin Turbo Pascal 2.3.0

Komponen	Kebutuhan
macOS	11 Big Sur atau lebih baru, Intel (x86_64) atau Apple Silicon (arm64).
Windows	Windows 10 versi 1809 atau lebih baru, 64-bit (x86_64).
Linux	x86_64 atau arm64 dengan glibc 2.34+ (Ubuntu 22.04+, Debian 12+, Fedora 35+).
Python	3.10 atau lebih baru, lengkap dengan Tk (antarmuka grafis).
Kompiler	Kompiler Odin sudah termasuk dalam paket. Free Pascal dan Xcode tidak diperlukan.
Rust	1.85+ (hanya di Linux atau saat membangun dari source, untuk membuat kompiler Odin).

A.2 macOS

1. **Unduh dan ekstrak.** Buka `github.com/Syamsuddin/ODIN-TURBO-PASCAL`, unduh paketnya (**Code ▸ Download ZIP**), lalu ekstrak seluruhnya. Jangan menjalankan pemasang dari dalam pratinjau ZIP.
2. **Buka `Install.command`.** Pemasang berjalan di Terminal. Ia mendeteksi chip Intel atau Apple Silicon; memeriksa Python (Tk), lalu menawarkan Homebrew atau tautan unduhan resmi bila belum ada.
3. **Tunggu pemeriksaan.** Pemasang mengompilasi dan menjalankan sebuah program uji kecil untuk memastikan hasilnya cocok dengan CPU Mac Anda. Bila gagal, pemasangan berhenti dengan pesan kesalahan.
4. **Jalankan aplikasi** dari `~/Applications/Odin Turbo Pascal.app`.

AWAS

Bundel belum ditandatangani Apple. Bila macOS menahan pembukaannya, buka `System Settings ▸ Privacy & Security ▸ Open Anyway`. Jangan menonaktifkan Gatekeeper secara menyeluruh.

TIPS

Bila atribut *executable* hilang saat ZIP diekstrak, jalankan pemasang lewat Terminal: `/bin/bash "/lokasi/paket/Install.command"`.

A.3 Windows

1. **Unduh dan ekstrak** paket dari `github.com/Syamsuddin/ODIN-TURBO-PASCAL` (**Code ▸ Download ZIP**), lalu ekstrak seluruhnya.
2. **Klik dua kali** `Install.cmd`. Pemasang berjalan di PowerShell: memeriksa Windows 10 1809+ 64-bit; mencari Python (Tk), lalu menawarkan winget atau tautan resmi bila belum ada; mengompilasi dan menjalankan program uji dengan kompiler Odin.
3. **Jalankan** Odin Turbo Pascal dari menu Start atau pintasan Desktop.

Aplikasi dipasang di `%LOCALAPPDATA%\Programs\OdinTurboPascal` tanpa memerlukan hak administrator.

A.4 Linux

Di Linux, Odin Turbo Pascal dijalankan dari source. Contoh untuk Ubuntu atau Debian:

```
sudo apt install python3-tk git          # IDE (Python + Tk)
curl https://sh.rustup.rs -sSf | sh      # Rust, sekali saja, untuk membangun odinpc
git clone https://github.com/Syamsuddin/ODIN-TURBO-PASCAL.git
cd ODIN-TURBO-PASCAL/toolchain
cargo build --release                    # membangun kompiler odinpc
cd ..
python3 main.py                          # jalankan IDE
```

Kompiler, debugger, dan Odin Vision langsung aktif.

INFO

Sebagian hal belum diuji di perangkat keras sungguhan, hanya di runner CI dan mesin virtual, antara lain pemasangan di Mac Apple Silicon, peluncuran lewat Finder, Gatekeeper, serta pemasang Windows lengkap. Rincian status pengujian ada di `README` dan `docs/TEST_REPORT.txt`.

LAMPIRAN B

Pintasan Keyboard

Daftar lengkap pintasan Odin Turbo Pascal. Sumber: menu IDE dan `docs/SHORTCUTS.txt`. Di Mac, tekan fn bersama tombol fungsi bila tombol itu dipakai untuk kontrol perangkat.

Tabel B.1 Berkas dan editor

Tombol	Perintah
F2	Simpan (Save)
F3	Buka (Open)
Ctrl + N	Berkas baru (New)
Ctrl + O	Buka
Ctrl + S	Simpan
Alt + Backspace	Undo
Ctrl + Y	Redo
Ctrl + L	Hapus baris
Tab / Shift + Tab	Indentasi / kurangi
Insert	Mode Insert / Overwrite
Alt + X	Keluar (Exit)

Tabel B.2 Cari, kompilasi, jalankan

Tombol	Perintah
Ctrl + F	Find
Ctrl + H	Replace
Ctrl + G	Search again
Ctrl + J	Go to line
F9	Make (kompilasi)
Alt + F9	Compile
Ctrl + F9	Run (kompilasi + jalankan)
Ctrl + F2	Stop program
Alt + F5	User screen
Alt + F8	Diagnostik berikutnya
F1	Help

Tabel B.3 Debugger

Tombol	Perintah
F7	Trace into (masuk prosedur)
F8	Step over (lewati prosedur)
F4	Go to cursor
Ctrl + F8	Toggle breakpoint
Ctrl + F7	Evaluate / Modify
Ctrl + F9	Continue

Tabel B.4 Jendela dan Mac

Tombol	Perintah
F5	Zoom jendela
F6	Jendela berikutnya
Alt + F3	Tutup jendela
F10	Menu File
Cmd + R	Run (Mac)
Cmd + B	Compile (Mac)

LAMPIRAN C

Kata Tercadang dan Rutin Bawaan

Daftar ini diambil langsung dari penyorot sintaks Odin Turbo Pascal, sehingga sama persis dengan kata yang diberi warna di editor. **Kata tercadang** (*reserved word*) tidak boleh dipakai sebagai nama buatan sendiri. **Rutin bawaan** boleh ditimpa, tetapi sebaiknya tidak.

C.1 Kata Tercadang

absolute	and	array	asm	assembler
begin	case	class	const	constructor
destructor	div	do	downto	else
end	except	exports	external	far
file	finally	for	forward	function
goto	if	implementation	in	inherited
inline	interface	interrupt	label	library
mod	near	not	object	of
or	out	packed	private	procedure
program	property	protected	public	raise
record	repeat	set	shl	shr
string	then	threadvar	to	try
type	unit	until	uses	var
virtual	while	with	xor	

C.2 Tipe Bawaan

boolean	byte	bytebool	cardinal	char
comp	double	extended	int64	integer
longbool	longint	pointer	qword	real
shortint	shortstring	single	smallint	text
word	wordbool			

C.3 Konstanta Bawaan

false	maxint	maxlongint	nil	true

C.4 Prosedur dan Fungsi Bawaan

Sebagian memerlukan unit tertentu, misalnya `ClrScr`, `GotoXY`, dan `ReadKey` ada di unit `Crt`. Rincian tiap rutin ada di `Help ▸ Pascal reference`.

abs	append	arctan	assign	assigned
blockread	blockwrite	break	chdir	chr
close	clreol	clrscr	concat	continue
copy	cos	dec	delay	delete
delline	dispose	eof	eoln	erase
exclude	exit	exp	filepos	filesize
fillchar	frac	freemem	getdir	getmem
gotoxy	halt	high	highvideo	inc
include	insert	insline	int	ioresult
keypressed	length	ln	low	lowvideo
maxavail	memavail	mkdir	move	new
normvideo	nosound	odd	ord	paramcount
paramstr	pi	pos	pred	random
randomize	read	readkey	readln	rename
reset	rewrite	rmdir	round	runerror
seek	seekeof	seekeoln	sin	sizeof
sound	sqr	sqrt	str	succ
textbackground	textcolor	textmode	trunc	truncate
upcase	val	wherex	wherey	window
write	writeln			

LAMPIRAN D

Memahami Pesan Kesalahan

Ada dua jenis kesalahan. **Kesalahan kompilasi** terjadi saat kompiler memeriksa program; tidak ada executable yang dibuat sampai diperbaiki. **Kesalahan runtime** terjadi saat program yang sudah jadi dijalankan, lalu berhenti dengan *Runtime error* dan sebuah kode.

D.1 Kesalahan Kompilasi yang Sering Muncul

Semua pesan di bawah ini adalah keluaran kompiler Odin yang sebenarnya, diambil dengan sengaja membuat program yang salah. Bentuk pesan: `berkas(baris,kolom) Error kode: pesan`.

Tabel D.1 Contoh kesalahan kompilasi dan penyebabnya

Pesan kompiler	Penyebab umum
`titikkoma.pas(4,3) Error: ";" diharapkan, tetapi ditemukan identifier WRITELN`	Titik koma terlupa di akhir baris sebelumnya. Periksa akhir baris di atas yang ditunjuk.
`takdikenal.pas(3,3) Error: Identifier tidak dikenal: Jumlah`	Variabel dipakai tanpa diumumkan di bagian `var`, atau namanya salah ketik.
`tipe.pas(4,8) Error: Tipe tidak cocok: String dan Integer`	Nilai yang diberikan tidak sesuai tipe variabel (di sini String ke Integer).
`real.pas(4,8) Error: Tipe tidak cocok: Real dan Integer`	Operator `/` selalu menghasilkan Real. Untuk hasil bulat pakai `div`.
`samadengan.pas(4,5) Error: ";" diharapkan, tetapi ditemukan "="`	Penugasan ditulis `:=`, bukan `=`. Tanda `=` hanya untuk perbandingan.
`elsekoma.pas(7,3) Error: ";" diharapkan, tetapi ditemukan ELSE`	Ada titik koma sebelum `else` yang berpasangan dengan `if`.
`petik.pas(3,11) Error: Konstanta string melewati akhir baris`	Tanda petik penutup string terlupa.
`result.pas(4,3) Error: Identifier tidak dikenal: Result`	Dalam mode Turbo Pascal tidak ada `Result`; nilai fungsi diberikan lewat nama fungsi.

D.2 Kesalahan Runtime

Program yang lolos kompilasi masih bisa berhenti saat dijalankan. Odin Turbo Pascal menandai baris penyebabnya dengan latar merah (gunakan `Search ▸ Find error` untuk kembali ke sana). Kode yang sering ditemui:

Tabel D.2 Kode kesalahan runtime yang umum

Kode	Arti	Contoh penyebab
2	Berkas tidak ditemukan	`Reset` pada berkas yang belum ada.

Kode	Arti	Contoh penyebab
103	Berkas belum dibuka	`Write` ke berkas tanpa `Rewrite` / `Reset` lebih dulu.
106	Masukan tidak sah	`ReadLn` angka, tetapi pengguna mengetik huruf.
200	Pembagian dengan nol	`A div 0` atau `A / 0`.
201	Di luar jangkauan	Indeks array di luar batas, atau nilai melampaui tipe (range/overflow check).
202	Tumpukan (stack) penuh	Rekursi tanpa kondisi berhenti.
207	Operasi pecahan tidak sah	`Sqrt` dari bilangan negatif.
216	Akses memori tidak sah	Membaca atau menulis lewat pointer `nil`.

TIPS

Range check dan overflow check aktif secara bawaan di kompiler Odin, sehingga banyak kesalahan tertangkap sebagai kode 201, bukan menghasilkan nilai yang salah secara diam-diam. Biarkan tetap aktif selama belajar.

LAMPIRAN E

Daftar Program Contoh

Kesembilan puluh berkas di `File ▸ Examples...`, urut sesuai bab. Angka di kanan adalah halaman tempat listing lengkapnya dibahas.

1.1	bab01_01_halo_dunia.pas	Halo, Dunia!	7
1.2	bab01_02_cek_kompiler.pas	Memeriksa kompiler	8
2.1	bab02_01_struktur_dasar.pas	Struktur dasar program Pascal	14
2.2	bab02_02_komentar.pas	Komentar	17
2.3	bab02_03_blok_begin_end.pas	Blok begin..end	18
3.1	bab03_01_tipe_integer_real.pas	Tipe Integer dan Real	23
3.2	bab03_02_tipe_char_string.pas	Tipe Char dan string	25
3.3	bab03_03_tipe_boolean.pas	Tipe Boolean	26
3.4	bab03_04_konstanta.pas	Konstanta	28
3.5	bab03_05_batas_tipe_data.pas	Batas tipe data	29
4.1	bab04_01_write_writeln.pas	Write dan WriteLn	34
4.2	bab04_02_read_readln.pas	Read dan ReadLn	35
4.3	bab04_03_format_keluaran.pas	Format keluaran	37
4.4	bab04_04_biodata_sederhana.pas	Latihan: biodata sederhana	38
5.1	bab05_01_operator_aritmetika.pas	Operator aritmetika	42
5.2	bab05_02_div_mod.pas	div dan mod	43
5.3	bab05_03_operator_relasional.pas	Operator relasional (perbandingan)	45
5.4	bab05_04_operator_logika.pas	Operator logika	46
5.5	bab05_05_prioritas_operator.pas	Prioritas operator	48
5.6	bab05_06_luas_keliling.pas	Latihan: luas dan keliling	49
6.1	bab06_01_if_then.pas	if..then	55
6.2	bab06_02_if_else.pas	if..then..else	58
6.3	bab06_03_if_bersarang.pas	if bersarang	61
6.4	bab06_04_case_of.pas	case..of	64
6.5	bab06_05_nilai_huruf.pas	Latihan: nilai angka menjadi nilai huruf	67
6.6	bab06_06_tahun_kabisat.pas	Latihan: tahun kabisat	70
7.1	bab07_01_for_to.pas	Perulangan for..to	78
7.2	bab07_02_for_downto.pas	Perulangan for..downto	81
7.3	bab07_03_while_do.pas	Perulangan while..do	83
7.4	bab07_04_repeat_until.pas	Perulangan repeat..until	87

7.5	bab07_05_perulangan_bersarang.pas	Perulangan bersarang	89
7.6	bab07_06_menu_berulang.pas	Latihan: menu berulang	92
8.1	bab08_01_bilangan_prima.pas	Bilangan prima	102
8.2	bab08_02_fpb_kpk.pas	FPB dan KPK	106
8.3	bab08_03_deret_fibonacci.pas	Deret Fibonacci	109
8.4	bab08_04_segitiga_bintang.pas	Segitiga bintang	111
8.5	bab08_05_bilangan_sempurna.pas	Bilangan sempurna	114
8.6	bab08_06_tebak_angka.pas	Permainan tebak angka	116
9.1	bab09_01_prosedur_dasar.pas	Prosedur	126
9.2	bab09_02_fungsi_dasar.pas	Fungsi	129
9.3	bab09_03_parameter_nilai.pas	Parameter nilai	131
9.4	bab09_04_parameter_var.pas	Parameter var	133
9.5	bab09_05_lokal_global.pas	Variabel lokal dan global	135
9.6	bab09_06_kalkulator_modular.pas	Latihan: kalkulator modular	139
10.1	bab10_01_faktorial_rekursif.pas	Faktorial secara rekursif	147
10.2	bab10_02_fibonacci_rekursif.pas	Fibonacci secara rekursif	151
10.3	bab10_03_pangkat_rekursif.pas	Pangkat secara rekursif	154
10.4	bab10_04_menara_hanoi.pas	Menara Hanoi	157
11.1	bab11_01_array_satu_dimensi.pas	Array satu dimensi	168
11.2	bab11_02_maks_min_rata.pas	Nilai terbesar, terkecil, dan rata-rata	172
11.3	bab11_03_array_dua_dimensi.pas	Array dua dimensi	176
11.4	bab11_04_penjumlahan_matriks.pas	Penjumlahan matriks	178
11.5	bab11_05_perkalian_matriks.pas	Perkalian matriks	181
12.1	bab12_01_fungsi_string.pas	Fungsi dan prosedur string	188
12.2	bab12_02_balik_kata.pas	Membalik kata	190
12.3	bab12_03_palindrom.pas	Palindrom	193
12.4	bab12_04_hitung_vokal.pas	Menghitung vokal	196
12.5	bab12_05_konversi_angka_string.pas	Konversi angka dan string	199
13.1	bab13_01_record_dasar.pas	Record	207
13.2	bab13_02_array_of_record.pas	Array of record	211
13.3	bab13_03_record_bersarang.pas	Record bersarang	214
13.4	bab13_04_data_mahasiswa.pas	Latihan: data mahasiswa	217
14.1	bab14_01_tipe_enumerasi.pas	Tipe enumerasi	226
14.2	bab14_02_tipe_subrange.pas	Tipe subrange	229
14.3	bab14_03_tipe_set.pas	Tipe set (himpunan)	233
14.4	bab14_04_validasi_huruf_set.pas	Latihan: validasi dengan set	235
15.1	bab15_01_sequential_search.pas	Pencarian beruntun (sequential search)	244

15.2	`bab15_02_binary_search.pas`	Pencarian biner (binary search)	248
15.3	`bab15_03_cari_data_record.pas`	Mencari data di array of record	254
16.1	`bab16_01_bubble_sort.pas`	Bubble sort	265
16.2	`bab16_02_selection_sort.pas`	Selection sort	269
16.3	`bab16_03_insertion_sort.pas`	Insertion sort	272
16.4	`bab16_04_urut_record.pas`	Mengurutkan array of record	276
16.5	`bab16_05_banding_jumlah_langkah.pas`	Membandingkan jumlah langkah	278
17.1	`bab17_01_tulis_file_teks.pas`	Menulis file teks	290
17.2	`bab17_02_baca_file_teks.pas`	Membaca file teks	293
17.3	`bab17_03_tambah_file_teks.pas`	Menambah isi file teks	296
17.4	`bab17_04_file_bertipe.pas`	File bertipe	298
17.5	`bab17_05_simpan_baca_record.pas`	Menyimpan dan membaca record	301
18.1	`bab18_01_pointer_dasar.pas`	Pointer	308
18.2	`bab18_02_new_dispose.pas`	New dan Dispose	312
18.3	`bab18_03_linked_list_tambah.pas`	Linked list: menambah simpul	315
18.4	`bab18_04_linked_list_hapus.pas`	Linked list: menghapus simpul	318
19.1	`bab19_01_pakai_unit_sendiri.pas`	Memakai unit buatan sendiri	329
19.2	`bab19_02_unit_crt.pas`	Unit Crt	334
19.3	`bab19_03_menu_berwarna.pas`	Menu berwarna	338
20.1	`bab20_01_kasir_sederhana.pas`	Proyek: kasir sederhana	349
20.2	`bab20_02_nilai_siswa.pas`	Proyek: pengolah nilai siswa	358
20.3	`bab20_03_buku_telepon.pas`	Proyek: buku telepon	365
Unit	`matematika.pas`	Unit buatan sendiri: matematika	326

LAMPIRAN F

Glosarium

Istilah penting yang dipakai di buku ini.

Algoritma Urutan langkah yang jelas dan berhingga untuk menyelesaikan suatu masalah.

Array Kumpulan data bertipe sama yang disimpan berurutan dan diakses lewat indeks.

Breakpoint Titik henti yang dipasang pada sebuah baris agar debugger berhenti di situ.

Debugger Alat untuk menjalankan program langkah demi langkah dan memeriksa nilai variabel.

Deklarasi Pengumuman nama (variabel, konstanta, tipe, prosedur) beserta sifatnya sebelum dipakai.

Direktif kompiler Perintah untuk kompiler, ditulis di antara `{$ ... }`, misalnya `{$R+}`.

Ekspresi Gabungan nilai, variabel, dan operator yang menghasilkan sebuah nilai.

Executable Program hasil kompilasi yang bisa langsung dijalankan prosesor.

Fungsi Subprogram yang mengembalikan sebuah nilai.

IDE Lingkungan pengembangan terpadu: editor, kompiler, dan alat bantu dalam satu aplikasi.

Kata tercadang Kata yang maknanya sudah ditetapkan bahasa dan tidak boleh dipakai sebagai nama sendiri.

Kode sumber Teks program yang ditulis manusia, misalnya berkas `.pas`.

Komentar Catatan untuk pembaca manusia yang diabaikan kompiler, ditulis di antara `{ ... }`.

Kompiler Program yang menerjemahkan kode sumber menjadi executable. Odin Turbo Pascal memakai kompiler Odin (`odinpc`).

Kompilasi Proses menerjemahkan kode sumber menjadi executable.

Konstanta Nilai bernama yang tidak berubah selama program berjalan.

Linked list Rangkaian node yang saling terhubung lewat pointer, bisa tumbuh dan menyusut saat program berjalan.

Parameter Data yang dikirim ke prosedur atau fungsi. Parameter nilai dikirim sebagai salinan, parameter `var` sebagai acuan.

Pointer Variabel yang menyimpan alamat variabel lain di memori.

Prosedur Subprogram yang mengerjakan tugas tanpa mengembalikan nilai.

Record Tipe yang menggabungkan beberapa field bertipe berbeda menjadi satu kesatuan.

Rekursi Prosedur atau fungsi yang memanggil dirinya sendiri.

Runtime error Kesalahan yang terjadi saat program berjalan, menyebabkannya berhenti.

Set Himpunan nilai dari tipe ordinal, dengan operasi gabungan, irisan, dan selisih.

Tipe data Jenis nilai yang bisa disimpan sebuah variabel, misalnya `Integer` atau `Char`.

Unit Berkas berisi kumpulan deklarasi yang bisa dipakai banyak program lewat `uses`.

User screen Layar 80×25 tempat program berjalan dan menampilkan keluaran di Odin Turbo Pascal.

Variabel Tempat bernama di memori untuk menyimpan nilai yang bisa berubah.

Belajar Pascal dengan cara yang menyenangkan: ketik, jalankan, pahami.

Buku ini mengajak Anda belajar pemrograman dari nol dengan bahasa Pascal, bahasa yang dirancang khusus untuk mengajarkan cara berpikir terstruktur. Setiap konsep dijelaskan dengan bahasa sederhana, infografik berwarna, dan program lengkap yang bisa langsung dibuka dan dijalankan di Odin Turbo Pascal, IDE bergaya klasik yang berjalan di macOS, Windows, dan Linux.

YANG AKAN ANDA KUASAI

- Memasang dan memakai IDE Odin Turbo Pascal
- Tipe data, operator, percabangan, dan perulangan
- Prosedur, fungsi, dan rekursi
- Array, string, record, enumerasi, dan set
- Algoritma pencarian dan pengurutan
- File, pointer, linked list, dan unit
- Tiga proyek akhir yang utuh

Semua 89 program contoh tersedia di menu **File ▸ Examples**, dan semua keluaran di buku ini adalah hasil menjalankan program tersebut dengan kompiler Odin.

Syams Ideris
syamsuddin.ideris@gmail.com
Odin Turbo Pascal v2.3.0 · GNU GPL v3 atau yang lebih baru